

ML STUDIO | A VISUAL, HANDS-ON COURSE

MACHINE LEARNING FOR EVERYONE

From First Principles to Modern Models

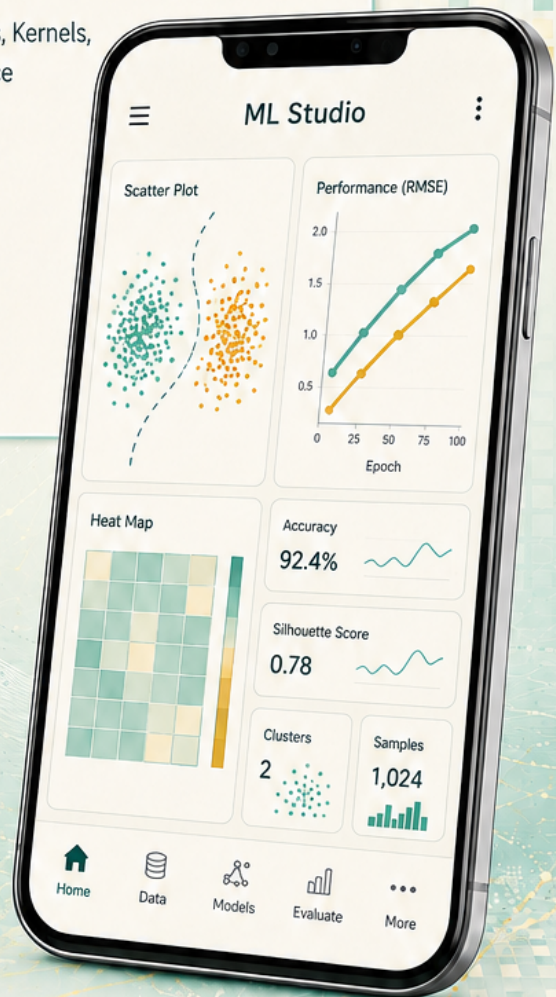
Regression, Classification, Decision Trees, Random Forests, SVMs, Kernels,
Naive Bayes, Ensembles, Clustering, PCA, and Hands-On Practice



FREE ANDROID APP



COMPANION TEACHER'S GUIDE



STUDENT EDITION

Dr. Ismael Khorshed Abdulrahman

First Edition · 2026

MLFE | STUDENT EDITION

MACHINE LEARNING FOR EVERYONE

A Practical Guide from First Principles to Classical Models

Learn by hand, then check it with Python

Regression • Classification • Trees and Ensembles
Kernels • Clustering • Dimensionality Reduction
Model Selection • Responsible Machine Learning

Dr. Ismael Khorshed Abdulrahman

*PhD in Electrical and Computer Engineering
Tennessee Technological University, USA*

First Edition | 2026

Machine Learning for Everyone:

A Practical Guide from First Principles to Classical Models

Learn by hand, then check it with Python

Regression • Classification • Trees and Ensembles

Kernels • Clustering • Dimensionality Reduction

Model Selection • Responsible Machine Learning

Dr. Ismael Khorshed Abdulrahman

PhD in Electrical and Computer Engineering

Free Electronic Edition

Licensed under Creative Commons Attribution (CC BY).

This book is freely available for educational, personal, and scholarly use. You are welcome to share, reproduce, and adapt its content, including for commercial purposes, provided that appropriate credit is given to the author and the original source is properly cited.

[This book and its companion app, *ML Studio*, are provided free of charge for anyone who wishes to learn.](#)
[Voluntary donations in support of their continued development are warmly welcomed and appreciated.](#)

A note on AI assistance: AI tools were used during the preparation of this book under the author's direction and review. AI assistance may have contributed to the drafting, refinement, explanation, organization, or development of certain examples and other content. Every effort has been made to carefully check and verify the facts, figures, explanations, mathematical derivations, examples, solutions, and other content before publication. Nevertheless, despite these efforts, errors or inaccuracies may remain. Readers are kindly encouraged to inform the author of any mistakes, inaccuracies, or corrections they identify. Such feedback is greatly appreciated and may be considered for correction in future editions (updates).

A Note on the Numbers: The datasets, prices, measurements and organizations in this book's worked examples, activities and case studies are illustrative teaching examples, chosen to make a method clear and checkable by hand. They are not verified statistics about any real person, company or institution. Where a real historical fact is stated — who first published a method, and in what year — it is named in the text and listed in the References at the end of the book. Always confirm current figures from a primary source before using them in a real decision.

Last updated: 31 August 2026 · AI model families and dated examples checked as of this date.

How to Use This Book

This book teaches machine learning in three steps. You read an idea in plain words. You see it worked out with real numbers, one step at a time. Then you try it yourself — first on paper, then in Python.

Machine learning can feel like two subjects at once: a wall of unfamiliar words, and a wall of unfamiliar mathematics. This book takes down both walls at the same time, slowly and honestly. Every technical word is explained in plain English the first time it appears, and collected again in the Glossary at the end. Sentences are kept short, because many readers of this book are learning in a second or third language. New ideas are built only from ideas you already have.

Every chapter has the same twelve parts

Once you have read one chapter, you know the shape of all of them:

1. **Learning goals** — what you will be able to do by the end.
2. **A plain-language introduction** — an everyday situation, before any mathematics.
3. **Numbered lessons** — the ideas, one small step at a time.
4. **WORD TO KNOW boxes** — every new term, defined the moment it appears.
5. **EQUATION boxes** — the formula in proper mathematical notation, restated in plain words, with a table defining every symbol and its unit, and a note on when to use it.
6. **WORKED EXAMPLES** — fully solved with numbered steps, small enough to check with a pencil.
7. **Figures and tables** — plots, diagrams, flowcharts and datasets, each followed by a short note on what to observe.
8. **COMMON MISTAKE boxes** — the error, then the correction, marked **X** and **✓**.
9. **Practical activities** — one on paper, one in Python, sometimes a simulation.
10. **A real-world case study** — the method used in a real kind of decision.
11. **A chapter summary** — the whole chapter in a dozen lines.
12. **Practice problems** — labeled Basic, Standard and Challenge.

How to read a lesson

Read the plain explanation first and do not worry about the equation yet. When you reach the EQUATION box, read the “In words” line before the symbols — it says the same thing in ordinary English. Then read the symbol table, so you know what every letter means and what unit it carries. Only then look at the formula itself. Finally, work through the WORKED EXAMPLE with a pencil and a calculator, writing each step yourself rather than reading it. If your numbers match the book’s, you have understood the equation.

Hand first, software second

Every method in this book is introduced on a dataset small enough to work by hand — often four to eight rows. You will fit lines, split trees, compute distances, build confusion matrices and move centroids yourself, with arithmetic. Only then does the PYTHON activity repeat the same calculation with NumPy or scikit-learn, and get the same answer. This order matters. A person who has computed a Gini impurity by hand once will never be confused by what a decision tree is doing; a person who has only called a function will be.

You do not need a computer to learn from this book. Every lesson, every worked example and most practice problems can be done with paper and a calculator. The Python activities are there for when you have a computer, and Appendix B tells you exactly how to set one up.

The five parts

Part One (Chapters 1–7) builds the foundations every project needs, whatever model you eventually choose: what learning is, a safe project workflow, how to describe and clean data, how to split it honestly, how to measure a model, and why a model that is perfect on its training data can still fail.

Part Two (Chapters 8–9) predicts numbers: linear regression, and the gradient descent that fits it.

Part Three (Chapters 10–17) predicts categories: logistic regression, k -nearest neighbors, naive Bayes, decision trees, random forests, boosting and stacking, support vector machines, and kernel methods.

Part Four (Chapters 18–21) works without labels: k -means, hierarchical and density-based clustering, principal component analysis, and anomaly detection.

Part Five (Chapters 22–25) is about building models you can defend: feature engineering, tuning and model selection, interpretability, fairness and privacy, and two complete capstone projects.

Part One is written to be read in order. After that, the model chapters can mostly be read on their own, although each one assumes the vocabulary of Part One, and a few build directly on the chapter before them — Chapter 17 continues Chapter 16, and Chapter 15 continues Chapter 14.

The three levels of practice problem

Basic problems check that you can state a definition or do one calculation. **Standard** problems are the ones to spend most of your time on: they take a method through several steps, like the worked examples. **Challenge** problems ask you to compare two situations, explain why something went wrong, or design a small project of your own; they usually have more than one defensible answer. Try every problem on paper before checking anything, and write out the steps even when you can see the answer. Full step-by-step solutions are collected in the companion Teacher's Guide.

If you get stuck

Three places will usually help. The **Glossary** at the end defines every term in the book in one place. **Appendix A** collects the small amount of mathematics this book leans on — means, sums, distances, probability, slopes and logarithms — each with its own worked example. **Appendix B** is the Python field guide. And if a chapter refers to an idea you have forgotten, follow its chapter reference back; the book cross-references itself deliberately so that nothing has to be memorized.

A word about using these models on people

Several chapters predict things about people: which students may need support, which patients may be ill, which applicants may repay a loan. Those chapters carry a short box marked RESPONSIBLE USE, and Chapter 24 is devoted to the subject. The short version, worth carrying from the first page: a prediction is a guide for a human decision, never a fact about a person, and never proof of what will happen.

— *Dr. Ismael Khorshed Abdulrahman*

Notation and Conventions

This book uses a small, consistent set of symbols from the first chapter to the last. You do not need to memorize this page — every symbol is also explained in plain words, in a symbol table, the first time it is actually used. Keep this page as a quick reference.

Data

Symbol	Meaning	Notes
x	one input feature	plain lowercase letter
x_j	the j -th feature of an example	subscript names the feature
$\mathbf{x}$	all features of one example, as a vector	bold lowercase
X	the whole feature table: one row per example, one column per feature	capital letter; X in code
y	the true label or target value of one example	y in code
$\hat{y}$	the model's prediction for one example	"y hat"
n	the number of examples in a dataset	a count
p	the number of features	a count
i	index over examples, $i = 1, 2, \dots, n$	row number
j	index over features, $j = 1, 2, \dots, p$	column number
k	a count chosen by you: neighbors, clusters, or folds	meaning is always stated

Models and learning

Symbol	Meaning	Notes
w	a weight, or coefficient, learned by the model	w_j for feature j
$\mathbf{w}$	all the weights together, as a vector	bold
b	the intercept, or bias term	a single number
α	the learning rate in gradient descent	a small positive number
λ	the strength of a regularization penalty	larger means simpler model
C	an inverse penalty strength (SVM, logistic regression)	larger means <i>less</i> regularization
γ	the width parameter of an RBF kernel	larger means narrower
L	a loss: the number the model tries to make small	also called cost or error
$p(\cdot)$	a probability	between 0 and 1
$\sigma(z)$	the sigmoid function	turns a score into a probability

Statistics

Symbol	Meaning	Notes
$\bar{x}$	the mean of a set of values	" x bar"
μ	a mean, especially of a whole population	Greek letter mu
s, σ	a standard deviation	s for a sample, σ for a population
s^2, σ^2	a variance	the standard deviation squared
r	the correlation coefficient, from -1 to $+1$	Chapter 3
R^2	the fraction of variance a regression explains	Chapter 6
z	a z -score: how many standard deviations from the mean	Chapters 4 and 21
d	a distance between two points	d^2 is the squared distance

Mathematical operators

Symbol	Meaning	Notes
$\times, \cdot$	multiply	$\mathbf{a} \cdot \mathbf{b}$ is a dot product
$\div, /$	divide	
$\sum$	add up a list of numbers	Greek capital sigma
$\prod$	multiply a list of numbers together	Greek capital pi; Chapter 12
$\sqrt{\quad}$	square root	
$ a $	absolute value: drop the minus sign	
$\ \mathbf{w}\ $	the length of a vector	Chapters 16 and 17
$\approx$	approximately equal to	
$\propto$	proportional to	Chapter 12
$\log, \ln$	logarithm; in this book always the natural logarithm	Appendix A.9
e	the number 2.71828...	base of the natural logarithm
$\frac{\partial L}{\partial w}$	the partial derivative of L with respect to w	the slope in one direction; Chapter 9
$\arg \min, \arg \max$	the input that makes an expression smallest or largest	not the value itself

How the boxes are colored

Blue **WORD TO KNOW** boxes introduce new vocabulary. Orange-edged **EQUATION** boxes give a formula, a plain-English restatement, a note on when it is used, and a table defining every symbol. Green **WORKED EXAMPLE** boxes are fully solved, numbered and checkable by hand. **COMMON MISTAKE** boxes pair a **X** Wrong line with a **✓** Right line. Orange-edged **ACTIVITY** boxes are things to do, on paper or in Python. Blue-edged **CASE STUDY** boxes describe a real kind of decision. Shaded **RESPONSIBLE USE** and **IMPORTANT** boxes carry warnings worth stopping for. Light **WHAT TO OBSERVE** boxes follow a figure and say what to look at in it.

Numbers, units and rounding

Money in the Kurdistan Region and Iraq is written in Iraqi dinars (IQD), often in thousands; other examples use dollars. Distances are in kilometers, masses in kilograms, energy in kilowatt-hours. Intermediate results in worked examples are shown to two or three decimal places and the final answer is rounded sensibly for its unit — a predicted rent to the nearest thousand dinars, an accuracy to the nearest whole percent. Where rounding a middle step would change the final answer, the book says so.

Contents

How to Use This Book	iv
Notation and Conventions	vii
I Foundations of Machine Learning	1
1 What Is Machine Learning?	2
1.1 Learning from Examples	2
1.2 Data, Samples, Features and Labels	4
1.3 Three Common Learning Tasks	6
1.4 Supervised, Unsupervised and Reinforcement Learning	8
1.5 A Precise Learning Goal: Task, Experience and Performance	9
1.6 Training, Testing and Accuracy	10
1.7 Build a Classifier by Hand: Nearest Centroid	12
1.8 Where This Book Goes Next	15
2 The Machine Learning Workflow	20
2.1 Begin with the Decision, Not the Algorithm	21
2.2 Name the Unit, Target and Prediction Time	22
2.3 Inspect and Split the Data Early	24
2.4 Build a Baseline Before a Complex Model	29
2.5 Train, Validate, Test Once	32
2.6 The Workflow as a Checklist	34
3 Data Collection and Dataset Structure	39
3.1 Where Data Comes From	39
3.2 The Dataset as a Table: Samples, Features and Targets	41
3.3 Describing One Column: Mean, Median, Mode, Range, Variance and Standard Deviation	43
3.4 Seeing One Column: Histograms and Box Plots	46
3.5 Seeing Two Columns: Scatter Plots and Correlation	49
3.6 Seeing Many Columns: the Correlation Heat Map and Pair Plot	51
3.7 A First-Look Checklist for Any New Dataset	53
4 Data Cleaning and Preprocessing	58
4.1 Why Raw Data Is Never Ready	58
4.2 Missing Values: Find, Understand, Then Fill or Drop	60
4.3 Outliers and Errors: z-Scores, the IQR Rule, and Domain Checks	62
4.4 Duplicate Records	66
4.5 Encoding Categorical Variables	66
4.6 Feature Scaling: Standardization and Min–Max Normalization	68
4.7 Fit on Training Data Only: Pipelines and Leakage	71
5 Training, Validation, Testing, and Cross-Validation	77
5.1 Why One Number From the Training Data Lies	77
5.2 The Three Sets: Training, Validation, Test	78

5.3	Random, Stratified and Time-Based Splits	80
5.4	k-Fold Cross-Validation	83
5.5	Stratified k-Fold, Leave-One-Out and Repeated CV	85
5.6	Reporting a Score With Its Spread	86
5.7	Data Leakage Through Splitting	88
6	Measuring Model Performance	96
6.1	A Metric Must Match the Real Goal	96
6.2	Regression Errors in Real Units: MAE, MSE, RMSE and R^2	97
6.3	Reading a Confusion Matrix	100
6.4	Accuracy, Precision, Recall, Specificity and F1	100
6.5	From Probabilities to Decisions: Thresholds, ROC Curves and AUC	102
6.6	When 99% Accuracy Means Nothing: Imbalanced Data	104
6.7	Multi-Class Confusion Matrices and Averaging	108
6.8	Calibration: Do Probabilities Mean What They Say?	110
7	Overfitting, Underfitting, and Regularization	116
7.1	Two Ways a Model Can Fail	116
7.2	Bias and Variance	119
7.3	The Bias-Variance Trade-off	120
7.4	Hyperparameters and Model Capacity	122
7.5	Training vs Validation Loss Curves	124
7.6	Learning Curves: More Data or a Different Model?	125
7.7	Regularization: Paying a Price for Complexity	127
II	Supervised Learning: Predicting Numbers	135
8	Linear Regression	136
8.1	A Line That Predicts a Number	136
8.2	Slope, Intercept and Predictions by Hand	138
8.3	Residuals and the Least-Squares Loss	139
8.4	The Best Line Without Guessing	142
8.5	Checking a Fit: Plots and R^2	144
8.6	Multiple Linear Regression	146
8.7	Curves From Lines: Polynomial Regression	148
8.8	Ridge and Lasso: Regularized Regression	150
9	Gradient Descent	157
9.1	Searching for the Bottom of a Valley	157
9.2	The Gradient: Which Way Is Downhill?	160
9.3	One Step of Gradient Descent	161
9.4	The Learning Rate: Too Small, Too Large, Just Right	165
9.5	Reading a Loss Curve	167
9.6	Batch, Stochastic and Mini-Batch Gradient Descent	169
9.7	Feature Scaling and Convergence	172
9.8	When Gradient Descent Is Used	174
III	Supervised Learning: Predicting Categories	179
10	Logistic Regression	180
10.1	Why a Straight Line Cannot Predict Yes or No	180
10.2	The Sigmoid Function	182
10.3	Log Loss: Scoring a Probability Honestly	184

10.4	The Decision Boundary	186
10.5	Fitting Logistic Regression with Gradient Descent	188
10.6	Reading Coefficients: Odds and Log-Odds	189
10.7	More Than Two Classes: One-vs-Rest and Softmax	191
10.8	Regularized Logistic Regression: the C Dial	193
11	k-Nearest Neighbors	199
11.1	Predict Like Your Neighbors	199
11.2	Distance: Euclidean, Manhattan and Why Scaling Matters	202
11.3	Choosing k : Small Against Large	206
11.4	Weighted Voting and Ties	209
11.5	kNN for Predicting Numbers	211
11.6	Cost, Strengths and the Curse of Dimensionality	212
12	Naive Bayes and Probabilistic Classifiers	219
12.1	Probability Refresher: Joint, Conditional and Independent	219
12.2	Bayes' Theorem in Plain Words	221
12.3	The "Naive" Assumption and the Naive Bayes Classifier	222
12.4	Categorical Naive Bayes by Hand	224
12.5	Text Classification: Bag of Words, Multinomial Naive Bayes and Smoothing	225
12.6	Gaussian Naive Bayes for Numerical Features	229
12.7	Related Probabilistic Classifiers: LDA and QDA	231
12.8	Strengths, Weaknesses and When to Use Naive Bayes	233
13	Decision Trees	238
13.1	A Tree Asks One Question at a Time	238
13.2	Measuring Purity: Gini Impurity and Entropy	241
13.3	Choosing the Best Split	243
13.4	Growing the Whole Tree	246
13.5	Trees Overfit Easily: Depth, Leaf Size and Pruning	247
13.6	Regression Trees Predict Local Averages	250
13.7	Reading a Tree as Rules, and What Trees Do Well	252
14	Random Forests and Bagging	259
14.1	Why Many Trees Can Beat One	259
14.2	Bootstrap Sampling	262
14.3	Bagging: Bootstrap Aggregating	264
14.4	Random Forests Add Feature Randomness	266
14.5	Out-of-Bag Examples Give a Free Check	268
14.6	Feature Importance in a Forest	270
14.7	Tuning a Forest, and What It Cannot Do	273
15	Boosting and Stacking	280
15.1	Weak Learners and the Boosting Idea	281
15.2	AdaBoost: Re-weighting the Mistakes	282
15.3	Gradient Boosting: Fitting the Residuals	285
15.4	Learning Rate, Number of Rounds and Early Stopping	288
15.5	Modern Gradient-Boosting Libraries	290
15.6	Voting and Stacking Ensembles	291
15.7	Bagging, Boosting and Stacking Side by Side	294
16	Support Vector Machines	302
16.1	Many Lines Separate the Data — Which One Is Best?	302
16.2	The Margin and the Support Vectors	305

16.3	The Hard-Margin SVM	307
16.4	Soft Margins: Slack and the C Dial	309
16.5	The Hinge Loss View	313
16.6	Support Vector Regression: a Tube Instead of a Street	314
16.7	Scaling, Several Classes, and Practical Advice	316
17	Kernel Methods	322
17.1	When No Straight Line Works	322
17.2	Adding Features by Hand	324
17.3	The Kernel Trick: Similarity Instead of Coordinates	327
17.4	Common Kernels and the Gamma Parameter	330
17.5	Kernel SVM in Practice: Choosing C and Gamma Together	332
17.6	Kernels Beyond SVM: Kernel Ridge Regression	336
17.7	Cost, Scaling and When to Prefer a Kernel Method	338
IV	Unsupervised Learning: Finding Structure in Data	344
18	k-Means Clustering	345
18.1	Grouping Without Answers	345
18.2	The k-Means Algorithm, Step by Step	347
18.3	Inertia and Choosing k : the Elbow Method	350
18.4	The Silhouette Score	353
18.5	Initialization Matters: Random Starts and k-means++	355
18.6	Limits: Shapes k-Means Cannot Find, and Scale	357
18.7	Soft Clustering: Gaussian Mixture Models in Brief	358
18.8	Using Clusters: Profiles, Names, and Far-Out Points	360
19	Hierarchical and Density-Based Clustering	366
19.1	Clusters Inside Clusters: the Hierarchical Idea	366
19.2	Agglomerative Clustering Step by Step	367
19.3	Linkage Rules: Single, Complete, Average and Ward	371
19.4	Reading a Dendrogram and Cutting It	375
19.5	Density-Based Clustering: DBSCAN	376
19.6	Comparing k-Means, Hierarchical Clustering and DBSCAN	379
19.7	Distance Matrices, Scaling and Cost	381
20	Principal Component Analysis and Dimensionality Reduction	388
20.1	Too Many Features, Too Little Insight	389
20.2	Centering, Variance and the Direction of Greatest Spread	389
20.3	Projecting Points onto a Direction	391
20.4	Principal Components, One at a Time	393
20.5	Explained Variance and How Many Components to Keep	396
20.6	Loadings, Visualization and Rebuilding an Example	398
20.7	PCA Needs Scaling — and What It Cannot Do	401
20.8	Two Other Projections in Brief: LDA and t-SNE	403
21	Anomaly Detection	409
21.1	What Makes an Example Unusual?	409
21.2	Distance from the Center	412
21.3	z-Scores and Setting a Threshold	414
21.4	Local Outlier Factor: Crowded or Alone?	417
21.5	Isolation Forest: Anomalies Are Easy to Isolate	419
21.6	One-Class SVM in Brief	422

21.7	Evaluating a Detector and Acting on Alerts	423
V	Building Reliable and Responsible Models	431
22	Feature Engineering and Feature Selection	432
22.1	A Good Feature Is Worth More Than a Fancy Model	432
22.2	Combining Columns: Dates, Differences and Interactions	435
22.3	Reshaping One Column: Log Transforms and Bins	438
22.4	Encoding Text and Crowded Category Columns	440
22.5	Selecting Fewer, Better Features	441
22.6	Feature Importance Charts and Their Traps	445
22.7	A Feature-Engineering Checklist	447
23	Hyperparameter Tuning, Model Comparison, and Model Selection	453
23.1	Every Model Has Dials to Turn	454
23.2	Grid Search: Trying Every Combination	455
23.3	Random Search: Trying Many, Not All	458
23.4	Nested Cross-Validation and the Test-Set Trap	460
23.5	Comparing Models Fairly: Same Splits, Same Metric, With Spread	463
23.6	Model-Comparison Tables and Charts	465
23.7	Beyond the Score: Cost, Speed, Interpretability, Data Needs	467
23.8	A Checklist, and Reporting the Final Result	468
24	Interpretability, Fairness, Privacy, and Responsible Machine Learning	475
24.1	Why “It Works” Is Not Enough	476
24.2	Reading Coefficients and Feature Importance	477
24.3	Permutation Importance and Partial Dependence	478
24.4	Fairness: The Same Word, Different Meanings	482
24.5	Protected Attributes and Proxy Variables	484
24.6	Privacy: Personal Data, Anonymization and Consent	485
24.7	Drift, Monitoring and Safe Rollback After Launch	488
24.8	A Responsible-ML Checklist and the Human in the Loop	491
25	Capstone Projects	497
25.1	Project 1: Framing, Auditing and Splitting	497
25.2	Project 1: Baseline, Models, Test and Explanation	501
25.3	Project 2: Framing a Churn Decision and Splitting It Safely	507
25.4	Project 2: Calibrate, Choose a Threshold, Test Once, Watch	509
25.5	Six Project Ideas You Can Do	514
25.6	How to Present a Machine-Learning Project	515
VI	Reference	521
A	Mathematics When You Need It	522
A.1	Numbers, Rounding and Percentages	523
A.2	Scalars, Vectors and Matrices	524
A.3	Sums and the Sigma Notation	526
A.4	Mean, Variance and Standard Deviation	528
A.5	The Dot Product	529
A.6	Distance and Norm	531
A.7	Probability Basics	533
A.8	Functions, Slopes, Derivatives and Gradients	535
A.9	Exponentials and Logarithms	539

B	Python, NumPy, pandas and scikit-learn	543
B.1	Installing Python and the Libraries	543
B.2	The Small Amount of Python This Book Uses	545
B.3	NumPy Arrays	547
B.4	pandas DataFrames	549
B.5	The scikit-learn Pattern	551
B.6	Pipelines and ColumnTransformer	554
B.7	Plotting with matplotlib	556
B.8	Reproducibility	558
B.9	A Safe Checklist for Every Code Example	559
B.10	Common Error Messages and What They Mean	559
C	Datasets Used in This Book	563
	Glossary	568
	References	589

PART ONE

FOUNDATIONS OF MACHINE LEARNING

CHAPTER ONE

What Is Machine Learning?

How computers learn useful rules from examples

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what machine learning is and how it differs from normal, rule-based programming.
- Identify the samples, the features and the label inside a small dataset table.
- Tell classification, regression and clustering apart, and choose the right one for a new project.
- Describe the difference between supervised and unsupervised learning, and name reinforcement learning as a third path.
- Write a project's task, experience and performance measure (T, E, P) before choosing any model.
- Split data into a training set and a test set, and compute a model's accuracy by hand.
- Build and use a nearest-centroid classifier with nothing more than a pencil and a calculator.

Introduction

Think about buying watermelons at a market in Duhok. You want a sweet one, but you cannot taste it before you buy it. So you look at the color of the skin. You feel the weight in your hand. You knock on it with one knuckle and listen to the sound it makes. After buying and tasting many melons over many visits, you slowly get better at guessing which ones will be sweet. Nobody taught you an exact rule. You built your own rule from experience, one melon at a time.

This is the basic idea behind machine learning. A computer is given many examples, together with the correct answer for each one, and it works out a useful rule by itself. In this chapter you will meet the vocabulary that the rest of the book uses on every page: dataset, feature, label, model, training and testing. You will learn to recognize the three most common learning tasks, to write down a project's goal in three plain sentences, and to measure a model honestly. At the end you will build a complete classifier by hand, using only addition, subtraction, squaring and division.

1.1 Learning from Examples

A normal computer program starts with rules that a person writes. A programmer thinks carefully about a problem, writes exact steps, and the computer follows those steps every single time. Such a set of exact steps has its own name.

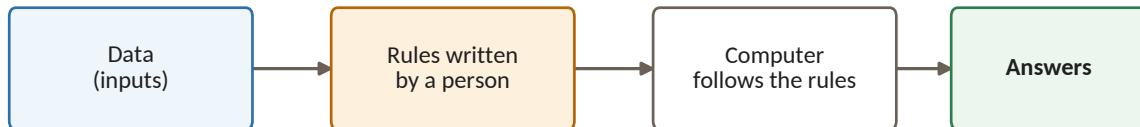
WORD TO KNOW

Algorithm — A clear, exact set of steps that solves a problem or finishes a task. A *learning algorithm* is the exact set of steps a computer follows to search for a useful pattern inside data.

In normal programming, the program receives data as input, follows the programmer's rules exactly, and produces an answer as output. If the rules are correct, the answer is correct. If the rules are wrong or incomplete, the answer is wrong, and only a person can go back and fix the rules by hand.

Machine learning changes one important part of this picture, shown in Figure 1.1. Instead of a person writing every rule, we give the computer many examples, along with the correct answer for each example. A learning algorithm studies these examples and searches for a useful rule by itself, without a person writing that rule directly.

Traditional programming



Machine learning

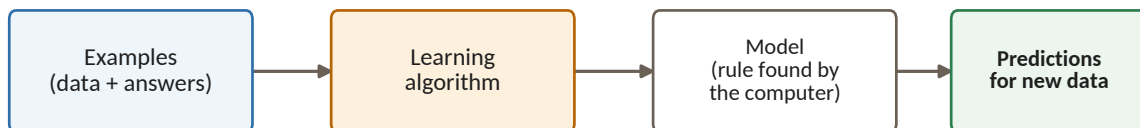


Figure 1.1 – Traditional programming (top) starts from rules written by a person. Machine learning (bottom) starts from examples and lets a learning algorithm find the rule.

WHAT TO OBSERVE

In the top row the rules are an *input*. In the bottom row the rule (the model) is an *output*. This single difference is the whole idea of machine learning: the computer writes the rule, and the person supplies the examples.

WORD TO KNOW

Machine learning — A way to build computer programs that find useful rules from data, instead of a person writing every rule by hand. The computer studies many examples and works out a pattern on its own.

The rule that comes out of this process also has its own name, and it is important not to confuse it with the algorithm that produced it.

WORD TO KNOW

Model — The rule that a learning algorithm produces after studying data. A model takes a new input and returns an output, such as a prediction.

The algorithm and the model are not the same thing, even though beginners often mix them up. Think about a cooking teacher and a recipe card. The teacher's method is how the recipe card gets written in the first place. The recipe card is what you actually use in your kitchen, again and again, afterward. In the same way, a learning algorithm is the method that studies data and builds a model. The model is the finished rule you actually use afterward to make new predictions. You run the learning algorithm only once, or once in a while,

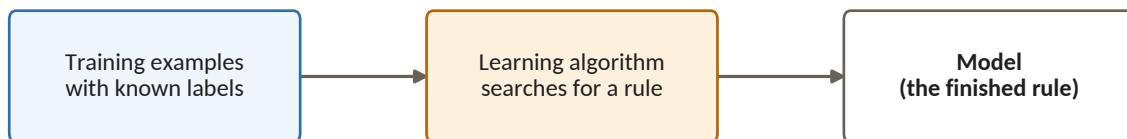
to train a model. You then use that trained model again and again, sometimes thousands or millions of times, without ever running the learning algorithm again.

WORD TO KNOW

Prediction — The output a model gives for one new example. A prediction can be a category, such as “sweet”, or a number, such as a weight in kilograms.

So the full loop looks like Figure 1.2. A learning algorithm studies training examples, together with their known answers, and produces a model. The finished model then looks at new inputs it has never seen before, and returns a prediction for each one. This one loop — examples in, model out, then new input in, prediction out — is the heart of every idea in this book. Every chapter after this one is really just a different way to build a better model, check it more honestly, or use it more responsibly.

TRAINING (done once)



USING THE MODEL (again and again)

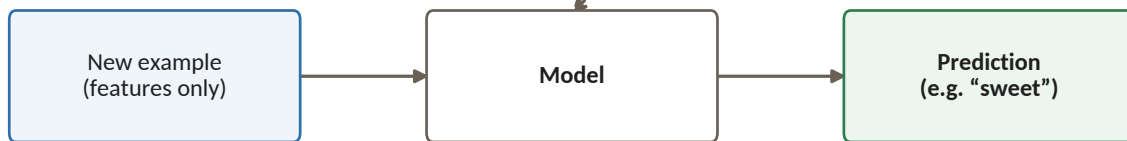


Figure 1.2 – The learning loop. Training happens once (top); the finished model is then used many times on new examples (bottom).

COMMON MISTAKE

✗ Wrong: Calling every computer program that makes automatic decisions “machine learning”, even a program that only follows fixed rules a person typed in by hand, such as “if the temperature is above 40, send a warning.”

✓ Right: Ask where the rule came from. If a person wrote the exact rule, it is normal programming. If a learning algorithm found the rule by studying labeled examples, it is machine learning.

1.2 Data, Samples, Features and Labels

Machine learning always begins with data. In this book, data almost always means a table, very much like a spreadsheet. Each row in the table is one example. Each column in the table is one measured property of that example.

WORD TO KNOW

Dataset — An organized collection of examples used for training or testing a model. In a table, one row is one example (also called a *sample*), and one column is one measured property.

Let us go back to the melon market and turn it into a small dataset. Imagine a market inspector who tastes four melons and writes down what she notices about each one. Melon A weighs 4.2 kilograms, made a hollow sound when tapped (a high sound score), and tasted sweet. Melon B weighs 2.8 kilograms, made a dull sound, and did not taste sweet. Melon C weighs 3.9 kilograms, made a hollow sound, and tasted sweet. Melon D weighs 3.1 kilograms, made a dull sound, and did not taste sweet. Table 1.1 shows this as a table.

Table 1.1 – The melon dataset: four samples, two features and one label

Melon (sample)	Weight (kg)	Sound score (0–10)	Sweet? (label)
A	4.2	8	Yes
B	2.8	3	No
C	3.9	7	Yes
D	3.1	2	No

This small collection of four melons, each with three pieces of information, is already a dataset. Weight and sound are the columns we could measure *before* tasting a melon. Sweetness is the column we are actually trying to predict.

WORDS TO KNOW

Feature — One measured property of an example, used as an input to a model. Weight, sound, age and price can all be features.

Label — The known, correct answer attached to one training example. A label can be a category, such as “sweet”, or a number, such as a price. When the answer is a number, it is often called the *target*.

In the melon dataset, weight and sound are features. Sweetness is the label. A model trained on this data studies the features of each melon and tries to predict the label. Later, when a new melon arrives without a known sweetness result, the model can look at its weight and sound and guess whether it will be sweet, before anyone has to cut it open. Figure 1.3 shows how the parts of the table are named.

FEATURES (inputs the model may use)			LABEL (the answer)
Melon	Weight (kg)	Sound score (0–10)	Sweet?
A	4.2	8	Yes
B	2.8	3	No
C	3.9	7	Yes
D	3.1	2	No

one row = one example (one sample) →

one column = one feature

Figure 1.3 – The anatomy of a dataset table. Each row is one sample; the blue columns are features (inputs); the green column is the label (the answer to predict).

WHAT TO OBSERVE

The label column is the only column the model is *not* allowed to see when it makes a prediction. Everything else in the row is available. Keeping that one column separate is the first rule of honest machine learning.

COMMON MISTAKE

✗ Wrong: Letting the label leak into the list of features by accident, for example allowing a model to see the sweetness result while it is also trying to predict sweetness.

✓ Right: Keep the label completely separate from the features you feed into a model. The model should only ever see columns that would be available before the true answer is known.

1.3 Three Common Learning Tasks

Not every machine learning project wants the same kind of answer. Some projects want a category. Some want a number. Some do not even have a known answer to learn from, only a pile of examples to sort into groups. This book begins with three of the most common learning tasks, shown side by side in Figure 1.4. You will meet all three again, in far more detail, in Parts Two, Three and Four.

WORDS TO KNOW

Classification — A learning task where the model predicts a category, chosen from a fixed, small list of possible answers. “Spam or not spam” and “sweet or not sweet” are both classification tasks.

Regression — A learning task where the model predicts a number that can take almost any value on a scale. Tomorrow’s temperature and a house’s selling price are both regression targets.

Clustering — A learning task where the model groups similar examples together, even though nobody gave the groups a name in advance. The computer finds the groups; a person studies each group afterward and decides what it seems to represent.

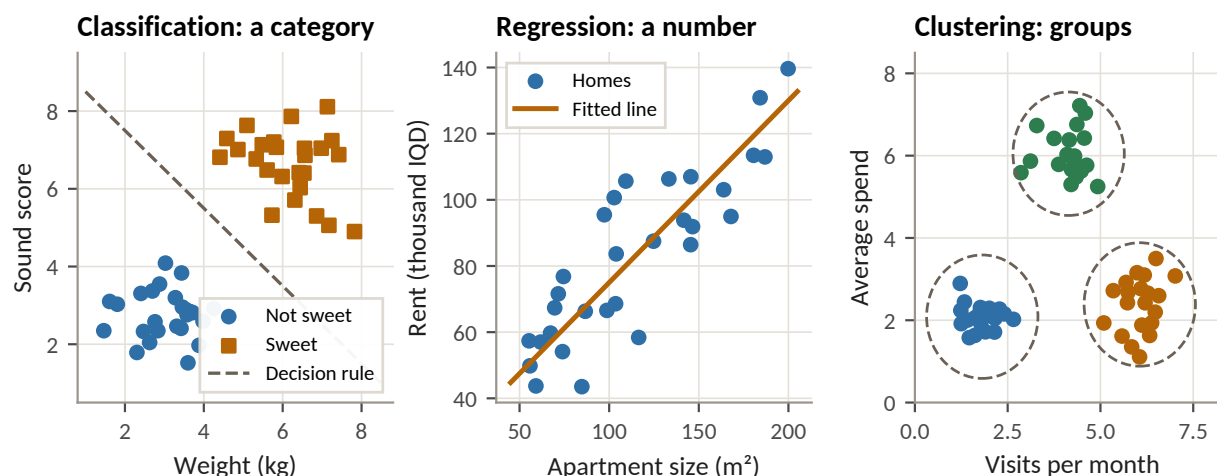


Figure 1.4 – The three most common learning tasks. Left: classification separates two categories with a decision rule. Middle: regression fits a line that predicts a number. Right: clustering finds groups in data that has no labels at all.

WHAT TO OBSERVE

In the left and middle panels every point already has an answer (a color, or a rent). In the right panel the points arrive with no answer; the dashed circles are groups the computer discovered by itself. Whether or not the data carries answers is the biggest dividing line in machine learning, and the next lesson gives it a name.

Classification and regression feel similar at first, because both predict something about a new example using its features. The real difference is the *shape of the answer*. Classification answers always come from a short, fixed list, such as “spam” or “not spam”. Regression answers can be almost any number on a continuous scale, such as 3.7 kilograms, 21.5 degrees Celsius, or 184,000 dollars. Here is a simple rule of thumb: if you can count the possible answers on your fingers, the task is probably classification; if the answer could be almost any number, the task is probably regression. Table 1.2 summarizes the three tasks.

Table 1.2 – Comparing the three learning tasks

Task	Shape of the answer	Everyday examples	Needs labels?
Classification	One category from a short fixed list	Spam or not spam; sweet or not sweet; which of five diseases	Yes
Regression	A number on a continuous scale	House price; tomorrow’s temperature; days until a pump fails	Yes
Clustering	A group number the computer invents	Types of shoppers; groups of similar villages; kinds of sensor readings	No

Clustering is a different kind of task altogether, because it does not use labels at all. Imagine an online store that wants to understand its customers better, but has never asked any customer to describe their own shopping “type”. Clustering can still group customers who buy similar products, at similar times, in similar amounts, into the same cluster, purely by noticing which customers look alike in the data. A person can then look inside each cluster and give it a sensible name, such as “weekend bargain shoppers”, but the clustering method itself never invents that name. It only finds who belongs together.

WORKED EXAMPLE 1.1 — Identify the Learning Task for Four Real Projects

A data team has four projects this month. Project 1: sort incoming e-mails into “spam” or “not spam”. Project 2: predict the price a house will sell for, in dollars. Project 3: group website visitors into similar groups, with no existing group names to learn from. Project 4: predict whether a bank loan will be repaid on time or not. Identify the learning task for each project.

1. Project 1 asks for one of two fixed categories, spam or not spam, so Project 1 is **classification**.
2. Project 2 asks for a price in dollars, which could be almost any number, so Project 2 is **regression**.
3. Project 3 has no known groups and no labels to learn from, only a request to find similar examples, so Project 3 is **clustering**.
4. Project 4 asks for one of two fixed categories, repaid or not repaid, so Project 4 is **classification**.

Answer: Projects 1 and 4 are classification, Project 2 is regression, and Project 3 is clustering. Two projects can use the same learning task even when their subjects, e-mails and bank loans, have nothing else in common.

COMMON MISTAKE

X Wrong: Assuming that any label written as a number must mean regression, even when the number is really just a category code, such as “1” for spam and “0” for not spam.

✓ Right: Look at what the number actually means. If the values come from a small, fixed set of categories, the task is still classification, even when those categories happen to be written as digits.

1.4 Supervised, Unsupervised and Reinforcement Learning

Classification and regression share one important feature: both of them learn from examples that already carry a known, correct answer. Clustering is different, because it learns from examples that have no known answer at all. This difference is important enough that machine learning gives it its own pair of names (Figure 1.5).

WORDS TO KNOW

Supervised learning — Learning from examples that already have a correct label attached. Classification and regression are both supervised learning, because every training example already carries its own answer.

Unsupervised learning — Learning from examples that have no label at all. Clustering is unsupervised learning, because the model only ever sees the features, never a correct answer.

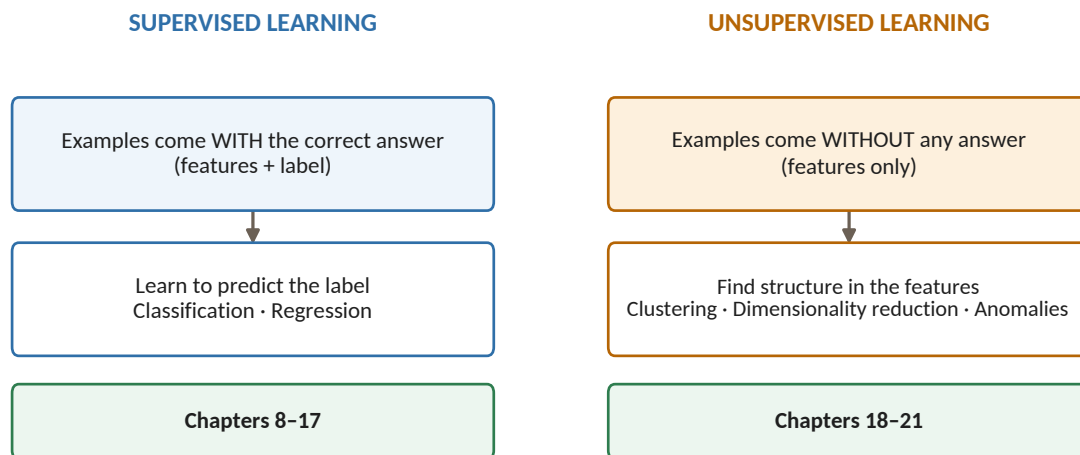


Figure 1.5 – Supervised learning uses examples that carry answers; unsupervised learning finds structure in examples that do not.

The word “supervised” is a helpful picture to hold in your mind. Imagine a teacher supervising a student while the student practices a new skill, correcting each mistake as it happens. Supervised learning works in a similar way: every training example already carries the correct answer, so the learning algorithm can check its own guesses against a known truth while it learns, mistake by mistake. Unsupervised learning has no such supervisor. The learning algorithm only ever sees the raw features, with no answer key at all, and it must find structure using the features alone. This is exactly why clustering can group similar customers together, but can never tell you on its own what makes that group special. Only a person, looking at the finished groups, can add that meaning.

Most of this book focuses on these two families, because together they cover the large majority of real machine learning projects. But it is worth naming one more family here, even though this book will not teach it in depth.

WORD TO KNOW

Reinforcement learning — A third kind of learning, not covered in this book, where a computer program called an *agent* takes actions inside an environment and learns from rewards and penalties, instead of from labeled examples.

Reinforcement learning shows up in projects such as game-playing programs and robots learning to walk, where there is no ready-made table of correct answers, only a score that goes up or down depending on what the agent does. Nobody hands a chess program a table of “correct moves” for every possible position, but the program can play millions of practice games against itself and slowly learn which actions tend to lead to a win. This book stays with supervised and unsupervised learning, which cover the great majority of everyday business, science and engineering projects, but it is useful to recognize the name “reinforcement learning” when you meet it elsewhere. (Deep neural networks, another famous family of models, are covered in the companion book *Artificial Neural Networks for Everyone*; everything in this book stands on its own without them.)

1.5 A Precise Learning Goal: Task, Experience and Performance

Before choosing any model, a careful machine learning project first answers three simple questions, in plain words. What exactly must the model do? What data will it learn from? How will we know whether it worked? These three questions have a short name in this book: T, E and P, for Task, Experience and Performance. This way of defining learning was written down by the computer scientist Tom Mitchell in 1997, and it is still the clearest starting point for any project.

WORDS TO KNOW

Task (T) — A plain sentence describing what the model must predict, and for what kind of example.

Experience (E) — The exact data the model will learn from: how many examples, and where they came from.

Performance (P) — How success will be measured once a trained model exists — the number that tells us whether the model is good enough.

Writing all three down, in order, before touching any data or any model, keeps a project honest and focused from the very first day. Table 1.3 shows the three parts for the project in the next worked example.

WORKED EXAMPLE 1.2 — Define a Project Using Task, Experience and Performance

An online store in Erbil ships thousands of orders every week. A small number of orders turn out to be fraudulent, paid for with a stolen card. The store manager wants to use machine learning to flag risky orders before they ship. Define this project using Task, Experience and Performance.

1. Task (T): for each new order, decide whether it is likely fraudulent or likely genuine, before the order ships.
2. Experience (E): 5,000 past orders, each one already labeled by the store’s fraud team as “fraudulent” or “genuine”.
3. Performance (P): measure how many of the truly fraudulent orders the model catches, and separately measure how many genuine orders it wrongly flags, because these two kinds of mistakes cost the store very differently.
4. Only after T, E and P are written down in plain sentences should the team choose which model to train.

Answer: This is a classification task: predict “fraudulent” or “genuine” for each order, learn from 5,000 labeled past orders, and judge success by how many real fraud cases are caught and how many honest customers are wrongly bothered — not by accuracy alone.

Table 1.3 – Task, Experience and Performance for the Erbil fraud project

Part	Plain-language statement
Task (T)	Flag each new order as likely fraudulent or likely genuine, before it ships
Experience (E)	5,000 past orders, each labeled “fraudulent” or “genuine” by the fraud team
Performance (P)	Fraud cases caught, and genuine orders wrongly flagged, counted separately

IMPORTANT

A high score on the wrong performance measure can hide a serious problem. For a health project, missing one sick patient is usually far more costly than sending one healthy patient for an extra check. Always choose the performance measure P before choosing a model, and make sure it truly matches what matters most in the real project. Chapter 6 is entirely about choosing P well.

Notice that none of the three questions mentions any specific algorithm. That is intentional. A common beginner habit is to start a project by picking a famous algorithm, then forcing the data to fit it. A far safer habit, and the one this whole book follows, is to start with T, E and P, and only choose an algorithm once those three things are completely clear.

1.6 Training, Testing and Accuracy

Once T, E and P are clear, and a dataset is ready, the actual learning can begin. But there is an important trap waiting here, and this book returns to it again and again: a model that looks perfect on the data it studied can still fail badly on new data it has never seen.

WORD TO KNOW

Training — The process of using examples to fit, or adjust, a model. During training, the model changes its internal rule so that it matches the training examples more closely.

To catch this trap, we never judge a model using only the data it trained on. Instead, we set aside a separate group of examples before training even begins, and we do not let the model see this group until training is completely finished (Figure 1.6).

WORDS TO KNOW

Training set — The examples a model learns from.

Test set — A group of examples kept completely separate from training, then used once to check how well the finished model performs on data it has never seen before.

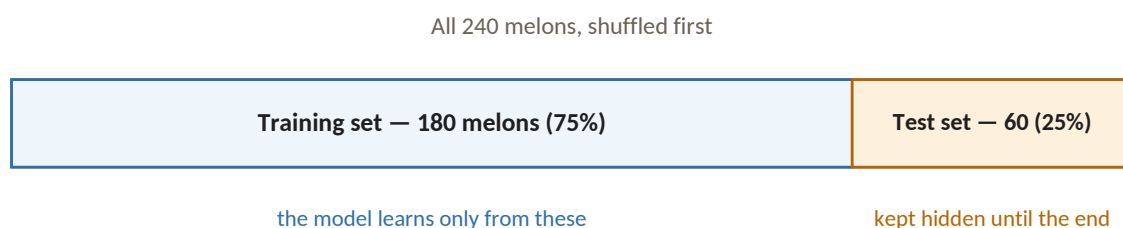


Figure 1.6 – Splitting 240 melons into a training set and a test set. The data is shuffled first so that both sets are a fair mix.

WORKED EXAMPLE 1.3 — Split 240 Melons into a Training Set and a Test Set

A market inspector collects careful measurements on 240 melons across one harvest season. The team decides to keep 25% of the melons aside as a test set, and use the rest to train the model. How many melons go into each set?

1. Start with all 240 melons.
2. Find 25% of 240, because that fraction becomes the test set: $0.25 \times 240 = 60$.
3. The test set therefore contains 60 melons.
4. The training set gets the rest: $240 - 60 = 180$.
5. Check by adding the two groups back together: $180 + 60 = 240$, which matches the original total.

Answer: 180 melons go into the training set, and 60 melons go into the test set. The model will learn its rule only from the 180 training melons, and the 60 test melons stay hidden until the very end.

Once a model is trained, we can finally check it honestly, by letting it make predictions on the test set and comparing each prediction to the true, known label. One simple, common performance measure for classification counts how many of those predictions were exactly correct.

WORD TO KNOW

Accuracy — The fraction of predictions a model gets exactly right, usually written as a percentage. Accuracy is one common performance measure, though later chapters show it is not always the right one.

In symbols, if the model made n predictions and c of them were correct, accuracy is the ratio of the two counts.

EQUATION: ACCURACY

$$\text{Accuracy} = \frac{c}{n}$$

In words: Accuracy equals the number of correct predictions divided by the total number of predictions made.

When it is used: After a classifier has made predictions on a test set, to report the fraction it got exactly right. Use it when every kind of mistake costs about the same.

Symbol	Meaning	Unit or type
c	number of correct predictions on the test set	count
n	total number of predictions made on the test set	count

Multiply the result by 100 to write accuracy as a percentage, so 0.85 becomes 85%.

WORKED EXAMPLE 1.4 — Compute the Test Accuracy of the Melon Classifier

The trained melon classifier is used on all 60 test melons. It correctly predicts sweetness or non-sweetness for 51 of them, and gets the other 9 wrong. Find its test accuracy.

1. Identify the two numbers needed: correct predictions $c = 51$, and total predictions $n = 60$.
2. Write the equation: $\text{Accuracy} = c \div n$.
3. Put in the numbers: $51 \div 60 = 0.85$.
4. Turn the result into a percentage: $0.85 \times 100 = 85$.

Answer: The model's test accuracy is 85%. Out of every 100 melons like the ones in the test set, the model is expected to judge sweetness correctly for about 85 of them.

COMMON MISTAKE

X Wrong: Reporting the accuracy a model achieves on the very same rows it was trained on, and presenting that number as proof the model is ready to use.

✓ Right: Always compute accuracy on a separate test set that the model never saw during training. Training accuracy is almost always too optimistic, sometimes wildly so; Chapter 7 explains exactly why.

1.7 Build a Classifier by Hand: Nearest Centroid

This lesson builds a complete, simple classifier by hand, from start to finish, using only arithmetic you already know: addition, subtraction, division, and squaring a number. The method is called *nearest-centroid classification*. Despite its long name, the idea is simple: find the “average melon” for each class, then classify a new melon by seeing which average melon it resembles most closely.

WORD TO KNOW

Centroid — The average position of a group of points. To find a centroid, average each feature separately across every point in the group.

A centroid is found one feature at a time. For each feature, add up that feature’s values across the group and divide by the number of points.

EQUATION: CENTROID OF A GROUP, ONE FEATURE AT A TIME

$$\bar{v} = \frac{\sum_{i=1}^n v_i}{n}$$

In words: A centroid’s value for one feature is the sum of that feature’s values across every point in the group, divided by the number of points. Repeat this once for each feature, completely separately.

When it is used: Whenever you need the “typical” or “average” point of a group — in nearest-centroid classification here, and again in k-means clustering in Chapter 18.

Symbol	Meaning	Unit or type
$\bar{v}$	the centroid’s value for one feature	same unit as that feature
v_i	the feature’s value for point i in the group	same unit as that feature
n	the number of points in the group	count

A centroid with two features, such as weight and sound score, is really just two of these one-feature averages, computed side by side and never mixed together.

Suppose the inspector measures two features for four melons she has already tasted: weight, in kilograms, and a sound score from 0 to 10 given by a handheld ripeness meter, where a higher score means a hollower, riper-sounding thump. Two of the melons were ripe. Two were not yet ripe. We will find the centroid, the “average melon”, for each class, then use both centroids to judge a brand-new melon. To decide which centroid a new melon is closer to, we need one more small formula: a way to measure distance between two points that each have two features.

EQUATION: SQUARED DISTANCE BETWEEN TWO POINTS (TWO FEATURES)

$$d^2 = (a_1 - a_2)^2 + (b_1 - b_2)^2$$

In words: Subtract the two points’ first-feature values and square the result; subtract their second-feature values and square that result; then add the two squared results together.

When it is used: To compare how close a point is to several other points. Because we only compare distances, the square root is not needed — the smaller squared distance is always the smaller distance.

Symbol	Meaning	Unit or type
d^2	squared distance between point 1 and point 2	squared feature units
a_1, a_2	the first feature of point 1 and of point 2	same unit as that feature
b_1, b_2	the second feature of point 1 and of point 2	same unit as that feature

The true (Euclidean) distance is $\sqrt{d^2}$. Chapter 11 uses the square-rooted version; this chapter only needs to know which distance is smaller.

WORKED EXAMPLE 1.5 — Classify a New Melon Using Nearest Centroid

Ripe melon 1 weighs 4 kg with a sound score of 8. Ripe melon 2 weighs 6 kg with a sound score of 10. Unripe melon 1 weighs 2 kg with a sound score of 2. Unripe melon 2 weighs 2 kg with a sound score of 4. A new melon, Q, weighs 5 kg and has a sound score of 7. Use the nearest-centroid method to classify melon Q as Ripe or Unripe.

1. Find the Ripe centroid's weight by averaging the two Ripe weights: $(4 + 6) \div 2 = 5$.
2. Find the Ripe centroid's sound score: $(8 + 10) \div 2 = 9$. The Ripe centroid is (5, 9).
3. Find the Unripe centroid's weight: $(2 + 2) \div 2 = 2$.
4. Find the Unripe centroid's sound score: $(2 + 4) \div 2 = 3$. The Unripe centroid is (2, 3).
5. Squared distance from Q (5, 7) to the Ripe centroid (5, 9): weight difference $5 - 5 = 0$, so $0^2 = 0$; sound difference $7 - 9 = -2$, so $(-2)^2 = 4$. Add: $0 + 4 = 4$.
6. Squared distance from Q (5, 7) to the Unripe centroid (2, 3): weight difference $5 - 2 = 3$, so $3^2 = 9$; sound difference $7 - 3 = 4$, so $4^2 = 16$. Add: $9 + 16 = 25$.
7. Compare: 4 is smaller than 25, so melon Q sits closer to the Ripe centroid.

Answer: Melon Q is classified as Ripe. Its weight and sound score, taken together, sit much closer to the typical ripe melon than to the typical unripe melon.

Figure 1.7 shows the four training melons, the two centroids, melon Q, and a second new melon R that sits between the two groups.

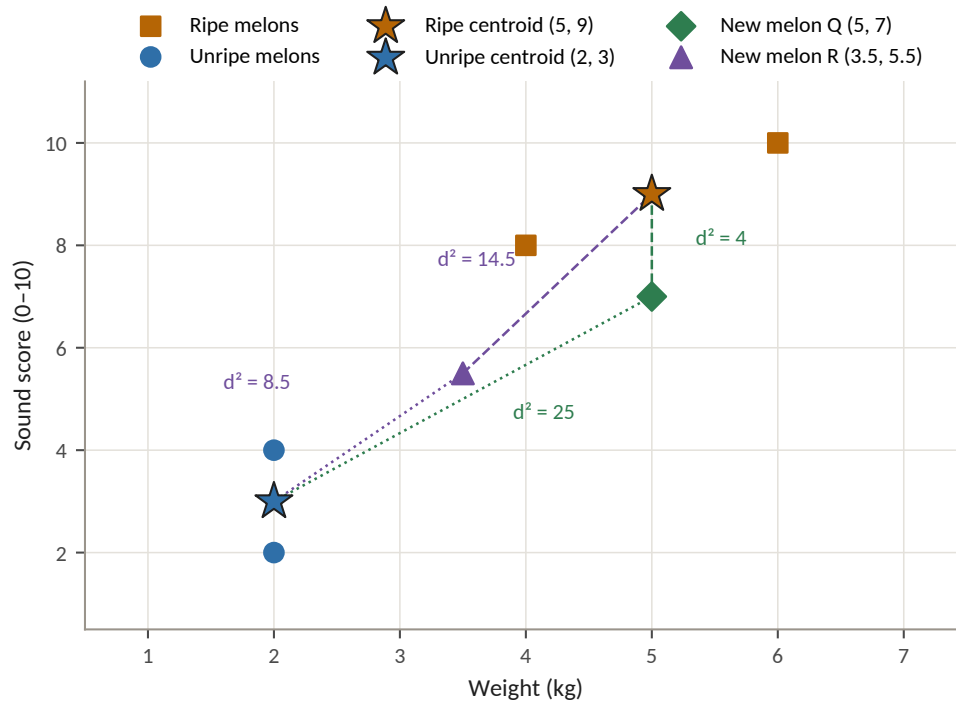


Figure 1.7 – Nearest-centroid classification of two new melons. Stars mark the centroids; dashed lines lead to the Ripe centroid and dotted lines to the Unripe centroid, labeled with the squared distances.

WHAT TO OBSERVE

Melon Q is clearly nearer to the Ripe star. Melon R looks “in between”, but its dotted line to the Unripe centroid ($d^2 = 8.5$) is still shorter than its dashed line to the Ripe centroid ($d^2 = 14.5$), so the method classifies it as Unripe. The classifier always has an answer, even for points near the boundary — which is why a test set is needed to find out how often that answer is right.

WORKED EXAMPLE 1.6 — Classify a Second Melon Near the Boundary Between Classes

Using the same two centroids as before — Ripe at (5, 9) and Unripe at (2, 3) — classify a second new melon, R, which weighs 3.5 kg and has a sound score of 5.5.

1. Squared distance from R (3.5, 5.5) to the Ripe centroid (5, 9): weight difference $3.5 - 5 = -1.5$, so $(-1.5)^2 = 2.25$; sound difference $5.5 - 9 = -3.5$, so $(-3.5)^2 = 12.25$. Add: $2.25 + 12.25 = 14.5$.
2. Squared distance from R (3.5, 5.5) to the Unripe centroid (2, 3): weight difference $3.5 - 2 = 1.5$, so $1.5^2 = 2.25$; sound difference $5.5 - 3 = 2.5$, so $2.5^2 = 6.25$. Add: $2.25 + 6.25 = 8.5$.
3. Compare: 8.5 is smaller than 14.5, so melon R sits closer to the Unripe centroid, even though its numbers sit between the two groups.

Answer: Melon R is classified as Unripe. The method always picks whichever centroid is closer, and here Unripe wins clearly, 8.5 against 14.5.

Notice that neither example ever takes a square root. We only need to know which distance is smaller, and squared distances put the two candidates in the same order that true distances would, so the square-root step can simply be skipped to save work.

COMMON MISTAKE

X Wrong: Comparing raw distances when one feature uses much bigger numbers than another, for example weight measured in grams sitting next to a sound score from 0 to 10 — the large-number feature then quietly dominates every distance.

✓ Right: Put features on similar scales before comparing distances between them. Chapter 4 explains exactly how, using standardization, and Chapter 11 puts it to work on distance calculations just like this one.

1.8 Where This Book Goes Next

Everything you have met in this chapter — features and labels, three tasks, two learning families, T/E/P, training and testing — returns in every later chapter. Figure 1.8 shows the road ahead. Part One builds the foundations that every project needs: a safe workflow, clean data, honest testing and the right performance measure. Parts Two and Three teach the classical supervised models one at a time, always by hand first and then in Python. Part Four turns to data without labels. Part Five is about building models you can trust and defend, and finishes with two complete projects.

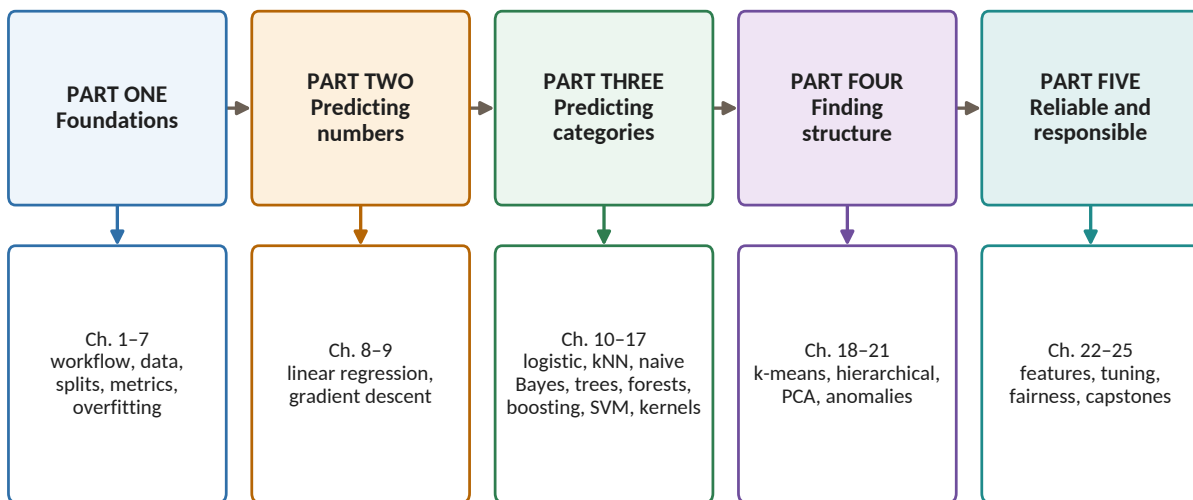


Figure 1.8 – The five parts of this book and what each one covers.

COMMON MISTAKES AND MISCONCEPTIONS

- Calling every automatic decision “machine learning”, even one that only follows fixed rules a person wrote by hand.
- Confusing the learning algorithm (the method) with the model (the finished rule it produces).
- Letting the label leak into the features, so the model can see the very answer it is supposed to predict.
- Assuming a number-shaped label always means regression, even when the number is really a small set of category codes.
- Reporting accuracy measured on the training data itself, instead of on a separate, untouched test set.
- Comparing raw distances between features that sit on very different scales, letting one large-number feature dominate.

- Choosing a model before writing down the task, the experience and the performance measure in plain sentences.

Practical Activities

ACTIVITY (PAPER) — Be a Data Detective

1. Choose something you buy often — fruit, bread, phone credit, a bus ticket. Write down five things about it you could measure or observe *before* you decide whether it was a good buy.
2. Make a table with six rows (six real or imagined purchases). Give each row a value for every one of your five columns, and add one final column that records the answer you would like to predict (for example “worth it: yes/no” or “price paid”).
3. Mark each column as a feature or as the label. Decide whether predicting your label would be classification or regression, and write one sentence explaining why.
4. Write the Task, Experience and Performance for your little project in three plain sentences.

You need: Paper and pen

ACTIVITY (PYTHON) — Nearest Centroid with NumPy and scikit-learn

The following program repeats the exact arithmetic of Worked Examples 1.5 and 1.6, first with NumPy, then with scikit-learn’s ready-made NearestCentroid classifier.

Python · NumPy · scikit-learn

```
import numpy as np
from sklearn.neighbors import NearestCentroid

X = np.array([[4, 8], [6, 10], [2, 2], [2, 4]]) # weight (kg), sound score
y = np.array(["Ripe", "Ripe", "Unripe", "Unripe"])

c_ripe = X[y == "Ripe"].mean(axis=0) # steps 1-2
c_unripe = X[y == "Unripe"].mean(axis=0) # steps 3-4
for name, q in [("Q", np.array([5, 7])), ("R", np.array([3.5, 5.5]))]:
    d_ripe = ((q - c_ripe) ** 2).sum() # squared distances
    d_unripe = ((q - c_unripe) ** 2).sum()
    print(name, "d2 to Ripe:", d_ripe, " d2 to Unripe:", d_unripe,
          "->", "Ripe" if d_ripe < d_unripe else "Unripe")

model = NearestCentroid().fit(X, y) # the same idea, built in
print(model.centroids_)
print(model.predict([[5, 7], [3.5, 5.5]]))
```

Output

```
Q d2 to Ripe: 4.0 d2 to Unripe: 25.0 -> Ripe
R d2 to Ripe: 14.5 d2 to Unripe: 8.5 -> Unripe
[[5. 9.]
 [2. 3.]]
['Ripe' 'Unripe']
```


The first lines store the four training melons and their labels. `mean(axis=0)` averages each column separately, which finds the two centroids exactly like steps 1 to 4. The two `d_` lines subtract, square and add, exactly like steps 5 and 6. Running the code prints the same centroids and the same squared distances, 4 and 25 for Q and 14.5 and 8.5 for R, that we found by hand — and scikit-learn’s built-in classifier agrees.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Finding Students Who May Need Extra Support at a Secondary School in Erbil

A secondary school in Erbil wants to find students who could benefit from extra academic support, early enough in the term for that support to actually help. Right now, teachers notice struggling students mostly by accident, often not until after a student has already failed a major exam. The school’s counseling office wonders whether machine learning could help them notice earlier, so a counselor can step in while there is still time to change the outcome.

The counseling team starts exactly the way this chapter recommends, with Task, Experience and Performance, instead of picking a model first. The *task* is to flag, by the end of the fourth week of term, which students may benefit from a short supportive check-in with a counselor. The *experience* is five years of past student records: attendance rate, number of completed homework assignments, and quiz results from the first four weeks of each past term, together with a label the school already knows for past students — whether that student later needed real academic support that term. The *performance measure* is not simple accuracy. The team cares much more about not missing a student who truly needed help than about one extra, unnecessary check-in, so they agree to track both kinds of mistakes separately, not just one combined score.

With T, E and P written down, the features and the label become obvious. The features are attendance rate, completed assignments and early quiz scores, all measured only through week four. The label is whether that student, looking back, turned out to need real support that term. This is a classification task, because the answer is one of two categories, not a number on an open scale. The team splits the five years of records into a training set and a test set before doing anything else, so that the final check is honest.

The team is also careful about what the data cannot tell them. A low attendance rate can come from many different causes: a part-time job, a family responsibility, a long walk to school, or an illness. The data does not know which cause applies to any one student, only the number itself. For this reason, the school agrees that the model’s output will only ever create a private list for a trained counselor to review, never an automatic mark on a student’s official record, and never a decision made without a caring adult looking at the whole picture first.

RESPONSIBLE USE

A prediction is not a fact about a person, and it is not proof of what will happen in the future. A model trained on old records can also learn old, unfair patterns without anyone intending it. Always use a sensitive prediction to offer help and to trigger a caring human review, never to remove a person’s opportunity automatically. Chapter 24 returns to this in depth.

CHAPTER SUMMARY

- Machine learning builds a model by studying labeled examples; the learning algorithm produces the model, and the model then produces predictions on new, unseen inputs.
- A dataset is a table of examples (samples); features are the input columns, and the label (or target) is the known-answer column.
- Classification predicts a category, regression predicts a number, and clustering finds groups with no given labels.

- Supervised learning uses labeled examples; unsupervised learning does not; reinforcement learning learns from rewards instead of labels.
- Every project should state its task (T), experience (E) and performance measure (P) before any model is chosen.
- Training fits a model to data; a separate test set gives an honest check on data the model has never seen.
- Accuracy is the fraction of test predictions that were exactly correct, $c \div n$, often written as a percentage.
- A nearest-centroid classifier finds the average point of each class, then predicts whichever class's centroid is closest to a new point, using squared distances.
- Features on very different scales must be rescaled before distances are compared.
- A prediction is a helpful guide for a human decision, never proof of the future and never a fact about a person.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain the difference between a learning algorithm and a model. Use the recipe-card picture or one of your own.

Problem 2 [Basic] A dataset has columns for a car's age, mileage and selling price. Which column is most likely the label, if the goal is to predict resale value? Is the task classification or regression?

Problem 3 [Basic] State whether each task is classification or regression: (a) predicting whether a tumor is dangerous or harmless, (b) predicting a runner's finishing time in minutes, (c) predicting which of six crops a farmer planted from a satellite photo.

Problem 4 [Basic] A dataset has 800 rows. The team keeps 20% for testing. How many rows are in the training set, and how many are in the test set?

Problem 5 [Basic] Define supervised learning and unsupervised learning in one sentence each, and give one example task of each kind.

Problem 6 [Standard] A classifier gets 72 correct predictions out of 80 test examples. Calculate its test accuracy as a percentage, showing every step.

Problem 7 [Standard] Class A contains the points (2, 2) and (4, 6). Find its centroid, showing the average of each feature separately.

Problem 8 [Standard] Class B has centroid (7, 5). A new point is $q = (5, 4)$. Find the squared distance from q to Class B's centroid, showing every step.

Problem 9 [Standard] Using the centroid of Class A from Problem 7 and the centroid of Class B from Problem 8, classify the point $q = (5, 4)$ by the nearest-centroid method.

Problem 10 [Standard] A shop groups its customers into similar buying patterns without using any pre-existing group names. Name the learning task, and explain in one sentence why it is not classification.

Problem 11 [Standard] Write the Task, Experience and Performance for a project that predicts whether a delivery will arrive late.

Problem 12 [Challenge] A model reaches 99% accuracy on its training data but only 71% accuracy on its test data. (a) What warning sign does this gap give? (b) Which of the two numbers should you trust more when deciding if the model is ready to use, and why?

Problem 13 [Challenge] Two ripe melons have features $(3, 9)$ and $(5, 7)$, and two unripe melons have features $(1, 3)$ and $(3, 1)$. A new melon is $(3, 5)$. (a) Find both centroids. (b) Show that the new melon is exactly the same squared distance from each centroid. (c) Suggest one sensible rule a program could use to break such a tie.

Problem 14 [Challenge] Design a small machine learning idea for your own school, workplace or town. State the task, the experience, the performance measure, and one possible risk if the prediction were used carelessly.

Problem 15 [Challenge] In the nearest-centroid method, weight is in kilograms and the sound score runs from 0 to 10. Suppose weight were recorded in grams instead (4000 g instead of 4 kg). Recompute the squared distances of melon Q to both centroids and explain what goes wrong.

CHAPTER TWO

The Machine Learning Workflow

A safe order for every project, from the decision to the final test

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a project should begin with a decision that a person needs to make, not with the name of an algorithm.
- Write a one-sentence decision statement and name the unit of observation, the target and the prediction time for a new project.
- Recognize data leakage, and check whether a candidate feature is safe to use at prediction time.
- Split a dataset into training, validation and test sets, by random choice or by date, and count the rows in each set.
- Build a mean baseline and a majority-class baseline by hand, and measure them with the absolute error and with accuracy.
- Compare candidate models on the validation set, and explain why the test set is opened only once.
- Use the eight-step workflow as a checklist for any new project.

Introduction

Every morning, a busy public clinic in Sulaymaniyah fills up with walk-in patients, and almost every one of them asks the same question at the front desk: “How long will I wait?” For years the honest answer has been “we do not know.” Patients grow frustrated, some leave without being seen, and the staff spend their time answering the same question again and again. The clinic director would like to give every patient an honest estimate at check-in. A friend suggests a famous algorithm she read about online. Is that the right place to start?

It is not. Building a machine learning project is a little like building a house. The tools matter, but the *order* of the steps matters just as much. Nobody paints the walls before the roof is on, and nobody should train a model before deciding what question it must answer, what one row of data means, and which information will really be available at the moment the prediction is needed.

This chapter gives you a safe order of steps that works for every project in this book, from a clinic in Sulaymaniyah to a factory in Da Nang: begin with the decision, name what one row means, look at the data, split it, build the simplest possible predictor first, compare candidate models fairly, test once, and keep watching after launch. Every calculation is done by hand with tiny datasets first, then repeated in Python with the same answers.

2.1 Begin with the Decision, Not the Algorithm

Chapter 1 gave you the full loop of machine learning: examples go in, a model comes out, and the model then makes predictions on new data. This chapter gives you something just as important: a safe order of steps for turning a real-world question into a working project.

WORD TO KNOW

Workflow — An ordered set of steps used to complete a project in a safe, repeatable way. Following the same workflow every time helps a whole team avoid easy-to-make mistakes.

Figure 2.1 shows the eight steps of the workflow this book follows. Notice that training a model is step 6, not step 1, and notice the arrow that loops back from the last step to the first: a real project is never quite finished.

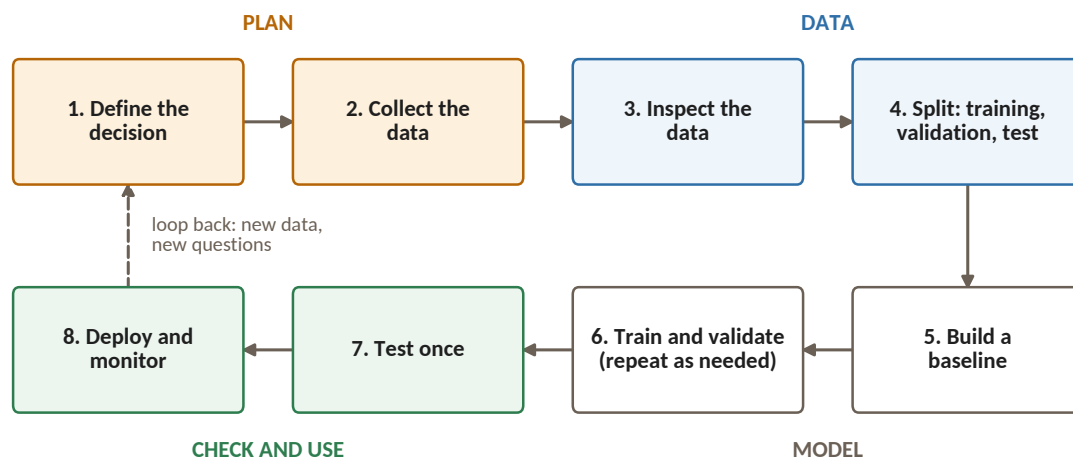


Figure 2.1 – The eight-step machine learning workflow. The first four steps happen before any model exists; the last step loops back to the first because a project is never finished for good.

WHAT TO OBSERVE

Four of the eight steps are finished before any learning algorithm runs, and only step 6 is about training a model. The dashed loop-back arrow means that deploying a model is not the end of the story: a model that is not watched slowly goes out of date.

A machine learning project should always begin with a decision that some person needs to make, not with the name of an algorithm. “Let’s use the newest algorithm” is not a project goal, no matter how impressive it sounds. “Predict how long a patient will wait, so the front desk can tell them at check-in” is a real project goal, because it names a decision, a person who needs it, and the moment when the answer becomes useful. One simple habit keeps this straight from day one: before touching any data, write one plain sentence with three blanks in it.

WORD TO KNOW

Decision sentence — One plain sentence, written before any data work begins, of the form: “At prediction time, the model will use ___ to predict ___, so that ___.” The first blank names the information available at that moment, the second names the answer wanted, and the third names who uses the answer and for what.

Table 2.1 shows what each blank means and how the clinic fills it in. If nobody can fill in the third blank, the project has no customer. If the first blank contains information that is only known *after* the answer is needed, the project will fail in a way that no algorithm can repair.

Table 2.1 – The decision sentence template and the clinic’s answer

Blank	Question it answers	The clinic’s answer
“will use ____”	What information is really available at the moment of prediction?	the weekday, the check-in hour, the patients already waiting, the clinicians on duty
“to predict ____”	What answer do we want, for one example?	how many minutes this patient will wait
“so that ____”	Who uses the answer, and for which decision?	the front desk can tell the patient an honest estimate at check-in

WORKED EXAMPLE 2.1 — Write the Decision Sentence for the Clinic Project

The clinic manager in Sulaymaniyah wants to write one decision sentence before any data work begins: “At prediction time, the model will use ____ to predict ____, so that ____.” Fill in the three blanks.

1. Decide the moment of prediction, the instant a patient checks in, and list the information available at that exact moment: the day of the week, the check-in hour, the number of patients already waiting, and the number of clinicians on duty.
2. Name the answer the model must produce: the number of minutes this patient will wait before being seen.
3. Name the purpose: so the clinic can show the patient an honest estimate at check-in, instead of no information at all.

Answer: “At prediction time, the model will use the day, the hour, the number of patients waiting and the number of clinicians on duty to predict how many minutes this patient will wait, so that the front desk can show the patient an honest estimate at check-in.” One clear sentence keeps the whole project pointed at a real decision.

The decision sentence is a sharper version of the Task from the T, E, P questions in Chapter 1: it also fixes *when* the prediction is made and *who* acts on it. Notice that it still says nothing about which algorithm to use. That choice comes later, almost as a detail.

COMMON MISTAKE

✗ Wrong: Choosing an impressive-sounding algorithm first, then trying to force the data and the question to fit it.

✓ Right: Write the plain decision sentence first: what the model will use, what it will predict, and why. Choose an algorithm only after that sentence is completely clear.

2.2 Name the Unit, Target and Prediction Time

The decision sentence hides three separate ideas inside it, and each one deserves its own attention before any data is collected. Think about the clinic’s spreadsheet. One receptionist types one row per patient. Another receptionist, on Thursdays, types one row for the whole morning, with the total number of patients and the total minutes waited. Both kinds of row contain a “waiting time” column, and both look perfectly tidy. A model trained on this table would receive two completely different kinds of example with no way to tell them apart.

WORD TO KNOW

Unit of observation — The real-world thing that one row of a dataset represents. A unit of observation might be one person, one house, one e-mail, one machine, or, as in this chapter's clinic example, one patient visit.

Always state the unit of observation in one plain sentence before collecting a single row, and check that every row matches it. The second idea inside the decision sentence is the *target*, the answer to be predicted for each unit. You met this word in Chapter 1: the target is the project-level idea of what we want to predict, and the label is that same answer once it is attached to real training rows. Most people use the two words interchangeably once a project is running, and this book does too. The third idea is the one beginners most often forget, and it decides which columns are allowed to be features at all.

WORD TO KNOW

Prediction time — The exact moment in the real world at which the model must produce its answer. Only information that already exists at that moment may be used as a feature; everything that becomes known later is off limits.

WORKED EXAMPLE 2.2 — Name the Unit, Target and Prediction Time for the Clinic Project

Using the clinic project from Lesson 2.1, name the unit of observation, the target and the prediction time in one short phrase each, and check that the planned features fit.

1. Identify the unit of observation: one patient visit, meaning one check-in at the clinic on one particular day.
2. Identify the target: the number of minutes that visit's patient waits before being seen by a clinician.
3. Identify the prediction time: the moment of check-in, before any of that waiting has happened.
4. Check every planned feature against that moment: the day, the hour, the queue length and the number of clinicians on duty are all known at check-in, and the target itself is not yet known then.

Answer: Unit of observation: one patient visit. Target: waiting time in minutes. Prediction time: the moment of check-in. Naming these three things stops a common, costly error: using information from later in the visit, such as the consultation length, to predict something that must be guessed before the visit begins.

Figure 2.2 draws one clinic visit as a timeline. The dashed line marks the prediction time, the moment of check-in. Everything to the left of the line is known and may be used; everything to the right, including the target itself, is still in the future.

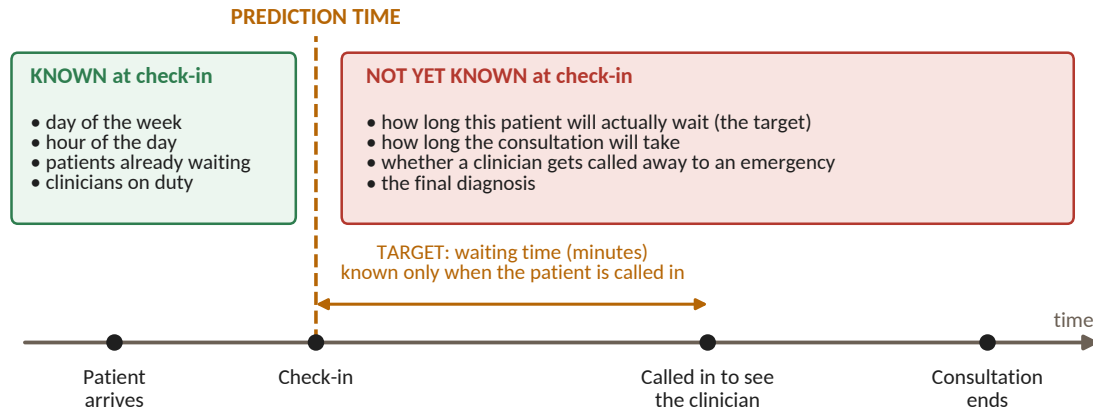


Figure 2.2 – One patient visit as a timeline. The prediction is made at check-in; the target (the waiting time) only becomes known when the patient is called in, and the consultation length even later.

WHAT TO OBSERVE

The target is the gap between two events on the timeline, check-in and being called in, so it cannot be known at check-in by definition. Everything in the red box is invisible to the model at prediction time, even though all of it will sit in the same spreadsheet row once the visit is over.

Table 2.2 names the same three things for two more projects. Notice how different the prediction times are: a parcel company needs its answer when the parcel leaves the depot, while a farmer needs a yield estimate weeks before the harvest, when there is still time to act on it.

Table 2.2 – Unit of observation, target and prediction time for three projects

Project	Unit of observation	Target	Prediction time
Clinic waiting time, Sulaymaniyah	one patient visit	minutes waited (a number)	the moment of check-in
Late deliveries, parcel company in Erbil	one parcel delivery	late or on time (a category)	the moment the parcel leaves the depot
Wheat yield, farm near Zakho	one field in one growing season	tonnes per hectare (a number)	the end of spring, weeks before harvest

COMMON MISTAKE

X Wrong: Mixing two different units of observation inside one dataset, for example letting some rows represent a single patient visit while other rows secretly represent a whole morning at the clinic.

✓ Right: Choose exactly one unit of observation for a dataset, and check that every row matches it. If a project truly needs both kinds of rows, build two separate datasets instead of blending them together.

2.3 Inspect and Split the Data Early

With the decision, the unit, the target and the prediction time all written down, it is finally time to look at real data. But there is a strict order here too: first *inspect*, then *split*, and only then let any learning happen. Inspecting means looking before doing anything else: check the column names, the units, the number of missing values, the balance between categories, and any value that cannot possibly be true. In the clinic data,

a recorded wait of -5 minutes is impossible, and a wait of 400 minutes, almost seven hours, is far more likely to be “40” with an extra zero than a real visit. Catching such rows now, against the paper records, takes a minute; teaching a model from them would quietly damage every later step.

Inspection is about *looking*, not *changing*. Do not clean, fill, rescale or otherwise learn anything from the complete dataset before splitting it. That rule sounds fussy until you meet the problem it prevents.

WORD TO KNOW

Data leakage — Information entering training that would not actually be available at the real moment of prediction. Leakage makes a model’s test score look better than the model will truly perform in real use.

Leakage is dangerous precisely because it feels like success: the model scores wonderfully on the test set, and then the deployed model fails, because in real use the leaked information does not exist yet. A study by Shachar Kaufman, Saharon Rosset and Claudia Perlich in 2011 showed that leakage had quietly spoiled the results of several well-known data-mining competitions. The defense is the prediction-time check from the previous lesson, applied to every candidate column; Figure 2.3 draws it for the clinic.

WORKED EXAMPLE 2.3 — Check Three Candidate Features for Leakage

The clinic team is choosing columns to use as features. Three candidates are suggested: (1) the number of patients already waiting when this patient checks in, (2) the length of this patient’s consultation with the clinician, and (3) the average waiting time for this same weekday and hour, calculated only from past visits. For each candidate, decide whether it is safe to use or whether it would cause leakage.

1. Check candidate 1 against the prediction time, the moment of check-in: the number of patients already waiting is fully known at that moment, so it is safe.
2. Check candidate 2: the consultation length only becomes known after the visit is completely finished, long after the prediction had to be made, so using it would be leakage.
3. Check candidate 3: an average built only from past visits, before today, is fully known at check-in, so it is safe.

Answer: Candidates 1 and 3 are safe features. Candidate 2 must be thrown out: it would let the model peek at the future, making test scores look wonderful while the real system could never produce that number in time.

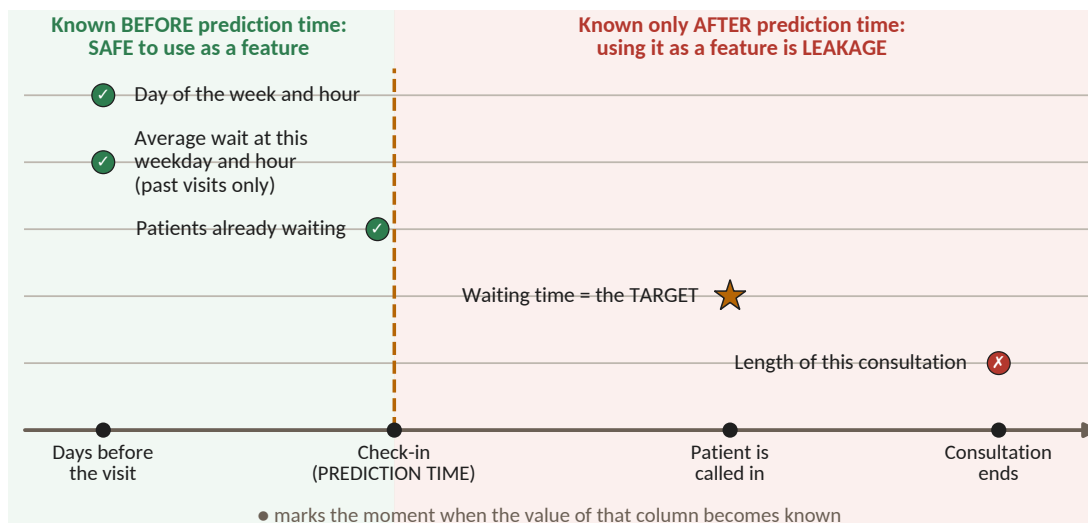


Figure 2.3 – When does each column’s value come into existence? Columns whose value is known before the prediction time are safe features; a column that is only filled in afterward leaks the future.

WHAT TO OBSERVE

The three green columns are all fixed by the time the patient reaches the front desk, so they can be features. The consultation length is not written down until the visit is over, long after the waiting time (the target, marked by the star) is known. A model trained with it would look brilliant on paper and be useless at the desk.

Table 2.3 extends the same check to five candidate columns. The last row is a special case that catches many beginners: a column that is really the target in disguise. The waiting time can be calculated directly from the time the patient was called in, so using that column as a feature leaks the whole answer.

Table 2.3 – Leakage check of five candidate features for the clinic project

Candidate feature	When its value becomes known	Safe at check-in?
Day of the week and check-in hour	known in advance	Yes
Number of patients already waiting	at check-in	Yes
Average wait for this weekday and hour, from past visits only	before today	Yes
Length of this visit’s consultation	after the visit ends	No — leakage
Time the patient was called in	when the wait ends	No — it is the target itself

Leakage can also enter through preprocessing. Some cleaning steps quietly *learn* something from the data, such as an average used to fill missing values or a typical range used for scaling; if that happens on the whole dataset before the split, facts about rows that should stay hidden have slipped into training. Chapter 4 shows how to do these steps safely; the rule for now is simple.

COMMON MISTAKE

X Wrong: Cleaning, scaling, or otherwise learning anything at all from the complete dataset before splitting it into training, validation and test groups.

✓ Right: Split the data first. Learn any cleaning rule, such as a typical value or a typical range, only from the training set, then apply that same rule to the validation and test sets afterward.

In Chapter 1 you split data into two groups. Real projects usually need three, because a team does not train one model and stop: it tries several candidates and settings, and it needs data to choose between them that is neither the training data nor the final test.

WORDS TO KNOW

Validation set — A group of examples, separate from both the training and test sets, used to compare candidate models and choose settings before the test set is ever opened. Settings may be adjusted based on validation results; the test set stays untouched until the very end.

Random split — A way of dividing a dataset by shuffling the rows first, so that every row has the same chance of landing in each set.

Time-based split — A way of splitting data by date instead of by chance, so that every training example happened before every validation example, and every validation example before every test example.

A common plan for the clinic's 1,200 visits is 60% for training, 20% for validation and 20% for testing, chosen at random after shuffling (Figure 2.4).

WORKED EXAMPLE 2.4 — Split 1,200 Clinic Visits into Training, Validation and Test Sets

The clinic has collected 1,200 past patient visits. The team decides on a 60% training, 20% validation and 20% test split. How many visits go into each group?

1. Find the training visits: $0.60 \times 1,200 = 720$.
2. Find the validation visits: $0.20 \times 1,200 = 240$.
3. Find the test visits: $0.20 \times 1,200 = 240$.
4. Check the total: $720 + 240 + 240 = 1,200$, which matches the original number of visits.

Answer: The training set gets 720 visits, the validation set gets 240 visits and the test set gets 240 visits. Every one of the 1,200 visits belongs to exactly one group.

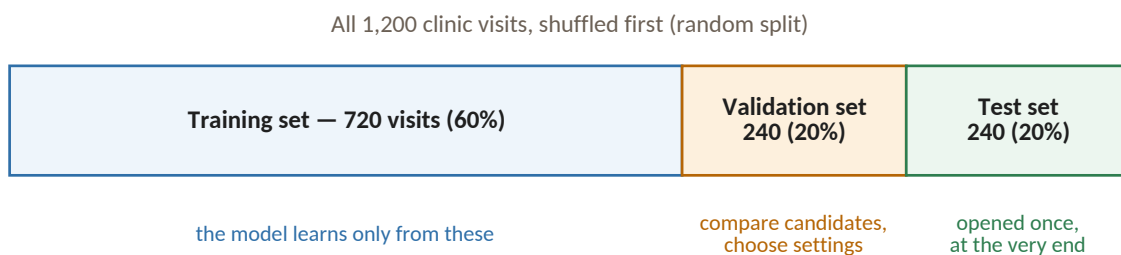


Figure 2.4 – A random 60/20/20 split of 1,200 clinic visits into training, validation and test sets. Each visit belongs to exactly one set.

WHAT TO OBSERVE

The three sets have three different jobs, not just three different sizes: the training set is the only one a model learns from, the validation set is opened many times to compare candidates, and the test set is opened once. Every visit is in exactly one set.

A random split like this works well when every row is roughly independent of every other row. But the clinic's visits happened in a real order, month after month, and waiting times change with the seasons, with staffing and with local events. When time matters this way, a random split can quietly leak the future into training, because visits from a later month land in the training set by chance while visits from an earlier month land in the test set.

WORKED EXAMPLE 2.5 — Choose a Time-Based Split Instead of a Random Split

The clinic's 1,200 visits were recorded over the last 12 months, spread evenly at 100 visits per month. Instead of splitting randomly, the team chooses a time-based split: the most recent 2 months become the test set, the 2 months before that become the validation set, and the remaining older months become the training set. How many visits are in each group?

1. Find how many months belong to training: $12 - 2 - 2 = 8$ months.
2. Find the training visits: $8 \times 100 = 800$ visits.
3. Find the validation visits: $2 \times 100 = 200$ visits.
4. Find the test visits: $2 \times 100 = 200$ visits.
5. Check the total: $800 + 200 + 200 = 1,200$, which matches the original number of visits.

Answer: The training set has 800 visits from the oldest 8 months, the validation set has 200 visits from the next 2 months, and the test set has 200 visits from the most recent 2 months. The model can never learn from the future by accident.

Figure 2.5 compares the two plans month by month. In the random plan every month is a mixture of the three sets; in the time-based plan the three sets sit one after another on the calendar.

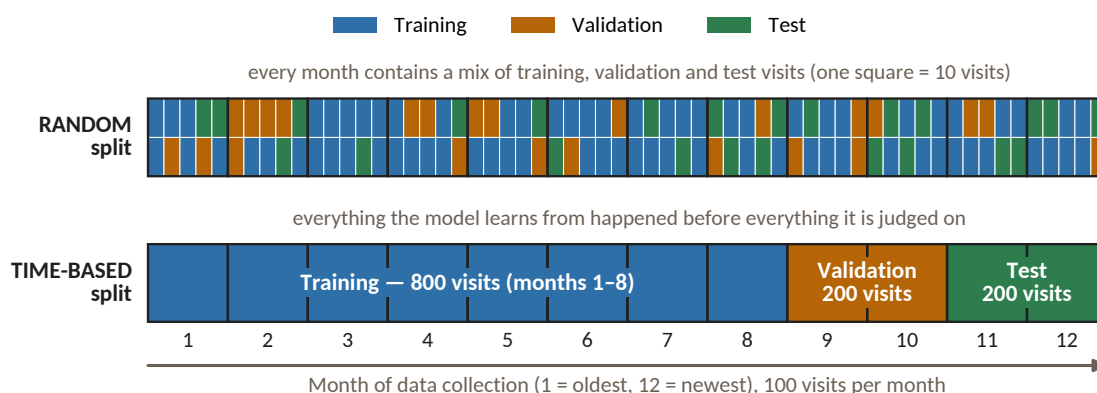


Figure 2.5 – Twelve months of clinic visits (100 per month) split two ways. Top: a random split mixes all three sets inside every month. Bottom: a time-based split trains on the oldest eight months, validates on the next two and tests on the newest two.

WHAT TO OBSERVE

In the top strip, a model trained on the blue squares of month 12 has already “seen” the future of the green test squares in month 3. In the bottom strip that is impossible: everything the model learns from happened before everything it is judged on, which is exactly the situation the deployed model will face.

The two plans give slightly different counts, 720/240/240 against 800/200/200, and neither is “wrong.” A random split is the usual default. A time-based split becomes the right choice whenever the order of events could matter to the target, which is true for demand forecasting, equipment failure, weather, prices and clinic queues. A simple rule of thumb: ask whether tomorrow’s example could be influenced by something that only became true today. If the answer is yes, split by time. Chapter 5 adds more splitting tools, including cross-validation.

2.4 Build a Baseline Before a Complex Model

Before a shop owner in Erbil buys an expensive new machine, she asks a simple question: how much better is it than what I already have? Machine learning needs the same question, and the honest way to answer it is to build the simplest possible predictor first.

WORD TO KNOW

Baseline — A deliberately simple method used as the minimum standard of comparison for a new model. A useful model must clearly beat the baseline on the chosen performance measure.

A baseline is simple on purpose. For a regression project, the usual baseline ignores every feature and always predicts the average of the training targets. For a classification project, it always predicts the most common class in the training data. A baseline takes minutes to build, and it answers an important question: is a complicated model actually worth its extra cost, or would a two-line rule have done almost as well? The mean baseline for regression is just an average, computed from the training rows only.

EQUATION: MEAN BASELINE PREDICTION

$$\hat{y}_{\text{base}} = \frac{1}{n} \sum_{i=1}^n y_i$$

In words: Add up the target values of all the training examples, divide by how many there are, and predict that single number for every new example, whatever its features say.

When it is used: As the first predictor in any regression project, before any real model is trained. Every later model is judged against it.

Symbol	Meaning	Unit or type
$\hat{y}_{\text{base}}$	the baseline’s prediction, the same for every new example	unit of the target
n	number of training examples	count
y_i	true target of training example i	unit of the target

Only training rows go into the average. Using validation or test rows here would be a small leakage of exactly the kind Lesson 2.3 warns about.

To judge any regression prediction, baseline or not, we need to measure how far it was from the truth. The simplest measure is the distance between the two numbers, ignoring the direction of the miss.

EQUATION: ABSOLUTE ERROR OF ONE PREDICTION

$$e_i = |y_i - \hat{y}_i|$$

In words: Subtract the prediction from the true value and drop the minus sign if there is one. A prediction 3 minutes too high and a prediction 3 minutes too low both have an absolute error of 3.

When it is used: For every single regression prediction, to say how far off it was in the target's own unit.

Symbol	Meaning	Unit or type
e_i	absolute error of prediction i	unit of the target
y_i	true value of example i	unit of the target
$\hat{y}_i$	predicted value of example i	unit of the target

The vertical bars mean “ignore the sign.” One error is not enough to judge a predictor; over m predictions we average them.

WORD TO KNOW

Mean absolute error (MAE) — The average of the absolute errors over a group of predictions, $\text{MAE} = (e_1 + e_2 + \cdots + e_m) \div m$. It says how far off a regression model typically is, in the same unit as the target. A smaller MAE is better; Chapter 6 studies it in detail.

WORKED EXAMPLE 2.6 — Build a Mean Baseline for Waiting Time and Find Its MAE

Before building any complex model, the clinic team builds a baseline from five training visits whose waiting times were 18, 25, 30, 12 and 20 minutes. Find the baseline's prediction, then find its MAE on four validation visits whose patients actually waited 24, 33, 10 and 17 minutes.

1. Add the five training waiting times: $18 + 25 + 30 + 12 + 20 = 105$.
2. Divide by the number of training visits, $n = 5$: $105 \div 5 = 21$. The baseline predicts $\hat{y}_{\text{base}} = 21$ minutes for every new visit, whatever the day, hour or crowd.
3. Find the absolute error on each validation visit: $|24 - 21| = 3$, $|33 - 21| = 12$, $|10 - 21| = 11$, $|17 - 21| = 4$.
4. Add the four errors: $3 + 12 + 11 + 4 = 30$.
5. Divide by the number of predictions, $m = 4$: $\text{MAE} = 30 \div 4 = 7.5$.

Answer: The baseline predicts 21 minutes for every visit, and its MAE on the four validation visits is 7.5 minutes. Telling every patient “about 21 minutes” is off by seven and a half minutes on average; this is the bar that every real model must clear on the same validation visits.

Table 2.4 collects the nine visits of the example. The baseline row is flat, because a baseline ignores the features; a real model must produce smaller errors than the last row, on average, to earn its place.

Table 2.4 – The tiny clinic dataset of Worked Example 2.6 (minutes)

Visit	1	2	3	4	5	6	7	8	9
Set	train	train	train	train	train	val.	val.	val.	val.
True waiting time	18	25	30	12	20	24	33	10	17
Baseline prediction	—	—	—	—	—	21	21	21	21
Absolute error	—	—	—	—	—	3	12	11	4

A baseline can also expose a hidden weakness that a single performance number might otherwise hide, especially for classification.

WORD TO KNOW

Majority class — The category that appears most often in the training data. The majority-class baseline predicts this category for every example, no matter what its features say.

EQUATION: MAJORITY-CLASS BASELINE ACCURACY

$$\text{Accuracy}_{\text{majority}} = \frac{n_{\text{majority}}}{n}$$

In words: The baseline is right exactly on the examples that belong to the most common class, so its accuracy is the share of examples in that class.

When it is used: As the first score in any classification project. It tells you how much of a model's accuracy comes from the data being lopsided rather than from the model being clever.

Symbol	Meaning	Unit or type
$\text{Accuracy}_{\text{majority}}$	accuracy of the majority-class baseline	fraction (or %)
n_{majority}	number of examples in the most common class	count
n	total number of examples	count

Compare with the accuracy equation of Chapter 1: this is the same $c \div n$, with c equal to the size of the largest class.

WORKED EXAMPLE 2.7 — Evaluate a Baseline That Always Predicts the Majority Class

Recall the online store's fraud project from Chapter 1: 5,000 past orders, of which 4,900 are genuine and 100 are fraudulent. A baseline predicts "genuine" for every order. Find its accuracy, then explain its hidden weakness.

1. Identify the majority class: "genuine," with $n_{\text{majority}} = 4,900$ of $n = 5,000$ orders.
2. Count the correct predictions: the baseline is right on every genuine order (4,900) and wrong on every fraudulent order (100).
3. Divide: $4,900 \div 5,000 = 0.98$.
4. Turn the result into a percentage: $0.98 \times 100 = 98$.

Answer: The baseline reaches 98% accuracy, which sounds excellent, but it catches exactly zero of the 100 fraudulent orders, the very orders the project exists to catch. A high accuracy number can hide a completely useless model; a real fraud model has done nothing until it beats 98% *and* catches real fraud. Chapter 6 introduces measures that look at each kind of mistake separately.

COMMON MISTAKE

X Wrong: Reporting a new model's score by itself, with no baseline next to it, which can make an ordinary result look like real progress.

✓ Right: Always report the baseline's score next to the new model's score, measured on the same data, so that anyone reading the result can judge how much the model really adds.

2.5 Train, Validate, Test Once

With a baseline in hand, the team can finally train real candidate models and compare them fairly. Training fits each candidate to the training set. The validation set is then used to compare the candidates and choose between them, changing one thing at a time so the team can understand what actually helped. If a team changes the model type, the features and a setting all at once, and the validation score improves, nobody can say afterward which change deserves the credit. Only after one final model has been chosen does the team touch the test set, and only once.

Table 2.5 shows the clinic team's comparison. Both candidates were trained on the same training visits and judged on the same validation visits as the baseline, so the three MAE values can be compared directly. The two model scores are illustrative example numbers; the baseline score is the one you computed by hand.

Table 2.5 – Model comparison on the validation set, and the single test-set check

Candidate	Validation MAE (min)	Test MAE (min)	Decision
Mean baseline	7.5	—	the bar every model must clear
Model 1	6.4	—	beats the baseline, loses to Model 2
Model 2	5.1	5.6 (opened once)	chosen; 5.6 minutes is the reported result

WORKED EXAMPLE 2.8 — Compare Two Candidate Models, Then Test Once

The clinic team trains two candidate models on the training set. On the validation set, Model 1 has an MAE of 6.4 minutes and Model 2 has an MAE of 5.1 minutes; the mean baseline scored 7.5 minutes on the same visits. The team selects the better model, then checks it, one time only, on the untouched test set, where it scores an MAE of 5.6 minutes. Which model does the team choose, and what final error do they report?

1. Compare the two validation errors: 6.4 minutes for Model 1 and 5.1 minutes for Model 2.
2. A smaller MAE means better predictions, and $5.1 < 6.4$, so Model 2 performs better on the validation set.
3. Check that the winner beats the baseline: $7.5 - 5.1 = 2.4$ minutes better, so the extra complexity is earning its place.
4. Select Model 2 based only on this validation comparison, and open the test set for the first and only time: MAE = 5.6 minutes.
5. Report 5.6 minutes as the final number, even though it is $5.6 - 5.1 = 0.5$ minutes worse than the validation score.

Answer: The team chooses Model 2 and reports a test MAE of 5.6 minutes. A small rise from validation to test is normal and expected, because the validation set was used to pick the winner; the test score is trusted precisely because the team looked at it exactly once.

Figure 2.6 shows the same numbers as a chart. Reporting them side by side, baseline and models on the same validation visits, is the honest way to show what the modeling work achieved.

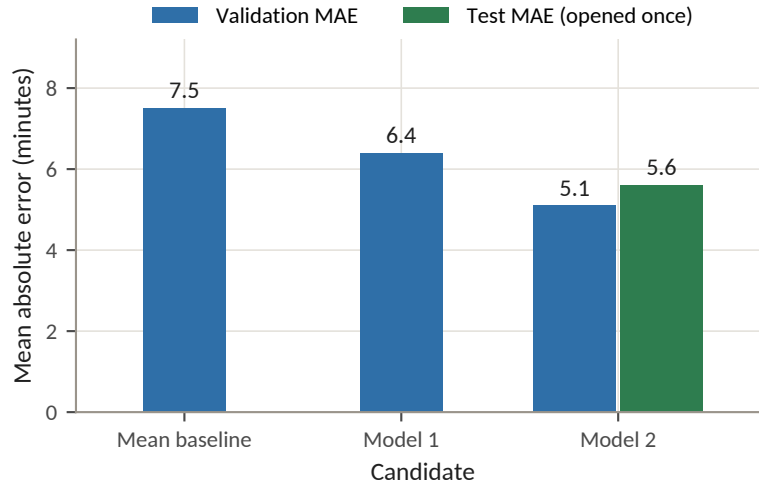


Figure 2.6 – Validation MAE of the mean baseline and two candidate models, and the single test-set MAE of the chosen model. Illustrative example numbers.

WHAT TO OBSERVE

Each bar is a little shorter than the one before it, and the green test bar for Model 2 is slightly taller than its blue validation bar. That small gap is the price of having used the validation set to choose the winner. What matters is that 5.6 minutes is still well below the 7.5-minute baseline.

Why be so strict about the test set? Figure 2.7 shows the rule as a picture: the test set is sealed at the moment of splitting and the seal is broken exactly once, at the very end.

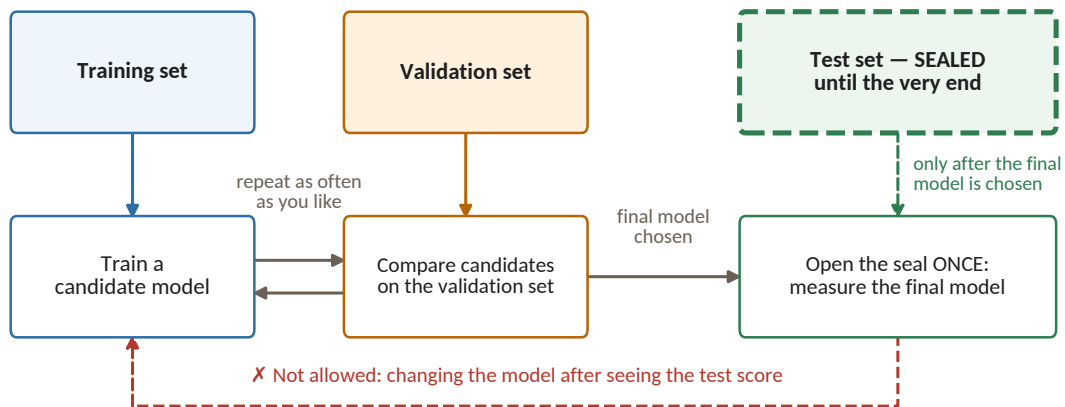


Figure 2.7 – The sealed test set. Training and validation may be repeated as often as needed; the test set is opened once, after the final model is chosen, and the model is never changed afterward.

WHAT TO OBSERVE

The two gray arrows between the first two boxes form a loop that may run many times. There is no loop through the test set: one dashed green arrow goes in, one number comes out, and the red dashed path back to training is forbidden.

IMPORTANT

If you check the test result repeatedly, and keep changing the model based on what you see, the test set quietly turns into another validation set, and its score stops being an honest estimate of real-world performance. The opposite mistake is just as common: stopping when the validation score looks good and never checking a genuine, untouched test set at all. Always finish with exactly one test-set evaluation.

2.6 The Workflow as a Checklist

The eight steps of Figure 2.1 are easiest to use as a checklist, with one question to ask at each step. Table 2.6 collects them. Many organizations follow a very similar published sequence called CRISP-DM, written by Pete Chapman and colleagues in 2000, which also ends with deployment and also loops back to the beginning. The two final steps deserve their own names.

WORDS TO KNOW

Deployment — Putting a trained model to work on real, new examples, for example connecting the waiting-time model to the clinic's check-in screen.

Monitoring — Regularly checking a deployed model's predictions against what really happened, so that a model whose accuracy is slipping is noticed and retrained.

Table 2.6 – The workflow as a checklist: one question per step

Step	Ask yourself	Trap it prevents
1. Define the decision	Who needs which answer, at what moment?	a model nobody uses
2. Collect the data	Does every row match one unit of observation?	mixed units in one table
3. Inspect the data	Any impossible values, missing values or lopsided classes?	learning from broken data
4. Split	Random or time-based? Split before any cleaning rule is learned?	leakage through preprocessing
5. Build a baseline	What does the simplest rule score?	mistaking an ordinary result for progress
6. Train and validate	One change at a time, judged on the validation set?	not knowing what helped
7. Test once	Is the test set still untouched?	a test set turned into a validation set
8. Deploy and monitor	Is the model still accurate months later?	a model that quietly goes out of date

Monitoring is the step most often forgotten, because the project feels finished at deployment. But a clinic is a living place: new clinicians join, patient flow shifts, a vaccination campaign changes the queue. A model trained on last year's pattern slowly goes out of date if nobody keeps watching it; Chapter 24 returns to this problem, called drift. The rest of Part One fills in steps 2 to 7: data in Chapter 3 and Chapter 4, splitting and cross-validation in Chapter 5, performance measures in Chapter 6, and overfitting in Chapter 7.

COMMON MISTAKES AND MISCONCEPTIONS

- Starting a project with an algorithm's name instead of a plain decision sentence.
- Mixing two different units of observation inside a single dataset.
- Using a feature that would not actually be known yet at the true moment of prediction, or learning a cleaning rule from the complete dataset before splitting it.
- Using a random split when the data's time order could matter to the target.
- Reporting a model's score with no baseline next to it for comparison.
- Checking the test set repeatedly and adjusting the model afterward based on what it shows.

Practical Activities

ACTIVITY (PAPER) — Plan a Project of Your Own, Step by Step

1. Choose a prediction that would help someone you know: a shopkeeper, a teacher, a farmer, a bus company, a football coach. Write the decision sentence with all three blanks filled in, then write the Task, Experience and Performance (T, E, P) in three plain sentences, as in Chapter 1.
2. Name the unit of observation, the target and the prediction time. Then list six candidate features and mark each one “safe” or “leaks,” with one reason each.
3. Decide whether a random split or a time-based split fits your project, and say why. If you had 2,000 rows, write down how many would go into training, validation and test. Finally, describe the baseline you would build first.

You need: Paper and pen

ACTIVITY (PYTHON) — Splits and Baselines with scikit-learn

This program repeats Worked Examples 2.4, 2.6 and 2.7 with scikit-learn, the free Python library described by Fabian Pedregosa and colleagues in 2011. `train_test_split` only makes two groups at a time, so the three-way split is done in two steps; `DummyRegressor` and `DummyClassifier` are scikit-learn's ready-made baselines. Run the two parts in order.

Python · NumPy · scikit-learn (part 1: the three-way split)

```
import numpy as np
from sklearn.model_selection import train_test_split

# 1. Split 1,200 visit numbers 60/20/20 with a fixed seed (Worked Example 2.4)
visits = np.arange(1, 1201)
train, rest = train_test_split(visits, test_size=0.40, random_state=42)
val, test = train_test_split(rest, test_size=0.50, random_state=42)
print("train / validation / test:", len(train), len(val), len(test))
print("first five training visits:", train[:5])
```

Output

```
train / validation / test: 720 240 240
first five training visits: [862 579 449 446 260]
```

The first split pulls out 40% of the visits as a temporary group, leaving 60% for training; the second split cuts that group in half, giving 20% each for validation and test, exactly the 720/240/240 of Worked

Example 2.4. `random_state=42` makes the shuffle repeatable, so a classmate gets the same five training visits.

Python • scikit-learn (part 2: the two baselines)

```
from sklearn.dummy import DummyRegressor, DummyClassifier
from sklearn.metrics import mean_absolute_error, accuracy_score

# 2. Mean baseline for waiting time (Worked Example 2.6)
X_train = np.array([[1], [2], [3], [4], [5]]) # a baseline ignores the features
y_train = np.array([18, 25, 30, 12, 20])      # minutes waited
X_val = np.array([[6], [7], [8], [9]])
y_val = np.array([24, 33, 10, 17])
base = DummyRegressor(strategy="mean").fit(X_train, y_train)
pred = base.predict(X_val)
print("baseline predicts:", pred)
print("absolute errors:", np.abs(y_val - pred))
print("baseline MAE on validation:", mean_absolute_error(y_val, pred))

# 3. Majority-class baseline for the 5,000 orders (Worked Example 2.7)
y_orders = np.array(["genuine"] * 4900 + ["fraud"] * 100)
X_orders = np.zeros((5000, 1))
majority = DummyClassifier(strategy="most_frequent").fit(X_orders, y_orders)
print("majority baseline accuracy:", accuracy_score(y_orders, majority.predict(X_orders)))
```

Output

```
baseline predicts: [21. 21. 21. 21.]
absolute errors: [ 3. 12. 11.  4.]
baseline MAE on validation: 7.5
majority baseline accuracy: 0.98
```

`DummyRegressor` learns nothing but the training mean, 21, and `mean_absolute_error` returns the 7.5 minutes you found by hand. `DummyClassifier` always answers “genuine” and scores the same 98% as Worked Example 2.7. Try changing `random_state` in part 1 and watch the visit numbers change while the group sizes stay the same.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Predicting Patient Waiting Time at a Clinic in Sulaymaniyah

The director of the clinic from the introduction decides that “how long will I wait?” is exactly the kind of decision machine learning can support. She asks her data team to follow this chapter’s workflow and to tick every line of the checklist in Table 2.6 before anything goes near the check-in screen. The team starts with the decision sentence of Worked Example 2.1. The unit of observation is one patient visit, and the target is the waiting time in minutes. While inspecting a year of records they find a handful of impossible values, such as a wait of -5 minutes, and check them against the paper log before going further. They exclude any column that would not truly be known at check-in, above all the consultation length, because it would leak the future into a prediction made before the future has happened.

Because clinic life changes with the seasons, flu outbreaks and staffing schedules, the team chooses a time-based split, as in Worked Example 2.5: training on the oldest eight months, validating on the next two and testing on the most recent two. Before training anything, they build the mean baseline of Lesson 2.4, which sets the bar at an MAE of 7.5 minutes. They then train two candidate models, compare

them on the validation months, choose Model 2, and open the test months exactly once, reporting the 5.6-minute result of Table 2.5.

The finished screen shows a friendly range, such as “about 15 to 20 minutes,” never a falsely precise “17 minutes,” with a note that urgent cases are always seen first. Three months after launch, the monitoring step pays for itself: the estimates are noticeably worse on the first Saturday of each month, when a nearby vaccination campaign brings a wave of walk-ins that the training data barely captured. The team adds a campaign-day flag and plans to retrain once enough such days have been recorded. That is the spirit of a safe workflow: a habit of checking, learning and improving.

RESPONSIBLE USE

A predicted waiting time is a helpful estimate, not a promise and not a medical judgment. Before deploying any model about people, add two lines to the decision sentence: what happens when the prediction is wrong, and who may overrule it. A waiting-time estimate must never decide who is seen first; a nurse’s judgment about urgency always comes before any prediction (Chapter 24).

CHAPTER SUMMARY

- A project begins with a plain decision sentence, “use ___ to predict ___, so that ___,” not with the name of an algorithm.
- Every row is one unit of observation, kept the same across the whole dataset. Only information that exists at the prediction time may be a feature.
- Data leakage is information entering training that is not available at prediction time; it inflates test scores. Inspect, then split, before any cleaning rule is learned.
- The training set fits the model, the validation set compares candidates, and the test set is opened once, at the end. A random split suits independent rows; a time-based split protects projects where order matters.
- A baseline is a deliberately simple predictor: the training mean for regression, the majority class for classification. Every real model must beat it on the same data.
- The MAE averages the absolute errors $|y - \hat{y}|$ over a set; a majority-class baseline can score a very high accuracy on lopsided data while catching none of the rare cases that matter.
- Compare candidates one change at a time on the validation set; checking the test set repeatedly turns it into a validation set. After deployment, monitor the model and loop back when the world changes.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher’s Guide.

Problem 1 [Basic] In your own words, explain why a project should start with a decision sentence instead of the name of an algorithm.

Problem 2 [Basic] Define *unit of observation*, and give an example that is not from this chapter. Then name a second unit that could slip into the same dataset by mistake.

Problem 3 [Basic] A dataset has 800 rows. The team plans a 60/20/20 random split. How many rows go into the training, validation and test sets? Show every step.

Problem 4 [Basic] Four training visits waited 14, 22, 18 and 26 minutes. What does the mean baseline predict for every new visit? Show your arithmetic, and say why such a simple rule is still useful.

Problem 5 [Standard] Split 2,000 rows into 70% training, 15% validation and 15% test, showing every step and checking that the three counts add up to 2,000.

Problem 6 [Standard] Explain why scaling an entire dataset before splitting it counts as data leakage, even though the target column is never touched.

Problem 7 [Standard] A regression baseline predicts 50 for every example. The true values for three new examples are 44, 52 and 61. Find each absolute error, then find the MAE to two decimal places.

Problem 8 [Standard] A dataset has 9,000 negative examples and 1,000 positive examples. State the majority-class baseline, find its accuracy, and say how many of the positive examples it catches.

Problem 9 [Standard] A bakery in Duhok predicts tomorrow's bread demand each evening at closing time. Say whether each candidate feature is safe or leaks: (a) today's sales, (b) tonight's weather forecast, (c) tomorrow's actual number of customers, (d) whether tomorrow is a public holiday.

Problem 10 [Challenge] A ride-hailing company wants to predict how long a driver will take to reach a passenger. (a) Write the decision sentence. (b) Name the unit of observation, the target and the prediction time. (c) Would a random or a time-based split fit better, and why?

Problem 11 [Challenge] On the validation set, Model A scores an MAE of 4.9 minutes and Model B scores 5.3 minutes. A team member then opens the test set for both: Model A scores 6.0 and Model B 5.5. (a) Which model should have been chosen, and on what evidence? (b) Why does choosing Model B "because it did better on the test set" break the workflow? (c) What number should honestly be reported, and what has been lost by opening the test set twice?

Problem 12 [Challenge] Six months after deployment, the clinic model's estimates are usually too short. Which of the eight workflow steps should the team repeat, in what order, and why might step 1 need to change as well as step 6?

CHAPTER THREE

Data Collection and Dataset Structure

Samples, features, targets, and the first look at any dataset

IN THIS CHAPTER YOU WILL LEARN TO

- Explain where the rows of a dataset come from, and judge whether a sample represents the population a model will serve.
- Name the parts of a dataset table: samples, features, the target, the feature matrix $\mathbf{X}$ and the target vector $\mathbf{y}$.
- Classify every column as numerical (continuous or discrete), categorical (nominal, ordinal or binary), text or date.
- Calculate the mean, median, mode, range, variance, standard deviation, quartiles and interquartile range of a column by hand.
- Read a histogram, a box plot, a scatter plot, a correlation heat map and a pair plot, and calculate the Pearson correlation coefficient of two columns by hand.
- Run a first-look checklist on any new dataset with pandas.

Introduction

A friend in Sulaymaniyah is moving and asks you: “What is a fair rent for a two-room apartment near the center?” You do not know, but you know how to find out. You ask neighbors, you read listings, and you write every rent you hear about in a notebook, next to the size of the apartment and its street. After a week the notebook holds forty lines, and only then can you say something useful: “Most two-room apartments near the center rent for between 400 and 550 thousand dinars, and one of the numbers I was given looks like a mistake.”

Machine learning starts the same way. Before any model is trained, someone has to collect the examples, arrange them in a table, and look at them carefully. This chapter is about that first step: where data comes from, how a table is organized into samples, features and a target, what kinds of values a column can hold, and how to describe and plot one column, then two, then many. Everything is done first by hand on a small table; the Python activity then repeats the same calculations on forty listings with pandas.

3.1 Where Data Comes From

A small rental website lists apartments across Erbil, Duhok and Sulaymaniyah, and its founders want a model that suggests a fair rent for each new listing. The first question is not “which model?” but “where will the rows come from?” Figure 3.1 shows the five most common answers.

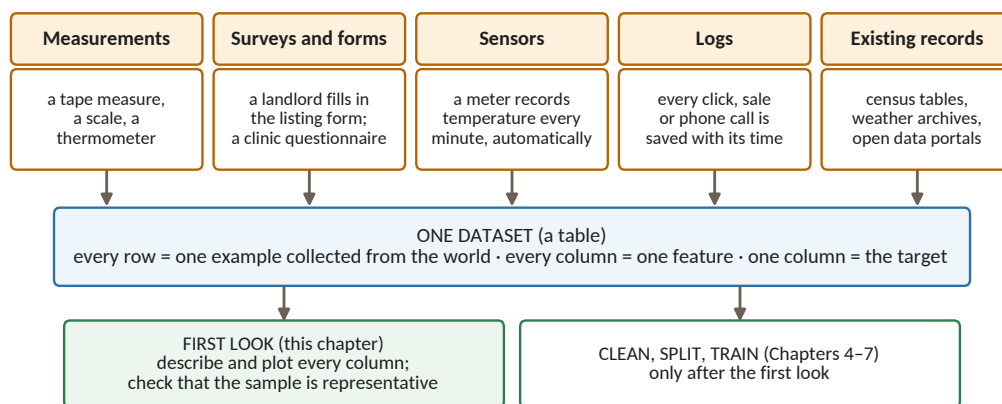


Figure 3.1 – Where the rows of a dataset come from. Whatever the source, the result is one table, and the first job is to look at it before any cleaning or training begins.

Some data is *measured* on purpose, one example at a time: an agent visits an apartment with a tape measure. Some is *reported* by people on a form: the landlord types the size and the rent into the website. Some is recorded automatically by *sensors*, such as a meter in a Duhok classroom that stores the temperature every minute; some is a *log* of every click or phone call, saved with its time; and a great deal already exists in *records* collected by someone else. Every source has a typical weakness — people skip questions and round their answers, sensors fail and record zeros, logs only record what happened on the platform, old records use other units — so knowing the source tells you what to check first. Whatever the source, the collected rows are used to say something about a much larger group, and statistics has two words for this.

WORDS TO KNOW

Population — The complete group a model is meant to work for — every apartment for rent in the three cities, including the ones nobody has listed yet.

Sample — The rows actually collected, always only part of the population. (Machine learning also uses *sample* for one single row; this book says “a sample of 40 listings” for the group and “one sample” or “one example” for a row.)

Representative sample — A sample whose mix of examples looks like the population’s mix, so that what is true of the sample is roughly true of the population.

Sampling bias — A systematic difference between sample and population caused by the way the data was collected, such as collecting listings from only one city.

A model can only learn what its sample shows it. If the website launched in Erbil first, a model trained on its early data will quietly apply Erbil prices to an apartment in Duhok, and more rows do not fix this: ten thousand Erbil listings still say nothing about Duhok. The cure is to collect from the population the model must serve, and to check the sample’s mix against it. Suppose the website’s first export holds 200 listings: 150 from Erbil, 30 from Sulaymaniyah and 20 from Duhok, that is $150 \div 200 = 75\%$, $30 \div 200 = 15\%$ and $20 \div 200 = 10\%$, while the founders estimate (illustratively) that the region’s rental market is 50% Erbil, 30% Sulaymaniyah and 20% Duhok. The sample over-represents Erbil by 25 percentage points and under-represents the other two cities by half, so a model trained on it would mostly learn Erbil’s prices: the founders should collect more listings from the other two cities, or say plainly that the model is only trustworthy for Erbil.

COMMON MISTAKE

X Wrong: Believing that a very large dataset is automatically a good one — “we have 50,000 rows, so the model will be fine.”

✓ **Right:** Ask how the rows were collected and whom they leave out. A large sample with a systematic gap (one city, one season, only daytime readings, only customers who complained) is still biased; size does not cure bias.

Two more collection details matter: the *unit* of every column should be fixed by the form and printed in the column name (a size typed as 850 might be square feet), and every row should carry the *date* it was collected, because a rent from 2019 and a rent from this month are not the same fact.

RESPONSIBLE USE

Data about people is not like data about pumps. Collect only the columns the project needs, tell people what will be done with their information, and store it safely. A free-text field such as a listing's description can contain a landlord's phone number or address; remove such details before the data is shared or used for training. Chapter 24 returns to this in depth.

3.2 The Dataset as a Table: Samples, Features and Targets

Once collected, the data becomes a table. Table 3.1 shows the first eight listings from the website; every calculation in this chapter uses this table or a small part of it.

Table 3.1 – Eight apartment listings — the dataset used throughout this chapter (illustrative numbers)

ID	Size (m ²)	Rooms	Age (years)	Distance to center (km)	City	Rent (thousand IQD)
1	85	2	5	2.0	Erbil	550
2	60	1	12	4.5	Duhok	300
3	105	3	3	1.5	Erbil	700
4	75	2	8	3.0	Sulaymaniyah	400
5	200	4	1	6.0	Erbil	1150
6	95	3	15	2.5	Sulaymaniyah	450
7	75	2	20	5.0	Duhok	330
8	105	2	6	3.5	Sulaymaniyah	520

Each row is one apartment: one *sample*, also called an example, an observation, a record or an instance. Each column is one *feature*, also called a variable or an attribute. The last column, rent, is the value to be predicted, so it is the *target* ("label" when the answer is a category, as in Chapter 1). The ID column identifies the row but says nothing about the apartment, so it is not a feature. Figure 3.2 names every part: $n = 8$ samples and $p = 5$ features, so $\mathbf{X}$ has 8 rows and 5 columns and $\mathbf{y}$ has 8 values.

WORDS TO KNOW

Feature matrix $\mathbf{X}$ — All the feature values arranged as a grid with n rows (one per sample) and p columns (one per feature); row i holds the feature values of sample i .

Target vector $\mathbf{y}$ — The n target values, one per sample, in the same row order as $\mathbf{X}$. Together, $\mathbf{X}$ and $\mathbf{y}$ are what a supervised learning algorithm receives.

Identifier — A column, such as an ID number, that names a row but carries no information about it; kept for bookkeeping, dropped before training.

identifier (not a feature)

FEATURES X (n rows $\times$ p columns)

TARGET y

ID	Size (m ²)	Rooms	Age (yr)	Dist. (km)	City	Rent (thousand IQD)
1	85	2	5	2.0	Erbil	550
2	60	1	12	4.5	Duhok	300
3	105	3	3	1.5	Erbil	700
4	75	2	8	3.0	Sulaymaniyah	400
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

one row = one sample (example) →

one column = one feature (Table 3.1 has $n = 8$ rows and $p = 5$ features)

Figure 3.2 – The anatomy of the listings table. Blue columns are features (the matrix X), the green column is the target (y), and the gray ID column is an identifier that is not used for learning.

One feature, City, holds words rather than numbers, and a model cannot multiply “Erbil” by anything. Naming the type of every column is therefore the first job on any new dataset: which summary numbers make sense, which plot to draw and, in Chapter 4, which cleaning and encoding steps to use all depend on it. Figure 3.3 shows the family tree of feature types.

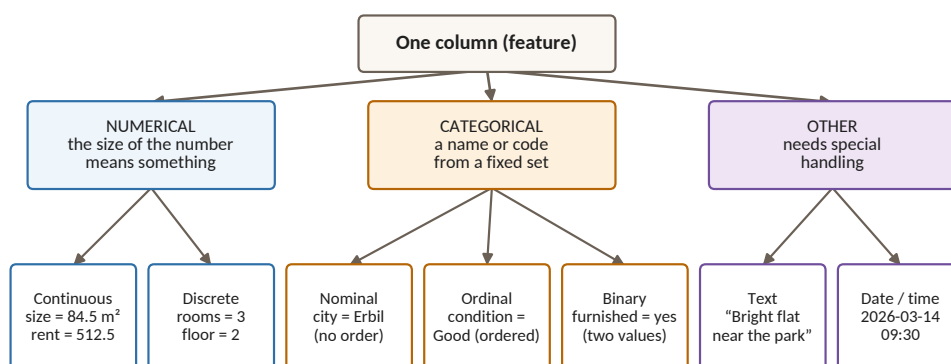


Figure 3.3 – The family tree of feature types. For a numerical column, ask whether it is measured or counted; for a categorical column, ask whether its values have a natural order.

WORDS TO KNOW

Numerical feature — A number whose size means something: 100 m² really is twice 50 m². A *continuous* numerical feature can take any value on a scale (size, rent, temperature); a *discrete* one counts whole things (rooms, children, visits).

Categorical feature — A value chosen from a fixed set of names or codes. A *nominal* feature has no natural order (city, blood type). An *ordinal* feature has an order but no fixed distance between steps (condition: needs repair, fair, good, excellent — nobody can say that the step from fair to good equals the step from good to excellent). A *binary* feature has exactly two values (furnished: yes or no).

Text feature — Free sentences typed by a person, such as a listing’s description. Text must be turned into numbers before a model can use it.

Date/time feature — A date or a moment, such as the day a listing was posted; usually turned into numerical features (month, day of the week, days since an event) before training.

Table 3.2 classifies every column of one full listing in the form a project keeps in its data dictionary, and Worked Example 3.1 shows the reasoning behind each line.

Table 3.2 – Feature-type classification of the listing columns

Column (example)	Type	Role
ID (3)	Identifier	Bookkeeping only, never a feature
Size (105 m ²), Distance (1.5 km)	Numerical, continuous	Features, with fixed units
Rooms (3), Age (3 years)	Numerical, discrete	Features (counts)
City (Erbil)	Categorical, nominal	Feature, no order between cities
Condition (Good)	Categorical, ordinal	Feature, ordered set of four values
Furnished (yes)	Binary	Feature
Listed (2026-03-14)	Date	Feature after conversion (month, days on the site)
Description (“Bright ...”)	Text	Feature only after text processing; check for private details
Rent (700 thousand IQD)	Numerical, continuous	Target

WORKED EXAMPLE 3.1 – Classify the Feature Types of One Listing

One full row of the website’s table reads: ID = 3; Size = 105 m²; Rooms = 3; City = “Erbil”; Condition = “Good” (from needs repair, fair, good, excellent); Furnished = “yes”; Listed = 2026-03-14; Description = “Bright apartment near the park, recently painted”; Rent = 700 thousand IQD. Classify every column.

1. ID = 3 is written with a digit, but listing 6 is not “twice” listing 3 in any sense: an **identifier**, not a feature.
2. Size = 105 m² is a number whose size means something, and 105.5 would also be a valid size: **numerical, continuous**. Rooms = 3 counts whole rooms: **numerical, discrete**.
3. City = “Erbil” is one name from a small set with no natural order: **categorical, nominal**. Condition = “Good” comes from an ordered set, needs repair < fair < good < excellent: **categorical, ordinal**. Furnished = “yes” has exactly two possible values: **binary**.
4. Listed = 2026-03-14 is a **date**; Description is a sentence typed by a person: **text**.
5. Rent = 700 is numerical and continuous, and it is the column to be predicted: the **target**.

Answer: One listing contains an identifier, two numerical features (one continuous, one discrete), three categorical features (nominal, ordinal and binary), a date, a text field and a numerical target — seven kinds of value in nine columns, which is why this classification step comes first.

COMMON MISTAKE

X Wrong: Treating every column that contains digits as a numerical feature: an ID number, a postal code, a phone number or a category written as “1, 2, 3”.

✓ Right: Ask whether the size of the number means anything. 100 m² is twice 50 m², so size is numerical. Postal code 44002 is not “more” than 44001, so it is a nominal category written with digits.

3.3 Describing One Column: Mean, Median, Mode, Range, Variance and Standard Deviation

A shopkeeper in Duhok who is asked “how big are the apartments on your street?” will not read out forty numbers. She will say “about 90 square meters, though a few are much bigger.” That sentence contains a *typical value* and a *spread*, and this lesson gives both exact definitions using the eight sizes in Table 3.1.

WORDS TO KNOW

Mean — The everyday average: the sum of the values divided by how many there are. Every value pulls the mean toward itself, so one very large value can move it a lot.

Median — The middle value after sorting. Half of the values are below it and half above it, so a single huge value hardly moves it.

Mode — The value that appears most often — the only “typical value” that also makes sense for a categorical column (the most common city).

Range — The largest value minus the smallest: a quick but fragile measure of spread, because it depends on only two values.

The mean is written with a bar over the variable’s name; it is the first equation of the chapter and appears inside almost every other one. The median needs no arithmetic, only sorting, but its rule depends on whether the number of values is odd or even.

EQUATION: MEAN AND MEDIAN OF A COLUMN

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \text{Median} = \begin{cases} x_{(m+1)} & \text{if } n = 2m + 1 \text{ is odd} \\ \frac{x_{(m)} + x_{(m+1)}}{2} & \text{if } n = 2m \text{ is even} \end{cases}$$

In words: Mean: add up every value, then divide by the number of values. Median: sort the values; with an odd number take the middle one, with an even number average the two middle ones.

When it is used: The mean gives one typical value and is the first step of the variance, the standard deviation and the correlation coefficient. The median gives a typical value that is not pulled around by a few extreme values (rents, incomes, waiting times); Chapter 4 uses it to fill missing values.

Symbol	Meaning	Unit or type
$\bar{x}$	the mean (read “x bar”)	same unit as x
x_i	the i -th value	unit of the column
$x_{(k)}$	the k -th smallest value after sorting	unit of the column
n	number of values in the column	count
m	half of n , rounded down	count

The centroid of Chapter 1 is the mean of each feature. With $n = 8$ values, $m = 4$, so the median is the average of the 4th and 5th sorted values.

WORKED EXAMPLE 3.2 — Mean and Median of the Eight Apartment Sizes

The eight sizes in Table 3.1 are 85, 60, 105, 75, 200, 95, 75 and 105 m². Find the mean and the median.

1. Add the eight values: $85 + 60 + 105 + 75 + 200 + 95 + 75 + 105 = 800$.
2. Divide by $n = 8$: $\bar{x} = 800 \div 8 = 100$ m².
3. Sort the values: 60, 75, 75, 85, 95, 105, 105, 200. Since $n = 8$ is even, $m = 4$: the median is the average of the 4th and 5th sorted values, 85 and 95.
4. Median = $(85 + 95) \div 2 = 180 \div 2 = 90$ m².

Answer: The mean size is 100 m² and the median size is 90 m². The mean is higher because the one very large apartment (200 m²) pulls it up; the median only counts that apartment as “one value above the middle.” When the two disagree like this, report both.

The mode and the range need no formula. The Rooms column reads 2, 1, 3, 2, 4, 3, 2, 2, so 2 rooms appears four times and is the mode. The sizes run from 60 to 200 m², a range of $200 - 60 = 140$ m² — and without the single 200 m² apartment the range would be only $105 - 60 = 45$ m². That fragility is why the standard measure

of spread uses every value: take each value's distance from the mean, square it so that distances below and above the mean both count as positive, average the squares, and take the square root to return to the original unit.

WORDS TO KNOW

Variance — The average squared distance of the values from their mean (dividing by $n - 1$ for a sample). Its unit is the column's unit squared, which is why it is rarely reported on its own.

Standard deviation — The square root of the variance: a typical distance of a value from the mean, in the column's own unit. Small means the values sit close to the mean; large means they are spread out.

EQUATION: SAMPLE VARIANCE AND STANDARD DEVIATION

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}$$

In words: Subtract the mean from every value, square each difference, add the squares, divide by one less than the number of values, and take the square root.

When it is used: To report how spread out a numerical column is, in its own unit; also to scale features in Chapter 4 and to report the spread of a model's scores in Chapter 5.

Symbol	Meaning	Unit or type
s^2	the sample variance	unit of x , squared
s	the sample standard deviation	same unit as x
$x_i, \bar{x}, n$	the values, their mean and their number, as above	—

Why $n - 1$ and not n ? The mean was computed from the same values, so the deviations from it are slightly smaller than the deviations from the true population mean; dividing by $n - 1$ corrects for this. pandas uses $n - 1$; NumPy's `np.std` divides by n unless you pass `ddof=1`.

To keep the arithmetic short, the next example uses five more listings from the website (listings 9 to 13 of its dataset), whose sizes are 50, 70, 110, 120 and 150 m² and whose rents are 440, 400, 520, 540 and 600 thousand IQD. The same five listings return in the correlation example later in the chapter.

WORKED EXAMPLE 3.3 — Standard Deviation of Five Apartment Sizes

Find the sample standard deviation of the five sizes 50, 70, 110, 120 and 150 m².

1. Find the mean: $(50 + 70 + 110 + 120 + 150) \div 5 = 500 \div 5 = 100$ m².
2. Subtract the mean from each value: $-50, -30, 10, 20, 50$. Square each deviation: 2500, 900, 100, 400, 2500.
3. Add the squares: $2500 + 900 + 100 + 400 + 2500 = 6400$.
4. Divide by $n - 1 = 4$: $s^2 = 6400 \div 4 = 1600$.
5. Take the square root: $s = \sqrt{1600} = 40$ m².

Answer: The standard deviation is 40 m²: a typical apartment in this group sits about 40 m² from the mean of 100 m². The same steps for the five rents give a mean of 500 and a standard deviation of 80 thousand IQD (deviations $-60, -100, 20, 40, 100$; sum of squares 25,600; variance 6,400).

Table 3.3 collects the summary numbers for size and rent over the eight listings; the standard deviations were computed by pandas in the Python activity, using the same steps as the example. For both columns the mean sits above the median and the standard deviation is large compared with the typical value. Both facts point at the same row: apartment 5, more than twice as large and twice as expensive as the rest — an *outlier*, a word the next lesson defines properly.

Table 3.3 – Descriptive statistics of size and rent for the eight listings of Table 3.1

Column	Mean	Median	Min	Max	Range	Std. dev. <i>s</i>
Size (m ²)	100	90	60	200	140	43.3
Rent (thousand IQD)	550	485	300	1150	850	274.3

3.4 Seeing One Column: Histograms and Box Plots

Numbers summarize; pictures reveal. A nurse in Sulaymaniyah who plots the ages of last month's patients sees at once whether they were mostly children, mostly elderly, or two separate groups — something no mean can show. The two standard pictures of one column are the histogram and the box plot. Eight listings are too few for either, so this lesson uses the website's sample of 40 listings, stored in the file `datasets/ch03_apartments.csv` (its first eight rows are Table 3.1 and rows 9 to 13 are the five listings of Worked Example 3.3).

WORD TO KNOW

Histogram — A plot that cuts the range of a numerical column into equal intervals called *bins* and draws one bar per bin, with height equal to the number of values in it. Too few bins hide the shape and too many make it ragged; 6 to 15 bins is a sensible start for 30 to 300 values. The name was introduced by the statistician Karl Pearson in 1895.

Figure 3.4 is the histogram of the 40 sizes, with the mean and the median drawn as vertical lines.

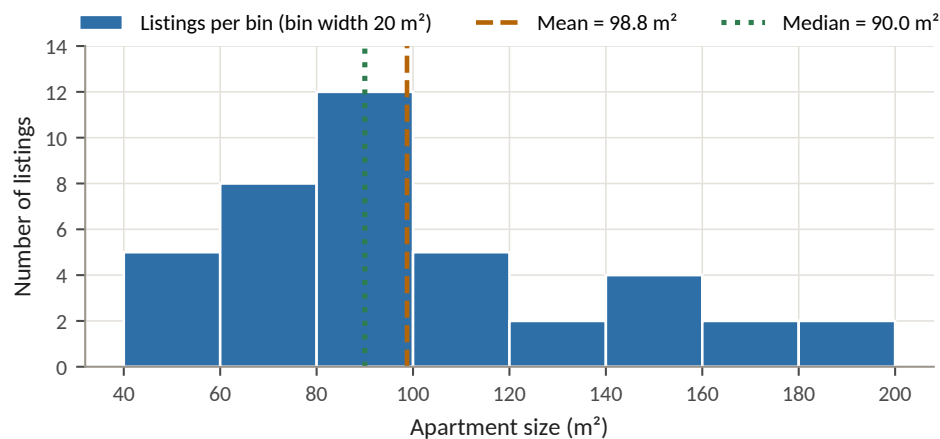


Figure 3.4 – Histogram of the sizes of 40 listings, with bins 20 m² wide. The dashed line is the mean (98.8 m²) and the dotted line the median (90 m²).

WHAT TO OBSERVE

Most listings sit between 60 and 120 m², and the bars then thin out to the right in a long tail of large apartments up to 200 m². A tail on the right pulls the mean to the right of the median — exactly the gap between the two lines. Such a shape is called *right-skewed*; rents, incomes and waiting times usually are.

The box plot compresses the same column into five numbers and a few dots, and needs a few more words.

WORDS TO KNOW

Percentile — The value below which a given percentage of the column falls; the 25th percentile is the value that a quarter of the values are below.

Quartiles — The three values that cut the sorted column into four equal parts: the first quartile Q_1 (25th percentile), the median (50th) and the third quartile Q_3 (75th).

Interquartile range (IQR) — $Q_3 - Q_1$: the width of the middle half of the values. Like the median, it is barely affected by outliers.

Box plot — A picture of a column: a box from Q_1 to Q_3 with a line at the median, whiskers to the most extreme values that are not outliers, and a dot for each outlier. It was introduced by the statistician John Tukey in 1977.

Outlier — A value that sits far from most of the other values in its column. It may be a genuine but unusual example, such as a real luxury apartment, or a mistake, such as an extra digit typed by accident; only checking can tell the two apart.

By hand, the quartiles are simply medians of halves: sort the values, split them at the median (leaving the middle value out when n is odd), and take the median of each half. The box plot also needs a rule for which values count as outliers, and Tukey's rule uses the IQR.

EQUATION: QUARTILES, INTERQUARTILE RANGE AND OUTLIER FENCES

$$\text{IQR} = Q_3 - Q_1, \quad \text{lower fence} = Q_1 - 1.5 \text{ IQR}, \quad \text{upper fence} = Q_3 + 1.5 \text{ IQR}$$

In words: Find the first and third quartiles as the medians of the lower and upper halves of the sorted values. Their difference is the IQR. Any value more than one and a half IQRs below Q_1 or above Q_3 is drawn as an outlier; the whiskers stop at the last values inside the fences.

When it is used: To draw a box plot, to compare the spread of several groups, and (in Chapter 4) as a first rule for flagging values that deserve a check.

Symbol	Meaning	Unit or type
Q_1	first quartile, the median of the lower half	unit of the column
Q_3	third quartile, the median of the upper half	unit of the column
IQR	interquartile range	unit of the column
1.5	Tukey's multiplier for the fences	no unit

Software places Q_1 and Q_3 with slightly different rules (pandas interpolates between the two nearest sorted values); for a small column the rules can differ by a few units, for hundreds of values they agree closely. This chapter's box plots use the hand rule.

WORKED EXAMPLE 3.4 — Quartiles, IQR and Fences for the Eight Sizes

Find Q_1 , Q_3 , the IQR and the fences for the eight sizes in Table 3.1, and decide which listings would appear as outlier dots in a box plot.

1. Sort the values: 60, 75, 75, 85, 95, 105, 105, 200. The median is $(85 + 95) \div 2 = 90$, as before.
2. Lower half (the four smallest): 60, 75, 75, 85. Its median is $(75 + 75) \div 2 = 75$, so $Q_1 = 75 \text{ m}^2$.
3. Upper half (the four largest): 95, 105, 105, 200. Its median is $(105 + 105) \div 2 = 105$, so $Q_3 = 105 \text{ m}^2$.
4. $\text{IQR} = 105 - 75 = 30 \text{ m}^2$.
5. Fences: $75 - 1.5 \times 30 = 30 \text{ m}^2$ and $105 + 1.5 \times 30 = 150 \text{ m}^2$. All sizes are above 30, and only 200 is above 150.

Answer: $Q_1 = 75 \text{ m}^2$, $Q_3 = 105 \text{ m}^2$, $\text{IQR} = 30 \text{ m}^2$, fences 30 and 150 m^2 . The 200 m^2 apartment is the only outlier; the middle half of the listings is only 30 m^2 wide, even though the full range is 140 m^2 .

Figure 3.5 shows every part of a box plot, drawn for the 40 sizes. With 40 values the lower half has 20 values, so Q_1 is the average of the 10th and 11th sorted sizes and Q_3 the average of the 30th and 31st.

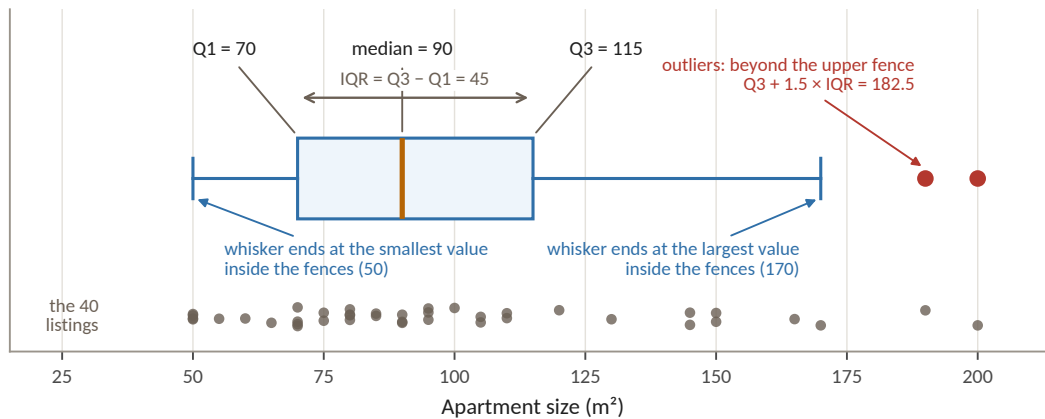


Figure 3.5 – Anatomy of a box plot, drawn for the sizes of the 40 listings ($Q_1 = 70$, median = 90, $Q_3 = 115$ m²). The gray dots below show the raw values that the box summarizes.

WHAT TO OBSERVE

The box holds the middle half of the listings, from 70 to 115 m². The upper whisker stops at 170 m², the largest size inside the fence of $115 + 1.5 \times 45 = 182.5$ m², and the two apartments beyond it (190 and 200 m²) become dots. The plot hides the exact positions of the 40 values but shows center, spread and outliers at a glance.

The real power of the box plot appears when several groups sit side by side. Figure 3.6 draws one box per city for the rent column.

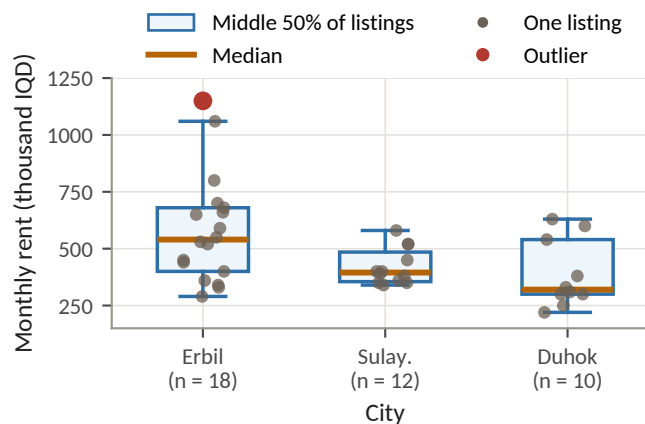


Figure 3.6 – Rent by city for the 40 listings (Erbil 18, Sulaymaniyah 12, Duhok 10). The gray dots are the individual listings.

WHAT TO OBSERVE

The medians fall from Erbil (540) to Sulaymaniyah (395) to Duhok (320 thousand IQD), so city clearly carries information about rent. Erbil also has by far the widest box and one outlier dot, the 1150 listing of Table 3.1 — a flag to check the row, not an order to delete it (Chapter 4 gives the procedure). Duhok's

median sits near the bottom of its box: most Duhok rents are low, a few higher ones stretch the box upward.

3.5 Seeing Two Columns: Scatter Plots and Correlation

A landlord in Erbil knows without any statistics that bigger apartments rent for more. A model needs that knowledge as a number: how strongly, and in which direction, do two columns move together? The picture is the scatter plot; the number is the correlation coefficient.

WORDS TO KNOW

Scatter plot — A plot with one feature on the horizontal axis and another on the vertical axis, and one point per sample. It shows whether two columns move together, how tightly, and whether the relationship is a straight line or a curve.

Correlation coefficient r — A number between -1 and $+1$ that measures how closely two numerical columns follow a straight line together. Values near $+1$ mean they rise together, values near -1 mean one falls as the other rises, and values near 0 mean no straight-line relationship.

Figure 3.7(a) plots size against rent for the 40 listings, with the five listings of Worked Example 3.3 marked as squares, and Figure 3.7(b) shows four simulated clouds of 40 points with $r = 0.9, 0.5, 0$ and -0.8 , to build a feel for the size of r .

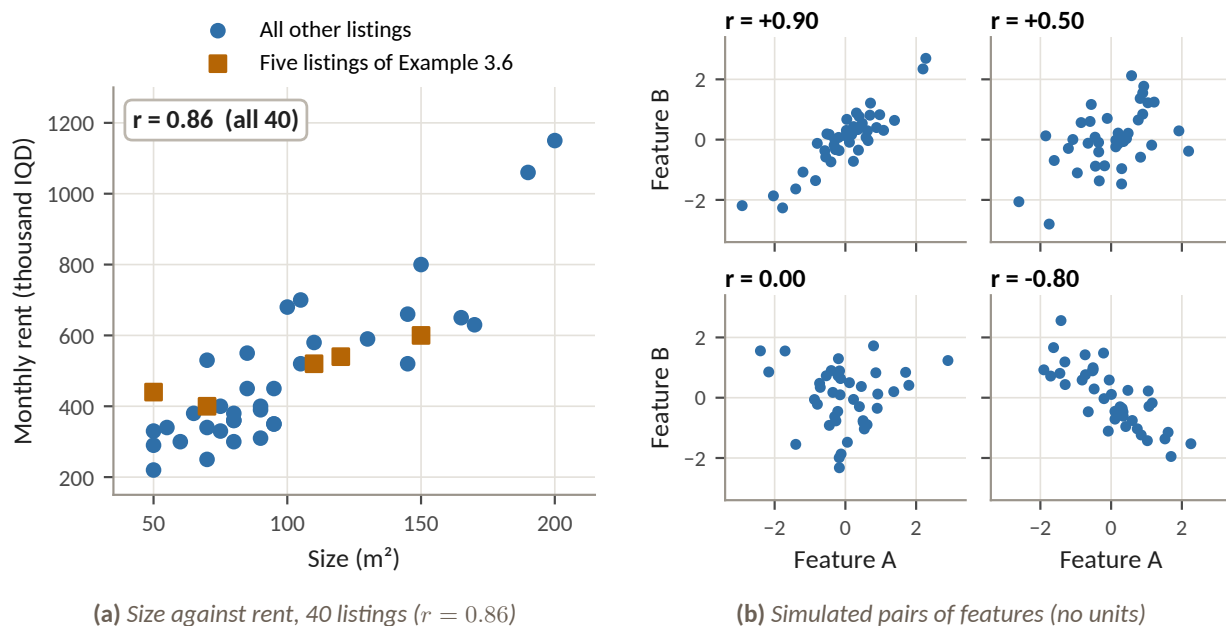


Figure 3.7 – Scatter plots and the correlation coefficient. Left: the real listings. Right: four clouds with the same number of points but different values of r .

WHAT TO OBSERVE

In (a) the cloud slopes upward: larger apartments almost always rent for more. It is not a perfect line, because city, age and distance also matter, but it is narrow enough that size alone would give a useful prediction. In (b), at $r = 0.9$ the points hug a rising line, at 0.5 the trend is visible but loose, at 0 the cloud has no direction, and at -0.8 the line falls steeply. Rough guide: $|r|$ above 0.7 is strong, 0.3 to 0.7 moderate, below 0.3 weak; the sign gives the direction, the size the tightness.

The idea of measuring such a relationship with one number was introduced by Francis Galton in 1888, and the formula used today was worked out by Karl Pearson in 1895. It multiplies each sample's two deviations from the means: a large size with a large rent gives a positive product, a small size with a small rent also gives a positive product, and only mismatched pairs give negative products. The sum of the products is then divided by a scaling term so that the result always lies between -1 and $+1$.

EQUATION: PEARSON CORRELATION COEFFICIENT

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

In words: For every sample, multiply its x deviation by its y deviation and add the products. Divide by the square root of the sum of squared x deviations times the sum of squared y deviations.

When it is used: To measure the straight-line relationship between two numerical columns: a feature and the target (is the feature useful?) or two features (do they repeat each other?). Not for categorical columns; it misses curved relationships.

Symbol	Meaning	Unit or type
r	the correlation coefficient	no unit, from -1 to $+1$
x_i, y_i	the two values of sample i	units of the two columns
$\bar{x}, \bar{y}$	the means of the two columns; n is the number of samples	units of the two columns

The denominator equals $(n - 1) s_x s_y$, the product of the two standard deviations times $n - 1$. Changing a unit (dinars to dollars, meters to feet) changes numerator and denominator by the same factor, so r has no unit and does not change.

WORKED EXAMPLE 3.5 — Correlation Between Size and Rent for Five Listings

Compute r for the five listings of Worked Example 3.3: sizes 50, 70, 110, 120, 150 m² and rents 440, 400, 520, 540, 600 thousand IQD.

- Means: $\bar{x} = 500 \div 5 = 100$ m² and $\bar{y} = 2500 \div 5 = 500$ thousand IQD.
- Size deviations $x_i - \bar{x}$: $-50, -30, 10, 20, 50$. Rent deviations $y_i - \bar{y}$: $-60, -100, 20, 40, 100$.
- Products of paired deviations: $(-50)(-60) = 3000$; $(-30)(-100) = 3000$; $(10)(20) = 200$; $(20)(40) = 800$; $(50)(100) = 5000$. Sum: 12,000.
- Sums of squared deviations: sizes $2500 + 900 + 100 + 400 + 2500 = 6400$ (from Worked Example 3.3); rents $3600 + 10,000 + 400 + 1600 + 10,000 = 25,600$.
- Denominator: $\sqrt{6400 \times 25,600} = \sqrt{163,840,000} = 12,800$ (check: $(n-1) s_x s_y = 4 \times 40 \times 80 = 12,800$). Divide: $r = 12,000 \div 12,800 = 0.9375$.

Answer: $r = 0.9375$, about 0.94: a strong positive correlation. Every product in step 3 was positive because every listing with a below-average size also had a below-average rent, and vice versa; the one imperfection is listing 10, larger than listing 9 but cheaper.

COMMON MISTAKE

X Wrong: Reading a high correlation as proof that one column *causes* the other — “ice-cream sales and drownings both rise in summer, so ice cream causes drowning.”

✓ Right: Correlation only says that two columns move together; a third factor (the summer heat) may drive both. And r only sees straight lines: two columns can have a perfect U-shaped relationship and still give $r \approx 0$. Always look at the scatter plot too.

3.6 Seeing Many Columns: the Correlation Heat Map and Pair Plot

A dataset with five numerical columns has ten pairs, and a real project may have fifty columns and more than a thousand pairs. Nobody can read a thousand scatter plots, so the correlations are collected in a table and colored.

WORDS TO KNOW

Correlation matrix — A square table with one row and one column per numerical feature, holding the correlation of every pair; the diagonal is always 1 and the table is symmetric.

Heat map — A grid whose cells are colored by their value, so that patterns can be seen without reading every number. A correlation heat map colors the correlation matrix.

Pair plot — A grid of scatter plots showing every pair of a few chosen features, with the histogram of each feature on the diagonal.

Table 3.4 is the correlation matrix of the five numerical columns of the 40 listings, computed with pandas in the Python activity, and Figure 3.8 shows the same table as a heat map.

Table 3.4 – Correlation matrix of the five numerical columns of the 40 listings

	Size	Rooms	Age	Distance	Rent
Size	1.00	0.89	−0.07	0.01	0.86
Rooms	0.89	1.00	−0.01	0.00	0.75
Age	−0.07	−0.01	1.00	0.08	−0.36
Distance	0.01	0.00	0.08	1.00	−0.22
Rent	0.86	0.75	−0.36	−0.22	1.00

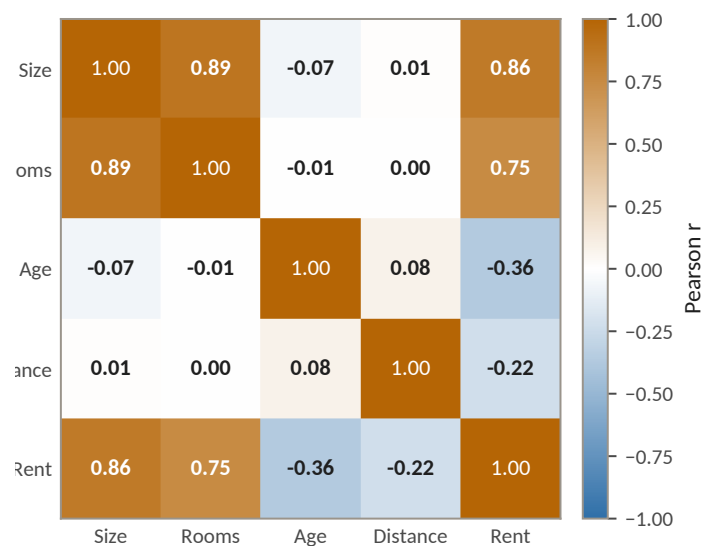


Figure 3.8 – Correlation heat map of the five numerical columns. Orange cells are positive correlations, blue cells negative, white cells near zero.

WHAT TO OBSERVE

Read the last row first: it shows how every feature relates to the target. Size (0.86) and rooms (0.75) are strongly and positively related to rent; age (-0.36) and distance (-0.22) weakly and negatively. The diagonal of 1.00 carries no information.

WORKED EXAMPLE 3.6 — Read a Correlation Heat Map

Using Table 3.4 or Figure 3.8, answer three questions from the website's founders: (a) Which single feature is most useful for predicting rent? (b) Which two features say almost the same thing? (c) Do older apartments rent for more or less?

1. For (a), read the Rent row and ignore the 1.00 on the diagonal: Size 0.86, Rooms 0.75, Age -0.36 , Distance -0.22 . The largest size of r is 0.86, for Size.
2. For (b), look for the largest correlation between two *features*, away from the Rent row and the diagonal: Size and Rooms, $r = 0.89$. Bigger apartments have more rooms, so the two columns largely repeat each other.
3. For (c), read the sign of the Age–Rent cell: -0.36 is negative, so as age goes up, rent tends to go down, but only weakly.

Answer: Size is the most useful single feature ($r = 0.86$); size and rooms are largely redundant ($r = 0.89$), so a simple model may not need both; and older apartments rent for a little less ($r = -0.36$). Distance shows only a weak negative relationship ($r = -0.22$), which may partly be chance — 40 listings cannot pin down a weak correlation precisely.

The heat map shows numbers; the pair plot shows shapes. Figure 3.9 draws every pair of three chosen columns, with each column's histogram on the diagonal. Features that repeat each other, like size and rooms, are called *redundant*; redundancy is not an error, but it wastes effort and can confuse some models (Chapter 22 chooses among redundant features, Chapter 20 merges them).

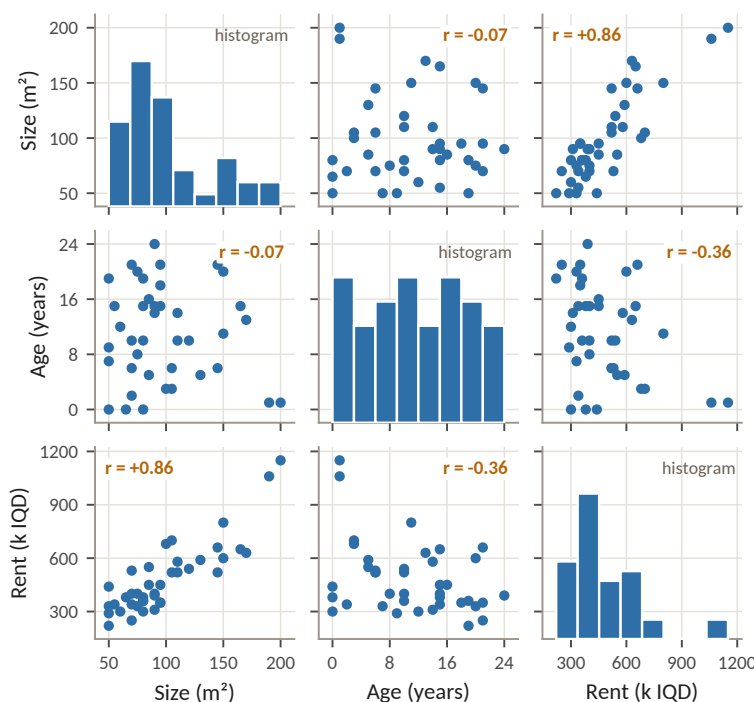


Figure 3.9 – Pair plot of size, age and rent for the 40 listings. Off-diagonal panels are scatter plots (with r in the corner); diagonal panels are histograms.

WHAT TO OBSERVE

The size-rent panel repeats Figure 3.7(a). The age-rent panel shows a loose downward drift: $r = -0.36$ is a weak relationship, not a tight one. The size-age panel is a shapeless cloud ($r = -0.07$). The diagonal histograms show that size and rent are right-skewed while age is spread evenly from 0 to 24 years.

3.7 A First-Look Checklist for Any New Dataset

Everything in this chapter comes together as a short routine that a data team runs on every new table before cleaning it (Chapter 4), splitting it (Chapter 5) or training anything. John Tukey, who introduced the box plot, called this habit *exploratory data analysis* in his 1977 book of the same name. Table 3.5 is the checklist; every line names the tool from this chapter that answers the question, and what a warning sign looks like.

WORD TO KNOW

Exploratory data analysis (EDA) — The habit of describing and plotting a dataset — one column at a time, then pairs, then everything — before building any model, in order to understand what the data holds and to catch problems early.

Table 3.5 – The first-look checklist for a new dataset

Question	Tool	Warning signs
Where did the rows come from, what is one row, and whom does the sample leave out?	Source description; row count; counts per group (city, month, device)	One group dominates; a period or group is missing; duplicated IDs
What type and unit does each column have, and which column is the target?	Feature-type table (data dictionary); task statement	Digits used as categories; mixed units; dates stored as text; a feature that is a copy of the target
What is typical, and how spread out, in each column?	<code>describe()</code> : mean, median, std, min, max	Mean far from median; impossible minimum or maximum (a negative size, an age of 250)
What shape does each column have?	Histogram; box plot	Long tails; a spike at zero; outlier dots; several separate humps
Which features relate to the target, and which repeat each other?	Scatter plots; correlation matrix and heat map; pair plot	No feature relates to the target; two features with r near 1; a curved pattern r cannot see

One rule from Chapter 2 still applies: looking at all the rows to understand them is fine, but any *decision* learned from the data — a fill-in value, a scaling rule, a choice of features — must later be learned from the training rows only, or it leaks test-set information into the model.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing that a large dataset must be a representative one; a million rows from one city still describe only that city.
- Treating any column written with digits (an ID, a postal code, a category code) as a numerical feature, or treating an ordinal feature as if the steps between its values were equal.
- Reporting the mean as “the typical value” when a long tail or an outlier pulls it far from the median.
- Dividing by n instead of $n - 1$ in the sample variance, and deleting every value a box plot marks as an outlier without checking whether it is real.

- Reading a strong correlation as proof of cause, or reading $r \approx 0$ as proof that two columns are unrelated when the relationship is curved.

Practical Activities

ACTIVITY (PAPER) — Draw a Histogram and a Box Plot by Hand

A shop assistant in Duhok collected the monthly rents (thousand IQD) of twelve apartments on her street: 350, 250, 400, 900, 320, 450, 380, 600, 350, 500, 300, 420.

1. Sort the twelve values. Make bins 100 wide (200–299, 300–399, ..., 900–999), count the rents in each, and draw the histogram. Describe its shape in one sentence.
2. Find the median (average of the 6th and 7th sorted values), then Q_1 (median of the six smallest) and Q_3 (median of the six largest).
3. Compute the IQR and the two fences, $Q_1 - 1.5 \text{ IQR}$ and $Q_3 + 1.5 \text{ IQR}$. Which value, if any, lies outside them?
4. Draw the box plot on the same scale as the histogram: box from Q_1 to Q_3 , a line at the median, whiskers to the smallest and largest values inside the fences, and a dot for each outlier. Then compute the mean, compare it with the median, and decide which of the two you would tell a friend who asks “what do apartments on this street cost?”.

You need: Graph paper, pencil, ruler and a calculator

ACTIVITY (PYTHON) — The First Look With pandas

The program first repeats the hand calculations of Worked Examples 3.2 to 3.5 on the same rows, then runs the first-look checklist on all 40 listings: `describe()`, a histogram, box plots by city, the correlation matrix and a heat map drawn with `matplotlib` alone. Run it from the book's folder so that it finds `datasets/ch03_apartments.csv`.

Python • pandas • matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("datasets/ch03_apartments.csv")
eight = df.head(8)  # the 8 listings of Table 3.1
print("mean", eight["size_m2"].mean(), " median", eight["size_m2"].median(),
      " Q1, Q3", eight["size_m2"].quantile([0.25, 0.75]).tolist())
five = df.iloc[8:13]  # listings 9-13 (Examples 3.3, 3.5)
print("std size", five["size_m2"].std(), " std rent", five["rent"].std(),
      " r", five["size_m2"].corr(five["rent"]))

num = ["size_m2", "rooms", "age_years", "distance_km", "rent"]
print(df[num].describe().round(1).T)  # all 40 listings
corr = df[num].corr()  # Table 3.4

fig, ax = plt.subplots(1, 3, figsize=(12, 3.6))
df["size_m2"].hist(ax=ax[0], bins=8, edgecolor="white")
ax[0].set_xlabel("Size (m²)"); ax[0].set_ylabel("Number of listings")
df.boxplot(column="rent", by="city", ax=ax[1]); ax[1].set_ylabel("Rent (thousand IQD)")
im = ax[2].imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax[2].set_xticks(range(5), num, rotation=45); ax[2].set_yticks(range(5), num)
fig.colorbar(im, ax=ax[2], label="Pearson r"); fig.tight_layout();
fig.savefig("first_look.png")
```

Output

```
mean 100.0  median 90.0  Q1, Q3 [75.0, 105.0]
std size 40.0  std rent 80.0  r 0.9375

count  mean  std  min  25%  50%  75%  max
size_m2  40.0  98.8  39.2  50.0  70.0  90.0  112.5  200.0
rooms    40.0   2.5   1.0   1.0   2.0   2.0   3.0   4.0
age_years 40.0  11.0   7.0   0.0   5.8  10.5  15.2  24.0
distance_km 40.0  3.8   1.9   0.5   2.2   3.9   5.2   7.4
rent      40.0 485.0 199.9 220.0 347.5 420.0 582.5 1150.0
```

The first two printed lines are the hand results: mean 100 and median 90 (Worked Example 3.2), quartiles 75 and 105 (3.4), standard deviations 40 and 80 (3.3) and $r = 0.9375$ (3.5). `describe()` then summarizes every numerical column of the 40 listings (its standard deviation uses $n - 1$); the mean size 98.8 and median 90 are the two lines of Figure 3.4, and pandas puts the 75% point of size at 112.5 where the hand rule of Figure 3.5 gives 115 — the small difference between quartile rules noted in the equation box. `corr()` computes Table 3.4, and the last block draws the three plots into `first_look.png` for comparison with Figures 3.4, 3.6 and 3.8.

You need: A computer with Python, pandas and matplotlib installed (see Appendix B) and the file `datasets/ch03_apartments.csv`

CASE STUDY — A Rental-Listings Website Builds Its First Dataset — Erbil, Kurdistan Region

Two friends in Erbil run a website where landlords list apartments for rent. After six months they have about 2,000 listings and an idea: a model that suggests a fair rent to every landlord who posts a new apartment. Before touching any model, they run the first look of this chapter, and each step changes

their plan. The first question, where the rows come from, is uncomfortable. The website launched in Erbil and only later added Duhok and Sulaymaniyah, so about three quarters of the listings are in Erbil — the pattern of Lesson 3.1. Worse, every row is a listing a landlord chose to post online, so apartments rented by word of mouth in smaller towns are missing completely. The friends decide to state plainly that the first model is for the three cities only, and to collect listings in the two smaller cities for another three months before training.

The second question, what type and unit each column has, uncovers three problems in the collection form: a few landlords who had lived abroad entered sizes in square feet, rent was sometimes typed in dinars and sometimes in dollars, and “condition” was a free-text box in which the same idea appeared as “good”, “Good”, “very good” and “fine.” They redesign the form — size in square meters only, rent in thousand IQD per month with the unit printed next to the box, condition as a fixed ordinal list of four values — and add a time stamp to every listing, because a rent from last winter is not the same fact as a rent from this month.

The third question, what is typical and what is spread out, is answered by `describe()` and a few plots, which flag a size of 8 m², a rent of 45 and an apartment with 11 rooms for checking. Two are typing mistakes and one is a real dormitory building; none is deleted silently, and each decision goes into a short data log. Finally, the heat map shows that size and rooms repeat each other, and that “days on the site” is strongly related to rent because overpriced apartments stay listed longer. That column is dropped: it is only known *after* a listing is posted, so a model using it would be cheating in the sense of Chapter 2. Nothing here required a learning algorithm; when the founders train a model in a later chapter, it will learn from a table they understand.

CHAPTER SUMMARY

- Data comes from measurements, surveys and forms, sensors, logs and existing records; units and time stamps must be fixed at collection time, and the sample must represent the population the model will serve — a large sample with a systematic gap is still biased.
- A dataset is a table: rows are samples, columns are features, one column is the target; the feature matrix **X** has n rows and p columns, the target vector **y** has n values, and identifiers are not features. Every column is numerical (continuous or discrete), categorical (nominal, ordinal or binary), text or a date, and the type decides how it is summarized, plotted and later cleaned.
- The mean $\bar{x} = \frac{1}{n} \sum x_i$ is pulled by extreme values; the median is not; the mode is the most common value; the range is the largest value minus the smallest.
- The sample variance $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$ and its square root, the standard deviation s , measure spread in the column’s own unit.
- A histogram shows the shape of one column (a right-skewed shape pushes the mean above the median); a box plot shows the quartiles, $IQR = Q_3 - Q_1$, and the values beyond $Q_1 - 1.5 IQR$ or $Q_3 + 1.5 IQR$ as outlier dots, which deserve a check, not automatic deletion.
- A scatter plot shows how two columns move together; the Pearson correlation coefficient r , between -1 and $+1$, measures the straight-line part of that relationship, has no unit, and says nothing about cause.
- A correlation matrix, its heat map and a pair plot show all pairs at once; read the target’s row first, then look for redundant features. The first-look checklist runs before any cleaning, splitting or training.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher’s Guide.

Problem 1 [Basic] A hospital dataset has these columns: patient age, blood type, pain level rated from 1 to 10 by the patient, date of admission, smoker (yes/no), a doctor's written notes, and a patient number. Classify each column by type, and say which one is an identifier.

Problem 2 [Basic] Five delivery times, in minutes, are 22, 18, 35, 20 and 26. Find the median and the mean, showing every step.

Problem 3 [Basic] The Rooms column of eight listings reads 1, 2, 2, 3, 2, 4, 3, 2. Find the mode and the range.

Problem 4 [Basic] For each correlation coefficient say whether the relationship is positive or negative and whether it is strong, moderate or weak: (a) $r = 0.92$, (b) $r = -0.15$, (c) $r = -0.78$, (d) $r = 0.05$.

Problem 5 [Standard] Five daily temperatures in Zakho were 15, 17, 21, 22 and 25 °C. Find the mean, the sum of squared deviations, the sample variance and the sample standard deviation, showing every step.

Problem 6 [Standard] Eight monthly rents (thousand IQD) are 300, 450, 250, 900, 350, 300, 450, 400. Sort them, find Q_1 , the median, Q_3 and the IQR, compute both fences, and say which value (if any) a box plot would draw as an outlier.

Problem 7 [Standard] A café in Sulaymaniyah records the hours of sunshine and the number of cold drinks sold on five days: sunshine 1, 2, 3, 4, 5 hours; drinks 14, 20, 17, 23, 26. Compute the Pearson correlation coefficient by hand, showing the deviations, the products and the two sums of squares.

Problem 8 [Standard] Using Figure 3.6, (a) which city has the widest middle half (the biggest IQR)? (b) In which city is the median closest to the bottom of the box, and what does that say about the shape of that city's rents? (c) Which listing in Table 3.1 is the outlier dot?

Problem 9 [Challenge] Seven recorded temperatures, in °C, are 19, 21, 20, 22, 18, 20 and 95. (a) Find the mean and the median of all seven values. (b) Find the mean of the six values without 95. (c) Explain, using these numbers, why the median is called a *robust* typical value. (d) Say what a careful team should do with the value 95 instead of deleting it.

Problem 10 [Challenge] The website decides to record rent in US dollars instead of thousand IQD, dividing every rent of the five listings of Worked Example 3.5 by the same exchange rate. (a) Without recomputing everything, explain why the correlation between size and rent stays exactly 0.9375. (b) What happens to the mean and the standard deviation of the rent column? (c) Use the equation note $(n - 1) s_x s_y$ to check the denominator 12,800 of Worked Example 3.5 from the standard deviations 40 and 80.

Problem 11 [Challenge] Design the data collection for a small project at your school, workplace or town: state the population, how you would collect a representative sample, five features with their types and units, the target, how you would record the time of each row, and one sampling bias to guard against.

Problem 12 [Challenge] Two columns are related by a perfect U shape: $y = (x - 5)^2$ for $x = 1, 2, \dots, 9$. (a) Without computing, explain why the Pearson correlation of x and y is exactly zero. (b) Compute $\bar{x}$ and $\bar{y}$, then the products $(x_i - \bar{x})(y_i - \bar{y})$ for $x = 1$ and for $x = 9$, and show that they cancel. (c) State the lesson for reading a correlation heat map.

Data Cleaning and Preprocessing

Missing values, outliers, duplicates, encoding, and scaling — done safely

IN THIS CHAPTER YOU WILL LEARN TO

- Find the missing values, outliers and duplicate rows hiding in a raw table, and fill a missing value with the training median.
- Compute a z-score and the IQR fences by hand, and use them to flag values that need a human check.
- Encode ordered categories as ordered codes and unordered categories as one-hot 0/1 columns, and avoid the dummy-variable trap.
- Standardize and min-max scale numerical features by hand, and explain why distance-based models need it.
- Explain why every preprocessing rule must be learned from the training rows only, and build a scikit-learn pipeline that guarantees it.

Introduction

A good cook never starts cooking with the vegetables straight from the market bag. First they are washed, the bruised parts are cut away, and every ingredient is measured in the same units. None of this is the meal, but skipping it almost always ruins the meal later. Data works the same way. A rental website that lists apartments in Erbil, Duhok and Sulaymaniyah receives its listings from hundreds of landlords who type on their phones, in a hurry. One forgets to enter the size. Another types 800 square meters instead of 80. A third presses “submit” twice and creates the same listing two times. None of them did anything unusual; this is simply what raw data looks like, everywhere in the world.

This chapter is the kitchen work that follows the first look of Chapter 3: finding the problems in a raw table, fixing them carefully, turning words into numbers a model can use, and putting every number on a fair scale. Along the way you will meet one rule that protects every project in this book — every cleaning rule is learned from the training rows only — and the tool, the pipeline, that makes it impossible to forget.

4.1 Why Raw Data Is Never Ready

The website’s data team exports its nine most recent listings, shown in Table 4.1. Before reading on, spend a minute looking for anything strange yourself.

Table 4.1 – Nine raw apartment listings exported from the rental website (illustrative example data)

ID	Size (m ²)	Age (years)	City	Condition	Rent (thousand IQD)
1	60	12	Erbil	Fair	300
2	65	6	Duhok	Good	340
3	71	22	Erbil	Good	360
4	75	2	Sulaymaniyah	Excellent	380
5	(blank)	42	Duhok	Good	410
6	92	15	erbil	Fair	440
7	98	8	Sulaymaniyah	Good	480
8	104	30	Erbil	Excellent	950
9	71	22	Erbil	Good	360

A careful look finds four problems. Listing 5 has no size. Listing 6 spells its city “erbil”, which a computer counts as a fourth city. Listing 8 asks a rent more than twice as high as any other — a typing mistake, or a genuinely special apartment? Listing 9 copies listing 3 in every column, almost certainly a landlord who pressed “submit” twice. Trained on this table as it stands, a model would learn from a phantom city, a ghost duplicate and a blank it cannot read.

Getting the table ready is called preprocessing, and Figure 4.1 shows the order of the work. Cleaning comes first, because every later step depends on trustworthy values; encoding turns words such as “Duhok” into numbers; scaling puts sizes, ages and rents on a comparable footing. Only then does a model see the data. Two habits protect the whole process: keep a *data-quality log* (for every value you change, note the row, the old value, the new value and why), and check free-text columns for private details a landlord may have typed in — a phone number, a name, an address — and remove them before the data is shared or used for training.

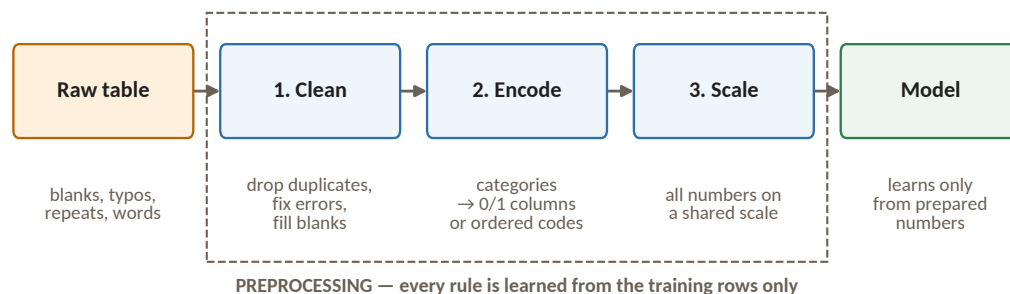


Figure 4.1 – The preprocessing pipeline. Raw data is cleaned, then encoded, then scaled, before any model sees it. Every rule inside the dashed box is learned from the training rows only.

WHAT TO OBSERVE

The three middle boxes are all *rules*: which value fills a blank, which number stands for which city, which mean and standard deviation rescale a column. Each rule is learned from the training rows and then simply *reused* on every other row (Lesson 4.7).

WORD TO KNOW

Preprocessing — All the steps that prepare raw data for a model before training begins. *Data cleaning* — finding and fixing missing values, errors, outliers, inconsistent spellings and duplicate rows — comes first; turning categories into numbers and putting numbers on a shared scale come next.

4.2 Missing Values: Find, Understand, Then Fill or Drop

Listing 5 arrived without a size. Perhaps the landlord never measured the apartment, or the form allowed the box to stay empty. The value is unknown — and unknown is not the same as zero.

WORD TO KNOW

Missing value — A feature value that was not recorded, and is simply blank in the table (software often shows it as NaN, “not a number”). It is not automatically zero and not automatically an error; it means the true value is unknown.

Before deciding what to do about a blank, look at the pattern of blanks, because the pattern decides the treatment. Figure 4.2 shows a larger table from the same website with its blanks shaded.

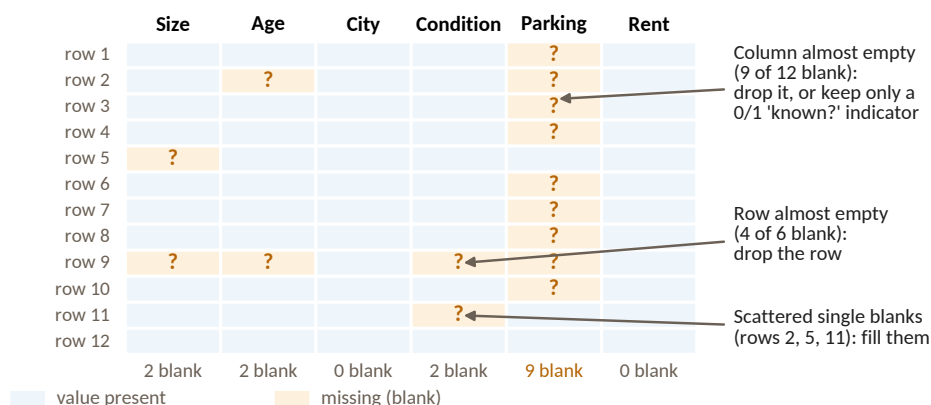


Figure 4.2 – A missing-value map of twelve listings and six columns. Shaded cells are blank. Three patterns appear: an almost-empty column, an almost-empty row, and scattered single blanks.

WHAT TO OBSERVE

The *Parking* column is blank in 9 of 12 rows: too little is known to fill it, so the column is dropped or replaced by a 0/1 column meaning “parking information was given”. Row 9 is blank in 4 of its 6 cells, so the row is dropped. The three scattered blanks can be filled in.

Filling a blank with a sensible substitute is called imputation: the training median or mean for a numerical column, the most frequent category for a categorical one. When the blankness itself may carry information — landlords of cheap apartments may skip the size box more often — an extra column records which rows were filled.

WORDS TO KNOW

Imputation — Filling a missing value with a substitute learned from the training data, such as the column’s median, mean or most frequent value.

Indicator column — An extra 0/1 column that records whether the original value was missing (1) or present (0). It lets a model notice that “unknown” may itself carry information.

The median is usually the safest single fill value, because a typing mistake elsewhere in the column barely moves it. The median is the middle value after sorting.

EQUATION: MEDIAN IMPUTATION RULE

$$x_{\text{fill}} = \text{median}(x_{(1)}, x_{(2)}, \dots, x_{(m)}) = \begin{cases} x_{(k+1)} & \text{if } m = 2k + 1 \text{ (odd count)} \\ \frac{x_{(k)} + x_{(k+1)}}{2} & \text{if } m = 2k \text{ (even count)} \end{cases}$$

In words: Sort the known training values. With an odd count the fill value is the middle one; with an even count, average the two middle ones. Write that number into every blank cell of the column.

When it is used: To fill blanks in a numerical column before training, using training rows only; the same fill value is reused for blanks in test rows and new listings.

Symbol	Meaning	Unit or type
x_{fill}	the value written into every blank of the column	unit of the feature
m	number of known training values in the column	count
$x_{(k)}$	the k -th smallest known value after sorting	unit of the feature

Replacing the median by the mean gives mean imputation; replacing it by the most frequent value gives the rule for categorical columns.

WORKED EXAMPLE 4.1 — Fill a Missing Size With the Training Median

Listing 9 is a duplicate and will be removed in Lesson 4.4, so the training table has eight listings, seven of them with a known size: 60, 65, 71, 75, 92, 98 and 104 square meters. Fill the blank size of listing 5 with the median of the training sizes.

1. Sort the seven known sizes: 60, 65, 71, 75, 92, 98, 104. They are already in order.
2. Count them: $m = 7$, an odd number, so the median is the single middle value, the 4th: $x_{(4)} = 75$.
3. Write 75 into the blank size of listing 5. (Had the 75 been typed as 800, the mean would jump from about 80.7 to about 184, while the median would only move to 92.)

Answer: The missing size is filled with 75 square meters. This number came from the seven training sizes alone; computing it from the test rows too would be data leakage, the trap Chapter 2 warned about.

Median imputation leaves a fingerprint. Figure 4.3 shows what happens on a larger table where 28 of 200 sizes were blank.

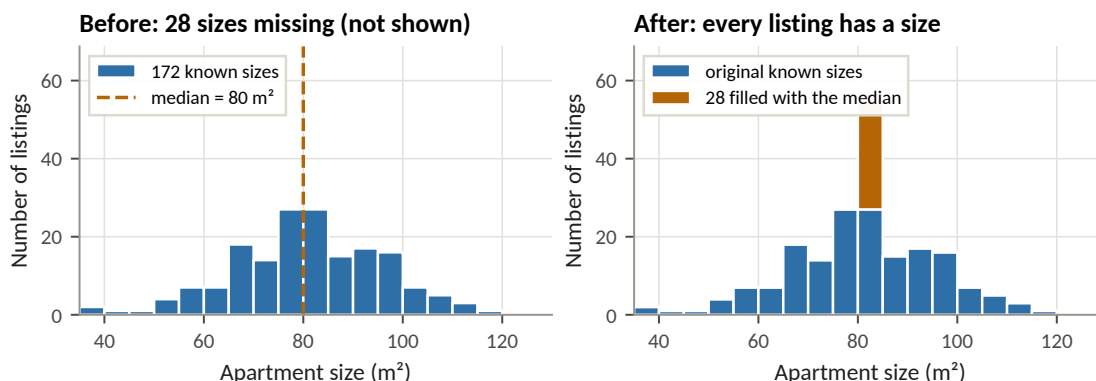


Figure 4.3 – Sizes of 200 listings before (left) and after (right) filling 28 blanks with the training median of 80 m². The filled values all land in one bar.

WHAT TO OBSERVE

The right-hand histogram has the same shape plus a tall orange spike at the median: every imputed apartment is now “exactly average”, which shrinks the column’s spread and overstates how common 80 m² is. An indicator column lets the model tell “size unknown, filled with 80” from “size measured as 80”.

COMMON MISTAKE

✗ Wrong: Assuming every blank cell means zero, such as reading a missing rent as “0 thousand IQD” or a missing age as “a brand-new building”.

✓ Right: Treat a blank as unknown. Fill it with a value learned from the training set, such as the median, add an indicator column if the blankness itself may matter, and note the change in the log.

RESPONSIBLE USE

When rows describe people, dropping every row with a blank can silently remove a whole group — patients never given a certain test, applicants who left the income box empty. Ask *who* is missing, and why, before dropping anything.

4.3 Outliers and Errors: z-Scores, the IQR Rule, and Domain Checks

One morning a landlord submits an apartment of 800 square meters. Is it a palace, or did a “0” get typed twice? A value that sits far from the rest of its column is called an outlier, and the first job is to *notice* it. The usual way to measure “far from the rest” is to count how many standard deviations a value sits from the mean of a trusted reference column — here, the cleaned training sizes.

WORDS TO KNOW

Outlier — A value that sits far away from most of the other values in its column. It may be a genuine but unusual example, or a mistake such as an extra digit. Only checking can tell the two apart.

z-score — How many standard deviations a value sits above or below the mean of its reference data. A z-score of 0 means exactly average; +2 means two standard deviations above average; −2 means two below.

EQUATION: z-SCORE (STANDARDIZED VALUE)

$$z = \frac{x - \mu}{\sigma}$$

In words: Subtract the reference mean from the value, then divide by the reference standard deviation.

When it is used: To judge how unusual one value is against a trusted reference column — a common starting rule flags any value with $z \geq 3$ or $z \leq -3$ for a human check — and, under the name *standardization*, to rescale every value of a column before training (Lesson 4.6).

Symbol	Meaning	Unit or type
z	the z-score of the value	no unit (a plain number)
x	the value being checked	unit of the feature
μ	mean of the reference (training) column	unit of the feature
σ	standard deviation of the reference (training) column	unit of the feature

The habit of drawing warning lines three standard deviations from the mean goes back to the factory control charts of Walter Shewhart in 1931. Three is a convention, not a law.

The reference numbers come from Chapter 3: μ is the ordinary average and $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$ is the square root of the average squared distance from it (we divide by n , as scikit-learn does, rather than by $n - 1$). After Worked Example 4.1 the eight training sizes are 60, 65, 71, 75, 75, 92, 98 and 104. They add up to 640, so $\mu = 640 \div 8 = 80$. Their differences from 80 are $-20, -15, -9, -5, -5, 12, 18, 24$; the squares, 400, 225, 81, 25, 25, 144, 324, 576, add up to 1800, so $\sigma = \sqrt{1800 \div 8} = \sqrt{225} = 15$. These two numbers, 80 and 15, are the reference for every z-score and for standardization in Lesson 4.6.

WORKED EXAMPLE 4.2 — Spot a Likely Entry Error Among Six Submitted Sizes

Six new listings, A to F, arrive with sizes 74, 95, 65, 101, 86 and 800 square meters. Using the training mean of 80 and standard deviation of 15, find each z-score and flag any listing with $z \geq 3$ or $z \leq -3$.

1. Listing A: $z = (74 - 80) \div 15 = -6 \div 15 = -0.4$. Listing B: $z = (95 - 80) \div 15 = 15 \div 15 = 1.0$. Neither is flagged.
2. Listing C: $z = (65 - 80) \div 15 = -15 \div 15 = -1.0$. Listing D: $z = (101 - 80) \div 15 = 21 \div 15 = 1.4$. Neither is flagged.
3. Listing E: $z = (86 - 80) \div 15 = 6 \div 15 = 0.4$. Not flagged.
4. Listing F: $z = (800 - 80) \div 15 = 720 \div 15 = 48$. Far beyond 3: flagged.
5. Think about a real-world explanation: a “0” typed twice, for a true size of 80 m², would fit the other listings perfectly.

Answer: Listing F, with $z = 48$, is a likely data-entry error, probably 80 m² typed as 800. The right next step is not to delete or silently correct the row but to flag it and ask the landlord — who confirms 80 m².

Figure 4.4 draws the six submitted sizes against the band of “normal” sizes, from three standard deviations below the training mean to three above.

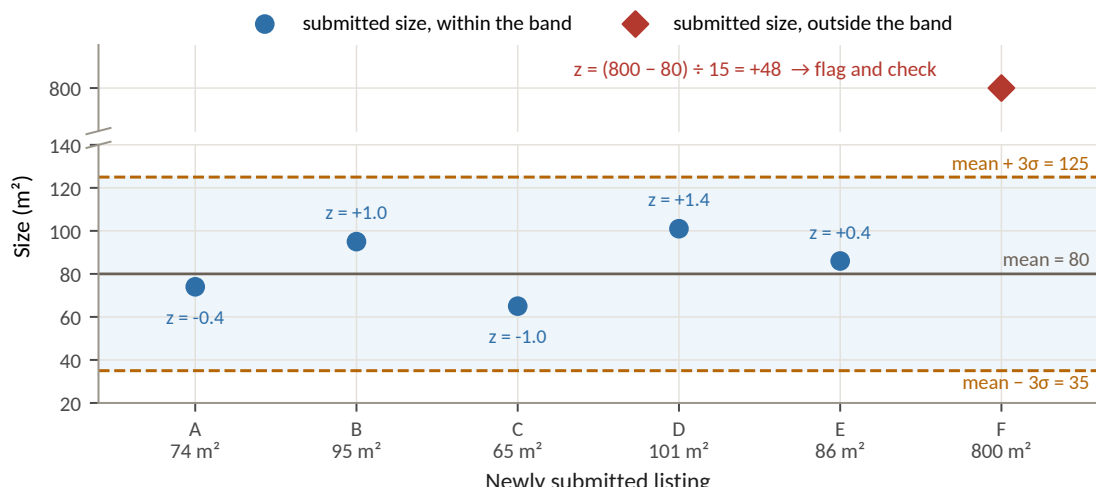


Figure 4.4 – The six submitted sizes against the training mean (80 m²) and the lines three standard deviations away (35 m² and 125 m²). The vertical axis is broken so that listing F fits on the page.

WHAT TO OBSERVE

Five listings sit inside the shaded band from 35 to 125 m²; listing F is so far outside that the axis had to be cut. The band comes from the *training* mean and standard deviation — the six new values did not move the lines at all.

COMMON MISTAKE

✗ Wrong: Computing the mean and standard deviation from the same handful of values you are checking. For the six submissions above that mean is 203.5 and that standard deviation about 267, so the z-score of the 800 m² listing is only about 2.2: the outlier inflated the very standard deviation used to judge it.

✓ Right: Compare new values with a large, trusted reference, such as the cleaned training column, or use the IQR rule below, which resists exactly this problem.

The z-score leans on the mean and standard deviation, and both are pulled by extreme values. A second rule uses the quartiles instead, which barely move when one value is wild. The middle half of a sorted column runs from the first quartile Q_1 to the third quartile Q_3 ; fences are drawn one and a half times that width beyond each quartile. This rule, and the box plot that draws it, were introduced by John Tukey in 1977.

WORD TO KNOW

Interquartile range (IQR) — The distance between the first and third quartiles, $Q_3 - Q_1$: the width of the middle half of the sorted values. The two cut-off lines $Q_1 - 1.5 \text{ IQR}$ and $Q_3 + 1.5 \text{ IQR}$ are called the *fences*; any value outside them is flagged as a possible outlier.

EQUATION: IQR OUTLIER FENCES

$$\text{IQR} = Q_3 - Q_1, \quad \text{lower fence} = Q_1 - 1.5 \times \text{IQR}, \quad \text{upper fence} = Q_3 + 1.5 \times \text{IQR}$$

In words: Sort the values and split them into a lower and an upper half (leave out the middle value if the count is odd). Q_1 is the median of the lower half, Q_3 the median of the upper half. Step one and a half IQRs below Q_1 and above Q_3 ; anything beyond those fences is flagged.

When it is used: To flag possible outliers in any numerical column, even a small one that itself contains the outlier. Box plots draw exactly these fences.

Symbol	Meaning	Unit or type
Q_1	first quartile (median of the lower half)	unit of the feature
Q_3	third quartile (median of the upper half)	unit of the feature
IQR	interquartile range, $Q_3 - Q_1$	unit of the feature
1.5	Tukey's multiplier; 3 is sometimes used for "extreme" outliers	no unit

Software computes quartiles by slightly different recipes; the fences land in almost the same place.

WORKED EXAMPLE 4.3 — Find the IQR Fences for Eight Rents

The eight training listings ask monthly rents of 300, 340, 360, 380, 410, 440, 480 and 950 thousand IQD. Find the fences and decide which rents to flag.

1. Sort the rents: 300, 340, 360, 380, 410, 440, 480, 950. The count is 8 (even), so the halves are 300, 340, 360, 380 and 410, 440, 480, 950.
2. Q_1 is the median of the lower half: $(340 + 360) \div 2 = 350$.
3. Q_3 is the median of the upper half: $(440 + 480) \div 2 = 460$.
4. $IQR = 460 - 350 = 110$, and $1.5 \times 110 = 165$.
5. Lower fence: $350 - 165 = 185$. Upper fence: $460 + 165 = 625$.
6. Compare: 300 to 480 all lie between 185 and 625; 950 lies above 625, so it is flagged.

Answer: The fences are 185 and 625 thousand IQD, and only the rent of 950 is flagged — even though 950 itself sat inside the calculation, because the quartiles ignore how far the largest value is from the rest.

Figure 4.5 is the box plot of these rents. A flag is a question, not a verdict, and only a person who knows the subject can answer it. Such domain checks also catch mistakes no statistic sees: a negative size, a rent of exactly 0, a building age of 240 years, a temperature column that switches from Celsius to Fahrenheit halfway through. The routine is always the same: flag, check with someone who knows, correct or keep, and write it in the log — never delete a flagged row silently.

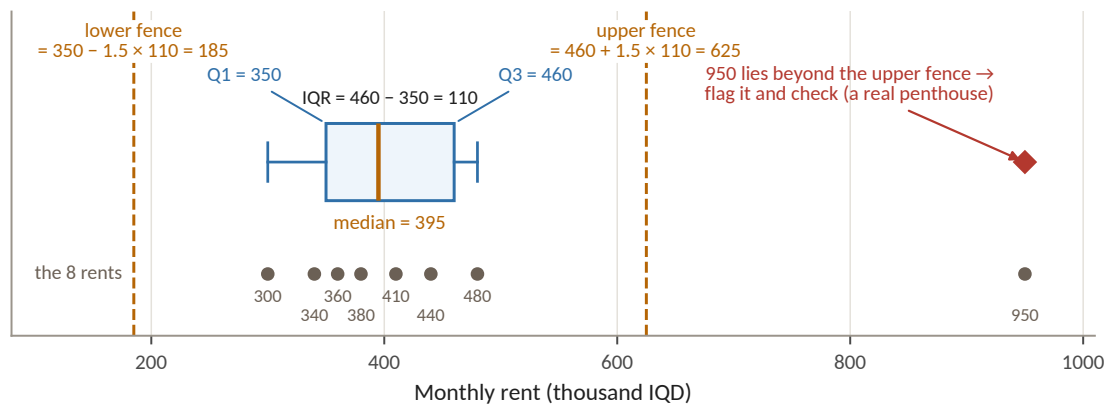


Figure 4.5 – Box plot of the eight rents with the IQR fences drawn in. The box spans Q_1 to Q_3 , the whiskers reach the last values inside the fences, and the rent of 950 is drawn alone beyond the upper fence.

WHAT TO OBSERVE

The box covers the middle half of the rents, 350 to 460, with the median 395 inside it; the whiskers stop at 300 and 480, the most extreme rents inside the fences; the lone point at 950 is what a box plot calls an outlier. The team phones the landlord: listing 8 is a furnished penthouse with a roof garden, so the 950 is genuine and is kept, with a note in the log.

4.4 Duplicate Records

Listing 9 in Table 4.1 repeats listing 3 in every cell, right down to the rent. The landlord's phone showed a spinning wheel, so he pressed “submit” again, and the website saved two rows for one apartment.

WORD TO KNOW

Duplicate record — A row that describes the same example as another row. An *exact* duplicate matches in every column; a *near* duplicate matches in every column except one that is generated automatically, such as an ID or a time stamp, or differs only by spelling or spacing.

A duplicate does quiet damage. The model treats the repeated apartment as two independent pieces of evidence: with the duplicate present, listing 3 supplies 2 of 9 rows, about 22% of all the evidence, instead of 1 of 8, or 12.5%. Worse, if duplicates survive until the data is split, the same apartment can land in both the training set and the test set, and the model is then tested on a row it has already memorized — a leak that Chapter 5 returns to.

Exact duplicates are easy to find: sort the rows, or let software compare every column (`drop_duplicates()` in pandas). Near duplicates take more thought: two rows that share a landlord, a street and a size but differ in the time stamp are probably one apartment posted twice, while identical features with different IDs may be two real apartments in the same building. Use an identifying column as the key, check a few suspected pairs by hand, and do it before the train/test split. Table 4.2 sums up the cleaning stage.

Table 4.2 – Before-and-after summary of the cleaning stage for the website's table

Check	Before	After	Action taken and logged
Rows in the table	9	8	Listing 9 dropped (exact copy of listing 3)
Blank sizes	1	0	Listing 5 filled with the training median, 75 m ²
Spellings of city	4	3	“erbil” merged into “Erbil” (listing 6)
Rents outside the IQR fences	1	1	Listing 8 checked with the landlord: genuine penthouse, kept
New sizes with $ z \geq 3$	1	0	Listing F corrected from 800 to 80 m ² after confirmation

4.5 Encoding Categorical Variables

Underneath everything, a model only does arithmetic. It can multiply a size by a weight, but it cannot multiply the word “Duhok” by anything. Every categorical column must become numbers before training, and how depends on whether the categories have a real order.

WORDS TO KNOW

Encoding — Turning a categorical column into numbers a model can use.

Ordinal encoding — Replacing ordered categories by ordered whole numbers, such as Needs Repair = 0, Fair = 1, Good = 2, Excellent = 3. Correct only when the order is real (condition, pain level, shirt size).

One-hot encoding — Replacing an unordered (nominal) categorical column — city, blood type, day type — by one new 0/1 column per category. In each row exactly one of the new columns is 1 (“hot”) and the rest are 0.

The Condition column is ordinal: Excellent really is better than Good, which really is better than Fair, so ordered codes 0, 1, 2, 3 keep that information. The City column is different. Numbering the cities Duhok = 1, Erbil = 2, Sulaymaniyah = 3 would tell the model that Sulaymaniyah is “three times Duhok” and that Erbil lies halfway between them — nonsense that many models would happily learn. One-hot encoding avoids this with a rule applied once per category: for each city seen in the training data, create a column that is 1 when the row’s city matches and 0 otherwise.

EQUATION: ONE-HOT ENCODING RULE

$$x_c = \begin{cases} 1 & \text{if the row's category equals } c \\ 0 & \text{otherwise} \end{cases} \quad \text{for every category } c \text{ seen in the training data}$$

In words: Make one new 0/1 column for every category in the training list. In each row, put a 1 in the column of that row’s own category and a 0 in every other new column.

When it is used: For any categorical column whose categories have no real order: cities, colors, product types.

Symbol	Meaning	Unit or type
c	one category from the training list	a name
x_c	the new 0/1 column for category c	0 or 1

The new columns of one row always add up to exactly 1. scikit-learn lists the categories in alphabetical order, so the City columns come out as Duhok, Erbil, Sulaymaniyah.

WORKED EXAMPLE 4.4 — One-Hot Encode the City Column for Four Listings

Listings 1 to 4 have the cities Erbil, Duhok, Erbil and Sulaymaniyah. One-hot encode the City column using the alphabetical column order City_Duhok, City_Erbil, City_Sulaymaniyah.

1. List the distinct training cities in alphabetical order: Duhok, Erbil, Sulaymaniyah — three categories, so three new columns.
2. Listing 1 is in Erbil: City_Duhok = 0, City_Erbil = 1, City_Sulaymaniyah = 0.
3. Listing 2 is in Duhok: 1, 0, 0. Listing 3 is in Erbil, like listing 1: 0, 1, 0.
4. Listing 4 is in Sulaymaniyah: 0, 0, 1.

Answer: Each listing gets exactly one 1, in the column of its own city, and 0 elsewhere (Table 4.3). The model can now use the city without ever being told that one city outranks another.

Table 4.3 – One-hot encoding of the City column for listings 1 to 4

Listing	City (original)	City_Duhok	City_Erbil	City_Sulaymaniyah
1	Erbil	0	1	0
2	Duhok	1	0	0
3	Erbil	0	1	0
4	Sulaymaniyah	0	0	1

Figure 4.6 shows the same transformation as a picture: one word column goes in, three number columns come out.

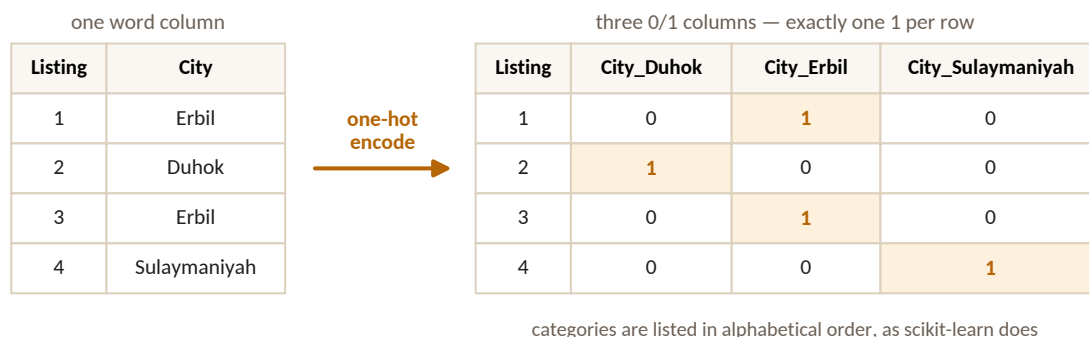


Figure 4.6 – One-hot encoding turns one word column into one 0/1 column per category. The shaded cells are the single 1 in each row.

WHAT TO OBSERVE

Row by row, the 1 moves to whichever column names the listing's city. Column by column, each new column is a yes/no question — “is this listing in Duhok?” — something a model can multiply by a weight.

WORD TO KNOW

Dummy-variable trap — Redundancy caused by keeping every one-hot column of a category together with a linear model's intercept term. Because the three city columns always add up to 1, any one of them can be computed from the other two ($\text{City_Erbil} = 1 - \text{City_Duhok} - \text{City_Sulaymaniyah}$), so a linear model (Chapter 8) has two ways to express the same thing and its weights become ambiguous. The fix is to drop one column (scikit-learn: `drop="first"`); tree-based models are not affected. A related point: the category list is learned from the training rows, so a test listing from an unseen city should be encoded as all zeros (`handle_unknown="ignore"`) rather than crash the program.

4.6 Feature Scaling: Standardization and Min-Max Normalization

The cleaned table holds numerical columns with very different ranges: sizes from 60 to 104 square meters, ages from 2 to 42 years, rents from 300 to 950 thousand IQD. Chapter 1 met this trap with melon weights in grams: whichever column uses the biggest numbers quietly takes over any calculation that adds features together, such as the distances of nearest-neighbor classifiers (Chapter 11) or the weighted sums of gradient descent (Chapter 9). Figure 4.7 shows the problem and the cure.

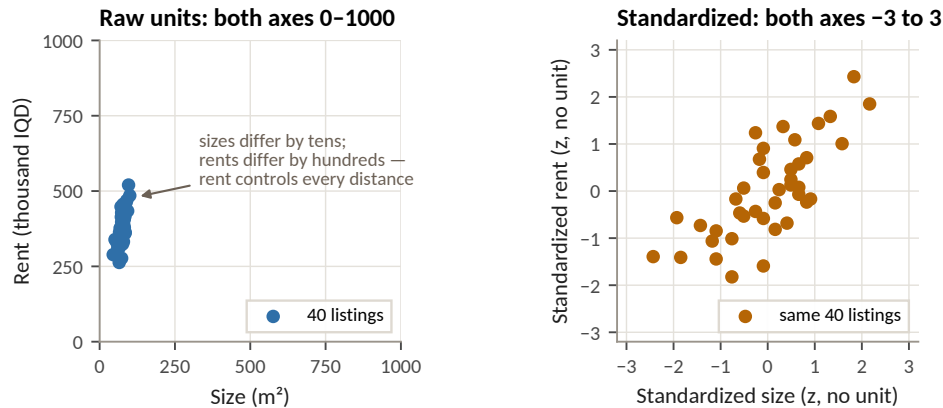


Figure 4.7 – Forty listings plotted twice. Left: raw units, with both axes drawn from 0 to 1000 so that one step on the page means the same number on each axis. Right: the same listings after standardizing both columns.

WHAT TO OBSERVE

In the left panel the points form a thin vertical stripe: sizes differ by tens, rents by hundreds, so a distance between two listings would be, in effect, a distance in rent alone. In the right panel both columns run from about -3 to 3 , and the relationship between size and rent finally shows.

WORDS TO KNOW

Feature scaling — Rescaling a numerical column so that different columns have comparable ranges. The two common methods are standardization and min-max normalization.

Standardization — Subtracting the training mean and dividing by the training standard deviation, so that the column is centered on 0 and most values fall between about -3 and 3 .

Min-max normalization — Subtracting the training minimum and dividing by the training range, so that the training values run from exactly 0 to exactly 1.

Standardization is the z-score formula of Lesson 4.3 given a new job. There it measured how unusual one value was; here it is applied to every value of a column — training, test and future rows alike — always with the mean and standard deviation of the *training* column, so that the whole column is rewritten in units of “standard deviations from the training mean”:

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

After standardizing, the training column itself has mean 0 and standard deviation 1; scikit-learn’s `StandardScaler` does exactly this. Standardize before any model that compares or adds features on different scales — distance-based models, regression trained by gradient descent, support vector machines, principal component analysis; tree-based models do not need it.

WORKED EXAMPLE 4.5 — Standardize a Large and a Small Apartment

The training sizes have mean 80 m^2 and standard deviation 15 m^2 . Two new apartments arrive, one of 110 m^2 and one of 50 m^2 . Standardize both sizes.

1. Identify the training numbers: $\mu_{\text{train}} = 80$, $\sigma_{\text{train}} = 15$.
2. Large apartment: subtract the mean, $110 - 80 = 30$; divide by the standard deviation, $30 \div 15 = 2$.
3. Small apartment: subtract the mean, $50 - 80 = -30$; divide, $-30 \div 15 = -2$.

Answer: The standardized sizes are 2 and -2 . The two apartments sit exactly the same distance from the training average, two standard deviations, in opposite directions — something the raw numbers 110 and 50 did not show at a glance.

Min-max normalization squeezes a column into the range from 0 to 1 instead; it is natural when a feature has hard limits, such as a percentage or a score out of 100.

EQUATION: MIN-MAX NORMALIZATION

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

In words: Subtract the smallest training value, then divide by the training range (largest minus smallest). The smallest training value becomes 0, the largest 1, and everything else lands in between.

When it is used: When a model or a later step needs every feature inside a fixed interval, or when the feature has natural hard limits and no wild outliers.

Symbol	Meaning	Unit or type
x'	the min-max scaled value	no unit; 0 to 1 for training rows
x	the original value	unit of the feature
$x_{\min}$	smallest value in the training column	unit of the feature
$x_{\max}$	largest value in the training column	unit of the feature

A new value outside the training range simply lands outside 0 to 1. One outlier in the training column squeezes every other value into a corner of the interval, so standardization is preferred when outliers are present.

WORKED EXAMPLE 4.6 — Min-Max Scale Five Building Ages

The first five training listings have building ages of 12, 6, 22, 2 and 42 years (the other three ages, 15, 8 and 30, lie inside the same range). Min-max scale the five ages.

1. Find the training minimum and maximum: $x_{\min} = 2$, $x_{\max} = 42$, so the range is $42 - 2 = 40$.
2. Listing 1: $(12 - 2) \div 40 = 10 \div 40 = 0.25$.
3. Listing 2: $(6 - 2) \div 40 = 4 \div 40 = 0.1$.
4. Listing 3: $(22 - 2) \div 40 = 20 \div 40 = 0.5$.
5. Listing 4: $(2 - 2) \div 40 = 0$. Listing 5: $(42 - 2) \div 40 = 40 \div 40 = 1$.

Answer: The scaled ages are 0.25, 0.1, 0.5, 0 and 1: the newest building becomes exactly 0, the oldest exactly 1, and a 22-year-old building lands halfway.

Table 4.4 applies both rules to the whole training table, and Figure 4.8 shows that the choice between them is mostly a matter of convenience.

Table 4.4 – The eight training listings with size standardized and building age min-max scaled

Listing	Size (m ²)	Standardized size, $(x - 80)/15$	Age (years)	Min-max age, $(x - 2)/40$
1	60	−1.33	12	0.25
2	65	−1.00	6	0.10
3	71	−0.60	22	0.50
4	75	−0.33	2	0.00
5	75 (filled)	−0.33	42	1.00
6	92	0.80	15	0.325
7	98	1.20	8	0.15
8	104	1.60	30	0.70

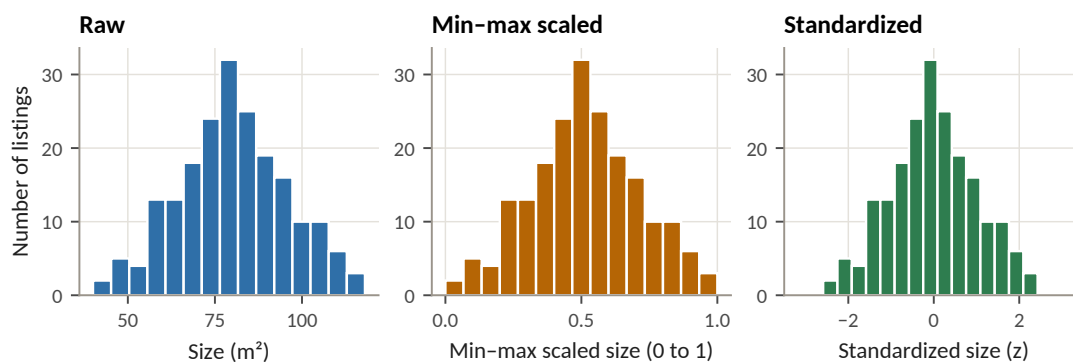


Figure 4.8 – One column of 200 sizes drawn three times: in raw square meters, after min-max scaling, and after standardization. Only the numbers on the horizontal axis change.

WHAT TO OBSERVE

The three histograms have exactly the same shape: scaling slides and stretches the axis but never reorders the listings or changes the pattern. Scale only the numerical feature columns — one-hot columns are already 0/1, and the target stays in its real units.

4.7 Fit on Training Data Only: Pipelines and Leakage

The website’s team has now learned several rules from the training rows: fill a blank size with 75, encode the cities alphabetically, standardize size with mean 80 and standard deviation 15, min-max scale age with minimum 2 and maximum 42. Two things can still go wrong. A teammate could compute a fresh mean from the test rows “to be more accurate”, which leaks test information into the preparation and makes the final score dishonest. Or the team could apply the rules by hand and forget one when a new listing arrives.

WORDS TO KNOW

Fit and transform — The two halves of every preprocessing step: *fit* learns the rule from the training rows (the median, the category list, the mean and standard deviation); *transform* applies an already-learned rule to any rows — training, test or brand-new.

Pipeline — An ordered chain of preprocessing steps and a model, bundled into one object. Fitting it fits every step in order on the training rows only; using it afterward transforms new rows with the learned rules and then predicts.

Figure 4.9 shows the rule this whole chapter has been building toward: fitting happens once, on the training rows; the test rows, the validation rows of Chapter 5 and every future listing are only ever *transformed*, with the numbers the training rows produced.

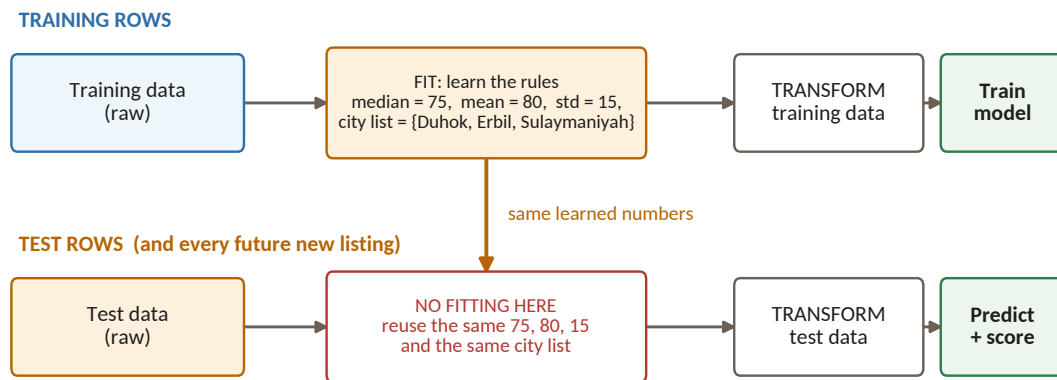


Figure 4.9 – Fit on the training rows, then transform the training rows and the test rows with the same learned numbers. Nothing is ever fitted on the test rows.

WHAT TO OBSERVE

The orange box is the only place where numbers are learned, and the arrow carrying “same learned numbers” downward is the whole point: the test data never gets a median, a mean or a category list of its own. If it did, the test score would describe a model that had peeked at the test set.

WORKED EXAMPLE 4.7 — Run a Trained Pipeline on One New Apartment

A pipeline fitted on the eight training listings has learned: fill a blank size with 75; standardize size with mean 80 and standard deviation 15; min-max scale age with minimum 2 and maximum 42; one-hot encode City as Duhok, Erbil, Sulaymaniyah; code Condition as Needs Repair = 0, Fair = 1, Good = 2, Excellent = 3. A new listing arrives: 110 m², 22 years old, Duhok, Good. Turn it into the row of numbers the model receives.

1. Size is present, so no filling is needed. Standardize it: $(110 - 80) \div 15 = 30 \div 15 = 2$.
2. Min-max scale the age: $(22 - 2) \div 40 = 20 \div 40 = 0.5$.
3. One-hot encode the city: Duhok gives City_Duhok = 1, City_Erbil = 0, City_Sulaymaniyah = 0.
4. Encode the condition: Good = 2.
5. Assemble the row in the pipeline’s column order: [2, 0.5, 1, 0, 0, 2].

Answer: The raw listing becomes the six numbers 2, 0.5, 1, 0, 0, 2, computed with the training rules and nothing else. No step could be forgotten, because the pipeline carries all of them together.

COMMON MISTAKE

X Wrong: Filling blanks, computing the scaler’s mean or listing the categories on the complete dataset first, and splitting into training and test sets afterward.

✓ Right: Split first. Fit every preprocessing step on the training rows only, then transform the test rows with the learned rules. A pipeline built with Pipeline and ColumnTransformer does this automatically.

COMMON MISTAKES AND MISCONCEPTIONS

- Treating a blank cell as zero instead of as “unknown”.
- Deleting an unusual value without checking whether it is genuine, or judging it with a mean and standard deviation computed from the same few values, so that it hides itself.
- Ignoring duplicate rows, which double-count one example and can leak it into both the training and the test set, or numbering unordered categories 1, 2, 3, so that the model invents an order.
- Learning a fill value, a category list or a scaling rule from the complete dataset instead of the training rows.
- Applying preprocessing by hand and forgetting a step when the model is used on new data.

Practical Activities

ACTIVITY (PAPER) — Clean a Ten-Row Table by Hand

A second rental website sends the ten listings in Table 4.5 (rents in thousand IQD). Clean the table on paper, in this order, with one log line for every change.

1. Duplicates: find the two rows that repeat another row in every column and cross them out.
2. Outliers: sort the remaining known sizes, find Q_1 , Q_3 , the IQR and the fences, and flag any size outside them. Say what the true value probably is and what to do before changing it.
3. Missing values: fill the blank size with the median of the known, corrected sizes. Explain why the blank rent (the target) cannot be filled the same way, and what to do with that row.
4. Encoding: one-hot encode the City column, listing the categories in alphabetical order.
5. Scaling: standardize the size of listing 110 with the mean and standard deviation of the cleaned sizes, and say in one sentence what the result means.

You need: Paper, pencil and a calculator

Table 4.5 – Ten raw listings for the paper activity (illustrative example data)

ID	Size (m ²)	City	Rent	ID	Size (m ²)	City	Rent
101	55	Zakho	280	106	88	Duhok	420
102	70	Duhok	350	107	650	Zakho	310
103	62	Zakho	300	108	75	Akre	(blank)
104	70	Duhok	350	109	62	Zakho	300
105	(blank)	Akre	260	110	95	Duhok	470

ACTIVITY (PYTHON) — The Same Rules With scikit-learn’s Pipeline and ColumnTransformer

Fit this chapter’s rules on the eight cleaned listings stored in `datasets/ch04_listings.csv`.

Python · pandas · scikit-learn — part 1: fit the rules on the training rows

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
train = pd.read_csv("datasets/ch04_listings.csv") # 8 cleaned listings (size 5 blank)
size_steps = Pipeline([("fill", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler())])
order = [
    ["Needs Repair", "Fair", "Good", "Excellent"]
]
prep = ColumnTransformer(
    [
        ("size", size_steps, ["size"]),
        ("age", MinMaxScaler(), ["age"]),
        ("city", OneHotEncoder(), ["city"]),
        ("cond", OrdinalEncoder(categories=order), ["condition"])
    ]
)
prep.fit(train) # FIT: rules learned from training rows only
size_fit = prep.named_transformers_["size"]
print(
    "fill value:", size_fit["fill"].statistics_, " mean:", size_fit["scale"].mean_,
    " std:", size_fit["scale"].scale_
)
```

Output

```
fill value: [75.] mean: [80.] std: [15.]
```

The `ColumnTransformer` sends each column to its own rule: size passes through a small Pipeline that fills blanks and then standardizes; age is min-max scaled; city is one-hot encoded (alphabetically); condition gets ordered codes. One call, `prep.fit(train)`, learns every rule from the eight training rows, and the printed fill value 75, mean 80 and standard deviation 15 are exactly the hand results. Now transform two new listings with the same fitted object.

Python · pandas · scikit-learn — part 2: transform new rows with the learned rules

```
new = pd.DataFrame(
    {
        "size": [110, 50],
        "age": [22, 2],
        "city": ["Duhok", "Erbil"],
        "condition": ["Good", "Fair"]
    }
)
print(prepare.transform(new)) # TRANSFORM: same rules, new rows
```

Output

```
[ [ 2.  0.5  1.  0.  0.  2. ]
  [-2.  0.  0.  1.  0.  1. ]]
```

The first row, 2, 0.5, 1, 0, 0, 2, matches Worked Example 4.7; the second gives the -2 of Worked Example 4.5. Add a model as a last step, `Pipeline([("prep", prep), ("model", LinearRegression())])`, and fit learns every rule and trains the model in one call, while predict preprocesses and predicts in one call.

You need: A computer with Python, pandas and scikit-learn installed (see Appendix B)

CASE STUDY — Cleaning Building-Energy Meter Data at a University in Duhok

A university in Duhok installed smart electricity meters on its main classroom building two years ago, partly to track ordinary demand and partly to see how much of it the new rooftop solar panels cover on a sunny day. The facilities office wants to prepare the hourly readings for a forecasting model that could

predict tomorrow's demand and help decide how much power to draw from the grid rather than from the building's own panels and batteries. First, someone has to make the raw readings trustworthy. (The details are an illustrative example, not a report on a real installation.)

The first look sorts the columns: hour of day and outdoor temperature are numerical; day type ("Class Day", "Weekend", "Holiday") is categorical; a building-occupancy rating of Low, Medium or High is ordinal; the engineer's free-text maintenance log is text, set aside for now after a check for private details such as contractors' phone numbers. Then the problems appear, and every one of them is a problem this chapter has met. A handful of hourly readings show exactly 0 kilowatt-hours in the middle of a normal class day — almost certainly a communication failure, not real zero demand — so those cells are marked as missing, filled with the training median for that hour, and an indicator column records which hours were affected. The old temperature sensor reported Fahrenheit for six months, so those readings are converted to Celsius before any statistic is computed. Some hours appear twice because the meter re-sent a reading after a lost connection; the second copy is dropped, using the time stamp as the key. The spring holiday appears as "Newroz", "Nawroz" and "New Roz", merged into one category. The IQR rule flags a few very high readings; the engineer confirms they fall on exam days when every hall was air-conditioned at once, so they are kept. Every decision goes into the data-quality log.

Then the rest of the toolkit follows. Day type becomes one-hot columns, because there is no order between a class day and a holiday; occupancy keeps ordered codes (Low = 0, Medium = 1, High = 2), because that order is real; hour of day and temperature are standardized with a mean and standard deviation learned only from the training months; and every step is wrapped in one pipeline. The team splits the data by time, training on the oldest months and testing on the most recent ones, so that no rule ever sees the future — the order-sensitive split that Chapter 2 recommended. Even before a model is trained, the cleaned table shows that midday demand on sunny class days is visibly lower than on cloudy ones at the same hour and occupancy, a first hint that the panels really are covering part of the building's own use. The director also insists on one more habit: rerunning the whole cleaning pipeline on a fresh export every few months, because new sensors get installed, old ones get replaced, and a spelling nobody has seen yet can always appear in next winter's holiday schedule. Careful preprocessing is not a detour on the way to "the real work" of modeling; a brilliant model trained on mixed units, unmerged spellings or silently wrong zeros will confidently produce a wrong answer, without ever announcing that anything is wrong.

CHAPTER SUMMARY

- Raw data arrives with blanks, typing mistakes, inconsistent spellings and duplicates; preprocessing cleans, encodes and scales it before a model sees it, and every change goes into a data-quality log.
- A missing value means "unknown", never zero. Drop an almost-empty column or row, fill scattered blanks with the training median (or most frequent category), and add an indicator column when blankness itself may matter.
- A z-score, $z = (x - \mu)/\sigma$, counts standard deviations from a trusted reference; values with $|z| \geq 3$ deserve a check. The reference must be large and clean, because an outlier inflates a standard deviation computed from the same few values.
- The IQR rule flags values beyond $Q_1 - 1.5 \text{ IQR}$ or $Q_3 + 1.5 \text{ IQR}$; it resists outliers and is what box plots draw. A flag is a question, not a verdict: check, then correct or keep, and log the decision.
- Duplicate rows double-count one example and can leak it into both the training and the test set; remove them before splitting.
- Ordinal categories become ordered codes; unordered categories become one-hot 0/1 columns (drop one of them for linear models with an intercept).
- Standardization, $x' = (x - \mu_{\text{train}})/\sigma_{\text{train}}$, centers a column on 0 with spread 1; min-max normalization, $x' = (x - x_{\min})/(x_{\max} - x_{\min})$, maps the training range to 0–1. Neither changes the shape of the data.

- Every preprocessing rule is fitted on the training rows only and reused, unchanged, on the test rows and every future row; a pipeline bundles the steps and the model so that this always happens automatically.
- Every change made during cleaning is written in a data-quality log, and the whole cleaning pipeline is rerun whenever fresh data arrives.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain, in your own words, what preprocessing means and why it happens before training.

Problem 2 [Basic] Find the median of five delivery times, in minutes: 22, 18, 35, 20, 26. Then find the median of six waiting times: 9, 14, 7, 20, 11, 16.

Problem 3 [Basic] Four fruit-stand records have the Fruit values apple, banana, apple, cherry. One-hot encode this column (categories in alphabetical order), writing the 0/1 values for each record.

Problem 4 [Basic] A hospital dataset in Chengdu has the columns patient age, blood type, pain level (none, mild, moderate, severe) and the doctor's written notes. For each column, say whether it needs no encoding, ordinal encoding, one-hot encoding or special handling as text.

Problem 5 [Standard] A feature has a training mean of 50 and standard deviation of 5. Standardize a new value of 62, showing every step, and say whether the $|z| \geq 3$ rule would flag it.

Problem 6 [Standard] A feature has a training mean of 200 and standard deviation of 25. Standardize a new value of 150, showing every step, and say whether it is above or below average.

Problem 7 [Standard] Eight temperatures from a sensor at a factory in Da Nang, in °C, are 19, 21, 20, 22, 18, 20, 95 and 23. Sort them, find Q_1 , Q_3 , the IQR and both fences, and state which readings are flagged.

Problem 8 [Standard] Min-max scale the four values 10, 20, 35 and 50 using their own minimum and maximum, showing every step. Then give the scaled value of a new value of 60 without changing the minimum or maximum.

Problem 9 [Standard] Explain why fitting a scaler or an imputer on the complete dataset before splitting counts as data leakage, and how a pipeline prevents it.

Problem 10 [Challenge] Seven recorded temperatures, in °C, are 19, 21, 20, 22, 18, 20 and 95. (a) Identify the likely outlier and find the median of the other six. (b) Compute the mean and standard deviation (divide by n) of all seven values and show that the z-score of 95 is below 3. (c) Why did the z-score rule miss this outlier, and which rule would have caught it?

Problem 11 [Challenge] A teammate standardizes sizes with the mean (82 m²) and standard deviation (16 m²) of the training and test rows combined; you use the training rows only (mean 80 m², standard deviation 15 m²). For a test apartment of 110 m², compute both standardized values, say which one is legitimate, and explain why the difference matters more than it looks.

Problem 12 [Challenge] The 200-listing table of Figure 4.3 had 28 blank sizes. Suppose landlords of the cheapest apartments most often skipped the size box. (a) What would median imputation alone get wrong for a model that predicts rent? (b) How does an indicator column help? (c) Suggest a better fill rule that uses another column.

Training, Validation, Testing, and Cross-Validation

How to measure a model honestly with data it has never seen

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a score measured on the training rows is too optimistic, and what it means for a model to generalize.
- Split a dataset into training, validation and test sets, and calculate the size of each set by hand.
- Choose between a random, a stratified, a time-based and a group split for a new project.
- Carry out k -fold cross-validation by hand on a small dataset and calculate the mean score.
- Report a cross-validation score with its spread (mean $\pm$ standard deviation) and its standard error, and compare two models fairly.
- Recognize the three ways a split can leak information: group leakage, time leakage and test-set peeking.

Introduction

Imagine a student in Erbil preparing for a history exam. She finds last year's exam paper and memorizes every answer on it, word for word. If the teacher reuses the same paper, she scores 100%. But if the teacher writes new questions on the same topics, her score will show what she actually understood. A careful teacher never lets the practice paper and the real paper be the same paper.

A machine learning model is in the same position: the rows it studied are its practice paper, and a score on them tells you how well the model remembers, not how it will do on the patients or machines that arrive next month. This chapter shows how to keep the real exam honest: cut the data into sets with one job each, get a reliable score from a small dataset with cross-validation, report that score with its spread, and spot the quiet ways information leaks from the exam back into the practice.

5.1 Why One Number From the Training Data Lies

A tailor in Erbil measures a customer, cuts the cloth, and sews a suit. Of course the suit fits that customer. The real question is whether the tailor's method produces suits that fit the *next* customers. A model was cut to fit its training rows; what matters is how it fits rows it has never seen.

WORDS TO KNOW

Generalization — A model's ability to make good predictions on new examples that were not in its training data. Generalization is the whole point of training a model.

Generalization gap — The difference between a model's score on its training rows and its score on new, unseen rows. A large gap means the model has memorized more than it has learned.

Why is the training score too optimistic? During training, a flexible model does two things at once: it learns real patterns (busy hours mean long waits), and it memorizes accidents of the particular rows it saw (the visit at 10:15 on a Tuesday happened to be short). Both raise the score on the training rows; only the first helps with new rows. The training score therefore mixes real skill with memory, and that one number cannot tell the two apart. Figure 5.1 shows the effect with a simulation: one flexible classifier (a decision tree, met properly in Chapter 13) was trained on clinic-visit data of different sizes, then scored on its training rows and on 1,000 unseen rows.

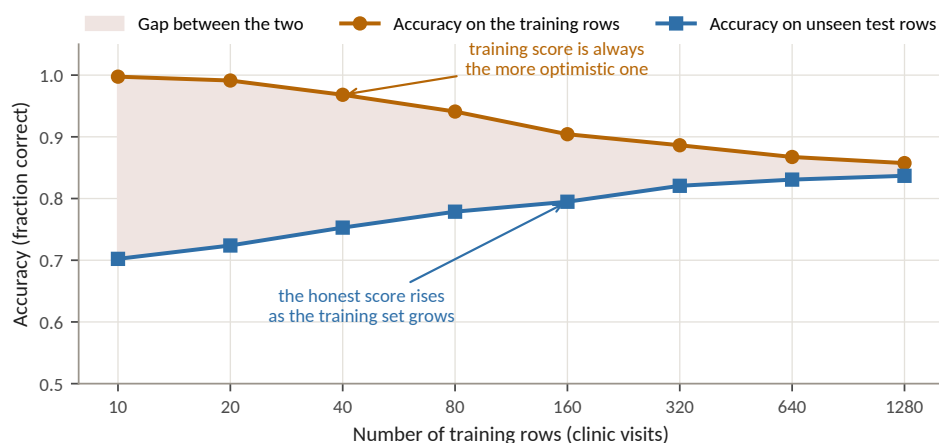


Figure 5.1 – Training accuracy and unseen-data accuracy of the same classifier as the training set grows. Each point is an average over 40 random training sets of that size.

WHAT TO OBSERVE

With only 10 training rows the training score is a perfect 1.00 while the honest score is about 0.70, a gap of 0.30. As the training set grows, the honest score rises and the gap shrinks, but at every size the orange line sits above the blue one. The training score is never the number to report; the rest of this chapter is about how to hold rows back, even from a small dataset, without wasting them.

5.2 The Three Sets: Training, Validation, Test

Return to the clinic in Sulaymaniyah from Chapter 2. The team has 1,200 past visits and a new question: at check-in, will this visit turn into a long wait of more than 30 minutes? They want to try three candidate models, each with settings to choose. If they compare every candidate on the test set and keep whichever scores best there, the test set has been used to make choices; it is no longer data the finished model has never “seen”, and its score is no longer an honest final check. The solution is to hold back two separate groups of rows, one for choosing and one for the final check.

WORDS TO KNOW

Holdout set — Any group of rows kept out of training so that the model can be scored on data it did not learn from. The validation set and the test set are both holdout sets.

Validation set — A holdout set used to compare candidate models and to choose their settings. Its score may be looked at as often as needed, because it is part of building the model.

The training set fits the model. The validation set is used again and again while the team compares candidates and adjusts settings. The test set, defined in Chapter 1, stays locked until every decision has been made, then is opened once to produce the number that gets reported (Figure 5.2).

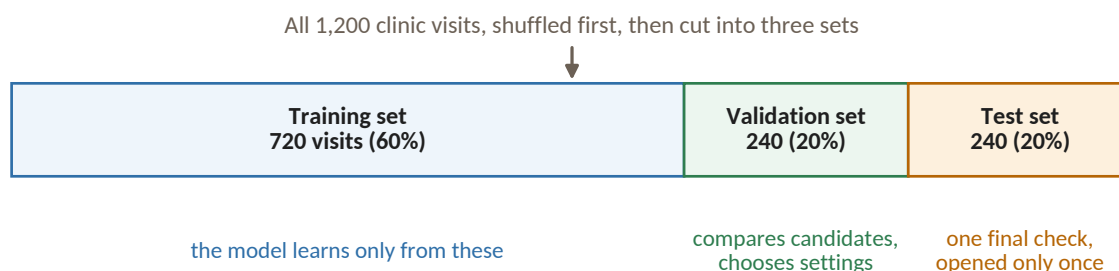


Figure 5.2 – The 1,200 clinic visits cut into training, validation and test sets in a 60/20/20 split. Each row belongs to exactly one set.

WHAT TO OBSERVE

Three sets, three jobs. The training set is the largest because learning needs data; the validation set is reused freely; the test set is opened once. If you ever change a model because of its test score, the test set has quietly become a second validation set.

To cut the data, multiply the total number of rows by each set's fraction. Give the test set whatever is left, so that rounding never loses or invents a row.

EQUATION: SPLIT SIZES

$$n_{\text{train}} = f_{\text{train}} n, \quad n_{\text{val}} = f_{\text{val}} n, \quad n_{\text{test}} = n - n_{\text{train}} - n_{\text{val}}$$

In words: Multiply the total number of rows by the training fraction and by the validation fraction; the test set gets all the remaining rows, so the three counts add up to the total exactly.

When it is used: Whenever a dataset is cut into sets, to work out how many rows land in each set before any training starts.

Symbol	Meaning	Unit or type
n	total number of rows in the dataset	count
$f_{\text{train}}, f_{\text{val}}$	fraction of rows for training and for validation	number between 0 and 1
$n_{\text{train}}, n_{\text{val}}, n_{\text{test}}$	rows in the training, validation and test sets	count

If a fraction gives a decimal, round down to whole rows; the test set absorbs the difference. With only a training set and a test set, drop the middle term.

WORKED EXAMPLE 5.1 — Split 1,200 Clinic Visits 60/20/20

The clinic team decides on a 60% training, 20% validation, 20% test split of its 1,200 past visits. How many visits go into each set?

1. Find the training visits: $0.60 \times 1,200 = 720$.
2. Find the validation visits: $0.20 \times 1,200 = 240$.
3. Give the test set what is left: $1,200 - 720 - 240 = 240$.
4. Check the total: $720 + 240 + 240 = 1,200$, which matches the original number of visits.

Answer: 720 visits train the model, 240 visits are used to compare candidates and choose settings, and 240 visits stay locked for one final check. Every visit belongs to exactly one set.

Table 5.1 collects the counts and the job of each set. The fractions are a habit, not a law: 60/20/20, 70/15/15 and 80/10/10 are all common. With millions of rows even 1% is thousands of rows, so the holdout sets can be smaller fractions. With only a few hundred rows a separate validation set costs too many training rows, and Lesson 5.4 shows a better way.

Table 5.1 – The 1,200 clinic visits after a 60/20/20 split

Set	Fraction	Calculation	Rows	Used for
Training	60%	$0.60 \times 1,200$	720	Fitting each candidate model
Validation	20%	$0.20 \times 1,200$	240	Comparing candidates and choosing settings
Test	20%	$1,200 - 720 - 240$	240	One final, honest check of the chosen model
Total	100%		1,200	

COMMON MISTAKE

✗ Wrong: Comparing five candidate models on the test set, keeping the best one, and reporting its test score as the honest final result.

✓ Right: Compare candidates on the validation set (or with cross-validation). Open the test set once, after the final model is chosen, and report that single number even if it is a little lower.

5.3 Random, Stratified and Time-Based Splits

The counts tell you *how many* rows go into each set. The next question is *which* rows. The simplest answer is to shuffle the rows like a deck of cards and cut, so that every row has the same chance of landing in every set.

WORDS TO KNOW

Random split — A split made by shuffling all the rows and cutting the shuffled list at the chosen counts. Every row has the same chance of landing in each set.

Stratified split — A random split made separately inside each class, so that every set contains the same share of each class as the whole dataset.

Time-based split — A split made by date instead of by chance: the oldest rows train the model, and the newest rows validate and test it, so the model is always scored on data from after the period it learned from.

A random split works well when the rows are independent and no class is rare. Now suppose the clinic also wants to predict which booked patients will not turn up, so that a receptionist can send a friendly reminder. Of 200 booked appointments, only 30 were no-shows, which is 15%. A random 20% test set of 40 appointments is *expected* to contain 6 no-shows, but chance can easily give it 2 or 10, and with 2 the test score for

the very class the project cares about rests on two people. A stratified split removes this luck: it cuts the 30 no-shows and the 170 kept appointments separately, by the same fraction.

WORKED EXAMPLE 5.2 — Stratified Split of 200 Appointments With 30 No-Shows

The 200 booked appointments contain 30 no-shows and 170 kept appointments. Make a stratified 80% training, 20% test split, and check that both sets keep the 15% no-show share.

1. Test no-shows: $0.20 \times 30 = 6$.
2. Test kept appointments: $0.20 \times 170 = 34$. The test set has $6 + 34 = 40$ rows.
3. Training no-shows: $30 - 6 = 24$.
4. Training kept appointments: $170 - 34 = 136$. The training set has $24 + 136 = 160$ rows.
5. Check the shares: whole data $30 \div 200 = 0.15$; training $24 \div 160 = 0.15$; test $6 \div 40 = 0.15$.

Answer: The training set has 160 rows (24 no-shows) and the test set has 40 rows (6 no-shows). Both sets carry exactly the 15% no-show share of the original data, so neither set is unusually easy or unusually hard.

Table 5.2 lists the counts, and Figure 5.3 compares the stratified split with an unlucky random one.

Table 5.2 – Stratified 80/20 split of the 200 booked appointments, counted by class

Set	No-shows	Kept	Total rows	No-show share
All appointments	30	170	200	15%
Training (80%)	24	136	160	15%
Test (20%)	6	34	40	15%

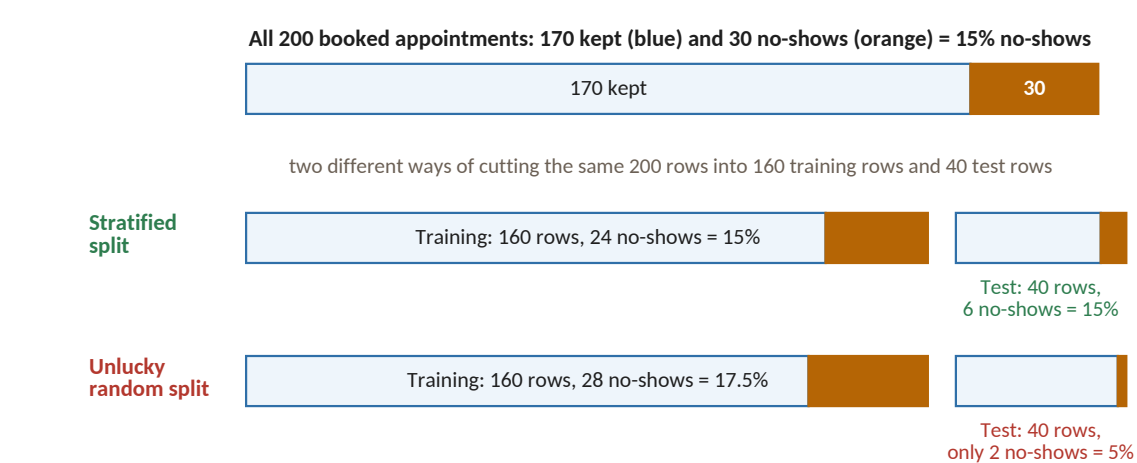


Figure 5.3 – A stratified split (middle) keeps the 15% no-show share in both sets. An unlucky random split (bottom) can leave the test set with only 2 no-shows.

WHAT TO OBSERVE

The orange blocks are the no-shows. In the stratified rows the orange share of every bar is the same 15%. In the unlucky random split the test bar has almost no orange: a model could score well there while never catching a single no-show. Stratifying costs nothing and removes this risk.

Some data has a third property that neither split respects: time order. The clinic's 1,200 visits were recorded over 12 months, about 100 per month, and the model will be used on *future* visits. If the rows are shuffled,

December visits land in the training set while March visits land in the test set, so the model is scored on the past after learning from the future. Real use never looks like that. A time-based split keeps the calendar intact: the oldest 8 months (800 visits) train, the next 2 months (200) validate, and the newest 2 months (200) test (Figure 5.4).

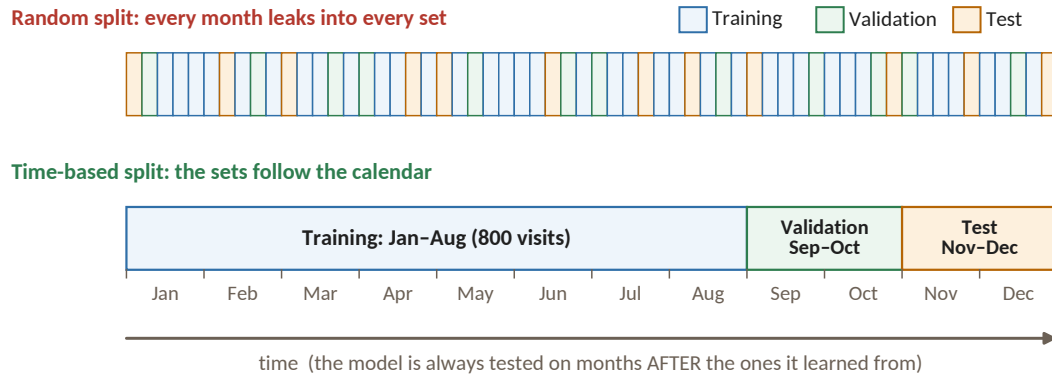


Figure 5.4 – Two ways to split 12 months of clinic visits. A random split (top) scatters every month across all three sets; a time-based split (bottom) trains on the past and tests on the future.

WHAT TO OBSERVE

In the top strip every month contains slivers of all three colors, so the training set already knows what December looked like. In the bottom strip the colors follow the arrow of time: the model never sees a visit from after the period it must predict, and the test months are the most recent ones, exactly the situation the deployed model will face.

Table 5.3 summarizes when to use each way of splitting, together with a fourth kind, the group split, which Lesson 5.7 explains.

Table 5.3 – Four ways to decide which rows go into which set

Strategy	How rows are assigned	Use it when	Watch out for
Random	Shuffle all rows, then cut at the chosen counts	Rows are independent and no class is rare	An unlucky shuffle can starve one set of a class
Stratified	Shuffle and cut each class separately, same fraction	Classification, especially with a rare class	Needs a label for every row
Time-based	Cut by date: oldest rows train, newest rows test	Rows have a time order and the model predicts the future	Seasons or trends differ between sets; that is the honest difficulty
Group	Keep every row of one patient, machine or farm on one side	Several rows belong to the same person or thing	Sets are slightly unequal; needs a group ID column

COMMON MISTAKE

✗ Wrong: Shuffling rows that have a time order (sensor readings, sales, visits) and reporting the random-split score, because it is usually higher.

✓ Right: Ask: “When this model is used, will it predict the future from the past?” If yes, split by time, accept the lower score, and trust it more.

5.4 k -Fold Cross-Validation

A validation score from one split is honest, but it is also partly luck. Take 100 clinic visits and the nearest-centroid classifier from Chapter 1, split them 80/20 twenty times with a different shuffle each time, and score the model on each held-out 20%. Nothing about the model changes between the twenty splits, yet the held-out accuracy ranges from 0.75 to 1.00. A team that happened to make the lucky split would believe the model is perfect; a team that made the unlucky one would think it mediocre. One split is one draw from a wide spread, and 20 held-out rows are too few to pin the score down.

Cross-validation reduces this luck by making *every* row take a turn as validation data. The rows are shuffled and cut into k equal pieces called folds, and k rounds are run: in each round one fold is held out, the model is trained on the other $k - 1$ folds, and the held-out fold is scored. After k rounds every row has been validated exactly once and trained on $k - 1$ times, and the k scores are averaged. The idea was worked out by the statisticians Mervyn Stone in 1974 and Seymour Geisser in 1975, and it is now the standard way to compare candidate models.

WORDS TO KNOW

Fold — One of the k equal-sized pieces into which the rows are cut for cross-validation.

k -fold cross-validation — A procedure that trains and scores a model k times, each time holding out a different fold for validation and training on the other $k - 1$ folds, then averages the k scores.

Figure 5.5 shows the five rounds of 5-fold cross-validation. The test set is not part of the picture at all: it stays locked, exactly as before. Cross-validation replaces the validation set, never the test set.

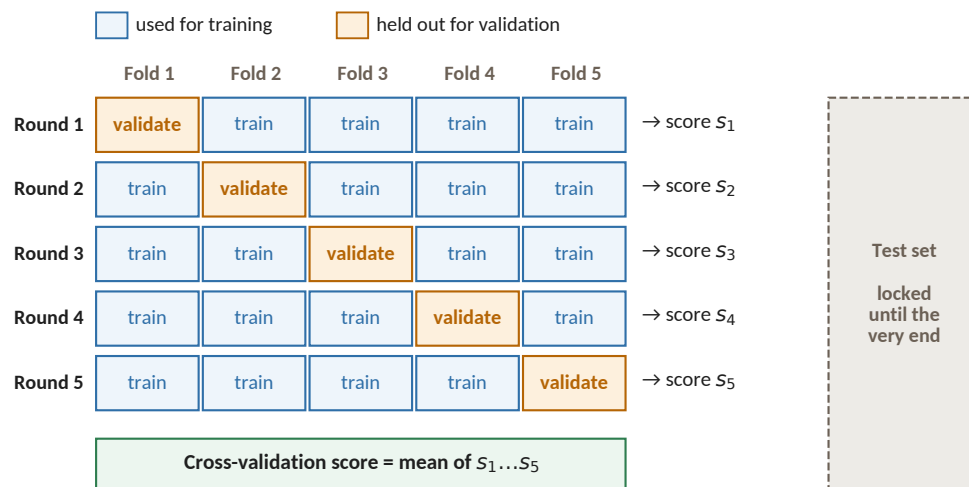


Figure 5.5 – Five-fold cross-validation. In each round one fold (orange) is held out for validation and the other four (blue) train the model. The test set stays locked throughout.

WHAT TO OBSERVE

Read the grid row by row: the orange cell moves one step to the right each round, so each fold is held out exactly once. Five models are trained, each on 80% of the rows, and the reported cross-validation score is the mean of the five scores.

WORKED EXAMPLE 5.3 — Assign 20 Rows to Five Folds

A small dataset of 20 clinic visits is to be cross-validated with $k = 5$. After shuffling, the rows are renumbered 1 to 20 in their new order. Which rows form each fold, how many rows does each round train on, and how many validation predictions are made altogether?

1. Find the fold size: $20 \div 5 = 4$ rows per fold.
2. Cut the shuffled list into blocks of four: fold 1 is rows 1–4, fold 2 is rows 5–8, fold 3 is rows 9–12, fold 4 is rows 13–16, and fold 5 is rows 17–20.
3. In each round, one fold of 4 rows is held out, so the model trains on $20 - 4 = 16$ rows.
4. Five rounds each produce 4 validation predictions: $5 \times 4 = 20$ predictions in total, one for every row.
5. Each row is held out once and used for training in the other $5 - 1 = 4$ rounds.

Answer: The five folds are rows 1–4, 5–8, 9–12, 13–16 and 17–20. Every round trains on 16 rows and validates on 4, and after five rounds all 20 rows have been validated exactly once, so no row is wasted.

Each round produces one score, and the cross-validation score is their average.

EQUATION: MEAN CROSS-VALIDATION SCORE

$$\bar{s} = \frac{1}{k} \sum_{j=1}^k s_j$$

In words: Add up the score of every fold and divide by the number of folds.

When it is used: After the k rounds of cross-validation, to turn k separate scores into the one number used to compare candidate models.

Symbol	Meaning	Unit or type
$\bar{s}$	mean cross-validation score	same unit as the score (a fraction for accuracy)
k	number of folds	count
s_j	score of the model on held-out fold j	same unit as the score

The score can be any performance measure: accuracy here, or MAE, recall or F_1 from Chapter 6. The folds are averaged the same way whatever the measure.

WORKED EXAMPLE 5.4 — Mean of Five Fold Accuracies

The clinic's 100 long-wait visits are cross-validated with $k = 5$, so each fold holds 20 visits. The nearest-centroid classifier gets 18, 16, 17, 18 and 16 of the 20 held-out visits right in the five rounds, which are accuracies of 0.90, 0.80, 0.85, 0.90 and 0.80. Find the mean cross-validation accuracy and the range of the fold scores.

1. Add the five scores: $0.90 + 0.80 + 0.85 + 0.90 + 0.80 = 4.25$.
2. Divide by $k = 5$: $4.25 \div 5 = 0.85$.
3. Find the highest and lowest fold scores: 0.90 and 0.80.
4. Subtract to get the range: $0.90 - 0.80 = 0.10$.

Answer: The mean cross-validation accuracy is 0.85, and the fold scores spread over a range of 0.10. On average the classifier gets 85 visits in 100 right, and no single fold was much luckier than the others.

How many folds? With $k = 5$ each model trains on 80% of the rows; with $k = 10$ on 90%, closer to the full dataset, at the cost of twice as many training runs. Both are common; five is a comfortable default for the small datasets in this book.

Figure 5.6 plots the five scores with their mean. Because the folds are equal in size, the mean of the folds, 0.85, is also the total number of correct predictions, $18 + 16 + 17 + 18 + 16 = 85$, divided by 100.

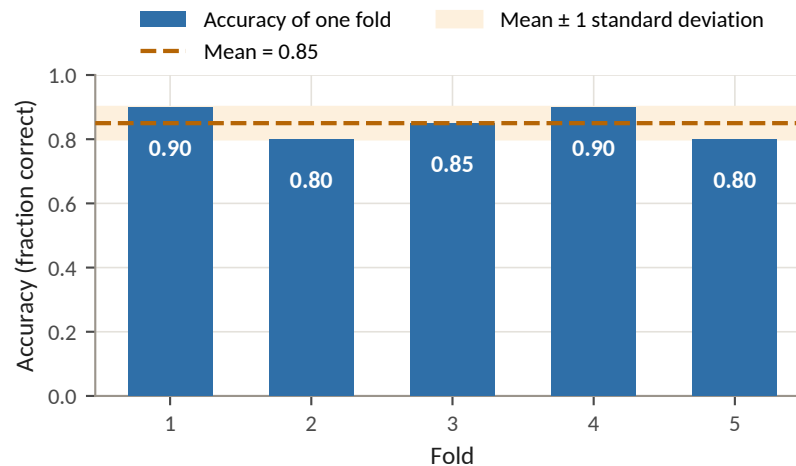


Figure 5.6 – The five fold accuracies of Worked Example 5.4 with their mean (dashed line) and the band of one standard deviation on either side, calculated in Lesson 5.6.

WHAT TO OBSERVE

The bars rise and fall around the dashed mean, and no fold is far outside the shaded band. When one bar sits far below the others, look at that fold: it may contain a cluster of unusual rows.

5.5 Stratified k -Fold, Leave-One-Out and Repeated CV

The no-show problem from Lesson 5.3 returns inside cross-validation. Cutting the 200 appointments into five random folds of 40 gives each fold 6 no-shows on average, but chance can leave one fold with 2 and another with 10, and the fold scores then jump around for reasons that have nothing to do with the model. The fix is the same as before: stratify.

WORDS TO KNOW

Stratified k -fold — Cross-validation in which each fold is built separately inside every class, so that all k folds carry the same class shares as the whole dataset.

Leave-one-out cross-validation — The extreme case $k = n$: every single row is its own fold, so n models are trained, each on $n - 1$ rows, and each is scored on the one row it did not see.

Repeated cross-validation — Running k -fold cross-validation several times with a fresh shuffle each time, then averaging all the fold scores, to smooth out the luck of any one set of folds.

With stratified 5-fold cross-validation, every one of the five folds of 40 appointments contains exactly 6 no-shows and 34 kept appointments. Ron Kohavi's 1995 study of many datasets found that stratified 10-fold cross-validation gave the most dependable estimates in practice, and stratification is scikit-learn's default for classifiers today. Leave-one-out, described by Peter Lachenbruch and M. Ray Mickey in 1968, pushes k as far as it will go: every model trains on all rows but one, so each is almost the model you will finally use. The price is computing time, one model per row. Repeated cross-validation attacks the luck of the shuffle from the other direction, by reshuffling and running the whole procedure again.

WORKED EXAMPLE 5.5 — Count the Models Trained by Three Cross-Validation Schemes

The clinic's 100 long-wait visits are cross-validated three ways: ordinary 5-fold, leave-one-out, and 5-fold repeated three times. For each scheme, how many models are trained, and on how many rows does each one train? How many times more models does leave-one-out train than one 5-fold run?

1. Ordinary 5-fold: 5 models, each trained on $100 - 20 = 80$ rows.
2. Leave-one-out: one fold per row, so $k = n = 100$ models, each trained on $100 - 1 = 99$ rows.
3. Repeated 5-fold, three repeats: $3 \times 5 = 15$ models, each trained on 80 rows with a different shuffle in each repeat.
4. Compare leave-one-out with one 5-fold run: $100 \div 5 = 20$ times as many models.

Answer: Five models (80 rows each), 100 models (99 rows each) and 15 models (80 rows each). Leave-one-out trains 20 times more models than 5-fold cross-validation to gain 19 extra training rows per model, a trade that is worth it only when the dataset is very small.

Table 5.4 lines up the schemes, including two that respect time order and groups, which the next lessons use.

Table 5.4 – Cross-validation schemes compared for a dataset of n rows

Scheme	Models trained	Each model trains on	Best for
k -fold ($k = 5$)	5	80% of the rows	The everyday default
Stratified k -fold	5	80%, same class shares in every fold	Classification with a rare class
Leave-one-out	n	$n - 1$ rows	Very small datasets
Repeated k -fold (3×5)	15	80%, reshuffled for each repeat	When one k -fold run looks noisy
Time-series split	k	Only rows from before the validation block	Time-ordered data (Lesson 5.7)
Group k -fold	k	About 80% of the groups, never a split group	Several rows per patient or machine (Lesson 5.7)

5.6 Reporting a Score With Its Spread

Two taxi drivers in Erbil both average 25 minutes to the airport. One always takes 23 to 27 minutes; the other sometimes 12 and sometimes 40. Same average, very different reliability. A cross-validation mean has the same weakness: alone, it hides how much the folds disagreed, so good practice reports the mean together with the standard deviation of the fold scores (introduced in Chapter 3), written as mean $\pm$ standard deviation, for example 0.85 ± 0.05 .

EQUATION: STANDARD DEVIATION OF THE FOLD SCORES

$$s_{sd} = \sqrt{\frac{1}{k-1} \sum_{j=1}^k (s_j - \bar{s})^2}$$

In words: Find how far each fold score sits from the mean, square those distances, add them up, divide by one less than the number of folds, and take the square root.

When it is used: Every time a cross-validation score is reported, to show how much the fold scores disagreed: mean $\pm$ standard deviation.

Symbol	Meaning	Unit or type
s_{sd}	standard deviation of the fold scores	same unit as the score
s_j	score of fold j	same unit as the score
$\bar{s}$	mean of the k fold scores	same unit as the score
k	number of folds	count

Dividing by $k - 1$ is the sample version from Chapter 3. NumPy's `std()` divides by k unless you write `std(ddof=1)`, which gives a slightly smaller number (0.045 instead of 0.05 for the scores below).

WORKED EXAMPLE 5.6 — Standard Deviation of the Five Fold Accuracies

The five fold accuracies of Worked Example 5.4 are 0.90, 0.80, 0.85, 0.90 and 0.80, with mean 0.85. Find their standard deviation and write the cross-validation result as mean $\pm$ standard deviation.

1. Subtract the mean from each score: 0.05, -0.05 , 0.00, 0.05, -0.05 .
2. Square each difference: 0.0025, 0.0025, 0, 0.0025, 0.0025.
3. Add the squares: $0.0025 + 0.0025 + 0 + 0.0025 + 0.0025 = 0.0100$.
4. Divide by $k - 1 = 4$: $0.0100 \div 4 = 0.0025$.
5. Take the square root: $\sqrt{0.0025} = 0.05$.

Answer: The standard deviation is 0.05, so the result is reported as accuracy 0.85 ± 0.05 . A typical fold landed within about five percentage points of the mean.

The standard deviation describes how much single folds wander. A second, smaller number describes how uncertain the *mean itself* is: the standard error. Averaging k folds pins the mean down more tightly than any one fold, by a factor of $\sqrt{k}$.

WORD TO KNOW

Standard error — The standard deviation of the fold scores divided by the square root of the number of folds. It estimates how far the cross-validation mean might sit from the score you would get with unlimited data, and it shrinks as more folds are averaged.

EQUATION: STANDARD ERROR OF THE CROSS-VALIDATION MEAN

$$SE = \frac{s_{sd}}{\sqrt{k}}$$

In words: Divide the standard deviation of the fold scores by the square root of the number of folds.

When it is used: To judge whether the difference between two models' cross-validation means is large enough to matter. A difference smaller than about two standard errors could easily be luck.

Symbol	Meaning	Unit or type
SE	standard error of the mean score	same unit as the score
s_{sd}	standard deviation of the fold scores	same unit as the score
k	number of folds	count

The folds share most of their training rows, so their scores are not fully independent and this SE is a rough guide rather than an exact one. Treat it as a warning light, not a proof.

Table 5.5 shows the fold scores of the nearest-centroid model, called Model A from now on, next to a second candidate, Model B, on the same five folds.

Table 5.5 – Five-fold accuracies of two candidate long-wait models on the same folds

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std. dev.
A (nearest centroid)	0.90	0.80	0.85	0.90	0.80	0.85	0.05
B (second candidate)	0.93	0.81	0.93	0.81	0.87	0.87	0.06

WORKED EXAMPLE 5.7 — Is Model B Really Better Than Model A?

Model A scores 0.85 ± 0.05 and Model B scores 0.87 ± 0.06 over the same five folds. Find the standard error of each mean and decide whether B is clearly better.

1. Find $\sqrt{k}$ for $k = 5$: $\sqrt{5} \approx 2.236$.
2. Standard error of A: $0.05 \div 2.236 \approx 0.022$.
3. Standard error of B: $0.06 \div 2.236 \approx 0.027$.
4. Difference between the means: $0.87 - 0.85 = 0.02$.
5. Compare: the difference of 0.02 is smaller than one standard error of either model, and far smaller than two standard errors of B, $2 \times 0.027 \approx 0.054$.

Answer: The standard errors are about 0.022 for A and 0.027 for B. The 0.02 gap between the two means is within the noise of the folds, so the team cannot say B is better. It should choose on other grounds (simplicity, speed, cost of mistakes) or collect more data.

COMMON MISTAKE

X Wrong: Declaring Model B the winner because 0.87 is larger than 0.85, and putting only the two means in the report.

✓ Right: Report mean $\pm$ standard deviation for every model, compare the gap between means with the standard errors, and say “no clear difference” when that is the truth.

5.7 Data Leakage Through Splitting

Chapter 2 defined data leakage as information reaching the model that would not be available at prediction time. This lesson looks at three ways the *split itself* can leak, even when every feature is perfectly safe. The general problem was studied carefully by Shachar Kaufman, Saharon Rosset and Claudia Perlich in 2011, who found leakage behind many results that looked too good to be true.

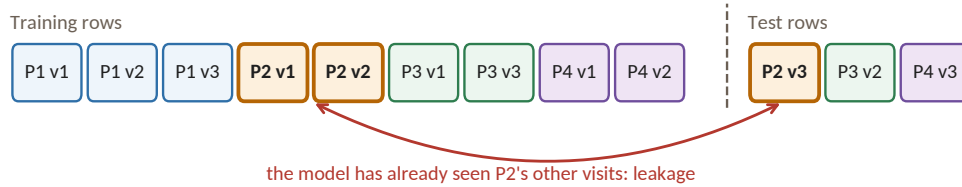
WORDS TO KNOW

Group leakage — Leakage that happens when rows belonging to the same person, machine or place land on both sides of a split, so the model can recognize the group instead of learning a general rule.

Test-set peeking — Looking at the test score more than once and using it to make choices, which turns the test set into a validation set and makes its final score optimistic.

Group leakage. A hospital dataset has one row per *visit*, and most patients visit several times. A random split by row sends two visits of patient P2 to the training set and a third to the test set. The model then does not need to understand medicine; it only needs to notice that the test row looks like the P2 rows it has already seen, and copy their label. The test score rises for a reason that vanishes the moment a genuinely new patient walks in. Figure 5.7 shows the leak and the repair: a group split keeps every visit of a patient on the same side. In cross-validation the same idea is group k -fold (GroupKFold in scikit-learn), and the case study shows how much difference it can make.

Split by visit (random rows): patient P2 is on BOTH sides



Split by patient (group split): every visit of a patient stays on one side

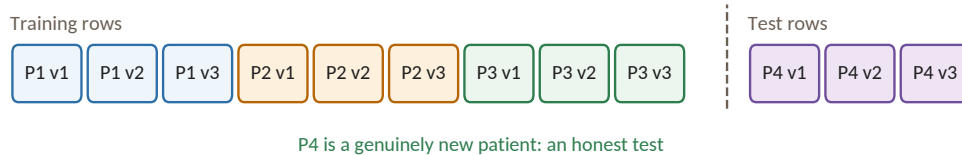


Figure 5.7 – Twelve visits from four patients. Splitting by visit (top) puts patient P2 on both sides of the cut; splitting by patient (bottom) keeps each patient entirely on one side.

WHAT TO OBSERVE

In the top panel, P2 v3 is in the test set while P2 v1 and P2 v2 are in the training set: the model is being tested on a patient it has already studied. In the bottom panel the test set contains only P4, whom the model has never seen, which is exactly the situation at the hospital door. The group split has slightly uneven sets, which is a small price.

Time leakage. Lesson 5.3 showed that shuffling time-ordered rows lets the model learn from the future. Ordinary folds on time-ordered data leak the same way, because some folds train on rows recorded after the validation fold. The repair is a time-series split, in which every validation block is later than everything it is compared against; scikit-learn's `TimeSeriesSplit` builds these growing windows, and the Python activity prints one.

Test-set peeking. The third leak needs no unusual data at all, only impatience. Every look at the test score that changes what you do next quietly turns the test set into a validation set. The reported score then contains luck, because you kept the setting that happened to shine on those particular rows.

WORKED EXAMPLE 5.8 — Test-Set Peeking Inflates a Score

A team tries five settings and checks accuracy on the 200-row test set after every try: A scores 82%, B 85%, C 79%, D 88% and E 84%. They keep D and report “88% test accuracy”. After launch, D scores 81% on 200 genuinely new rows. A second team compares the same settings with 5-fold cross-validation on the training data only (81%, 84%, 78%, 83%, 85%), keeps E, and opens the test set once: 84%. Find the drop for each team.

1. First team: the highest of the five peeked scores is 88%, from Setting D.
2. Compare with the fresh data: $88 - 81 = 7$ percentage points too optimistic.
3. Second team: the highest cross-validation mean is 85%, from Setting E.
4. Compare with its single test score: $85 - 84 = 1$ percentage point.

Answer: Peeking inflated the reported score by 7 points; choosing with cross-validation and testing once left a gap of only 1 point. Setting D was not necessarily the best setting, only the luckiest on that one test set.

Figure 5.8 places the two teams side by side.

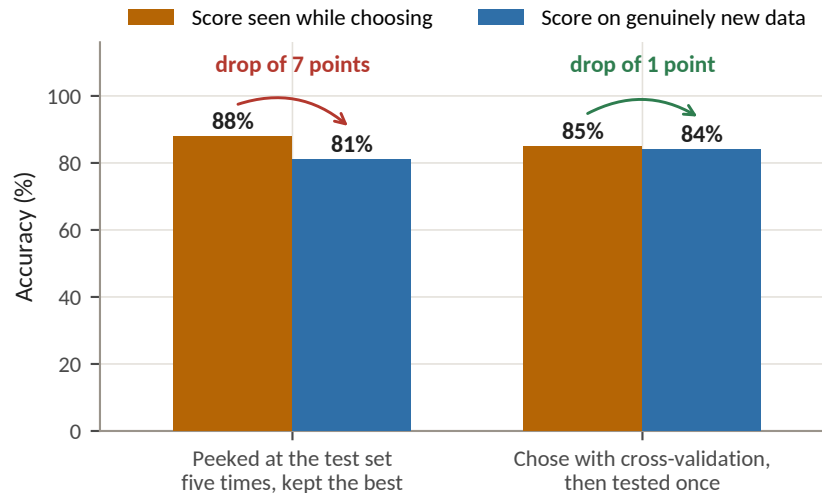


Figure 5.8 – Score used to choose a setting versus score on genuinely new data, for a team that peeked at the test set five times and a team that chose with cross-validation and tested once.

WHAT TO OBSERVE

The orange bars are the numbers each team believed; the blue bars are what the world delivered afterward. Peeking bought a higher orange bar and a bigger disappointment. Cross-validation's estimate was lower but nearly right, the kind of number a clinic can plan around.

IMPORTANT

A test set can be used honestly only once. The moment its score is used to choose between models or settings, it becomes another validation set and its number is no longer a fair estimate. Finish every choice with the validation set or cross-validation, then open the test set exactly one time and report what you see. Chapter 23 shows how to nest cross-validation inside cross-validation when many settings must be searched.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing that a high training accuracy proves a model is good; the training score mixes real skill with memorization and is always the most optimistic number.
- Using the test set to compare candidate models or choose settings, then reporting its score as if the model had never seen it.
- Making a random split on data with a time order, so that the model learns from the future and is scored on the past.
- Thinking cross-validation replaces the test set; it replaces the validation set, and the final test still happens once.
- Reporting only the mean of the fold scores, or calling a model “better” because its mean is 0.02 higher when the gap is smaller than the standard error.
- Splitting by row when several rows belong to the same patient or machine, so the model recognizes the group instead of learning the rule.

Practical Activities

ACTIVITY (PAPER) — Five Folds From Fifteen Cards

Each card is one clinic visit: the number of patients already waiting when the visit began, and whether the wait turned out long (Y) or not (N). Your “model” is the one-feature nearest-centroid rule from Chapter 1: predict Y if the card’s waiting count is closer to the average of the training Y cards than to the average of the training N cards.

Card	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Waiting	2	3	4	5	6	7	8	11	6	10	12	13	14	15	16
Long wait?	N	N	N	N	N	N	N	N	Y	Y	Y	Y	Y	Y	Y

1. Write the 15 cards, shuffle them, and deal five piles of three. To compare with our answer, use these piles instead: fold 1 = cards 3, 10, 8; fold 2 = cards 1, 12, 9; fold 3 = cards 6, 14, 4; fold 4 = cards 2, 11, 7; fold 5 = cards 5, 13, 15.
2. Round 1: set fold 1 aside. From the other 12 cards, average the waiting counts of the N cards and of the Y cards. (Shortcut: all eight N cards add to 46 and all seven Y cards add to 86, so subtract the held-out cards from these totals. For fold 1: N average = $(46 - 4 - 11) \div 6 \approx 5.17$, Y average = $(86 - 10) \div 6 \approx 12.67$.)
3. Predict each held-out card with the nearest average, compare with its true label, and write down how many of the three are correct.
4. Repeat for folds 2 to 5. Add the five correct counts and divide by 15 to get the mean cross-validation accuracy.
5. Check: you should find 13 correct out of 15, a mean accuracy of about 0.87, with the two mistakes on cards 8 and 9, the visits that broke the usual pattern. If you dealt your own piles, why are your fold accuracies different but your mean similar?

You need: 15 index cards or slips of paper, a pencil and a calculator

ACTIVITY (PYTHON) — Folds, Stratified Folds and Cross-Validation Scores with scikit-learn

The program repeats Worked Examples 5.2, 5.3, 5.4 and 5.6 with scikit-learn. Part 3 reads `datasets/ch05_clinic_visits.csv`, 100 clinic visits with the hour of arrival, the number of patients already waiting, the number of clinicians on duty and the label `long_wait`; the rows are already in random order, so no extra shuffling is needed.

```

import numpy as np, pandas as pd
from sklearn.model_selection import KFold, StratifiedKFold, TimeSeriesSplit
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import NearestCentroid

# 1. Cut 20 rows into 5 folds (Worked Example 5.3)
folds = list(KFold(n_splits=5).split(np.arange(20)))
print("rows held out per fold:", [(val + 1).tolist() for train, val in folds])
print("training rows each time:", [len(train) for train, val in folds])
# 2. Stratified folds keep 6 of the 30 no-shows in every fold (Worked Example 5.2)
y = np.array([1] * 30 + [0] * 170)
folds = StratifiedKFold(n_splits=5).split(np.zeros(200), y)
print("no-shows held out per fold:", [int(y[val].sum()) for train, val in folds])
# 3. Real fold scores on 100 clinic visits (Worked Examples 5.4 and 5.6)
data = pd.read_csv("datasets/ch05_clinic_visits.csv")
X, y = data[["hour", "waiting", "clinicians"]].values, data["long_wait"].values
scores = cross_val_score(NearestCentroid(), X, y, cv=KFold(n_splits=5))
sd = scores.std(ddof=1)
print("fold accuracies:", scores)
print("mean = %.2f  sd = %.2f  se = %.3f" % (scores.mean(), sd, sd / 5 ** 0.5))
# 4. Time-ordered folds: the validation months always come AFTER the training months
for train, val in TimeSeriesSplit(n_splits=3).split(np.arange(12)):
    print("train on months", train + 1, "-> validate on months", val + 1)

```

Output

```

rows held out per fold: [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16],
    [17, 18, 19, 20]]
training rows each time: [16, 16, 16, 16, 16]
no-shows held out per fold: [6, 6, 6, 6, 6]
fold accuracies: [0.9  0.8  0.85 0.9  0.8 ]
mean = 0.85  sd = 0.05  se = 0.022
train on months [1 2 3] -> validate on months [4 5 6]
train on months [1 2 3 4 5 6] -> validate on months [7 8 9]
train on months [1 2 3 4 5 6 7 8 9] -> validate on months [10 11 12]

```

Part 1 prints the same five blocks of four rows as Worked Example 5.3 (KFold numbers rows from 0, so the code adds 1), each round training on 16 rows. Part 2 shows that StratifiedKFold gives every fold exactly 6 of the 30 no-shows, as in Worked Example 5.2. Part 3 is the real thing: cross_val_score trains the nearest-centroid classifier five times and returns the fold accuracies 0.90, 0.80, 0.85, 0.90 and 0.80, and NumPy reproduces the mean 0.85, the standard deviation 0.05 (with ddof=1) and the standard error 0.022 of Worked Examples 5.4, 5.6 and 5.7. Part 4 shows TimeSeriesSplit: the training window grows and the validation months always come after it. Try KFold(n_splits=5, shuffle=True, random_state=1) in part 3 and watch the five scores change while the mean stays close to 0.85.

You need: A computer with Python, NumPy, pandas and scikit-learn installed (see Appendix B)

CASE STUDY — Grouped Cross-Validation for Repeat Patients at a Hospital in Chengdu

A large hospital in Chengdu, China, follows about 400 patients with type 2 diabetes. Each patient visits several times a year, and two years of records add up to 2,000 visits, about five per patient. The diabetes team wants a model that flags, at each visit, whether the patient's blood-sugar control is likely to be poor at the next visit, so that a nurse can schedule a supportive phone call in between. The features are known at the visit: age, weight, number of medications, the last three laboratory results and the days since the previous visit. All numbers in this case study are illustrative.

The data team first ran ordinary 5-fold cross-validation with one row per visit. The fold accuracies were 0.92, 0.90, 0.91, 0.93 and 0.89, a mean of 0.91. Then a physician asked: "When we use this model, will it ever see a visit from a patient who is already in the training data?" For newly diagnosed patients, the ones the hospital most wants to help, the answer was no. Yet in the visit-level folds almost every test visit belonged to a patient whose other visits were in the training folds. Age, weight and stable laboratory values made each patient easy to recognize, so the model could copy the label from that patient's other visits: the group leakage of Lesson 5.7.

The team repeated the cross-validation with `GroupKFold`, using the patient ID as the group, so that all visits of a patient stayed in one fold: 80 patients per fold, slightly uneven numbers of visits (412, 398, 405, 391 and 394), and fold accuracies of 0.80, 0.76, 0.79, 0.77 and 0.78, a mean of 0.78. The gap of 0.13 between the two estimates was leakage, not skill. Six months after launch, the model's accuracy on newly enrolled patients was 0.77, almost exactly the grouped estimate and nowhere near 0.91. Because the team had reported the honest number, nobody was surprised, and the nurses' call list was sized correctly from the first week.

The lesson reaches far beyond hospitals: several readings from one water pump, several photographs of one apple, several essays by one student. Whenever rows share an owner, the split must keep the owner on one side.

RESPONSIBLE USE

Predictions about patients, students or customers must be used to offer help, never to withhold it: the Chengdu model schedules a supportive call and does not decide who receives care, and a no-show prediction may trigger a friendly reminder, never a cancelled appointment. Report the honest, grouped score to the people who rely on the model, and keep checking it after launch, because even a leak-free estimate drifts as the world changes. Chapter 24 returns to these duties.

CHAPTER SUMMARY

- A score on the training rows mixes real skill with memorization; only a score on unseen rows measures generalization, and the gap between the two shrinks with more data but never disappears.
- The training set fits the model, the validation set compares candidates and settings as often as needed, and the test set is opened once for the final number.
- Split sizes come from the fractions: $n_{\text{train}} = f_{\text{train}} n$, $n_{\text{val}} = f_{\text{val}} n$, and the test set takes the rest.
- A random split shuffles and cuts; a stratified split keeps each class's share in every set; a time-based split trains on the past and tests on the future; a group split keeps every row of one patient or machine on one side.
- In k -fold cross-validation each fold is held out once while the other $k - 1$ folds train the model; the k scores are averaged, and every row is validated exactly once.
- Cross-validation replaces the validation set, not the test set; stratified k -fold protects rare classes, leave-one-out trains n models on $n - 1$ rows each, and repeated cross-validation reshuffles and averages again.
- Report mean $\pm$ standard deviation of the fold scores, and use the standard error $s_{\text{sd}}/\sqrt{k}$ to judge whether two models really differ.

- A gap between two models' means that is smaller than about two standard errors is not evidence that one is better.
- Group leakage, time leakage and test-set peeking all make a score look better than the model really is; the repairs are group splits, time-series splits and opening the test set only once.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain why a model's accuracy on its training rows is not an honest measure of how it will perform on new data. Use the exam-paper picture or one of your own.

Problem 2 [Basic] Name the three sets used to build and check a model, state in one sentence what each one is for, and explain why cross-validation can replace one of them but not the other two.

Problem 3 [Basic] A dataset has 900 rows. Split it 70% training, 15% validation and 15% test, showing every step, and check that the counts add up.

Problem 4 [Basic] Ten-fold cross-validation is run on 500 rows. How many rows are in each fold, how many rows does each model train on, and how many models are trained in total?

Problem 5 [Standard] A clinic's 400 booked appointments include 40 no-shows. Explain in one sentence why a stratified split matters here, then make a stratified 75% training, 25% test split and give the number of no-shows and kept appointments in each set. Check that both sets keep the original no-show share.

Problem 6 [Standard] Five fold accuracies are 0.72, 0.78, 0.75, 0.81 and 0.69. Find the mean cross-validation accuracy and the range, showing every step.

Problem 7 [Standard] For the five scores of the previous problem, find the standard deviation (divide by $k - 1$) and the standard error, showing every step. Write the result as mean $\pm$ standard deviation.

Problem 8 [Standard] Twelve rows, numbered 1 to 12 after shuffling, are cross-validated with $k = 4$. Draw the grid of rounds and folds in the style of Figure 5.5, list the rows held out in each round, and state how many rows each model trains on and how many times each row is used for training.

Problem 9 [Standard] A shop's sales records cover 36 months with 50 rows per month. Design a time-based split that keeps the most recent 6 months for testing and the 6 months before that for validation. How many rows are in each set, and why would a random split be wrong here?

Problem 10 [Standard] Model P has fold accuracies 0.88, 0.84, 0.86, 0.90 and 0.82; Model Q has 0.91, 0.79, 0.95, 0.83 and 0.87. Find each model's mean, standard deviation and standard error. Which model would you choose, and what would you say to a colleague who insists that Q is better because its mean is higher?

Problem 11 [Challenge] A dataset of 600 sensor readings comes from 30 water pumps, 20 readings each. A colleague proposes ordinary 5-fold cross-validation. (a) Explain the leakage risk in one paragraph. (b) Design a cross-validation scheme that avoids it, and state how many pumps and how many readings are held out in each fold. (c) Predict whether the honest score will be higher or lower than the naive one, and why.

Problem 12 [Challenge] Count the models trained by (a) leave-one-out on 60 rows, (b) 5-fold cross-validation repeated 10 times, and (c) a single 6-fold run. Rank the three by computing cost, then explain which one you would choose for a dataset of 60 rows and which for a dataset of 60,000 rows.

Problem 13 [Challenge] A team tries eight settings, checks each one on the test set, and reports the best score, 91%. On fresh data after launch the model scores 84%. (a) Name the leak and explain how it produced the 7-point drop. (b) Describe the safe procedure the team should have followed, including exactly how many times the test set may be opened. (c) Explain why the drop tends to grow as more settings are tried.

Measuring Model Performance

Errors for regression, the confusion matrix for classification, and what to do when classes are rare

IN THIS CHAPTER YOU WILL LEARN TO

- Choose a performance metric that matches what each kind of mistake really costs, before comparing models.
- Calculate MAE, MSE, RMSE and R^2 by hand for a regression model's predictions.
- Build a confusion matrix and calculate accuracy, precision, recall, specificity and F1 from it.
- Turn predicted probabilities into decisions with a threshold, and read an ROC curve, its AUC and a precision-recall curve.
- Explain why accuracy misleads on imbalanced data; apply resampling, class weights and threshold moving; read a multi-class matrix and a reliability diagram.

Introduction

A regional water utility keeps hundreds of hand pumps and small motorized pumps running across farming villages in Punjab. A pump that fails without warning can leave a whole village without safe drinking water for days, until a technician can travel out to repair it. The utility cannot inspect every pump every week, so it fits simple sensors — vibration, flow rate and motor temperature — to each pump and trains a model to flag the pumps likely to fail soon. After the first month the team proudly reports that the model is “92% accurate.”

Is that good? You cannot tell from one number. Ninety-two percent could describe a model that catches almost every failing pump, or one that never catches a single failure and simply gets the healthy pumps right. To judge a model honestly you have to look at *what kind* of mistakes it makes, how often, and what each mistake costs. This chapter is the toolbox for that judgment, from error metrics for regression and the confusion matrix for classification to thresholds, rare classes, many classes, and a final check on whether a model's “70% chance” really means 70%. Every idea is worked by hand on a handful of pumps or patients first, then repeated in Python.

6.1 A Metric Must Match the Real Goal

In Chapter 1 you wrote down a project's performance measure, P , before choosing a model, and you met accuracy as one such measure. Accuracy is not the only choice, and often not the right one.

WORD TO KNOW

Metric — A number used to measure how well a model performs. Different metrics pay attention to different kinds of mistakes, so choosing the right metric matters as much as choosing the right model.

A metric is a statement about which mistakes matter most. The pump model can go wrong in two ways: it can miss a pump that is truly about to fail, leaving a village without water and without warning, or it can raise a false alarm on a pump that was fine, costing one wasted trip. These mistakes do not cost the same, and Table 6.1 shows that in other projects the balance tips the other way. Choose a metric that reflects what a mistake actually costs, not whichever number is easiest to compute.

Table 6.1 – Two kinds of mistakes, and which one hurts more, in three projects (illustrative)

Project	“Positive” means	A missed positive costs...	A false alarm costs...	Worse mistake
Pump failure, Punjab	Pump will fail soon	A village without water for days	One wasted technician visit	Missed positive
E-mail spam filter	Message is spam	One junk message in the inbox	A real message lost in the spam folder	False alarm
Rare-disease screening	Patient has the disease	Treatment starts late	One extra test and some worry	Missed positive

COMMON MISTAKE

X Wrong: Training several candidate models first, then picking whichever metric makes the best-looking model look even better.

✓ Right: Decide which metric matters most *before* training or comparing any models, based on what each kind of mistake would truly cost.

One more fact matters for the whole chapter: on most days the great majority of pumps are fine, so a lazy model that always predicts “Normal” would reach a very high accuracy without ever catching a failure. Lesson 6.6 returns to this trap.

6.2 Regression Errors in Real Units: MAE, MSE, RMSE and R^2

The utility later builds a second model that predicts a number: how many days remain before a pump needs maintenance. This is a regression task, and its metrics describe error in the target’s own unit — days. Everything starts from the error of one prediction, $e_i = y_i - \hat{y}_i$, positive when the model predicted too low and negative when it predicted too high (Chapter 8 calls it a *residual*). Table 6.2 lists four pumps whose true days-until-maintenance were later recorded, together with the model’s predictions.

Table 6.2 – Four pump predictions and their errors (days until maintenance)

Pump	True y_i (days)	Predicted $\hat{y}_i$ (days)	Error $y_i - \hat{y}_i$	Absolute error $ y_i - \hat{y}_i $	Squared error $(y_i - \hat{y}_i)^2$
1	10	12	−2	2	4
2	25	20	5	5	25
3	15	18	−3	3	9
4	30	26	4	4	16
Sum			4	14	54

The plain errors partly cancel ($-2 + 5 - 3 + 4 = 4$), which flatters the model. The first three metrics below remove the signs in different ways; the fourth compares the model with the simplest possible guess, the *baseline* of Chapter 2 that always predicts the mean of the training targets.

WORDS TO KNOW

Mean absolute error (MAE) — The average size of the errors, ignoring whether each was too high or too low; in the same unit as the target.

Mean squared error (MSE) — The average of the squared errors; large errors count for much more than small ones, and the unit is the target's unit squared.

Root mean squared error (RMSE) — The square root of MSE, which brings the number back into the target's real unit.

R^2 (coefficient of determination) — A score comparing a model's squared errors with those of always predicting the mean: 1 is perfect, 0 is no better than the mean, and a negative value is worse than the mean. The name was introduced by the geneticist Sewall Wright in 1921.

EQUATION: THE FOUR REGRESSION METRICS: MAE, MSE, RMSE AND R^2

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

In words: MAE: drop the minus sign of every error, then average. MSE: square every error, then average. RMSE: take the square root of MSE. R^2 : divide the model's total squared error by the total squared error of the mean baseline, and subtract from one.

When it is used: MAE to report a typical error in the target's unit. MSE is the loss most regression algorithms minimize during training (Chapter 8); RMSE is the version to report, and the better choice when a few large errors are much worse than many small ones. R^2 to say how much of the variation in the target the model explains, on a unit-free scale; report it together with MAE or RMSE.

Symbol	Meaning	Unit or type
MAE, RMSE	mean absolute error, root mean squared error	same unit as y
MSE	mean squared error	unit of y , squared
R^2	coefficient of determination	number, at most 1
n	number of predictions	count
$y_i, \hat{y}_i, \bar{y}$	true value, predicted value, mean of the true values	unit of the target

The vertical bars mean "ignore the sign." RMSE is never smaller than MAE on the same predictions. In R^2 the numerator is the model's sum of squared errors (SSE) and the denominator the total sum of squares (SST) of the baseline.

WORKED EXAMPLE 6.1 — Calculate MAE, MSE and RMSE for Four Pump Predictions

Using Table 6.2, find the three error metrics of the four predictions.

- Find each error, true minus predicted: $10 - 12 = -2$, $25 - 20 = 5$, $15 - 18 = -3$, $30 - 26 = 4$.
- Drop the signs and add: $2 + 5 + 3 + 4 = 14$. Divide by $n = 4$: $\text{MAE} = 14 \div 4 = 3.5$ days.
- Square each error and add: $4 + 25 + 9 + 16 = 54$. Divide by 4: $\text{MSE} = 54 \div 4 = 13.5$ days squared.
- Take the square root: $\text{RMSE} = \sqrt{13.5} \approx 3.67$ days.

Answer: $MAE = 3.5$ days, $MSE = 13.5$ days², $RMSE \approx 3.67$ days. RMSE sits above MAE because the one large error, 5 days on pump 2, contributes 25 of the 54 squared units.

WORKED EXAMPLE 6.2 — Calculate R^2 for Five Pump Predictions

At a second site the true days until maintenance for five pumps were 10, 20, 30, 40 and 50, and the model predicted 12, 18, 33, 38 and 49. Find R^2 .

1. Mean of the true values: $\bar{y} = 150 \div 5 = 30$.
2. Model's squared errors: $(10 - 12)^2 = 4$, $(20 - 18)^2 = 4$, $(30 - 33)^2 = 9$, $(40 - 38)^2 = 4$, $(50 - 49)^2 = 1$; $SSE = 22$.
3. Baseline's squared errors, each true value minus 30: 400, 100, 0, 100, 400; $SST = 1000$.
4. $R^2 = 1 - 22 \div 1000 = 0.978$.

Answer: $R^2 = 0.978$; the model removes 97.8% of the squared error that always predicting the mean would make.

Figure 6.1 shows the two standard plots for a regression model, drawn for the five pumps of Worked Example 6.2 together with 25 other pumps from the same site.

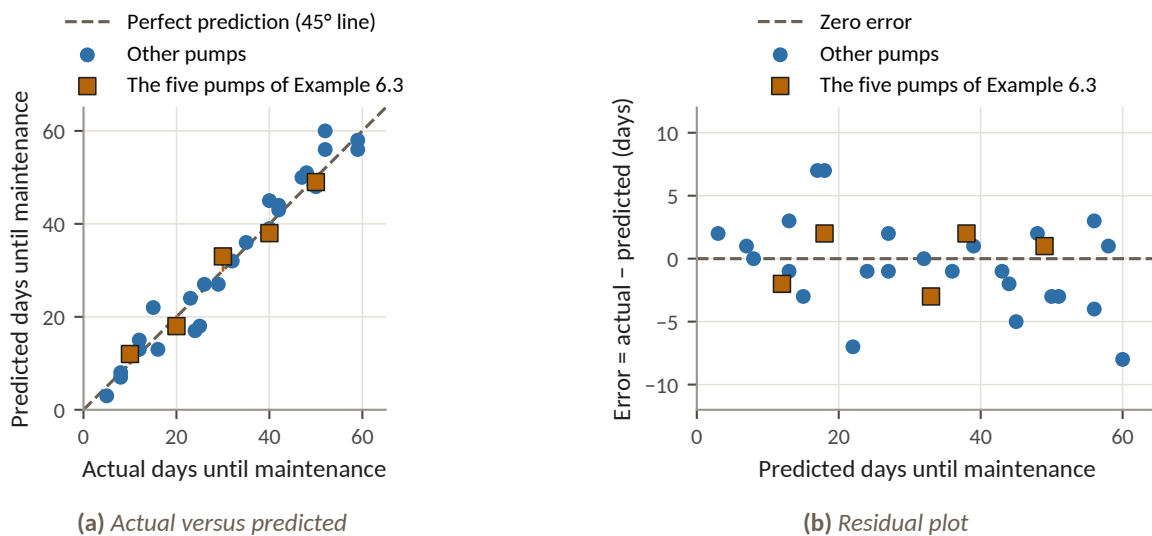


Figure 6.1 – Two plots for checking a regression model. Left: each pump is a point at (actual, predicted); perfect predictions sit on the dashed 45° line, and the orange segments are the errors of the five example pumps. Right: the same errors plotted against the prediction.

WHAT TO OBSERVE

In the left panel the points hug the 45° line, which is what a high R^2 looks like. In the right panel the errors scatter evenly above and below zero with no curve and no widening fan, so the model is not systematically too high or too low anywhere; a residual plot with a clear shape means the model is missing a pattern (Chapter 8).

6.3 Reading a Confusion Matrix

Back to the first pump model, the one that answers yes or no. Call a pump the model flags as “Will fail soon” a *positive* prediction and a pump it leaves as “Normal” a *negative* prediction. Every prediction then falls into one of four groups.

WORDS TO KNOW

Confusion matrix — A table that sorts every prediction of a classifier into one of four groups by comparing the prediction with the truth. Rows are the actual classes, columns the predicted classes.

True positive (TP) — Actually positive, predicted positive — a correct alarm.

False negative (FN) — Actually positive, predicted negative — a miss.

False positive (FP) — Actually negative, predicted positive — a false alarm.

True negative (TN) — Actually negative, predicted negative — correctly left alone.

TP and TN are correct; FN and FP are the two kinds of mistake. For the utility a false negative is a failure the model missed, and a false positive is a wasted trip.

WORKED EXAMPLE 6.3 — Build a Confusion Matrix for 50 Pump Inspections

Over one month, technicians follow up on 50 pumps. The model flagged 12 of them as “Will fail soon” and left 38 as “Normal.” In fact 10 of the 50 failed within two weeks and 40 did not, and the model had flagged 9 of the 10 that failed. Find TP, FN, FP and TN.

1. True positives: failed *and* flagged. Given directly: $TP = 9$.
2. False negatives: failed but not flagged. Ten failed and 9 were caught: $FN = 10 - 9 = 1$.
3. False positives: flagged but did not fail. Twelve were flagged, 9 correctly: $FP = 12 - 9 = 3$.
4. True negatives: did not fail and not flagged. Forty did not fail, 3 of them wrongly flagged: $TN = 40 - 3 = 37$.
5. Check: $9 + 1 + 3 + 37 = 50$.

Answer: $TP = 9$, $FN = 1$, $FP = 3$, $TN = 37$. The model caught 9 of the 10 pumps that needed urgent attention, missed 1, and sent technicians on 3 unnecessary trips.

Figure 6.2 lays the four numbers out as a matrix, with darker cells for larger counts. Row totals are the actual classes (10 failed, 40 did not); column totals are the predictions (12 flagged, 38 left as Normal).

6.4 Accuracy, Precision, Recall, Specificity and F1

Accuracy (Chapter 1) is the fraction of predictions that were correct: the two diagonal cells over the total. It treats every mistake the same. Precision and recall split it into two more specific questions, each looking at a different half of the matrix; specificity does for the negatives what recall does for the positives; and F1 combines precision and recall into one number. The words precision and recall come from information retrieval, the science of searching document collections, where Allen Kent and colleagues introduced them in 1955.

WORDS TO KNOW

Precision — Of all the examples the model predicted as positive, the fraction that really were positive: “when this model raises an alarm, how often is it right?”

Recall (sensitivity, true positive rate) — Of all the examples that really were positive, the fraction the model found: “of all the true cases, how many did the model catch?”

Specificity (true negative rate) — Of all the examples that really were negative, the fraction the model correctly left alone. Its complement, $1 - \text{specificity}$, is the *false positive rate*.

Actual Will fail	9 TP	1 FN
Actual Normal	3 FP	37 TN
	Predicted Will fail	Predicted Normal

Figure 6.2 – Confusion matrix for the 50 pump inspections. Rows are the actual condition, columns the prediction.

WHAT TO OBSERVE

The two diagonal cells (9 and 37) hold the 46 correct predictions. The single false negative in the top right is the most important pump of the month: it failed with no warning at all. The three false positives in the bottom left cost three wasted trips. Always read all four cells, and keep the layout straight: here and in scikit-learn rows are the actual class and columns the prediction, but some texts swap them.

WORD TO KNOW

F1 score — A single score combining precision and recall with the harmonic mean, an average that stays high only when *both* are high. It is the best-known member of the F-measure family proposed by C. J. van Rijsbergen in 1979.

EQUATION: ACCURACY, PRECISION, RECALL, SPECIFICITY AND F1

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad \text{Specificity} = \frac{TN}{TN + FP}$$

In words: Accuracy: the two correct cells over everything. Precision: true positives over everything the model called positive (first column). Recall: true positives over everything that was positive (first row). Specificity: true negatives over everything that was negative (second row). F1: twice the product of precision and recall, divided by their sum.

When it is used: Accuracy only when the classes are balanced and both mistakes cost about the same. Precision when false alarms are expensive (spam filters, fraud investigations); recall when misses are expensive (disease screening, pump failures); recall and specificity together to describe a screening test. F1 to rank models with one number when the positive class is rare.

Symbol	Meaning	Unit or type
TP, TN	correct positive and negative predictions	count
FP, FN	false alarms, misses	count
F1	harmonic mean of precision and recall	0 to 1

Recall and specificity are fractions of an actual class, so they do not change when a class becomes rarer; precision and accuracy do, as Lesson 6.6 shows. A shortcut for F1 straight from the matrix: $F1 = 2 TP \div (2 TP + FP + FN)$.

WORKED EXAMPLE 6.4 — Calculate the Pump Model's Accuracy, Precision, Recall, Specificity and F1

Using $TP = 9$, $FN = 1$, $FP = 3$ and $TN = 37$, find the five scores.

1. Accuracy: correct = $9 + 37 = 46$ out of 50, so $46 \div 50 = 0.92$, or 92%.
2. Positive predictions: $9 + 3 = 12$. Precision = $9 \div 12 = 0.75$.
3. Actual positives: $9 + 1 = 10$. Recall = $9 \div 10 = 0.90$.
4. Actual negatives: $37 + 3 = 40$. Specificity = $37 \div 40 = 0.925$; false positive rate = $1 - 0.925 = 0.075$.
5. F1: $2 \times 0.75 \times 0.90 = 1.35$ and $0.75 + 0.90 = 1.65$, so $F1 = 1.35 \div 1.65 \approx 0.818$. Shortcut check: $18 \div (18 + 3 + 1) = 18 \div 22 \approx 0.818$.

Answer: Accuracy = 0.92, precision = 0.75, recall = 0.90, specificity = 0.925, F1 $\approx$ 0.818. The 92% hides the one missed failure; recall shows it. F1 sits between precision and recall, pulled toward the weaker of the two.

Precision and recall often pull in opposite directions: flagging more pumps raises recall, but some of the extra flags are wrong, so precision falls. Table 6.3 collects the five scores. No single score, F1 included, tells the whole story: always ask what a model's false negatives and false positives would cost in the real project.

Table 6.3 – Five classification metrics from one confusion matrix

Metric	Formula	Question it answers	Use when	Pump model
Accuracy	$(TP + TN)/n$	What fraction of all predictions were right?	Classes balanced, mistakes cost the same	0.92
Precision	$TP/(TP + FP)$	Of the alarms, how many were right?	False alarms are costly	0.75
Recall	$TP/(TP + FN)$	Of the true cases, how many were caught?	Misses are costly	0.90
Specificity	$TN/(TN + FP)$	Of the true negatives, how many were cleared?	Describing a screening test	0.925
F1	$2 TP/(2 TP + FP + FN)$	Are precision and recall both high?	Ranking models on rare positives	0.818

COMMON MISTAKE

X Wrong: Assuming that precision and recall always rise and fall together, or using the two words interchangeably.

✓ Right: Keep the questions separate: precision asks “of the alarms, how many were right?”; recall asks “of the true cases, how many did we find?” A model can have high precision and low recall, or the reverse.

6.5 From Probabilities to Decisions: Thresholds, ROC Curves and AUC

Every example so far used a model that gave a plain yes or no. Most classifiers first output a *probability* — a number between 0 and 1 for how likely the positive class is — and only afterward does a cutoff turn it into a

decision. The cutoff is a choice the team makes; two rates describe its effect on each actual class, and a curve of those two rates over all possible cutoffs is the most widely used picture of a classifier.

WORDS TO KNOW

Threshold — The probability a model's output must reach before the prediction counts as positive. A threshold of 0.5 is the common default; lowering it makes the model flag more examples as positive, raising it makes the model flag fewer.

ROC curve — A plot of the true positive rate (up the side) against the false positive rate (along the bottom) for every threshold from 1 down to 0.

AUC (area under the ROC curve) — One number, from 0.5 (random guessing) to 1.0 (perfect), for how well a model *rank*s positives above negatives. It equals the probability that a random positive example scores higher than a random negative one (Hanley and McNeil, 1982).

Precision-recall curve — A plot of precision against recall at every threshold. It looks only at the positive class, which makes it more informative than the ROC curve when positives are rare; the area under it (average precision) summarizes it.

EQUATION: THE THRESHOLD RULE, TRUE POSITIVE RATE AND FALSE POSITIVE RATE

$$\hat{y} = \begin{cases} \text{positive} & \text{if } p \geq t \\ \text{negative} & \text{if } p < t \end{cases} \quad \text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

In words: Predict the positive class whenever the model's probability is at or above the threshold. At that threshold, TPR is the fraction of actual positives that were flagged (recall) and FPR is the fraction of actual negatives that were wrongly flagged.

When it is used: Every time a probability-producing classifier must give a yes-or-no answer. TPR and FPR place the chosen threshold as one point on the ROC curve below.

Symbol	Meaning	Unit or type
p	the model's predicted probability for one example	0 to 1
t	the threshold chosen by the team	0 to 1
TPR	true positive rate, the same as recall	0 to 1
FPR	false positive rate, equal to 1 – specificity	0 to 1

The threshold is applied after training, so it can be changed at any time without retraining. The ideal threshold has TPR = 1 and FPR = 0.

Ten pumps at one site, A to J, receive predicted probabilities of failure of 0.08, 0.15, 0.25, 0.35, 0.40, 0.55, 0.62, 0.70, 0.82 and 0.93. Pumps D, F, H, I and J later failed; A, B, C, E and G did not.

WORKED EXAMPLE 6.5 — Classify the Ten Pumps at Threshold 0.5 and Find the ROC Point

Using the ten probabilities above and a threshold of 0.5, find TP, FN, FP, TN, precision, recall (TPR) and FPR.

1. Apply $p \geq 0.5$: pumps F, G, H, I and J are flagged; A to E are left as Normal.
2. Flagged pumps against the truth: F, H, I and J truly failed (TP = 4); G did not (FP = 1). Unflagged: A, B, C and E did not fail (TN = 4); D failed but was missed (FN = 1). Check: $4 + 1 + 1 + 4 = 10$.
3. Precision = $4 \div 5 = 0.80$. TPR (recall) = $4 \div (4 + 1) = 0.80$. FPR = $1 \div (1 + 4) = 0.20$.

Answer: At threshold 0.5: TP = 4, FN = 1, FP = 1, TN = 4; precision and recall are both 0.80, and the ROC point is (FPR, TPR) = (0.20, 0.80).

Now move the threshold. Figure 6.3 shows the matrix for the same ten pumps at 0.7, 0.5 and 0.3, with the scores of each threshold in its title.

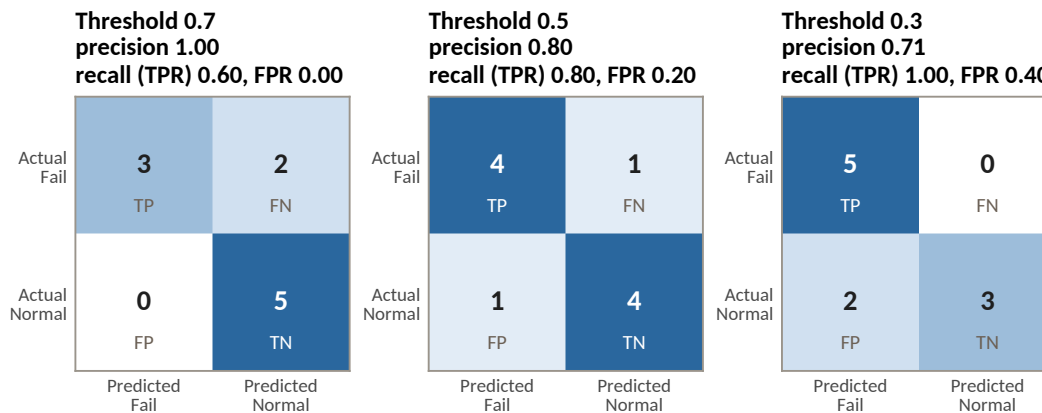


Figure 6.3 – The same ten pumps and the same model at three thresholds. Only the cutoff changes, yet every cell of the matrix moves.

WHAT TO OBSERVE

Lowering the threshold from 0.7 to 0.3 moves pumps from the right column to the left: false negatives become true positives (recall rises from 0.60 to 1.00), but true negatives become false

positives (precision falls to 0.71, FPR rises to 0.40). No threshold removes both mistakes; each one chooses a different trade-off. Plotting TPR against FPR for every threshold gives the ROC curve. It was first drawn for radar operators in the 1940s — and entered medical decision-making through the work of John Swets (Swets, 1988). Its companion, the precision-recall curve, does the same for precision and recall. Figure 6.4 shows both for the ten pumps.

COMMON MISTAKE

✗ Wrong: Choosing the threshold by repeatedly checking scores on the test set and keeping whichever cutoff scores best there.

✓ Right: Choose the threshold on the validation set or with cross-validation (Chapter 5); save the test set for one last, honest check.

6.6 When 99% Accuracy Means Nothing: Imbalanced Data

A hospital network in Chengdu, China, screens patients for a rare disease with a low-cost blood test. A model reads the laboratory measurements and predicts whether the patient is likely to have the disease, so that only patients with a worrying prediction are sent for a slower, more certain test. In one month's batch of 1,000 patients, only 10 truly have the disease (illustrative numbers, not medical statistics, but the pattern is extremely common). That one fact, that the disease is rare, matters more than anything else in this lesson.

WORDS TO KNOW

Class imbalance — A situation where one class has far more examples than another. The common class is the *majority class*, the rare class the *minority class*.

Oversampling — Adding more copies, or new similar examples, of the minority class to the training set.

Undersampling — Removing examples of the majority class from the training set.

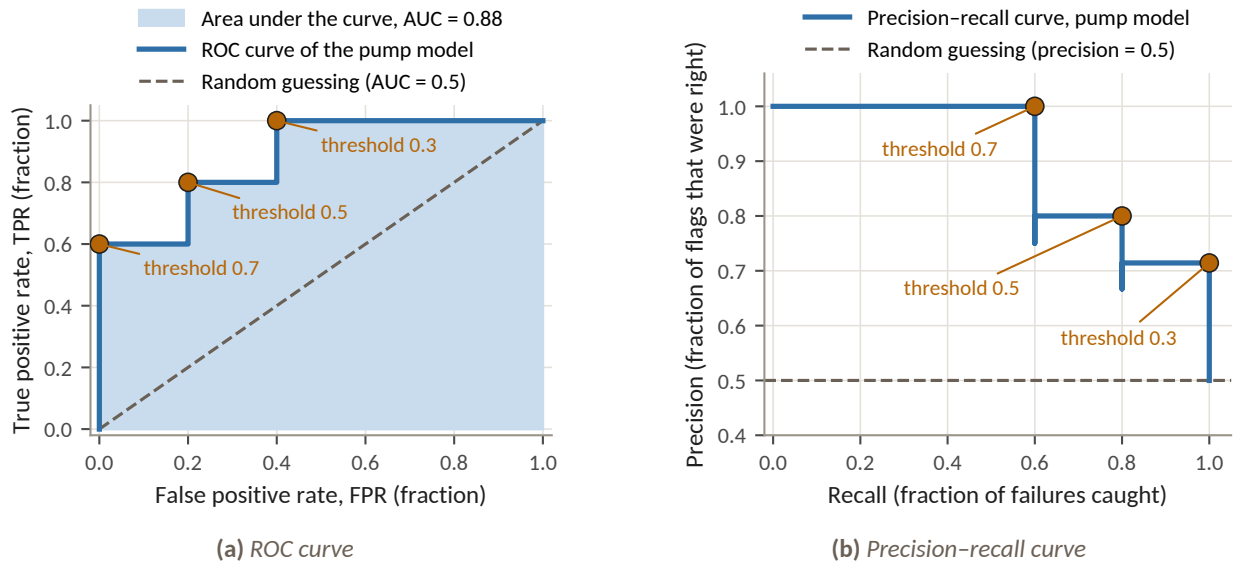


Figure 6.4 – Two threshold-free views of the ten-pump model. The orange points are the three thresholds of Figure 6.3.

WHAT TO OBSERVE

With only ten pumps the curves are staircases: each step is one pump crossing the threshold. The ROC curve stays well above the diagonal of random guessing, and the shaded area is $AUC = 0.88$. Check it by hand: of the $5 \times 5 = 25$ (failing, healthy) pairs of pumps, the failing pump scores higher in 22, and $22 \div 25 = 0.88$. On the precision-recall curve, precision stays at 1.0 until recall reaches 0.6, then drops as healthy pumps G and E are flagged; its average precision is about 0.90.

Class weight — A number attached to a class before training that multiplies how much a mistake on that class costs inside the learning algorithm's loss; a high weight on the minority class pushes the algorithm to avoid missing it.

EQUATION: IMBALANCE RATIO AND INVERSE-FREQUENCY CLASS WEIGHT

$$\text{Imbalance ratio} = \frac{N_{\text{majority}}}{N_{\text{minority}}} \quad w_{\text{positive}} = \frac{N_{\text{negative}}}{N_{\text{positive}}}, \quad w_{\text{negative}} = 1$$

In words: The imbalance ratio divides the size of the larger class by the size of the smaller one. The inverse-frequency class weight is the same fraction used as a multiplier: the rare class counts that many times more during training, and the common class counts once.

When it is used: Compute the imbalance ratio first on any classification dataset; a ratio above about 4-to-1 warns that accuracy will mislead. Use the class weight whenever a model supports a `class_weight` setting (most scikit-learn classifiers do) and the minority class must not be ignored.

Symbol	Meaning	Unit or type
$N_{\text{majority}}, N_{\text{minority}}$	examples in the more common and the rarer class	count
w_{positive}	weight of the minority (positive) class in training	multiplier

A ratio written “99-to-1” means the majority class has 99 times as many examples as the minority class. `scikit-learn`’s `class_weight=“balanced”` computes inverse-frequency weights automatically.

WORKED EXAMPLE 6.6 — Find the Imbalance Ratio and the Accuracy of a Model That Always Says “No Disease”

Of 1,000 screened patients, 10 have the disease and 990 do not. Find the imbalance ratio, then TP, FN, FP, TN, accuracy and recall for a lazy model that predicts “no disease” for everyone.

1. Imbalance ratio: $990 \div 10 = 99$, or 99-to-1.
2. The lazy model never predicts positive: $TP = 0$, $FP = 0$. All 10 sick patients are missed ($FN = 10$) and all 990 healthy ones are cleared ($TN = 990$).
3. Accuracy: $(0 + 990) \div 1000 = 0.99$, or 99%. Recall: $0 \div 10 = 0$.

Answer: The imbalance ratio is 99-to-1, and the lazy model scores 99% accuracy while catching none of the 10 patients who have the disease. A number that sounds almost perfect hides a model that is useless for the one job it exists to do.

Figure 6.5 shows why: the lazy model’s accuracy is simply the share of the majority class. A real model that catches 8 of the 10 sick patients while wrongly flagging 30 healthy ones ($TP = 8$, $FN = 2$, $FP = 30$, $TN = 960$) has a lower accuracy, $968 \div 1000 = 96.8\%$, yet a recall of 80%. Judged by accuracy the useless model wins; judged by recall the useful one does. On imbalanced data, always report recall, precision and F1 alongside accuracy.

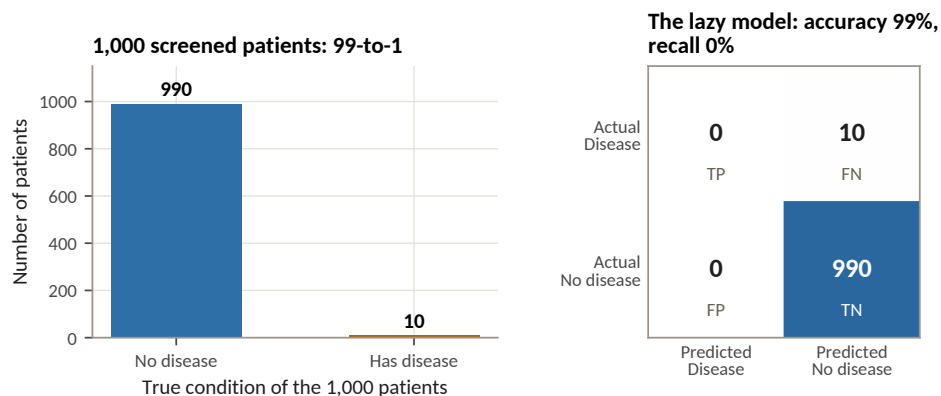


Figure 6.5 – Left: 990 healthy patients and 10 with the disease, a 99-to-1 imbalance. Right: the confusion matrix of the lazy model that predicts “no disease” for everyone.

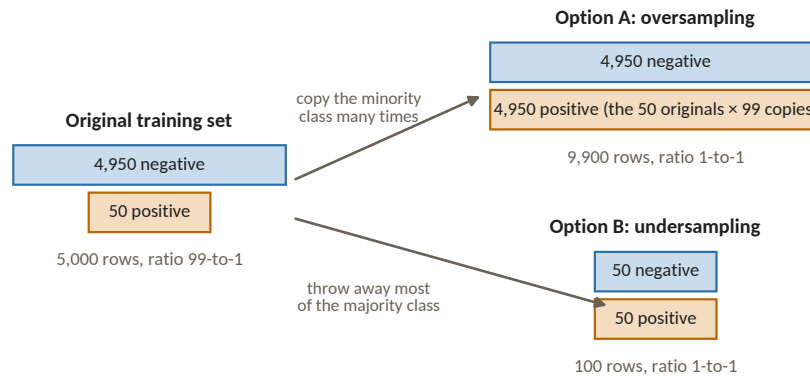
WHAT TO OBSERVE

The lazy model’s left column is empty: it never raises an alarm, and its 99% accuracy comes entirely from the 990 true negatives. Any metric that looks only at the positive row (recall) or the positive column (precision) exposes it at once.

A model trained on such data learns to lean toward the common class. Three fixes are in common use: two act before or during training, one afterward.

Fix 1 — resample the training set. The hospital’s training set has 5,000 past patients with confirmed results: 50 positive and 4,950 negative, the same 99-to-1 ratio. Two strategies change the balance before training (Figure 6.6).

Fix 2 — class weights. Resampling changes which rows exist. Class weights leave every row where it is and change how much a mistake on that row costs during training, using the inverse-frequency formula above.



Bar widths are schematic, not to scale. Resample the training set only — never the test set.

Figure 6.6 – Two ways to rebalance the same 5,000-row training set. Oversampling copies each of the 50 positive rows 99 times ($50 \times 99 = 4,950$), giving 9,900 rows; undersampling keeps only 50 of the 4,950 negative rows, giving 100 rows.

WHAT TO OBSERVE

Both options end at 1-to-1, but at different prices: oversampling repeats the same 50 patients 99 times each, so the model may memorize them, while undersampling throws away 4,900 real records. SMOTE (Chawla, Bowyer, Hall and Kegelmeyer, 2002) instead invents new minority examples partway between a real one and its nearest same-class neighbor. Either way, split first and resample only the training set; a copy of one patient landing in both sets makes the test score dishonestly optimistic (Chapter 5).

WORKED EXAMPLE 6.7 — Find a Class Weight by Inverse Frequency

The training set has 50 positive and 4,950 negative patients. Find the inverse-frequency weight for the positive class, then compare the weighted and unweighted cost of 2 false negatives and 5 false positives.

1. $w_{\text{positive}} = 4950 \div 50 = 99$; $w_{\text{negative}} = 1$.
2. Unweighted cost: $2 + 5 = 7$ mistakes, all counted equally.
3. Weighted cost: $2 \times 99 + 5 \times 1 = 198 + 5 = 203$.

Answer: The positive class gets a weight of 99, the same number as the copies made by oversampling. The 2 missed patients now account for 198 of the 203 units of cost — exactly the pressure that pushes a model to stop missing positives.

Fix 3 — move the threshold. The third fix works on a model that is already trained. Ten patients receive predicted probabilities of 0.05, 0.10, 0.15, 0.22, 0.30, 0.40, 0.55, 0.60, 0.75 and 0.90 (patients 1 to 10); patients 5, 7, 9 and 10 truly have the disease. Figure 6.7 lines them up.

WORKED EXAMPLE 6.8 — Reclassify the Ten Patients at Thresholds 0.5 and 0.2

Using the ten probabilities above, find TP, FN, FP, TN, recall and precision at a threshold of 0.5, then at 0.2.

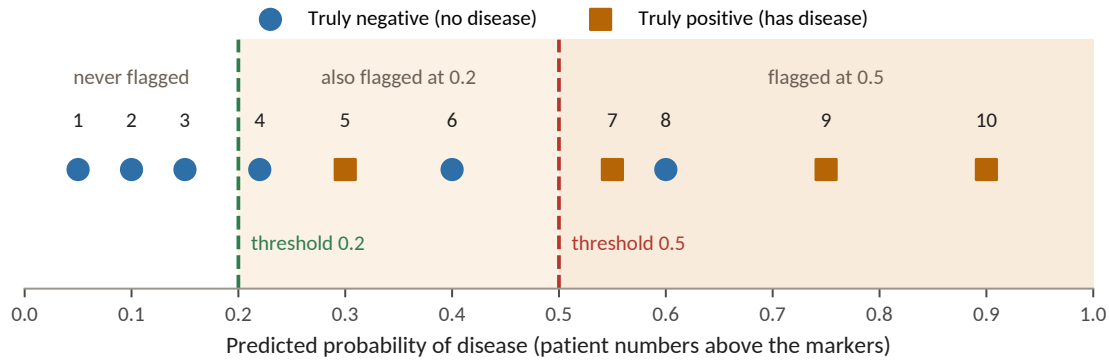


Figure 6.7 – Ten patients placed by predicted probability of disease. Orange squares truly have the disease. The dashed lines are the thresholds 0.5 and 0.2; everything to the right of a line is flagged at that threshold.

WHAT TO OBSERVE

At 0.5 the flagged group is patients 7 to 10, and sick patient 5 is missed. Sliding the line left to 0.2 catches patient 5 but drags healthy patients 4, 6 and 8 along. Threshold moving cannot create information the model does not have; it only decides which mistake to make more often.

1. Threshold 0.5 flags patients 7, 8, 9 and 10: three are sick (TP = 3), patient 8 is not (FP = 1). Of the unflagged, patients 1, 2, 3, 4 and 6 are healthy (TN = 5) and patient 5 is sick but missed (FN = 1). Recall = $3 \div 4 = 0.75$; precision = $3 \div 4 = 0.75$.
2. Threshold 0.2 flags patients 4 to 10: four are sick (TP = 4), patients 4, 6 and 8 are not (FP = 3). The unflagged patients 1, 2 and 3 are healthy (TN = 3, FN = 0). Recall = $4 \div 4 = 1.00$; precision = $4 \div 7 \approx 0.571$.

Answer: Lowering the threshold from 0.5 to 0.2 raises recall from 75% to 100% and lowers precision from 75% to about 57%, because three healthy patients are also flagged. For a screening program that is usually a good trade.

RESPONSIBLE USE

In a screening program a missed case can mean an illness goes untreated until it is harder to help, while a false alarm usually means one more test and some worry. Choose the threshold with the people closest to the decision, never by keeping 0.5 out of habit, and treat a flag as a reason for a follow-up test and a clinician's judgment, never as a diagnosis.

6.7 Multi-Class Confusion Matrices and Averaging

Suppose the utility upgrades its model to predict one of three pump conditions: *Normal*, *Worn* (schedule service soon) and *Failing* (send a technician now). The confusion matrix simply grows to three rows and three columns; Figure 6.8 shows it for 60 inspected pumps.

With three classes, precision and recall are computed *per class*, treating that class as positive and everything else as negative: recall for *Worn* is the *Worn* diagonal cell divided by the *Worn* row total, and precision for *Worn* is the same cell divided by the *Worn* column total. The per-class scores are combined in two ways.

Actual Normal	24	4	2	30 pumps recall 0.80
Actual Worn	4	14	2	20 pumps recall 0.70
Actual Failing	0	1	9	10 pumps recall 0.90
	Predicted Normal	Predicted Worn	Predicted Failing	

Figure 6.8 – A three-class confusion matrix for 60 pumps. The diagonal cells are correct predictions; each row adds up to the number of pumps that truly belonged to that class.

WHAT TO OBSERVE

Read a row to see where an actual class “leaks”: of the 20 truly worn pumps, 4 were called Normal (a risky miss) and 2 were called Failing (a harmless early visit). Read a column to see what a prediction is made of: the 13 pumps predicted Failing include 9 that really were, 2 worn and 2 normal. There is no single TP or FP now; each class has its own.

WORDS TO KNOW

Macro average — The plain average of a metric across the classes, so that every class counts equally, however rare.

Micro average — The metric computed from the pooled counts of all classes. For a single-label problem, micro-averaged precision, recall and F1 all equal the accuracy, so it is dominated by the common classes.

In symbols, with K classes and Recall_k the recall of class k ,

$$\text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k,$$

and macro precision and macro F1 are defined the same way. `scikit-learn`’s `classification_report` prints the per-class scores, the macro average, and a “weighted” average in which each class counts in proportion to its size.

WORKED EXAMPLE 6.9 — Compute Per-Class Recall and the Macro Average for the Three-Class Model

Using Figure 6.8, find the recall of each class, the macro-averaged recall, and the accuracy.

1. Normal: 24 correct out of $24 + 4 + 2 = 30$, so recall = $24 \div 30 = 0.80$.
2. Worn: 14 correct out of $4 + 14 + 2 = 20$, so recall = $14 \div 20 = 0.70$.
3. Failing: 9 correct out of $0 + 1 + 9 = 10$, so recall = $9 \div 10 = 0.90$.
4. Macro-averaged recall: $(0.80 + 0.70 + 0.90) \div 3 = 0.80$.
5. Accuracy (the micro average): $24 + 14 + 9 = 47$ correct out of 60, so $47 \div 60 \approx 0.783$.

Answer: Per-class recall is 0.80, 0.70 and 0.90; the macro average is 0.80 and the accuracy about 0.78. The macro average gives the 10 failing pumps the same voice as the 30 normal ones, which is what the utility wants.

6.8 Calibration: Do Probabilities Mean What They Say?

A model can be excellent at *ranking* pumps from most to least at risk, with a high AUC, and still be wrong about what its probability numbers mean: “a 70% chance of failing” is only useful if pumps with such a score really do fail about 70% of the time.

WORD TO KNOW

Calibration — How well a model’s predicted probabilities match real-world frequencies. A model is well calibrated if, among all the cases where it predicts a 70% chance, the event really happens about 70% of the time. A *reliability diagram* shows this check for several probability buckets at once.

EQUATION: EXPECTED POSITIVES IN A CALIBRATION BUCKET

$$E = \bar{p} \times n_{\text{bucket}}$$

In words: If the model is well calibrated, the number of positives to expect in one bucket equals the bucket’s average predicted probability times the number of examples in it.

When it is used: After training, on validation data: group predictions into buckets by probability, compare the expected count with the observed count in each bucket, and plot the result.

Symbol	Meaning	Unit or type
E	expected number of positives in the bucket	count
$\bar{p}$	average predicted probability in the bucket	0 to 1
n_{bucket}	number of examples in the bucket	count

Dividing both sides by n_{bucket} gives the same test as fractions: observed fraction of positives versus $\bar{p}$.

The utility checks its failure model on 200 validation inspections in five probability buckets (0.0–0.2 up to 0.8–1.0) holding 80, 50, 30, 20 and 20 pumps, with mean predicted probabilities 0.10, 0.30, 0.50, 0.70 and 0.90; in fact 6, 12, 12, 10 and 14 of them failed.

WORKED EXAMPLE 6.10 — Check Whether the “70%” Bucket Is Well Calibrated

Twenty pumps received predicted probabilities between 0.6 and 0.8, averaging 0.70, and ten of them actually failed. Find the expected number of failures if the model is well calibrated, the observed fraction, and the gap.

1. Expected failures: $E = 0.70 \times 20 = 14$.
2. Observed fraction: $10 \div 20 = 0.50$, or 50%.
3. Gap: $70 - 50 = 20$ percentage points.

Answer: The bucket is not well calibrated: the model claims a 70% chance of failure, but only 50% of these pumps failed. A technician who took the 70% at face value would overestimate the risk; the model is overconfident here.

Doing this for every bucket and plotting observed fraction against mean predicted probability gives the reliability diagram in Figure 6.9, an idea from weather forecasting, where Allan Murphy and Robert Winkler used such diagrams in 1977 to check whether “70% chance of rain” forecasts were honest.

Expect this after oversampling or class weighting: both make the positive class look more common during training than it is in the real world, which pushes probabilities upward. After either fix, recalibrate, or use the outputs only as a ranking and a threshold.

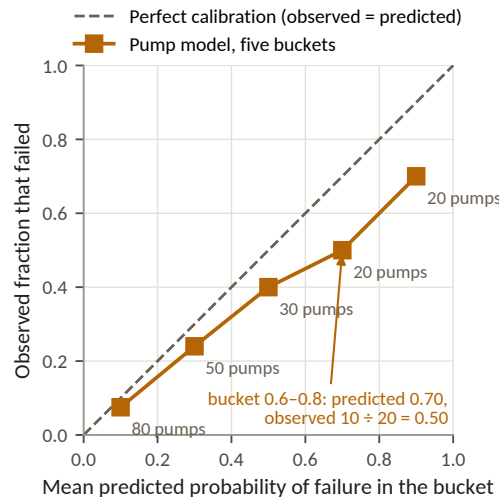


Figure 6.9 – Reliability diagram for the pump-failure model on 200 validation inspections. Each square is one bucket; the dashed diagonal is perfect calibration.

WHAT TO OBSERVE

Every square sits below the diagonal: in every bucket fewer pumps failed than the probabilities promised (observed 0.075, 0.24, 0.40, 0.50 and 0.70 against predicted 0.10, 0.30, 0.50, 0.70 and 0.90), and the gap widens as the probabilities grow. Points below the line mean an

overconfident model; points above it would mean an underconfident one. The ranking is still fine, but the raw numbers should not be shown as literal chances until they are recalibrated, for example with the method John Platt proposed in 1999, which fits a simple curve mapping scores onto honest probabilities.

COMMON MISTAKES AND MISCONCEPTIONS

- Judging a classifier by accuracy alone, especially when the positive class is rare, or choosing the metric only after seeing which model it favors.
- Reading only the diagonal cells of a confusion matrix and ignoring what each false negative and false positive means in the real project.
- Assuming precision and recall always rise and fall together, or using the two words interchangeably.
- Comparing MAE or RMSE across different targets or units, as if the smaller number always means the better model.
- Resampling the whole dataset before splitting it, so that copies of one row land in both the training and the test set.
- Treating a model's probabilities as literal chances without checking calibration, especially after class weighting or resampling.

Practical Activities

ACTIVITY (PAPER) — Fill a Confusion Matrix by Hand

Twenty pumps were checked after the model's prediction. Each entry is *actual / predicted*, where F means "will fail" and N "normal":

1: F/F 2: F/N 3: N/N 4: N/F 5: F/F 6: N/N 7: F/F 8: F/F 9: N/N 10: N/F
11: F/N 12: N/N 13: N/N 14: F/F 15: N/N 16: F/F 17: N/F 18: N/N 19: N/N 20: N/N

1. Draw an empty 2×2 confusion matrix (actual classes as rows, predicted as columns) and add a tally mark to the right cell for each pump. Write the totals TP, FN, FP and TN and check that they add up to 20.
2. Compute accuracy, precision, recall, specificity and F1, showing every step.
3. The utility says a missed failure is far worse than a wasted trip. Which of your numbers should it look at first, and is the model doing well by that measure?
4. Check: TP = 6, FN = 2, FP = 3, TN = 9; accuracy 0.75, precision 0.67, recall 0.75, specificity 0.75, F1 0.71.

You need: Paper, pencil and a calculator

ACTIVITY (PYTHON) — The Same Numbers with scikit-learn

The program below repeats the arithmetic of Worked Examples 6.1 to 6.5 with scikit-learn.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score,
                             confusion_matrix, classification_report,
                             roc_curve, roc_auc_score)

# Lesson 6.2: days until maintenance (Examples 6.1 and 6.2)
y_true, y_pred = [10, 25, 15, 30], [12, 20, 18, 26]
print("MAE:", mean_absolute_error(y_true, y_pred),
      " RMSE:", round(np.sqrt(mean_squared_error(y_true, y_pred)), 3))
print("R2 :", r2_score([10, 20, 30, 40, 50], [12, 18, 33, 38, 49]))

# Lessons 6.3-6.4: the 50 inspections (1 = will fail, 0 = normal)
actual = np.array([1] * 10 + [0] * 40)
flag = np.array([1] * 9 + [0] * 1 + [1] * 3 + [0] * 37)
print(confusion_matrix(actual, flag, labels=[1, 0])) # row 1 = actual fail
print(classification_report(actual, flag, target_names=["Normal", "Will fail"]))

# Lesson 6.5: ten pumps with predicted probabilities
p = np.array([0.08, 0.15, 0.25, 0.35, 0.40, 0.55, 0.62, 0.70, 0.82, 0.93])
failed = np.array([0, 0, 0, 1, 0, 1, 0, 1, 1, 1])
fpr, tpr, thr = roc_curve(failed, p) # the staircase of Figure 6.4
print("FPR:", fpr[1:], " TPR:", tpr[1:])
print("AUC:", roc_auc_score(failed, p))
```


Output

```
MAE: 3.5 RMSE: 3.674
R2 : 0.978
[[ 9  1]
 [ 3 37]]

              precision    recall  f1-score   support

   Normal         0.97         0.93         0.95         40
   Will fail       0.75         0.90         0.82         10

 accuracy                   0.92         50
 macro avg         0.86         0.91         0.88         50
 weighted avg       0.93         0.92         0.92         50

FPR: [0.  0.  0.2 0.2 0.4 0.4 1. ] TPR: [0.2 0.6 0.6 0.8 0.8 1.  1. ]
AUC: 0.88
```

`labels=[1, 0]` makes scikit-learn print the positive class first so that the matrix matches Figure 6.2; the “Will fail” row of `classification_report` gives precision 0.75, recall 0.90 and F1 0.82; and `roc_curve` lists the corners of the staircase in Figure 6.4. Every number matches the hand calculations.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Detecting Faulty Water Pumps from Sensor Data — Rural Punjab

The water utility from this chapter’s running example serves more than three hundred villages, most of them relying on a single shared pump, so a sudden failure can mean a full day’s walk to the next working water source. With a small maintenance team and a large territory, the engineers build the sensor-based failure model and grade it as this chapter recommends. They begin by writing down what each mistake would cost: a missed failure could leave a village without water for days, while a false alarm costs one wasted visit. Recall therefore matters more than precision, though precision cannot be ignored, because a technician who receives constant false alarms will stop trusting the system. The team tracks recall, precision and F1 together and treats accuracy as almost meaningless, since a lazy “always Normal” model would score above 90% without catching anything.

Using cross-validation (Chapter 5) on two years of sensor readings, the team compares three candidate models, keeps the one with the strongest and most consistent recall, chooses a threshold below 0.5 on validation data, and only then opens the untouched test set once. The final model reaches a recall of 90% and a precision of 75%, the numbers of this chapter’s worked examples: it finds 9 of every 10 pumps that are truly about to fail, at the cost of about one unnecessary visit for every three real warnings. A calibration check shows the class-weighted model is overconfident, so technicians see a three-level flag — low, medium, high — rather than a raw percentage. Three months after launch, flagged pumps do fail noticeably more often than pumps chosen at random, but recall drops during the monsoon season, when water pressure across the network becomes unpredictable. Rather than hiding this weakness, the team documents it, plans to retrain with a full monsoon season of data, and explains the model to village councils in plain words: “it finds about 9 out of every 10 pumps that are about to fail, and sometimes sends a technician who finds nothing wrong.”

The disease-screening program in Chengdu from Lesson 6.6 learned the same lesson. Its first model reported 99% accuracy, and a junior analyst nearly celebrated until a senior colleague asked how many truly sick patients it had actually caught: almost none. The team started over with class weights, a threshold chosen on validation data together with the clinic’s physicians, and a calibration check before any probability was shown to a doctor. In both projects the model’s flag is a reason for a human to take a closer look, never a decision by itself.

CHAPTER SUMMARY

- A metric is a statement about which mistakes matter; choose it from the real cost of each mistake, before comparing models.
- MAE is the average absolute error in the target's unit; MSE squares the errors, punishing large ones; RMSE is the square root of MSE; R^2 compares the model with the mean baseline.
- A confusion matrix sorts every classification into TP, FN, FP or TN. Precision = $TP/(TP + FP)$ asks how often an alarm is right; recall = $TP/(TP + FN)$ asks how many true cases were found; specificity is recall's mirror for negatives; F1 stays high only when precision and recall are both high.
- A threshold turns a probability into a decision; lowering it raises recall and lowers precision. The ROC curve plots TPR against FPR for every threshold and AUC summarizes it; the precision-recall curve is more informative when positives are rare.
- On imbalanced data accuracy can be high for a useless model: check the imbalance ratio, then use recall, precision and F1. Resampling and class weights rebalance training; threshold moving works afterward; never resample before the split.
- A multi-class matrix has one row and column per class; macro averaging gives every class an equal vote, micro averaging (accuracy) favors the common classes. Calibration asks whether a predicted 70% really happens 70% of the time; a reliability diagram shows it, and class weighting or resampling usually breaks it.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define *metric* in your own words, and explain in one or two sentences why the metric should be chosen before any models are compared.

Problem 2 [Basic] Without looking back at the chapter, define TP, FP, FN and TN in your own words, and say which two are mistakes.

Problem 3 [Basic] A classifier has $TP = 30$, $TN = 50$, $FP = 10$ and $FN = 10$. Find its accuracy, showing every step.

Problem 4 [Basic] True values are 20, 30, 50 and 40; predicted values are 18, 34, 45 and 44. Find the MAE, the MSE and the RMSE, showing every step.

Problem 5 [Basic] A dataset has 4,500 majority-class examples and 500 minority-class examples. Find the imbalance ratio, and state what accuracy a model that always predicts the majority class would reach.

Problem 6 [Standard] Using $TP = 30$, $TN = 50$, $FP = 10$ and $FN = 10$ from Problem 3, calculate precision, recall, specificity and F1, showing every step.

Problem 7 [Standard] True values are 4, 6, 8 and 10; predicted values are 5, 5, 9 and 9. Find the mean of the true values, SSE, SST and R^2 , showing every step.

Problem 8 [Standard] A training set has 50 positive and 750 negative examples. Find the inverse-frequency class weight for the positive class, and the new total size of the training set after (a) oversampling the positive class to match the negative class and (b) undersampling the negative class to match the positive class.

Problem 9 [Standard] A lazy model always predicts "negative" on 2,000 examples, of which 30 are truly positive. Find its TP, FN, FP, TN and accuracy. A real model on the same data has $TP = 24$, $FN = 6$, $FP = 150$ and $TN = 1,820$; find its accuracy and recall, and say which model a hospital should prefer and why.

Problem 10 [Standard] At a chosen threshold a model flags 10 patients as positive; 6 of them are truly positive and 4 are not. There are 8 truly positive patients in total. Find recall and precision, say what would happen to each if the threshold were lowered, and describe one situation where the lower threshold is the safer choice.

Problem 11 [Challenge] A bucket of 50 patients all received a predicted probability near 0.40. (a) Find the expected number of true positives if the model is perfectly calibrated. (b) If only 15 of the 50 were truly positive, find the observed fraction and the calibration gap. (c) Is the model overconfident or underconfident here? Explain.

Problem 12 [Challenge] Six pumps receive probabilities 0.1, 0.2, 0.4, 0.6, 0.7 and 0.9; the pumps with 0.4, 0.7 and 0.9 truly failed. (a) List every (failing, healthy) pair and count those in which the failing pump has the higher probability. (b) Use the count to find the AUC. (c) Find TPR and FPR at threshold 0.5 and mark that point on a sketch of the ROC axes.

Overfitting, Underfitting, and Regularization

Why a model that is perfect on training data can fail on new data, and how to stop it

IN THIS CHAPTER YOU WILL LEARN TO

- Tell underfitting and overfitting apart, and recognize the “too simple, too flexible, just right” pattern in a fitted model.
- Compute a model’s mean squared error on training and on validation data by hand, and read the gap between the two.
- Measure bias and variance from a small table of predictions, and add them to the noise to get a model’s expected error.
- Explain what a hyperparameter is, and read a loss curve and a learning curve to decide whether the next step is more data, more capacity or less.
- Compute a regularized loss by hand, and choose the penalty strength λ with a validation set.

Introduction

Two students in Sulaymaniyah prepare for the same driving-theory test. The first memorizes last year’s question sheet word for word, including the numbers in every question. The second learns the rules of the road. On last year’s sheet, the first student scores a perfect 100%. On the real test, which uses new questions, she fails, while the second student passes comfortably. The first student did not learn the rules; she learned one particular sheet of paper.

A machine learning model can make exactly the same mistake: it can fit the examples it trained on so closely that it copies their accidents and their noise, and then fails on the first new example it meets. The opposite failure is also possible: a model can be so simple that it misses a pattern that is plainly visible in the data, like a student who learned only “always slow down.” This chapter is about the space between those two failures, and every later chapter leans on it. You will learn to recognize both failures, measure them with training and validation errors, explain them with the words *bias* and *variance*, read the curves that reveal them, and control them with a dial called regularization — by hand first, on a tiny dataset of wheat fields, and then in Python.

7.1 Two Ways a Model Can Fail

An agricultural researcher near Duhok studies wheat fields. For each field she records the total rainfall during the growing season, in millimeters, and the field’s final yield, in tons per hectare (t/ha). More rainfall generally means a higher yield, but the points never sit on one perfect line, because real fields differ in soil, timing and luck. Table 7.1 shows eight of her fields. She trains models on the first five and keeps the last three hidden, as a validation set (Chapter 5), to check the models afterward.

Table 7.1 – Eight wheat fields near Duhok: five for training, three for validation (illustrative numbers)

Field	Rainfall (mm)	Yield (t/ha)	Used for
1	300	2.1	Training
2	350	2.8	Training
3	400	3.0	Training
4	450	3.2	Training
5	500	3.9	Training
6	325	2.2	Validation
7	425	3.1	Validation
8	475	3.8	Validation

Every model in this book is built the same way: a learning algorithm studies a training set and produces a model. But the real test is never how well the model explains the training set it has already seen; it is how well it handles a brand-new example — a new patient, a new customer, a new field of wheat.

WORD TO KNOW

Generalization — A model's ability to perform well on new examples it has never seen, not only on the examples it trained on. Generalization, not training performance, is the true goal of every machine learning project.

A model can fail to generalize in two opposite ways. It can be too simple to capture the real pattern in the data, or it can cling too closely to the exact training data it happened to see.

WORDS TO KNOW

Underfitting — Failing to learn enough of the real pattern, so that the model performs poorly even on the training data. An underfit model is usually too simple for the problem.

Overfitting — Learning the noise and small accidents of the training data so well that the model performs worse on new data. An overfit model has memorized its training examples instead of learning a general rule.

Loss — The number a learning algorithm tries to make as small as possible during training; it measures how badly the predictions miss the known answers. In this chapter the loss is the mean squared error (MSE).

Picture three models fitted to the five training fields, shown in Figure 7.1. The *Flat model* ignores rainfall and predicts the same yield, 3.0 t/ha, for every field. The *Line model* is a straight line that rises with rainfall: predicted yield = $0.008 \times \text{rainfall} - 0.2$, so each extra 100 mm of rain adds 0.8 t/ha. (It is exactly the least-squares line that Chapter 8 teaches you to compute.) The *Wiggly model* is a very flexible curve, free to bend as much as it likes, that passes exactly through every training field.

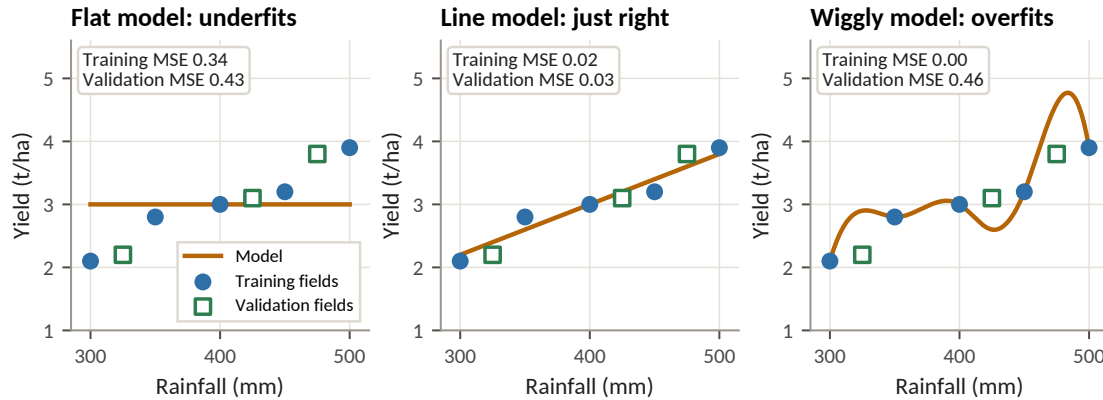


Figure 7.1 – Three models fitted to the same five training fields (blue dots). The hollow green squares are the three validation fields, which no model saw during training.

WHAT TO OBSERVE

The flat line misses the obvious upward trend: it underfits. The wiggly curve touches every blue dot, yet between the dots it swings through empty space and misses the green validation squares badly: it overfits. The straight line touches no dot exactly but stays close to all eight fields: too straight, too wiggly, just right.

To turn the picture into numbers we need one way of scoring a model: a loss. This chapter uses the mean squared error from Chapter 6, because it is what most learning algorithms actually minimize during training and because, as Lesson 7.3 shows, it splits into meaningful parts. In words: take every prediction's error, square it, add the squares up, and divide by the number of examples. Squaring removes the minus signs and punishes big misses more than small ones.

EQUATION: MEAN SQUARED ERROR AS THE LOSS

$$L = \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In words: Subtract each prediction from the true value, square the difference, add all the squares together, and divide by the number of examples.

When it is used: To score a regression model during training (on the training set) and afterward (on the validation or test set). Comparing the two scores reveals underfitting and overfitting.

Symbol	Meaning	Unit or type
L	the loss, here the mean squared error	squared unit of y , e.g. (t/ha) ²
n	number of examples being scored	count
$y_i, \hat{y}_i$	true value and model's prediction for example i	unit of the target

$\sqrt{\text{MSE}}$ (the RMSE of Chapter 6) brings the error back to t/ha. An MSE of 0 means every prediction was exactly right.

WORKED EXAMPLE 7.1 — Compare Three Models Using Only Training Data

Using the five training fields of Table 7.1 (true yields 2.1, 2.8, 3.0, 3.2 and 3.9 t/ha), the Flat model predicts 3.0 for every field. The Line model, predicted yield = $0.008 \times \text{rainfall} - 0.2$, predicts 2.2, 2.6, 3.0, 3.4 and 3.8. The Wiggly model predicts 2.1, 2.8, 3.0, 3.2 and 3.9, exactly the true values. Find the training MSE of each model.

1. Flat model errors, true minus predicted: $2.1 - 3.0 = -0.9$, $2.8 - 3.0 = -0.2$, $3.0 - 3.0 = 0$, $3.2 - 3.0 = 0.2$, $3.9 - 3.0 = 0.9$.
2. Square them: 0.81, 0.04, 0, 0.04, 0.81. Add: 1.70. Divide by $n = 5$: $1.70 \div 5 = 0.34$.
3. Line model errors: $2.1 - 2.2 = -0.1$, $2.8 - 2.6 = 0.2$, $3.0 - 3.0 = 0$, $3.2 - 3.4 = -0.2$, $3.9 - 3.8 = 0.1$.
4. Square them: 0.01, 0.04, 0, 0.04, 0.01. Add: 0.10. Divide by 5: $0.10 \div 5 = 0.02$.
5. Wiggly model: every prediction equals the true yield, so all five errors are 0 and the MSE is $0 \div 5 = 0$.

Answer: Training MSE: Flat 0.34, Line 0.02, Wiggly 0.00 (t/ha)². Judged by training error alone the Wiggly model is the perfect winner; Lesson 7.4 shows that this judgment is wrong.

COMMON MISTAKE

✗ Wrong: Deciding that a model is the best choice because it has the lowest error, or the highest accuracy, on the training data it learned from.

✓ Right: A perfect or near-perfect training score is not proof of a good model; it is just as often a warning sign of overfitting. Always check a separate validation set before trusting a training score.

7.2 Bias and Variance

Underfitting and overfitting describe what a model's mistakes look like from the outside. Bias and variance describe *why* those mistakes happen, and they are the words the rest of this book uses for a model's weaknesses. A schoolyard game in Zakho makes both ideas easy to picture. A target is drawn on a wall, and four children each throw nine small stones at the bullseye. Figure 7.2 shows their results.

WORDS TO KNOW

Bias — A model's tendency to make the same kind of mistake, in the same direction, again and again, because its assumptions are too simple for the real pattern. High bias shows up as underfitting.

Variance — A model's tendency to change a great deal when it is trained on a different sample of the same kind of data. High variance shows up as overfitting.

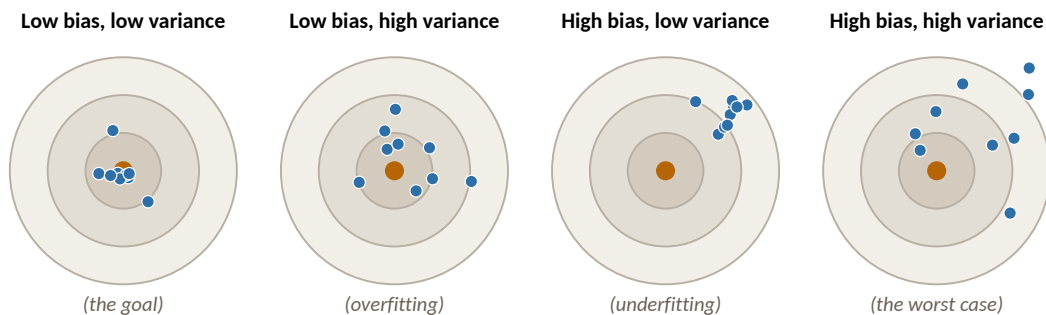


Figure 7.2 – Four throwers, four mixtures of bias and variance. The orange dot is the bullseye (the truth); the blue dots are the throws (predictions from models trained on different samples).

WHAT TO OBSERVE

The first thrower is tight and centered: low bias, low variance, the goal. The second is centered on average but scattered everywhere: low bias, high variance, the overfit model. The third is tight but consistently off to one side: high bias, low variance, the underfit model; her stones are close to *each other*, not to the target. The fourth has both problems.

The Flat model predicted 3.0 t/ha for every field, so it was too low for every wet field and too high for every dry field, every single time: high bias, a systematic mistake caused by a model too simple to represent the rising pattern. Variance shows up differently. Imagine four research teams, each sampling its own set of wheat fields near Duhok in the same season, so that the four datasets are similar but not identical. Each team fits a Flat, a Line and a Wiggly model to its own fields, and every model then predicts the yield of one particular test field that received 500 mm of rain. Table 7.2 shows the twelve predictions.

Table 7.2 – Predictions for one test field (500 mm of rain) by models trained on four different samples (illustrative)

Model	Team 1 (t/ha)	Team 2 (t/ha)	Team 3 (t/ha)	Team 4 (t/ha)
Flat	2.8	3.0	3.2	3.0
Line	3.5	3.9	4.1	3.7
Wiggly	3.2	4.6	3.4	4.0

To measure bias, average the m predictions, $\bar{\hat{y}} = \frac{1}{m} \sum_{j=1}^m \hat{y}^{(j)}$, and subtract the true value: $\text{bias} = \bar{\hat{y}} - y^*$. To measure variance, look at how far the individual predictions spread around their own average, exactly as for the variance of any list of numbers in Chapter 4:

$$\text{variance} = \frac{1}{m} \sum_{j=1}^m \left(\hat{y}^{(j)} - \bar{\hat{y}} \right)^2,$$

that is, subtract the average from each prediction, square, and take the mean of the squares.

7.3 The Bias-Variance Trade-off

Lesson 7.2 sorted the models into extremes: the Flat model is consistently wrong in one direction, the Wiggly model is right on average but wildly unstable. Real projects are rarely so extreme. Choosing a model usually means choosing how much of each problem to accept, because you almost never get to erase both at once.

WORDS TO KNOW

Bias-variance trade-off — The pattern where lowering a model's bias, usually by giving it more capacity, tends to raise its variance, and lowering its variance tends to raise its bias. The best model is usually not the simplest one and not the most flexible one, but somewhere in between.

Irreducible noise — Randomness that no model can predict away: two fields with identical rainfall still give slightly different yields. It sets a floor below which no error can fall.

The three ideas fit together in one equation, written down for machine learning by Stuart Geman, Elie Bienenstock and René Doursat in 1992. A model's expected squared error on a new example is the sum of three separate parts: its bias squared, its variance, and the irreducible noise.

EQUATION: TOTAL ERROR HAS THREE PARTS

$$\text{expected error} = \text{bias}^2 + \text{variance} + \sigma_{\text{noise}}^2$$

In words: A model's expected squared error on new data equals how wrong it is on average, squared, plus how much its predictions swing from one training sample to another, plus the randomness that no model could ever remove.

When it is used: As a way of thinking whenever you choose between a simpler and a more flexible model: the winner is the one with the smallest sum, not the smallest single part.

Symbol	Meaning	Unit or type
bias	average prediction minus the true value, $\bar{\hat{y}} - y^*$; squared so it cannot cancel out	unit of y
variance	spread of the predictions across samples, $\frac{1}{m} \sum_j (\hat{y}^{(j)} - \bar{\hat{y}})^2$	squared unit of y
σ_{noise}^2	variance of the irreducible noise in the target	squared unit of y

Pushing bias toward zero tends to push variance up, and the other way around. The noise term is the same for every model, so it never changes the ranking.

WORKED EXAMPLE 7.2 — Measure Bias and Variance Across Samples and Rank the Three Farm Models

Long records show that a field receiving 500 mm of rain yields 3.8 t/ha on average, with season-to-season randomness of about ± 0.2 t/ha, that is, a noise variance of $0.2^2 = 0.04$. Using the twelve predictions of Table 7.2, find each model's average prediction, variance, bias and expected error, then rank the models.

1. Flat model: average $(2.8 + 3.0 + 3.2 + 3.0) \div 4 = 3.0$; deviations $-0.2, 0, 0.2, 0$; squares $0.04, 0, 0.04, 0$; variance $= 0.08 \div 4 = 0.02$.
2. Line model: average $(3.5 + 3.9 + 4.1 + 3.7) \div 4 = 15.2 \div 4 = 3.8$; deviations $-0.3, 0.1, 0.3, -0.1$; squares $0.09, 0.01, 0.09, 0.01$; variance $= 0.20 \div 4 = 0.05$.
3. Wiggly model: average $(3.2 + 4.6 + 3.4 + 4.0) \div 4 = 15.2 \div 4 = 3.8$; deviations $-0.6, 0.8, -0.4, 0.2$; squares $0.36, 0.64, 0.16, 0.04$; variance $= 1.20 \div 4 = 0.30$.
4. Bias, average minus 3.8: Flat $3.0 - 3.8 = -0.8$, so $\text{bias}^2 = 0.64$; Line $3.8 - 3.8 = 0$; Wiggly 0 .
5. Expected error, $\text{bias}^2 + \text{variance} + \text{noise}$: Flat $0.64 + 0.02 + 0.04 = 0.70$; Line $0 + 0.05 + 0.04 = 0.09$; Wiggly $0 + 0.30 + 0.04 = 0.34$.
6. Rank by bias (highest first): Flat, then Line and Wiggly tied at zero. By variance: Wiggly 0.30, Line 0.05, Flat 0.02. By expected error (lowest first): Line 0.09, Wiggly 0.34, Flat 0.70.

Answer: The Flat model's predictions barely move between samples (variance 0.02) but sit 0.8 t/ha too low; the Wiggly model is right on average but swings fifteen times more (variance 0.30). The Line model wins with an expected error of 0.09 (t/ha)², even though it has neither the lowest bias (a tie) nor the lowest variance: it is the only model that keeps both parts small at the same time. No model can go below the noise floor of 0.04.

Figure 7.3 shows the same three parts measured in a computer simulation, where the capacity dial is the degree of a polynomial curve fitted to 20 noisy fields at a time: degree 0 is the Flat model, degree 1 the Line model, and higher degrees bend more. Panel (a) splits the error into parts that only a simulation can see; panel (b) shows the two numbers a real project can measure, the picture Lesson 7.4 builds by hand.

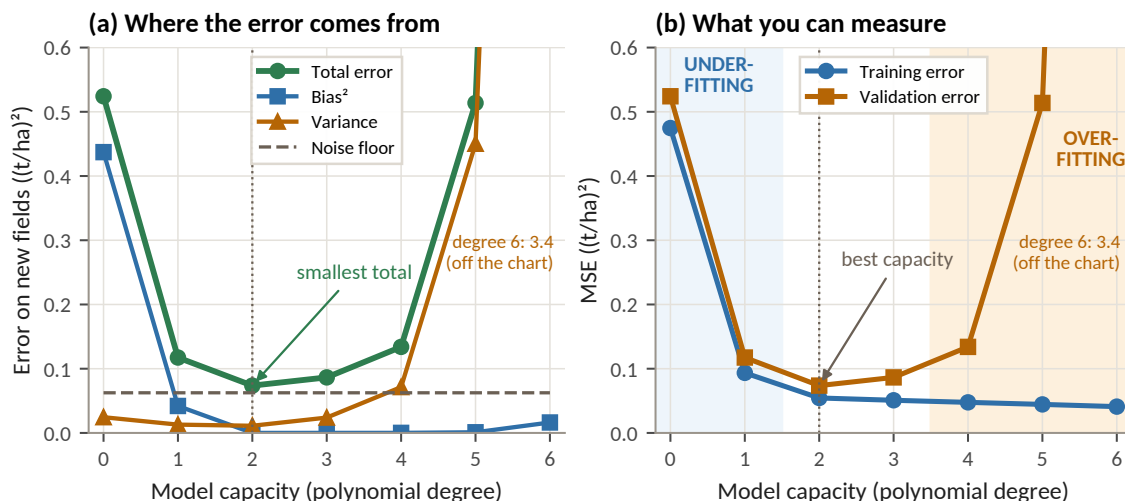


Figure 7.3 – The bias-variance trade-off against model capacity, averaged over 400 simulated training sets of 20 wheat fields. (a) Bias squared, variance, noise and their total. (b) Training and validation error, with the two failure zones shaded.

WHAT TO OBSERVE

In (a), bias squared falls quickly and is essentially zero from degree 2 onward, while variance is small at first and then explodes: at degree 5 it is seven times the noise floor. The total is U-shaped, and its bottom, at degree 2, is where the *sum* is smallest, not where either part is. In (b), the training error only ever goes down, while the validation error follows the U of panel (a): where both errors are high and close, the model underfits; where they pull apart, it overfits.

7.4 Hyperparameters and Model Capacity

A tailor in Erbil keeps one adjustable jacket pattern with a dial that sets how closely the cut follows the customer's body. Turned all the way down, every jacket comes out the same loose shape; turned all the way up, the jacket follows every bump of the one customer who was measured and fits nobody else. Every model in this book has such a dial, and a person sets it *before* training starts.

WORDS TO KNOW

Hyperparameter — A setting chosen before training begins, rather than a value the learning algorithm discovers from the data. Changing a hyperparameter changes how a model learns, including how much freedom it has to bend.

Model capacity — How wide a range of patterns a model is able to represent. A low-capacity model can only represent simple patterns, such as a straight line; a high-capacity model can represent complicated, wiggly ones.

Generalization gap — The validation error minus the training error. A large gap means the model does much better on the examples it memorized than on new ones: the signature of overfitting.

A hyperparameter is different from the numbers a model learns during training, such as the slope of the Line model or a centroid in Chapter 1: those are chosen automatically by the algorithm, while a hyperparameter is chosen beforehand, by a person or by the systematic search of Chapter 23. Most familiar hyperparameters are capacity dials in disguise. The degree of a polynomial curve (Chapter 8), the maximum depth of a decision tree (Chapter 13) and the number of boosting rounds (Chapter 15) raise capacity as they grow; the number of neighbors k (Chapter 11) works the other way, since a *smaller* k lets each prediction react to fewer, closer

points. Every one of them answers the same question: how much freedom should this model have to bend toward the training data?

In Lesson 7.1 the capacity dial had three positions: Flat (capacity level 1), Line (level 2) and Wiggly (level 3), and training error fell as capacity rose. What matters is what happens on the three hidden validation fields.

WORKED EXAMPLE 7.3 — Reveal the Validation Error for Three Capacity Levels

The three models are now scored on the validation fields of Table 7.1: Field 6 (325 mm, true yield 2.2), Field 7 (425 mm, 3.1) and Field 8 (475 mm, 3.8). The Flat model predicts 3.0 for all three; the Line model predicts $0.008 \times 325 - 0.2 = 2.4$, then 3.2 and 3.6; the Wiggly curve of Figure 7.1 reads 2.9, 2.6 and 4.6 at the three rainfall values. Find each model's validation MSE and its generalization gap.

1. Flat errors: $2.2 - 3.0 = -0.8$, $3.1 - 3.0 = 0.1$, $3.8 - 3.0 = 0.8$. Squares 0.64, 0.01, 0.64; sum 1.29; $\text{MSE} = 1.29 \div 3 = 0.43$.
2. Line errors: $2.2 - 2.4 = -0.2$, $3.1 - 3.2 = -0.1$, $3.8 - 3.6 = 0.2$. Squares 0.04, 0.01, 0.04; sum 0.09; $\text{MSE} = 0.09 \div 3 = 0.03$.
3. Wiggly errors: $2.2 - 2.9 = -0.7$, $3.1 - 2.6 = 0.5$, $3.8 - 4.6 = -0.8$. Squares 0.49, 0.25, 0.64; sum 1.38; $\text{MSE} = 1.38 \div 3 = 0.46$.
4. Gaps, validation minus training (Worked Example 7.1): Flat $0.43 - 0.34 = 0.09$; Line $0.03 - 0.02 = 0.01$; Wiggly $0.46 - 0.00 = 0.46$.

Answer: Training error kept falling with capacity (0.34, 0.02, 0.00) but validation error did not (0.43, 0.03, 0.46). The Wiggly model, perfect in training, is now the worst of the three on new fields, with a gap of 0.46. The Line model wins on the number that matters. Table 7.3 collects the results: it is panel (b) of Figure 7.3, built from numbers you calculated yourself.

Table 7.3 – Training and validation MSE of the three farm models at three capacity levels

Capacity	Model	Training MSE ((t/ha) ²)	Validation MSE ((t/ha) ²)	Gap	Diagnosis
Level 1	Flat	0.34	0.43	0.09	Both high: underfitting (high bias)
Level 2	Line	0.02	0.03	0.01	Both low and close: generalizes well
Level 3	Wiggly	0.00	0.46	0.46	Perfect in training, poor on new fields: overfitting (high variance)

A real project finds the best capacity exactly this way: by trying several settings and comparing validation error, never training error alone. The same pattern appears in classification, where capacity shows in the shape of the decision boundary. Figure 7.4 classifies wheat plants as healthy or infected by rust disease from two features, using three models of increasing capacity.

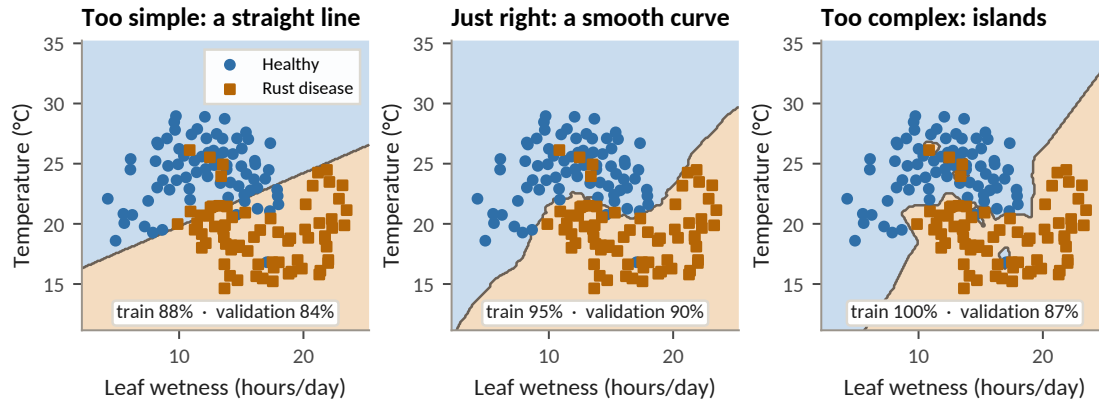


Figure 7.4 – Three decision boundaries for the same 150 training plants (illustrative data), with training and validation accuracy (the validation set holds 500 more plants).

WHAT TO OBSERVE

The straight line cannot follow the curved border between the groups, so it is wrong on both sets (88% and 84%). The smooth curve (k -nearest neighbors with $k = 15$, Chapter 11) does best on new plants, 90%. The jagged model ($k = 1$) draws a private island around every training point: 100% in training, only 87% on new plants. The islands are memorized noise.

7.5 Training vs Validation Loss Curves

Many learning algorithms improve their model gradually, round after round: gradient descent (Chapter 9) takes one small step downhill per iteration, and boosting (Chapter 15) adds one small tree per round. This gives a second dial, turned *during* training: how long to keep going. A student who practices twenty exam questions for one evening learns the method; one who drills the same twenty for a month starts memorizing the printing errors on the sheet.

WORDS TO KNOW

Loss curve — A plot of the loss against the number of training rounds (iterations or epochs). Plotting the training loss and the validation loss on the same axes shows when the model stops learning the pattern and starts learning the noise.

Early stopping — Stopping training at the round where the validation loss is lowest, instead of continuing until the training loss is lowest.

Figure 7.5 shows a boosted-tree model trained round by round on 20 of the researcher's fields (the dataset file `ch07_wheat_fields.csv` used in the activities), with the loss recorded on those 20 training fields and on 10 validation fields after every round.

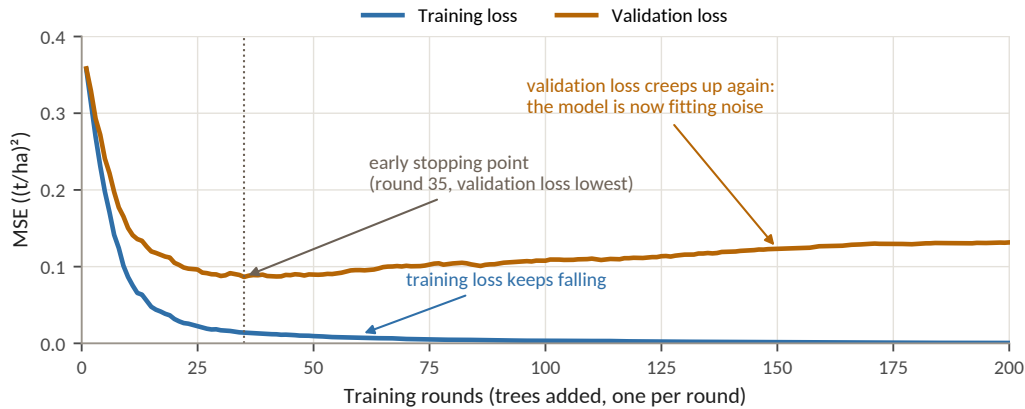


Figure 7.5 – Training and validation loss of a boosted-tree model against the number of training rounds, on 20 training and 10 validation fields.

WHAT TO OBSERVE

Both curves fall steeply at first: the model is learning the real rainfall–yield pattern, which helps on every field. After about 35 rounds they separate. The training loss keeps sliding toward zero (0.017 at round 30, 0.001 at round 200), while the validation loss bottoms out at 0.088 near round 30 and creeps up to 0.132 by round 200, because every further round fits accidents of the 20 training fields. Early stopping freezes the model at the bottom of the orange curve; $0.132 - 0.088 = 0.044$ is what over-training costs.

7.6 Learning Curves: More Data or a Different Model?

Lesson 7.4 changed the capacity while holding the amount of data fixed. This lesson does the opposite: it holds the model fixed and changes the amount of training data, to answer the question every team asks when a validation score disappoints — would collecting more data fix it?

WORD TO KNOW

Learning curve — A plot of training error and validation error against the size of the training set. The shape of the two lines together shows whether a model is underfitting, overfitting or generalizing well, and whether more data would help.

Three patterns come up again and again. If both errors stay high and close together as data is added, the model is underfitting: it lacks the capacity (or the features) to learn the pattern, and more examples cannot supply capacity. If training error is low but validation error is much higher, with the gap narrowing as data is added, the model is overfitting, and more data genuinely helps. If both errors are low and close, the model generalizes well and extra data changes little. Figure 7.6 shows all three, using a pool of 400 simulated fields.

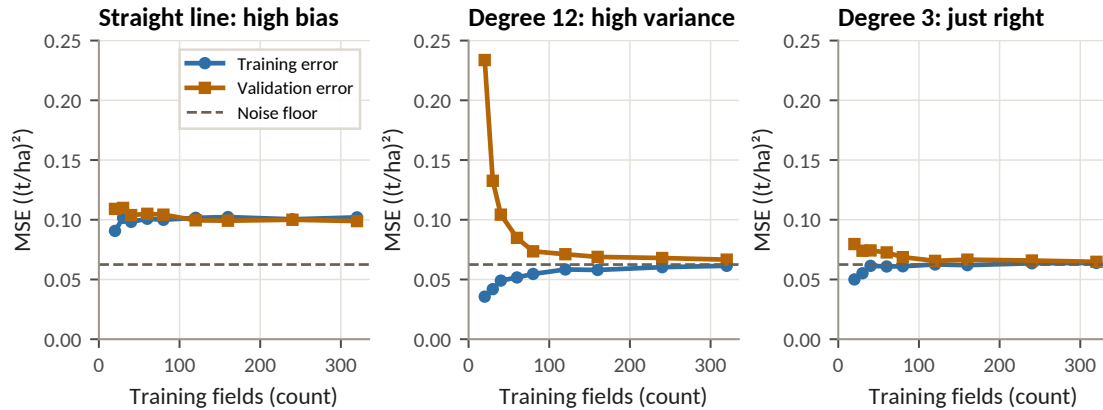


Figure 7.6 – Learning curves for three models of different capacity (median over 30 random splits of a pool of 400 simulated fields). The dashed line is the noise floor, 0.0625 (t/ha)^2 .

WHAT TO OBSERVE

Left: the straight line’s two curves meet almost immediately, but at 0.10, well above the noise floor; more fields will never lower them. Middle: the degree-12 curve starts with a huge gap that closes steadily as fields are added; here more data is exactly the cure. Right: the degree-3 curve reaches the noise floor with about 100 fields and stays there.

Table 7.4 – Learning-curve readings for two models from Figure 7.6, in $(\text{t/ha})^2$

Model	Training fields	Training error	Validation error	Gap
Straight line	20	0.091	0.109	0.018
Straight line	320	0.102	0.099	−0.003
Degree-12 curve	20	0.036	0.234	0.198
Degree-12 curve	320	0.061	0.067	0.006

WORKED EXAMPLE 7.4 — Diagnose Two Learning Curves for the Wheat-Yield Model

Using Table 7.4, diagnose the straight line and the degree-12 curve, and say for which of them collecting more fields is worthwhile.

1. Straight line at 20 fields: gap $0.109 - 0.091 = 0.018$. At 320 fields: gap $0.099 - 0.102 = -0.003$, effectively zero (a tiny negative gap just means the two errors are equal within random noise).
2. Straight line: both errors stay near 0.10 from 20 to 320 fields. Sixteen times more data changed nothing.
3. Degree-12 curve at 20 fields: gap $0.234 - 0.036 = 0.198$. At 320 fields: gap $0.067 - 0.061 = 0.006$. The validation error fell from 0.234 to 0.067 as fields were added.

Answer: The straight line underfits: its small gap looks reassuring, but both errors are stuck at 0.10, well above the noise floor of 0.0625, so it needs more capacity (or better features, Chapter 22), not more data. The degree-12 curve overfits at 20 fields, and more data cured it almost completely — at 320 fields it is nearly as good as the best model. Always check the training error before collecting more data.

The readings come from scikit-learn’s `learning_curve` function, which Part B of the SIMULATION activity uses to produce them.

7.7 Regularization: Paying a Price for Complexity

So far we have fought overfitting by choosing between separate models: Flat, Line or Wiggly. Real projects often prefer a smoother approach: keep one flexible model, capable of wiggling if the data truly demands it, but charge it a price for every unnecessary bend. Think of a toll road between Erbil and Duhok: a driver may take any detour she likes, but every extra kilometer costs money, so she leaves the main road only when it is really worth it.

WORD TO KNOW

Regularization — Adding a penalty, inside training itself, that discourages a model from becoming more complex than the data justifies. The size of the penalty is set by the *regularization strength* λ (lambda), a hyperparameter: $\lambda = 0$ means no penalty, and a large λ forces the model to stay simple.

In practice the penalty is computed from the model's coefficients: large coefficients make steep, wiggly curves, so the most common penalty is simply the sum of the squared coefficients, and training minimizes the fit error plus λ times that sum. This recipe is *ridge regression*, introduced by Arthur Hoerl and Robert Kennard in 1970; Chapter 8 applies it alongside its cousin the *lasso*, proposed by Robert Tibshirani in 1996, which penalizes the sum of the absolute coefficients instead.

EQUATION: REGULARIZED LOSS

$$L_{\lambda} = \underbrace{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{fit error}} + \lambda \underbrace{\sum_{j=1}^p w_j^2}_{\text{penalty}}$$

In words: Training no longer minimizes the fit error alone, but the fit error plus a penalty that grows with the size of the model's coefficients, multiplied by the strength λ chosen beforehand.

When it is used: Whenever a flexible model overfits: regression (Chapter 8), logistic regression (Chapter 10), support vector machines (Chapter 16). Choose λ with a validation set, never with the test set.

Symbol	Meaning	Unit or type
L_{λ}	the regularized loss	squared unit of y
λ	regularization strength, a hyperparameter	no unit
$w_j, \sum_j w_j^2$	the model's p coefficients, and the ridge penalty: the sum of their squares	coefficient units (squared in the sum)

With $\lambda = 0$ training cares only about fitting the training data. As λ grows, training cares more and more about keeping the coefficients small, even at the cost of fitting the training data a little less closely. scikit-learn calls λ `alpha`.

WORKED EXAMPLE 7.5 — Compute the Regularized Loss of Two Candidate Curves

A flexible curve with four coefficients is fitted to the training fields in two ways. Curve A is nearly straight: coefficients (0.8, 0.6, 0, 0), fit error (training MSE) 0.02. Curve B bends through every field: coefficients (1.0, -2.0, 2.0, -1.0), fit error 0.00. Which curve has the lower regularized loss at $\lambda = 0.01$, and which at $\lambda = 0.001$?

1. Penalty of A: $0.8^2 + 0.6^2 + 0^2 + 0^2 = 0.64 + 0.36 = 1.0$.
2. Penalty of B: $1.0^2 + (-2.0)^2 + 2.0^2 + (-1.0)^2 = 1 + 4 + 4 + 1 = 10.0$.
3. At $\lambda = 0.01$: $L_A = 0.02 + 0.01 \times 1.0 = 0.03$; $L_B = 0.00 + 0.01 \times 10.0 = 0.10$. Curve A wins.
4. At $\lambda = 0.001$: $L_A = 0.02 + 0.001 \times 1.0 = 0.021$; $L_B = 0.00 + 0.001 \times 10.0 = 0.010$. Curve B wins.

Answer: At $\lambda = 0.01$ the toll is high enough that the nearly straight Curve A (loss 0.03) beats the wiggly Curve B (0.10); at $\lambda = 0.001$ the toll is so cheap that B (0.010) beats A (0.021). Same two curves, same data — only λ decided the winner. This is why λ is a capacity dial.

Figure 7.7 shows what the dial does to a real fit: one degree-12 curve, fitted by ridge regression to the 20 training fields of the chapter dataset at three values of λ .

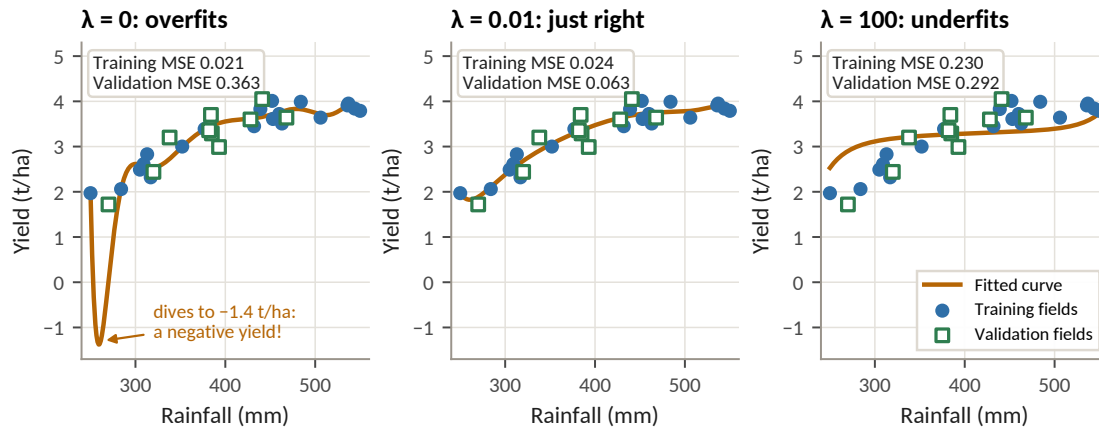


Figure 7.7 – The same degree-12 curve fitted to the same 20 training fields with three regularization strengths. Hollow green squares are the 10 validation fields.

WHAT TO OBSERVE

With no penalty the curve chases every training point and, between two of them, plunges to a negative yield: validation MSE 0.363. With $\lambda = 0.01$ the curve keeps its useful bend (yield rises steeply, then levels off) and reaches 0.063. With $\lambda = 100$ the penalty flattens the curve until it barely follows the data, and both errors are high again. One flexible model, one dial, all three behaviors.

Table 7.5 – The regularization dial: degree-12 curve on 20 training and 10 validation fields

λ	0	0.001	0.01	0.1	1	10	100
Training MSE ((t/ha) ²)	0.021	0.023	0.024	0.025	0.027	0.076	0.230
Validation MSE ((t/ha) ²)	0.363	0.062	0.063	0.065	0.070	0.132	0.292

WORKED EXAMPLE 7.6 – Turn the Regularization Dial

Using Table 7.5, find the generalization gap at $\lambda = 0$ and at $\lambda = 100$, identify which end of the dial overfits and which underfits, and choose the best λ .

- Gap at $\lambda = 0$: $0.363 - 0.021 = 0.342$. Training is nearly perfect and validation is poor: overfitting.
- At $\lambda = 100$: training 0.230 and validation 0.292, both high and close (gap 0.062): underfitting.
- Scan the validation row for the smallest value: 0.062 at $\lambda = 0.001$.
- Compare the neighbors: $\lambda = 0.01, 0.1$ and 1 give 0.063, 0.065 and 0.070, all within 0.008 of the best.

Answer: $\lambda = 0.001$ gives the lowest validation MSE, 0.062 (t/ha)². The exact value matters little: anything from 0.001 to 1 is nearly as good, while $\lambda = 0$ overfits (0.363) and $\lambda = 100$ underfits (0.292). Regularization strength and model capacity are two dials that control the very same trade-off.

Figure 7.8 fills in the whole dial. Because useful values of λ span many powers of ten, the horizontal axis is logarithmic: each step to the right multiplies λ by ten.

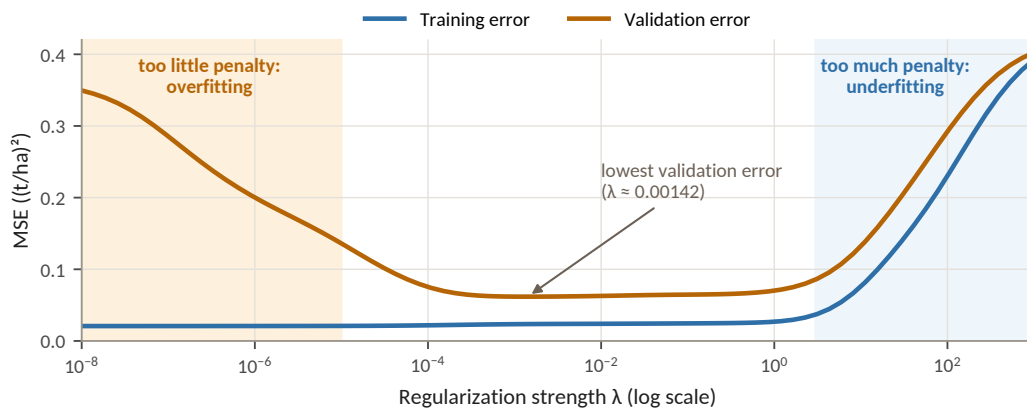


Figure 7.8 – Training and validation error of the degree-12 curve against regularization strength λ (logarithmic axis).

WHAT TO OBSERVE

Read from left to right, this is panel (b) of Figure 7.3 again, with capacity decreasing instead of increasing: too little penalty (left) gives a small training error and a large validation error; too much (right) drags both errors up together. The floor of the valley is wide, which is typical: choosing λ within the right power of ten is usually enough.

Regularization belongs to a family of tools that all trade a little training performance for better generalization: early stopping (Lesson 7.5) limits how long a model can chase the training data; a simpler model, or a capacity dial turned down, removes the freedom to overfit; more data shrinks the variance directly, as the learning curves showed. The preference for the simpler of two equally good explanations is far older than machine learning (Occam's razor, after the fourteenth-century philosopher William of Ockham). Whichever tool you reach for, the decision starts from the same two numbers: high validation error *with* high training error calls for more capacity, better features or a smaller λ ; low training error with much higher validation error calls for less capacity, a larger λ , early stopping or more data.

COMMON MISTAKE

✗ Wrong: Turning up the regularization strength to repair a poor validation score that is really caused by data leakage, or by a training-validation split that was never truly separate (Chapter 5).

✓ Right: Regularization only helps with overfitting caused by too much capacity. Check for leakage and a clean split first. No value of λ repairs a validation set that was not kept separate.

IMPORTANT

A model that fits its training data almost perfectly can still sound completely confident while being badly wrong on a new field, a new season or a new farmer's land. Before any prediction guides a real decision — an irrigation budget, a patient recall, a loan — check its validation performance honestly, and say so plainly if it is weak.

COMMON MISTAKES AND MISCONCEPTIONS

- Judging a model by its training score alone, without a separate validation set.
- Believing that training automatically finds the best hyperparameter — tree depth, k , or λ — on its own.
- Assuming that more training data always fixes a weak validation score, even when the training score is weak too.
- Trying to repair data leakage or a broken split by increasing the regularization strength.
- Choosing capacity or λ by repeatedly checking the test set instead of the validation set, which quietly turns the test set into a second training set.

Practical Activities

ACTIVITY (PAPER) — Sketch the U-Curve and Mark the Best Capacity

A team trained decision trees of depth 1 to 8 on a delivery-time dataset and recorded training and validation errors (illustrative, in minutes²): depth 1: 0.40, 0.42; depth 2: 0.22, 0.25; depth 3: 0.12, 0.14; depth 4: 0.07, 0.10; depth 5: 0.04, 0.12; depth 6: 0.02, 0.16; depth 7: 0.01, 0.21; depth 8: 0.00, 0.27.

1. Draw axes on graph paper: depth from 1 to 8 across, error from 0 to 0.45 up. Plot the training errors and join them; plot the validation errors in a second color and join them.
2. Compute the gap at every depth and write it under the axis.
3. Shade the depths where both errors are high and close (underfitting) and the depths where the gap is wide (overfitting). Circle the depth with the lowest validation error, and write one sentence explaining why depth 8, with a training error of exactly 0, is a worse choice.

You need: Graph paper, pencil, two colors

ACTIVITY (PYTHON) — Training and Validation MSE with NumPy and scikit-learn

This program repeats Worked Examples 7.1 and 7.3: `LinearRegression` finds the Line model by least squares (exactly slope 0.008 and intercept -0.2), and `mean_squared_error` does the subtract-square-average arithmetic.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

x_train = np.array([300, 350, 400, 450, 500])
y_train = np.array([2.1, 2.8, 3.0, 3.2, 3.9])
x_val = np.array([325, 425, 475])
y_val = np.array([2.2, 3.1, 3.8])

line = LinearRegression().fit(x_train[:, None], y_train) # least squares
print("Line model: yield =", round(line.coef_[0], 3), "x rainfall",
      round(line.intercept_, 1))
p_line_train = line.predict(x_train[:, None])
p_line_val = line.predict(x_val[:, None])
models = {"Flat": (np.full(5, 3.0), np.full(3, 3.0)),
          "Line": (p_line_train, p_line_val),
          "Wiggly": (y_train.copy(), np.array([2.9, 2.6, 4.6]))} # exact
for name, (p_train, p_val) in models.items(): # Worked Examples 7.1 and 7.3
    print(f"{name:7s} training MSE {mean_squared_error(y_train, p_train):.2f}"
          f" validation MSE {mean_squared_error(y_val, p_val):.2f}")
```

Output

```
Line model: yield = 0.008 x rainfall -0.2
Flat      training MSE 0.34    validation MSE 0.43
Line      training MSE 0.02    validation MSE 0.03
Wiggly    training MSE 0.00    validation MSE 0.46
```

The fitted line matches the chapter's Line model, so the MSE lines reproduce Table 7.3 exactly; `np.var([3.2, 4.6, 3.4, 4.0])` repeats the variance arithmetic of Worked Example 7.2.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

ACTIVITY (SIMULATION) — Fit Curves of Degree 1, 3 and 12, Then Grow the Training Set

Part A fits three polynomial curves to the 20 training fields of `ch07_wheat_fields.csv` and scores them on the 10 validation fields. Part B simulates a pool of 400 fields and uses `learning_curve` to train the degree-12 curve on 30 random subsets of 20, 80 and 320 fields: the degree-12 rows of Table 7.4.

```

import numpy as np, pandas as pd
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import learning_curve, ShuffleSplit
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge

fields = pd.read_csv("datasets/ch07_wheat_fields.csv")
train, val = fields[fields.set == "train"], fields[fields.set == "validation"]
z_tr, z_va = (train.rainfall_mm - 400) / 100, (val.rainfall_mm - 400) / 100
for d in [1, 3, 12]:
    # Part A: three polynomial curves
    coef = np.polyfit(z_tr, train.yield_t_ha, d) # least squares, degree d
    mse_tr = mean_squared_error(train.yield_t_ha, np.polyval(coef, z_tr))
    mse_va = mean_squared_error(val.yield_t_ha, np.polyval(coef, z_va))
    print(f"degree {d:2d}: training MSE {mse_tr:.3f}"
          f" validation MSE {mse_va:.3f}")

rng = np.random.default_rng(11) # Part B: 400 simulated fields
rain = np.round(rng.uniform(250, 550, 400))
y = np.round(3.9 - 0.00003 * (rain - 520) ** 2 + rng.normal(0, 0.25, 400), 2)
splits = ShuffleSplit(n_splits=30, test_size=0.2, random_state=0)
model = make_pipeline(PolynomialFeatures(12), StandardScaler(),
                      Ridge(alpha=0.001)) # the degree-12 curve
n, tr, va = learning_curve(model, (rain[:, None] - 400) / 100, y,
                           train_sizes=[20, 80, 320], cv=splits,
                           scoring="neg_mean_squared_error")
for k in range(3):
    # median over the 30 random splits
    print(f"degree 12, {n[k]:3d} fields: training {np.median(-tr[k]):.3f}"
          f" validation {np.median(-va[k]):.3f}")

```

Output

```

degree 1: training MSE 0.065 validation MSE 0.158
degree 3: training MSE 0.028 validation MSE 0.080
degree 12: training MSE 0.021 validation MSE 0.363
degree 12, 20 fields: training 0.036 validation 0.234
degree 12, 80 fields: training 0.055 validation 0.074
degree 12, 320 fields: training 0.061 validation 0.067

```

1. Compute the three gaps of Part A, then change 12 to 6 and to 2, rerun, and decide which degree you would choose. To see the curves, plot `np.polyval(coef, zz)` against `400 + 100 * zz` for `zz = np.linspace(-1.5, 1.5, 300)`.
2. Part B fits the model 90 times (3 sizes $\times$ 30 splits). Replace the model with `LinearRegression()` (imported from `sklearn.linear_model`) for the straight-line rows of Table 7.4. Which model is stuck, and which is rescued by more fields?

You need: A computer with Python, NumPy, pandas and scikit-learn installed; the file `datasets/ch07_wheat_fields.csv` (Appendix C)

CASE STUDY — Choosing Model Capacity for a Wheat-Yield Model — Duhok Governorate

Anwar manages a small agricultural extension office near Duhok. Every spring, farmers ask the same question: given this season's rainfall so far, how much wheat should they expect, and is paying for extra irrigation worth it? For years the office gave a rough guess from experience. Anwar hires a young data analyst, Sara, to see whether the office's records from thirty fields (the dataset used in this chapter's activities) can produce a more careful estimate.

Sara is eager to impress. She fits a highly flexible curve to the twenty fields she uses for training, and on those fields it looks remarkable: a training MSE of 0.021, almost zero. She is ready to present it as the office's new tool when a senior colleague asks one question: what does it score on fields it never trained on? Sara has not checked. When she does, the validation MSE is 0.363, seventeen times the training error — the Wiggly model of Lesson 7.4, which memorized twenty fields instead of learning a rule that transfers.

Sara starts over, following the chapter's method. The flat average underfits (training MSE 0.438, validation 0.427). The straight line is much better (0.065 and 0.158) but leaves a clear gap and, on a learning curve, its errors flatten well above the noise floor: it lacks capacity, not data. A gently curving degree-2 model does best of the simple candidates (0.029 and 0.077). Finally she keeps her flexible curve but adds a ridge penalty of $\lambda = 0.01$, chosen on the validation fields alone, so that the curve can still respond where the data supports it but must pay for every extra bend. The regularized curve reaches a validation MSE of 0.063, the best of all, with the "rises, then levels off" shape that agronomists expect.

The finished tool prints a range rather than a single confident number, with a note that the estimate is guidance, not a guarantee, and Anwar's office pairs every estimate with the judgment of a local agronomist before advising any farmer to spend money on irrigation. Sara also schedules a retraining at the end of every season, because a rule learned from a few years of rainfall need not hold in a year that behaves differently. The lesson of her first attempt stays with her: a training score alone never proves a model is ready.

CHAPTER SUMMARY

- Generalization — performing well on new data — is the true goal of a model; a strong training score alone can never prove it.
- Underfitting means the model is too simple and performs poorly even on the training data; overfitting means it has memorized the training data and performs worse on new data.
- The loss is the number training minimizes, here the MSE; the generalization gap is the validation error minus the training error.
- Bias is a consistent, systematic mistake caused by an overly simple model; variance is how much the model's predictions swing across different training samples. Both can be measured from a table of predictions.
- Expected error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$. Lowering one part usually raises the other, so the best model has the smallest sum, somewhere between the simplest and the most flexible.
- A hyperparameter is a setting chosen before training that controls capacity: polynomial degree, tree depth, k in nearest neighbors, the number of boosting rounds, or the penalty strength λ .
- As capacity rises, training error keeps falling while validation error falls, then rises again: a U-curve. Choose the capacity at the bottom of the validation curve.
- A loss curve plots both errors against training rounds (stop early when the validation loss turns upward); a learning curve plots them against training-set size (a high plateau means underfitting, a wide narrowing gap means overfitting and more data will help).
- Regularization adds λ times a penalty on the model's coefficients to the loss, letting one flexible model behave like a simple or a complex one depending on λ ; choose λ with the validation set. Regularization, early stopping, simpler models and more data all treat overfitting, but none repairs data leakage or a broken split.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define underfitting and overfitting in your own words, one sentence each, and name the failure that the driving-test student in the Introduction suffered from.

Problem 2 [Basic] A model scores 55% accuracy on its training data and 54% on its validation data. Name the more likely problem, underfitting or overfitting, and explain your choice in one sentence.

Problem 3 [Basic] A model scores 99% accuracy on its training data and 61% on its validation data. Name the more likely problem and explain your choice in one sentence.

Problem 4 [Basic] A fourth candidate, Model D, predicts 2.3, 2.7, 2.8, 3.6 and 3.8 t/ha for the five training fields of Table 7.1 (true yields 2.1, 2.8, 3.0, 3.2 and 3.9). Find Model D's training MSE, showing every step.

Problem 5 [Standard] A team predicting exam scores from study hours tries three capacity levels: training and validation MSE (points²) of 64 and 81, then 16 and 25, then 1 and 49. Say which level underfits, which overfits and which is best, then convert the best level's validation MSE into an RMSE in points.

Problem 6 [Standard] Trained on four different samples, Model X predicts 2.9, 3.3, 3.1 and 2.7 t/ha for one field, and Model Y predicts 3.0, 3.0, 3.2 and 2.8. The field's true long-run yield is 3.4 t/ha and the noise variance is 0.04. For each model find the average prediction, the bias, bias², the variance and the expected error. Which model is better, and by how much?

Problem 7 [Standard] A spam filter trained on 200 e-mails has training accuracy 96% and validation accuracy 70%. Trained on 2,000 e-mails, its accuracies are 93% and 89%. Find the gap at each size, diagnose the model, and say whether collecting even more e-mails is likely to help.

Problem 8 [Standard] At $\lambda = 1$ a model's training MSE is 0.30 and its validation MSE 0.55. At $\lambda = 0.05$ they are 0.05 and 0.20. At $\lambda = 20$ they are 0.60 and 0.65. Choose the best λ of the three and explain your choice in one sentence. Which setting overfits and which underfits?

Problem 9 [Standard] In Figure 7.5 the validation loss is 0.088 at round 30 and 0.132 at round 200, while the training loss falls from 0.017 to 0.001. A colleague insists on training for all 200 rounds "because the training loss is only 0.001." Compute the generalization gap at both rounds, and write two sentences explaining why the training loss is the wrong number to look at.

Problem 10 [Challenge] A team tests a decision tree at three depths on a customer-churn dataset: depth 2 (training accuracy 74%, validation 73%), depth 8 (92%, 84%), depth 20 (99%, 78%). (a) Which depth would you choose, and why? (b) Which depth shows the clearest overfitting, and what is the evidence? (c) The best depth still leaves a gap of 8 points; name two things to try next, other than another depth.

Problem 11 [Challenge] Curves A and B of Worked Example 7.5 have fit errors 0.02 and 0.00 and penalties 1.0 and 10.0. (a) Write the regularized loss of each as a formula in λ . (b) Find the value of λ at which the two losses are equal. (c) For which values of λ does training prefer Curve A? (d) Explain why a validation set, not this calculation, must decide which λ to use.

Problem 12 [Challenge] Pick a prediction problem from your own field of study or interest. Name one hyper-parameter of a model you could use that controls its capacity, and explain in a few sentences how you would use a validation set, not the test set, to choose its value, what a learning curve could additionally tell you, and how you would decide whether to regularize.

PART TWO

SUPERVISED LEARNING: PREDICTING NUMBERS

CHAPTER EIGHT

Linear Regression

A line that predicts a number, fitted by least squares

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what linear regression predicts, and write its equation for one feature.
- Compute a prediction by hand from a coefficient, an intercept and a feature value.
- Calculate residuals and use them to score a candidate line with SSE and MSE.
- Find the best slope and intercept directly, with the least-squares formulas, without guessing.
- Judge a finished fit with R^2 , an actual-versus-predicted plot and a residual plot.
- Extend a line to several features at once, and interpret one coefficient while the others are held fixed.
- Bend a line into a curve with polynomial features, and tame a model with ridge or lasso regression.

Introduction

Meet Rania. She works in the customer office of the electricity company in Erbil, where callers ask her the same question every month: “roughly how much will my bill be this time?” The meter reading is still days away, so she cannot look up the answer. But she does have records for hundreds of homes: the floor area of each, in square meters, and the monthly bill it actually paid.

Picture those records as a graph. Home size runs along the bottom, the monthly bill up the side, and every home becomes one dot. The dots do not fall in a perfect pattern, but they drift upward: bigger homes tend to pay more. If Rania can draw one straight line through the middle of that cloud, she can answer every caller in seconds — read the home size along the bottom, go up to the line, read off a predicted bill.

A straight line needs only two numbers to describe it, and this chapter shows how to find the two best ones by hand, with arithmetic you already know. Chapter 1 called this kind of task *regression*; Chapter 6 gave the error measures used to grade a line; Chapter 7 warned about models that fit their training data too closely. Now we build the first real model of the book and check every number ourselves.

8.1 A Line That Predicts a Number

Rania starts small. She pulls five homes out of her records and looks for the simplest rule that turns the first column of Table 8.1 into the last one. These same five homes return in every lesson of this chapter.

Table 8.1 – Rania’s five recorded homes in Erbil (illustrative example data)

Home	Size (m ²)	Major appliances (count)	Monthly bill (dollars)
1	50	1	50
2	70	2	66
3	90	3	82
4	110	3	94
5	130	5	110

Lessons 8.1 to 8.5 use only the size column and the bill column; the appliance column waits until Lesson 8.6.

WORD TO KNOW

Linear regression — A method that predicts a number by multiplying each feature by its own number, adding the results together, and then adding one more fixed number at the end. With a single feature, this is exactly a straight line drawn through the data.

Its output is always a number, such as a bill in dollars, never a category; Chapter 10 adapts the same shape of rule to yes-or-no questions. The word *regression* comes from the English scientist Francis Galton, who in 1886 studied how children’s heights “regressed” toward their parents’ generation’s average and drew the first fitted line of this kind. A one-feature line needs two letters: w for the number the feature is multiplied by, and b for the number added at the end. The hat on $\hat{y}$ marks a *predicted* value, as in Chapter 6.

EQUATION: THE ONE-FEATURE REGRESSION LINE

$$\hat{y} = wx + b$$

In words: The predicted value equals a number we multiply by the feature, plus a fixed number we add at the end.

When it is used: Whenever one numerical feature is used to predict one number, and a straight line is a reasonable description of how the two move together.

Symbol	Meaning	Unit or type
$\hat{y}$	the predicted value	same unit as the target
x	the feature value for one example	unit of the feature
w	how much the prediction changes for each extra unit of x	target unit per feature unit
b	the value the line takes when $x = 0$	same unit as the target

Lesson 8.2 gives w and b their proper names. For now, notice the shape of the recipe: multiply by w , then add b . Lesson 8.4 shows how to compute the best w and b from the data itself.

A colleague suggests a first rule: multiply the size by 0.7 and add 15, or $\hat{y} = 0.7x + 15$. Call this candidate *Line A*.

WORKED EXAMPLE 8.1 — Predict One Home’s Bill, Then All Five

Line A predicts a monthly bill from home size with the rule $\hat{y} = 0.7x + 15$, where x is the size in square meters and $\hat{y}$ is the bill in dollars. Predict the bill for a 90 m² home, then for all five homes of Table 8.1.

1. Write the rule with the home size filled in: $\hat{y} = 0.7 \times 90 + 15$.
2. Multiply first: $0.7 \times 90 = 63$.

3. Add the fixed number: $63 + 15 = 78$ dollars.
4. Now multiply every size by 0.7: $0.7 \times 50 = 35$, $0.7 \times 70 = 49$, $0.7 \times 90 = 63$, $0.7 \times 110 = 77$, $0.7 \times 130 = 91$.
5. Add the intercept 15 to each: 50, 64, 78, 92, 106.

Answer: Line A predicts 78 dollars for the 90 m² home, and 50, 64, 78, 92 and 106 dollars for the five homes in order. Lesson 8.3 compares these predictions with the true bills already recorded in Table 8.1.

Notice what the rule does not claim. Home 3 in Table 8.1 is 90 m² and really paid 82 dollars, not 78. Homes differ in insulation, in how many people live there and in how often the air conditioner runs. A line describes a general tendency across many homes, never a promise about a single one.

8.2 Slope, Intercept and Predictions by Hand

The two numbers inside a regression line each deserve their own name, because you will read and interpret them constantly.

WORDS TO KNOW

Coefficient — The number a feature is multiplied by inside a regression equation, also called a *weight* or, for a one-feature line, the *slope*. A coefficient of 0.7 means the prediction rises by 0.7 for every extra unit of that feature.

Intercept — The fixed number added at the end, after every coefficient has been multiplied by its feature. The intercept is the model's prediction when every feature equals zero.

In Line A, $\hat{y} = 0.7x + 15$, the coefficient is 0.7 and the intercept is 15. The coefficient carries a unit made of two others, dollars per square meter: each extra square meter of floor area adds 0.7 dollars to the predicted bill. The intercept, 15 dollars, is where the line crosses the vertical axis. Figure 8.1 shows both numbers, the five homes and the prediction of Worked Example 8.1 on one picture.

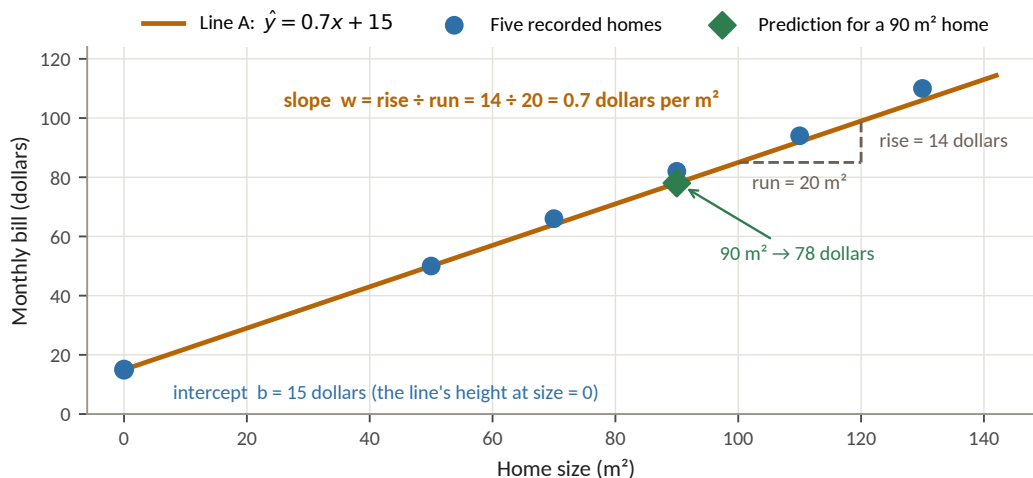


Figure 8.1 – The anatomy of a regression line, drawn on Rania's five homes. The intercept is where the line meets the vertical axis; the slope is the rise divided by the run of any step along the line.

WHAT TO OBSERVE

Moving 20 m² to the right always lifts the prediction by exactly 14 dollars, anywhere on the line, because $0.7 \times 20 = 14$. That constant rate of change is what makes the model *linear*. Notice too that the line misses every dot but the first — the subject of Lesson 8.3.

No home has 0 m² of floor space, so this intercept is a mathematical anchor, not a real prediction. An intercept only needs a real-world meaning when $x = 0$ is a value the data could reach. Predicting far outside the training range has its own name and its own risk.

WORD TO KNOW

Extrapolation — Using a model to predict for feature values that lie outside the range covered by the training data. Rania's five homes run from 50 to 130 m², so a prediction for a 400 m² warehouse is extrapolation, and nothing in the data supports it.

The fitted line of Lesson 8.4 cheerfully predicts $0.74 \times 400 + 13.8 = 309.8$ dollars for that warehouse, but the model has never seen a building near that size and cannot know whether the pattern still holds there.

COMMON MISTAKE

X Wrong: Computing a prediction by multiplying the coefficient by the feature, then stopping, and forgetting to add the intercept.

✓ Right: Always finish both steps of the recipe: multiply first, then add the intercept. Skipping the intercept shifts every single prediction by the same fixed amount, and here that shift would be 15 dollars on every home.

IMPORTANT

A coefficient describes a pattern found in past data, not a cause. A positive coefficient on home size does not prove that floor space by itself raises a bill: bigger homes also have more rooms to cool, more appliances and more people. Treat a coefficient as a clue about a pattern, never as proof of what would happen if one thing in the real world changed.

8.3 Residuals and the Least-Squares Loss

Rania already knows the true bills of these five homes, because they are past customers. Comparing a prediction with the truth, one home at a time, gives every prediction its own small report card.

WORD TO KNOW

Residual — The true value minus the predicted value for one example, $y - \hat{y}$. A positive residual means the model predicted too low; a negative one means it predicted too high; zero means the prediction was exactly right. Some books define it the other way round; this book always uses true minus predicted, as Chapter 6 did.

WORKED EXAMPLE 8.2 — Find the Residual of Each of Rania's Five Homes

The five homes' true monthly bills are 50, 66, 82, 94 and 110 dollars. Line A predicted 50, 64, 78, 92 and 106 dollars for the same five homes, in the same order. Find the residual of each home.

1. Home 1: $50 - 50 = 0$.
2. Home 2: $66 - 64 = 2$.
3. Home 3: $82 - 78 = 4$.

4. Home 4: $94 - 92 = 2$.
5. Home 5: $110 - 106 = 4$.

Answer: The five residuals are 0, 2, 4, 2 and 4 dollars. Every residual is zero or positive, so Line A never once predicted too high; it either matched exactly or predicted a little too low. A pattern in which every residual points the same way is exactly the consistent, one-directional error that Chapter 7 called high bias.

The left panel of Figure 8.2 draws these five residuals as vertical segments. Every segment is measured straight up and down, never sideways: the model is judged only on how far off its predicted bill was, not on the home's size. Five separate residuals check one home at a time, but Rania needs a single number that grades a whole line, so she can compare candidates fairly. Chapter 7 called that number the *loss*. For a regression line the standard choice squares the residuals first.

WORDS TO KNOW

Sum of squared errors (SSE) — The total obtained by squaring every residual and adding all the squares together. Squaring removes the signs, so errors above and below the line can never cancel out.

Least squares — The rule that says the best line is whichever line makes the sum of squared errors as small as possible. It was published by the French mathematician Adrien-Marie Legendre in 1805 and independently developed by Carl Friedrich Gauss, who published his own account in 1809.

EQUATION: RESIDUAL, SUM OF SQUARED ERRORS AND MEAN SQUARED ERROR

$$e_i = y_i - \hat{y}_i \quad \text{SSE} = \sum_{i=1}^n e_i^2 \quad \text{MSE} = \frac{\text{SSE}}{n}$$

In words: A residual is the true value minus the predicted value, sign kept. Square every residual and add the squares to get the SSE. Divide that total by the number of examples to get the MSE.

When it is used: Residuals whenever a fitted model is checked; they are the raw material of every loss function and every regression metric of Chapter 6. SSE is what least squares makes as small as possible; MSE is what to report or compare across datasets of different sizes.

Symbol	Meaning	Unit or type
e_i	residual of example i	same unit as the target
$y_i, \hat{y}_i$	true value and predicted value of example i	unit of the target
SSE, MSE	sum of squared errors, mean squared error	unit of y , squared
n	number of examples	count

Squaring does two jobs: it removes the sign, so -4 counts exactly as much as $+4$, and it punishes large residuals far more than small ones. The MSE of Chapter 6 divides by n so that lines scored on different numbers of homes stay comparable; its square root is the RMSE, back in dollars.

WORKED EXAMPLE 8.3 — Find the Loss of Line A

Line A's five residuals are 0, 2, 4, 2 and 4 dollars. Find its sum of squared errors and its mean squared error.

1. Square each residual: $0^2 = 0$, $2^2 = 4$, $4^2 = 16$, $2^2 = 4$, $4^2 = 16$.
2. Add the five squares: $0 + 4 + 16 + 4 + 16 = 40$, so $\text{SSE} = 40$.
3. Divide by the number of homes, $n = 5$: $40 \div 5 = 8$.

Answer: Line A's SSE is 40 and its MSE is 8 dollars squared. On its own, 8 is hard to picture; it becomes useful the moment a second line is scored the same way.

WORKED EXAMPLE 8.4 — Compare Line A With a Second Candidate, Line B

A second rule, Line B, is $\hat{y} = 0.8x + 10$. For the same five homes (sizes 50, 70, 90, 110 and 130 m²; true bills 50, 66, 82, 94 and 110 dollars), find Line B's predictions, residuals, SSE and MSE, then decide which line fits better.

1. Predictions: $0.8 \times 50 + 10 = 50$, $0.8 \times 70 + 10 = 66$, $0.8 \times 90 + 10 = 82$, $0.8 \times 110 + 10 = 98$, $0.8 \times 130 + 10 = 114$.
2. Residuals, true minus predicted: $50 - 50 = 0$, $66 - 66 = 0$, $82 - 82 = 0$, $94 - 98 = -4$, $110 - 114 = -4$.
3. Square each residual: 0 , 0 , 0 , $(-4)^2 = 16$, $(-4)^2 = 16$.
4. Add them: $SSE = 0 + 0 + 0 + 16 + 16 = 32$.
5. Divide by 5: $MSE = 32 \div 5 = 6.4$.
6. Compare the two mean squared errors: Line A scored 8, Line B scored 6.4.

Answer: Line B fits better: MSE 6.4 dollars squared against Line A's 8. Its first three predictions are exactly right, and it misses only on the two largest homes, predicting a little too high both times.

Table 8.2 collects both lines, and the right panel of Figure 8.2 draws them on the same axes.

Table 8.2 – Predictions and residuals of Line A and Line B on the five homes (dollars). Line B's squared residuals are 0, 0, 0, 16 and 16, so its SSE is 32.

Home	Size (m ²)	True bill	A: $\hat{y}$	A: residual	A: squared	B: $\hat{y}$	B: residual
1	50	50	50	0	0	50	0
2	70	66	64	2	4	66	0
3	90	82	78	4	16	82	0
4	110	94	92	2	4	98	-4
5	130	110	106	4	16	114	-4
Total				12	SSE 40		-8

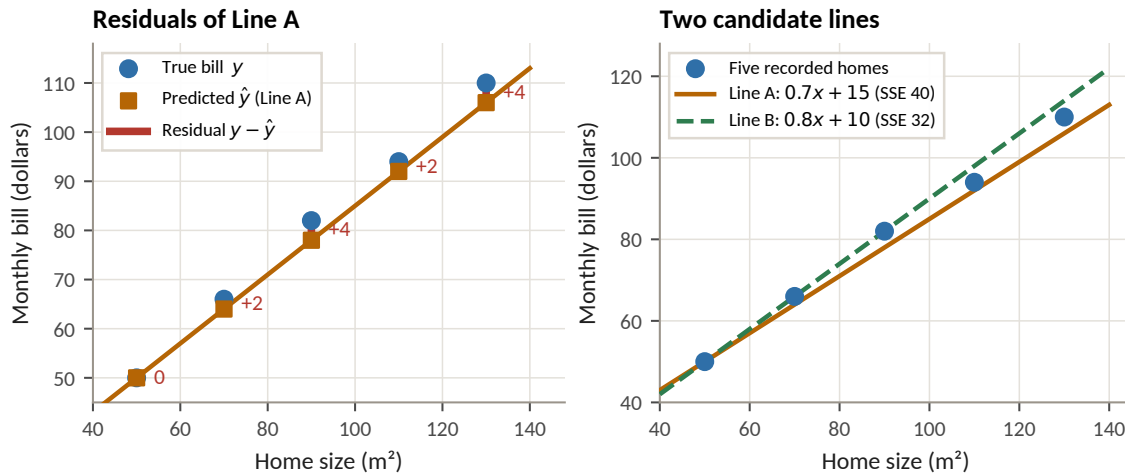


Figure 8.2 – Left: the five residuals of Line A, drawn as red segments from each true bill (round marker) down to the prediction (square marker). Right: Line A and the steeper Line B on the same five homes.

WHAT TO OBSERVE

All five red segments point upward, the picture of a line sitting a little too low across the whole range. On the right, Line B passes exactly through the three smallest homes and then rises above the two largest, while Line A sits below every home except the smallest. Neither is obviously right by eye; only the SSE decides, 32 for B against 40 for A.

COMMON MISTAKE

X Wrong: Comparing two candidate lines using only the raw sum of squared errors, especially when the two lines were scored on different numbers of homes.

✓ Right: Compare lines with the mean squared error, the sum divided by the number of examples, exactly as Chapter 6 recommended. A larger dataset almost always produces a larger raw sum, even for an equally good line.

8.4 The Best Line Without Guessing

Line B beat Line A — but is it the best line possible? Guessing candidates one at a time is no strategy once a project has thousands of homes. For a straight line, happily, the best answer can be written down directly, with two short formulas and no guessing at all.

WORDS TO KNOW

Ordinary least squares (OLS) — Fitting a line by choosing the coefficient and the intercept that make the sum of squared errors as small as it can possibly be. It is the standard way to fit a linear regression model.

Closed-form solution — An answer given by a formula that can be evaluated once, directly, instead of being searched for step by step. Linear regression is one of the few models in this book with a closed-form solution; most models in later chapters must be searched for, as Chapter 9 explains.

Both formulas work with *deviations from the mean*: how far each size sits from the average size, and each bill from the average bill.

EQUATION: THE LEAST-SQUARES SLOPE AND INTERCEPT

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad b = \bar{y} - w \bar{x}$$

In words: For the slope, multiply each example's size deviation by its own bill deviation, add all those products, and divide by the sum of the squared size deviations. For the intercept, take the average bill and subtract the slope times the average size.

When it is used: To fit a one-feature linear regression exactly, in one pass through the data, with no searching and no repeated trials.

Symbol	Meaning	Unit or type
w, b	the fitted slope and intercept	target unit per feature unit; target unit
x_i, y_i	feature value and true target of example i	their own units
$\bar{x}, \bar{y}$	the mean of the features and the mean of the targets	their own units
n	number of examples	count

Compute w first; b needs it. The formulas guarantee that the fitted line passes through the point $(\bar{x}, \bar{y})$, and that no other straight line has a smaller SSE on this data.

By hand, the safest route is a small table with one row per example and one column per quantity, as in Table 8.3.

Table 8.3 – Least-squares calculation table for the five homes ($\bar{x} = 90 \text{ m}^2$, $\bar{y} = 80.4 \text{ dollars}$)

Home	Size x	Bill y	$x - \bar{x}$	$y - \bar{y}$	Product	$(x - \bar{x})^2$
1	50	50	−40	−30.4	1216	1600
2	70	66	−20	−14.4	288	400
3	90	82	0	1.6	0	0
4	110	94	20	13.6	272	400
5	130	110	40	29.6	1184	1600
Sum	450	402	0	0	2960	4000

WORKED EXAMPLE 8.5 — Fit the Least-Squares Line to the Five Homes by Hand

Using the five homes of Table 8.1, find the slope and intercept that minimize the sum of squared errors.

1. Find the mean size: $(50 + 70 + 90 + 110 + 130) \div 5 = 450 \div 5 = 90 \text{ m}^2$.
2. Find the mean bill: $(50 + 66 + 82 + 94 + 110) \div 5 = 402 \div 5 = 80.4 \text{ dollars}$.
3. Find each size deviation $x - \bar{x}$: −40, −20, 0, 20, 40.
4. Find each bill deviation $y - \bar{y}$: −30.4, −14.4, 1.6, 13.6, 29.6.
5. Multiply the two deviations of each home: 1216, 288, 0, 272, 1184. Add them: 2960.
6. Square each size deviation: 1600, 400, 0, 400, 1600. Add them: 4000.
7. Divide to get the slope: $w = 2960 \div 4000 = 0.74 \text{ dollars per m}^2$.
8. Anchor the intercept: $b = 80.4 - 0.74 \times 90 = 80.4 - 66.6 = 13.8 \text{ dollars}$.

Answer: The least-squares line is $\hat{y} = 0.74x + 13.8$. Its slope sits between the two guesses and its SSE is 4.8, far below Line A's 40 and Line B's 32. No other straight line can beat it on this data.

Figure 8.3 draws the fitted line with the two guesses behind it.

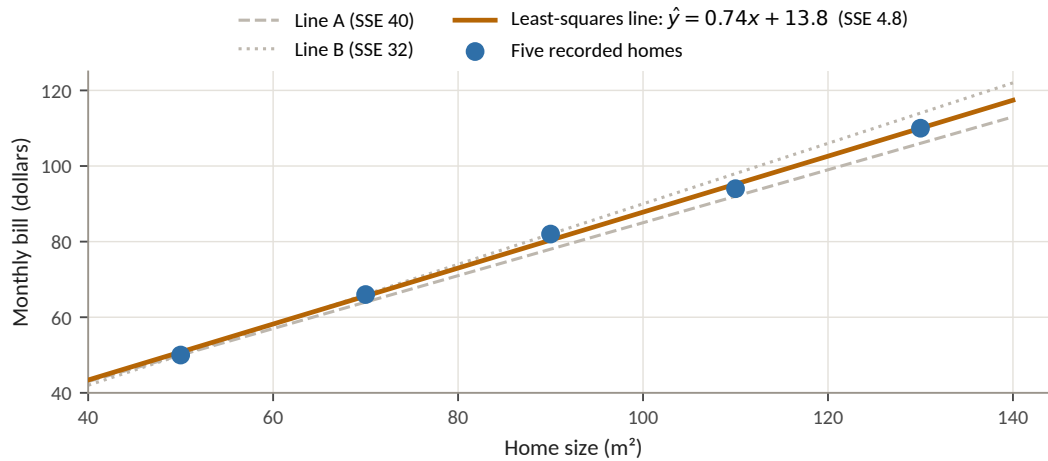


Figure 8.3 – The least-squares line for Rania’s five homes, with the two guessed lines shown faintly behind it for comparison.

WHAT TO OBSERVE

The fitted line threads the dots more evenly than either guess: two homes sit above it and two below, instead of all five on one side. That balance is no coincidence — whenever a line is fitted by least squares with an intercept, its residuals add up to exactly zero.

8.5 Checking a Fit: Plots and R^2

A fitted line always exists, even for data with no real pattern. Before trusting one, grade it with a number, then look at two standard plots. Table 8.4 holds everything the grading needs.

Table 8.4 – The least-squares line $\hat{y} = 0.74x + 13.8$ on the five homes (dollars)

Home	Size (m ²)	True y	Predicted $\hat{y}$	Residual	Residual ²	$(y - \bar{y})^2$
1	50	50	50.8	−0.8	0.64	924.16
2	70	66	65.6	0.4	0.16	207.36
3	90	82	80.4	1.6	2.56	2.56
4	110	94	95.2	−1.2	1.44	184.96
5	130	110	110.0	0.0	0.00	876.16
Sum				0	SSE 4.8	SST 2195.2

An SSE of 4.8 sounds small, but small compared with what? Chapter 6 answered that with R^2 , which compares the model’s squared error against that of the laziest possible model: always predicting the mean bill.

EQUATION: R^2 OF A FITTED LINE

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

In words: Divide the model’s total squared error by the total squared error that always predicting the mean would make, then subtract the result from one.

When it is used: To say, on a unit-free scale, how much of the variation in the target the fitted line explains. Report it beside RMSE or MAE, never instead of them.

Symbol	Meaning	Unit or type
R^2	coefficient of determination	number, at most 1
SSE	the model's sum of squared errors	unit of y , squared
SST	total sum of squares: $\sum (y_i - \bar{y})^2$	unit of y , squared
$\bar{y}$	mean of the true target values	unit of the target

$R^2 = 1$ means every residual is zero; $R^2 = 0$ means the line is no better than the mean; a negative R^2 means it is worse. The measure was named by the geneticist Sewall Wright in 1921 (Chapter 6).

WORKED EXAMPLE 8.6 — Grade the Fitted Line With MSE, RMSE and R^2

Using Table 8.4, find the MSE, the RMSE and the R^2 of the least-squares line, and compare its R^2 with Line A's.

1. Add the squared residuals: $0.64 + 0.16 + 2.56 + 1.44 + 0 = 4.8$, so $SSE = 4.8$.
2. Divide by $n = 5$: $MSE = 4.8 \div 5 = 0.96$ dollars squared.
3. Take the square root: $RMSE = \sqrt{0.96} \approx 0.98$ dollars.
4. Add the last column to get the total sum of squares: $924.16 + 207.36 + 2.56 + 184.96 + 876.16 = 2195.2$.
5. Apply the formula: $R^2 = 1 - 4.8 \div 2195.2 = 1 - 0.0022 = 0.9978$.
6. Do the same for Line A, whose SSE was 40: $R^2 = 1 - 40 \div 2195.2 = 0.9818$.

Answer: The fitted line has $RMSE \approx 0.98$ dollars and $R^2 = 0.9978$: it removes 99.78% of the squared error the mean baseline would make, and its typical miss is about one dollar. Line A's R^2 of 0.9818 looks high too — which is why R^2 is never read alone: Line A's typical miss, $\sqrt{8} \approx 2.83$ dollars, is almost three times worse.

Numbers alone can hide a bad fit, so every regression model deserves two quick plots. The first, Figure 8.4, puts the true value along the bottom and the predicted value up the side.

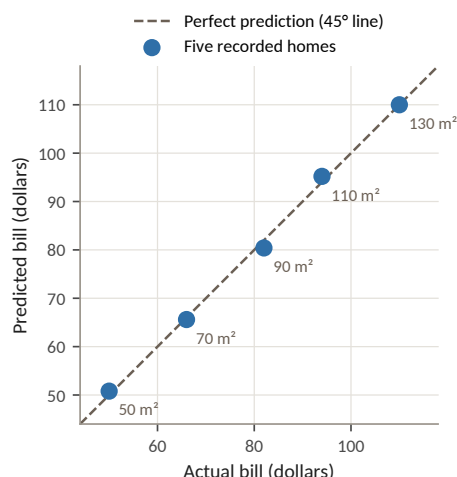


Figure 8.4 – Actual versus predicted bills for the five homes under the fitted line. Perfect predictions would sit exactly on the dashed 45° line.

WHAT TO OBSERVE

All five points hug the dashed line, which is what an R^2 of 0.998 looks like. The 90 m² home sits farthest below it: its true bill of 82 dollars was 1.6 above the 80.4 predicted. A plot like this makes one bad prediction jump out, however good the average score.

WORD TO KNOW

Residual plot — A plot of the residuals up the side against the predicted values (or the feature) along the bottom, with a horizontal line at zero. A good fit shows a shapeless band around zero; any visible shape means the model is missing a pattern.

The second plot is the more revealing, because it magnifies what the first squashes. Figure 8.5 shows both outcomes: on the left thirty homes from Rania's larger records, on the right a straight line fitted to data that genuinely curves.

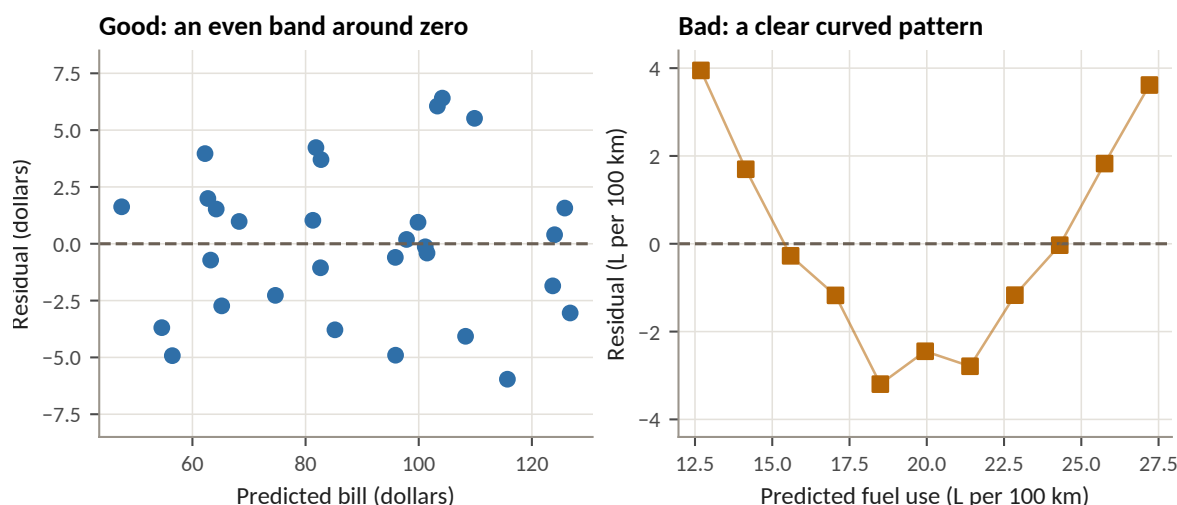


Figure 8.5 – Two residual plots. Left: a healthy fit, residuals scattered evenly above and below zero. Right: a straight line fitted to curved data, where the residuals form an obvious U.

WHAT TO OBSERVE

On the left there is no shape to find: points sit above and below zero at every predicted bill and do not fan out. On the right they start high, dive below zero in the middle and climb again — proof that a straight line is the wrong shape here. Lesson 8.7 fixes exactly this.

8.6 Multiple Linear Regression

Home size alone is a useful clue, but not the only one: two homes of the same size can have very different bills if one runs three air conditioners around the clock and the other one. Rania's records carry a second column, the count of major appliances, listed in Table 8.1 but unused until now.

WORD TO KNOW

Multiple linear regression — Linear regression with more than one feature. Each feature gets its own coefficient, all the products are added together, and a single intercept is added at the end.

EQUATION: A REGRESSION MODEL WITH SEVERAL FEATURES

$$\hat{y} = b + w_1x_1 + w_2x_2 + \cdots + w_kx_k = b + \sum_{j=1}^k w_jx_j$$

In words: The predicted value equals the intercept plus every feature multiplied by its own coefficient, with all those products added together, one term for each feature the model uses.

When it is used: Whenever several numerical features together explain the target better than any one of them alone.

Symbol	Meaning	Unit or type
$\hat{y}$	the predicted value	same unit as the target
b	the intercept	same unit as the target
$x_1, \dots, x_k$	the feature values of one example	each feature's own unit
$w_1, \dots, w_k$	the coefficient matching each feature	target unit per unit of that feature
k	number of features	count

With one feature this is exactly the line of Lesson 8.1. With two features the model is a flat plane; with more it is called a hyperplane. The recipe never changes: multiply each feature by its coefficient, add everything, then add the intercept.

Fitted on her office's full records, Rania's two-feature model is $\hat{y} = 5 + 0.4 \times \text{size} + 9 \times \text{appliances}$. Table 8.5 reports its three numbers with their units.

Table 8.5 – Coefficients of Rania's two-feature model (illustrative fitted values)

Part of the model	Symbol	Value	Plain-language meaning
Intercept	b	5 dollars	Baseline of the model when both features are zero
Home size	w_1	0.4 dollars per m ²	Each extra square meter adds 0.4 dollars, appliances held fixed
Major appliances	w_2	9 dollars per appliance	Each extra appliance adds 9 dollars, size held fixed

Every coefficient has a precise meaning: the change in the prediction for one extra unit of that feature, while every other feature stays exactly the same. Two homes differing in one feature only make this concrete.

WORKED EXAMPLE 8.7 — Predict With Two Features, and Read One Coefficient

Home X is 120 m² with 4 major appliances; Home Y is also 120 m² but has 5. Using $\hat{y} = 5 + 0.4 \times \text{size} + 9 \times \text{appliances}$, predict both bills and compare them.

1. Multiply the size coefficient by Home X's size: $0.4 \times 120 = 48$.
2. Multiply the appliance coefficient by its appliance count: $9 \times 4 = 36$.
3. Add both results to the intercept: $5 + 48 + 36 = 89$ dollars for Home X.
4. Repeat for Home Y with 5 appliances: $5 + 48 + 9 \times 5 = 5 + 48 + 45 = 98$ dollars.
5. Find the difference: $98 - 89 = 9$ dollars.

Answer: Home X is predicted to pay 89 dollars and Home Y 98 — and that 9-dollar gap is exactly the appliance coefficient, because size was held fixed at 120 m² for both. This is what a coefficient measures: the effect of one feature on its own, with every other feature held perfectly still.

Figure 8.6 shows the same idea as two pictures: vary one feature, freeze the other.

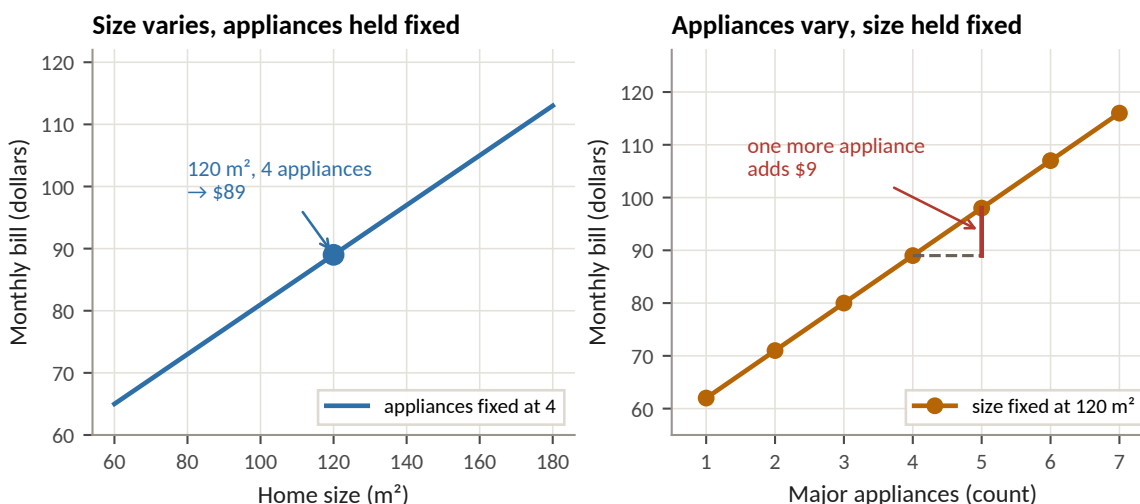


Figure 8.6 – Reading a two-feature model one feature at a time. Left: bills predicted as size changes with appliances frozen at 4. Right: bills predicted as appliances change with size frozen at 120 m².

WHAT TO OBSERVE

Each panel is a straight line, because holding one feature fixed leaves an ordinary one-feature line behind: the left rises by 0.4 dollars per square meter, the right by 9 dollars per appliance. Freezing the other features is what makes a coefficient readable.

COMMON MISTAKE

✗ Wrong: Concluding that appliances “matter more” than home size because the coefficient 9 is a much bigger number than 0.4.

✓ Right: A coefficient’s size depends partly on its feature’s unit. Home size runs over a hundred square meters or more, while the appliance count runs from about 1 to 6, so the size term contributes far more in total. Compare coefficients only after standardizing every feature, as Chapter 4 explains.

IMPORTANT

Adding features is not automatically an improvement. Chapter 7 warned that capacity chases noise, and every new feature adds some. Two features measuring nearly the same thing — floor area and number of rooms — also make coefficients unstable, because the model cannot tell which deserves the credit.

8.7 Curves From Lines: Polynomial Regression

The right-hand panel of Figure 8.5 came from a real problem. A transport company in Duhok records the average driving speed and the fuel used per 100 kilometers on each delivery trip. Fuel use is high at low speeds, falls to a minimum near 50 km per hour and climbs steeply after that. No straight line can bend like that — yet linear regression still works, with one small trick: add the square of the feature as a new feature.

WORD TO KNOW

Polynomial regression — Linear regression fitted to powers of a feature (x , x^2 , x^3 and so on) as if each power were a separate feature. The fitted model is a curve in the original feature, but it is still a linear model in its coefficients, so all the formulas and software of this chapter apply unchanged.

EQUATION: POLYNOMIAL REGRESSION MODEL OF DEGREE d

$$\hat{y} = b + w_1x + w_2x^2 + \cdots + w_dx^d$$

In words: Square the feature, cube it, and so on up to the chosen degree; treat each power as its own feature with its own coefficient; then fit exactly as in Lesson 8.6.

When it is used: When a residual plot shows a clear curve, so a straight line is systematically too high in one part of the range and too low in another.

Symbol	Meaning	Unit or type
d	the degree: the highest power used	count
$x, x^2, \dots, x^d$	the feature and its powers, used as separate features	feature unit, raised to each power
$w_1, \dots, w_d$	one coefficient per power	target unit per unit of that power
b	the intercept	same unit as the target

Degree 1 is the ordinary straight line. Degree is a hyperparameter of the kind Chapter 7 described: a high degree buys flexibility and pays for it with overfitting. Training error always falls as the degree rises, so it can never choose the degree for you — use a validation set or cross-validation (Chapter 5) and pick the simplest degree whose validation error is close to the best.

WORKED EXAMPLE 8.8 — Predict Fuel Use With a Degree-2 Model

The fitted degree-2 model for the Duhok trips is $\hat{y} = 26 - 0.40v + 0.004v^2$, where v is the average speed in km per hour and $\hat{y}$ is fuel use in liters per 100 km. Predict fuel use at 90 km per hour, and compare it with the prediction at 50 km per hour.

1. Multiply the first coefficient by the speed: $-0.40 \times 90 = -36$.
2. Square the speed: $90^2 = 8100$.
3. Multiply the second coefficient by the squared speed: $0.004 \times 8100 = 32.4$.
4. Add all three parts: $26 - 36 + 32.4 = 22.4$ liters per 100 km.
5. Repeat at 50 km per hour: $26 - 0.40 \times 50 + 0.004 \times 2500 = 26 - 20 + 10 = 16.0$.
6. Find the difference: $22.4 - 16.0 = 6.4$ liters per 100 km.

Answer: The model predicts 22.4 liters per 100 km at 90 km per hour against 16.0 at 50 — 6.4 liters more for every 100 km driven. Because the squared term is positive the curve bends upward, so the model also predicts higher fuel use at very low speeds.

Figure 8.7 fits three degrees to the same eleven trips.

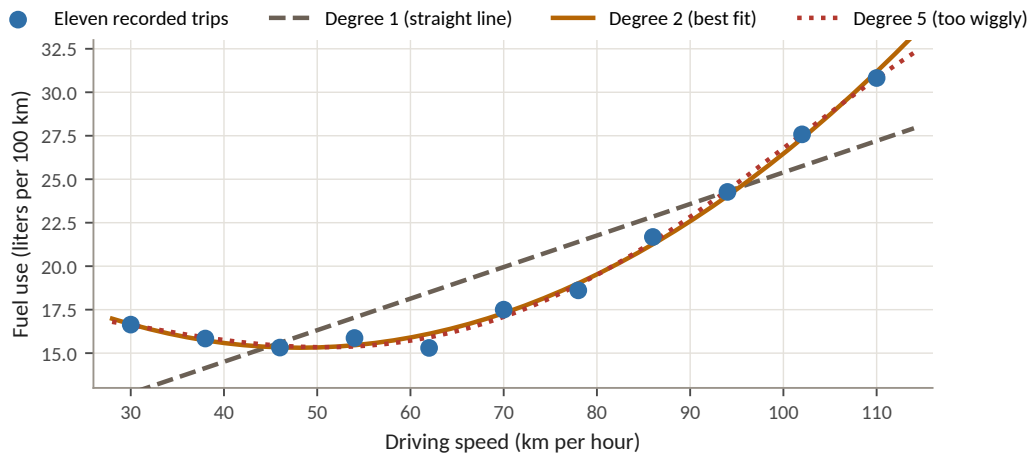


Figure 8.7 – Three polynomial fits to the same eleven delivery trips. Degree 1 is a straight line, degree 2 follows the real curve, and degree 5 bends toward individual points.

WHAT TO OBSERVE

The straight line is right nowhere: it runs far below the slow trips and far above the middle ones. The degree-2 curve matches the data’s shape with one extra coefficient. The degree-5 curve wanders toward single points and gains almost nothing — overfitting produced by raising one hyperparameter.

8.8 Ridge and Lasso: Regularized Regression

Lessons 8.6 and 8.7 both showed how easily a regression model gains too much flexibility. Chapter 7 gave the cure: *regularization*, a penalty for complexity added into the loss — fit error plus λ times a penalty — so that a model must earn every large coefficient. Ridge and lasso are the two standard ways of measuring what “complexity” means for a set of coefficients.

WORDS TO KNOW

Ridge regression — Linear regression with an L2 penalty added to the loss: λ times the sum of the squared coefficients. Ridge shrinks every coefficient toward zero, shrinking the largest ones most, but almost never lands exactly on zero. It was introduced by Arthur Hoerl and Robert Kennard in 1970.

Lasso regression — Linear regression with an L1 penalty added to the loss: λ times the sum of the absolute values of the coefficients. Lasso can push a weak coefficient all the way to exactly zero, which removes that feature from the model entirely. It was proposed by Robert Tibshirani in 1996.

EQUATION: THE RIDGE PENALTY AND THE LASSO PENALTY

$$\text{ridge penalty} = \lambda \sum_{j=1}^k w_j^2 \quad \text{lasso penalty} = \lambda \sum_{j=1}^k |w_j|$$

In words: The ridge penalty multiplies λ by the sum of the squared coefficients. The lasso penalty multiplies λ by the sum of the coefficients’ absolute values, ignoring their signs. Training minimizes the fit error plus the chosen penalty.

When it is used: Whenever a regression model has many features, features that overlap heavily, or fewer examples than a plain fit can safely support.

Symbol	Meaning	Unit or type
λ	regularization strength, a hyperparameter	no unit
w_j	the model's j th coefficient	target unit per feature unit
k	number of coefficients penalized	count
$ w_j $	the coefficient with its minus sign dropped	same unit as w_j

Neither penalty ever includes the intercept. *scikit-learn* calls λ *alpha*, in both *Ridge* and *Lasso*.

WORKED EXAMPLE 8.9 — Compare the Ridge and Lasso Penalties for One Model

A fuller version of Rania's model has a coefficient of 0.5 for home size and 6 for major appliances, with a fit error of $\text{SSE} = 40$ before any penalty. Using $\lambda = 0.2$, find the ridge penalty, the lasso penalty and the total regularized loss under each.

1. Square each coefficient: $0.5^2 = 0.25$ and $6^2 = 36$.
2. Add them and multiply by λ : $(0.25 + 36) \times 0.2 = 36.25 \times 0.2 = 7.25$. That is the ridge penalty.
3. Take the absolute value of each coefficient: $|0.5| = 0.5$ and $|6| = 6$.
4. Add them and multiply by λ : $(0.5 + 6) \times 0.2 = 6.5 \times 0.2 = 1.3$. That is the lasso penalty.
5. Total loss under ridge: $40 + 7.25 = 47.25$.
6. Total loss under lasso: $40 + 1.3 = 41.3$.

Answer: The ridge penalty is 7.25 and the lasso penalty only 1.3, from the very same pair of coefficients. Squaring punishes the single large coefficient far more heavily than the plain absolute value does: 36 of ridge's 36.25 came from the appliance coefficient alone.

The two penalties also behave differently as λ grows. Figure 8.8 follows six coefficients of a larger Erbil model — home size, appliances, occupants, building age, and two features with no real effect on the bill — as the dial is turned from almost nothing up to 1000. Every feature was standardized first, so the six coefficients are directly comparable.

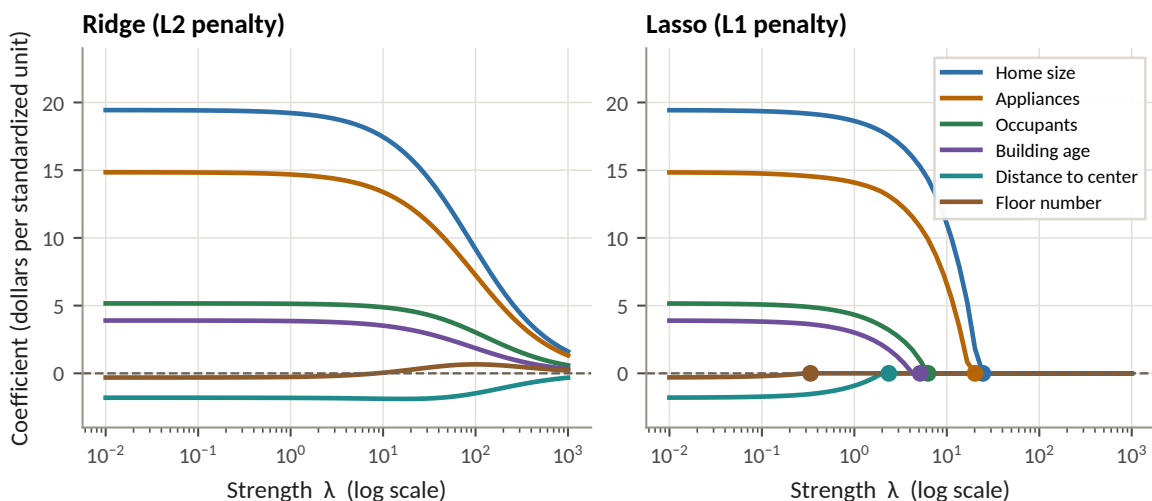


Figure 8.8 – Coefficient paths for the same six-feature bill model under both penalties. Left: ridge. Right: lasso, where a dot marks the strength at which a coefficient first becomes exactly zero.

WHAT TO OBSERVE

Under ridge the curves slide down together and flatten, but not one reaches zero even at $\lambda = 1000$: every feature stays, with its influence turned down. Under lasso the curves land on zero and stay there. The two features with no real effect drop out first (floor number near $\lambda = 0.3$, distance near 2), then building age and occupants, and by $\lambda \approx 22$ even home size is gone — a quick ranking of which features the data really needs.

Table 8.6 sums up the difference.

Table 8.6 – Ridge and lasso compared

Question	Ridge regression (L2)	Lasso regression (L1)
Penalty added to the loss	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Effect on coefficients	Shrinks all of them smoothly toward zero	Shrinks them and sets weak ones to exactly zero
Removes features?	No, every feature stays	Yes, a zero coefficient drops its feature
Best when	Many features that all matter a little, or features that overlap	Many features of which only a few really matter
Standardize first?	Yes, always	Yes, always

COMMON MISTAKE

✗ Wrong: Applying the ridge or lasso penalty to the intercept along with the coefficients, or penalizing features that were never put on a common scale.

✓ Right: Penalize the coefficients only, never the intercept, and standardize every feature first (Chapter 4). Otherwise a feature measured in small units gets a large coefficient purely because of its unit, and the penalty punishes it for that alone.

COMMON MISTAKES AND MISCONCEPTIONS

- Multiplying the coefficient by the feature and then forgetting to add the intercept.
- Reading a coefficient as proof of a cause, or comparing candidate lines by raw SSE when they were scored on different numbers of examples instead of by mean squared error.
- Putting raw feature values into the least-squares slope formula instead of deviations from the mean, and trusting a high R^2 without ever plotting the residuals.
- Comparing coefficients of features measured in very different units without standardizing first.
- Choosing the polynomial degree, or the regularization strength λ , using the training error instead of a validation set.
- Penalizing the intercept, or applying ridge or lasso to features that were never standardized.
- Extrapolating far outside the range the model was trained on.

Practical Activities

ACTIVITY (PAPER) — Fit a Line by the Least-Squares Formulas

A workshop in Zakho records how many hours its diesel generator ran each day and how many liters of fuel it used. Five days gave: 1 hour and 3 liters; 2 hours and 5 liters; 3 hours and 4 liters; 4 hours and 7 liters; 5 hours and 6 liters.

1. Draw axes on graph paper, hours along the bottom (0 to 6) and liters up the side (0 to 8), and plot the five points.
2. Make a six-column table like Table 8.3: x , y , $x - \bar{x}$, $y - \bar{y}$, the product, and $(x - \bar{x})^2$. Fill in the means first: $\bar{x} = 3$ hours and $\bar{y} = 5$ liters.
3. Add the product column and the squared column. Divide the first total by the second to get the slope w ; then compute $b = \bar{y} - w\bar{x}$. Write out the fitted line.
4. Draw your line on the graph and check that it passes through the point $(3, 5)$ — it must. Predict all five days, list the residuals, square them and add them to get the SSE. Check that your residuals add up to zero.
5. Try the line $\hat{y} = 1.0x + 2.0$ instead. Compute its SSE the same way and confirm that it is larger than yours.

You need: Graph paper, pencil, ruler and a calculator

ACTIVITY (PYTHON) — The Same Line From scikit-learn

This program repeats Worked Examples 8.5, 8.6 and 8.8 with scikit-learn, then turns the regularization dial.

Python · NumPy · scikit-learn

```
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

X = np.array([[50], [70], [90], [110], [130]])    # home size (m2)
y = np.array([50, 66, 82, 94, 110])              # true monthly bill (dollars)

lin = LinearRegression().fit(X, y)
p = lin.predict(X)
print("w and b      :", round(lin.coef_[0], 4), round(lin.intercept_, 4))
print("predictions :", np.round(p, 2))
print("residuals    :", np.round(y - p, 2))
print("SSE and MSE :", round(np.sum((y - p) ** 2), 3), round(mean_squared_error(y, p), 3))
print("R2          :", round(r2_score(y, p), 4))

speed = np.array([30, 38, 46, 54, 62, 70, 78, 86, 94, 102, 110])
fuel = np.array([16.64, 15.84, 15.32, 15.87, 15.30, 17.50, 18.61, 21.68, 24.27, 27.58,
                 30.82])
S2 = PolynomialFeatures(degree=2, include_bias=False).fit_transform(speed.reshape(-1, 1))
quad = LinearRegression().fit(S2, fuel)
print("degree-2 fit:", round(quad.intercept_, 2), np.round(quad.coef_, 4))

Xz = (X - X.mean()) / X.std()                    # standardize before penalizing
for name, model in [("ridge", Ridge), ("lasso", Lasso)]:
    for a in (1.0, 50.0):
        print(name, "lambda =", a, "-> coefficient", np.round(model(alpha=a).fit(Xz,
            y).coef_, 3))
```

Output

```
w and b      : 0.74 13.8
predictions : [ 50.8  65.6  80.4  95.2 110. ]
residuals    : [-0.8   0.4   1.6  -1.2  -0. ]
SSE and MSE : 4.8 0.96
R2           : 0.9978
degree-2 fit: 24.89 [-0.3982  0.0041]
ridge lambda = 1.0 -> coefficient [17.442]
ridge lambda = 50.0 -> coefficient [1.903]
lasso lambda = 1.0 -> coefficient [19.93]
lasso lambda = 50.0 -> coefficient [0.]
```

LinearRegression returns exactly the slope 0.74 and intercept 13.8 found by hand in Worked Example 8.5, with the same residuals, SSE, MSE and R^2 as Worked Example 8.6. The degree-2 fit recovers Worked Example 8.8's model, $26 - 0.40v + 0.004v^2$, from the data alone. The last four lines penalize the standardized size feature, whose unpenalized coefficient is about 20.9: ridge shrinks it to 17.4 and then 1.9 but never to zero, while lasso drops it to exactly 0.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Estimating Electricity Bills Before the Meter Reading — Erbil

Rania's five-home examples were built for hand calculation, far too small to trust with real customers. Her office's records cover about 4,000 homes over two years, with home size, appliance count, occupant count and building age beside each month's true bill. The decision the model must support is narrow: give a representative an estimated bill range while the customer is still on the phone, days before the meter is read.

The team splits the homes into training, validation and test sets before looking at any model, as Chapter 5 requires, and agrees in advance that success means a mean absolute error under 6 dollars on the test set. An ordinary least-squares fit on all four features looks good at first, but the building-age coefficient comes out implausibly large and its sign flips whenever the team refits on a different split. A few very old buildings with high bills, for reasons unrelated to age, are dragging it around — exactly the instability Lesson 8.8 predicts.

So the team standardizes every feature and refits with ridge regression, testing several values of λ against validation error. A moderate λ steadies the building-age coefficient at a small, sensible value without hurting validation error. Lasso, tried out of curiosity, drives occupant count to exactly zero: once size and appliances are known, it adds almost nothing. The validation residual plot is a clean band, so no polynomial term is needed.

The finished tool never shows one falsely confident number. It shows a range, built from the model's typical validation error, with a plain sentence saying that new construction, a recent renovation or an unusually hot month makes the estimate less reliable. The office retrains once a year, because habits and tariffs change and a rule learned from old bills need not fit new ones.

RESPONSIBLE USE

An estimated bill is a prediction, not an invoice. A customer must never be charged, disconnected or refused service on the basis of a model's estimate; the real meter reading must always decide. When a predicted number affects what a person pays, publish the typical error beside the estimate and give the person an easy way to reach a human who can check it.

CHAPTER SUMMARY

- Linear regression predicts a number by multiplying each feature by its own coefficient and adding one fixed intercept: $\hat{y} = wx + b$ for a single feature.
- The coefficient (slope) is how much the prediction changes per extra unit of the feature; the intercept is the prediction when every feature is zero.
- A residual is the true value minus the predicted value, $y - \hat{y}$; positive means the model predicted too low. The sum of squared errors adds up the squared residuals, and the mean squared error divides that total by the number of examples.
- Least squares picks the line with the smallest SSE; for one feature its slope and intercept come from two closed-form formulas built on deviations from the mean. The fitted line always passes through $(\bar{x}, \bar{y})$ and its residuals always add to zero.
- R^2 compares the model's squared error against always predicting the mean: 1 is perfect, 0 is no better than the mean; report it beside RMSE or MAE.
- An actual-versus-predicted plot and a residual plot catch what a single score hides; any visible shape in a residual plot means a missing pattern.
- Multiple linear regression adds one coefficient per feature, each meaning "the change per unit of this feature with all others held fixed"; polynomial regression fits powers of a feature as extra features, so a linear model can follow a curve.
- Ridge (L2) shrinks all coefficients smoothly toward zero, while lasso (L1) can set weak coefficients to exactly zero and drop their features; both need standardized features and never penalize the intercept.

- A regression prediction is a best estimate with a typical error attached, never a fact, and must not be extrapolated far beyond the data it was fitted on.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define coefficient and intercept in your own words, one sentence each, and give the unit of each for a model that predicts a monthly bill in dollars from a home size in square meters.

Problem 2 [Basic] A regression line is $\hat{y} = 12 + 0.6x$, where x is home size in square meters. What does it predict for a 0 m^2 home, and why is that particular prediction not meaningful in the real world?

Problem 3 [Basic] A model predicts 75 dollars for a home whose true bill is 81 dollars. Find the residual and state whether the model predicted too high or too low.

Problem 4 [Basic] Explain in your own words the main difference between ridge regression and lasso regression, and name one situation where you would prefer each.

Problem 5 [Standard] A line's five residuals, in dollars, are 3, -2 , 1, -4 and 2. Find the sum of squared errors and the mean squared error, showing every step.

Problem 6 [Standard] Using $\hat{y} = 8 + 0.5 \times \text{size} + 7 \times \text{appliances}$, predict the bill of a 100 m^2 home with 3 major appliances, showing every step. Then state, in one sentence, what the coefficient 7 means.

Problem 7 [Standard] A tiny dataset has $x = 1, 2, 3, 4$ and $y = 4, 6, 7, 11$. Build the least-squares calculation table, then find the slope and the intercept. (Check: $\bar{x} = 2.5$ and $\bar{y} = 7$.)

Problem 8 [Standard] A fitted model has $\text{SSE} = 12$ and $\text{SST} = 60$. Find R^2 and say in one sentence what it means.

Problem 9 [Standard] A model has a coefficient of 3 for one feature and 1 for another. Using $\lambda = 0.5$, find the ridge penalty and the lasso penalty, showing every step.

Problem 10 [Challenge] A model has an R^2 of 0.93 on its training data, but its residual plot shows residuals that are positive at both ends of the range and negative in the middle. (a) What is the model missing? (b) Which technique from this chapter would you try first, and why? (c) How would you decide how far to take it without overfitting?

Problem 11 [Challenge] A degree-2 model of crop yield is $\hat{y} = 12 - 0.5x + 0.01x^2$, with x the fertilizer in kg per hectare. (a) Predict the yield at 30 and at 40 kg per hectare. (b) Explain why predicting for $x = 200$ would be unwise even though the arithmetic works.

Problem 12 [Challenge] Rania's fitted line is $\hat{y} = 0.74x + 13.8$ with x in square meters. A colleague re-records every size in hundreds of square meters instead, so 130 m^2 becomes 1.3. (a) What is the new slope, and what is its unit? (b) Does the intercept change? (c) Does any prediction change? (d) What does this tell you about judging a feature's importance from the size of its coefficient?

Problem 13 [Challenge] Design your own two-feature linear regression project for a place you know well — a shop, a farm, a school or a clinic. Name the two features and the target with their units, write a plausible model with invented coefficients, say what each coefficient would mean, and explain why you would consider ridge or lasso before trusting it.

Gradient Descent

How a computer finds the best parameters, one small step at a time

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why the loss of a model can be pictured as a landscape with hills and valleys.
- Calculate the loss of a candidate line, and the two partial derivatives of that loss, by hand.
- Carry out several rounds of gradient descent with a pencil, updating the coefficient and the intercept each round.
- Choose a learning rate, and recognize the loss curves produced by a rate that is too small, just right, or far too large.
- Tell batch, stochastic and mini-batch gradient descent apart, and count the iterations in one epoch.
- Explain why standardizing the features makes gradient descent converge much faster.
- Decide when a closed-form formula is better than gradient descent, and when it is not.

Introduction

Imagine walking down a mountain path above Zakho on a moonless night. You know the village is somewhere at the bottom of the valley, but you cannot see it, and your torch is dead. You cannot plan the whole route. What you *can* do is feel the ground under your feet: test which way it slopes downward, take one careful step that way, feel again, repeat. Step after step, without ever seeing the valley floor, you get lower and lower — and eventually you arrive.

This is how a computer trains most of the models in this book. Chapter 8 showed that a straight line is described by two numbers, a coefficient w and an intercept b , and that a loss function grades any pair by how badly its line misses the data. What it did not explain is how a computer *finds* the pair with the smallest loss when there are millions of pairs and no time to try them all.

The answer is gradient descent. It starts from any guess, works out which direction is downhill in the loss, takes one small step that way, and repeats until the steps stop mattering. In this chapter you will run it by hand on four homes in Erbil, watch the loss fall from 41 to under 4 in three rounds, see what happens when the steps are too big, and reproduce the whole thing in fifteen lines of Python.

9.1 Searching for the Bottom of a Valley

Rania works for the electricity company in Erbil, and Chapter 8 left her with a question. She has a line, $\hat{y} = wx + b$, and a way to grade it, the mean squared error from Chapter 6. But how does a computer choose the best w and the best b ? To keep every number checkable by hand, this chapter shrinks her problem to four homes and one very simple feature: the number of air conditioners. The target is the home's electricity cost for one hot day, in dollars, and Table 9.1 is the whole training set.

Table 9.1 – Four Erbil homes on one hot day (illustrative example data)

Home	Air conditioners x (count)	Daily cost y (dollars)
A	1	3
B	2	5
C	3	7
D	4	9

A person can see the pattern here at a glance: each extra air conditioner adds \$2, and there is a fixed \$1 on top. In symbols, $w = 2$ and $b = 1$ fit all four homes exactly. The computer knows nothing of this. It only knows how to try a pair of numbers and measure how badly that pair misses.

WORD TO KNOW

Loss surface — A picture of the loss for every possible choice of a model's parameters. Each pair (w, b) is a location on a map, and the loss for that pair is the height of the ground there. A low place means a good model; a high place means a bad one.

The loss surface is not something a computer draws; it is a way for us to think about the search. With two parameters it is a landscape you could walk on, and training means finding its lowest point. The loss itself is the mean squared error from Chapter 6, written now as a function of w and b instead of as one fixed number.

EQUATION: THE LOSS SURFACE OF A STRAIGHT LINE

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

In words: Pick a coefficient and an intercept. Predict every training example with them, subtract each prediction from the true value, square every difference, and take the average. That average is the height of the loss surface at that pair of numbers.

When it is used: Whenever you need one single number that grades a whole candidate line, so that two candidates can be compared, or so that a search method knows which way to move.

Symbol	Meaning	Unit or type
$L(w, b)$	the loss for this particular pair of parameters	target unit squared
w	the coefficient being tested	target unit per unit of x
b	the intercept being tested	same unit as y
x_i, y_i	the feature value and true value of example i	their own units
n	number of training examples	count

This is exactly the mean squared error of Chapter 6. The only new idea is to treat it as a function of the parameters, with the data held fixed, instead of as a score for one finished model.

WORKED EXAMPLE 9.1 — Find the Height of the Loss Surface at Three Places

Using the four homes in Table 9.1, find the loss $L(w, b)$ at three candidate pairs: $(w = 0, b = 0)$, $(w = 1, b = 1)$ and $(w = 2, b = 1)$.

1. With $w = 0$ and $b = 0$ every prediction is $0 \times x + 0 = 0$, so the residuals are the true values themselves: 3, 5, 7, 9. Square them: 9, 25, 49, 81; add: 164; divide by $n = 4$: $164 \div 4 = 41$.

2. With $w = 1$ and $b = 1$ the predictions are 2, 3, 4, 5, so the residuals are 1, 2, 3, 4. Square and average: $1 + 4 + 9 + 16 = 30$, and $30 \div 4 = 7.5$.
3. With $w = 2$ and $b = 1$ the predictions are 3, 5, 7, 9, so every residual is 0 and the loss is 0.

Answer: The three heights are $L(0, 0) = 41$, $L(1, 1) = 7.5$ and $L(2, 1) = 0$ dollars squared. The pair $(2, 1)$ sits at the very bottom of this landscape; the pair $(0, 0)$ sits high on a slope.

Trying pairs one at a time worked here only because we already knew the answer. A model with 20 features has 21 parameters, and even a coarse grid of ten values for each would mean 10^{21} combinations — more than a computer could test before the sun goes out. A search method has to be smarter than guessing. Luckily, the mean squared error of a linear model has a shape that makes searching easy.

WORDS TO KNOW

Convex function — A function whose graph is bowl-shaped: a straight line joining any two points on the graph never dips below the graph itself. A convex loss has exactly one lowest region and no misleading dips along the way.

Global minimum — The lowest point of the whole loss surface — the best parameters that exist for this model and this data.

Local minimum — A dip that is lower than everything immediately around it, but still higher than the global minimum. A search that walks only downhill can get stuck in one.

The least-squares loss of a linear model is convex. That is a strong promise: the landscape is one clean bowl, so any method that keeps walking downhill must reach the global minimum, wherever it starts. Losses of other models, such as the ones in Chapter 18, are not convex and can trap a downhill walker in a local minimum. Figure 9.1 shows a slice through Rania's bowl, taken by freezing the intercept at $b = 1$ so that the whole picture fits on one axis.

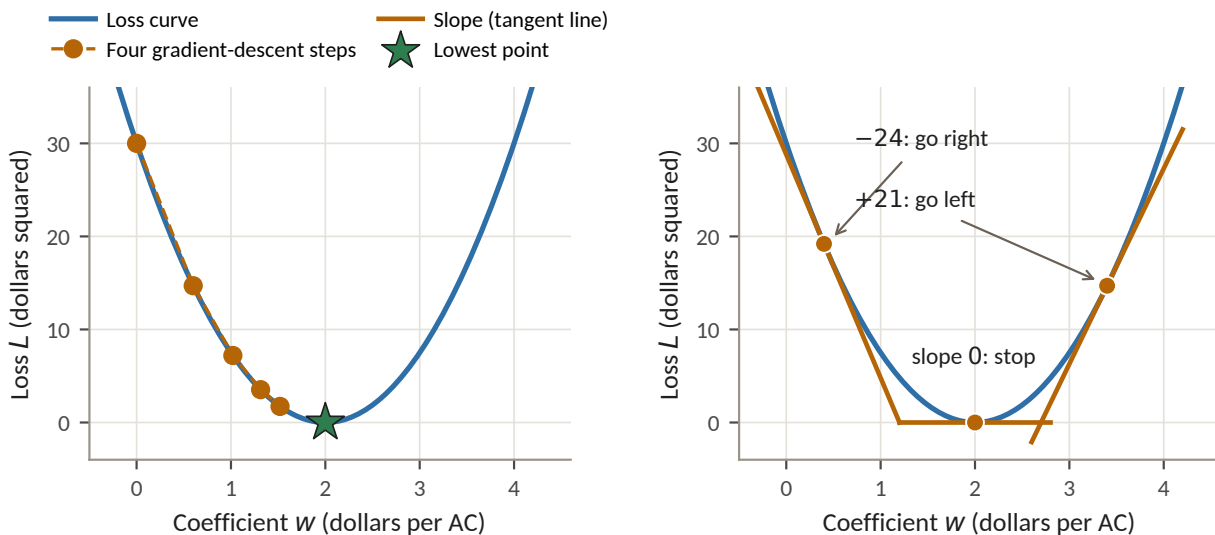


Figure 9.1 — A slice through the loss surface of the four homes, with the intercept frozen at $b = 1$ so that only w varies. Left: the first four steps of gradient descent, computed later in this chapter. Right: the slope of the loss at three places, used in Lesson 9.2.

WHAT TO OBSERVE

Both panels show one clean bowl with a single lowest point at $w = 2$, where the loss is 0. On the left the four steps get smaller as the ground flattens — 0.60, then 0.42, then 0.29, then 0.21 — without anyone telling the walker to slow down. On the right the tangent at $w = 0.4$ falls steeply to the right, the one at $w = 3.4$ rises to the right, and the one at $w = 2$ is flat.

9.2 The Gradient: Which Way Is Downhill?

Standing on the hillside above Zakho in the dark, you do not need a map, only one piece of local information: which way the ground tilts under your feet. Mathematics has a precise name for that tilt.

WORD TO KNOW

Partial derivative — The slope of a function when you change one variable and hold all the others perfectly still. Written $\partial L / \partial w$ and read “the partial derivative of L with respect to w ”, it answers: if w grew by a tiny amount and everything else stayed the same, how fast would L change?

A partial derivative is a number with a sign, and the sign is the useful part, as the right-hand panel of Figure 9.1 and Table 9.2 both show. Its size matters too: a steep slope says the correction is urgent.

Table 9.2 – Reading the slope of the loss at three values of w (right panel of Figure 9.1)

w	Slope $\partial L / \partial w$	Downhill is	What gradient descent does
0.4	−24 (negative)	to the right	Increases w by a lot
2.0	0	nowhere	Leaves w where it is: the bottom
3.4	+21 (positive)	to the left	Decreases w

A model has one partial derivative for each parameter. Collected together they form a single object with its own name.

WORD TO KNOW

Gradient — The list of all the partial derivatives of the loss, one for each parameter. The gradient points in the direction in which the loss increases fastest, so the *opposite* of the gradient points in the direction in which the loss falls fastest — straight downhill.

For a straight line there are exactly two partial derivatives, one for w and one for b . Both are built from the residuals of Chapter 8: the true value minus the prediction, $r_i = y_i - \hat{y}_i$. The derivation belongs to a calculus course (Appendix A shows where it comes from); the two formulas themselves use nothing but multiplication and addition.

EQUATION: THE GRADIENT OF THE MEAN SQUARED ERROR

$$\frac{\partial L}{\partial w} = -\frac{2}{n} \sum_{i=1}^n x_i r_i \quad \frac{\partial L}{\partial b} = -\frac{2}{n} \sum_{i=1}^n r_i \quad \text{where } r_i = y_i - (wx_i + b)$$

In words: For the coefficient: multiply every example’s feature value by that same example’s residual, add all those products, divide by the number of examples, double it, and change the sign. For the intercept: do the same with the plain residuals, without multiplying by the feature first.

When it is used: Once per round of training, to find out which way to move the coefficient and the intercept, and how far the loss is currently tilting.

Symbol	Meaning	Unit or type
$\partial L / \partial w$	slope of the loss in the w direction	loss unit per unit of w
$\partial L / \partial b$	slope of the loss in the b direction	loss unit per unit of b
r_i	residual of example i with the current w and b	same unit as y
x_i	feature value of example i	unit of the feature
n	number of training examples	count

Both formulas carry a minus sign. If the residuals are mostly positive — the model is predicting too low — both derivatives are negative, which correctly says that raising w and b would lower the loss.

WORKED EXAMPLE 9.2 — Compute the Gradient for the Four Homes at $w = 0$, $b = 0$

Using Table 9.1, find both partial derivatives at the starting guess $w = 0$ and $b = 0$.

1. Predict every home with the current guess: $0 \times x + 0 = 0$, so all four predictions are 0.
2. Find each residual, true minus predicted: 3, 5, 7, 9. Add them: $3 + 5 + 7 + 9 = 24$.
3. Multiply each residual by its own feature value and add: $(1 \times 3) + (2 \times 5) + (3 \times 7) + (4 \times 9) = 3 + 10 + 21 + 36 = 70$.
4. Coefficient derivative: $\partial L / \partial w = -\frac{2}{4} \times 70 = -0.5 \times 70 = -35$.
5. Intercept derivative: $\partial L / \partial b = -\frac{2}{4} \times 24 = -0.5 \times 24 = -12$.

Answer: The gradient is $(-35, -12)$. Both numbers are negative, so the loss falls if both w and b are increased — which makes sense, because a line through the origin with zero slope predicts \$0 for every home and is far too low.

Both numbers here are large, a warning that a step of ordinary size could carry us a long way. The next example shows how large “large” gets when a feature is measured in big units.

WORKED EXAMPLE 9.3 — Compute the Gradient for Rania’s Five Houses at $w = 0.5$, $b = 0$

Rania’s five houses from Chapter 8 have sizes 50, 70, 90, 110 and 130 square meters and true monthly bills of \$50, \$66, \$82, \$94 and \$110. Find both partial derivatives at $w = 0.5$, $b = 0$.

1. Predict each bill: $0.5 \times 50 = 25$, then 35, 45, 55, 65.
2. Residuals, true minus predicted: 25, 31, 37, 39, 45. Add them: 177.
3. Multiply each residual by its own size and add: $(50 \times 25) + (70 \times 31) + (90 \times 37) + (110 \times 39) + (130 \times 45) = 1250 + 2170 + 3330 + 4290 + 5850 = 16,890$.
4. Coefficient derivative: $-\frac{2}{5} \times 16,890 = -0.4 \times 16,890 = -6756$.
5. Intercept derivative: $-\frac{2}{5} \times 177 = -0.4 \times 177 = -70.8$.

Answer: The gradient is $(-6756, -70.8)$. Both point the same way, but the coefficient derivative is about 95 times larger, only because home size is measured in numbers near 100 while the intercept is not multiplied by anything. Lesson 9.7 shows why this imbalance causes trouble and how to remove it.

9.3 One Step of Gradient Descent

Knowing the downhill direction is not enough; you still have to move. Gradient descent moves by subtracting a small multiple of the gradient from the current parameters — subtracting is what turns “uphill” into “downhill”.

WORDS TO KNOW

Gradient descent — A method that finds good parameter values by repeatedly computing the gradient of the loss and taking a small step in the opposite direction. It was first described by the mathematician Augustin-Louis Cauchy in 1847, long before computers existed.

Learning rate — A small positive number, written α , chosen before training begins, that decides how far each step goes. It is a hyperparameter (Chapter 7), not something training discovers by itself.

Iteration — One complete round of gradient descent: predict, find the residuals, compute the gradient, update every parameter once.

EQUATION: ONE STEP OF GRADIENT DESCENT

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

In words: Multiply each partial derivative by the learning rate, and subtract the result from the parameter it belongs to. Do this for every parameter, using the derivatives computed before any of them changed.

When it is used: Once per iteration, for every parameter of any model whose loss can be differentiated: linear regression here, logistic regression in Chapter 10, support vector machines in Chapter 16.

Symbol	Meaning	Unit or type
$\leftarrow$	“becomes”: the value on the right replaces the old value	—
α	the learning rate, chosen before training	no unit
$\partial L / \partial w$	the coefficient’s partial derivative, from the box above	loss unit per unit of w
$\partial L / \partial b$	the intercept’s partial derivative	loss unit per unit of b

Compute both derivatives first, then update both parameters. Updating w and then recomputing the residuals before updating b is a different method, and it is not the one described here.

Every iteration repeats the same four actions, shown as a flowchart in Figure 9.2: predict, subtract, differentiate, step.

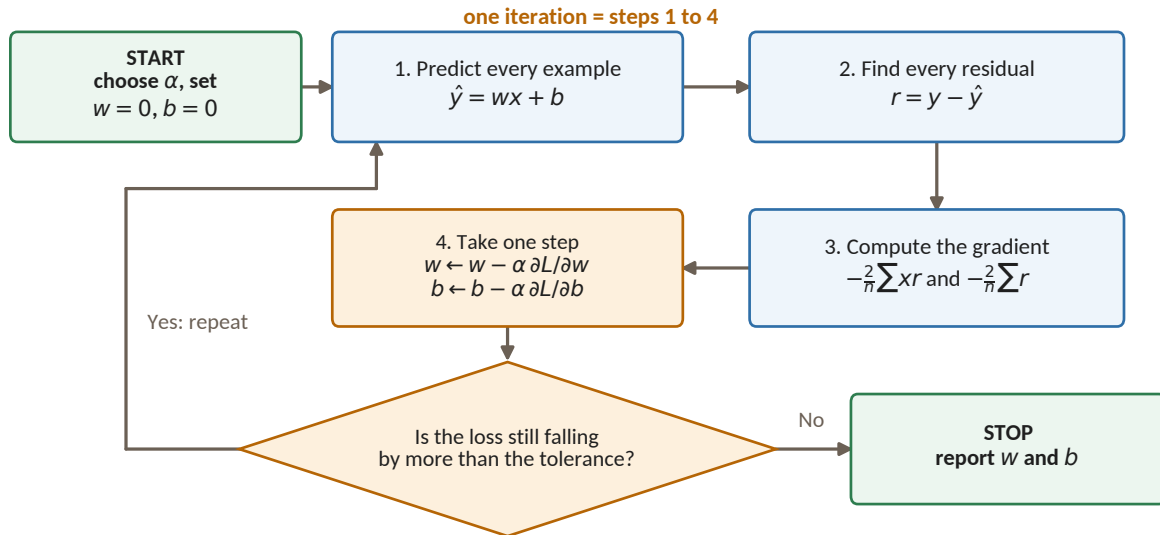


Figure 9.2 – The gradient-descent algorithm. Steps 1 to 4 are one iteration; the loop repeats until the loss stops falling.

WHAT TO OBSERVE

The loop returns to step 1, not to START: the learning rate and the starting values are chosen once, and after that only w and b change. The only decision inside the loop is the diamond at the bottom; Lesson 9.5 explains how to answer it.

The next three worked examples run this loop three times on the four homes, starting from $w = 0$ and $b = 0$ with $\alpha = 0.02$, rounding w and b to two decimals at the end of every round.

WORKED EXAMPLE 9.4 – Round One of Gradient Descent

Start from $w = 0$, $b = 0$, with $\alpha = 0.02$ and the four homes of Table 9.1. Find the parameters after one iteration, and the new loss.

1. The gradient at this point was already found above: $\partial L / \partial w = -35$ and $\partial L / \partial b = -12$.
2. Update the coefficient: $w \leftarrow 0 - 0.02 \times (-35) = 0 + 0.70 = 0.70$.
3. Update the intercept: $b \leftarrow 0 - 0.02 \times (-12) = 0 + 0.24 = 0.24$.
4. The new line $\hat{y} = 0.70x + 0.24$ predicts 0.94, 1.64, 2.34, 3.04, so the new residuals are 2.06, 3.36, 4.66, 5.96.
5. New loss: $(2.06^2 + 3.36^2 + 4.66^2 + 5.96^2) \div 4 = 72.77 \div 4 = 18.19$.

Answer: After one round $w = 0.70$ and $b = 0.24$, and the loss has fallen from 41 to 18.19 dollars squared — a drop of more than half from a single step.

WORKED EXAMPLE 9.5 – Round Two of Gradient Descent

Continue from $w = 0.70$ and $b = 0.24$, with the same learning rate.

1. The residuals here were found in the previous example: 2.06, 3.36, 4.66, 5.96. Their sum is 16.04.
2. Multiply each by its feature value and add: $2.06 + 6.72 + 13.98 + 23.84 = 46.60$.
3. Gradient: $\partial L / \partial w = -0.5 \times 46.60 = -23.30$ and $\partial L / \partial b = -0.5 \times 16.04 = -8.02$.

4. Update: $w \leftarrow 0.70 + 0.02 \times 23.30 = 1.17$ and $b \leftarrow 0.24 + 0.02 \times 8.02 = 0.40$, both rounded to two decimals.
5. The line $\hat{y} = 1.17x + 0.40$ predicts 1.57, 2.74, 3.91, 5.08; its residuals are 1.43, 2.26, 3.09, 3.92, and its loss is $32.07 \div 4 = 8.02$.

Answer: After two rounds $w = 1.17$ and $b = 0.40$, and the loss is 8.02 dollars squared. The gradient has shrunk from -35 to -23.30 , so the second step was smaller than the first without anyone shrinking it by hand.

WORKED EXAMPLE 9.6 — Round Three of Gradient Descent

Continue from $w = 1.17$ and $b = 0.40$.

1. Residuals from the previous round: 1.43, 2.26, 3.09, 3.92, summing to 10.70; and $1.43 + 4.52 + 9.27 + 15.68 = 30.90$.
2. Gradient: $\partial L / \partial w = -0.5 \times 30.90 = -15.45$ and $\partial L / \partial b = -0.5 \times 10.70 = -5.35$.
3. Update: $w \leftarrow 1.17 + 0.02 \times 15.45 = 1.48$ and $b \leftarrow 0.40 + 0.02 \times 5.35 = 0.51$.
4. The line $\hat{y} = 1.48x + 0.51$ has residuals 1.01, 1.53, 2.05, 2.57 and loss $14.17 \div 4 = 3.54$.

Answer: After three rounds $w = 1.48$ and $b = 0.51$, and the loss is 3.54 dollars squared. Three small steps have taken the loss from 41 down to 3.54, and the line is closing in on the perfect answer $w = 2$, $b = 1$.

Table 9.3 collects the three rounds side by side, and Figure 9.3 draws the same three rounds as a path across the loss surface, seen from above.

Table 9.3 – Three rounds of gradient descent on the four homes, $\alpha = 0.02$. Each row shows the parameters and loss after that round, together with the gradient measured there, which drives the next round.

Round	w	b	$\sum r_i$	$\sum x_i r_i$	$\partial L / \partial w$	Loss
start	0.00	0.00	24.00	70.00	-35.00	41.00
1	0.70	0.24	16.04	46.60	-23.30	18.19
2	1.17	0.40	10.70	30.90	-15.45	8.02
3	1.48	0.51	7.16	20.50	-10.25	3.54

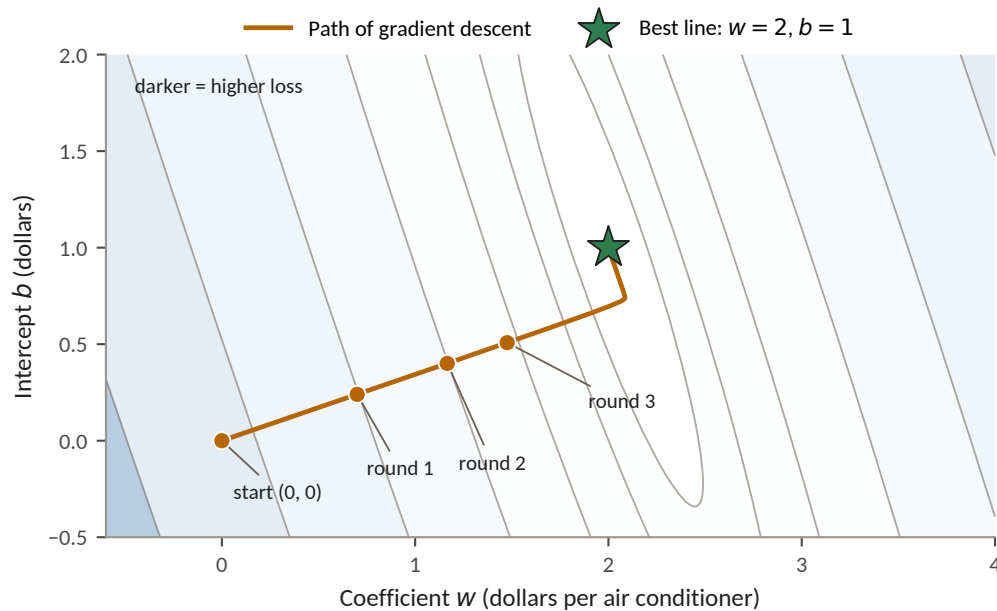


Figure 9.3 – The loss surface of the four homes seen from above, with the path taken by gradient descent from the starting guess to the best line.

WHAT TO OBSERVE

Every ring joins pairs (w, b) with the same loss, exactly like the height lines on a hiking map. The rings are long ovals rather than circles, so the surface is a narrow valley, not a round bowl. The path crosses the valley quickly, then turns to creep along its floor toward the star — a pattern that returns in Lesson 9.7.

COMMON MISTAKE

✗ Wrong: Writing $w \leftarrow w + \alpha \partial L / \partial w$, adding the gradient instead of subtracting it.

✓ Right: Always subtract. The gradient points *uphill*, toward a larger loss. Adding it makes the loss grow every round, which is the single most common reason a hand calculation of gradient descent goes wrong.

IMPORTANT

A model trained on four rows is a teaching device, never a tool. Everything Chapter 7 said about overfitting still applies: real training needs many examples and a separate validation set, however neatly the loss falls on a practice dataset.

9.4 The Learning Rate: Too Small, Too Large, Just Right

The learning rate α is the one number a person must supply before gradient descent can run, and it decides almost everything about how the run behaves. Think of it as the length of your stride on the dark hillside: tiny shuffling steps are safe but slow, while enormous leaps may carry you straight over the valley and up the opposite slope. Figure 9.4 runs the same slice of Rania's bowl three times with three different rates.

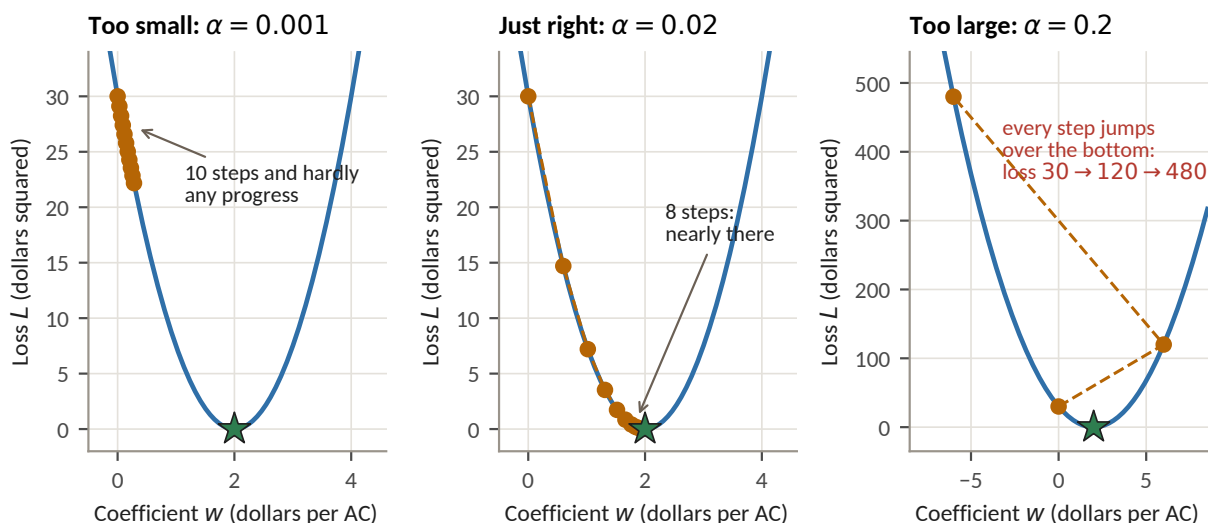


Figure 9.4 – The same loss slice with three learning rates. Note that the third panel needs a completely different scale on both axes.

WHAT TO OBSERVE

With $\alpha = 0.001$ the dots barely move: ten steps take w only from 0 to 0.28, and the loss is still 22. With $\alpha = 0.02$ eight steps bring w to 1.88 and the loss down to 0.10. With $\alpha = 0.2$ each step jumps clean over the bottom and lands higher than it started: the loss goes $30 \rightarrow 120 \rightarrow 480$, and the next step would put w at 18, far off the chart. Nothing about the third run recovers on its own.

The third panel shows a failure with its own name: when steps overshoot so badly that the loss grows round after round, the run is said to *diverge*. Divergence is not slow learning but the opposite of learning, and it usually shows up within the first few iterations.

WORKED EXAMPLE 9.7 – One Step With a Learning Rate That Is Far Too Large

Start again from $w = 0$, $b = 0$ on the four homes, but with $\alpha = 0.2$ instead of 0.02. Find the parameters and the loss after one step, then after a second step.

1. The gradient at the start is unchanged: $\partial L / \partial w = -35$, $\partial L / \partial b = -12$.
2. Update: $w \leftarrow 0 + 0.2 \times 35 = 7$ and $b \leftarrow 0 + 0.2 \times 12 = 2.4$.
3. The line $\hat{y} = 7x + 2.4$ predicts 9.4, 16.4, 23.4, 30.4 — far above the true costs of \$3 to \$9. Residuals: $-6.4, -11.4, -16.4, -21.4$; loss $897.84 \div 4 = 224.46$.
4. The residuals are now all negative, so the next gradient points the other way: $\partial L / \partial w = +82$ and $\partial L / \partial b = +27.8$.
5. Second update: $w \leftarrow 7 - 0.2 \times 82 = -9.4$ and $b \leftarrow 2.4 - 0.2 \times 27.8 = -3.16$, giving a loss of 1229.13.

Answer: One step took the loss from 41 up to 224.46, and the second took it to 1229.13. The run is diverging: each step corrects too hard in the opposite direction, and the swings grow instead of settling.

Table 9.4 compares the three runs after 1, 5 and 20 steps.

Table 9.4 – Training loss (dollars squared) after 1, 5 and 20 iterations, from the same starting guess

Rate α	After 1	After 5	After 20	Verdict
0.001	39.64	34.65	20.91	Safe but far too slow
0.02	18.19	0.72	0.01	Just right
0.2	224.46	201,848	over 10^{16}	Diverging

For this small dataset there is a hard ceiling: any rate above about 0.12 makes the run diverge. That ceiling depends on the data, not the model, which is why no single learning rate is right for every problem. In practice teams start around 0.01 or 0.1, watch the first ten iterations, and multiply or divide by 3 until the loss falls steadily. Chapter 23 turns that habit into a proper search.

COMMON MISTAKE

✗ Wrong: Raising the learning rate to finish training sooner, on the reasoning that bigger steps must mean faster progress.

✓ Right: Raise it only while the loss keeps falling steadily. A rate above the ceiling does not train slowly — it destroys the model completely, often within three iterations. Halve the rate and restart the moment the loss rises.

Two refinements are worth naming, because you will meet them in software. *Momentum*, introduced by Boris Polyak in 1964, lets a step remember the direction of the previous one, so the run rolls along a valley floor instead of bouncing across it. *Adaptive* methods such as AdaGrad, published by John Duchi, Elad Hazan and Yoram Singer in 2011, give every parameter its own learning rate. Both are variations on the same four steps.

9.5 Reading a Loss Curve

The most useful picture in training is the *loss curve* of Chapter 7: the training loss plotted against the iteration number. It answers three questions at a glance — is the learning rate sensible, is the run finished, is anything badly wrong. Figure 9.5 shows the healthy case and, beside it, all three learning rates on one logarithmic scale.

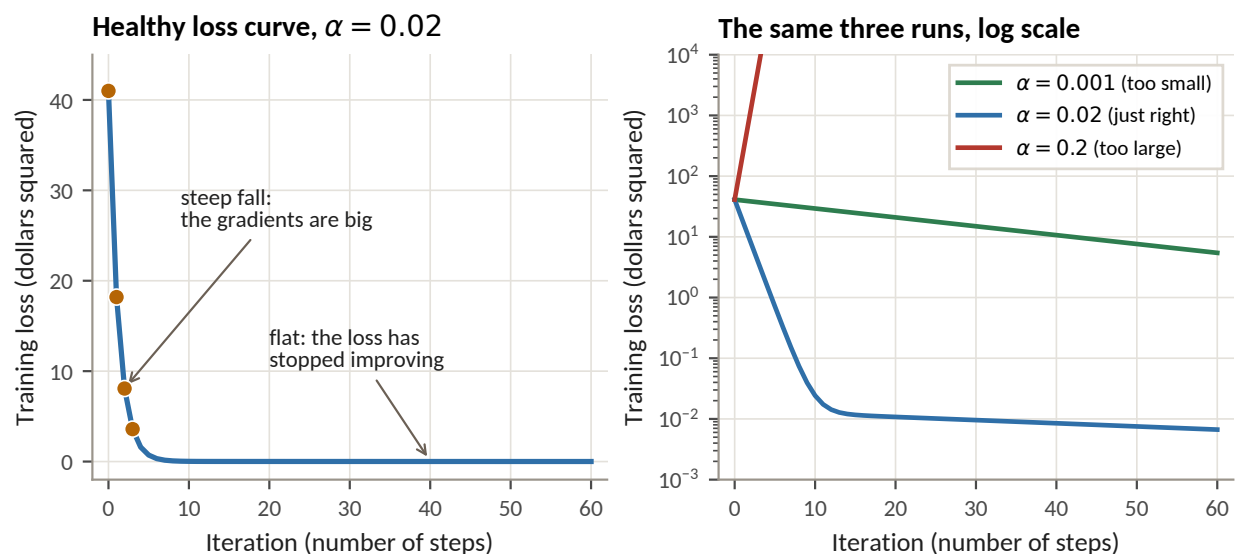


Figure 9.5 – Left: the training loss of the four-home model against the iteration number, with $\alpha = 0.02$. Right: the same run and two others on a logarithmic scale, which lets very small and very large losses share one axis.

WHAT TO OBSERVE

A healthy curve falls steeply, then bends, then flattens: 41 at the start, 18.19 after one iteration, 0.024 after ten, 0.0075 after fifty, 0.0012 after two hundred. On the right, the green curve is almost a straight slow slide (rate too small), the blue one drops and levels off (rate just right), and the red one shoots off the top of the chart within ten iterations (rate too large).

Training has to stop somewhere. The usual rule watches how much the loss improved during the last iteration and stops once that improvement stops being interesting.

WORD TO KNOW

Convergence — The point at which further iterations no longer change the parameters or the loss in any useful way. A run that reaches this point has *converged*; a run whose loss keeps growing instead has *diverged*.

EQUATION: THE STOPPING RULE

$$L_{t-1} - L_t < \varepsilon \implies \text{stop training}$$

In words: Subtract the current iteration's loss from the previous iteration's loss. If that improvement is smaller than a small number chosen in advance, further steps are not worth taking, so stop.

When it is used: At the end of every iteration, to decide automatically when a run has converged instead of fixing the number of iterations by guesswork.

Symbol	Meaning	Unit or type
L_t	training loss after iteration t	target unit squared
L_{t-1}	training loss after the previous iteration	target unit squared
ε	the tolerance: the smallest improvement still worth a step	same unit as the loss

Software usually also sets a maximum number of iterations, so that a run which never converges still ends. In scikit-learn these two settings are called `tol` and `max_iter`.

WORKED EXAMPLE 9.8 — Decide When to Stop Using a Tolerance of 0.05

The four-home run with $\alpha = 0.02$ produced these training losses: iteration 3, 3.5896; iteration 4, 1.5993; iteration 5, 0.7165; iteration 6, 0.3248; iteration 7, 0.1510; iteration 8, 0.0739; iteration 9, 0.0395. Stop when one iteration improves the loss by less than $\varepsilon = 0.05$.

1. Improvement at iteration 4: $3.5896 - 1.5993 = 1.9903$. Much bigger than 0.05, so keep going.
2. Iteration 5: $1.5993 - 0.7165 = 0.8828$; iteration 6: $0.7165 - 0.3248 = 0.3917$; iteration 7: $0.3248 - 0.1510 = 0.1738$. All above 0.05, so keep going.
3. Iteration 8: $0.1510 - 0.0739 = 0.0771$. Still above 0.05, so keep going.
4. Iteration 9: $0.0739 - 0.0395 = 0.0344$. This is below 0.05, so training stops here.

Answer: Training stops after iteration 9, with a loss of about 0.04 dollars squared. A smaller tolerance, such as 0.01, would have let the run continue to iteration 11 for a barely visible gain.

COMMON MISTAKE

✗ Wrong: Treating a flat training loss curve as proof that the model is good.

✓ Right: A flat training curve only proves that the *search* has finished. Whether the model is any good is a separate question, answered on a validation set with the measures of Chapter 6. A badly overfitted model can have a beautifully flat training loss curve, as Chapter 7 showed.

9.6 Batch, Stochastic and Mini-Batch Gradient Descent

Every gradient so far used all four homes. With four rows that costs nothing. With four million rows, one iteration would have to touch four million rows before the parameters moved even once, and 500 iterations would read the whole dataset 500 times. There are three ways to organize the work, differing only in how many examples each gradient is built from.

WORDS TO KNOW

Batch gradient descent — The version used so far: every iteration computes the gradient from all n training examples. Each step is accurate but expensive.

Stochastic gradient descent (SGD) — Every iteration computes the gradient from one single randomly chosen example. Each step is cheap and noisy. The idea goes back to Herbert Robbins and Sutton Monro in 1951.

Mini-batch gradient descent — Every iteration computes the gradient from a small group of examples, typically 16 to 256 of them. It is the compromise almost all software uses.

Once the parameters move more than once per pass, we need a word for a full pass.

WORD TO KNOW

Epoch — One complete pass through every training example. Batch gradient descent performs exactly one iteration per epoch; stochastic gradient descent performs n of them; a mini-batch run performs one iteration per mini-batch.

EQUATION: ITERATIONS IN ONE EPOCH

$$\text{iterations per epoch} = \left\lceil \frac{n}{B} \right\rceil$$

In words: Divide the number of training examples by the size of one mini-batch, and round up, because a final short batch still counts as one iteration.

When it is used: To work out how many parameter updates a training run will actually make, and therefore how long it will take and how many times the loss can be recorded.

Symbol	Meaning	Unit or type
n	number of training examples	count
B	mini-batch size, from 1 up to n	count
$\lceil \rceil$	round up to the next whole number	—

$B = n$ gives batch gradient descent (1 iteration per epoch); $B = 1$ gives stochastic gradient descent (n iterations per epoch). Everything in between is a mini-batch.

The gradient formula barely changes: the sum runs over the mini-batch, and the division uses the size of that batch.

EQUATION: THE MINI-BATCH GRADIENT

$$\frac{\partial L}{\partial w} \approx -\frac{2}{B} \sum_{i \in \mathcal{B}} x_i r_i \quad \frac{\partial L}{\partial b} \approx -\frac{2}{B} \sum_{i \in \mathcal{B}} r_i$$

In words: Use exactly the same recipe as before, but add up only the examples in the current mini-batch, and divide by how many examples that batch holds.

When it is used: Whenever the training set is too large for every iteration to read all of it, which is most real projects.

Symbol	Meaning	Unit or type
$\mathcal{B}$	the set of examples in the current mini-batch	a group of rows
B	how many examples are in $\mathcal{B}$	count
r_i	residual of example i with the current parameters	same unit as y

The $\approx$ sign matters: a mini-batch gradient is an estimate of the true gradient. It points roughly downhill, not exactly downhill, and that is why the path looks jagged.

WORKED EXAMPLE 9.9 — Count the Iterations for Rania's Full Dataset

Rania's real training set has 2,000 homes. She plans to train for 30 epochs. How many iterations will she run (a) with mini-batches of 50, (b) with batch gradient descent, (c) with stochastic gradient descent?

1. Mini-batches of 50: iterations per epoch = $2000 \div 50 = 40$.
2. Over 30 epochs that is $40 \times 30 = 1200$ iterations.
3. Batch gradient descent uses the whole training set for one gradient, so it makes 1 iteration per epoch: $1 \times 30 = 30$ iterations.
4. Stochastic gradient descent uses one home per gradient, so it makes 2,000 iterations per epoch: $2000 \times 30 = 60,000$ iterations.

Answer: 1,200 iterations with mini-batches of 50, only 30 with batch gradient descent, and 60,000 with stochastic gradient descent — all three reading the data exactly 30 times. The difference is not how much data is read, but how often the parameters are allowed to move.

Figure 9.6 draws the paths of batch and stochastic gradient descent across the loss surface of 40 simulated homes; Figure 9.7 compares all three methods epoch by epoch.

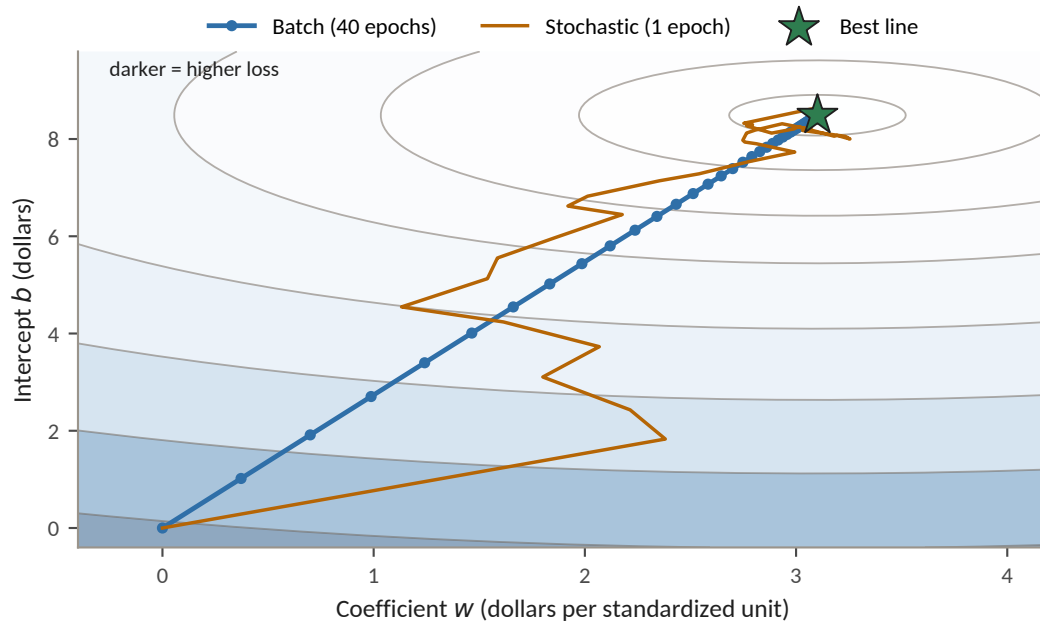


Figure 9.6 – Forty parameter updates by batch gradient descent (blue) and by stochastic gradient descent (orange) on the same 40 homes, with the same learning rate. The feature has been standardized first, as Lesson 9.7 explains.

WHAT TO OBSERVE

The blue path is smooth, because every step used all 40 homes. The orange path staggers, because every step used one single home and that home's own accident of a residual. Yet after the same 40 updates both are close to the star — and the orange path got there after reading the data once, while the blue path needed 40 full passes.

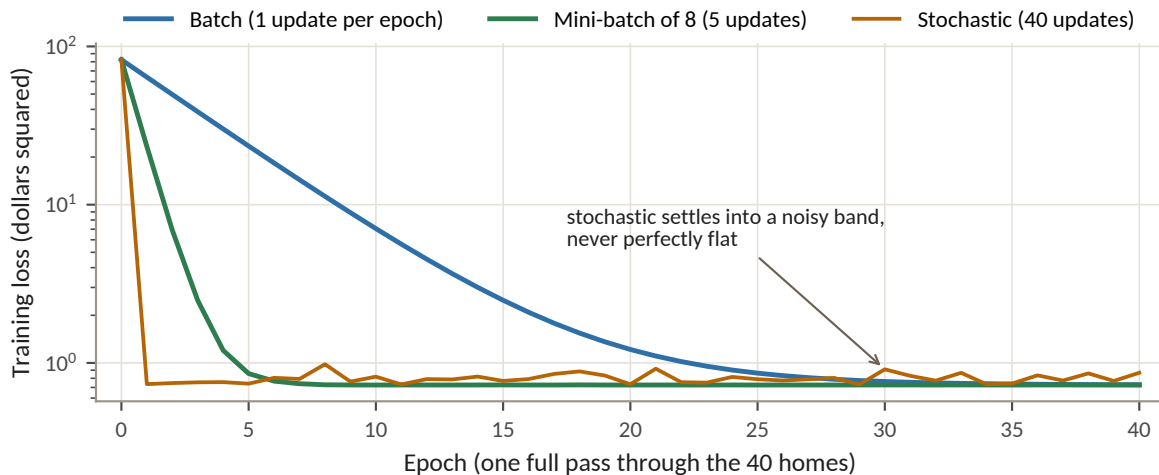


Figure 9.7 – Training loss after each epoch for the three methods on the same 40 homes, on a logarithmic scale.

WHAT TO OBSERVE

Measured per epoch — that is, per unit of reading work — stochastic gradient descent wins easily: it is near the bottom after one epoch, mini-batches take about six, and batch gradient descent about thirty. But the orange curve never becomes perfectly flat: each noisy step knocks the parameters slightly off the best point, so the run settles into a band rather than at a point.

Table 9.5 sums up the trade-off.

Table 9.5 – Choosing between the three forms of gradient descent

Method	Examples per step	Strengths	Weaknesses
Batch	all n	Smooth path; the loss falls every iteration	One step needs a full pass; no good if the data does not fit in memory
Stochastic	1	Very fast early progress	Noisy; never settles exactly; needs a shrinking rate
Mini-batch	16 to 256	Most of the speed with far less noise	One more hyperparameter: the batch size

Because stochastic and mini-batch runs never settle exactly, teams usually let the learning rate shrink as training goes on — large steps early, tiny steps at the end. In scikit-learn's `SGDRegressor` this is the `learning_rate` setting, whose default `'invscaling'` divides the rate by a growing power of the iteration number.

9.7 Feature Scaling and Convergence

Look again at Figure 9.3: the contour rings were long thin ovals, not circles, and the path had to creep along the valley floor. The cause is unequal feature scales. A coefficient's partial derivative is built by multiplying residuals by that feature's values, so a feature measured in the hundreds produces a gradient in the thousands while a feature measured in single digits produces one in the tens. A rate small enough to be safe for the steep direction is hopelessly slow for the shallow one, so the run crawls. The cure is standardization from Chapter 4: subtract the training mean and divide by the training standard deviation, so no feature can shout louder than another simply because of its unit.

WORKED EXAMPLE 9.10 — Standardize the Five House Sizes and Recompute the Gradient

Rania's five house sizes are 50, 70, 90, 110 and 130 square meters, and the bills are \$50, \$66, \$82, \$94 and \$110. Standardize the sizes, then find both partial derivatives again at $w = 0.5$, $b = 0$ and compare them with the raw result.

1. Mean size: $(50 + 70 + 90 + 110 + 130) \div 5 = 90$. Deviations: $-40, -20, 0, 20, 40$; squared they sum to 4000.
2. Standard deviation: $\sqrt{4000 \div 5} = \sqrt{800} = 28.28$.
3. Standardized sizes: $-40 \div 28.28 = -1.414$, then $-0.707, 0, 0.707, 1.414$.
4. Predictions at $w = 0.5$, $b = 0$ are $0.5 \times z$, so the residuals are 50.71, 66.35, 82.00, 93.65, 109.29, summing to 402.00.
5. Multiply each residual by its standardized size and add: $(-1.414 \times 50.71) + (-0.707 \times 66.35) + 0 + (0.707 \times 93.65) + (1.414 \times 109.29) = 102.15$.
6. Gradients: $\partial L / \partial w = -0.4 \times 102.15 = -40.86$ and $\partial L / \partial b = -0.4 \times 402.00 = -160.80$.

Answer: The two derivatives are now -40.86 and -160.80 , within a factor of about 4 of each other. Before standardizing they were -6756 and -70.8 , a factor of about 95 apart. The loss surface has changed from a knife-edge valley into something far closer to a round bowl.

Figure 9.8 shows the difference for the four-home dataset, whose feature is only 1 to 4 air conditioners — a modest range, yet enough to bend the bowl out of shape.

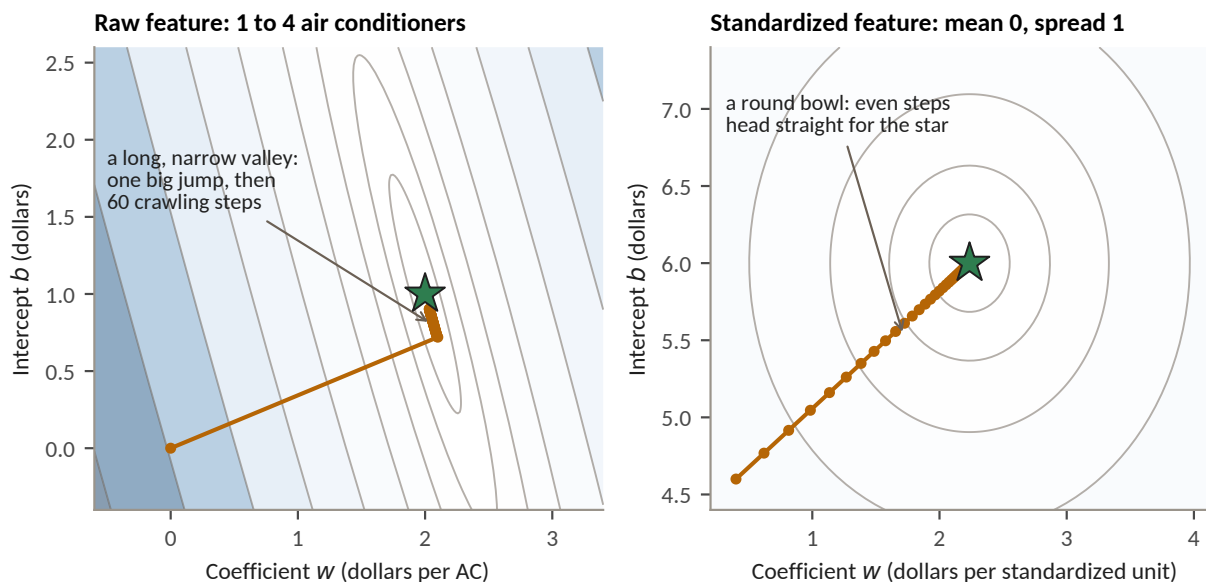


Figure 9.8 – The same four homes, before and after standardizing the feature. Both runs use the same learning rate, $\alpha = 0.06$, and 60 iterations.

WHAT TO OBSERVE

On the left the contours are squeezed into a narrow diagonal valley. The first step lands on the valley floor, and the remaining 59 steps crawl along it without reaching the star. On the right the contours are circles, and the run walks straight in and arrives after about 35 steps. There is a second, hidden gain: the raw version cannot use a learning rate above about 0.12 without diverging, while the standardized version is safe all the way up to 1.0.

COMMON MISTAKE

X Wrong: Computing the mean and standard deviation for standardization from the whole dataset, including the test rows, before splitting.

✓ Right: Fit the scaler on the training rows only, then apply it to validation and test rows, exactly as Chapter 4 showed. Using test rows to compute a mean is leakage, and it quietly flatters every score that follows.

Scaling is not cosmetic here; it is often the difference between a model that trains in seconds and one that never finishes. The same is true of the distance-based models in Chapter 11 and the support vector machines in Chapter 16.

9.8 When Gradient Descent Is Used

Linear regression is unusual: it has an exact formula for the best line, given in Chapter 8 and first published by Adrien-Marie Legendre in 1805. So why iterate at all?

WORD TO KNOW

Closed-form solution — A formula that computes the exact best parameters in one shot, with no repetition and no learning rate. Least squares for linear regression has one; most other models do not.

The closed form has to build a square matrix with one row and column per feature and then invert it: roughly nk^2 multiplications to build and k^3 to invert, where n is the number of rows and k the number of features. Gradient descent instead costs one pass over the data per iteration, roughly nk multiplications. For small problems the formula wins; for very large ones the iteration does.

Put numbers on that. A text model with $n = 50,000$ rows and $k = 20,000$ features needs about $nk^2 + k^3 = 2 \times 10^{13} + 8 \times 10^{12} \approx 2.8 \times 10^{13}$ multiplications for the closed form, but only $nk = 10^9$ per iteration, or 3×10^{11} for 300 iterations, with gradient descent: roughly 90 times less arithmetic. Shrink the problem to 800 rows and 6 features and the comparison reverses completely — about 29,000 multiplications for the closed form against 24 million for a 5,000-iteration run. Table 9.6 gives the practical rule.

Table 9.6 – Closed-form formula or gradient descent?

Situation	Better choice	Why
Few features, data fits in memory	Closed form	Exact answer, no learning rate, no stopping rule
Thousands of features, or millions of rows	Gradient descent	Avoids inverting a huge matrix; can work through the data in pieces
Data arrives continuously	Stochastic gradient descent	The model can be updated as each new example arrives
Logistic regression, support vector machines, most other losses	Gradient descent	No closed-form formula exists for these losses

The last row is the important one. Most useful losses have no formula for their minimum. Logistic regression (Chapter 10) minimizes log loss with these very same four steps, replacing the residual $y - \hat{y}$ with $y - p$, where p is a predicted probability; support vector machines (Chapter 16) minimize hinge loss the same way; ridge and lasso (Chapter 8) simply add the penalty's own derivative to the gradient before the step. Learning one method here has bought you the training procedure for a large part of this book.

COMMON MISTAKES AND MISCONCEPTIONS

- Adding the gradient to a parameter instead of subtracting it, so the loss climbs instead of falling.
- Believing a larger learning rate always means faster training, when above a data-dependent ceiling it destroys the run entirely.
- Updating the coefficient, recomputing the residuals, and only then updating the intercept, instead of computing both derivatives before either parameter moves.
- Reading a flat training loss curve as proof that the model is good, when it only proves the search has stopped.
- Expecting stochastic gradient descent to settle exactly at the best parameters, when its noise keeps it moving in a small band around them.

- Skipping feature scaling and then blaming gradient descent, or the model, for a run that crawls or diverges, or computing the scaling mean and standard deviation from the whole dataset instead of the training rows only.
- Assuming every model needs gradient descent, when a closed-form formula is faster and exact for a small linear regression.

Practical Activities

ACTIVITY (PAPER) — Two Rounds of Gradient Descent on Three Shops

Three small shops in Duhok each run a chest freezer or two. Shop 1 has 1 freezer and a daily electricity cost of \$3; shop 2 has 2 freezers and \$6; shop 3 has 3 freezers and \$9. Start from $w = 0$ and $b = 0$ with a learning rate of $\alpha = 0.02$, and use $n = 3$ in the gradient formulas.

1. Make a table with columns for x , y , $\hat{y}$ and $r = y - \hat{y}$. Fill in the first round's predictions and residuals, then compute $\sum r_i$ and $\sum x_i r_i$.
2. Compute both partial derivatives, $-\frac{2}{3} \sum x_i r_i$ and $-\frac{2}{3} \sum r_i$. Check that both are negative and explain in one sentence why they must be.
3. Take the step, rounding w and b to two decimals, and compute the new loss. Then repeat the whole table for a second round and take a second step.
4. Check your work: the loss should fall from 42 to about 25.5 after one round and about 15.4 after two, reaching $w = 1.00$ and $b = 0.43$.
5. Sketch the loss against the round number on graph paper. Where would the curve go if you kept going, and what value of w is it heading toward?

You need: Paper, pencil, a calculator and graph paper

ACTIVITY (PYTHON) — A Fifteen-Line Gradient Descent Loop, Checked Against scikit-learn

This is the whole of Figure 9.2 written out, run on the four homes of Table 9.1, then compared with scikit-learn's exact answer and its stochastic solver.

```

import numpy as np
from sklearn.linear_model import LinearRegression, SGDRegressor

x = np.array([1.0, 2, 3, 4])          # air conditioners in each home
y = np.array([3.0, 5, 7, 9])          # daily electricity cost, dollars
w, b, rate, n = 0.0, 0.0, 0.02, len(x)

for step in range(1, 201):
    residual = y - (w * x + b)          # step 2 of the algorithm
    grad_w = -(2 / n) * np.sum(x * residual)  # step 3: the two gradients
    grad_b = -(2 / n) * np.sum(residual)
    w = w - rate * grad_w                # step 4: one step downhill
    b = b - rate * grad_b
    if step in (1, 2, 3, 10, 50, 200):
        loss = np.mean((y - (w * x + b)) ** 2)
        print("step %3d   w = %.4f   b = %.4f   loss = %.4f" % (step, w, b, loss))

lin = LinearRegression().fit(x.reshape(-1, 1), y)
print("LinearRegression   w = %.4f   b = %.4f" % (lin.coef_[0], lin.intercept_))

sgd = SGDRegressor(learning_rate="constant", eta0=0.01, max_iter=20000,
                    tol=None, random_state=0).fit(x.reshape(-1, 1), y)
print("SGDRegressor      w = %.4f   b = %.4f" % (sgd.coef_[0], sgd.intercept_[0]))

```

Output

```

step   1   w = 0.7000   b = 0.2400   loss = 18.1926
step   2   w = 1.1660   b = 0.4004   loss = 8.0765
step   3   w = 1.4762   b = 0.5078   loss = 3.5896
step  10   w = 2.0558   b = 0.7179   loss = 0.0242
step  50   w = 2.0722   b = 0.7877   loss = 0.0075
step 200   w = 2.0293   b = 0.9138   loss = 0.0012
LinearRegression   w = 2.0000   b = 1.0000
SGDRegressor      w = 1.9999   b = 1.0004

```

Step 1 reproduces the hand calculation exactly. Steps 2 and 3 differ very slightly from the hand values (8.08 against 8.02, 3.59 against 3.54) only because the worked examples rounded w and b to two decimals each round while Python keeps every digit. After 200 iterations the loop has almost reached the exact answer that `LinearRegression` finds in one shot, and `SGDRegressor`, taking one home at a time, lands there too. Now experiment: set `rate` to 0.001 and then to 0.2 and rerun — the first crawls, the second explodes to `inf` within a few steps. Record every loss in a list and plot it to draw your own version of Figure 9.5.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Training the Bill Model on 2,000 Homes — Erbil

Rania's four-home example was a classroom toy. Her office's records cover about 4,000 homes, and after the split described in Chapter 8 her training set holds 2,000 of them, each with four features: floor area in square meters, number of major appliances, number of occupants, and building age in years. The target is

the monthly bill in dollars. The model had to be retrained every month from a script, so training had to be automatic, quick and reliable.

Her first attempt used gradient descent on the four raw features, with the learning rate her textbook suggested, 0.01. The training loss after the very first iteration was 800 million dollars squared, and it grew about eighty thousand times larger with every further iteration until the computer printed `inf`. She halved the rate — eight times in all — before the loss finally began to fall. The largest rate that did not diverge was around 0.00007, so small that the run had to be measured in tens of thousands of iterations. The cause was visible the moment she looked at the columns: floor area ran from 40 to 215, appliance count from 1 to 6, building age from 0 to 45. The area column's gradient was thousands of times larger than the appliance column's, purely because of the unit, and one learning rate had to serve all four. Even after 200,000 iterations the training mean squared error had only reached 34.5 dollars squared, short of the 34.0 that the closed-form formula proves is the best possible here.

Rania then put a `StandardScaler` in front of the model, fitted on the training rows only, exactly as Chapter 4 requires. With every feature centered on 0 and scaled to a spread of 1, a learning rate of 0.1 was perfectly safe — more than a thousand times larger than before. The run came within 1% of the best possible training error after 25 iterations and was indistinguishable from the exact answer after 100. Retraining went from several minutes to well under a second.

Two lessons stayed with the team. An untrainable model is not always a bad model; sometimes it is a badly scaled one, and the fix is a preprocessing step rather than a different algorithm. And the closed-form formula was worth keeping as a check, because it let Rania confirm that her iterative run had converged to the right place. When she later moved the same pipeline to logistic regression, that safety net disappeared and the loss curve became her only evidence.

CHAPTER SUMMARY

- The loss of a model can be pictured as a landscape whose height at each pair (w, b) is the loss of that pair; training means finding the lowest point.
- The least-squares loss of a linear model is convex — one clean bowl — so any downhill search reaches the global minimum wherever it starts.
- A partial derivative is the slope of the loss in one parameter's direction; the gradient collects all of them and points uphill. For the mean squared error, $\partial L / \partial w = -\frac{2}{n} \sum x_i r_i$ and $\partial L / \partial b = -\frac{2}{n} \sum r_i$.
- One iteration subtracts the learning rate times each derivative from its parameter: $w \leftarrow w - \alpha \partial L / \partial w$. Steps shrink automatically as the ground flattens, because the gradient itself shrinks near the bottom.
- A learning rate that is too small wastes time; one above a data-dependent ceiling makes the loss grow every round, which is divergence, not slow learning.
- A healthy loss curve falls steeply, bends and flattens; training stops when one iteration improves the loss by less than a chosen tolerance ϵ .
- Batch gradient descent uses every example per step, stochastic uses one, and mini-batch uses a small group; one epoch is one full pass through the data, and $\lceil n/B \rceil$ iterations.
- Standardizing the features turns a narrow valley into a round bowl, allowing a much larger learning rate and far fewer iterations.
- Linear regression also has an exact closed-form solution, which is better for small problems; gradient descent wins on very large ones and is the only option for most other losses.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion *Teacher's Guide*.

Problem 1 [Basic] In your own words, explain what the loss surface of a model is, and what its lowest point represents.

Problem 2 [Basic] A partial derivative of the loss with respect to w is -18 . Should w be increased or decreased to reduce the loss? Explain your reasoning in one sentence.

Problem 3 [Basic] Define learning rate, iteration and epoch in one sentence each.

Problem 4 [Basic] A training run's loss values for the first four iterations are 60, 140, 380 and 1,020. What has gone wrong, and what single setting should be changed?

Problem 5 [Basic] A training set has 4,000 rows and the mini-batch size is 200. How many iterations are there in one epoch, and how many in 25 epochs?

Problem 6 [Standard] A tiny dataset has $x = [2, 4]$ and $y = [6, 10]$. Starting from $w = 0$ and $b = 0$, compute both partial derivatives, showing every step.

Problem 7 [Standard] Continue Problem 6. With a learning rate of $\alpha = 0.05$, take one step and report the new w , the new b and the new loss.

Problem 8 [Standard] Using the four homes of Table 9.1, compute the loss at $w = 3$, $b = 0$ and at $w = 1.5$, $b = 1$. Which of the two lines is better, and by how much?

Problem 9 [Standard] A run records losses of 12.40, 8.10, 6.05, 5.20, 4.95 and 4.88 over six iterations. Using a tolerance of $\varepsilon = 0.10$, state the iteration at which training stops, showing the improvement at each step.

Problem 10 [Standard] Explain in your own words why a mini-batch gradient is only an estimate of the true gradient, what visible effect that has on the path across the loss surface, and why the run never settles exactly.

Problem 11 [Standard] One feature is measured in grams and runs from 200 to 900; another is a count from 1 to 5. Explain what this does to the two partial derivatives, what a single learning rate must then compromise between, and which preprocessing step fixes it.

Problem 12 [Challenge] Repeat Worked Example 9.4 with $\alpha = 0.05$ instead of 0.02. (a) Find w , b and the loss after one step and after two steps. (b) Does this rate converge faster than 0.02? (c) The ceiling for this dataset is about 0.12 — explain what would happen at $\alpha = 0.15$.

Problem 13 [Challenge] A team trains a model twice on the same data: once with batch gradient descent for 200 epochs, and once with stochastic gradient descent for 200 epochs. The stochastic run reaches a low loss far sooner but its final loss is slightly worse and changes a little every time it is run. (a) Explain both observations. (b) Suggest two changes that would let the stochastic run finish as accurately as the batch run.

Problem 14 [Challenge] A dataset has 800 rows and 6 features. A colleague insists on using gradient descent with 5,000 iterations rather than the closed-form formula. (a) Estimate the number of multiplications each approach needs. (b) Give one reason your colleague could still be right, and one reason the closed form is the better default here.

Problem 15 [Challenge] Design a short experiment, in words, that would find a good learning rate for a new dataset without wasting hours of computer time. State what you would try first, what you would measure, how you would decide to raise or lower the rate, and when you would stop searching.

PART THREE

SUPERVISED LEARNING: PREDICTING CATEGORIES

CHAPTER TEN

Logistic Regression

From a line to a probability, and from a probability to a yes/no answer

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a straight line fitted to 0/1 labels cannot be read as a probability.
- Apply the sigmoid function to a score by hand and read the result as a probability.
- Calculate log loss for one prediction and for a set of predictions.
- Find the decision boundary of a logistic regression model with one feature and with two.
- Take one round of gradient descent for logistic regression by hand.
- Translate a coefficient into odds and an odds multiplier, and say what it does not prove.
- Extend logistic regression to three or more classes, and turn the regularization dial C .

Introduction

Look at tomorrow's weather forecast. It almost never says "it will rain". It says something more careful: "a 70 percent chance of rain". That one number does two jobs at once. It answers a yes-or-no question — will it rain? — and it also says how sure the forecaster is. If you must decide whether to carry an umbrella, 70 percent and 51 percent feel very different, even though both lean the same way.

An enormous number of real questions have this shape. Will this loan be repaid? Is this message spam? Will this student pass the final exam? Chapter 8 taught you to fit a line that predicts a number, and Chapter 9 taught you how a computer searches for good coefficients. This chapter points that machinery at yes-or-no questions, using a method with a slightly confusing name: *logistic regression*. Despite the word "regression", it is a classification method. Build an ordinary score, squeeze it into the range 0 to 1, grade it honestly, and draw a line between yes and no.

10.1 Why a Straight Line Cannot Predict Yes or No

Soran teaches a first-year statistics course at a university in Sulaymaniyah. Every term a few students fail the final exam, and usually discover the trouble far too late to fix it. He would like to know, in the first month, which students may be heading for difficulty, so that he can offer help while help still matters.

Table 10.1 holds eight students from last year's section: hours studied per week, percentage of classes attended in the first four weeks, and the result. Soran writes Fail as 0 and Pass as 1 — a labeling convention, as in Chapter 1, not a claim that failing is worth "zero" of anything.

Table 10.1 – Soran’s eight past students: two early features and the final result

Student	Hours studied per week	Attendance (%)	Result	Label y
Ari	1	40	Fail	0
Bahar	2	55	Fail	0
Chnar	3	45	Fail	0
Dilan	4	95	Pass	1
Evin	6	35	Fail	0
Farhad	7	85	Pass	1
Gona	8	70	Pass	1
Hemin	9	90	Pass	1

The natural first idea is to reuse Chapter 8: fit a straight line to those 0s and 1s. The least-squares formulas do not object, returning $\hat{y} = -0.17 + 0.13 \times \text{hours studied}$. The trouble appears the moment we ask that line for a prediction, as Figure 10.1 shows.

WORKED EXAMPLE 10.1 — See What Goes Wrong When a Straight Line Predicts Pass or Fail

The line fitted to Soran’s 0/1 labels is $\hat{y} = -0.17 + 0.13 \times \text{hours}$. Find the predicted value for Ari, who studied 1 hour per week, and for an imaginary student who studies 12 hours.

1. Put Ari’s 1 hour into the line: $-0.17 + 0.13 \times 1 = -0.04$.
2. Put 12 hours into the line: $0.13 \times 12 = 1.56$, so $-0.17 + 1.56 = 1.39$.
3. Compare both with the range a probability may occupy, 0 to 1: -0.04 sits below 0 and 1.39 above 1, so neither can be a probability.

Answer: The line predicts -0.04 for Ari and 1.39 for the 12-hour student. A “probability” of -0.04 is meaningless, and Ari is a real student in the training data, not a far extrapolation. A straight line has no top and no bottom, so it can always be pushed outside the legal range.

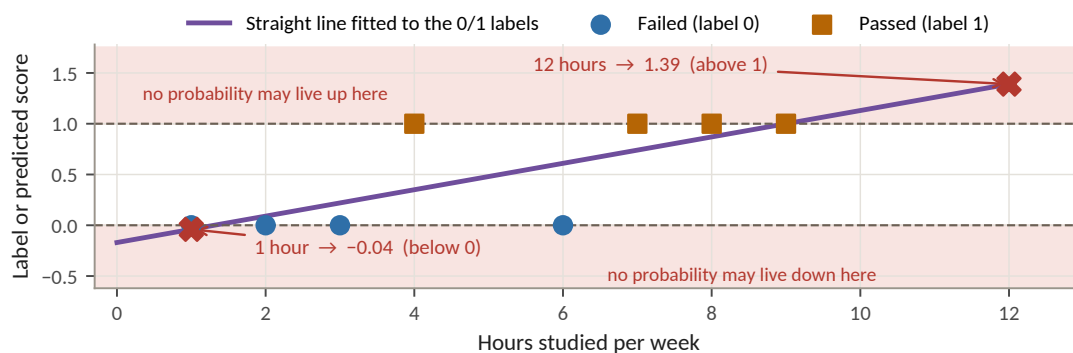


Figure 10.1 – A straight line fitted directly to the 0/1 pass-fail labels. The pink bands mark values a probability may never take.

WHAT TO OBSERVE

The line fits the middle of the data well enough but rises forever in both directions, leaving the legal range at about 1.3 hours on the left and 9 hours on the right — inside the range of Soran’s own students.

COMMON MISTAKE

✗ Wrong: Reading a plain linear regression score, such as 1.39 or -0.04 , as if it were already a probability.
✓ Right: A linear score has no upper and no lower limit, so it can never be trusted as a probability on its own. Logistic regression fixes this by passing the score through one extra step first.

WORD TO KNOW

Logistic regression — A method that predicts the probability of a yes-or-no answer. It builds the same score as linear regression — a coefficient for each feature plus an intercept — then squeezes it into the range 0 to 1, so the result can honestly be called a probability. Despite its name it is used for classification. It was set out in its modern form by the British statistician David Cox in 1958.

10.2 The Sigmoid Function

Soran needs a squeezing function: something that takes any number — huge, tiny, negative — and always hands back a value between 0 and 1. Such a function exists, with one shape worth memorizing: a flat left tail, a steep middle, a flat right tail. Because that looks like a stretched letter S, it is called the sigmoid.

WORD TO KNOW

Sigmoid function — A function that squeezes any real number into a value strictly between 0 and 1, tracing a smooth S-shaped curve. The curve is far older than machine learning: the Belgian mathematician Pierre Franois Verhulst introduced it in 1838 to describe a population that grows quickly and then levels off, and called it the *logistic* curve.

Before the squeezing there must be something to squeeze: the ordinary linear score from Chapter 8, which here gets its own letter, z . The squeezing step uses e , the fixed number about 2.71828 that appears in every growth formula; you can use it without deriving it, exactly as you use π for the area of a circle.

EQUATION: THE LINEAR SCORE AND THE SIGMOID

$$z = b + w_1x_1 + \cdots + w_kx_k, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

In words: First multiply each feature by its coefficient, add the products, and add the intercept; that is the score. Then raise e to the power of minus the score, add 1, and divide 1 by the result; that is the probability.

When it is used: Together these two steps are one logistic regression prediction; the first alone is what linear regression does.

Symbol	Meaning	Unit or type
z	the linear score for one example	no unit
b, w_j	the intercept and the coefficient of feature j	no unit; per unit of feature j
x_j, k	the value of feature j , and the number of features	unit of that feature; count
$p, \sigma(z)$	the predicted probability that the label is 1	between 0 and 1
e	a fixed number, about 2.71828	no unit

A large positive score gives a probability near 1, a very negative score one near 0, and $z = 0$ gives exactly 0.5. The output never actually reaches 0 or 1 — the guarantee a straight line could not give.

Suppose Soran's one-feature model, fitted on hours alone, is $z = -5 + 1 \times \text{hours}$; Lesson 10.5 shows where such numbers come from.

WORKED EXAMPLE 10.2 — Turn Three Study Times into Three Probabilities

Under $z = -5 + 1 \times \text{hours}$, find the predicted probability of passing for a student who studies 5 hours, for Farhad who studied 7, and for Chnar who studied 3. Use $e^{-2} \approx 0.1353$ and $e^2 \approx 7.389$.

1. The 5-hour student's score is $z = -5 + 5 = 0$, so $\sigma(0) = 1 \div (1 + e^{-0})$; any number to the power 0 is 1, so this is $1 \div 2 = 0.5$.
2. Farhad: $z = -5 + 7 = 2$, so $\sigma(2) = 1 \div (1 + 0.1353) = 1 \div 1.1353 = 0.881$.
3. Chnar: $z = -5 + 3 = -2$, so $\sigma(-2) = 1 \div (1 + 7.389) = 1 \div 8.389 = 0.119$.

Answer: The probabilities of passing are 0.500, 0.881 and 0.119. A score of exactly 0 always lands exactly halfway. Farhad's and Chnar's probabilities sit equally far from 0.5 in opposite directions, because their scores sit equally far from 0.

Figure 10.2 draws the curve, and Table 10.2 turns it into a lookup table for Soran's model.

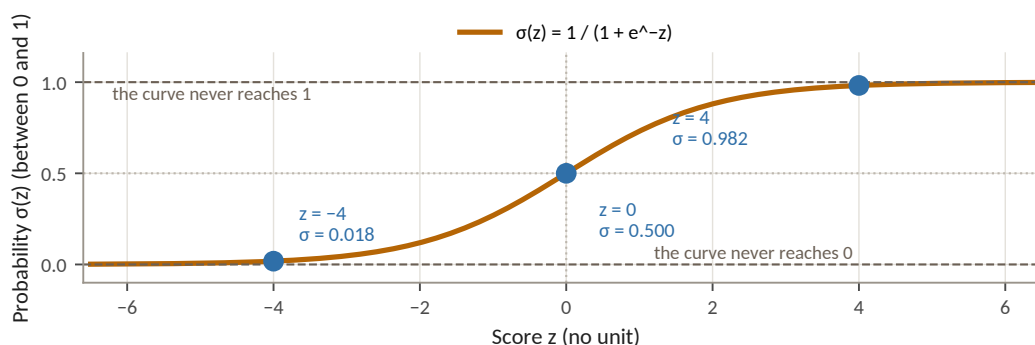


Figure 10.2 – The sigmoid curve. Every possible score on the horizontal axis becomes a probability between 0 and 1 on the vertical axis.

WHAT TO OBSERVE

The curve is steepest near $z = 0$, where a small change in the score moves the probability a lot, and almost flat beyond $z = \pm 4$. It is symmetric too: $\sigma(-4) = 0.018$ and $\sigma(4) = 0.982$ add to exactly 1.

Table 10.2 – Soran's one-feature model $z = -5 + \text{hours}$, worked out for seven study times

Hours studied per week	Score $z = -5 + \text{hours}$	Probability $\sigma(z)$	Prediction at cutoff 0.5
1	-4	0.018	Fail
2	-3	0.047	Fail
3	-2	0.119	Fail
4	-1	0.269	Fail
5	0	0.500	Pass
7	+2	0.881	Pass
9	+4	0.982	Pass

Laid over Soran's real students in Figure 10.3, the same curve shows what the model gets right and what it misses.

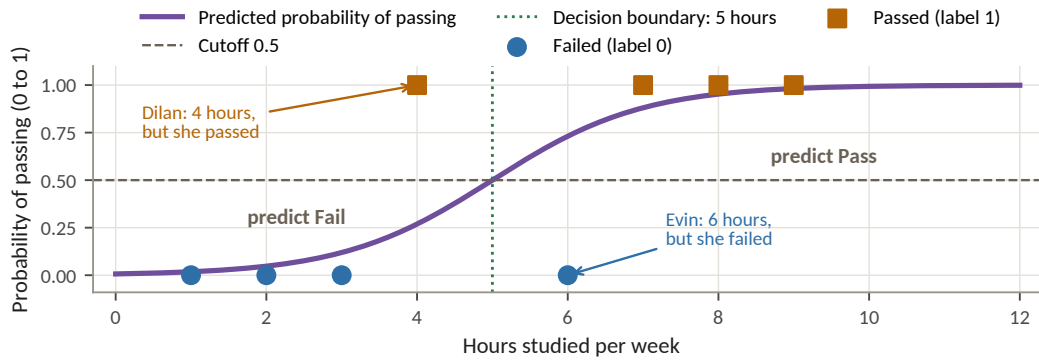


Figure 10.3 – Soran’s one-feature model drawn over his eight students. Each student is plotted at her true label, 0 or 1.

WHAT TO OBSERVE

Six of the eight students sit where the curve expects. Two do not: Dilan passed after only 4 hours, Evin failed after 6. Hours alone cannot explain them, which is why Lesson 10.4 adds a second feature.

10.3 Log Loss: Scoring a Probability Honestly

Chapter 8 graded a regression line with mean squared error. A probability needs something different, because it carries information a plain number does not: how sure the model claims to be. A model that says “95 percent sure” and is wrong deserves a far heavier penalty than one that says “52 percent sure”. The loss built for that job is log loss.

WORD TO KNOW

Log loss — A loss function for probabilities, also called cross-entropy loss. It compares the true 0/1 label with the predicted probability and punishes a confident, wrong prediction very heavily. A perfect, fully confident, correct prediction scores 0.

The formula looks like it has two halves, but only one is alive at a time: if the label is 1 the $(1 - y)$ half is multiplied by zero, and if it is 0 the y half disappears. The $\ln$ is the natural logarithm, the calculator button that undoes “raise e to a power”. To grade a whole model, average the individual losses.

EQUATION: LOG LOSS, FOR ONE EXAMPLE AND FOR A WHOLE DATASET

$$L = -[y \ln(p) + (1 - y) \ln(1 - p)], \quad \bar{L} = \frac{1}{n} \sum_{i=1}^n L_i$$

In words: If the true label is 1, the loss is minus the natural logarithm of the predicted probability; if it is 0, the loss is minus the natural logarithm of one minus the predicted probability. A dataset’s loss is the average of those.

When it is used: To grade a probability against the answer that happened, and — averaged over the training set — as the quantity training minimizes.

Symbol	Meaning	Unit or type
L, L_i	the log loss of one example, and of example i	no unit
$\bar{L}, n$	the average log loss, and the number of examples	no unit; count
y, p	the true label and the predicted probability that it is 1	0 or 1; between 0 and 1

The minus sign turns every loss into a positive penalty, since the logarithm of a number below 1 is negative. Lower is better, 0 is perfect, and a model that answers 0.5 to everything scores exactly 0.693 — the bar any useful model must beat.

WORKED EXAMPLE 10.3 — Score One Probability Two Ways, Then a Whole Set

(a) Soran's model gave Farhad a predicted probability of passing of $p = 0.881$. Find the log loss if Farhad passed ($y = 1$), then if the very same prediction had been followed by a Fail ($y = 0$). (b) Find the average log loss over four students, using Table 10.2: Ari (1 hour, failed), Chnar (3 hours, failed), Dilan (4 hours, passed) and Farhad (7 hours, passed). Use $\ln(0.982) \approx -0.018$, $\ln(0.881) \approx -0.127$, $\ln(0.269) \approx -1.313$ and $\ln(0.119) \approx -2.129$.

1. (a) With $y = 1$ only the first half survives: $L = -\ln(0.881) = -(-0.127) = 0.127$.
2. With $y = 0$ only the second half survives: $1 - 0.881 = 0.119$, so $L = -(-2.129) = 2.129$, about seventeen times more.
3. (b) Ari: $p = 0.018$, $y = 0$, so $L = -\ln(1 - 0.018) = -\ln(0.982) = 0.018$.
4. Chnar: $p = 0.119$, $y = 0$, so $L = -\ln(0.881) = 0.127$.
5. Dilan: $p = 0.269$, $y = 1$, so $L = -\ln(0.269) = 1.313$; Farhad: $p = 0.881$, $y = 1$, so $L = 0.127$.
6. Add and divide: $0.018 + 0.127 + 1.313 + 0.127 = 1.585$, and $1.585 \div 4 = 0.396$.

Answer: (a) The same prediction costs 0.127 when it turns out right and 2.129 when it turns out wrong; only the outcome changed. (b) The average log loss is 0.396, comfortably better than the 0.693 a shrugging model scores — though Dilan alone contributes 1.313 of the 1.585 total, because she passed after only 4 hours.

Table 10.3 shows the same idea from the other side, holding the answer at Pass and changing only the model's confidence; Figure 10.4 draws the curve behind it.

Table 10.3 – Log loss for four predictions, all about students who really did pass ($y = 1$)

Predicted probability p	Log loss $-\ln(p)$	What the model was doing
0.95	0.051	Confident and right — almost no penalty
0.60	0.511	Leaning the right way, but unsure
0.50	0.693	No opinion at all; the price of a shrug
0.05	2.996	Confident and wrong — a very heavy penalty

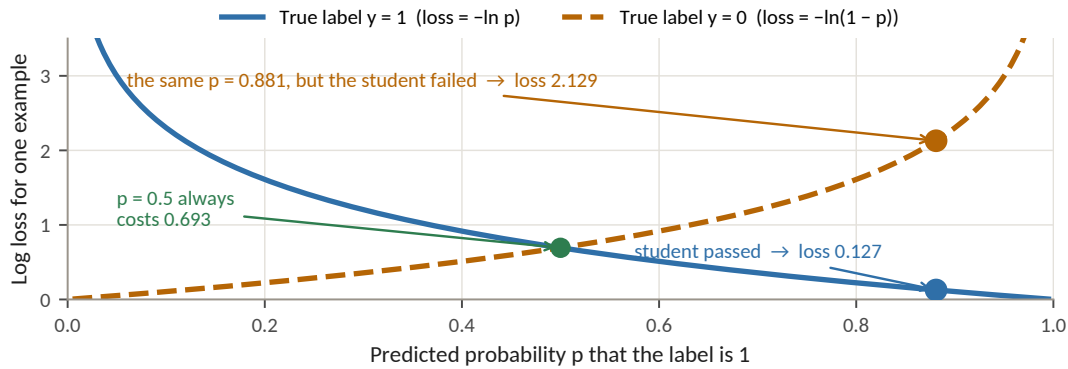


Figure 10.4 – Log loss as the predicted probability changes. The solid curve applies when the true label is 1, the dashed curve when it is 0.

WHAT TO OBSERVE

Both curves touch zero where the model was completely right and shoot upward without limit where it was completely wrong. They cross at $p = 0.5$, where the loss is 0.693 whatever happens.

COMMON MISTAKE

✗ Wrong: Training a logistic regression model with mean squared error, the loss Chapter 8 used for a straight line.

✓ Right: Logistic regression trains with log loss, which reacts correctly to probabilities and punishes confident, wrong predictions far more harshly than mean squared error ever would.

10.4 The Decision Boundary

Sooner or later Soran must act: contact this student, or not. That means turning the probability into a decision with a cutoff — the simplest rule predicts Pass when the probability is 0.5 or more. Chapter 6 calls this cutoff the *threshold*; for now 0.5 is a sensible default. The place where the model switches answers has its own name.

WORD TO KNOW

Decision boundary — The line, or surface, separating the region where a classifier predicts one category from the region where it predicts the other. With the usual 0.5 cutoff it sits where the predicted probability is 0.5 — which, from Lesson 10.2, is exactly where the score z is 0.

That sentence is the whole recipe: never chase the probability, set the score to zero and solve. With one feature the answer is a point on the number line; with two it is a straight line; with many it is a flat hyperplane, the word Chapter 8 used for a regression surface in many dimensions. One feature was never going to be enough for Soran, since Dilan and Evin broke its rule, so he adds attendance: $z = -6 + 0.8 \times \text{hours} + 0.04 \times \text{attendance}$, with attendance as a percentage.

EQUATION: THE DECISION BOUNDARY

$$b + w_1x_1 + \cdots + w_kx_k = 0 \quad \text{and, with a single feature,} \quad x = -\frac{b}{w}$$

In words: The boundary is the set of points whose linear score is exactly zero. With one feature there is only one such point: minus the intercept divided by the coefficient.

When it is used: To find the feature value at which a model changes its mind, and to draw the dividing line on a two-feature plot.

Symbol	Meaning	Unit or type
x_j, w_j	feature j at the boundary, and its coefficient	unit of that feature; per unit of it
b	the intercept	no unit

This assumes the usual 0.5 cutoff. For any other cutoff t , solve $z = \ln(t/(1 - t))$ instead; Lesson 10.6 explains why.

WORKED EXAMPLE 10.4 — Find the Decision Boundary, With One Feature and With Two

(a) Find the number of study hours at which $z = -5 + 1 \times \text{hours}$ gives a probability of exactly 0.5. (b) For $z = -6 + 0.8 \times \text{hours} + 0.04 \times \text{attendance}$, find how many hours a student who attends 60% of classes must study to reach 0.5.

1. A probability of exactly 0.5 happens only where the score is 0, so set $z = 0$ in both parts.
2. (a) $-5 + 1 \times \text{hours} = 0$, so $\text{hours} = 5$. Check with the shortcut: $-b \div w = 5 \div 1 = 5$.
3. (b) $-6 + 0.8 \times \text{hours} + 0.04 \times 60 = 0$, and $0.04 \times 60 = 2.4$.
4. Move the constants across and divide: $0.8 \times \text{hours} = 3.6$, so $\text{hours} = 3.6 \div 0.8 = 4.5$.

Answer: (a) The one-feature boundary is a single point at 5 hours per week — exactly where Table 10.2 flips from Fail to Pass. (b) With two features the boundary is a line: a student at 60% attendance needs 4.5 hours, one with better attendance needs fewer, one with worse attendance needs more.

Figure 10.5 draws that line across the two features, with the probability shaded behind it.

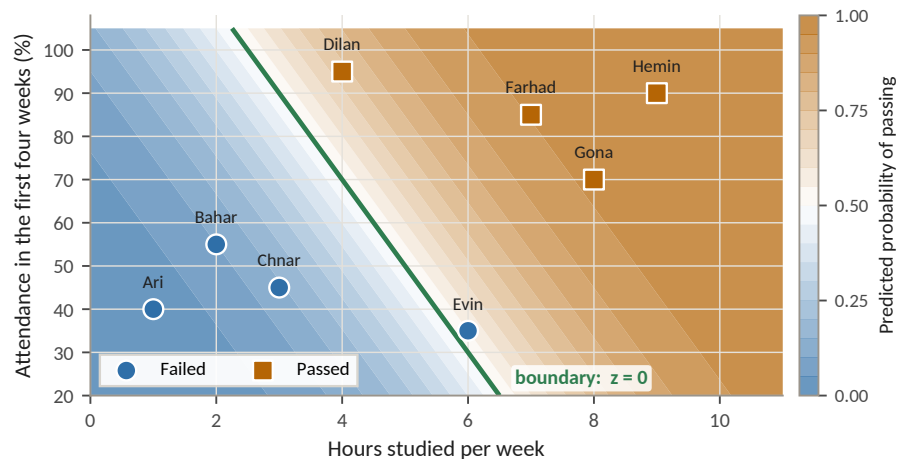


Figure 10.5 – The two-feature model. The green line is the decision boundary, where $z = 0$ and the probability is exactly 0.5.

WHAT TO OBSERVE

The boundary is perfectly straight — logistic regression can never bend it. Dilan, who studied only 4 hours, is now safely on the Pass side because she attended 95% of classes. Evin, at 6 hours but 35% attendance, sits just barely on the Pass side and still failed: one mistake out of eight, an accuracy of 87.5% against 75% for the hours-only model.

COMMON MISTAKE

X Wrong: Assuming that 0.5 is always the right cutoff for turning a probability into a decision.

✓ Right: 0.5 is only a default. Chapter 6 shows how to move the cutoff on purpose — for example lowering the bar for an alarm when missing a true case costs far more than a false alarm, as in disease screening or fraud detection.

10.5 Fitting Logistic Regression with Gradient Descent

Every coefficient so far was handed to you. Training finds those numbers itself, using the idea from Chapter 9: guess, work out which direction reduces the loss, take a small step that way, repeat. Only the loss has changed — log loss instead of mean squared error — and the update rule barely changes with it. Where Chapter 9 compared the true value with the predicted value, here we compare the true label y with the predicted probability p , and $y - p$ plays the role a residual played before.

WORD TO KNOW

Maximum likelihood — Choosing the coefficients that make the outcomes actually observed in the training data as probable as possible. Minimizing average log loss and maximizing likelihood are the same thing written two ways; the principle was set out by the statistician Ronald A. Fisher in 1922.

EQUATION: ONE GRADIENT DESCENT STEP FOR LOGISTIC REGRESSION

$$w_{\text{new}} = w + \alpha \sum_{i=1}^n x_i (y_i - p_i), \quad b_{\text{new}} = b + \alpha \sum_{i=1}^n (y_i - p_i)$$

In words: For every example, subtract the predicted probability from the true label. To update a coefficient, multiply each difference by that example's feature value, add them up, multiply by the learning rate, and add the result. Update the intercept the same way with the plain differences.

When it is used: Every round of training, until the steps stop changing the coefficients.

Symbol	Meaning	Unit or type
w, b	the current coefficient and intercept	per unit of x ; no unit
α	the learning rate, chosen before training	no unit
$x_i, y_i - p_i$	example i 's feature value, and its label minus its predicted probability	unit of that feature; between -1 and 1

Some libraries divide the sum by n . That rescales every step by the same factor, which can be absorbed into the learning rate; the direction of the step is identical.

WORKED EXAMPLE 10.5 — Take One Round of Gradient Descent

A tiny practice set has two students: one studied 2 hours and failed ($y = 0$), the other studied 8 hours and passed ($y = 1$). Starting from $w = 0$ and $b = 0$ with a learning rate of $\alpha = 0.1$, find the coefficient and intercept after one round.

1. Score both students with the current guess — $0 \times 2 + 0 = 0$ and $0 \times 8 + 0 = 0$ — and squeeze: $\sigma(0) = 0.5$ for both.
2. Find $y - p$: first student $0 - 0.5 = -0.5$; second student $1 - 0.5 = 0.5$.
3. For the intercept, add those differences: $-0.5 + 0.5 = 0$.
4. For the coefficient, multiply each difference by that student's hours and add: $(2 \times -0.5) + (8 \times 0.5) = -1 + 4 = 3$.

5. Update: $w_{\text{new}} = 0 + 0.1 \times 3 = 0.3$ and $b_{\text{new}} = 0 + 0.1 \times 0 = 0$.

Answer: After one round $w = 0.3$ and $b = 0$. The 8-hour student passed when the model expected only a coin flip, and her larger feature value pulled the coefficient upward. The two pulls on the intercept were equal and opposite, so it did not move.

Every later round repeats the same four actions: score, squeeze, compare with the truth, step. Figure 10.6 applies that rule to all eight students four thousand times, with a learning rate of 0.01. Too large a learning rate makes such a curve jump around or climb instead of falling, the overshooting problem Chapter 9 warned about; too small a rate takes many more rounds to arrive.

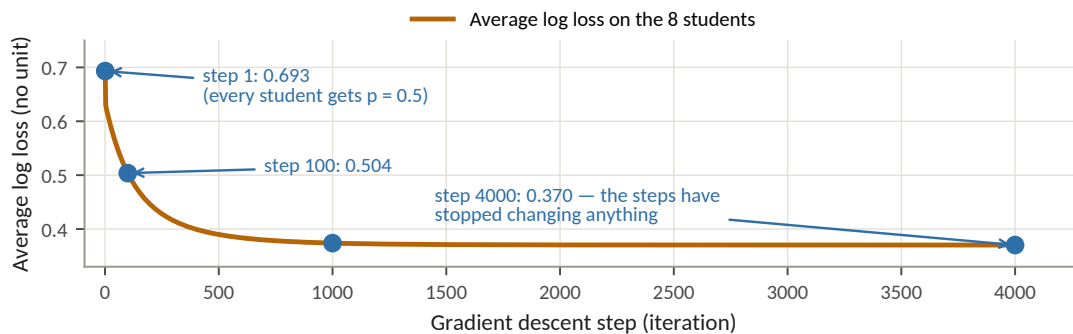


Figure 10.6 – The average log loss on Soran’s eight students during 4,000 rounds of gradient descent, starting from $w = 0$ and $b = 0$.

WHAT TO OBSERVE

The curve starts at exactly 0.693, because with $w = 0$ and $b = 0$ every student gets 0.5. By round 4,000 it has settled at 0.370 with $w = 0.824$ and $b = -4.119$, putting the boundary at $-b \div w = 5.00$ hours — exactly where Lesson 10.4 put it.

10.6 Reading Coefficients: Odds and Log-Odds

What do the fitted coefficients mean? In linear regression a coefficient of 0.8 meant “add 0.8 to the prediction for each extra unit”. Here it cannot, because the sigmoid bends everything: in Table 10.2, going from 1 hour to 2 moves the probability by 0.029, while 4 hours to 5 moves it by 0.231 — the same step, eight times the effect. To say what a coefficient really is, we need one old idea from betting.

WORDS TO KNOW

Odds — The probability that something happens divided by the probability that it does not. A probability of 0.8 means odds of $0.8 \div 0.2 = 4$, read as “4 to 1 on”; odds run from 0 upward with no upper limit.

Log-odds (logit) — The natural logarithm of the odds, running from very negative to very positive, with 0 meaning a probability of exactly 0.5. The statistician Joseph Berkson named it the *logit* in 1944.

Here is the elegant fact at the center of the method: the score z is the log-odds, and the sigmoid is exactly the calculation that turns log-odds back into a probability. The model really is a straight line — one drawn in log-odds rather than in probability.

EQUATION: ODDS, LOG-ODDS, AND WHAT A COEFFICIENT DOES

$$\text{odds} = \frac{p}{1-p}, \quad \ln\left(\frac{p}{1-p}\right) = z, \quad \text{new odds} = \text{old odds} \times e^w$$

In words: The odds are the probability of yes divided by the probability of no. The logarithm of the odds is exactly the linear score. Adding one unit to a feature adds its coefficient to the log-odds, which multiplies the odds by e raised to that coefficient.

When it is used: To explain in plain words what a fitted coefficient means, and to compare two features in a fitted model.

Symbol	Meaning	Unit or type
p, z	the predicted probability, and the linear score (the log-odds)	between 0 and 1; no unit
w, e^w	a coefficient, and its odds multiplier (the odds ratio)	per unit of that feature; no unit

An odds multiplier above 1 means the feature pushes toward “yes”; below 1 means it pushes toward “no”; exactly 1 means the feature does nothing at all.

WORKED EXAMPLE 10.6 — From a Probability to Odds, and From a Coefficient to an Odds Multiplier

A student's predicted probability of passing is $p = 0.8$ under the two-feature model, whose coefficient on hours studied is 0.8. (a) Find her odds and log-odds of passing. (b) Find her new probability if she studies one more hour per week and nothing else changes. Use $\ln(4) \approx 1.386$ and $e^{0.8} \approx 2.226$.

1. Probability of not passing: $1 - 0.8 = 0.2$, so the odds are $0.8 \div 0.2 = 4$.
2. Take the logarithm: the log-odds are $\ln(4) = 1.386$. Check: $\sigma(1.386) = 1 \div (1 + 0.25) = 0.8$.
3. Turn the coefficient into an odds multiplier, $e^{0.8} = 2.226$, and multiply: $4 \times 2.226 = 8.904$.
4. Turn the new odds back into a probability: $8.904 \div 9.904 = 0.899$.

Answer: Her odds are 4 and her log-odds 1.386; one extra hour raises her probability from 0.800 to about 0.899. That extra hour multiplies every student's odds by the same 2.23, but moves each student's probability by a different amount, because the sigmoid is steep in the middle and flat at the ends.

Table 10.4 turns the whole model into plain sentences.

Table 10.4 – Reading the two-feature model $z = -6 + 0.8 \text{ hours} + 0.04 \text{ attendance}$

Change in one feature	Added to the log-odds	Odds multiplier e^w	Plain meaning
One more hour of study per week	+0.80	2.23	The odds of passing become about 2.2 times larger
One more percentage point of attendance	+0.04	1.04	The odds of passing become about 4% larger
Ten more points of attendance	+0.40	1.49	The odds become about 1.5 times larger

None of this proves cause. The table says that students who studied more tended to pass more often, not that an extra hour by itself would lift anyone's chances. A coefficient is a pattern in old records, never an experiment.

10.7 More Than Two Classes: One-vs-Rest and Softmax

Not every question has two answers. Soran would like to sort students into three groups: Fail, Pass and Distinction. Logistic regression handles this in two standard ways, both worth knowing because scikit-learn offers both.

WORDS TO KNOW

One-vs-rest (OvR) — Solving a several-class problem by training one ordinary yes/no model per class — “this class versus everything else” — and predicting whichever class its own model is most confident about. Also called one-vs-all.

Softmax function — A function that turns several raw scores into probabilities that are all positive and add up to exactly 1. It is the several-class version of the sigmoid.

Multinomial logistic regression — A single model that computes one score per class and converts them all at once with softmax. The economist Daniel McFadden developed it in 1974 to study how travelers choose between several ways of making a trip.

One-vs-rest is the simpler idea, and Figure 10.7 shows all of it. With three classes you train three ordinary models — “Fail or not Fail”, “Pass or not Pass”, “Distinction or not Distinction” — run a new student through all three, and let the most confident model win.

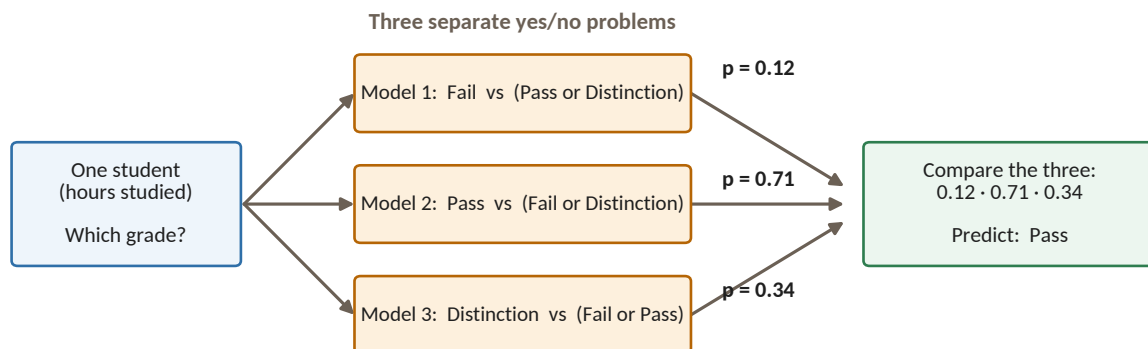


Figure 10.7 – One-vs-rest turns one three-class problem into three ordinary yes/no problems.

WHAT TO OBSERVE

Each model sees the same student but answers a different question, so the three probabilities have no reason to add up to 1 — here they total 1.17. That is not an error, only what happens when three separate models answer three separate questions.

Table 10.5 works through those numbers. Dividing each probability by the total forces them to add to 1 without changing which is largest.

Table 10.5 – One-vs-rest votes for a single student, and the same votes rescaled to add to 1

Model	Question it answers	Probability	Rescaled
Model 1	Fail, rather than anything else?	0.12	0.103
Model 2	Pass, rather than anything else?	0.71	0.607
Model 3	Distinction, rather than anything else?	0.34	0.291
Total		1.17	1.000

The second approach skips the separate models: one score per class inside a single model, converted together, so the probabilities add to 1 by construction.

EQUATION: THE SOFTMAX FUNCTION

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^K e^{z_j}}$$

In words: Raise e to the power of each class score, add the results, and divide each one by the total. Every answer is positive and they always add to exactly 1.

When it is used: Whenever one model must give probabilities for three or more classes at once.

Symbol	Meaning	Unit or type
p_c, z_c	the predicted probability and the linear score of class c	between 0 and 1; no unit
K	the number of classes	count

With exactly two classes, softmax reduces to the sigmoid. Because e raised to any power is positive, no probability can come out negative.

WORKED EXAMPLE 10.7 – Turn Three Scores into Three Probabilities with Softmax

A three-class model gives one student the scores $z_{\text{Fail}} = 2$, $z_{\text{Pass}} = 1$ and $z_{\text{Distinction}} = 0$. Find the three predicted probabilities. Use $e^2 \approx 7.389$, $e^1 \approx 2.718$ and $e^0 = 1$.

1. Raise e to each score: 7.389, 2.718 and 1.000, and add them: $7.389 + 2.718 + 1.000 = 11.107$.
2. Divide each by the total: $7.389 \div 11.107 = 0.665$; $2.718 \div 11.107 = 0.245$; $1.000 \div 11.107 = 0.090$.
3. Check: $0.665 + 0.245 + 0.090 = 1.000$.

Answer: The probabilities are 0.665 for Fail, 0.245 for Pass and 0.090 for Distinction, so the model predicts Fail. A score gap of 1 became a probability about 2.7 times larger, because $e^1 \approx 2.718$.

Figure 10.8 shows the same three numbers before and after the squeeze.

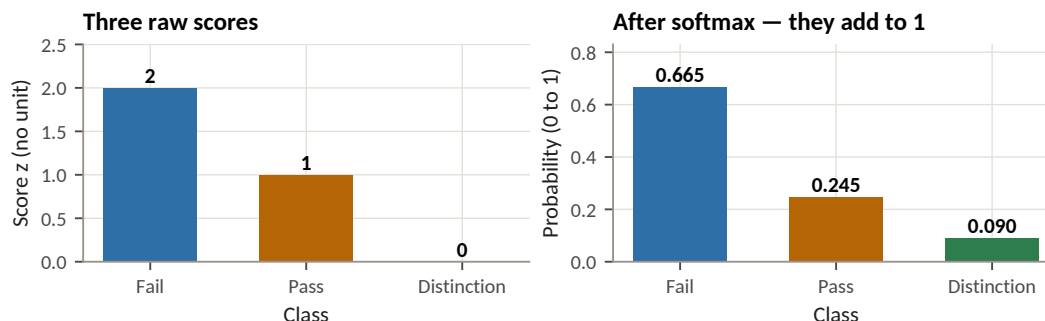


Figure 10.8 – Three raw class scores (left) and the probabilities softmax turns them into (right).

WHAT TO OBSERVE

The class with score 0 does not get probability 0. Softmax cares only about the *gaps* between scores: adding 10 to all three would leave all three probabilities unchanged.

10.8 Regularized Logistic Regression: the C Dial

Chapter 7 showed that a model can fit its training data too closely, and that a penalty on large coefficients is one cure. Logistic regression needs that cure badly: when the two classes can be separated cleanly, nothing stops training from making the coefficients larger and larger forever. Larger coefficients mean a steeper sigmoid, so probabilities are pushed ever closer to 0 and 1 — a model that is not merely confident but reckless. The penalty is the one from Chapter 7, dialed differently: scikit-learn asks not for the strength λ but for C , which sits on the other side.

WORD TO KNOW

C (inverse regularization strength) — The dial controlling how much a model is punished for large coefficients. It works backward from λ : a *small* C means a *strong* penalty and cautious coefficients, a *large* C almost no penalty. It is a hyperparameter, so it is chosen on a validation set, never on the test set.

EQUATION: REGULARIZED LOGISTIC REGRESSION

$$L_C = \underbrace{\frac{1}{2} \sum_{j=1}^k w_j^2}_{\text{penalty on big coefficients}} + C \underbrace{\sum_{i=1}^n L_i}_{\text{total log loss}}$$

In words: Training does not minimize the log loss alone. It minimizes the size of the coefficients plus C times the total log loss, so the two demands must be balanced against each other.

When it is used: Every scikit-learn fit — the penalty is on by default, with $C = 1$.

Symbol	Meaning	Unit or type
L_C, C	the penalized quantity training minimizes, and the inverse regularization strength	no unit
w_j, L_i	the coefficients (the intercept is not penalized), and the log loss of example i	per unit of feature j ; no unit

A large C makes the log-loss term dominate, so fitting wins; a small C makes the penalty dominate, so caution wins. This is λ from Chapter 7 turned upside down: roughly, C behaves like $1/\lambda$.

WORKED EXAMPLE 10.8 — Compare Two Models Under a Strong and a Weak Penalty

Four students studied 3, 4, 6 and 7 hours, with results Fail, Fail, Pass, Pass. Model P is steep, with $w = 2$ and a total log loss of 0.290 on these four. Model Q is gentle, with $w = 0.5$ and a total log loss of 1.575. Both put their boundary at 5 hours. Decide which model training would choose at $C = 10$ and at $C = 0.1$.

1. Penalty of Model P: $\frac{1}{2} \times 2^2 = 2$. Penalty of Model Q: $\frac{1}{2} \times 0.5^2 = 0.125$.
2. At $C = 10$, Model P scores $2 + 10 \times 0.290 = 4.90$ and Model Q scores $0.125 + 10 \times 1.575 = 15.875$, so P wins.
3. At $C = 0.1$, Model P scores $2 + 0.1 \times 0.290 = 2.029$ and Model Q scores $0.125 + 0.1 \times 1.575 = 0.2825$, so Q wins.

Answer: At $C = 10$ training picks the steep, confident Model P; at $C = 0.1$ it picks the gentle, cautious Model Q. Nothing about the data changed — only the dial, and with it the balance between fitting closely and staying modest.

Soran has records for 40 students: 24 for training and 16 kept aside for validation, following Chapter 5. Table 10.6 shows five settings of C , and Figure 10.9 shows three of them as pictures.

Table 10.6 – Five settings of C on Soran's 24 training and 16 validation students (example figures)

C	Coefficient on hours	Coefficient on attendance	Training log loss	Validation log loss
0.01	0.124	0.040	0.552	0.647
0.1	0.643	0.056	0.353	0.508
1	1.611	0.105	0.211	0.476
10	2.618	0.174	0.177	0.557
100	3.029	0.205	0.175	0.606

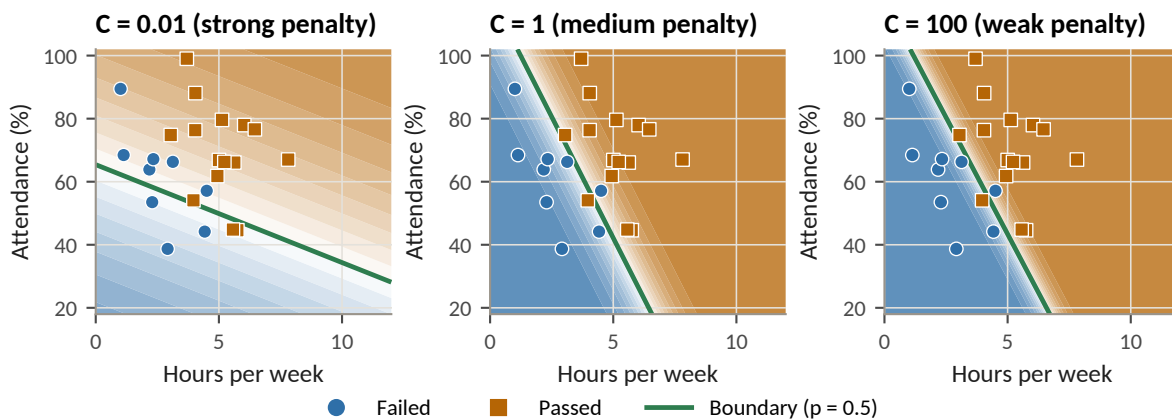


Figure 10.9 – The same 24 students fitted at three penalty strengths. The shading is the predicted probability of passing.

WHAT TO OBSERVE

Look at the shading, not only the line. At $C = 0.01$ the color changes gradually, because small coefficients keep every probability near 0.5. At $C = 100$ the change is almost instant: the model is nearly

certain everywhere, including where it has seen no data. Table 10.6 agrees with the middle picture — validation log loss bottoms out at $C = 1$, the U-shape Chapter 7 taught you to expect.

COMMON MISTAKES AND MISCONCEPTIONS

- Reading a plain linear regression score as if it were already a probability, even when it falls outside the range 0 to 1.
- Expecting a negative score z to produce a negative probability; the sigmoid never leaves the range 0 to 1.
- Training a logistic regression model with mean squared error instead of log loss.
- Believing that 0.5 is always the correct cutoff, whatever the two kinds of mistake cost.
- Reading a coefficient as a change in probability; it is a change in log-odds, a fixed multiplier e^w on the odds.
- Forgetting that in scikit-learn a *small* C means a *strong* penalty, and setting C backward.
- Treating a predicted probability as a fact about a person rather than an estimate that can be wrong.

Practical Activities

ACTIVITY (PAPER) — Probabilities and Two Different Cutoffs

Use Soran's one-feature model, $z = -5 + 1 \times \text{hours studied}$, and four students: Rojin 2 hours, Sara 3 hours, Tara 4.5 hours, Zana 7 hours.

1. Make a table with five columns: student, hours, score z , probability p , prediction. Fill in the whole score column first, before any sigmoid arithmetic.
2. Compute each probability with $\sigma(z) = 1 \div (1 + e^{-z})$ to three decimals, using $e^3 \approx 20.086$, $e^2 \approx 7.389$, $e^{0.5} \approx 1.649$, $e^{-2} \approx 0.135$.
3. Fill in the prediction column with a cutoff of 0.5 (predict Pass when $p \geq 0.5$), then add a sixth column using a cutoff of 0.3. Which student changes, and why is she the only one?
4. Soran wants to be sure he does not miss a student who really needs help. Should he move his cutoff for "predict Pass" up or down from 0.5? Write one or two sentences on what each direction costs him.

You need: Paper, pen and a calculator with an e^x key

ACTIVITY (PYTHON) — Logistic Regression with scikit-learn, in Two Classes and in Three

The first half fits Soran's eight students with `LogisticRegression`, prints the fitted coefficient and intercept, turns them into a decision boundary with $-b/w$, and asks for three predicted probabilities. The second half repeats the idea on a three-class toy set of twelve students, where `predict_proba` returns three numbers for each student instead of one. Run the program as printed, then change Evin's label from 0 to 1 and see how far the boundary moves.

Python · NumPy · scikit-learn

```
import numpy as np
from sklearn.linear_model import LogisticRegression
hours = np.array([[1], [2], [3], [4], [6], [7], [8], [9]])
passed = np.array([0, 0, 0, 1, 0, 1, 1, 1]) # 1 = passed the final
model = LogisticRegression().fit(hours, passed)
w, b = model.coef_[0][0], model.intercept_[0]
print("coefficient %.4f  intercept %.4f" % (w, b))
print("boundary %.2f hours  accuracy %.3f" % (-b / w, model.score(hours, passed)))
for h in [3, 5, 7]:
    print(" %d hours -> P(pass) = %.3f" % (h, model.predict_proba([[h]])[0, 1]))
hrs = np.array([[1], [2], [3], [2], [5], [6], [4], [5], [8], [9], [7], [8]])
grade = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2]) # 0 Fail, 1 Pass, 2 Distinction
multi = LogisticRegression(C=0.3).fit(hrs, grade)
for h in [3.0, 4.5, 7.0]:
    p = multi.predict_proba([[h]])[0]
    print("%.1f h: Fail %.3f Pass %.3f Distinction %.3f (sum %.3f)" % (h, *p, p.sum()))
```

Output

```
coefficient 0.6791  intercept -3.3956
boundary 5.00 hours  accuracy 0.750
 3 hours -> P(pass) = 0.205
 5 hours -> P(pass) = 0.500
 7 hours -> P(pass) = 0.795
3.0 h: Fail 0.605 Pass 0.363 Distinction 0.031 (sum 1.000)
4.5 h: Fail 0.303 Pass 0.553 Distinction 0.144 (sum 1.000)
7.0 h: Fail 0.031 Pass 0.363 Distinction 0.605 (sum 1.000)
```

The boundary is exactly 5.00 hours, the answer of Worked Example 10.4, and the accuracy of 0.750 is the 6 of 8 seen in Figure 10.3. The coefficient 0.6791 is gentler than the 1 used by hand, because scikit-learn applies its default penalty $C = 1$; the boundary is unaffected. In the three-class half the probabilities always sum to 1.000, as softmax guarantees.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — An Early-Warning List for a Statistics Course — Sulaymaniyah

Soran's goal was never to grade students in advance, but to notice in week four rather than week fourteen which students might benefit from a short, friendly conversation. He began as Chapter 1 recommends, with task, experience and performance measure. The task: estimate, from data available in the first four weeks, the probability that a student passes. The experience: 40 labeled records from last year's section. The performance measure: validation log loss, because a probability, not a label, was what he wanted. His first model used hours studied alone, and two of the eight students he checked by hand broke its rule. Adding attendance fixed one of them and turned the boundary from a point into a straight line, as in Figure 10.5. Two features on only 24 training rows made him nervous, so he compared validation log loss at five values of C , as Table 10.6 records; the pattern was the U-shape Chapter 7 had promised, and $C = 1$ won. The coefficients then told him something he had half suspected — an extra hour of weekly study multiplied a student's odds of passing by about 2.2, ten more points of attendance by about 1.5. He was careful, in his notes and with colleagues, to say “students who attend more tend to pass more” and never “attending more makes students pass”.

The hardest decision had nothing to do with mathematics: where to set the cutoff. Missing a struggling student costs that student a term; contacting one who was doing fine costs ten minutes and is rarely unwelcome when the message is warm. Because those mistakes are so unequal, he moved the cutoff away from 0.5, flagging any student whose predicted probability of passing fell below 0.65. The finished tool is a private weekly list, lowest probabilities first, that only Soran sees; it never touches an official record and never sends anything automatically. Each term he adds the new results and retrain — last year's model is a starting point, never a permanent rule.

RESPONSIBLE USE

A predicted probability is not a fact about a person, and a low one is not a verdict. A model never knows why a number is low: a student may be working a night job, caring for a relative, or simply new to studying alone. Use a model like this to offer help early and to trigger a caring human conversation — never to label a student, lower anyone's expectations, or decide something automatically. Chapter 24 returns to this in depth.

CHAPTER SUMMARY

- Logistic regression predicts a probability for a yes-or-no question; despite its name it is a classification method.
- A straight line fitted to 0/1 labels can return values below 0 or above 1, which no probability may take.
- Every prediction has two steps: build the score $z = b + w_1x_1 + \cdots + w_kx_k$, then squeeze it with $\sigma(z) = 1/(1 + e^{-z})$, which always lands strictly between 0 and 1 and gives exactly 0.5 when $z = 0$.
- Log loss grades a probability against the true label: a confident, correct prediction costs almost nothing, a confident, wrong one costs a great deal, and 0.5 always costs 0.693.
- Training minimizes the average log loss — the same thing as maximum likelihood — with the gradient descent step $w \leftarrow w + \alpha \sum x_i(y_i - p_i)$.
- The decision boundary is where $z = 0$: a point with one feature ($x = -b/w$), a straight line with two, a flat hyperplane with many.
- The score z is the log-odds, so a coefficient w multiplies the odds by e^w ; it does not add to the probability.
- Three or more classes are handled by one-vs-rest, one yes/no model per class, or by softmax, which turns several scores into probabilities adding to 1.
- In scikit-learn the penalty dial is C , the inverse regularization strength: small C means a strong penalty, large C the opposite. A predicted probability is an estimate that can be wrong — use it to offer help, never to hand down a verdict.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain in your own words why a plain linear regression score cannot be used directly as a probability. Give one example of a value it might return that no probability may take.

Problem 2 [Basic] State the probability the sigmoid function returns when $z = 0$, without doing any arithmetic, and explain in one sentence why.

Problem 3 [Basic] Define log loss in one or two sentences, and state what it costs for a perfect, fully confident, correct prediction and for a prediction of $p = 0.5$.

Problem 4 [Basic] A model uses $z = -6 + 2 \times \text{hours studied}$. Compute the score for a student who studies 4 hours per week and say whether the model predicts Pass or Fail at the 0.5 cutoff.

Problem 5 [Standard] A model gives a student a score of $z = 3$. Using $\sigma(3) = 1 \div (1 + e^{-3})$ and $e^{-3} \approx 0.0498$, find the predicted probability, showing every step.

Problem 6 [Standard] A model predicts $p = 0.7$ for a student. (a) Find the log loss if the student passed ($y = 1$), using $\ln(0.7) \approx -0.357$. (b) Find the log loss if the same student had failed ($y = 0$), using $\ln(0.3) \approx -1.204$. (c) State in one sentence what the comparison shows.

Problem 7 [Standard] A logistic regression model uses $z = -8 + 2 \times \text{hours studied}$. Find its decision boundary, showing every step, and state which students it predicts will pass.

Problem 8 [Standard] A tiny dataset has two examples: $x = 1$ with $y = 0$, and $x = 3$ with $y = 1$. Starting from $w = 0$ and $b = 0$ with a learning rate of 0.2, find w and b after one round of gradient descent, showing every step.

Problem 9 [Standard] A model predicts a probability of 0.75 that a loan will be repaid. Find the odds and the log-odds of repayment, showing every step. Use $\ln(3) \approx 1.099$.

Problem 10 [Standard] A three-class model gives one example the scores 1, 0 and -1 . Using $e^1 \approx 2.718$, $e^0 = 1$ and $e^{-1} \approx 0.368$, find the three softmax probabilities and check that they add to 1.

Problem 11 [Challenge] Using $z = -6 + 0.8 \text{ hours} + 0.04 \text{ attendance}$, find how many hours a student must study to reach a probability of exactly 0.5 (a) at 80% attendance and (b) at 40% attendance, then (c) say in one sentence what the difference tells you about the boundary.

Problem 12 [Challenge] A model has a training log loss of 0.20 and a validation log loss of 0.85. (a) Which problem from Chapter 7 does this suggest? (b) Suggest two fixes, saying for each whether it makes C larger or smaller. (c) After the fix, training log loss rises to 0.30 but validation log loss falls to 0.35 — why is that a good trade?

Problem 13 [Challenge] A credit model has a coefficient of -0.7 on “number of previous loans”. (a) Find the odds multiplier for one extra loan, using $e^{-0.7} \approx 0.497$. (b) A borrower’s odds of repaying are 2.0; find her new odds and new probability. (c) Say in one sentence why this is not evidence that another loan causes people to repay less often.

k-Nearest Neighbors

Predict like your neighbors — the simplest model that needs no training

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how k-Nearest Neighbors predicts a label by finding the closest training examples and letting them vote.
- Calculate Euclidean and Manhattan distances between two examples by hand, with two or more features.
- Standardize features before measuring distance, and show how scaling can flip a prediction.
- Compare a small k with a large k , and pick k from a validation curve instead of guessing.
- Use distance-weighted voting so that closer neighbors count more, and break ties safely.
- Predict a number with kNN by averaging the neighbors' target values.
- Describe the prediction cost of kNN and explain why too many features weaken every distance.

Introduction

Imagine you have just moved to a new street in Sulaymaniyah. You want fresh bread, but you do not know a single bakery in the area, and you have no guidebook. So you do the simplest thing that actually works: you ask the four or five neighbors who live closest to you. If most of them send you to the same bakery, you go there. You trust that answer because people who live close to you walk the same streets and notice the same things.

This chapter turns that ordinary habit into a complete machine learning method. Instead of asking human neighbors about bread, we ask a new example's closest neighbors inside the training data what label they carry, and we let those neighbors vote. There is no equation to fit and almost no training step. The method simply remembers every example it has seen, and does all of its thinking at the moment a question is asked.

That simplicity makes kNN a fine teaching model, and also a fussy one in practice, because everything depends on the word “close”. You will learn how to measure closeness, why the units you measure in can change the answer completely, and how many neighbors to ask. Every calculation is small enough to do with a pencil, and a short Python program at the end reproduces the same numbers.

11.1 Predict Like Your Neighbors

A fruit export company near Duhok runs a packing facility that sorts apples before shipping. An inspector picks up each apple, weighs it in her hand, presses the skin with a thumb and calls out a grade: Grade A for the premium boxes, Grade B for the standard boxes. She is good at her job, but two inspectors do not always agree on a borderline apple, and hand grading slows the line down during the harvest. The company wants a machine to give a fast, consistent first opinion.

WORDS TO KNOW

k-Nearest Neighbors (kNN) — A method that predicts the label of a new example by finding the k training examples most similar to it, then letting those k examples vote on the answer. The letter k stands for how many neighbors get a vote.

Instance-based learning — A family of methods, also called *lazy learning*, that store the training examples themselves instead of summarizing them into an equation. All the work happens at prediction time. kNN is the best-known member.

The nearest-neighbor idea is one of the oldest in the subject. It was first written down by Evelyn Fix and Joseph Hodges in 1951, and analyzed by Thomas Cover and Peter Hart in 1967, who proved that even a single nearest neighbor is a surprisingly strong classifier when there is plenty of data.

kNN is also the strangest model you have met so far. Linear regression in Chapter 8 and logistic regression in Chapter 10 each fit a fixed equation during training, then throw the original rows away. kNN does the opposite: it keeps every training example and waits. The nearest-centroid classifier of Chapter 1 squeezed each class into one average point; kNN never averages the data away, which lets it follow shapes one average point could never describe.

The company's training file holds 120 apples that inspectors have already graded by hand. Two numbers are recorded automatically for every apple: weight in grams, and a firmness score from 0 to 10 from a small probe, where a higher score means a firmer apple. Across the whole file the mean weight is 170 g with a standard deviation of 40 g, and the mean firmness is 5.0 with a standard deviation of 2.0. Table 11.1 lists eight of those apples, chosen to cover the range; every hand calculation in this chapter uses these eight.

Table 11.1 – Eight graded apples from the packing line — the hand dataset for this chapter

Apple	Weight (g)	Firmness (0–10)	Grade
A1	115	8	A
A2	195	7	A
A3	210	9	A
A4	220	8	A
B1	80	3	B
B2	90	2	B
B3	100	4	B
B4	150	3	B

Look carefully at apple A1. It weighs only 115 g, light for a premium apple, yet the inspector still graded it A — perhaps for a deep red color or a perfect shape, reasons the two recorded numbers cannot see. Keep this small apple in mind; it matters later. To keep the first calculation simple, this lesson uses weight only. With a single feature the distance between two apples is just the difference in their weights with the minus sign dropped. Once the distances are known, the k closest apples vote, and the label with the most votes wins.

EQUATION: MAJORITY VOTE AMONG THE k NEIGHBORS

$$V_c = \sum_{i \in N_k(q)} \mathbf{1}\{y_i = c\}, \quad \hat{y} = \arg \max_c V_c$$

In words: Count how many of the k nearest neighbors carry each class label. The class with the biggest count is the prediction.

When it is used: Every time a kNN classifier is asked for a label, once the k nearest training examples have been found.

Symbol	Meaning	Unit or type
q	the new example we want to label	—
$N_k(q)$	the set of the k training examples closest to q	k examples
$\mathbf{1}\{y_i = c\}$	equals 1 when neighbor i carries label c , and 0 otherwise	0 or 1
V_c	the number of votes class c receives	count
$\hat{y}$	the predicted class: the one with the largest V_c	a class name

$\arg \max$ is read “the value of c that makes V_c as large as possible”. With two classes and an odd k there is always a single winner.

WORKED EXAMPLE 11.1 — Classify a New Apple Using kNN with $k = 3$

A new apple weighs 205 g. Using the eight apples of Table 11.1, weight only and $k = 3$, predict its grade by majority vote.

- Find the distance from 205 g to every training apple by subtracting and dropping the sign:
 $|115 - 205| = 90$, $|195 - 205| = 10$, $|210 - 205| = 5$, $|220 - 205| = 15$, $|80 - 205| = 125$,
 $|90 - 205| = 115$, $|100 - 205| = 105$, $|150 - 205| = 55$.
- Sort the eight distances: 5 (A3, Grade A), 10 (A2, A), 15 (A4, A), 55 (B4, B), 90 (A1, A), 105 (B3, B), 115 (B2, B), 125 (B1, B).
- Because $k = 3$, keep only the three smallest: A3, A2 and A4, all Grade A.
- Count the votes: $V_A = 3$ and $V_B = 0$, so Grade A wins.

Answer: The new apple is classified as Grade A, by three votes to zero. All three of its nearest neighbors by weight are premium apples, so the vote is unanimous.

Table 11.2 lays out the same calculation as a table, the way a program would store it, and Figure 11.1 draws the eight apples along a weight axis.

Table 11.2 – Distance table for the new 205 g apple, sorted from nearest to farthest

Rank	Apple	Weight (g)	Distance (g)	Grade
1	A3	210	5	A
2	A2	195	10	A
3	A4	220	15	A
4	B4	150	55	B
5	A1	115	90	A
6	B3	100	105	B
7	B2	90	115	B
8	B1	80	125	B

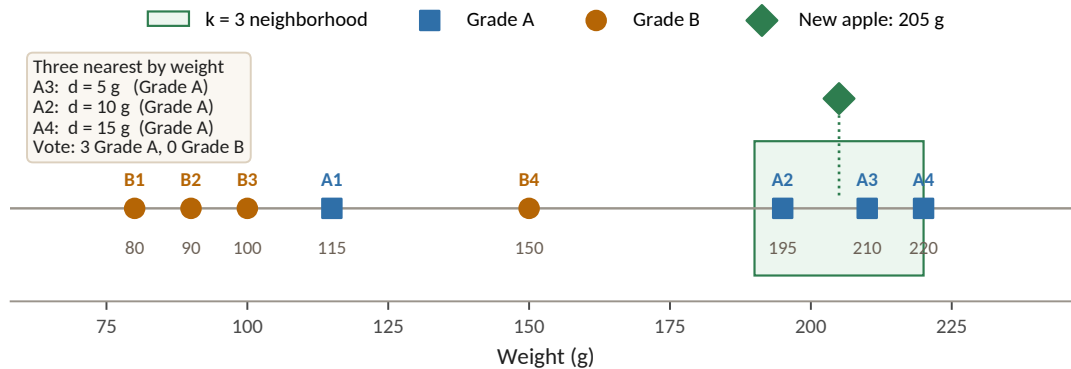


Figure 11.1 – The eight training apples on a weight axis. Grade A is premium, Grade B is standard. The shaded band holds the three nearest neighbors of the new 205 g apple.

WHAT TO OBSERVE

All three apples inside the shaded band are Grade A, so the vote is unanimous. The band is centered on the new apple, not on any training apple, and it stretches just far enough to catch exactly three neighbors; a different new apple would produce a different band.

COMMON MISTAKE

✗ Wrong: Believing kNN has no real work left to do at prediction time, just because it has almost no training step.

✓ Right: kNN moves its work from training time to prediction time. Every new example requires a distance to every stored training example, which becomes slow once the training set is large.

11.2 Distance: Euclidean, Manhattan and Why Scaling Matters

Weight alone is not enough for a real packing line. Once firmness is added, each apple becomes a point with two numbers, and “distance” needs a proper definition.

WORD TO KNOW

Feature space — The imaginary space in which each example is one point and each feature is one axis. Two features make a flat plane you can draw on paper; ten features make a space nobody can picture, though the arithmetic works exactly the same way.

There is more than one sensible way to measure how far apart two points are. Picture two points on graph paper that differ by 4 units across and 3 units up. You can cut straight across the diagonal, which is 5 units, or you can walk along the axes like a city street grid, 4 then 3, which is 7 units. Those are the two distances this chapter uses.

WORDS TO KNOW

Euclidean distance — The straight-line distance between two points: square the difference in each feature, add the squares, and take the square root. With two features this is the diagonal of a right triangle.

Manhattan distance — The distance you travel moving along one axis at a time: add up the size of each feature’s difference, with no squaring and no square root.

EQUATION: EUCLIDEAN DISTANCE BETWEEN TWO EXAMPLES

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{j=1}^p (a_j - b_j)^2}$$

In words: Subtract the two examples feature by feature, square every difference, add all the squares, and take the square root of the total.

When it is used: As the default distance for kNN whenever every feature is numerical and has been put on a shared scale.

Symbol	Meaning	Unit or type
$\mathbf{a}, \mathbf{b}$	the two examples being compared	—
p	the number of features	count
a_j, b_j	feature j of example $\mathbf{a}$ and of example $\mathbf{b}$	unit of feature j
d	how far apart the two examples are	no natural unit when features differ

With three or more features, add one more squared difference for each extra feature before taking the square root. Nothing else changes.

EQUATION: MANHATTAN DISTANCE BETWEEN TWO EXAMPLES

$$d_1(\mathbf{a}, \mathbf{b}) = \sum_{j=1}^p |a_j - b_j|$$

In words: Subtract the two examples feature by feature, drop every minus sign, and add the results.

When it is used: When you want a distance that is less disturbed by one very large difference in a single feature, or when the features are counts rather than smooth measurements.

Symbol	Meaning	Unit or type
d_1	Manhattan distance between the two examples	no natural unit
$ a_j - b_j $	the size of the difference in feature j , sign dropped	unit of feature j

Both belong to one family, the Minkowski distances: Manhattan uses powers of 1, Euclidean uses powers of 2. In scikit-learn this is the setting $p=1$ or $p=2$.

WORKED EXAMPLE 11.2 — Find the Euclidean and Manhattan Distance Between Two Apples

Apple A1 weighs 115 g with a firmness of 8. Apple B4 weighs 150 g with a firmness of 3. Find both distances, using the raw recorded numbers.

1. Subtract the weights and square: $115 - 150 = -35$, and $(-35)^2 = 1225$.
2. Subtract the firmness scores and square: $8 - 3 = 5$, and $5^2 = 25$.
3. Add the two squared differences: $1225 + 25 = 1250$.
4. Take the square root: $\sqrt{1250} \approx 35.36$. This is the Euclidean distance.
5. For Manhattan distance, drop the signs and simply add: $35 + 5 = 40$.
6. Compare the two parts of the squared total: weight contributed 1225 of the 1250, and $1225 \div 1250 = 0.98$, or 98%.

Answer: The Euclidean distance is about 35.36 and the Manhattan distance is 40. Neither has a real-world unit, because grams and firmness points were mixed together. Worse, weight supplied 98% of the squared distance by itself — firmness had almost no say.

Table 11.3 – Euclidean and Manhattan distance for four pairs of apples, using the raw recorded numbers

Pair	Weight gap (g)	Firmness gap	Manhattan	Euclidean
A1 and B4	35	5	40	35.36
A1 and B3	15	4	19	15.52
A2 and A3	15	2	17	15.13
B2 and B3	10	2	12	10.20

Table 11.3 repeats both distances for four pairs; here both measures rank the pairs the same way, which is common but not guaranteed. Real square roots rarely come out neatly, and kNN usually does not need them: comparing sums of squared differences gives exactly the same ordering as comparing true distances, because squaring never reorders positive numbers. Chapter 1 used the same shortcut for centroids, and so does the rest of this lesson.

Now for the trap that catches almost every beginner. Weight is measured in grams, with values in the hundreds; firmness runs from 0 to 10. When two such features go into one distance, the big-number feature quietly takes over.

The fix is the one Chapter 4 already taught: standardization. Subtract the training mean from each value and divide by the training standard deviation, so every feature ends up centered on 0 with a spread of about 1. Here the standardized weight is $(\text{weight} - 170) \div 40$ and the standardized firmness is $(\text{firmness} - 5) \div 2$. The next worked example classifies one apple twice, once each way.

WORKED EXAMPLE 11.3 – Classify One Apple Twice: Raw Features, Then Standardized

A new apple Q weighs 170 g and has a firmness of 7. Using the eight apples of Table 11.1, predict its grade at $k = 1$ with raw features, then standardize everything and predict again at $k = 1$ and $k = 3$.

1. Raw squared distance from Q (170, 7) to B4 (150, 3): weight $170 - 150 = 20$, so $20^2 = 400$; firmness $7 - 3 = 4$, so $4^2 = 16$. Add: $400 + 16 = 416$.
2. Raw squared distance from Q to A2 (195, 7): $(-25)^2 = 625$ plus $0^2 = 0$, total 625. The other six, worked the same way, are A3 1604, A4 2501, A1 3026, B3 4909, B2 6425 and B1 8116.
3. The smallest is 416, apple B4, so with raw features the prediction is **Grade B**.
4. Now standardize Q: $(170 - 170) \div 40 = 0$ and $(7 - 5) \div 2 = 1$, so Q becomes (0, 1).
5. Standardize A2: $(195 - 170) \div 40 = 0.625$ and $(7 - 5) \div 2 = 1$, giving (0.625, 1). Standardize B4: $(150 - 170) \div 40 = -0.5$ and $(3 - 5) \div 2 = -1$, giving (-0.5, -1).
6. Standardized squared distance from Q to A2: $(0 - 0.625)^2 = 0.390625$ plus $(1 - 1)^2 = 0$, total 0.390625. From Q to B4: $(0 + 0.5)^2 = 0.25$ plus $(1 + 1)^2 = 4$, total 4.25.
7. The other five, standardized the same way, are A4 1.8125, A3 2.0, A1 2.140625, B3 5.3125, B1 9.0625 and B2 10.25.
8. At $k = 1$ the winner is now A2 at 0.390625, Grade A. At $k = 3$ the three nearest are A2, A4 and A3, all Grade A, so the vote is 3 to 0.

Answer: Raw features say Grade B; standardized features say Grade A, at both $k = 1$ and $k = 3$. Q's firmness of 7 matches A2's exactly, but in raw units that signal was worth only 16 out of 416. Once weight and firmness sit on a fair, shared scale, firmness finally counts.

Figure 11.2 draws both calculations side by side.

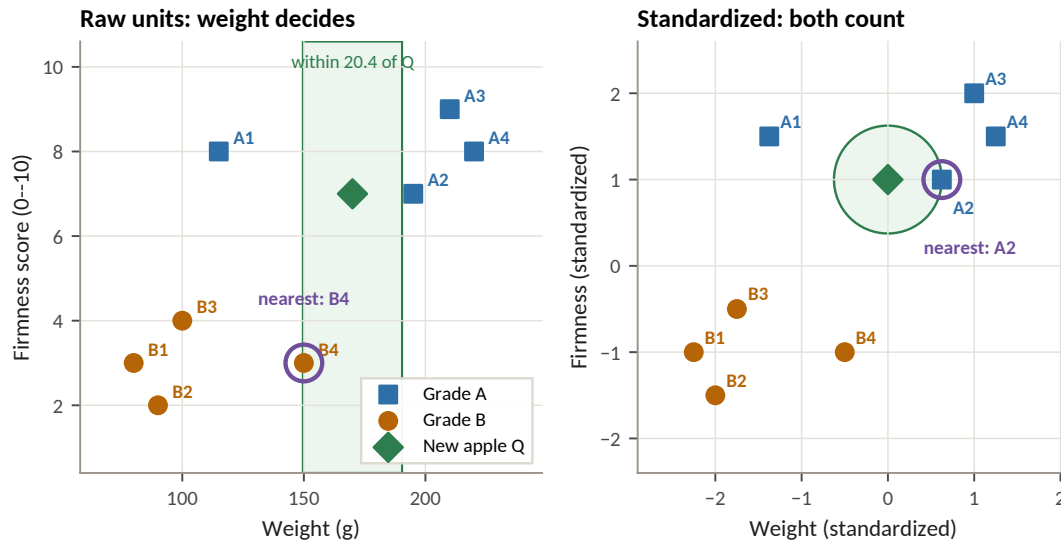


Figure 11.2 – The same eight apples and the same new apple Q, measured in raw units (left) and standardized units (right). The shaded area holds everything within the distance of the single nearest neighbor.

WHAT TO OBSERVE

On the left the shaded region is a tall band: an apple counts as “near” Q if its weight is close, whatever its firmness, and the nearest neighbor is B4, a standard apple. On the right the region is a proper circle, both features count equally, and the nearest neighbor becomes A2, a premium apple. Nothing about the apples changed — only the units did.

IMPORTANT

Compute the mean and the standard deviation from the training data alone, as Chapter 4 insisted, then reuse those two numbers for every validation, test and future apple. Recomputing them from data that is supposed to stay hidden is data leakage, and it spoils every distance measured afterward. In scikit-learn, wrap the scaler and the model in a Pipeline so the scaler can only ever be fitted on a training fold.

COMMON MISTAKE

✗ Wrong: Comparing raw distances between features on very different scales, letting a feature measured in grams silently drown out a score from 0 to 10.

✓ Right: Standardize every numerical feature before computing a distance for kNN. Distance-based methods are far more sensitive to feature scale than most other models in this book.

Figure 11.3 shows the finished two-feature prediction: apple Q, its three nearest neighbors in standardized space, and the vote they cast.

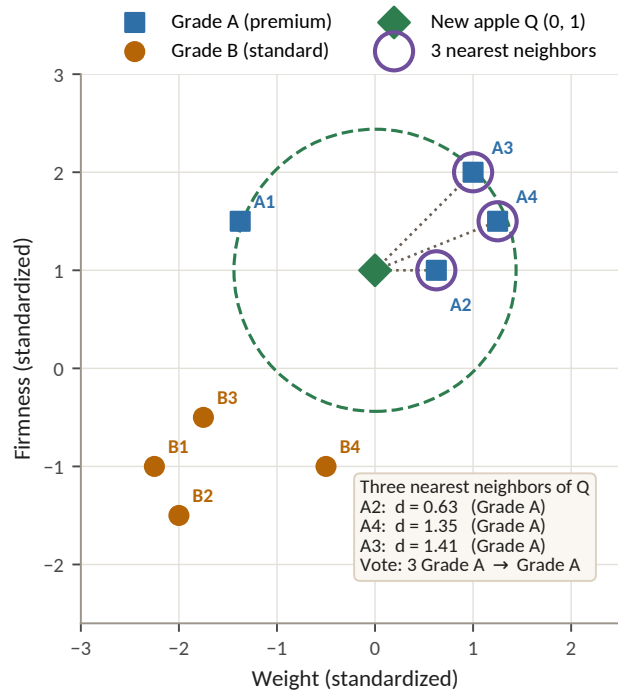


Figure 11.3 – Apple Q and its three nearest neighbors in standardized feature space. The dashed circle is drawn just wide enough to hold exactly three neighbors.

WHAT TO OBSERVE

The three circled apples are A2, A4 and A3, all Grade A, so the prediction is Grade A. Apple A1 sits only a little farther out at $d = 1.46$, close enough that a slightly larger circle would catch it — though the vote would stay 3–0, because A1 is a premium apple too.

11.3 Choosing k : Small Against Large

Nothing so far explained where the number 3 came from. The learning algorithm does not choose k ; a person does. That makes k a hyperparameter, the kind of setting Chapter 7 said to choose with a validation set rather than a guess, and it decides how much one unusual neighbor can influence the answer. Remember apple A1, the unusually light Grade A apple at 115 g. A new apple now arrives weighing 112 g, close to that odd apple and also fairly close to the standard apples. To keep the arithmetic visible, this lesson goes back to weight alone.

WORKED EXAMPLE 11.4 — Classify a Boundary Apple at Three Different Values of k

A new apple weighs 112 g. Using the eight apples of Table 11.1 and weight only, predict its grade with $k = 1$, then $k = 3$, then $k = 5$.

1. Find the distance from 112 g to every training apple: $|115 - 112| = 3$, $|195 - 112| = 83$, $|210 - 112| = 98$, $|220 - 112| = 108$, $|80 - 112| = 32$, $|90 - 112| = 22$, $|100 - 112| = 12$, $|150 - 112| = 38$.
2. Sort them: 3 (A1, A), 12 (B3, B), 22 (B2, B), 32 (B1, B), 38 (B4, B), 83 (A2, A), 98 (A3, A), 108 (A4, A).
3. At $k = 1$ keep only A1: $V_A = 1$, $V_B = 0$, so the prediction is Grade A.
4. At $k = 3$ keep A1, B3 and B2: $V_A = 1$, $V_B = 2$, so the prediction is Grade B.
5. At $k = 5$ keep A1, B3, B2, B1 and B4: $V_A = 1$, $V_B = 4$, so the prediction is Grade B.

Answer: $k = 1$ predicts Grade A, while $k = 3$ and $k = 5$ both predict Grade B. One very close but unusual neighbor controlled the whole prediction at $k = 1$; asking more neighbors gave the steadier answer that matches this apple's real neighborhood.

This is the bias-variance trade-off from Chapter 7 in a new disguise. A very small k lets one training example decide, so the model reacts to every accident in the data: high variance, and overfitting. A very large k asks so many neighbors that the vote barely notices where the new example sits; at the extreme, k equal to the whole training set predicts the majority class for every apple in the world: high bias, and underfitting.

Figure 11.4 repeats the same apple at $k = 1$, $k = 3$ and $k = 7$, so you can watch the neighborhood widen.

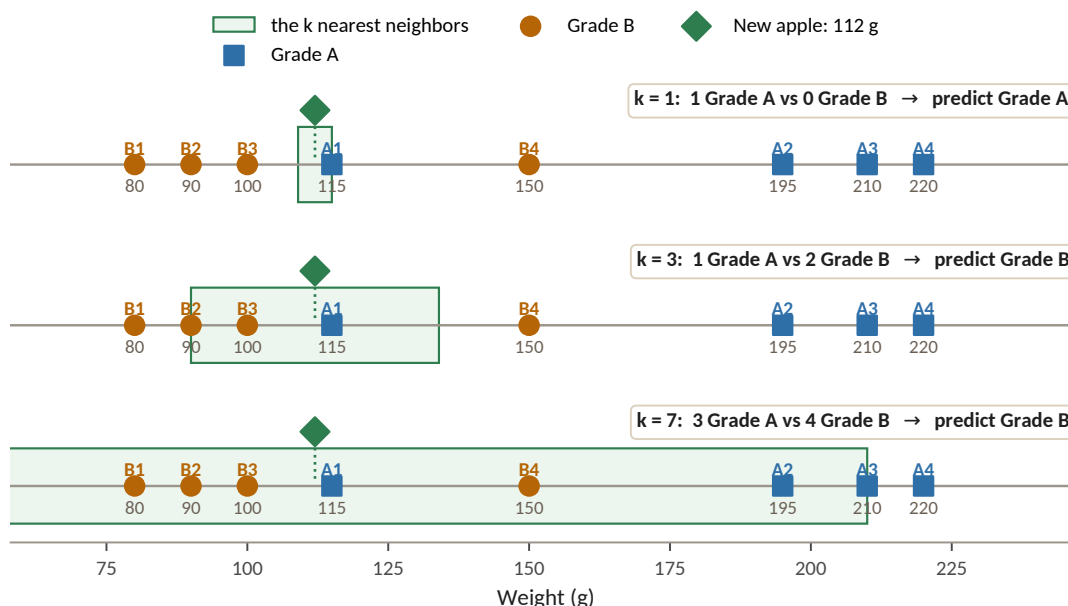


Figure 11.4 – The same 112 g apple judged by one, three and seven neighbors. The shaded band widens each time to hold exactly k training apples.

WHAT TO OBSERVE

At $k = 1$ the band is so narrow that it holds only apple A1, and one unusual apple decides everything. At $k = 3$ and $k = 7$ the band is wide enough to include the standard apples that really surround this weight. At $k = 8$ the band would swallow all eight apples, giving four votes each — an exact tie, and no answer at all.

WORD TO KNOW

Decision boundary — The dividing line, or curve, in feature space where a classifier switches from predicting one class to predicting another. Points on one side get one label; points on the other side get the other.

Figure 11.5 shows the trade-off directly, by coloring every point of the feature plane with the grade the model would predict there, using all 120 training apples.

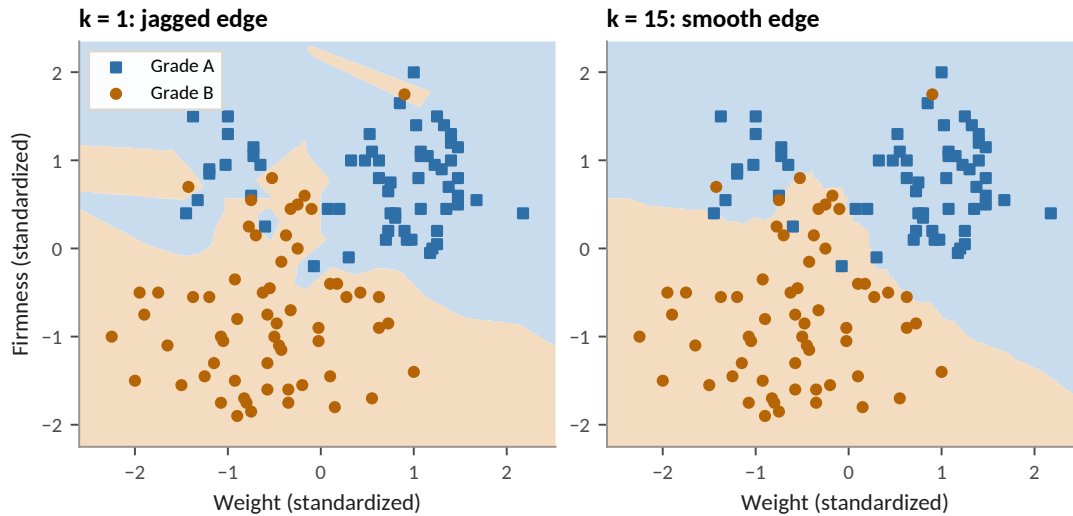


Figure 11.5 – Predicted grade for every point of the standardized feature plane, using all 120 training apples. Left: $k = 1$. Right: $k = 15$.

WHAT TO OBSERVE

At $k = 1$ the boundary is jagged, and every isolated apple carves out its own small island of color — a classic picture of overfitting. At $k = 15$ the boundary is a smooth curve that ignores stray apples. Both models came from exactly the same 120 apples; only k changed.

So which k is right? Training accuracy cannot say: at $k = 1$ every training apple is its own nearest neighbor, so training accuracy is a perfect 100% however noisy the data is. The honest method is the one Chapter 5 taught — score each candidate k with cross-validation and keep the winner, as Figure 11.6 and Table 11.4 do here.

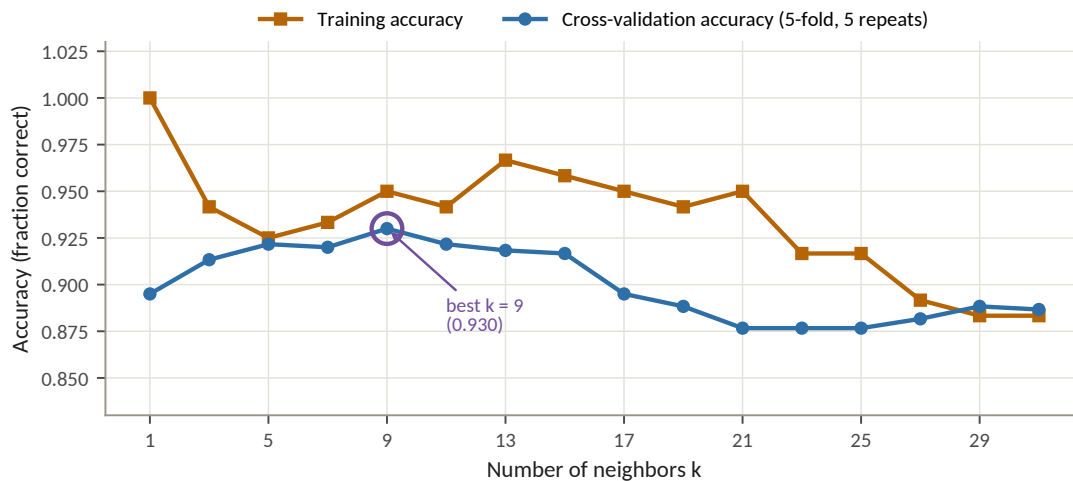


Figure 11.6 – Training accuracy and cross-validated accuracy against k for the 120 apples. The cross-validation curve rises, peaks and then falls again.

Table 11.4 – Cross-validated accuracy for eight values of k on the 120-apple training file (5-fold cross-validation, repeated 5 times)

k	1	5	9	13	17	21	25	29
Accuracy	0.895	0.922	0.930	0.918	0.895	0.877	0.877	0.888

WHAT TO OBSERVE

The training curve starts at a perfect 1.000 at $k = 1$ and stays high everywhere, so it never warns you about a bad choice. The cross-validation curve tells the real story: it climbs from 0.895 at $k = 1$ to a peak of 0.930 at $k = 9$, then slides down as larger neighborhoods blur the two grades together. Choose k from the blue curve, never the orange one.

With no time to search, a useful first guess is $k \approx \sqrt{n}$: with 120 apples that suggests about 11, close to the 9 that cross-validation chose. Treat it as a place to start, not as an answer.

COMMON MISTAKE

✗ Wrong: Choosing an even k in a two-class problem, such as $k = 4$, which can produce an exact tie with no winner.

✓ Right: Use an odd k whenever there are two classes, so a majority always exists. With three or more classes a tie is still possible, so fix the tie-breaking rule in advance.

11.4 Weighted Voting and Ties

Every vote has counted equally so far. Look again at the boundary apple: at $k = 3$ its neighbors were A1 at a distance of 3, B3 at 12 and B2 at 22. The Grade A neighbor sat more than seven times closer than the farthest Grade B neighbor, yet all three votes counted the same.

WORDS TO KNOW

Distance-weighted voting — A version of the kNN vote in which each neighbor's vote is multiplied by a weight that shrinks as distance grows, most often $1 \div d$, so close neighbors count more than distant ones.

Tie — A vote in which two or more classes collect the same total, so the majority rule has no winner. Ties are common with an even k in a two-class problem.

Weighted voting was introduced by Sahibsingh Dudani in 1976 and is now the standard alternative to a plain vote; in scikit-learn it is the setting `weights="distance"`.

EQUATION: DISTANCE-WEIGHTED VOTE

$$V_c = \sum_{i \in N_k(q)} \frac{\mathbf{1}_{\{y_i = c\}}}{d_i}, \quad \hat{y} = \arg \max_c V_c$$

In words: Give every one of the k neighbors a weight of one divided by its distance, add up the weights of the neighbors carrying each class, and predict the class with the largest total.

When it is used: When the k neighbors sit at very different distances, or when a sensible k still drags in neighbors that are clearly farther away than the rest.

Symbol	Meaning	Unit or type
d_i	the distance from the new example q to neighbor i	same units as the distance used
$1 \div d_i$	the weight of neighbor i 's vote	no unit
V_c	the total weight collected by class c	no unit

If a new example sits exactly on a training example, $d_i = 0$ and $1 \div d_i$ is undefined; software handles this by returning that example's label.

WORKED EXAMPLE 11.5 — Reclassify the Boundary Apple Using Distance-Weighted Voting

At $k = 3$ the 112 g apple has neighbors A1 (Grade A, $d = 3$), B3 (Grade B, $d = 12$) and B2 (Grade B, $d = 22$). Give each a weight of $1 \div d$ and predict the grade.

1. Weight of the Grade A neighbor A1: $1 \div 3 = 0.333$.
2. Weight of the first Grade B neighbor B3: $1 \div 12 = 0.083$.
3. Weight of the second Grade B neighbor B2: $1 \div 22 = 0.045$.
4. Add the Grade A weights: there is only one, so $V_A = 0.333$.
5. Add the Grade B weights: $V_B = 0.083 + 0.045 = 0.128$.
6. Compare: 0.333 is clearly larger than 0.128.

Answer: Weighted voting predicts Grade A, the opposite of the plain vote at $k = 3$, which said Grade B by two votes to one. The single Grade A neighbor sat so much closer that, once distance is allowed to matter, its opinion outweighs both other neighbors together.

Figure 11.7 draws both votes side by side.

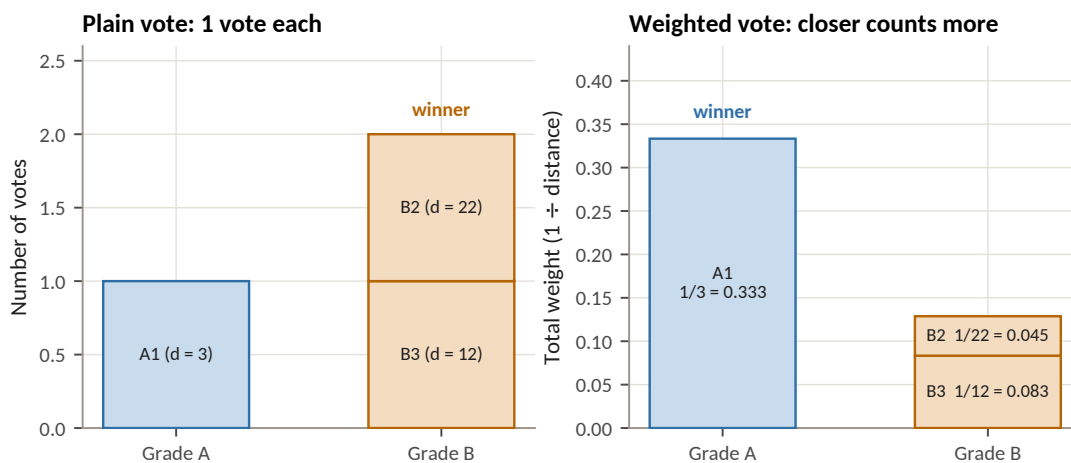


Figure 11.7 – The same three neighbors, counted two ways. Left: one vote each. Right: each vote multiplied by one divided by its distance.

WHAT TO OBSERVE

On the left the Grade B bar is twice as tall, because two neighbors beat one. On the right the two Grade B weights added together, 0.128, do not even reach half of the single Grade A weight of 0.333, because both Grade B neighbors are four to seven times farther away.

Weighted voting also softens the problem of ties, because two totals of real numbers are unlikely to be exactly equal. Three tie-breaking rules are used in practice, and a project should pick one in advance: fall back on the single nearest neighbor, switch to distance weighting, or choose whichever class is more common in training. Do not simply accept the software default — scikit-learn breaks ties alphabetically, which is an accident, not a decision about your problem.

11.5 kNN for Predicting Numbers

Everything so far predicted a category. The same idea predicts a number just as easily: instead of counting the neighbors' labels, average their target values.

WORD TO KNOW

kNN regression — Predicting a number for a new example by taking the mean of the target values of its k nearest training examples. The distance step is unchanged; only the last step changes, from counting to averaging.

EQUATION: kNN REGRESSION: AVERAGE THE NEIGHBORS

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(q)} y_i$$

In words: Add up the target values of the k nearest training examples and divide by k .

When it is used: To predict a number — a rent, a price, a temperature, a yield — when similar examples are expected to have similar answers.

Symbol	Meaning	Unit or type
$\hat{y}$	the predicted number for the new example	unit of the target
k	how many neighbors are averaged	count
y_i	the known target value of neighbor i	unit of the target

The distance-weighted version replaces the plain mean with $\hat{y} = (\sum w_i y_i) \div (\sum w_i)$, using $w_i = 1 \div d_i$, so nearer neighbors pull the prediction harder.

An agent in Erbil keeps a file of apartments she has already rented out, shown in Table 11.5. A new apartment of 100 m² has come onto her books and she needs a starting rent.

Table 11.5 – Six apartments already rented in Erbil — the training data for kNN regression

Apartment	P1	P2	P3	P4	P5	P6
Size (m ²)	60	75	90	105	120	150
Rent (thousand IQD)	300	360	420	500	550	660

WORKED EXAMPLE 11.6 — Predict a Rent from Three Neighbors, Plain and Weighted

Using Table 11.5, size as the only feature and $k = 3$, predict the monthly rent of a 100 m² apartment, first with a plain average and then with distance weighting.

- Find the distances from 100 m²: $|60 - 100| = 40$, $|75 - 100| = 25$, $|90 - 100| = 10$, $|105 - 100| = 5$, $|120 - 100| = 20$, $|150 - 100| = 50$.
- Sort them and keep the three nearest: P4 ($d = 5$, rent 500), P3 ($d = 10$, rent 420) and P5 ($d = 20$, rent 550).

3. Plain average: $500 + 420 + 550 = 1470$, then $1470 \div 3 = 490$.
4. Now the weights: $1 \div 5 = 0.20$, $1 \div 10 = 0.10$ and $1 \div 20 = 0.05$.
5. Multiply each rent by its weight: $0.20 \times 500 = 100$, $0.10 \times 420 = 42$, $0.05 \times 550 = 27.5$, and add: 169.5.
6. Add the weights: $0.20 + 0.10 + 0.05 = 0.35$.
7. Divide: $169.5 \div 0.35 \approx 484.29$.

Answer: The plain prediction is 490 thousand IQD per month and the weighted prediction is about 484.3. Weighting pulled the answer down because the closest apartment, P4 at 105 m², rents for 500, while the farthest neighbor at 550 lost most of its influence.

Figure 11.8 sweeps the new apartment's size across the whole axis, showing the entire prediction rule rather than this one answer.

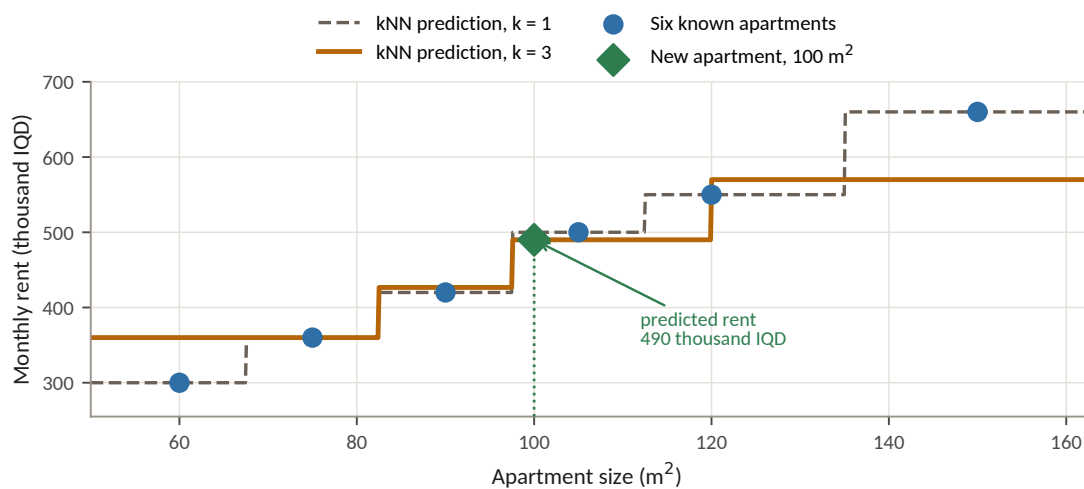


Figure 11.8 – kNN regression predictions across all apartment sizes. The prediction is a staircase, not a line, because it only changes when the set of k nearest neighbors changes.

WHAT TO OBSERVE

Both curves are flat steps: the prediction holds still until the new apartment moves close enough to swap one neighbor for another, then jumps. The $k = 1$ staircase has taller steps, because changing one of three neighbors moves an average less than changing the only neighbor. Neither curve ever leaves the range 300 to 660.

That last point is a real limitation. A linear model can extrapolate — Chapter 8 would quote a rent for a 400 m² apartment by continuing its line — but kNN regression cannot. Asked about one, it returns the average of the biggest apartments it has seen and stops responding to size at all.

11.6 Cost, Strengths and the Curse of Dimensionality

kNN's training step costs almost nothing — it just stores the data — and the bill arrives at every prediction instead.

WORDS TO KNOW

Query time — The work a model must do to answer one prediction request. For most models it is tiny; for kNN it grows with both the number of stored examples and the number of features.

Curse of dimensionality — The way distances lose their meaning as features are added, because every example ends up almost equally far from every other example. The phrase was coined by Richard Bellman in 1961.

WORKED EXAMPLE 11.7 — Count the Arithmetic in One kNN Prediction

A finished packing-line model stores $n = 50,000$ graded apples and uses $p = 12$ features. The line grades 20 apples per second. How many squared differences must the model compute per apple, and per second?

1. One distance needs one squared difference per feature, so $p = 12$ squared differences.
2. One prediction compares the new apple with every stored apple, so it needs $n = 50,000$ distances.
3. Squared differences for one prediction: $50,000 \times 12 = 600,000$.
4. Squared differences per second: $600,000 \times 20 = 12,000,000$.

Answer: Every apple costs 600,000 squared differences, and the line needs 12 million per second. A trained logistic regression from Chapter 10 would need only 12 multiplications per apple, because its training work is already finished.

Real software does not always search all n examples one by one: data structures such as the k-d tree, invented by Jon Bentley in 1975, and the ball tree find nearest neighbors far faster when there are few features. Their advantage fades as features are added, which is kNN's deepest weakness. Picture the line also measuring color, stem length, sugar content and a dozen more readings; any two apples differ a little in most of them, and those differences add up. Figure 11.9 drops 200 random points into a cube and measures the distance from a new point to its nearest and farthest neighbor as the cube gains dimensions.

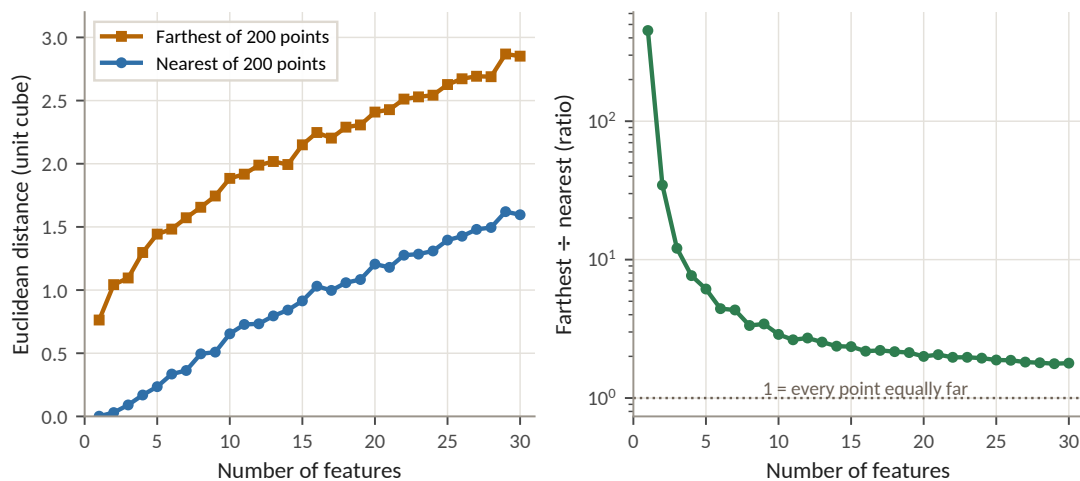


Figure 11.9 – Distances among 200 random points as the number of features grows. Left: the nearest and farthest distances. Right: how many times farther the farthest point is than the nearest.

WHAT TO OBSERVE

On the left both distances grow, but the nearest one grows much faster, so the gap closes. The right panel makes it plain: with one feature the farthest point is hundreds of times farther than the nearest,

but by 30 features it is less than twice as far. When every neighbor is about equally far, choosing the “ k nearest” is barely different from choosing k at random.

COMMON MISTAKE

✗ Wrong: Adding every available measurement to a kNN model, assuming more features can only help the distance calculation.

✓ Right: Check that each feature genuinely helps separate the classes before adding it. Extra, unhelpful features add noise to every distance and push truly similar examples apart. Chapter 22 is about choosing fewer, better features on purpose.

Table 11.6 sums up where kNN is a good choice and where it is not.

Table 11.6 – Where kNN is a good choice, and where it is not

Strengths	Weaknesses
No real training: a new labeled row can be used immediately	Every prediction costs one distance to every stored row
Assumes no shape, so the boundary can curve however the data does	Very sensitive to feature scale; always standardize in a pipeline
Easy to explain, and the same neighbors serve classification and regression	Useless features add noise; it copies training mistakes forever and cannot extrapolate

RESPONSIBLE USE

A kNN model can only repeat the patterns already in its training data, including its mistakes: if a past inspector graded a batch wrongly, kNN copies that mistake onto every similar apple. This matters far more when the examples are people. In a hospital or a school the model’s “nearest neighbors” are real past patients or students, and a prediction is only a summary of what happened to people who looked similar in a few recorded numbers. Use it to prompt a human review, never to close a door automatically. Chapter 24 returns to this.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing kNN has no real work to do at prediction time, just because it has almost no training step.
- Comparing raw distances between features on very different scales, or standardizing a test example with its own mean and standard deviation instead of the training set’s.
- Assuming a very small k , especially $k = 1$, is the most accurate choice, because it scores 100% on training data.
- Assuming a very large k is always safer, without noticing that it washes out local patterns and drifts toward the majority class.
- Choosing an even k in a two-class problem, and leaving ties to whatever the software does by default.
- Expecting distance weighting to rescue a badly chosen k , or to fix distances measured in unfair units.
- Adding every available feature, without checking whether each one helps separate the classes.
- Trusting a kNN regression prediction far outside the range of the training data, where it can only repeat the nearest edge value.

Practical Activities

ACTIVITY (PAPER) — Six Distances and a Vote

A cooperative near Halabja grades pomegranates as Sweet or Sour using two features that are already standardized, so no scaling is needed. The six graded fruits are P1 (0, 2) Sweet, P2 (3, 0) Sour, P3 (−3, −4) Sweet, P4 (6, 8) Sour, P5 (−1, 0) Sour and P6 (0, −6) Sweet. A new fruit Q sits at (0, 0).

1. Plot the six fruits on graph paper, one symbol for Sweet and another for Sour, and mark Q at the origin.
2. Compute the Euclidean distance from Q to each fruit, then list them sorted from smallest to largest with each label beside it. Every answer is a whole number; if one is not, check your arithmetic.
3. Predict Q's label by majority vote at $k = 1$, $k = 3$ and $k = 5$, and write down which values of k disagree.
4. Redo the $k = 5$ vote with distance weighting, giving each neighbor a weight of $1 \div d$ rounded to three decimals. Does the answer change? Explain why in one sentence.

You need: Graph paper, pencil, ruler and a calculator

ACTIVITY (PYTHON) — Reproduce the Hand Calculations with scikit-learn

This program repeats Worked Examples 11.3, 11.4 and 11.5 exactly. The mean and standard deviation come from the packing line's training file, not from these eight apples.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
X = np.array([[115, 8], [195, 7], [210, 9], [220, 8],          # Grade A apples
              [80, 3], [90, 2], [100, 4], [150, 3]])         # Grade B apples
y = np.array(["A", "A", "A", "A", "B", "B", "B", "B"])
mean, sd = np.array([170.0, 5.0]), np.array([40.0, 2.0])     # from the training file
q = np.array([[170.0, 7.0]])                                 # the new apple Q
print("raw features, k = 1 ->",
      KNeighborsClassifier(n_neighbors=1).fit(X, y).predict(q)[0])
Xs, qs = (X - mean) / sd, (q - mean) / sd                    # standardize both
model = KNeighborsClassifier(n_neighbors=3).fit(Xs, y)
dist, idx = model.kneighbors(qs, n_neighbors=3)
print("3 nearest (standardized) ->", y[idx[0]], np.round(dist[0], 3))
print("standardized, k = 3 ->", model.predict(qs)[0])
weight_only = X[:, [0]]                                     # the 112 g apple
for k, how in [(1, "uniform"), (3, "uniform"), (5, "uniform"), (3, "distance")]:
    m = KNeighborsClassifier(n_neighbors=k, weights=how).fit(weight_only, y)
    print("112 g apple, k = %d, %-8s ->" % (k, how), m.predict([[112]])[0])
```

Output

```
raw features, k = 1 -> B
3 nearest (standardized) -> ['A' 'A' 'A'] [0.625 1.346 1.414]
standardized, k = 3 -> A
112 g apple, k = 1, uniform -> A
112 g apple, k = 3, uniform -> B
112 g apple, k = 5, uniform -> B
112 g apple, k = 3, distance -> A
```

Every line matches a hand calculation: the three printed distances are the square roots of 0.390625, 1.8125 and 2.0 from Worked Example 11.3, and the last four lines repeat Worked Examples 11.4 and 11.5, including the flip back to Grade A once distance weighting is switched on.

You need: A computer with Python, NumPy and scikit-learn installed (see Chapter B)

ACTIVITY (SIMULATION) — Search for the Best k with Cross-Validation

This program tries eight values of k on the full 120-apple file and scores each with 5-fold cross-validation repeated five times, so no single lucky split can decide the winner.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, RepeatedStratifiedKFold

apples = pd.read_csv("datasets/ch11_apples.csv")
X = apples[["weight_g", "firmness"]].values
y = apples["grade"].values
print("rows %d | mean weight %.1f g | sd %.1f g" % (len(y), X[:, 0].mean(), X[:,
    0].std()))
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=0)
scores = {}
for k in range(1, 32, 4):
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    scores[k] = cross_val_score(model, X, y, cv=cv).mean()
    print("k = %2d    accuracy = %.3f" % (k, scores[k]))
best_k = max(scores, key=scores.get)
print("best k = %d with accuracy %.3f" % (best_k, scores[best_k]))
```

Output

```
rows 120 | mean weight 170.0 g | sd 40.0 g
k =  1    accuracy = 0.895
k =  5    accuracy = 0.922
k =  9    accuracy = 0.930
k = 13    accuracy = 0.918
k = 17    accuracy = 0.895
k = 21    accuracy = 0.877
k = 25    accuracy = 0.877
k = 29    accuracy = 0.888
best k = 9 with accuracy 0.930
```

The numbers match Table 11.4 and Figure 11.6. Note the `make_pipeline`: the scaler is refitted inside every training fold, so nothing from a validation fold leaks into the standardization. Set `weights="distance"` and run it again to see whether weighted voting helps here.

You need: A computer with Python, pandas and scikit-learn, and the file `datasets/ch11_apples.csv`

CASE STUDY — Grading Apples on a Packing Line Near Duhok

Lawand manages quality control at a fruit export company's packing facility near Duhok. Every day thousands of apples travel along a conveyor belt, and a small team of inspectors grades each one by hand, by weight and by feel. The team is skilled, but two inspectors do not always agree on a borderline apple, and hand grading slows the line down during the busiest weeks of the harvest.

The facility already owns a machine that records weight in grams and a firmness score for every apple. Lawand's team has 120 apples that inspectors graded by hand last season, and they train a kNN classifier on them, following this chapter's method step by step. They standardize both features first, remembering the trap of Lesson 11.2, because weight's much larger numbers would otherwise swamp firmness completely. They then run repeated cross-validation across many values of k , exactly as Figure 11.6 shows, and settle on $k = 9$, which scored 0.930 — clearly better than $k = 1$ at 0.895 and better than the oversmoothed $k = 21$ at 0.877.

The finished model runs on every apple as it crosses a sensor and suggests a grade before the apple reaches the boxes. Crucially, the team does not use the prediction alone; they also look at how the nine neighbors voted. An apple whose neighbors split five to four is sent to a human inspector, while a nine-to-zero vote goes straight into its box, keeping a trained eye on exactly the borderline cases.

Two rules complete the design. First, apple A1 — the light 115 g apple graded A for its color — taught the team that one unusual training example can pull nearby predictions toward its own label for reasons the two features cannot see, so they now note any apple graded on something the machine does not measure, and those notes say which feature is worth adding to the sensor next season. Second, no apple is graded by the machine alone in the first two weeks after a new variety arrives, because those apples have no near neighbors in the stored data at all — and a nearest neighbor that is not actually near is not evidence of anything.

CHAPTER SUMMARY

- k-Nearest Neighbors predicts a new example's label by finding the k closest training examples and letting them vote; it stores the data instead of fitting an equation, so almost all of its work happens at prediction time.
- Euclidean distance squares each feature's difference, adds the squares and takes the square root; Manhattan distance simply adds the sizes of the differences.
- Comparing squared distances gives the same ordering as true distances, so the square root can be skipped whenever only the ranking matters.
- Features must be standardized before distances are compared — raw units predicted Grade B and standardized units Grade A for the very same apple — using a mean and standard deviation taken from the training data only.
- A small k reacts to single unusual neighbors and overfits; a very large k washes out local patterns and drifts toward the majority class, which is underfitting.
- Training accuracy cannot choose k , because $k = 1$ always scores 100%; cross-validation chose $k = 9$ with 0.930 accuracy for the 120 apples.
- Distance-weighted voting counts closer neighbors more, using $1 \div d$, and can reverse a plain majority vote; prefer an odd k and fix a tie-breaking rule in advance.
- kNN regression averages the neighbors' target values, giving a staircase prediction that can never leave the range of the training targets.
- Each prediction costs one distance to every stored example, and too many features make every example almost equally far from every other — the curse of dimensionality.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain what the letter k means in k-Nearest Neighbors, and say who chooses it.

Problem 2 [Basic] Explain why kNN is described as having almost no training step, and say where its work happens instead.

Problem 3 [Basic] A two-class kNN model is set to $k = 6$. Explain in one sentence why this choice risks a tie, and give a better value.

Problem 4 [Basic] Class X has training weights 40 g and 60 g; Class Y has 100 g and 120 g. A new example weighs 55 g. Using weight only and $k = 1$, which class is it? Show the four distances.

Problem 5 [Basic] Name one strength and one weakness of kNN from Table 11.6, and give a one-sentence reason for each.

Problem 6 [Standard] Point P has features (12, 4) and point R has features (8, 1). Find both the Euclidean and the Manhattan distance between them, showing every step.

Problem 7 [Standard] A training point is (30, 300) and a new point is (35, 250). Without doing the arithmetic, explain why their raw distance would be misleading, and name the fix.

Problem 8 [Standard] Using a training mean of 50 and a training standard deviation of 10, standardize the values 70 and 35, showing every step. Then find the distance between the two standardized values.

Problem 9 [Standard] At $k = 5$, a new example's neighbors are labeled Yes, No, Yes, No and No, at distances 2, 3, 5, 6 and 9. (a) Find the plain majority vote. (b) Find the distance-weighted vote, rounding each weight to three decimals. (c) Do the two methods agree? Explain in one sentence.

Problem 10 [Standard] Three nearest neighbors of a new apartment have rents of 480, 520 and 560 thousand IQD. Find the kNN regression prediction at $k = 3$, then explain what would happen if the new apartment were twice the size of the largest one in the training file.

Problem 11 [Challenge] Using the eight apples of Table 11.1 and weight only, a new apple weighs 112 g. (a) Show that $k = 8$ produces an exact tie. (b) Apply each of the three tie-breaking rules from Lesson 11.4 and state the prediction each one gives. (c) Which rule would you choose for a packing line, and why?

Problem 12 [Challenge] A clinic wants to use kNN to flag patients needing a follow-up visit, using age in years, blood pressure in mmHg and number of past visits. (a) Why does standardizing matter especially here? (b) Give one risk of trusting the prediction without human review, and one way to reduce it. (c) Why must the clinic control who can see the stored model file?

Problem 13 [Challenge] Worked Example 11.7 counted 600,000 squared differences per prediction for $n = 50,000$ and $p = 12$. (a) Recompute the count if the training file doubles to 100,000 apples. (b) Recompute it if instead the number of features doubles to 24. (c) A colleague suggests keeping only the 5,000 most typical apples. Estimate the new cost and name one risk of throwing the other apples away.

Naive Bayes and Probabilistic Classifiers

Using probability rules to classify — fast, simple, and surprisingly strong for text

IN THIS CHAPTER YOU WILL LEARN TO

- Explain joint, conditional and independent probabilities from a set of counts.
- Apply Bayes' theorem by hand to turn a prior belief and new evidence into a posterior probability.
- State the “naive” assumption in plain words and explain why it makes a classifier possible from very little data.
- Classify a new example by hand with categorical naive Bayes, using counts from a 14-row dataset.
- Turn a short message into a bag-of-words vector and classify it with multinomial naive Bayes, using logarithms.
- Repair a zero count with Laplace (add-one) smoothing, and calculate a Gaussian likelihood for a numerical feature.
- Judge when naive Bayes, LDA or QDA is a good choice, and when the probabilities should not be trusted.

Introduction

A nurse at a busy clinic in Sulaymaniyah looks at the sky on a rainy Friday morning and says, “today will be quiet.” She has never written down a rule; she has simply seen many rainy weekend mornings, and most were quiet. Without any calculation she has combined three pieces of information — the weather, the day of the week, and the fact that it is not a holiday — into one judgment.

This chapter teaches a computer to do the same thing with arithmetic you can do on paper. The method is called *naive Bayes*: it counts how often each thing happened in the past, multiplies a few of those fractions together, and picks whichever answer comes out largest. It is one of the oldest classification methods in this book, one of the fastest, and — for problems made of words, such as sorting spam from real messages — still one of the best. We start with three ideas from probability, and from them comes Bayes' theorem, the engine of the whole chapter.

12.1 Probability Refresher: Joint, Conditional and Independent

A clinic records 200 visits over one month, noting for each whether the patient had a cough and whether flu was later diagnosed. Of the 200, 45 had both, 15 had flu without a cough, 35 had a cough without flu and 105 had neither — so 60 visits involved flu and 80 involved a cough. Figure 12.1 draws those four numbers as two overlapping groups.

WORDS TO KNOW

Joint probability — The probability that two events both happen, $P(A \text{ and } B)$: the count of cases where both happened, divided by the total number of cases. Like every probability it lies between 0 (never) and 1 (always).

Conditional probability — The probability that A happens *once B is already known*, written $P(A \mid B)$ and read “the probability of A given B ”. It changes the question from “out of everybody” to “out of this group”.

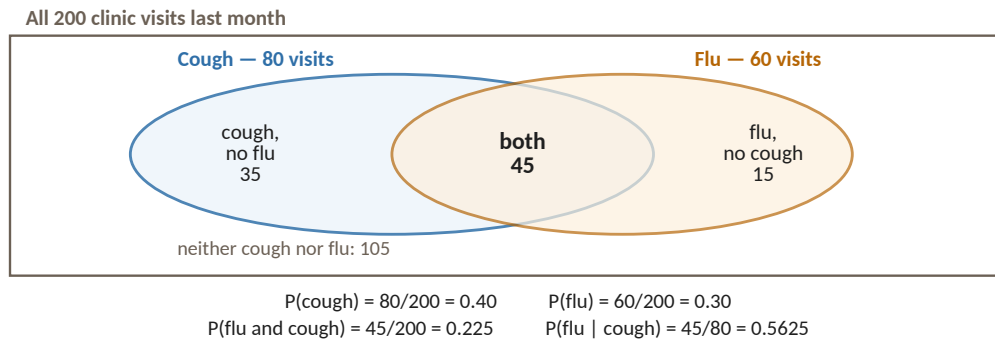


Figure 12.1 – The 200 clinic visits as two overlapping groups.

WHAT TO OBSERVE

The 45 in the middle is used in two fractions: divided by 200, the whole month, it gives the joint probability 0.225; divided by 80, only the coughing visits, it gives the conditional probability 0.5625. A conditional probability is a fraction with a shrunk bottom.

EQUATION: CONDITIONAL PROBABILITY

$$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)} = \frac{\text{count of } A \text{ and } B}{\text{count of } B}$$

In words: Count only the cases where B happened, then ask what fraction of those also had A .

When it is used: Whenever new information narrows the group you are looking at: a symptom is present, a test came back positive, a message contains a certain word.

Symbol	Meaning	Unit or type
$P(A \mid B)$	probability of A given that B happened	between 0 and 1
$P(A \text{ and } B),$ $P(B)$	probability that both happened, and that B happened at all	between 0 and 1

Chapter 6's precision is a conditional probability in disguise: it is $P(\text{really positive} \mid \text{flagged positive})$.

WORKED EXAMPLE 12.1 — Read Two Conditional Probabilities From the Counts

Using the counts in Figure 12.1, find the probability of flu given that a cough was recorded, and the probability of a cough given flu.

1. For $P(\text{flu} \mid \text{cough})$, keep only the cough column, which totals 80 visits. Of those 80, the number that also had flu is 45, so $45 \div 80 = 0.5625$.

2. For $P(\text{cough} \mid \text{flu})$, keep the flu row instead, which totals 60 visits. Of those 60, the number that also had a cough is 45, so $45 \div 60 = 0.75$.

Answer: $P(\text{flu} \mid \text{cough}) = 0.5625$ and $P(\text{cough} \mid \text{flu}) = 0.75$. The same 45 patients give two answers, because the questions divide by two different groups.

COMMON MISTAKE

X Wrong: Treating $P(A \mid B)$ and $P(B \mid A)$ as the same number — for example reading “95% of sick people test positive” as “95% of positive tests come from sick people”.

✓ Right: Check which group each fraction divides by. “Given a cough” divides by 80 visits; “given flu” divides by 60. Different bottom, different answer.

WORD TO KNOW

Independent events — Two events are independent when knowing that one happened does not change the probability of the other, so that $P(A \mid B) = P(A)$ and the joint probability is simply $P(A)$ times $P(B)$. Here $P(\text{flu}) \times P(\text{cough}) = 0.30 \times 0.40 = 0.12$, but the counted joint probability is 0.225, so a cough and flu are clearly *not* independent — which is exactly why a cough is useful evidence for flu.

12.2 Bayes’ Theorem in Plain Words

A pharmacy in Erbil offers a screening test for a rare condition, and a customer tests positive. How worried should the customer be? Most people answer “very”, because the test is 95% accurate. The correct answer is far less alarming, and it needs one equation — published in an essay by the English minister and mathematician Thomas Bayes, read to the Royal Society after his death by Richard Price in 1763, and rediscovered and generalized independently by Pierre-Simon Laplace in 1774. It takes what you believed before the evidence arrived, adjusts it by how strongly the evidence points one way, and returns an updated belief. Each part has a name.

WORDS TO KNOW

Prior probability — What you believe about a class before seeing the new example — usually just how common that class was in the training data.

Likelihood — How likely the evidence you saw would be if the class were true: a conditional probability read backwards, $P(\text{evidence} \mid \text{class})$.

Posterior probability — The updated belief after the evidence, $P(\text{class} \mid \text{evidence})$ — the number a classifier reports. The *evidence* itself is how likely the observation was overall, across every class that could have produced it; it makes the posteriors add to 1.

Base rate — How common a class is before any evidence is examined — the prior, expressed as a rate in the population. Ignoring a low base rate and judging a result only by a test’s accuracy is such a common error that it has its own name: the base-rate fallacy.

EQUATION: BAYES’ THEOREM

$$P(C \mid x) = \frac{P(x \mid C) P(C)}{P(x)}$$

In words: The updated probability of a class equals the likelihood of the evidence under that class, times the prior probability of the class, divided by how likely the evidence was overall.

When it is used: Whenever you know how often evidence appears *within* each class and need the reverse: how likely each class is, now that the evidence has appeared.

Symbol	Meaning	Unit or type
$P(C x)$, $P(x C)$	posterior: class C after seeing x ; likelihood: how usual x is inside C	between 0 and 1
$P(C)$, $P(x)$	prior of the class, and evidence across all classes	between 0 and 1

The evidence $P(x)$ is the same for every class, so when you only need to know which class wins you may skip it and compare the top halves alone.

Figure 12.2 turns the equation into a five-step recipe with the pharmacy's numbers filled in: the condition affects about 1 person in 100, and the test is positive for 95 of every 100 people who have it and for 5 of every 100 who do not.

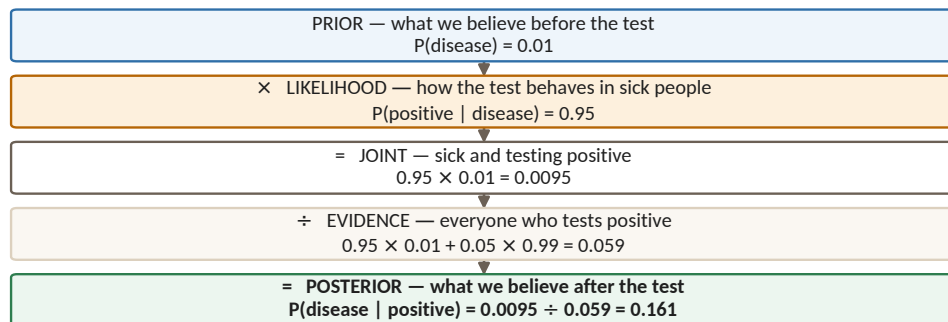


Figure 12.2 – Bayes' theorem as a five-step recipe, with the screening-test numbers. The prior never disappears — it multiplies everything after it.

WORKED EXAMPLE 12.2 — A Positive Result on a Test for a Rare Condition

A condition affects 1% of people. A screening test is positive for 95% of people who have it, and for 5% of people who do not. A customer tests positive. Find the probability that the customer really has the condition.

1. Write the prior: $P(\text{condition}) = 0.01$, so $P(\text{no condition}) = 0.99$.
2. Top half of Bayes' theorem for "has the condition": $0.95 \times 0.01 = 0.0095$; for "does not have it": $0.05 \times 0.99 = 0.0495$.
3. Add the two to get the evidence, the chance of a positive result from anyone: $0.0095 + 0.0495 = 0.059$, then divide: $0.0095 \div 0.059 = 0.161$.
4. Check it with whole people. Out of 10,000, the 100 who have the condition give $100 \times 0.95 = 95$ positive tests, and the 9,900 who do not give $9900 \times 0.05 = 495$ more. Of the $95 + 495 = 590$ positive tests, only 95 are true: $95 \div 590 = 0.161$.

Answer: About 0.161, or roughly 16%. Even after a positive result on a 95% accurate test, about 5 of every 6 people who test positive do not have the condition, because the healthy group is 99 times larger and its small false-alarm rate produces far more alarms. A rare-event screen is evidence, not a diagnosis: it should trigger a more specific second test and a conversation with a clinician, never an automatic conclusion. The same arithmetic governs fraud alerts, security warnings and machine-fault alarms.

12.3 The "Naive" Assumption and the Naive Bayes Classifier

Real examples carry several features at once. The clinic wants to predict whether tomorrow will be crowded from the weather, the day type and whether it is a public holiday. Bayes' theorem still applies, but the likeli-

hood becomes $P(x_1, x_2, x_3 \mid C)$ — the probability of that *whole combination* inside the class, and estimating that by counting needs training rows for every combination.

That is where the trouble starts. There are $3 \times 2 \times 2 = 12$ combinations, and in the 14-row dataset of the next lesson only 4 of them ever occur among the crowded days, so 8 of the counts would be zero and useless. Counting each feature on its own instead uses the same rows several times over, and every count then has real data behind it. Table 12.1 compares the two, and a fourth feature with 3 values would push the first approach to 72 numbers but the second only to 20.

Table 12.1 – Two ways to estimate the likelihood, for three features and two classes

Approach	Numbers per class	Numbers in total	Support from 14 training rows
Whole combinations	$3 \times 2 \times 2 = 12$	24	$14 \div 24 = 0.58$ rows each; most cells empty
Each feature on its own	$3 + 2 + 2 = 7$	14	a whole column of data behind every one

WORDS TO KNOW

Conditional independence (the naive assumption) — The assumption that *within a single class* the features carry no information about each other, which lets the probability of a whole combination be written as the separate feature probabilities multiplied together.

Naive Bayes classifier — A classifier that estimates each class prior and each feature likelihood by counting, multiplies them under the naive assumption, and predicts whichever class gives the largest product.

The word “naive” is honest: weather and day type are surely related, so the assumption is almost never true. What matters is whether breaking it changes which class wins, and often it does not — a point Pedro Domingos and Michael Pazzani explained in 1997, and David Hand and Keming Yu underlined in their 2001 study “Idiot’s Bayes — not so stupid after all”.

EQUATION: THE NAIVE BAYES DECISION RULE

$$\hat{y} = \arg \max_c P(c) \prod_{j=1}^d P(x_j \mid c)$$

In words: For each class, multiply how common that class is by the likelihood of every feature value of the new example, and predict whichever class gives the biggest answer.

When it is used: To classify a new example once the class priors and the per-feature likelihoods have been counted from training data.

Symbol	Meaning	Unit or type
$\hat{y}, \arg \max_c$	the predicted class: the c making what follows largest	a class label
$P(c)$	prior probability of class c	between 0 and 1
$\prod_{j=1}^d P(x_j \mid c)$	the d feature likelihoods multiplied together	between 0 and 1

The evidence $P(x)$ is dropped because it is identical for every class. Divide each product by the sum of all the products to turn the scores back into probabilities.

12.4 Categorical Naive Bayes by Hand

Now we build a complete classifier with a pencil. Table 12.2 is the whole training set — 14 days of weather, day type, holiday and the crowding that followed.

Table 12.2 – The clinic dataset: 14 days, days 1–7 left and 8–14 right (illustrative example data)

D	Weather	Day type	Holiday	Crowded?	D	Weather	Day type	Holiday	Crowded?
1	Sunny	Weekday	No	Yes	8	Cloudy	Weekday	No	Yes
2	Sunny	Weekday	No	Yes	9	Rainy	Weekday	No	Yes
3	Sunny	Weekend	No	No	10	Rainy	Weekday	No	Yes
4	Sunny	Weekday	Yes	No	11	Rainy	Weekend	No	No
5	Cloudy	Weekday	No	Yes	12	Rainy	Weekend	Yes	No
6	Cloudy	Weekend	No	Yes	13	Rainy	Weekday	Yes	No
7	Cloudy	Weekday	Yes	Yes	14	Sunny	Weekday	No	Yes

Training is pure counting. First the classes: 9 of the 14 days were crowded and 5 were not, giving priors $9/14$ and $5/14$. Then, feature by feature, count how often each value appears inside each class (Figure 12.3).

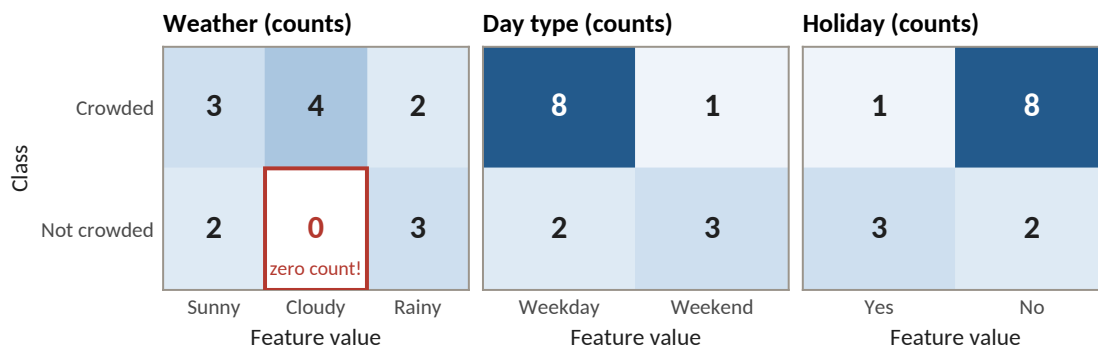


Figure 12.3 – Every count the classifier needs: training days with that feature value, in that class.

WHAT TO OBSERVE

In the middle panel, 8 of the 9 crowded days were weekdays but only 2 of the 5 quiet days were — that is what makes “weekday” useful evidence. The red cell is a warning: no cloudy day was ever quiet, and Lesson 12.5 shows why a zero count is dangerous.

Dividing each count by its class total turns counts into likelihoods. Table 12.3 is everything the classifier has learned — fourteen numbers, and nothing else. The 14 rows can now be thrown away, which is why naive Bayes trains in one pass and stays fast on millions of examples, unlike Chapter 11’s method, which must keep every row.

Table 12.3 – The trained model: counts and likelihoods

Feature value	Count, crowded	$P(\text{value} \mid \text{crowded})$	Count, not crowded	$P(\text{value} \mid \text{not})$
Weather = Sunny	3	$3/9 = 0.333$	2	$2/5 = 0.400$
Weather = Cloudy	4	$4/9 = 0.444$	0	$0/5 = 0.000$
Weather = Rainy	2	$2/9 = 0.222$	3	$3/5 = 0.600$
Day type = Weekday	8	$8/9 = 0.889$	2	$2/5 = 0.400$
Day type = Weekend	1	$1/9 = 0.111$	3	$3/5 = 0.600$
Holiday = Yes	1	$1/9 = 0.111$	3	$3/5 = 0.600$
Holiday = No	8	$8/9 = 0.889$	2	$2/5 = 0.400$

WORKED EXAMPLE 12.3 — Classify a New Day With Categorical Naive Bayes

Tomorrow is forecast Sunny, it is a Weekend day, and it is not a public holiday. Using the priors and Table 12.3, predict whether the clinic will be crowded.

1. Write the priors: $P(\text{crowded}) = 9 \div 14 = 0.643$ and $P(\text{not crowded}) = 5 \div 14 = 0.357$.
2. Take the three “crowded” likelihoods, 0.333, 0.111 and 0.889, and multiply with the prior: $0.643 \times 0.333 = 0.214$; $0.214 \times 0.111 = 0.0238$; $0.0238 \times 0.889 = 0.0212$.
3. Take the three “not crowded” likelihoods, 0.400, 0.600 and 0.400: $0.357 \times 0.400 = 0.143$; $0.143 \times 0.600 = 0.0857$; $0.0857 \times 0.400 = 0.0343$.
4. Compare: 0.0343 beats 0.0212, so “not crowded” wins. Divide each score by their total, 0.0555: $0.0343 \div 0.0555 = 0.618$ and $0.0212 \div 0.0555 = 0.382$.

Answer: The model predicts not crowded, with probability about 0.618 — roughly a 62% chance of a quiet day against 38% for a busy one. That is a prediction worth acting on cautiously, not a certainty.

The two scores are the top halves of Bayes’ theorem, and dividing by their sum performs the division by the evidence that the rule left out. Always do that when a person will read the result: “62% chance of a quiet day” tells a clinic manager far more than the bare word “quiet”.

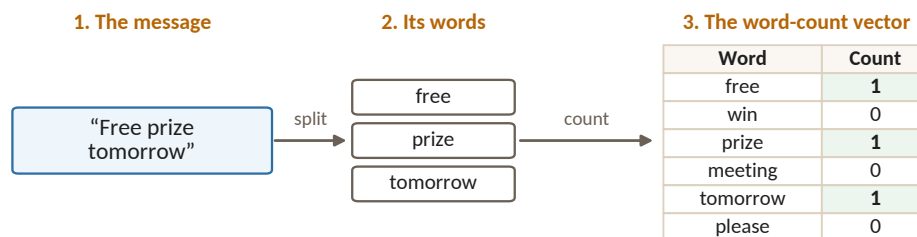
12.5 Text Classification: Bag of Words, Multinomial Naive Bayes and Smoothing

Naive Bayes truly shines on text: keeping spam out of an inbox, or routing referral letters to the right hospital department. A piece of writing is not a row of numbers, so the first job is to make it one.

WORD TO KNOW

Bag of words — A way of turning a document into numbers: choose a fixed list of words, the *vocabulary*, and count how many times each appears. Word order is discarded — hence “bag” rather than “sentence”.

Figure 12.4 follows one message through this conversion with a tiny six-word vocabulary, so everything can be checked by hand. A real filter uses thousands of words and its vectors are almost entirely zeros, which is exactly why counting words separately works so well here.



Word order is thrown away: “tomorrow prize free” gives exactly the same vector.

Figure 12.4 – Turning a message into a word-count vector.

The operator has hand-labeled ten short messages. The four marked spam are “Free prize! Win now”, “Win a free phone today”, “Claim your free prize” and “Free credit, win a big prize”. The six marked *ham* — the standard nickname for a message that is not spam — are “Meeting tomorrow at nine”, “Please send the report tomorrow”, “The meeting room is free tomorrow”, “Please join the meeting”, “Tomorrow the meeting starts early, please” and “Please confirm tomorrow”. Four of the ten are spam, so the priors are $P(\text{spam}) = 0.4$ and $P(\text{ham}) = 0.6$.

For text we do not ask “what fraction of spam messages contain this word” but “what fraction of all the word slots inside spam messages are filled by this word”. That is *multinomial* naive Bayes, the standard choice for word counts. Keeping only the six vocabulary words, the four spam messages hold 10 word slots and the six ham messages 14.

And at once there is a problem: “meeting” never appears in a spam message, so its raw likelihood inside spam is $0 \div 10 = 0$, and in a rule that multiplies, one zero destroys everything — any message containing “meeting” would score exactly zero for spam. The repair is old and simple: pretend you saw every word one extra time.

WORD TO KNOW

Laplace smoothing (add-one smoothing) — Adding a small constant, usually 1, to every count before dividing, and adding the matching amount to the denominator so the probabilities still add to 1. It removes zeros and pulls every estimate slightly toward “all values equally likely”. Pierre-Simon Laplace introduced it in 1814 as the rule of succession.

EQUATION: LAPLACE (ADD-ONE) SMOOTHING

$$P(x_j = v \mid c) = \frac{n_{v,c} + \alpha}{n_c + \alpha K_j}$$

In words: Add α to the count of this value in this class, add α once per possible value to the class total, then divide.

When it is used: Always, when a naive Bayes model is trained by counting, and especially with a small training set or a large vocabulary, where zero counts are certain.

Symbol	Meaning	Unit or type
$n_{v,c}, n_c$	how often value v was seen inside class c , and the class total	count
K_j, α	number of possible values of feature j ; smoothing strength, usually 1	count; 0 or more

Adding αK_j to the bottom keeps one feature’s likelihoods adding to exactly 1; setting $\alpha = 0$ gives the raw counts back.

The clinic data of Lesson 12.4 has the same disease: no cloudy day in the training set was ever quiet.

WORKED EXAMPLE 12.4 — Repair a Zero Count With Add-One Smoothing

The clinic forecast for the day after tomorrow is Cloudy, a Weekday, and a public holiday. Classify it with the raw counts of Table 12.3, then again with add-one smoothing ($\alpha = 1$).

1. Raw score for “crowded”: $0.643 \times 0.444 \times 0.889 \times 0.111 = 0.0282$.
2. Raw score for “not crowded”: $P(\text{Cloudy} \mid \text{not crowded}) = 0$, so the whole product $0.357 \times 0 \times 0.400 \times 0.600$ is 0, and the posterior becomes $0.0282 \div 0.0282 = 1.000$. The model claims complete certainty on the strength of five quiet days.
3. Smooth the weather counts for “not crowded”. With $K = 3$ weather values and $n = 5$ days, every count gains 1 and the bottom becomes $5 + 3 = 8$: $P(\text{Cloudy} \mid \text{not crowded}) = 1 \div 8 = 0.125$.
4. Smooth the other two features the same way, using $K = 2$: $(2 + 1) \div 7 = 0.4286$ and $(3 + 1) \div 7 = 0.5714$. Smoothed score: $0.357 \times 0.125 \times 0.4286 \times 0.5714 = 0.0109$.
5. Smooth the “crowded” class too: $(4 + 1) \div 12 = 0.4167$, $(8 + 1) \div 11 = 0.8182$, $(1 + 1) \div 11 = 0.1818$, giving $0.643 \times 0.4167 \times 0.8182 \times 0.1818 = 0.0398$, and $0.0398 \div (0.0398 + 0.0109) = 0.785$.

Answer: Both versions predict crowded, but the raw version claims probability 1.000 and the smoothed version reports 0.785. The prediction did not change; the false certainty did.

Applying the same repair to the ten messages, with $K = 6$ vocabulary words, gives Table 12.4; Figure 12.5 draws those likelihoods as bars.

Table 12.4 – Word counts and smoothed likelihoods ($\alpha = 1$); the denominators are $10 + 6 = 16$ and $14 + 6 = 20$

Word	Count in spam	$P(\text{word} \mid \text{spam})$	Count in ham	$P(\text{word} \mid \text{ham})$
free	4	$5/16 = 0.3125$	1	$2/20 = 0.10$
win	3	$4/16 = 0.25$	0	$1/20 = 0.05$
prize	3	$4/16 = 0.25$	0	$1/20 = 0.05$
meeting	0	$1/16 = 0.0625$	4	$5/20 = 0.25$
tomorrow	0	$1/16 = 0.0625$	5	$6/20 = 0.30$
please	0	$1/16 = 0.0625$	4	$5/20 = 0.25$

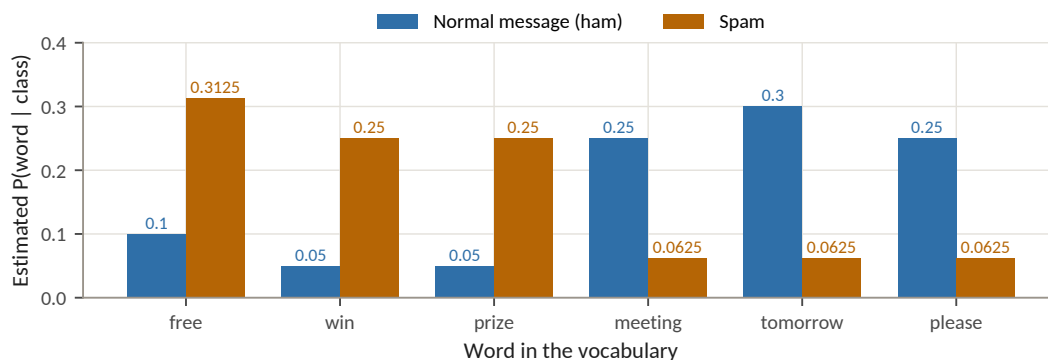


Figure 12.5 – Smoothed word likelihoods for the two classes.

WHAT TO OBSERVE

“free”, “win” and “prize” are four to five times likelier inside spam, and “meeting”, “tomorrow” and “please” four to five times likelier inside ham. No bar is zero, so no single word decides anything alone — but three or four multiplied together give a clear answer.

A three-word message can be scored by plain multiplication, but a 200-word message would multiply 200 likelihoods, most well below 0.01. Their product would be far smaller than the smallest positive number ordinary computer arithmetic can store, so it would round to zero for every class, leaving nothing to compare. The cure is logarithms, which turn multiplication into addition.

WORD TO KNOW

Log-probability — The natural logarithm of a probability. Every probability is at most 1, so every log-probability is 0 or negative, and a bigger (less negative) value means a more likely outcome. Adding log-probabilities is exactly the same as multiplying the probabilities.

EQUATION: THE NAIVE BAYES RULE IN LOGARITHMS

$$\text{score}(c) = \ln P(c) + \sum_{j=1}^d \ln P(x_j | c)$$

In words: Take the logarithm of the prior and of each feature likelihood, add them all, and predict whichever class has the largest (least negative) total.

When it is used: In every real implementation, whenever more than a handful of features are multiplied, so that tiny products never round down to zero.

Symbol	Meaning	Unit or type
$\text{score}(c)$	log-score of class c ; larger means more likely	0 or negative
$P(c), P(x_j c)$	the class prior and the feature likelihoods	between 0 and 1

Log-scores order the classes exactly as the products do, so the prediction never changes. The difference between two log-scores, put through $e^{\text{difference}}$, says how many times more likely one class is than the other.

WORKED EXAMPLE 12.5 — Classify One Message, Two Ways

A new message arrives: “Free prize tomorrow”. Its vocabulary words are free, prize and tomorrow, each once. Using the priors and Table 12.4, decide whether it is spam — first by multiplying, then by adding logarithms.

1. Spam, by multiplying: $0.4 \times 0.3125 = 0.125$; $0.125 \times 0.25 = 0.03125$; $0.03125 \times 0.0625 = 0.001953$.
2. Ham, by multiplying: $0.6 \times 0.10 = 0.06$; $0.06 \times 0.05 = 0.003$; $0.003 \times 0.30 = 0.0009$.
3. Compare: 0.001953 beats 0.0009, and $0.001953 \div (0.001953 + 0.0009) = 0.685$.
4. Spam, by logarithms: $\ln 0.4 = -0.916$, $\ln 0.3125 = -1.163$, $\ln 0.25 = -1.386$, $\ln 0.0625 = -2.773$; adding gives -6.238 .
5. Ham, by logarithms: $\ln 0.6 = -0.511$, $\ln 0.10 = -2.303$, $\ln 0.05 = -2.996$, $\ln 0.30 = -1.204$; adding gives -7.013 .
6. Compare: -6.238 beats -7.013 , so spam wins again; the difference is 0.775 and $e^{0.775} = 2.17$.

Answer: The message is spam, with probability about 0.685, and spam is 2.17 times likelier than ham — matching $0.685 \div 0.315 = 2.17$. Both routes agree, but only the logarithms would survive a 200-word

message. The result is far from certain, so a filter should send this message to a review folder rather than delete it.

COMMON MISTAKE

X Wrong: Multiplying hundreds of small likelihoods directly in code, then wondering why every class ends up with a score of exactly 0.0 and the classifier always picks the first class.

✓ Right: Add log-probabilities instead. Multiplying 0.0625 by itself 300 times gives a number so small that ordinary computer arithmetic stores it as exactly zero; adding $\ln 0.0625$ to itself 300 times gives a comfortable -832 .

Figure 12.6 shows both sides of smoothing: what it repairs, and what it costs.

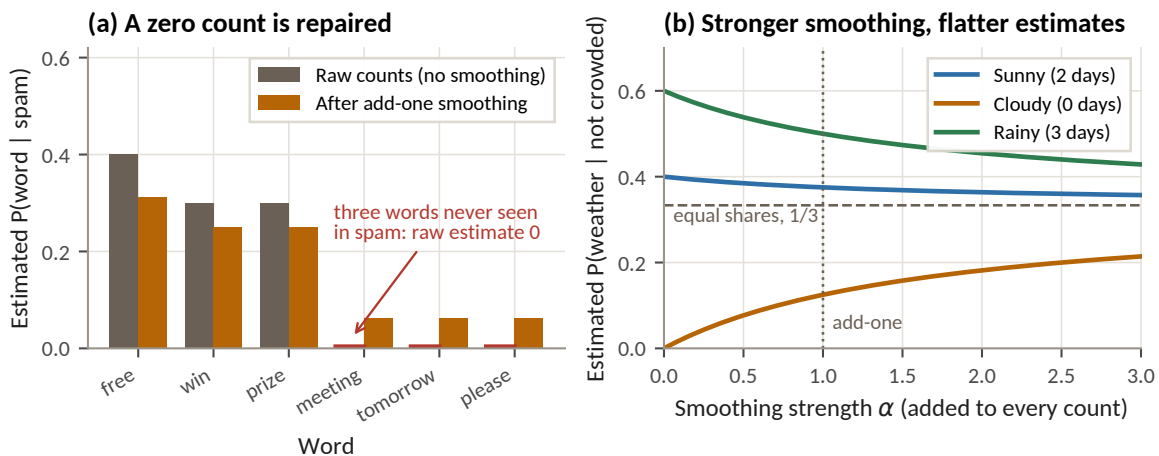


Figure 12.6 – Add-one smoothing lifts zero counts off the floor (left) and drags every estimate toward equal shares (right).

WHAT TO OBSERVE

On the left, three impossible zeros become a small but usable 0.0625, while frequent words give up only a little. On the right is the price: at $\alpha = 1$ “Cloudy” has risen from 0 to 0.125 but “Rainy” has fallen from 0.60 to 0.50, and larger α sends all three toward $1/3$ — what you would say with no data at all.

12.6 Gaussian Naive Bayes for Numerical Features

Counting works for a feature with a few named values, but fails for weight in grams, where almost every measurement is unique and every count would be 1 or 0. A packing house near Shaqlawa sorts apples into export Grade A and local-market Grade B by weight alone; instead of counting, we describe each class with a smooth curve.

WORD TO KNOW

Gaussian naive Bayes — A version of naive Bayes for numerical features: for each feature and class it stores just two numbers, the mean and the standard deviation, and uses the normal (bell-shaped) curve as the likelihood.

Five Grade A apples were weighed at 155, 165, 170, 175 and 185 g, and five Grade B apples at 131, 137, 140, 143 and 149 g. Their means are 170 g and 140 g and their standard deviations 10 g and 6 g — both summaries

come from Chapter 3, and here those four numbers are the entire trained model. The normal curve gives a *density* rather than a probability, but densities work in the naive Bayes rule the same way, because only the comparison between classes matters.

EQUATION: GAUSSIAN (NORMAL) LIKELIHOOD

$$f(x | c) = \frac{1}{\sigma_c \sqrt{2\pi}} \exp\left(-\frac{(x - \mu_c)^2}{2\sigma_c^2}\right)$$

In words: Measure how far the value sits from the class mean, square it, divide by twice the class variance, and use that as a negative power of e ; then scale by one over the standard deviation times $\sqrt{2\pi}$.

When it is used: As the likelihood of a numerical feature inside one class, when that feature's values form a roughly single-peaked, symmetric cloud.

Symbol	Meaning	Unit or type
$f(x c), x$	likelihood (density) of the new example's value x inside class c	density; feature unit
μ_c, σ_c	mean and standard deviation of that feature inside class c	unit of the feature

A density can be larger than 1, so it is not a probability by itself; divide each class score by the sum of all class scores as before. Before using this equation, check that the feature really is single-peaked inside each class — if it is strongly skewed, transform it (Chapter 22) or bin it and use categorical naive Bayes.

WORKED EXAMPLE 12.6 — Weigh One Apple and Decide Its Grade

A new apple weighs 158 g. Using the ten training apples above and equal priors of 0.5, find the likelihood under each grade and predict the grade.

1. Check Grade A: the weights total 850, so the mean is 170; the squared distances from 170 are 225, 25, 0, 25, 225, totalling 500, so the variance is $500 \div 5 = 100$ and the standard deviation is 10. For Grade B the weights total 700, the mean is 140, the squared distances total 180, and the standard deviation is $\sqrt{180 \div 5} = 6$.
2. Grade A: $(158 - 170)^2 = 144$ and $144 \div (2 \times 100) = 0.72$, so $e^{-0.72} = 0.4868$; the scaling factor is $1 \div (10\sqrt{2\pi}) = 0.03989$, so $f = 0.03989 \times 0.4868 = 0.01942$.
3. Grade B: $(158 - 140)^2 = 324$ and $324 \div 72 = 4.5$, so $e^{-4.5} = 0.01111$; the scaling factor is $1 \div (6\sqrt{2\pi}) = 0.06649$, so $f = 0.06649 \times 0.01111 = 0.00074$.
4. Multiply each by the prior 0.5, giving 0.00971 and 0.00037, then divide: $0.00971 \div (0.00971 + 0.00037) = 0.963$.

Answer: The apple is Grade A, with probability about 0.963. It sits 1.2 standard deviations from Grade A's mean but a full 3.0 from Grade B's — the spread matters as much as the distance.

Figure 12.7 draws both curves and marks the weight where the model changes its mind.

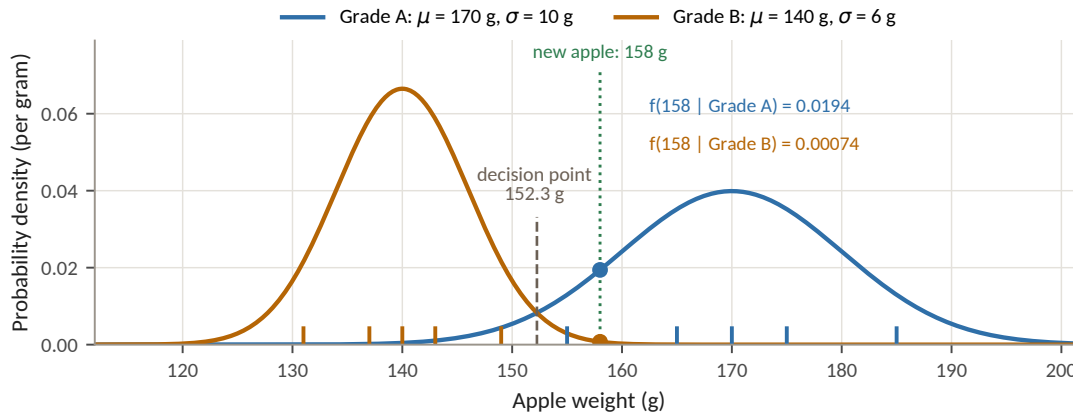


Figure 12.7 – The two class curves for apple weight; ticks mark the ten training apples.

WHAT TO OBSERVE

Grade B's curve is taller and narrower because its apples are packed into a smaller range. That is why the crossing point sits at 152.3 g rather than halfway between the means at 155 g: a tight class becomes unbelievable quickly away from its center, so it gives up the disputed ground first.

12.7 Related Probabilistic Classifiers: LDA and QDA

Gaussian naive Bayes assumes that, inside one class, the features vary independently: heavier apples are not systematically wider. That is rarely true. Two older methods keep the bell-shaped class model but drop that assumption, and they were the first statistical classifiers ever published — Ronald A. Fisher introduced the linear version in 1936.

WORDS TO KNOW

Linear discriminant analysis (LDA) — A classifier that models each class as a bell-shaped cloud and forces every class to share one common shape and spread. Its boundary is always a straight line (or a flat plane).

Quadratic discriminant analysis (QDA) — The same idea, but each class is allowed its own shape and spread, so the boundary it produces is curved.

The shape of a cloud of points in several dimensions is described by a small table of numbers called a *covariance matrix*: the spread of each feature down the diagonal, and how strongly each pair moves together off it. The equation below is the multi-feature bell curve — you never compute it by hand, and what matters is only what each method allows it to contain.

EQUATION: THE GAUSSIAN CLASS MODEL BEHIND LDA AND QDA

$$f(\mathbf{x} | c) = \frac{1}{(2\pi)^{d/2} |\Sigma_c|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \Sigma_c^{-1}(\mathbf{x} - \boldsymbol{\mu}_c)\right)$$

In words: Each class is a bell-shaped cloud with its own center and shape. An example is scored by how far it lies from that center, measured in units stretched by the shape of the cloud.

When it is used: As the class likelihood in LDA and QDA, when the features are numerical and vary together rather than separately.

Symbol	Meaning	Unit or type
$\mathbf{x}, \mu_c$	the new example's d features, and the center of class c	feature units
$\Sigma_c, \Sigma_c $	covariance matrix of class c , and its determinant	squared feature units

The three methods differ only in what Σ_c may be. QDA: a separate full matrix per class. LDA: one shared matrix. Gaussian naive Bayes: one per class, but with the off-diagonal entries forced to zero. With a single feature Σ_c is just σ_c^2 , and this becomes the bell curve of Lesson 12.6.

WORKED EXAMPLE 12.7 — Where LDA Puts the Boundary for the Apples

Fit LDA to the same ten apples by pooling the two classes into one shared variance, and find the weight where it switches from Grade B to Grade A. Compare with the Gaussian naive Bayes boundary of 152.3 g.

1. Take each class variance from Worked Example 12.6: 100 for Grade A and 36 for Grade B.
2. Pool them, weighting by class size: $(5 \times 100 + 5 \times 36) \div 10 = 68$, so the shared standard deviation is $\sqrt{68} = 8.25$ g and both classes must use it.
3. With identical curves and equal priors the densities are equal where the distances to the two means are equal — the midpoint, $(170 + 140) \div 2 = 155$ g.
4. Compare with the separate-spread boundary: $155 - 152.3 = 2.7$ g of difference.

Answer: LDA puts the boundary at exactly 155 g. One shared spread moved it 2.7 g away from the tighter class, because LDA may no longer notice that Grade B is more tightly packed.

With two features the difference becomes a shape: Figure 12.8 fits all three methods to the same 80 apples, measured for both weight and diameter.

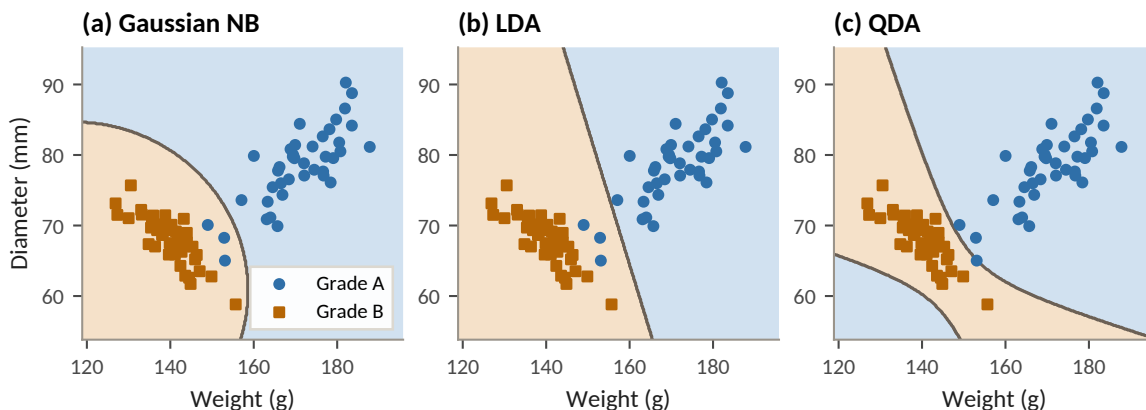


Figure 12.8 – The same 80 apples, three classifiers. Shading shows the predicted grade.

WHAT TO OBSERVE

LDA's boundary is perfectly straight, because one shared shape leaves no room for anything else. Gaussian naive Bayes curves, but only along the axes, because it may not notice that heavier apples are also wider. QDA bends with each cloud and separates these grades best, but it has the most numbers to estimate and so overfits a small sample first (Chapter 7).

12.8 Strengths, Weaknesses and When to Use Naive Bayes

Naive Bayes has an unusual profile. Training is one pass of counting, so it is often the fastest model available, and it handles tens of thousands of features without complaint. It needs very little data, because each number comes from a whole column rather than a single cell: Andrew Ng and Michael Jordan showed in 2001 that naive Bayes typically beats logistic regression (Chapter 10) when examples are scarce, though logistic regression overtakes it once data is plentiful. Its model is a small table anyone can read, and new data means updating counts rather than retraining.

The weaknesses come from the same source. Because it multiplies likelihoods as if the features were separate, correlated features get counted twice or ten times over: a filter treating “free”, “FREE” and “free!!!” as three words piles up three helpings of the same evidence, pushing the winning score far too close to 1. This is why naive Bayes probabilities are famously over-confident — the ranking is often good, but the numbers are poorly calibrated in exactly the sense Chapter 6 defined.

COMMON MISTAKE

X Wrong: Reporting a naive Bayes posterior of 0.99 to a decision-maker as though it meant “99 times out of 100 this will be right”.

✓ Right: Use naive Bayes probabilities to *rank* and sort, not as calibrated risks. If a real probability is needed, check the calibration curve first and recalibrate, as Chapter 6 describes.

Table 12.5 matches each method to the data in front of you. A rule of thumb: for any classification problem built out of text, start with multinomial naive Bayes on the day the data arrives — it costs minutes and gives a real accuracy number for heavier models to beat (Chapter 23).

Table 12.5 – Choosing among the probabilistic classifiers in this chapter

Method	Features it expects	Typical use, and what it assumes
Categorical naive Bayes	A few named values per feature	Survey answers, weather categories, day types
Multinomial naive Bayes	Counts, such as how often a word occurs	Text; the standard first model for spam filtering and topic sorting
Bernoulli naive Bayes	Yes/no flags: does the word occur at all?	Short texts, where presence matters more than count
Gaussian naive Bayes	Numbers with a single-peaked, symmetric spread	Measurements such as weight or temperature
LDA and QDA	Numbers that vary together	LDA shares one shape across classes (straight boundary); QDA gives each class its own (curved boundary, more data needed)

COMMON MISTAKES AND MISCONCEPTIONS

- Swapping a conditional probability for its reverse — reading $P(\text{positive} \mid \text{sick})$ as if it were $P(\text{sick} \mid \text{positive})$.
- Judging a test only by its accuracy and ignoring the base rate, so a rare condition seems far likelier than it is after a positive result.
- Rejecting naive Bayes because the features are obviously related: the assumption only has to be harmless to the ranking, not true.
- Leaving zero counts in the model, so one unseen word or category drives an entire class score to exactly zero.

- Multiplying many small likelihoods instead of adding their logarithms, and losing everything to underflow.
- Reading a posterior of 0.99 as a calibrated risk when correlated features have inflated it, or using Gaussian naive Bayes on a strongly skewed feature without looking at its shape.

Practical Activities

ACTIVITY (PAPER) — Train and Use a Naive Bayes Classifier by Hand

1. Copy Table 12.2 onto paper and cover Table 12.3. Count the 14 rows and rebuild all three count tables; your “crowded” counts must add to 9 for each feature and your “not crowded” counts to 5.
2. Divide each count by its class total to rebuild the likelihood table, and check that one feature’s likelihoods add to 1 within each class.
3. Classify Day A (Rainy, Weekday, no holiday) and Day B (Sunny, Weekend, holiday). For each, multiply the prior by three likelihoods for both classes, compare, and convert to probabilities. One answer should be far more confident — write one sentence saying which features made the difference.
4. Redo Day A with add-one smoothing, using $K = 3$ for weather and $K = 2$ for the others. Does the prediction change? Does the confidence?

You need: Paper, pen and a calculator

ACTIVITY (PYTHON) — Reproduce Every Worked Example With scikit-learn

The first program reproduces Worked Examples 12.3 and 12.4, once with almost no smoothing and once with add-one smoothing.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder

df = pd.read_csv("datasets/ch12_clinic_days.csv")
cols = ["weather", "day_type", "holiday"]
enc = OrdinalEncoder().fit(df[cols])
X, y = enc.transform(df[cols]), (df["crowded"] == "Yes").to_numpy()

Z = enc.transform(pd.DataFrame([["Sunny", "Weekend", "No"],      # Worked Example 12.3
                               ["Cloudy", "Weekday", "Yes"]],    # Worked Example 12.4
                      columns=cols))
for alpha, name in [(1e-10, "almost none"), (1.0, "add-one")]:
    model = CategoricalNB(alpha=alpha).fit(X, y)
    print(name, "smoothing: P(crowded) =", model.predict_proba(Z)[: , 1].round(3))
```

Output

```
almost none smoothing: P(crowded) = [0.382 1.    ]
add-one      smoothing: P(crowded) = [0.493 0.785]
```

OrdinalEncoder turns each word into a number, and alpha is the smoothing strength α . Without smoothing the model prints 0.382 for the first day and a suspicious 1.000 for the second — the two

answers found by hand. With add-one smoothing the second drops to the calmer 0.785 and the first slides to 0.493, so close to a coin flip that “not crowded” wins by almost nothing: fourteen rows never justified strong confidence. The second program repeats Worked Examples 12.5 and 12.6 with scikit-learn’s own naive Bayes classes.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, GaussianNB

msgs = ["Free prize! Win now", "Win a free phone today", "Claim your free prize",
        "Free credit, win a big prize", "Meeting tomorrow at nine",
        "Please send the report tomorrow", "The meeting room is free tomorrow",
        "Please join the meeting", "Tomorrow the meeting starts early, please",
        "Please confirm tomorrow"]
spam = np.array([1, 1, 1, 1, 0, 0, 0, 0, 0, 0]) # 1 = spam, 0 = ham
vec = CountVectorizer(vocabulary=["free", "win", "prize", "meeting", "tomorrow", "
    please"])
nb = MultinomialNB(alpha=1.0).fit(vec.transform(msgs), spam)
print("P(word | ham), then P(word | spam), in vocabulary order:")
print(np.exp(nb.feature_log_prob_).round(4))
print("P(spam | 'Free prize tomorrow') =",
      nb.predict_proba(vec.transform(["Free prize tomorrow"]))[0, 1].round(4))

grams = np.array([155, 165, 170, 175, 185, 131, 137, 140, 143, 149]).reshape(-1, 1)
grade = np.array(["A"] * 5 + ["B"] * 5)
apples = GaussianNB().fit(grams, grade)
print("means", apples.theta_.ravel(), "sds", np.sqrt(apples.var_).ravel().round(2))
print("P(Grade A | 158 g) =", apples.predict_proba([[158]])[0, 0].round(4))
```

Output

```
P(word | ham), then P(word | spam), in vocabulary order:
[[0.1    0.05  0.05  0.25  0.3    0.25 ]
 [0.3125 0.25  0.25  0.0625 0.0625 0.0625]]
P(spam | 'Free prize tomorrow') = 0.6846
means [170. 140.] sds [10.  6.]
P(Grade A | 158 g) = 0.9634
```

CountVectorizer does the bag-of-words step of Figure 12.4, and a fixed vocabulary keeps the six words of Table 12.4 and ignores the rest. The printed likelihoods match that table row for row, and 0.6846 matches the 0.685 found by hand. GaussianNB then recovers the means 170 and 140 and the standard deviations 10 and 6, and agrees on 0.9634 for the 158 g apple.

You need: Python, NumPy, pandas and scikit-learn, and the file datasets/ch12_clinic_days.csv

CASE STUDY — Filtering SMS Spam for a Mobile Operator — Kurdistan Region, Iraq

A mobile operator’s customers are being flooded with prize-draw and fake-loan text messages, and the engineering team has one week to act. They choose multinomial naive Bayes because it trains in seconds, fits on one page for the regulator, and can be updated nightly with newly labeled messages. All figures below are illustrative for a project of this kind, not measurements from a real network.

The team hand-labels 20,000 messages that customers had already reported or approved, of which about 2,400 are spam, giving a prior of $2400 \div 20000 = 0.12$. They lowercase everything, strip punctuation and keep the 8,000 most frequent words as the vocabulary, so every message becomes a bag-of-words vector like Figure 12.4, 8,000 entries long instead of six. Smoothing is not optional at this scale: thousands of ordinary words never appear in a spam message, and without it any message containing one of them would score exactly zero for spam and slip through. The team also works in logarithms, because an average message contributes 15 to 20 likelihoods.

The filter catches most spam on a held-out test month, but it is far better at *ranking* messages than at reporting honest probabilities: words such as “prize”, “winner” and “claim” travel together and each counts as fresh evidence, so many messages come back above 0.99 that deserve much less confidence. The team therefore ignores the raw probability and tunes a *threshold* on the score, as Chapter 6 recommends, so that fewer than one genuine message in a thousand is blocked: above the threshold a message goes to a spam folder the customer can open, in the middle band it is delivered but flagged, and nothing is deleted. A weekly sample of blocked messages goes to human review, because spammers change their wording constantly and a filter trained on last year’s words quietly stops working — the drift problem Chapter 24 returns to.

RESPONSIBLE USE

A message filter reads private messages, so build it with care: train only on messages customers have reported or consented to share, keep no more text than the model needs, and never let a score delete a message. Naive Bayes is also used to score people — for risk, eligibility or suspicion — and its over-confident probabilities make that especially dangerous: a 0.99 from a model with correlated features is not a fact about a person. Chapter 24 covers fairness, privacy and consent in detail.

CHAPTER SUMMARY

- A conditional probability $P(A \mid B)$ counts only the cases where B happened; it is not $P(B \mid A)$, and confusing the two is the commonest probability mistake there is.
- Bayes’ theorem turns a prior and a likelihood into a posterior, $P(C \mid x) = P(x \mid C)P(C)/P(x)$. When a class is rare most alarms are false alarms, so the base rate matters as much as the test’s accuracy.
- Naive Bayes assumes the features are independent inside each class, replacing one impossible joint table with a few small, well-supported ones. Its rule multiplies the class prior by one likelihood per feature and picks the largest product; dividing by the total of the products gives probabilities.
- Bag of words turns a document into word counts and throws the order away; multinomial naive Bayes is the standard classifier for them.
- Laplace (add-one) smoothing adds α to every count and αK to every total, removing zeros at the cost of pulling estimates toward equal shares; log-probabilities turn the long product into a sum and prevent underflow.
- Gaussian naive Bayes replaces counting with a bell curve per feature per class, keeping only a mean and a standard deviation; LDA shares one shape for a straight boundary, QDA gives each class its own for a curved one.
- Naive Bayes is fast, needs little data and excels on text, but correlated features make its probabilities over-confident — use it for ranking and as a baseline.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher’s Guide.

Problem 1 [Basic] Explain what the prior, the likelihood and the posterior each mean, using Worked Example 12.2.

Problem 2 [Basic] From Figure 12.1, find $P(\text{no flu} \mid \text{cough})$ and $P(\text{cough} \mid \text{no flu})$, showing which total you divided by each time.

Problem 3 [Basic] Why is naive Bayes called “naive”? State the assumption in one sentence and name two features that clearly break it.

Problem 4 [Basic] A vocabulary word never appears in any spam training message. What likelihood do raw counts give it, what happens to any message containing it, and what is the repair?

Problem 5 [Standard] A condition affects 2% of people; its test is positive for 90% of those who have it and 10% of those who do not. Find $P(\text{condition} \mid \text{positive})$.

Problem 6 [Standard] Using the priors and Table 12.3 without smoothing, classify a Rainy Weekday that is not a holiday. Give both scores and the final probability.

Problem 7 [Standard] Repeat Worked Example 12.4 with $\alpha = 2$. Report the new posterior probability of “crowded” and say whether stronger smoothing made the model more or less confident.

Problem 8 [Standard] Using Table 12.4 and the priors 0.4 and 0.6, classify “win free meeting” by multiplication and again by logarithms. Check that both give the same winner and the same ratio.

Problem 9 [Standard] An apple weighs 150 g. Using the means and standard deviations of Lesson 12.6 with equal priors, find each grade’s likelihood and the winner’s posterior.

Problem 10 [Challenge] A filter’s vocabulary holds “free”, “FREE” and “free!!!” as three words, and spam using one usually uses all three. (a) What does the naive assumption claim, and why is it wrong? (b) What happens to the spam posterior? (c) Give two fixes.

Problem 11 [Challenge] The test of Worked Example 12.2 is used where 20% of patients really have the condition. (a) Recompute the posterior after a positive result. (b) Why does the same test give a different answer? (c) What does that say about moving a classifier to a new population?

Problem 12 [Challenge] Design a naive Bayes classifier that routes messages at your school or workplace to the right office. State the classes, the vocabulary, the smoothing, the performance measure, and one privacy risk and its remedy.

CHAPTER THIRTEEN

Decision Trees

A model that asks one question at a time

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how a decision tree predicts by asking a short sequence of yes-or-no questions.
- Name the parts of a tree — root, internal node, branch, leaf, depth — and trace an example to its leaf.
- Calculate the Gini impurity and the entropy of a node by hand.
- Compare candidate splits using weighted impurity, and compute the information gain of the winner.
- Describe how a tree is grown one node at a time, and list the rules that stop it growing.
- Explain why trees overfit, and control it with maximum depth, minimum leaf size and pruning.
- Predict a number with a regression tree, and compute a feature's importance from a finished tree.

Introduction

Imagine phoning a repair hotline in Sulaymaniyah about a machine that has stopped working. A good technician never asks everything at once. She asks one question, listens, and lets your answer decide her next question. “Is the power light on?” If no, she asks about the cable. If yes, she asks something else entirely. Each answer throws away part of the problem, until only one sensible next step is left.

A decision tree predicts in exactly this way, and it is one of the friendliest models in this book because you can read a finished tree out loud. Where linear regression in Chapter 8 hands you a list of coefficients, a tree hands you a short list of questions — and a busy technician can actually remember questions. This chapter builds a small tree entirely by hand: how a tree measures how “mixed” a group of examples is, how it chooses the best question at each step, and why a tree left to grow freely memorizes its training data and then fails on new data. The Python activity at the end reproduces every number with scikit-learn.

13.1 A Tree Asks One Question at a Time

A factory in Erbil's industrial zone assembles small electric motors. Over three hundred machines sit on its floor, and the maintenance team can inspect only a handful closely each week. Every machine carries a sensor reporting its running temperature in degrees Celsius and a vibration level the maintenance sheet records simply as High or Low; the team also knows each machine's age. For ten machines it knows the answer that matters — did the machine turn out to need inspection soon afterward? Table 13.1 is that record.

Table 13.1 – Ten machines from the Erbil motor factory — three features and one label

Machine	Temperature (°C)	Vibration level	Age (years)	Needs inspection?
M1	95	High	9	Yes
M2	90	Low	3	Yes
M3	88	Low	7	Yes
M4	86	Low	2	Yes
M5	84	Low	5	Yes
M6	70	High	8	Yes
M7	78	Low	4	No
M8	76	Low	10	No
M9	72	Low	6	No
M10	62	Low	1	No

WORD TO KNOW

Decision tree — A model that predicts by asking a sequence of simple yes-or-no questions about an example's features, one at a time, until it reaches a final answer — exactly like a technician narrowing down a fault.

Six machines needed inspection and four did not, and the team wants a rule that turns the first three columns into the fourth. Every decision tree has the same shape, borrowed from a real tree turned upside down; Figure 13.1 shows one already trained on Table 13.1.

WORDS TO KNOW

Root node — The starting point of a tree, where the first question is asked, using every training example.

Internal node — Any node below the root that still asks a question and still splits its group in two.

Leaf node — An ending point that asks no further question and simply gives a prediction.

Branch — One of the two paths leaving a node, taken according to the answer to that node's question.

Depth — How many questions an example has answered on reaching a node. The root is depth 0; the deepest leaf gives the tree's depth.

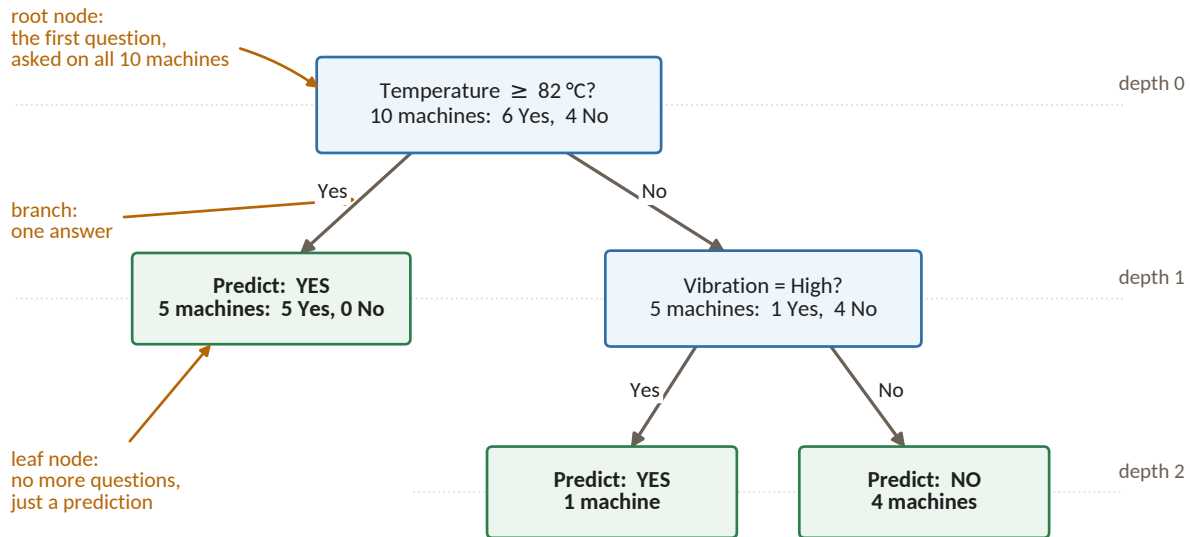


Figure 13.1 – A trained decision tree for the ten machines of Table 13.1. Blue boxes ask questions, green boxes give predictions, and every box shows how many machines reach it.

WHAT TO OBSERVE

The counts shrink as you move down: 10 machines, then 5 and 5, then 1 and 4. A leaf's counts show how much its prediction rests on — the left leaf holds 5 machines that all needed inspection, while the bottom-left leaf rests on a single machine.

WORKED EXAMPLE 13.1 — Follow Two Machines from the Root Node to a Leaf

Use the tree of Figure 13.1. Machine X runs at 78 °C with High vibration; Machine Y runs at 91 °C with Low vibration. Predict both.

1. Machine X, root question: is $78 \geq 82$? No, so take the “no” branch.
2. Machine X now meets the second question: is the vibration level High? Yes, so take the “yes” branch.
3. That branch ends in a leaf predicting “needs inspection: Yes”. Machine X answered two questions.
4. Machine Y, root question: is $91 \geq 82$? Yes, so take the “yes” branch.
5. That branch ends immediately in a leaf predicting “needs inspection: Yes”. Machine Y answered one question, and the tree never asked about its vibration at all.

Answer: Both machines are predicted to need inspection, but Machine X needed two questions and Machine Y only one. A tree asks only as many questions as it needs.

COMMON MISTAKE

✗ Wrong: Assuming every example must pass through the same number of questions on its way to a prediction.

✓ Right: An example's journey stops the moment it reaches a leaf. Two examples can take paths of very different lengths through the very same tree.

Because every question compares one feature with one threshold, the picture a tree draws is made only of rectangles with horizontal and vertical edges, as Figure 13.2 shows. The computer stores the vibration level as a number, 1 for High and 0 for Low, so “vibration = High?” is really “vibration ≥ 0.5 ?”.

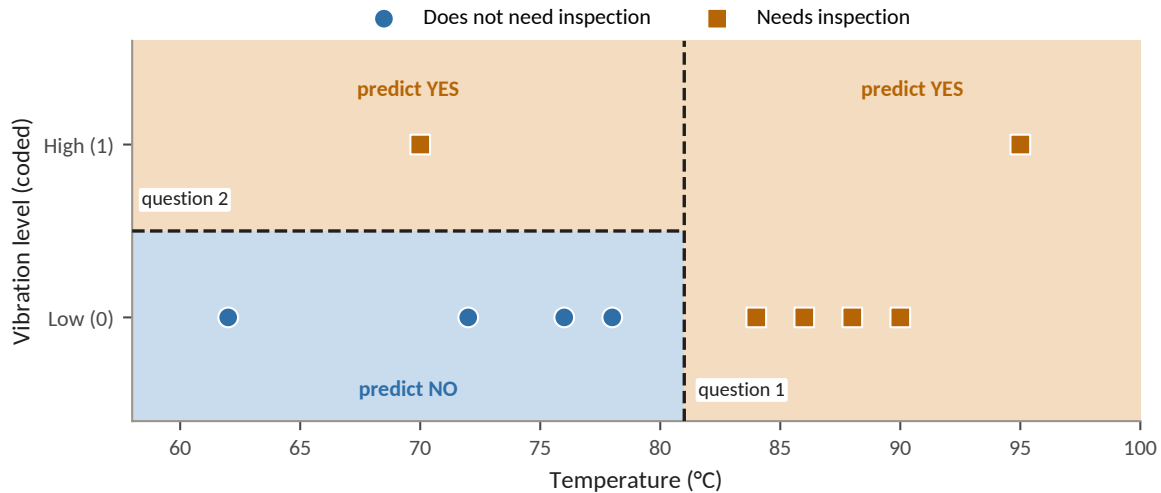


Figure 13.2 – The tree of Figure 13.1 drawn as a map of the two features it uses. Shaded areas show what the tree predicts anywhere on the map; the ten machines are drawn on top.

WHAT TO OBSERVE

The two dashed lines are the tree's two questions, cutting the map into three rectangles — never a diagonal, never a curve. The small orange rectangle on the left is the piece question 2 rescued, holding the one cool machine, M6, that still needed inspection.

13.2 Measuring Purity: Gini Impurity and Entropy

The tiny tree above was simply handed to you. Where did its questions come from — why temperature before vibration, and why 82 °C? Think of a shopkeeper in Duhok sorting a sack in which lentils and rice have been mixed. A good sorting step leaves one pile almost all lentils and the other almost all rice; a useless step leaves both piles as mixed as the sack. To compare candidate questions, a tree needs a number that says how mixed a group is.

WORDS TO KNOW

Impurity — A number measuring how mixed the labels are inside one node. One label only means zero impurity; an even mix means the highest impurity.

Gini impurity — The most common impurity measure in decision trees. For each label present in the node, square that label's share of the node, add the squares, and subtract the total from 1.

Gini impurity has a pleasant meaning: it is the chance you would label an example wrongly if you guessed at random using the node's own proportions.

EQUATION: GINI IMPURITY OF ONE NODE

$$G = 1 - \sum_{i=1}^k p_i^2$$

In words: Take each label's share of the node, square it, add all the squares together, and subtract the total from one.

When it is used: To score a node before splitting it, and to score the two children a candidate split would create.

Symbol	Meaning	Unit or type
G	Gini impurity of the node	0 (pure) to 0.5 for two classes
p_i	share of the node's examples carrying label i	a proportion between 0 and 1
k	number of different labels present	count

With two classes the largest possible value is 0.5, reached when the node is split exactly evenly; with k equally common classes it is $1 - 1/k$.

WORKED EXAMPLE 13.2 — Find the Root Node's Gini Impurity

The root node of Figure 13.1 holds all ten machines: 6 labeled Yes and 4 labeled No. Find its Gini impurity.

1. Share labeled Yes: $6 \div 10 = 0.6$. Share labeled No: $4 \div 10 = 0.4$.
2. Square each share: $0.6^2 = 0.36$ and $0.4^2 = 0.16$.
3. Add the squares: $0.36 + 0.16 = 0.52$.
4. Subtract from 1: $1 - 0.52 = 0.48$.

Answer: The root node's Gini impurity is $G = 0.48$, close to the two-class maximum of 0.5 — fair enough, since the ten machines are almost evenly divided.

Table 13.2 repeats those steps for the three nodes used later, so you can watch the number fall as nodes become one-sided.

Table 13.2 – Gini impurity, step by step, for three nodes of Figure 13.1

Node	Yes, No	p_{Yes}	p_{No}	Sum of squares	Gini
Root (all 10)	6, 4	0.6	0.4	0.52	0.48
Hot node (82 °C or more)	5, 0	1.0	0.0	1.00	0.00
Cool node (under 82 °C)	1, 4	0.2	0.8	0.68	0.32

Gini impurity is not the only choice. A second measure comes from information theory, where Claude Shannon introduced it in 1948 to measure the surprise a message carries.

WORD TO KNOW

Entropy — An impurity measure from information theory: 0 for a pure node and, for two classes, exactly 1 bit for an even mix. It answers the question: how many yes-or-no answers would you need, on average, to learn the label of an example drawn from this node?

EQUATION: ENTROPY OF ONE NODE

$$H = - \sum_{i=1}^k p_i \log_2 p_i$$

In words: For each label, multiply its share by the base-2 logarithm of that share, add the results, and change the sign of the total.

When it is used: As an alternative to Gini impurity. Trees built with entropy are usually very close to trees built with Gini impurity.

Symbol	Meaning	Unit or type
H	entropy of the node	bits, from 0 (pure) to $\log_2 k$
p_i	share of the node's examples carrying label i	a proportion between 0 and 1
$\log_2$	logarithm to base 2	no unit

A share of 0 contributes nothing, and every $\log_2 p_i$ is negative for $p_i < 1$, which is why the minus sign in front makes H positive.

Figure 13.3 compares the two measures before we put entropy to work.

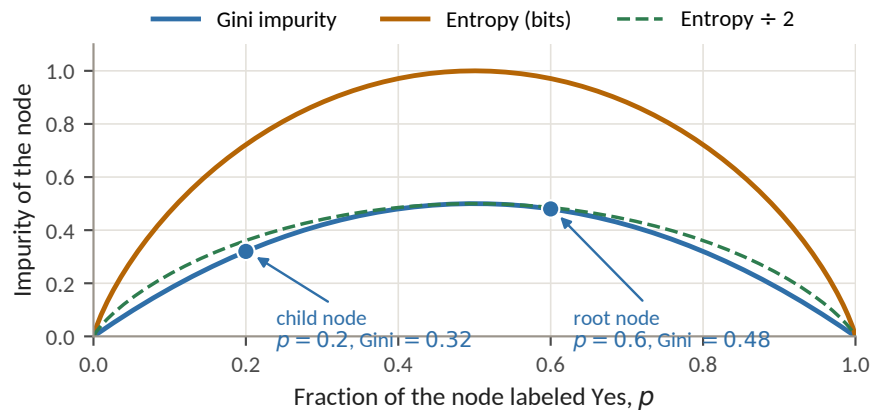


Figure 13.3 – Gini impurity and entropy for a node with two classes, as the share p of one class changes.

WHAT TO OBSERVE

Both curves peak at $p = 0.5$ and fall to zero at both ends: a node is purest when everything in it carries one label. Entropy climbs to 1.0 and Gini only to 0.5, but the dashed curve — entropy divided by two — lies almost on top of the Gini curve, so the two measures rank splits in nearly the same order.

WORKED EXAMPLE 13.3 — Find the Root Node's Entropy

Find the entropy of the same root node: 6 machines labeled Yes and 4 labeled No.

1. The shares are again $p_{\text{Yes}} = 0.6$ and $p_{\text{No}} = 0.4$.
2. Look up the logarithms: $\log_2 0.6 = -0.737$ and $\log_2 0.4 = -1.322$.
3. Multiply: $0.6 \times (-0.737) = -0.442$ and $0.4 \times (-1.322) = -0.529$.
4. Add, then change the sign: $-0.442 - 0.529 = -0.971$, so $H = 0.971$.

Answer: The root node's entropy is 0.971 bits, just under the two-class maximum of 1 bit. Learning whether one of these machines needs inspection is worth almost one full yes-or-no answer.

13.3 Choosing the Best Split

To choose a question, a tree tries many candidates and keeps the one whose two children are purest.

WORDS TO KNOW

Split — One question at a node, made of a feature and a threshold, that sends every example in the node down one of two branches.

Weighted impurity — The impurity of a split's two children combined into one number, each child counting in proportion to how many examples landed in it.

Information gain — How much impurity a split removes: the parent's impurity minus the split's weighted impurity. Larger is better.

Weighting matters: a pure child holding two examples has done far less work than a pure child holding fifty, so each child's impurity is multiplied by its share of the parent before the two are added.

EQUATION: WEIGHTED IMPURITY AND INFORMATION GAIN OF A SPLIT

$$I_{\text{split}} = \frac{n_L}{n} I_L + \frac{n_R}{n} I_R, \quad \text{Gain} = I_{\text{parent}} - I_{\text{split}}$$

In words: Multiply each child's impurity by the fraction of the parent's examples that landed in it and add the two results; the gain is what is left when that total is taken away from the parent's own impurity.

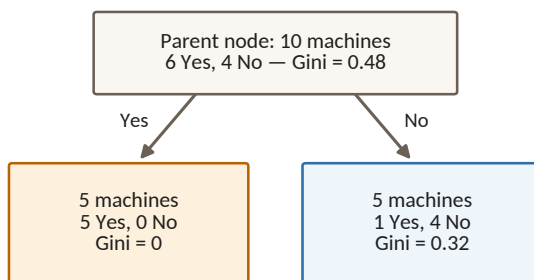
When it is used: To score and compare candidate splits at one node. The tree keeps the candidate with the lowest weighted impurity, which is always the one with the largest gain.

Symbol	Meaning	Unit or type
I_{split} , Gain	weighted impurity of the split, and the impurity it removes	same scale as I
I_{parent} , I_L , I_R	impurity of the parent and of the two children	Gini or entropy
n_L , n_R , n	examples in each child and in the parent, $n = n_L + n_R$	count

I can be Gini impurity or entropy — just use the same measure throughout; with Gini the gain is often called the impurity decrease. Lesson 13.6 puts mean squared error in the same place.

Figure 13.4 sets out two candidate questions for the ten machines, and the worked example below computes both scores by hand.

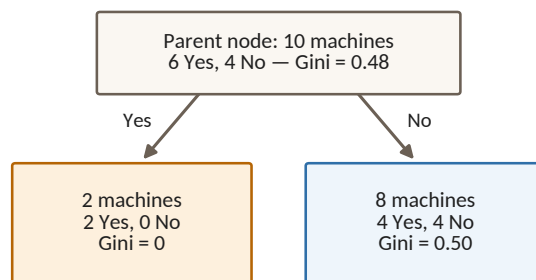
Split A: temperature $\geq 82^\circ\text{C}$?



Weighted Gini = 0.16

lower is better — Split A wins

Split B: vibration = High?



Weighted Gini = 0.40

leaves a large, half-mixed child

Figure 13.4 — The same ten machines split two different ways. Each child box shows its counts and its Gini impurity; the box at the bottom shows the weighted score that decides the contest.

WHAT TO OBSERVE

Both splits produce one perfectly pure child, so looking only at the best child would call the contest a tie. Weighting breaks it: Split B's pure child carries only 2 of the 10 machines, so it earns a weight of just $2 \div 10$, while its large, half-mixed child drags the score up to 0.40.

WORKED EXAMPLE 13.4 — Compare Two Candidate Splits Using Weighted Gini Impurity

Using the ten machines of Table 13.1, score Split A, “temperature $\geq 82^\circ\text{C}$ ”, against Split B, “vibration = High?”.

1. Sort by Split A. The machines at 95, 90, 88, 86 and 84°C go to the “yes” child: 5 machines, all Yes. The other five — 78, 76, 72, 70 and 62°C — go to the “no” child: 1 Yes (M6, at 70°C) and 4 No.
2. Gini of Split A's “yes” child: one label only, so $1 - (1^2 + 0^2) = 0$.
3. Gini of Split A's “no” child: shares $1 \div 5 = 0.2$ and $4 \div 5 = 0.8$, so $1 - (0.04 + 0.64) = 0.32$.
4. Combine: $(5 \div 10) \times 0 + (5 \div 10) \times 0.32 = 0.16$.
5. Sort by Split B. Only M1 and M6 have High vibration, and both are Yes: Gini = 0. The other eight are 4 Yes and 4 No, so their Gini is $1 - (0.25 + 0.25) = 0.50$.
6. Combine: $(2 \div 10) \times 0 + (8 \div 10) \times 0.50 = 0.40$.
7. Compare the two scores: 0.16 against 0.40.

Answer: Split A wins, because 0.16 is lower than 0.40. Split B's pure child is real but tiny, and it leaves eight machines behind in a perfectly mixed group — which is why the tree of Figure 13.1 asks about temperature first.

COMMON MISTAKE

X Wrong: Comparing two children's impurity scores directly, without weighting each one by how many examples landed in that child.

✓ Right: Always multiply each child's impurity by its share of the parent before adding. A small pure child must not count as much as a large pure child.

Subtracting the split's score from the parent's, as the second half of the box does, turns “lower is better” into “bigger is better”. Both rankings agree, because the parent's impurity is the same for every candidate at that node.

WORKED EXAMPLE 13.5 — Find the Information Gain of the Winning Split

Find the gain of Split A, measured first with Gini impurity and then with entropy.

1. With Gini: the parent scored 0.48 in Worked Example 13.2 and the split scored 0.16 in Worked Example 13.4, so $\text{Gain} = 0.48 - 0.16 = 0.32$.
2. With entropy: the parent scored 0.971 bits in Worked Example 13.3.
3. The “yes” child is pure, so its entropy is 0. The “no” child has shares 0.2 and 0.8, so $H = -(0.2 \times (-2.322) + 0.8 \times (-0.322)) = 0.464 + 0.258 = 0.722$ bits.
4. Weighted entropy of the split: $(5 \div 10) \times 0 + (5 \div 10) \times 0.722 = 0.361$ bits.
5. Subtract: $\text{Gain} = 0.971 - 0.361 = 0.610$ bits.

Answer: Split A removes 0.32 of Gini impurity, or 0.610 bits of entropy. The two numbers sit on different scales and cannot be compared with each other, but each ranks Split A first.

A real tree tries every feature, and for a numerical feature every useful threshold — the midpoints between neighboring values once that feature is sorted. Table 13.3 scores three candidates at the root.

Table 13.3 – Three candidate root splits for the ten machines, scored with Gini impurity

Candidate question	“Yes” child (Yes, No)	“No” child (Yes, No)	Weighted Gini	Gini gain
Temperature ≥ 82 °C?	5, 0	1, 4	0.16	0.32
Vibration = High?	2, 0	4, 4	0.40	0.08
Age ≥ 6 years?	3, 2	3, 2	0.48	0.00

The age row is the most instructive: it makes two groups of five mixed in exactly the same proportion as the parent, so its weighted Gini equals the parent’s 0.48 and its gain is exactly zero. Asking it would teach the tree nothing. This procedure — score every candidate, keep the lowest weighted impurity — is the heart of CART, the Classification and Regression Trees method published by Leo Breiman, Jerome Friedman, Richard Olshen and Charles Stone in 1984, and still what scikit-learn uses. The entropy version appeared in Ross Quinlan’s ID3 algorithm in 1986 and his C4.5 in 1993.

13.4 Growing the Whole Tree

One split gives a tree of depth 1. To grow further, the tree repeats the same search inside each child, as if that child were a brand-new, smaller problem.

WORD TO KNOW

Greedy algorithm — An algorithm that takes the best-looking step available right now, without checking whether a slightly worse step now would allow much better steps later. Tree growing is greedy: it picks the best split at each node in turn and never goes back to reconsider an earlier one.

Apply this to the cool node of Figure 13.1, the five machines below 82 °C, whose Gini impurity is 0.32. Splitting on “vibration = High?” sends M6 — the one machine there that needed inspection — into a child of its own and the remaining four, all No, into the other. Both children are pure, so the weighted impurity is $(1 \div 5) \times 0 + (4 \div 5) \times 0 = 0$ and the gain is the whole 0.32. Nothing can beat that, so the tree keeps it and every machine in Table 13.1 now lands in a pure leaf.

Growing must stop somewhere. Figure 13.5 shows the full loop with the three usual reasons to stop: the node is pure; it holds too few examples to split, or a split would leave a child that is too small; or it has reached the maximum depth a person allowed. The last two are hyperparameters in the sense of Chapter 7.

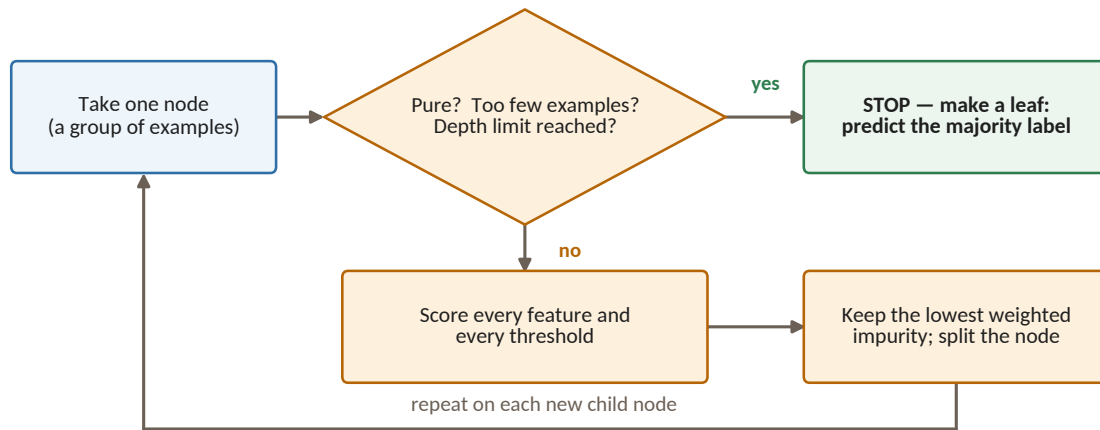


Figure 13.5 – How a tree is grown. The loop runs on the root first and then on every child it creates, so the two halves of a tree can reach different depths and use different features.

One practical detail explains a number in the Python activity. Searching thresholds for temperature, scikit-learn tries midpoints, and the best one here falls between 78 and 84, so it reports “temperature ≤ 81 ” rather than “temperature ≥ 82 ”. For whole-degree readings these are the same rule: the tree simply parks its cut in the empty gap between the two nearest machines.

COMMON MISTAKE

✗ Wrong: Believing that a tree searches for the best possible tree overall.

✓ Right: Tree growing is greedy and looks only one split ahead. A different first question might lead to a smaller tree, but finding the best tree overall is far too slow to compute, so every practical algorithm settles for the best next split.

13.5 Trees Overfit Easily: Depth, Leaf Size and Pruning

Nothing stops a tree from splitting again and again. Taken to the extreme, it keeps going until every leaf holds a single training example, every leaf is pure, and training accuracy is exactly 100%. Such a tree has memorized the noise of its training data instead of learning a pattern: overfitting (Chapter 7) in its plainest form, and trees fall into it faster than almost any other model in this book. To see it happen, the Erbil team pulled sensor records for 280 machines from the whole factory floor, this time keeping the raw vibration reading in millimeters per second, then trained on 160 machines and kept 120 as a validation set (Chapter 5).

Table 13.4 – Two ways to limit a tree, measured on the same 280 machines

Setting	Leaves	Training accuracy (%)	Validation accuracy (%)
Maximum depth 1	2	80.6	75.8
Maximum depth 3	8	92.5	88.3
Maximum depth 5	20	97.5	80.0
Maximum depth 7 (no limit)	24	100.0	80.0
Minimum leaf size 5	15	92.5	88.3
Minimum leaf size 10	10	88.8	85.0
Minimum leaf size 40	3	80.6	75.8

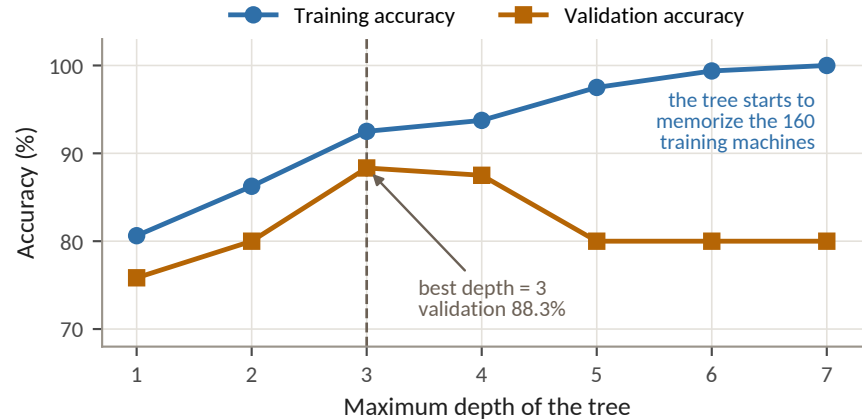


Figure 13.6 – Training and validation accuracy of the same trees as the maximum depth is raised, at every depth from 1 to 7.

WHAT TO OBSERVE

The training curve only ever rises, reaching a perfect 100% at depth 7. The validation curve rises to 88.3% at depth 3, then falls away and flattens at 80.0%. The widening space between the two is the generalization gap of Chapter 7, drawn by a decision tree.

WORKED EXAMPLE 13.6 – Compare Three Depths and Choose One

From the top half of Table 13.4, compare depth 1, depth 3 and depth 7. Find the training-validation gap at each and choose the best setting.

1. Gap at depth 1: $80.6 - 75.8 = 4.8$ percentage points.
2. Gap at depth 3: $92.5 - 88.3 = 4.2$ percentage points.
3. Gap at depth 7: $100.0 - 80.0 = 20.0$ percentage points.
4. Compare validation accuracies, because validation performance — not the size of the gap — is what the choice rests on: 75.8%, 88.3% and 80.0%.
5. The largest is 88.3%, at depth 3.

Answer: Choose depth 3. Depth 7 reaches a perfect 100% on training machines yet scores 20 points lower on machines it has never seen, the classic overfitting signature. Depth 1's small gap looks reassuring, but both of its scores are simply low — that is underfitting, not success.

Maximum depth is blunt: it limits every branch equally, even branches with plenty of data. A second hyperparameter is more direct.

WORD TO KNOW

Minimum leaf size — The smallest number of training examples a leaf is allowed to contain. Any split that would create a smaller leaf is forbidden, so a tree can never build a rule around one or two lucky examples.

WORKED EXAMPLE 13.7 – See What a Minimum Leaf Size Costs and What It Saves

The cool node of Figure 13.1 holds 5 machines: 1 Yes (M6) and 4 No. Its vibration split would put M6 alone in a leaf. Suppose a minimum leaf size of 3 is set. Find what the tree does instead, and its new training accuracy.

1. The vibration split would create leaves of 1 and 4 machines. Since 1 is below the minimum of 3, that split is forbidden.
2. Any other split of five machines also leaves one side with 2 or fewer, so every split of this node is forbidden and the node must become a leaf.
3. A classification leaf predicts the label with the most votes: No has 4 votes and Yes has 1, so the leaf predicts No.
4. That leaf now gets M6 wrong; the other nine machines are still correct, so training accuracy is $9 \div 10 = 0.9$, or 90%.

Answer: The tree loses one correct training prediction, dropping from 100% to 90%, and in exchange stops publishing a rule built on a single machine. This is the bias-variance trade-off of Chapter 7, controlled by a leaf-size setting.

The lower half of Table 13.4 shows the same story for leaf size: training accuracy falls steadily while validation accuracy rises, peaks at a minimum leaf size of 5 — matching the best depth exactly — then falls again as the limit starts to forbid useful splits too.

Both settings stop over-specific branches from ever being built. A third method works the other way round: let the tree grow first, then cut it back.

WORD TO KNOW

Pruning — Cutting branches off an already-grown tree, replacing a branch with a single leaf whenever it does not really help on held-out data. Pruning trades a little training accuracy for a simpler tree that generalizes better.

The standard method, published with CART in 1984, is cost-complexity pruning: charge a fixed price for every leaf the tree keeps, then remove the branches that are not worth their price.

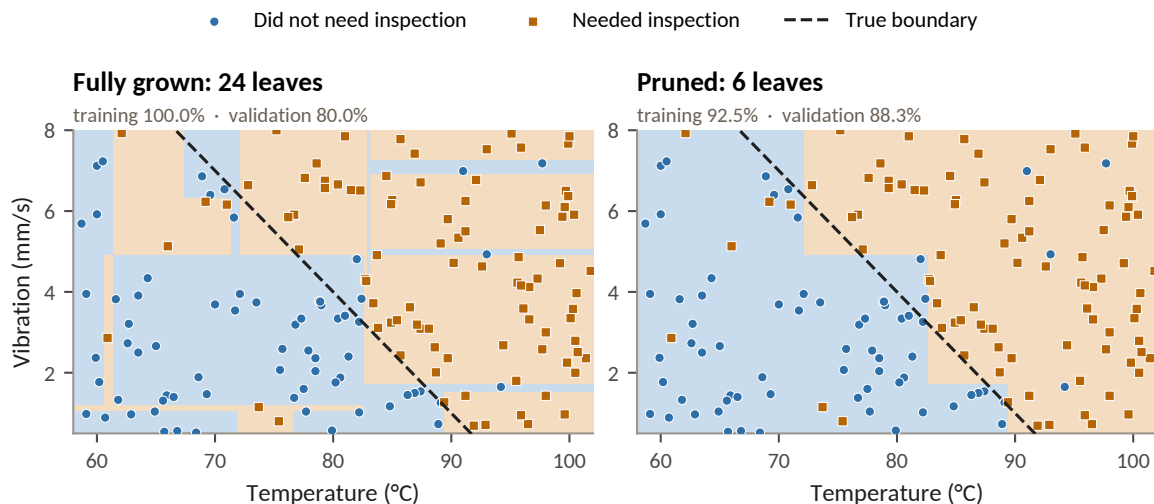


Figure 13.7 – The 160 training machines fitted by a fully grown tree (left) and by the pruned tree (right). The dashed line is the boundary the simulation actually used to generate the labels.

WHAT TO OBSERVE

The fully grown tree carves thin stripes and small islands around individual machines — each a rule built on noise. The pruned tree keeps six broad rectangles that follow the true dashed boundary in a rough staircase, getting 12 more training machines wrong and 10 more validation machines right.

COMMON MISTAKE

✗ Wrong: Growing a tree to unlimited depth and then judging it by its training accuracy.

✓ Right: A fully grown tree almost always reaches 100% training accuracy, so that number carries no information. Compare depths, leaf sizes or pruning strengths on a validation set, as Chapter 5 and Chapter 23 describe.

13.6 Regression Trees Predict Local Averages

Every tree so far has predicted a category. A tree can predict a number just as easily.

WORD TO KNOW

Regression tree — A decision tree whose leaves predict a number instead of a category. Each leaf predicts the plain average of the target values of the training examples that landed in it.

The machinery barely changes: only the meaning of “uniform” and the contents of a leaf differ. Instead of one-sided labels, a regression tree looks for children whose numbers sit close together, measured by the mean squared error of Chapter 6 around each child’s own average.

EQUATION: REGRESSION TREE: LEAF PREDICTION AND SPLIT SCORE

$$\hat{y}_{\text{leaf}} = \frac{1}{m} \sum_{j=1}^m y_j, \quad \text{MSE}_{\text{node}} = \frac{1}{m} \sum_{j=1}^m (y_j - \bar{y})^2$$

$$\text{Reduction} = \text{MSE}_{\text{parent}} - \left(\frac{n_L}{n} \text{MSE}_L + \frac{n_R}{n} \text{MSE}_R \right)$$

In words: A leaf predicts the average of the target values inside it; a node is scored by the average squared distance of its values from that average; and a split’s reduction is the parent’s score minus the weighted average of its children’s scores.

When it is used: The first formula every time the tree predicts; the other two to choose the best split at every node, exactly where weighted Gini impurity is used in a classification tree.

Symbol	Meaning	Unit or type
$\hat{y}_{\text{leaf}}, \bar{y}$	the leaf’s prediction, and a node’s average target	unit of the target
y_j, m	target value of example j in the node, and how many there are	unit of the target; count
n_L, n_R, n	examples in each child and in the parent	count
Reduction	squared error removed by the split	squared unit of the target

MSE_{node} is the variance of the node’s target values, and the largest reduction is the regression twin of the largest information gain.

WORKED EXAMPLE 13.8 — Predict a Repair Cost, and Score the Split That Produced It

Eight machines were repaired. Those aged 1, 2, 3 and 4 years cost 40, 50, 60 and 50 thousand IQD; those aged 7, 8, 9 and 10 years cost 200, 220, 180 and 200 thousand IQD. A depth-1 tree splits them with “age ≥ 5.5 years?”. Find each leaf’s prediction, predict the cost for a new 9-year-old machine, and find how much squared error the split removed.

1. Young leaf (age under 5.5): $40 + 50 + 60 + 50 = 200$, and $200 \div 4 = 50$ thousand IQD.
2. Old leaf (age 5.5 or more): $200 + 220 + 180 + 200 = 800$, and $800 \div 4 = 200$ thousand IQD.
3. Send the new machine down the tree: is $9 \geq 5.5$? Yes, so it reaches the old leaf and gets that leaf’s stored prediction, 200 thousand IQD.
4. Parent average: $1000 \div 8 = 125$. Its squared deviations 7225, 5625, 4225, 5625, 5625, 9025, 3025, 5625 add to 46,000, so $\text{MSE}_{\text{parent}} = 46,000 \div 8 = 5750$.
5. Young leaf: deviations $-10, 0, 10, 0$ square to 100, 0, 100, 0, adding to 200, so $\text{MSE}_L = 50$. Old leaf: deviations 0, 20, $-20, 0$ square to 0, 400, 400, 0, adding to 800, so $\text{MSE}_R = 200$.
6. Weighted score: $(4 \div 8) \times 50 + (4 \div 8) \times 200 = 25 + 100 = 125$.
7. Reduction: $5750 - 125 = 5625$.

Answer: The tree predicts 200 thousand IQD for the 9-year-old machine — an answer already sitting in the leaf, borrowed from four machines of similar age. The split removed 5625 of the parent’s 5750 mean squared error, about 98% of it, which is why the tree cuts at 5.5 years.

Table 13.5 – The two leaves of the repair-cost regression tree

Leaf	Age range (years)	Machines	Costs (thousand IQD)	Leaf mean	Leaf MSE
Young	under 5.5	4	40, 50, 60, 50	50	50
Old	5.5 or more	4	200, 220, 180, 200	200	200

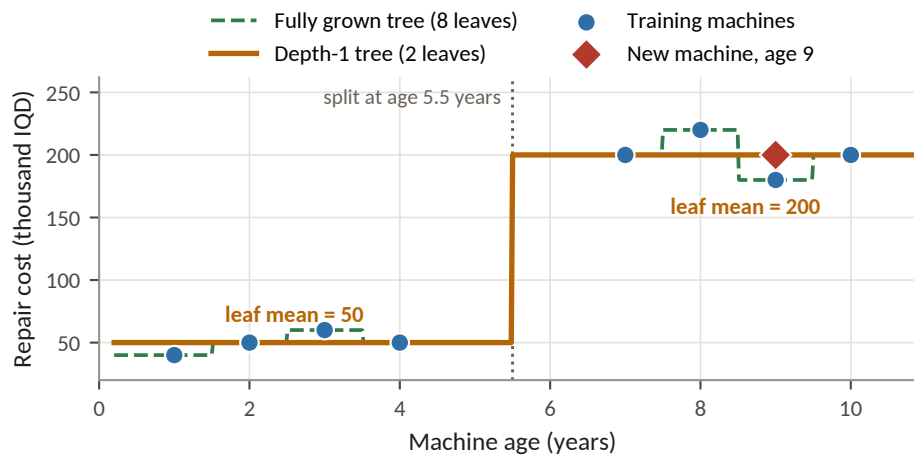


Figure 13.8 – The repair-cost regression tree as a staircase. The depth-1 tree has two steps; a fully grown tree gives every training machine its own step.

WHAT TO OBSERVE

The orange line is flat except at the single split, so every machine under 5.5 years gets the same answer, 50. The green staircase passes through all eight points perfectly, and its dip to 180 near age 9 shows it copying one unusual repair rather than a trend. Both lines stay flat past the last machine: a tree cannot continue a trend beyond the ages it has seen.

COMMON MISTAKE

X Wrong: Trusting a regression tree far outside the range of the training data — a 40-year-old machine when the oldest training machine was 10.

✓ Right: A tree can only repeat averages already stored in a leaf, so its prediction stays frozen at the last leaf's value forever. Unlike a straight line, it cannot extend a trend. Check that a new example's features fall inside the training range.

13.7 Reading a Tree as Rules, and What Trees Do Well

Every path from the root to a leaf is one rule, so the tree of Figure 13.1 becomes three sentences: if temperature is 82 °C or more, inspect this week (5 machines); if temperature is below 82 °C and vibration is High, inspect this week (1 machine); otherwise no inspection is needed (4 machines). A technician can carry those in his head, and no other model in this book converts so directly into instructions. A tree also reports which features it leaned on, by adding up the impurity each one removed.

WORD TO KNOW

Feature importance — A score for each feature saying how much of the tree's total impurity drop that feature was responsible for. The scores are shares of one whole, so they add up to 1 across all features.

EQUATION: FEATURE IMPORTANCE FROM A TREE

$$D_t = n_t I_t - n_L I_L - n_R I_R, \quad \text{Imp}(f) = \frac{\sum_{t \in T_f} D_t}{\sum_{t \in T} D_t}$$

In words: At every node, measure the impurity it removed, counted in examples; a feature's importance is its share of all the impurity removed anywhere in the tree.

When it is used: After training, to rank features by how much work each one did.

Symbol	Meaning	Unit or type
D_t	impurity removed at node t , weighted by its size	impurity $\times$ count
n_t, n_L, n_R	examples in node t and in its two children	count
I_t, I_L, I_R	impurity of node t and of its two children	Gini or entropy
T_f, T	nodes that split on feature f , and all splitting nodes	sets of nodes

Multiplying by n_t makes a split near the root count for more than one deep inside a small branch; dividing by the total makes the scores add up to 1.

WORKED EXAMPLE 13.9 — Compute the Importance of Each Feature

The tree of Figure 13.1 splits twice: on temperature at the root (10 machines, $I = 0.48$, children of 5 with $I = 0$ and 5 with $I = 0.32$) and on vibration at the cool node (5 machines, $I = 0.32$, children of 1 and 4, both pure). Find all three importances.

1. Impurity removed at the root: $D = 10 \times 0.48 - 5 \times 0 - 5 \times 0.32 = 4.8 - 1.6 = 3.2$.
2. Impurity removed at the cool node: $D = 5 \times 0.32 - 1 \times 0 - 4 \times 0 = 1.6$.
3. Total for the whole tree: $3.2 + 1.6 = 4.8$.
4. Temperature's share: $3.2 \div 4.8 = 0.667$. Vibration's share: $1.6 \div 4.8 = 0.333$.
5. Age never appears in any split, so its share is $0 \div 4.8 = 0$. Check: $0.667 + 0.333 + 0 = 1.000$.

Answer: Temperature scores 0.67, vibration 0.33 and age 0.00. Temperature did twice as much work as vibration, and age — whose Gini gain at the root was exactly zero — did none.

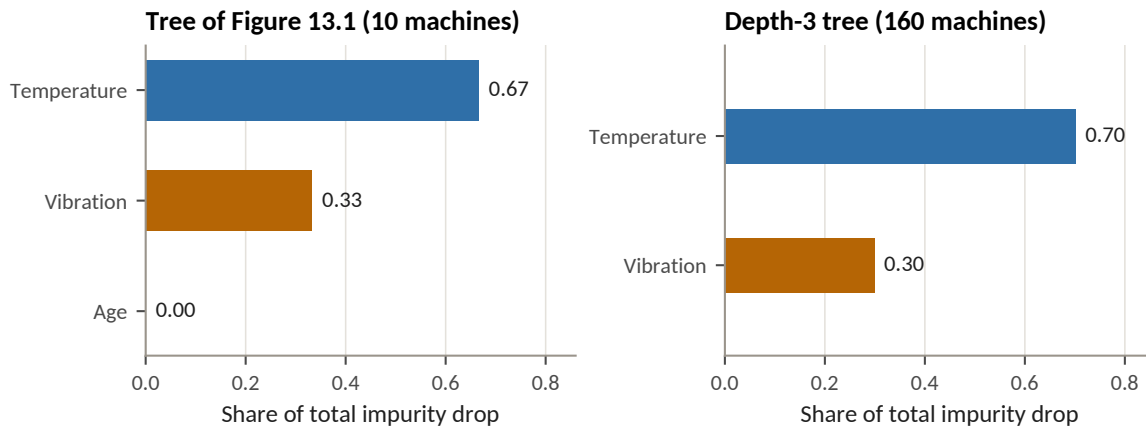


Figure 13.9 – Feature importances. Left: the hand-built tree of Figure 13.1. Right: a depth-3 tree fitted to all 160 training machines, where vibration is a raw reading in mm/s.

WHAT TO OBSERVE

Both panels tell the same story from different data: temperature carries roughly two-thirds of the importance and vibration one-third. Age scores exactly zero on the left — not because age is meaningless in the world, but because this tree never used it.

Read these scores carefully. If two features are closely related, a tree usually splits on one and gives the other a low score even though either would have worked, and a high score never proves cause: a tree that finds temperature useful has not shown that heat causes failures. Chapter 22 covers feature selection properly and Chapter 24 covers reading model explanations responsibly. One last property is worth naming, first seen in Figure 13.2.

WORD TO KNOW

Axis-aligned boundary — A decision boundary made only of straight pieces, each at right angles to one feature's axis, because every question compares a single feature with a single threshold. Trees can draw only this boxy kind of boundary.

Those boxy edges are the main price a tree pays. In exchange it needs no scaling or standardizing of features, it mixes numerical and category columns happily, it catches “if this and also that” combinations by itself, and every path it grows is a sentence a person can read. Against that, a deep tree has far too many rules to read, a diagonal boundary costs it a whole staircase of leaves, and one tree on its own is rarely the most accurate model available.

There is one more weakness, and it is why this chapter has a sequel: a single tree is unstable. Because the greedy search commits to one root question, a handful of different training rows can produce a completely different tree. That weakness becomes a strength once you grow many different trees and let them vote —

the idea behind random forests, introduced by Leo Breiman in 2001, which Chapter 14 builds directly on this chapter, and behind the boosted trees of Chapter 15.

COMMON MISTAKES AND MISCONCEPTIONS

- Assuming every example passes through the same number of questions on its way to a leaf.
- Comparing two children's impurities directly, without weighting each by how many examples landed in it.
- Treating a low impurity as proof of accuracy, when impurity only measures how mixed the labels are.
- Growing a tree to unlimited depth and then judging it by its 100% training accuracy, or assuming a deeper tree must be a better tree.
- Trusting a regression tree outside the range of feature values it was trained on, where its staircase is flat forever.
- Reading a feature importance of zero as proof that the feature does not matter in the real world.
- Choosing a depth, a leaf size or a pruning strength using training-set performance instead of a validation set.

Practical Activities

ACTIVITY (PAPER) — Score Three Splits for Eight Village Water Pumps

Eight hand pumps in villages near Zakho were checked last year. The table records each pump's age, whether the well's water level was Low or High, its brand, and whether it failed within a month.

Pump	P1	P2	P3	P4	P5	P6	P7	P8
Age (years)	12	10	9	5	6	8	3	2
Water level	Low	Low	Low	Low	Low	High	High	High
Brand	A	A	B	B	A	A	B	B
Failed?	Yes	Yes	Yes	Yes	No	No	No	No

1. Count the Yes and No labels in the whole group and compute the root node's Gini impurity, showing every step.
2. Split the pumps by "age ≥ 8 years?". Write the two children's counts, find each child's Gini impurity, and combine them into a weighted Gini.
3. Do the same for "water level = Low?" and for "brand = A?".
4. Rank the three candidates, say which question the tree should ask first, and compute its information gain. Check: the root scores 0.5 and the candidates score 0.375, 0.2 and 0.5, so the water-level question wins with a gain of $0.5 - 0.2 = 0.3$. Which candidate teaches the tree nothing at all, and why?
5. Grow one more level under the winning question by hand, then write your finished tree out as plain-language rules.

You need: Paper, pencil and a calculator

ACTIVITY (PYTHON) — Grow the Same Trees with scikit-learn

The program fits the tree of Figure 13.1 to the ten machines of Table 13.1, repeats Worked Examples 13.7 and 13.9, then reproduces Table 13.4 from `datasets/ch13_machines.csv`, which holds the 280 factory machines in the columns `temperature_c`, `vibration_mm_s`, `needs_inspection` and `set`.

```

import numpy as np, pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

# 1. The ten machines of Table 13.1 (vibration: 1 = High, 0 = Low)
X = np.array([[95, 1, 9], [90, 0, 3], [88, 0, 7], [86, 0, 2], [84, 0, 5],
              [70, 1, 8], [78, 0, 4], [76, 0, 10], [72, 0, 6], [62, 0, 1]])
y = ["Yes"] * 6 + ["No"] * 4
names = ["temperature", "vibration", "age"]
tree = DecisionTreeClassifier(criterion="gini", max_depth=2, random_state=0).fit(X, y)
print(export_text(tree, feature_names=names))
print("importances:", tree.feature_importances_.round(3))

# 2. Blocking tiny leaves (Worked Example 13.7)
small = DecisionTreeClassifier(min_samples_leaf=3, random_state=0).fit(X, y)
print("leaves:", small.get_n_leaves(), " training accuracy:",
      accuracy_score(y, small.predict(X)))

# 3. How deep should the tree be? 280 machines from the whole factory floor
d = pd.read_csv("datasets/ch13_machines.csv")
tr, va = d[d.set == "train"], d[d.set == "validation"]
cols = ["temperature_c", "vibration_mm_s"]
print("depth leaves training% validation%")
for depth in (1, 3, 5, 7):
    m = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(tr[cols],
        tr.needs_inspection)
    print(depth, m.get_n_leaves(),
          round(accuracy_score(tr.needs_inspection, m.predict(tr[cols])) * 100, 1),
          round(accuracy_score(va.needs_inspection, m.predict(va[cols])) * 100, 1))

```

Output

```

|--- temperature <= 81.00
|   |--- vibration <= 0.50
|   |   |--- class: No
|   |   |--- vibration > 0.50
|   |   |--- class: Yes
|--- temperature > 81.00
|   |--- class: Yes

importances: [0.667 0.333 0.   ]
leaves: 2 training accuracy: 0.9
depth leaves training% validation%
1 2 80.6 75.8
3 8 92.5 88.3
5 20 97.5 80.0
7 24 100.0 80.0

```

Part 1 prints exactly the tree of Figure 13.1, and its importances 0.667, 0.333 and 0.000 match Worked Example 13.9. The threshold reads 81.00 because scikit-learn cuts halfway between 78 and 84 — the

same rule as “82 °C or more” for whole-degree readings. Part 2 confirms Worked Example 13.7, and part 3 reprints the depth rows of Table 13.4. Try `criterion="entropy"` in part 1 and check that the tree does not change.

You need: Python with NumPy, pandas and scikit-learn (see Appendix B), and `datasets/ch13_machines.csv`

CASE STUDY — Prioritizing Equipment Inspections at a Motor Factory in Erbil

Shilan manages maintenance at the motor factory whose sensor data runs through this chapter. Her four technicians can inspect perhaps twenty of its three hundred machines properly in a week, and until now they worked a fixed rotation, walking the same route regardless of which machines showed warning signs. Shilan wants the week’s list to follow the evidence instead. A decision tree appeals to her for a reason beyond accuracy: whatever questions it learns can be read aloud, the way Lesson 13.7 reads a tree as three sentences. “Above 82 degrees, inspect this week” is a rule a busy person can remember, argue with, and correct — and being able to argue with it turned out to matter more than the team expected.

The team follows this chapter’s method. They score candidate splits with weighted Gini impurity and find that temperature and vibration both earn their place, while machine age barely lowers impurity at all — exactly the zero-gain row of Table 13.3 — so age is dropped. They then compare maximum depths on a validation set of machines the tree never trained on and settle on a moderate depth, just as Table 13.4 does: the fully grown tree scores perfectly on its training machines and clearly worse on new ones, carving out lonely one-machine leaves for accidents that never repeat.

The finished tree shuts nothing down. Every morning it produces a ranked list, together with the question path that put each machine on it, and a technician makes the final call. Two months in, the model exposes something the old rotation had hidden: a handful of machines checked every week almost never needed attention, while a few others, visited only occasionally, kept reappearing near the top. Shilan reports that the real gain is not one dramatic rescue but a steady, unglamorous improvement — the same four technicians, the same twenty inspections, aimed where the evidence points.

RESPONSIBLE USE

A ranked list is a good start to a technician’s morning, never a verdict on its own. A tree trained on past inspection records can only repeat the patterns inside those records, including past oversights: machines nobody ever checked cannot appear as failures in the training data. Keep a trained person reviewing the list and making the final decision — and be far more careful still when the rows are people rather than motors, as Chapter 24 explains.

CHAPTER SUMMARY

- A decision tree predicts by asking yes-or-no questions, starting at the root node and stopping at a leaf; different examples can take paths of different lengths.
- Impurity measures how mixed the labels are inside one node: Gini impurity is $1 - \sum p_i^2$ and entropy is $-\sum p_i \log_2 p_i$; both are 0 for a pure node and largest for an even mix.
- A split is scored by the weighted impurity of its two children, each counting in proportion to its size; information gain is the parent’s impurity minus that score.
- A tree is grown greedily: score every feature and threshold at a node, keep the best split, repeat inside each child until a stopping rule applies. Because every question uses one feature and one threshold, the boundary is always made of axis-aligned rectangles.
- Left alone, a tree splits until every leaf is pure, reaching 100% training accuracy and overfitting badly.
- Maximum depth and minimum leaf size limit a tree before it grows and pruning cuts back a grown tree; all three are chosen on a validation set, never on training accuracy.

- A regression tree predicts the average target value of the examples in a leaf, scores splits by the reduction in mean squared error, and cannot extend a trend beyond its training range.
- Feature importance is a feature's share of the total impurity the tree removed; a score of zero means the tree did not use the feature, not that the feature is useless.
- A single tree is easy to read but unstable and rarely the most accurate model; Chapter 14 combines many trees to fix that.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain the difference between a root node, an internal node and a leaf node.

Problem 2 [Basic] Explain why two different machines can answer a different number of questions in the same decision tree.

Problem 3 [Basic] A node contains 12 examples that all share the same label. State its Gini impurity and its entropy, and explain both answers. Then state what a leaf predicts in a classification tree and in a regression tree.

Problem 4 [Basic] Using the tree of Figure 13.1, trace a machine at 84 °C with Low vibration from the root to its prediction, writing each question and answer.

Problem 5 [Standard] A node contains 20 examples: 12 labeled “Fail” and 8 labeled “Pass”. Find its Gini impurity, showing every step, and then its entropy, using $\log_2 0.6 = -0.737$ and $\log_2 0.4 = -1.322$.

Problem 6 [Standard] A split of 20 examples creates two children. Child 1 has 4 examples, all “Yes”. Child 2 has 16 examples: 12 “Yes” and 4 “No”. Find the weighted Gini impurity of this split, showing every step.

Problem 7 [Standard] The parent node in Problem 6 held 16 “Yes” and 4 “No”. Find its Gini impurity, then the information gain of that split, and say whether the split was worth making.

Problem 8 [Standard] A regression-tree leaf contains three machines whose repair costs were 40, 55 and 70 thousand IQD. Find the leaf's prediction and then the leaf's mean squared error.

Problem 9 [Standard] A tree at maximum depth 2 scores 82% on training data and 80% on validation data; at maximum depth 12 it scores 99% and 71%. Say which depth is overfitting, which you would choose, and what evidence supports each answer.

Problem 10 [Challenge] A team compares two splits of the same 12 examples. Split A gives children of 6 and 6 with weighted Gini 0.28; Split B gives children of 2 and 10 with weighted Gini 0.19. (a) Which split does the algorithm choose, and why? (b) Give one reason Split B's two-example child deserves a second look, and name one hyperparameter that would forbid it.

Problem 11 [Challenge] A regression tree predicting delivery time has one leaf holding five past deliveries of 20, 25, 30, 90 and 35 minutes. (a) Find the leaf's prediction. (b) Find the prediction without the 90-minute delivery. (c) Explain what a single unusual value does to a leaf average, and give one question you would ask about that delivery before trusting the leaf.

Problem 12 [Challenge] A tree is trained on two features whose true class boundary runs diagonally across the feature map. (a) Sketch the tree's boundary and explain its shape. (b) Say what happens to the number of leaves, and to validation accuracy, as the tree follows the diagonal more closely.

Problem 13 [Challenge] A hospital in Sulaymaniyah builds a tree to decide whose laboratory results a doctor reviews first. (a) Give one advantage a tree offers over a model whose reasoning cannot be read aloud. (b) Describe one risk of trusting the ranking without human review. (c) Suggest one safeguard, and name the chapter of this book you would read before deploying it.

CHAPTER FOURTEEN

Random Forests and Bagging

Many different trees, voting together, beat one tree

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what an ensemble is, and why many different models can beat any one of them alone.
- Build a bootstrap sample by hand, and calculate the chance that a row is left out of one.
- Describe bagging, and make a forest's prediction by majority vote or by averaging.
- Explain how a random forest adds feature randomness on top of bagging, and why that extra randomness helps.
- Compute a forest's out-of-bag score by hand, and say when it can be trusted.
- Compute impurity-based and permutation feature importance, and read both carefully.
- Choose the number of trees and the other main settings of a forest, and name a forest's limits.

Introduction

In 1906 the English scientist Francis Galton visited a country fair where about eight hundred visitors each paid a small fee to guess the weight of an ox. Butchers guessed. Farmers guessed. So did people who had never stood near an ox in their lives. Almost nobody was right. But when Galton collected all the tickets afterward and looked at the middle of the whole pile of guesses, that middle number sat within about one percent of the animal's true weight. He published the result in 1907, and it has been retold ever since as the “wisdom of crowds”.

Chapter 13 ended with a warning about decision trees. A single tree is easy to read, but it is unstable: change a handful of training rows and the greedy search may pick a different root question and grow a visibly different tree. In the language of Chapter 7, a fully grown tree has high variance. That is exactly the weakness Galton's crowd fixes.

This chapter turns that idea into a working model. You will build small random training sets called bootstrap samples, grow one tree on each, and let the trees vote. You will see why forcing each tree to look at only some of the features makes the group stronger, and you will get a free accuracy check out of the rows each tree never saw. The result — the random forest, introduced by Leo Breiman in 2001 — is very often the strongest model you can train on an ordinary table of data in an afternoon.

14.1 Why Many Trees Can Beat One

Sunrise Parcels moves parcels between Erbil, Baghdad and Basra. Every afternoon somebody in its dispatch office phones a customer to apologize for a late delivery, and Rawa, who runs dispatch, would rather know before the van leaves. Her team wrote down four things about eight recent deliveries, plus the answer that matters: was it late? Table 14.1 is that record, and the whole chapter is built on it.

Table 14.1 – Eight Sunrise Parcels deliveries — four features and one label

Delivery	Distance (km)	Parcels on the van	Weather	Driver experience (years)	Late?
D1	5	6	Clear	8	No
D2	42	22	Rain	2	Yes
D3	8	17	Clear	5	No
D4	36	18	Clear	6	Yes
D5	11	7	Clear	9	No
D6	30	20	Rain	4	Yes
D7	7	12	Rain	3	No
D8	44	15	Rain	7	Yes

Four deliveries were late and four were not, so the root node's Gini impurity is $1 - (0.5^2 + 0.5^2) = 0.5$, the two-class maximum of Chapter 13. Scoring each feature as that chapter did gives distance the largest Gini gain, 0.50, against 0.30 for parcels, 0.17 for experience and 0.13 for weather. Distance is the strongest column here — remember that, because it causes trouble later.

Rawa's analyst, Karwan, grows a decision tree. Trained on last month's deliveries it asks about distance first; trained on this month's it sometimes asks about parcels first and gives a different answer to the same border-line delivery. The tree is not wrong so much as jumpy, and the fix is not a cleverer tree. It is more trees.

WORD TO KNOW

Ensemble — A model built by combining the predictions of many smaller models instead of relying on one. An ensemble usually predicts by letting its members vote, for a category, or by averaging their answers, for a number.

To see why combining helps, take an easier question than “late or not”: how many minutes late? Five trees, each grown on slightly different data, are asked about the same delivery.

WORKED EXAMPLE 14.1 — Average Five Trees' Guesses for One Delivery

Five trees predict that one delivery will run 14, 30, 22, 34 and 25 minutes late. It really ran 24 minutes late. Compare the typical single tree's error with the error of the five trees' average.

1. Find each tree's error, ignoring the sign: $|14 - 24| = 10$, $|30 - 24| = 6$, $|22 - 24| = 2$, $|34 - 24| = 10$, $|25 - 24| = 1$.
2. Average those errors: $(10 + 6 + 2 + 10 + 1) \div 5 = 5.8$ minutes.
3. Average the five predictions instead: $125 \div 5 = 25$ minutes, whose error is $|25 - 24| = 1$ minute.

Answer: A typical single tree is 5.8 minutes off; the average of the five is 1 minute off. Two trees guessed far too low and two far too high, so most of their mistakes canceled instead of adding up.

Nothing there required any tree to be good. It required them to be *wrong in different directions*. Figure 14.1 shows the same effect across a whole range of distances.

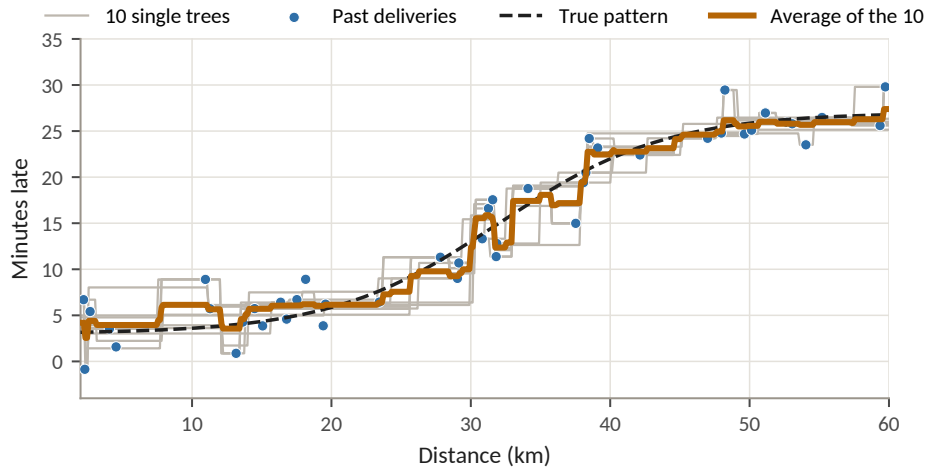


Figure 14.1 – Ten regression trees, each grown on its own random sample of the same 45 past deliveries, and the average of all ten.

WHAT TO OBSERVE

Each thin gray staircase is one tree. They agree roughly but never exactly, and each has its steps in different places. The thick orange line is their average, and it tracks the dashed true pattern far more closely than any single gray line does. Averaging did not remove the steps; it made them small.

How much closer? That depends on how much a single tree wobbles and on how much the trees resemble each other. Statistics gives an exact answer for the average of B predictions.

EQUATION: HOW MUCH AVERAGING REDUCES VARIANCE

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B t_b\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

In words: The wobble of an average has two parts: a shared part that never goes away, and a private part that shrinks as more models are added.

When it is used: To predict how much an ensemble of B models will steady a prediction, and to see why making the models less alike matters as much as adding more.

Symbol	Meaning	Unit or type
B	number of models in the ensemble	count
t_b	the prediction of model b for one fixed input	unit of the target
σ^2	variance of one single model's prediction	squared unit of the target
ρ	correlation between two models' predictions	between 0 and 1, no unit

If the models were unrelated, $\rho = 0$ and this collapses to σ^2/B . If they were identical, $\rho = 1$ and it collapses to σ^2 : a thousand copies of one tree are worth one tree.

WORKED EXAMPLE 14.2 — Find How Much Wobble Sixteen Trees Remove

A single tree's prediction of minutes late has a standard deviation of 8 minutes across training samples, so $\sigma^2 = 64$. Find the variance of an average of $B = 16$ trees, first if the trees were unrelated ($\rho = 0$), then if they were half alike ($\rho = 0.5$).

1. With $\rho = 0$ the first term vanishes and the second becomes σ^2/B : $64 \div 16 = 4$.
2. Take the square root to return to minutes: $\sqrt{4} = 2$ minutes instead of 8.
3. With $\rho = 0.5$, the shared part is $0.5 \times 64 = 32$.
4. The private part is $\frac{1 - 0.5}{16} \times 64 = \frac{0.5 \times 64}{16} = 2$.
5. Add them: $32 + 2 = 34$, and $\sqrt{34} = 5.83$ minutes.

Answer: Sixteen unrelated trees cut the wobble from 8 minutes to 2. Sixteen half-alike trees cut it only to 5.83 minutes, and adding thousands more would never take it below $\sqrt{32} = 5.7$ minutes. Making the trees different is not a detail — it sets the floor, and everything a random forest does beyond growing ordinary trees exists to push ρ down.

COMMON MISTAKE

X Wrong: Believing that any group of models must beat a single model, so a hundred weak or nearly identical models is always progress.

✓ Right: Combining helps only when the members are reasonably good *and* make different mistakes. A hundred copies of one tree predict exactly what one copy predicts, and a hundred models all wrong in the same direction stay wrong.

14.2 Bootstrap Sampling

So the trees must differ. But Sunrise Parcels has one training table, and a tree grown twice on the same rows comes out the same twice. Where is the variety to come from? The answer is a resampling trick published by the statistician Bradley Efron in 1979.

WORD TO KNOW

Bootstrap sample — A new dataset built by drawing rows at random from an original dataset, *with replacement*, until it has as many rows as the original. Because a row can be drawn more than once, a bootstrap sample usually repeats some rows and leaves others out completely.

“With replacement” is the important part. Picture writing each delivery’s name on a slip of paper, dropping all eight slips into a cup, drawing one slip, writing down what it says, and then *putting the slip back* before the next draw. Because every slip goes back, D3 can come out twice and D2 might never come out at all. Figure 14.2 shows one such round of eight draws.

The original 8 deliveries



Bootstrap sample for Tree 1: eight draws, with replacement

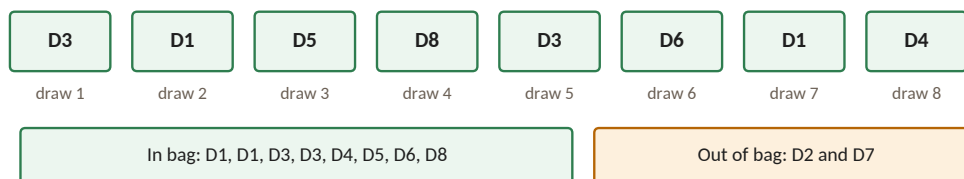


Figure 14.2 – One bootstrap sample drawn from the eight deliveries. Blue slips were drawn at least once; the two pale slips were never drawn.

WHAT TO OBSERVE

The bottom row still has eight entries, like the original, but D1 and D3 appear twice while D2 and D7 do not appear at all. The sample is the same size as the training set and yet it is not the training set — that is the whole trick.

WORKED EXAMPLE 14.3 — Build One Bootstrap Sample from Eight Deliveries

To grow its first tree, the computer draws eight of the deliveries at random, with replacement. The draws come out in this order: D3, D1, D5, D8, D3, D6, D1, D4. List which deliveries the tree will train on, and which are left out completely.

1. Count how many times each delivery was drawn: D1 twice (draws 2 and 7), D3 twice (draws 1 and 5), and D4, D5, D6, D8 once each.
2. Check the total: $2 + 2 + 1 + 1 + 1 + 1 = 8$, matching the eight draws made.
3. List the distinct deliveries that appear at least once: D1, D3, D4, D5, D6, D8. That is 6 different deliveries out of 8.
4. Compare that list against D1 through D8 to find the ones never drawn: D2 and D7.

Answer: This tree trains on D1, D1, D3, D3, D4, D5, D6, D8 — eight rows, but only six different deliveries. D2 and D7 are left out entirely, and Lesson 14.5 turns those two into a free accuracy check.

Two rows were left out here. Typical, or luck? The chance is worth computing, because it turns out to be nearly the same for every dataset size. On one draw a particular row is missed with probability $1 - 1/n$, and the n draws are independent, so it is missed on all of them with probability $(1 - 1/n)^n$.

EQUATION: CHANCE THAT ONE ROW IS LEFT OUT OF A BOOTSTRAP SAMPLE

$$P_{\text{out}} = \left(1 - \frac{1}{n}\right)^n, \quad P_{\text{out}} \rightarrow \frac{1}{e} \approx 0.368 \text{ as } n \text{ grows}$$

In words: The chance of missing a given row on one draw, multiplied by itself once for every draw. For any training set bigger than a few dozen rows the answer is close to 37 percent.

When it is used: To work out how many rows a bootstrap sample leaves out, and so how many are available for the free out-of-bag check of Lesson 14.5.

Symbol	Meaning	Unit or type
P_{out}	probability that one particular row is never drawn	between 0 and 1
n	number of rows in the training set, and the number of draws	count
e	the mathematical constant 2.71828...	no unit

The curve flattens almost at once: P_{out} is 0.344 for 8 rows, 0.364 for 50 and 0.368 for a million. The matching in-bag chance, $1 - P_{\text{out}} \approx 0.632$, is why the bootstrap is sometimes called the “63.2 percent rule”.

WORKED EXAMPLE 14.4 — Find the Chance That a Delivery Is Left Out

Sunrise Parcels has $n = 8$ deliveries. Find the chance that one delivery, say D2, is never drawn in the eight draws, and how many deliveries are expected to be left out.

1. Chance D2 is missed on a single draw: $1 - \frac{1}{8} = 0.875$.
2. The eight draws are independent, so multiply that chance by itself eight times: 0.875^8 .
3. Work it up by squaring: $0.875^2 = 0.7656$, then $0.7656^2 = 0.5862$, then $0.5862^2 = 0.3436$.
4. So $P_{\text{out}} = 0.3436$, about 34%.
5. Expected number of left-out deliveries: $8 \times 0.3436 = 2.75$, so about 3 of the 8.

Answer: Each delivery has about a 34% chance of being left out, so about 2.75 of the 8 are expected to be missing from any one bootstrap sample. The sample in Worked Example 14.3 left out 2 — an ordinary result, not a strange one.

That 34% barely moves as the dataset grows: whether a company has 8 deliveries or 8 million, every bootstrap sample leaves out roughly 37% of its rows and keeps roughly 63%.

COMMON MISTAKE

X Wrong: Thinking a bootstrap sample is just the training set handed to the tree again, so every tree in the forest sees identical data.

✓ Right: A bootstrap sample is drawn with replacement, so it repeats some rows and drops about 37% of them entirely. That difference is what makes each tree in the forest a different tree.

14.3 Bagging: Bootstrap Aggregating

Now put the two ideas together: draw many bootstrap samples, grow one model on each, combine their answers. Leo Breiman named this procedure in 1996, joining the two halves of its description into one word.

WORDS TO KNOW

Bagging — Short for *bootstrap aggregating*: training many copies of one kind of model, each on its own bootstrap sample, then combining every copy’s prediction. Bagging mainly reduces variance.

Majority vote — How a classification ensemble combines its members: count how many chose each category, and answer with the category chosen most often.

Vote share — The fraction of members that voted for the winning category — a rough confidence level for the prediction.

Figure 14.3 is the map of the whole chapter: one training table becomes several bootstrap samples, each sample grows its own tree, each tree casts one vote, and the votes are counted.

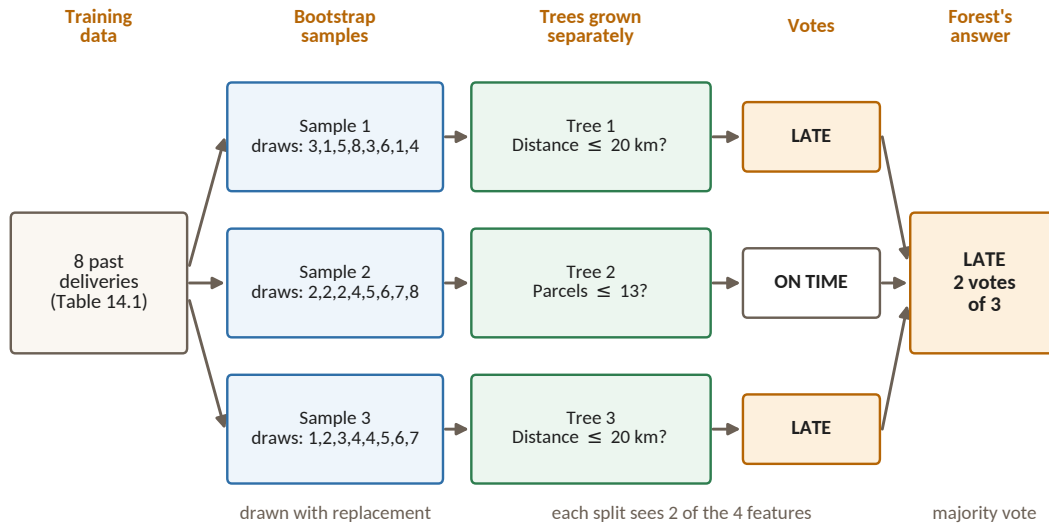


Figure 14.3 – From one small training table to one prediction. Each row of the diagram is one tree, grown on its own bootstrap sample and never told what the other trees are doing.

WHAT TO OBSERVE

Nothing joins the three rows until the last column. The trees are grown independently and in any order, which is why a forest can be trained on many processors at once. The last column is the only place they meet: two say Late, one says On time, and the majority wins.

Aggregating means one thing for a category and another for a number, so the rule comes in two halves.

EQUATION: HOW AN ENSEMBLE OF TREES PREDICTS

$$\hat{y}_{\text{number}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B t_b(\mathbf{x}), \quad \text{share}(c) = \frac{n_c}{B}$$

In words: To predict a number, average what every tree said. To predict a category, count the votes for each one, divide by the number of trees, and answer with the largest share.

When it is used: Every time a bagged ensemble or a forest predicts: the first formula for regression, the second for classification.

Symbol	Meaning	Unit or type
$\hat{y}_{\text{number}}$	the ensemble's predicted number	unit of the target
$t_b(\mathbf{x})$	what tree b predicts for the example $\mathbf{x}$	unit of the target, or a category
B	number of trees in the ensemble	count
n_c	number of trees voting for category c	count

The winning share is worth reporting next to the answer: "Late, 62 of 100 trees" and "Late, 98 of 100 trees" are the same prediction carrying very different agreement.

Bagging alone already helps, but on this dataset it hits a wall, and the wall is exactly the ρ of Lesson 14.1. Distance has the largest Gini gain on the whole table and keeps its lead inside almost every bootstrap sample,

so nearly every bagged tree reaches for it at the root and many look alike below it too: different, but not different enough.

14.4 Random Forests Add Feature Randomness

The fix is almost rude in its simplicity: at every single split, hide most of the features from the tree and make it choose from the few that are left.

WORDS TO KNOW

Feature subsampling — Choosing a node's split from only a small, randomly chosen subset of the features, freshly drawn at that one node. The setting that controls the subset size is usually called *max features*.

Random forest — An ensemble of decision trees where every tree is grown on its own bootstrap sample and every split is chosen from a random subset of the features. It predicts by majority vote, or by averaging for a number.

This sounds like sabotage. A tree that may not see distance must settle for a worse question, so each individual tree really is weaker. But the equation of Lesson 14.1 says the ensemble does not care much about σ^2 once B is large; it cares about ρ . Forbidding the favorite feature at random nodes is the most direct way to stop every tree telling the same story. Tin Kam Ho published the idea of growing trees on random feature subsets in 1998, and Breiman combined it with bagging into the random forest in 2001. A common default is to offer $\sqrt{p}$ features at each split for classification; with the four features of Table 14.1, that is two.

Let us grow three such trees by hand, each with its own bootstrap sample and its own pair of allowed features, and each stopped after one split so the arithmetic stays small.

WORKED EXAMPLE 14.5 — Grow Three Trees on Three Different Samples

Tree 1 trains on D3, D1, D5, D8, D3, D6, D1, D4 and may use only Distance or Weather. Tree 2 trains on D2, D2, D2, D4, D5, D6, D7, D8 and may use only Parcels or Weather. Tree 3 trains on D1, D2, D3, D4, D4, D5, D6, D7 and may use only Distance or Experience. Find the one question each tree asks, scoring candidates with weighted Gini impurity as in Chapter 13.

1. Tree 1's sample holds 3 late and 5 on-time rows, so its parent impurity is $1 - (0.375^2 + 0.625^2) = 0.46875$. Sorted by distance, its on-time rows sit at 5, 5, 8, 8 and 11 km and its late rows at 30, 36 and 44 km, so "distance ≤ 20 km?" gives two pure children: weighted Gini = 0.
2. Tree 1's only other option, "weather = Rain?", leaves 6 rows mixed and scores 0.208. Zero wins, so Tree 1 asks about distance.
3. Tree 2 may not look at distance at all. Its sample holds 6 late and 2 on-time rows, impurity $1 - (0.75^2 + 0.25^2) = 0.375$. Its on-time rows carry 7 and 12 parcels and its late rows 15, 18, 20, 22, 22 and 22, so "parcels ≤ 13 ?" also splits it perfectly, 0 against 0.333 for weather.
4. Tree 3's sample holds 4 late and 4 on-time rows, impurity 0.5. Again the on-time rows (5, 7, 8 and 11 km) sit below the late ones (30, 36, 36 and 42 km), so "distance ≤ 20 km?" scores 0, against 0.333 for its best experience question.

Answer: Tree 1 and Tree 3 ask "distance ≤ 20 km?" and Tree 2 asks "parcels ≤ 13 ". Every tree sorts its own sample perfectly, yet the forest holds two different questions — and it holds the second one only because Tree 2 was forbidden to look at distance.

Table 14.2 collects the three trees; its last column is the next worked example.

Table 14.2 – The three trees of the mini-forest, and how each one votes on Delivery Q

Tree	Bootstrap sample	Left out	Features allowed	Question learned	Vote on Q
1	D3, D1, D5, D8, D3, D6, D1, D4	D2, D7	Distance, Weather	distance ≤ 20 km?	Late
2	D2, D2, D2, D4, D5, D6, D7, D8	D1, D3	Parcels, Weather	parcels ≤ 13 ?	On time
3	D1, D2, D3, D4, D4, D5, D6, D7	D8	Distance, Experience	distance ≤ 20 km?	Late

WORKED EXAMPLE 14.6 – Let the Mini-Forest Vote on a New Delivery

A new delivery, Q, is about to leave: 25 km, 11 parcels, clear weather, a driver with 6 years of experience. Use the three trees of Table 14.2 to predict whether Q will be late, and report the vote share.

1. Tree 1 asks: is $25 \leq 20$? No, so Tree 1 votes **Late**.
2. Tree 2 asks: is $11 \leq 13$? Yes, so Tree 2 votes **On time**.
3. Tree 3 asks the same question as Tree 1: is $25 \leq 20$? No, so Tree 3 votes **Late**.
4. Count the votes: Late 2, On time 1.
5. Vote share of the winner: $2 \div 3 = 0.667$, about 67%.

Answer: The forest predicts Late by 2 votes to 1, a 67% share. The disagreement is honest information: Q is a middling delivery, long enough to worry Tree 1 but light enough to reassure Tree 2, and a forest reports that instead of hiding it.

Three stumps are a toy. Figure 14.4 shows 200 full-sized trees on 400 recorded deliveries, using distance and parcels so the picture fits on a page.

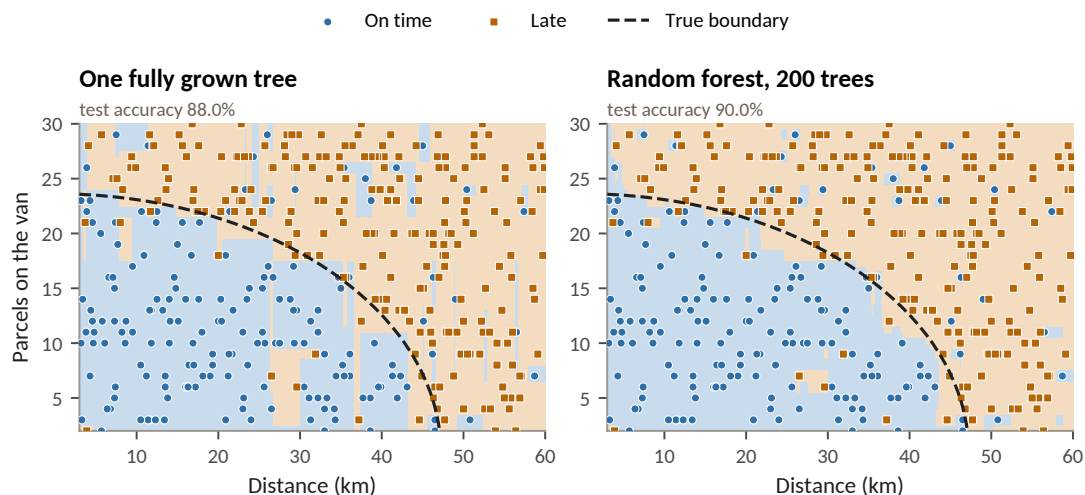


Figure 14.4 – One fully grown tree (left) and a 200-tree random forest (right) fitted to the same 400 training deliveries. Shading shows what each model predicts anywhere on the map.

WHAT TO OBSERVE

The single tree carves thin stripes and lonely islands, each a rule built around a handful of unusual deliveries. The forest's boundary is made of far smaller steps that hug the dashed true curve, because it averages two hundred different staircases and the places where they disagree get smoothed away. A forest is still built only from right-angled cuts, yet together they trace a curve.

Does the feature randomness earn its place, or would plain bagging have been enough? Figure 14.5 trains each model thirty times, on thirty resampled versions of the training data, scoring every one on the same untouched test set.

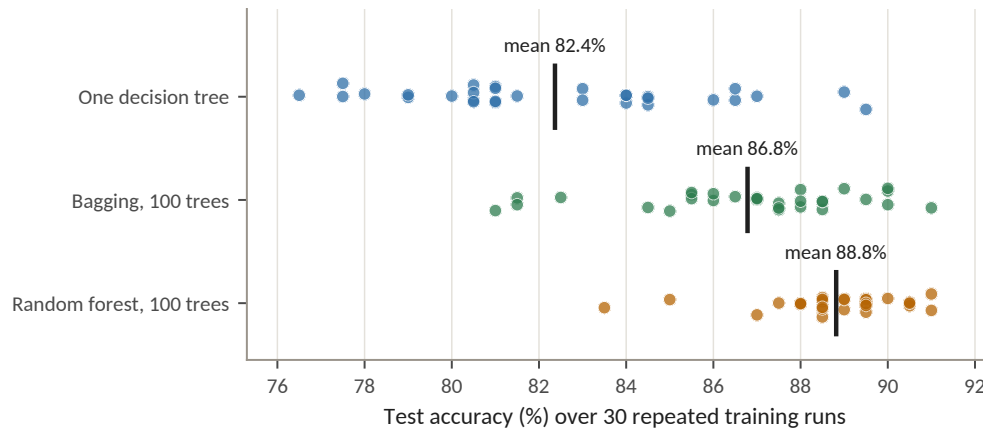


Figure 14.5 – Test accuracy of three models over 30 repeated training runs on resampled training data. Each dot is one trained model; the black bar marks the mean.

WHAT TO OBSERVE

Read it twice. Left to right, the means rise from 82.4% to 86.8% to 88.8%. Top to bottom, the dots pull in tighter: single trees are spread over 13 percentage points, bagging over 10, the forest over 7.5. Bagging bought most of the improvement and the random feature subsets bought the rest — in accuracy, and more visibly in steadiness.

COMMON MISTAKE

✗ Wrong: Assuming that every tree in a random forest is grown on the same data, or that hiding features must make the forest worse because it makes each tree worse.

✓ Right: Each tree gets its own bootstrap sample and, at every split, its own random handful of features. Individual trees do get weaker; the ensemble gets stronger, because more varied members leave less shared error behind.

14.5 Out-of-Bag Examples Give a Free Check

Look again at the “left out” column of Table 14.2. Tree 1 never saw D2 or D7, Tree 2 never saw D1 or D3, Tree 3 never saw D8. Those rows are not wasted: for each of them at least one tree sits where a test set puts a model, asked about an example it has genuinely never seen.

WORDS TO KNOW

Out-of-bag (OOB) example — For one tree, a training row its own bootstrap sample left out. That tree never learned from the row, so its prediction for it is honest.

Out-of-bag score — An accuracy estimate for the whole forest, made by predicting each training row with only the trees that left it out, then comparing those predictions with the true labels.

The recipe repeats three steps for every training row: find the trees whose bootstrap sample left it out; ask only those trees to predict it, combining their answers by majority vote; compare that vote with the true label. Rows that landed in every tree's sample are skipped.

EQUATION: OUT-OF-BAG SCORE

$$\text{OOB score} = \frac{c}{m}$$

In words: Count the training rows that at least one tree left out, count how many of those the left-out trees got right, and divide.

When it is used: As a quick, honest performance estimate for a forest without setting aside a validation set — especially valuable when data is scarce.

Symbol	Meaning	Unit or type
c	number of out-of-bag predictions that were correct	count
m	number of rows that received at least one out-of-bag prediction	count

m is much smaller than the training set in a small forest and almost equal to it in a large one. For regression, replace the count of correct answers with an error measure from Chapter 6.

Delivery	Tree 1	Tree 2	Tree 3	Judged by	OOB prediction	True label	Correct?
D1	in bag	OUT	in bag	Tree 2	On time	On time	yes
D2	OUT	in bag	in bag	Tree 1	Late	Late	yes
D3	in bag	OUT	in bag	Tree 2	Late	On time	NO
D4	in bag	in bag	in bag	none	none	Late	not tested
D5	in bag	in bag	in bag	none	none	On time	not tested
D6	in bag	in bag	in bag	none	none	Late	not tested
D7	OUT	in bag	in bag	Tree 1	On time	On time	yes
D8	in bag	in bag	OUT	Tree 3	Late	Late	yes

5 of the 8 deliveries could be judged and 4 of those 5 were right: out-of-bag score = $4 \div 5 = 0.80$

Figure 14.6 – Which trees are allowed to judge which deliveries. A tree may only judge a delivery marked OUT for it — one its own bootstrap sample left out.

WHAT TO OBSERVE

Only five of the eight rows carry an OUT mark; D4, D5 and D6 landed in all three bags and cannot be judged, which is expected with only three trees. With a hundred trees the chance of landing in every single bag is about 0.63^{100} , so almost every row gets judged.

WORKED EXAMPLE 14.7 — Compute the Mini-Forest's Out-of-Bag Score

Figure 14.6 shows which trees may judge which deliveries. Give each delivery its out-of-bag prediction and compute the forest's out-of-bag score.

1. D1 was left out only by Tree 2, which asks “parcels ≤ 13 ?”. D1 carries 6 parcels, so Tree 2 says On time: correct.
2. D2 was left out only by Tree 1, which asks “distance ≤ 20 km?”. D2 ran 42 km, so Tree 1 says Late: correct.
3. D3 was left out only by Tree 2. D3 carries 17 parcels, more than 13, so Tree 2 says Late while D3 was on time: **wrong** — it was a short 8 km run with a heavy load.
4. D7 was left out only by Tree 1. D7 ran 7 km, so Tree 1 says On time: correct.
5. D8 was left out only by Tree 3. D8 ran 44 km, so Tree 3 says Late: correct.
6. D4, D5 and D6 sat in every bootstrap sample, so no tree may judge them; they are skipped.
7. Count: $m = 5$ rows could be judged and $c = 4$ of them were right.

Answer: The out-of-bag score is $4 \div 5 = 0.80$, or 80%. Notice what caught the forest out: D3 could only be judged by the one tree forbidden to look at distance, and that tree's parcels rule does not fit a short, heavily loaded city route.

The out-of-bag score is a genuine convenience. Chapter 5 taught you to hold back a validation set before tuning anything; a forest hands you something similar for free, without giving up a single training row. It is not a replacement for a final untouched test set, though — treat it as a fast first check while you are still adjusting the model.

COMMON MISTAKE

X Wrong: Quoting an out-of-bag score from a forest with a handful of trees as if it carried the same weight as a large, separate test set.

✓ Right: A small forest leaves some rows in every bag, so its score rests on fewer rows and jumps around — 80% from five judged deliveries means very little. Use at least a hundred trees before leaning on one, and keep a test set for the final report.

14.6 Feature Importance in a Forest

Chapter 13 showed how one tree scores its features: measure the impurity each split removed, weight it by how many examples passed through that node, and turn each feature's total into a share of the tree's whole impurity drop. A forest does that once per tree, then averages the trees' scores.

EQUATION: FEATURE IMPORTANCE ACROSS A FOREST

$$\text{Imp}_{\text{forest}}(f) = \frac{1}{B} \sum_{b=1}^B \text{Imp}_b(f), \quad \text{Imp}_b(f) = \frac{\sum_{t \in T_f} D_t}{\sum_{t \in T} D_t}$$

In words: Inside each tree, a feature's importance is its share of that tree's total impurity drop; the forest's is the average of those shares over all the trees.

When it is used: After training, to rank the features by how much work the forest gave each one.

Symbol	Meaning	Unit or type
$\text{Imp}_b(f)$	tree b 's own importance score for feature f	share, between 0 and 1
D_t	impurity removed at node t , weighted by the node's size	impurity $\times$ count
T_f, T	nodes of tree b that split on f , and all its splitting nodes	sets of nodes
B	number of trees	count

Each tree's scores add up to 1 before averaging, so a large tree cannot outvote a small one, and the forest's scores also add up to 1. As in Chapter 13, $D_t = n_t I_t - n_L I_L - n_R I_R$.

WORKED EXAMPLE 14.8 — Compute Each Feature's Importance in the Mini-Forest

Each tree of Table 14.2 makes exactly one split, into two perfectly pure children. Find each tree's importance scores, then the forest's.

1. Tree 1's split: $D_t = 8 \times 0.46875 - 0 - 0 = 3.75$. It is the tree's only split, so distance takes the whole share: Tree 1 scores Distance 1, everything else 0.
2. Tree 2's split: $D_t = 8 \times 0.375 - 0 - 0 = 3.00$, its only split, so Tree 2 scores Parcels 1 and everything else 0.
3. Tree 3's split: $D_t = 8 \times 0.5 - 0 - 0 = 4.00$, its only split, so Tree 3 scores Distance 1 and everything else 0.
4. Average the Distance scores: $(1+0+1) \div 3 = 0.667$; average the Parcels scores: $(0+1+0) \div 3 = 0.333$.
5. Weather and Experience never appear in a split, so both score $0 \div 3 = 0$. Check: $0.667 + 0.333 = 1.000$.

Answer: Distance scores 0.667, Parcels 0.333, and Weather and Experience 0.000. Distance did twice the work of parcels — but only because two of the three trees were allowed to use it, which is a fact about our random draws as much as about deliveries.

That last remark points at a real weakness. Impurity importance measures how often a feature happened to be useful *inside this model*, and it can be fooled twice over: a feature that never got its turn looks useless, and one with many distinct values gets extra chances to look helpful, because more thresholds means more chances to shave off impurity by accident. Breiman proposed a second measure alongside random forests in 2001 that avoids both traps by testing the finished model instead of inspecting its splits.

WORD TO KNOW

Permutation importance — A feature's importance measured by damage: score the trained model, then shuffle that one feature's column so its values no longer match their rows, score the model again, and take the drop. A feature the model truly relies on causes a large drop; a feature it can do without causes none.

EQUATION: PERMUTATION IMPORTANCE OF ONE FEATURE

$$\text{PI}(f) = s - \frac{1}{R} \sum_{r=1}^R s_{f,r}$$

In words: Take the model's ordinary score and subtract the average score it gets after that one feature's column has been shuffled, repeating the shuffle several times so one unlucky shuffle cannot decide the answer.

When it is used: After training, to check how much a finished model really depends on each feature. Best computed on held-out data, so the answer describes new examples.

Symbol	Meaning	Unit or type
$PI(f)$	permutation importance of feature f	same unit as the score
s	the model's score on the untouched data	accuracy, or any metric
$s_{f,r}$	the score on shuffle r of feature f 's column	same unit as s
R	number of shuffles averaged	count

Scores near zero, and even slightly negative ones, both mean “the model does not need this feature”. Unlike impurity importance, these scores do not add up to 1.

WORKED EXAMPLE 14.9 — Find the Permutation Importance of Distance and of Parcels

The mini-forest gets all eight deliveries right, so its baseline accuracy is $8 \div 8 = 1.00$. Shuffle the distance column to 36, 42, 8, 30, 11, 5, 7, 44 km and re-score; then, starting again from the real data, shuffle the parcels column to 15, 6, 20, 17, 22, 7, 18, 12 and re-score.

- Two of the three trees ask about distance, so the majority always follows the distance rule: a shuffled distance of 20 km or less means the forest says On time.
- Work down the shuffled column. D1 now reads 36 km, so the forest says Late, but D1 was on time: wrong. D2 reads 42: Late, right. D3 reads 8: On time, right. D4 reads 30: Late, right. D5 reads 11: On time, right. D6 reads 5: On time, but D6 was late: wrong. D7 reads 7: On time, right. D8 reads 44: Late, right.
- Count: 6 of 8 correct, so the score after shuffling is 0.75, and $PI(\text{Distance}) = 1.00 - 0.75 = 0.25$.
- Now restore distance and shuffle parcels instead. Only Tree 2 uses parcels, so however its vote changes, Trees 1 and 3 still supply two votes of three and the majority never moves.
- Every delivery is still predicted correctly, so the score is again $8 \div 8 = 1.00$ and $PI(\text{Parcels}) = 0$.

Answer: Distance scores 0.25 and parcels 0.00 — even though impurity importance gave parcels a solid 0.333. Parcels is genuinely useful; it is simply not *needed*, because distance carries the same message and outvotes it. When two features tell the same story, permutation importance may credit neither.

COMMON MISTAKE

X Wrong: Reading a low importance score as proof that a feature does not matter in the real world.

✓ Right: Two features carrying the same information split the credit, or hand it all to whichever the model happened to use. Both measures describe this model on this data, not the world; Chapter 22 covers choosing features properly.

Figure 14.7 and Table 14.3 put the two measures side by side for a 100-tree forest grown on 400 deliveries, with driver experience deliberately left in as a column that has nothing to do with lateness in this simulated data.

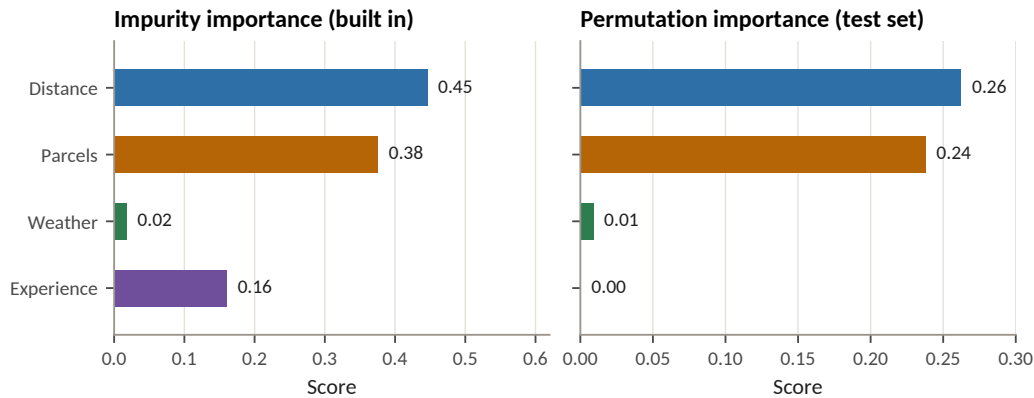


Figure 14.7 – Impurity importance (left) and permutation importance (right) for the same 100-tree forest.

WHAT TO OBSERVE

Distance and parcels lead on both measures and weather is small on both. Driver experience is the interesting one: impurity importance gives it 0.16, a bigger share than weather, while permutation importance gives it 0.00. Experience is a finely divided number, so it offered the trees thousands of thresholds and a few shaved off impurity by luck. Shuffling it costs the forest nothing, which settles the question.

Table 14.3 – Two importance measures for a 100-tree forest on 400 deliveries

Feature	Impurity importance	Permutation importance	Reading
Distance	0.45	0.26	Genuinely relied on
Parcels	0.38	0.24	Genuinely relied on
Weather	0.02	0.01	Small but real
Driver experience	0.16	0.00	Useless, but flattered

RESPONSIBLE USE

An importance score describes a model's own splits, never the real world, and never proves that a feature causes an outcome. Be especially careful when a column describes a person. If “driver experience” had earned a high score, the honest reading would still be that experienced drivers get different routes — not that any driver is at fault. Use importance to start a conversation and a human review, never to justify a penalty. Chapter 24 returns to this.

14.7 Tuning a Forest, and What It Cannot Do

A forest has fewer dials than most models in this book, and the most important behaves unusually well. Figure 14.8 shows accuracy as trees are added.

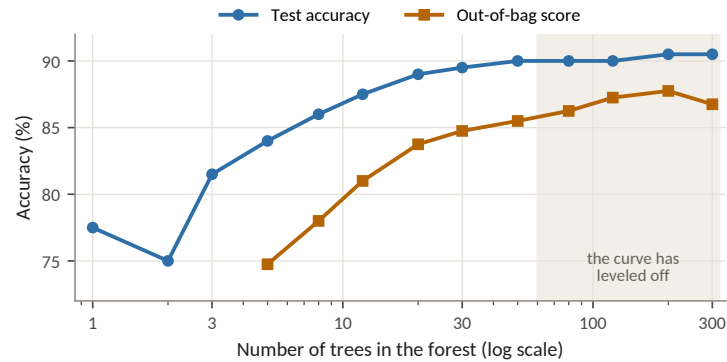


Figure 14.8 – Test accuracy and out-of-bag score as the forest grows from 1 to 300 trees.

WHAT TO OBSERVE

Both curves climb steeply and then flatten: almost all the benefit arrives by about 50 trees, and past 100 the line is level. Crucially, neither ever turns downward — more trees cost time and memory, not accuracy. The out-of-bag curve starts lower because with few trees hardly any row gets a fair judge, then climbs to meet the test curve.

That flat right-hand side is the most useful fact about forests. The number of trees is not a capacity dial in the sense of Chapter 7: each extra tree is one more independent opinion, and averaging more opinions cannot make the average wobble more. Set it generously — 100 to 500 is usual — and tune elsewhere. Table 14.4 lists the settings that repay attention, and the class weights of Chapter 6 are worth adding when one class is rare.

Table 14.4 – The main settings of a random forest

Setting	What it controls	Usual value	What happens if you raise it
Number of trees	How many trees are grown and averaged	100 to 500	Steadier predictions, slower training; never worse accuracy
Max features	How many features each split may choose from	$\sqrt{p}$ for classification	Trees get stronger but more alike; the forest loses variety
Max depth	How deep any one tree may grow	no limit	Deeper trees fit more detail; forests tolerate this far better than single trees
Min samples leaf	Smallest number of rows allowed in a leaf	1 to 5	Simpler trees, faster training, sometimes better on noisy data
Bootstrap	Whether each tree resamples its rows	on	Turning it off removes the out-of-bag check and most of the variety

Max features matters most. Offer too many and every tree reaches for the same strong column, pushing ρ up; offer too few and the trees become genuinely poor. Depth behaves differently here than in Chapter 13: one unpruned tree overfits badly, but a forest of unpruned trees usually does not, because each overfits its own bootstrap sample in its own direction and averaging cancels most of it. Table 14.5 shows the progression on the same 400 training and 200 test deliveries.

Table 14.5 – Four models on the same 400 training and 200 test deliveries

Model	Training accuracy (%)	Test accuracy (%)	Out-of-bag score (%)
One fully grown tree	100.0	84.0	—
Bagging, 100 trees	100.0	89.0	—
Random forest, 100 trees	100.0	90.0	88.2
Random forest, max depth 4	90.8	89.0	—

Every row except the last reaches a perfect 100% on its training rows, so training accuracy tells you nothing, exactly as Chapter 13 warned. The test column tells the story: bagging adds five points over one tree, feature randomness adds one more, and capping every tree at depth 4 gives a point back. The forest's out-of-bag score of 88.2% also lands close to its test score of 90.0%, and a little below it — the safe direction to be wrong in.

One relative is worth naming. Extremely randomized trees, published by Pierre Geurts, Damien Ernst and Louis Wehenkel in 2006, push the idea further: instead of searching for the best threshold among the allowed features, they pick thresholds at random and keep the best of those. Each tree is worse still, the forest is often just as accurate, and training is much faster.

Forests are not magic, and four limits are worth stating plainly. A forest cannot be read aloud: one tree of Chapter 13 becomes three sentences a technician can memorize, while a hundred trees become nothing a person can hold in their head. It inherits every tree's inability to extend a trend, so its prediction goes flat beyond the training range and a 90 km delivery gets the same answer as a 60 km one. It is bigger and slower to train and to run — rarely a problem for a table of this size, sometimes a problem on a phone or a sensor. And trained on badly unbalanced classes it will vote for the common class nearly every time, so read Chapter 6 before trusting its accuracy.

There is also a different way to combine trees: grow them one at a time, each aimed at the mistakes the previous ones still make. That is boosting, the subject of Chapter 15; where bagging attacks variance, boosting attacks bias.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing that any collection of models beats a single model, even when the members are all weak or nearly identical.
- Thinking a bootstrap sample is the training set reused, instead of a resample drawn with replacement that repeats some rows and drops about 37% of them — or that a forest's trees are all trained on the same data.
- Assuming that restricting the features each split may use must make the forest worse, because it makes each tree worse.
- Treating an out-of-bag score from a forest of a few trees as being as trustworthy as a large, separate test set.
- Believing that adding more trees can make a forest overfit, the way extra depth overfits a single tree.
- Reading a high feature importance as proof of cause, or a low one as proof that a feature is useless, when correlated features split the credit — and trusting impurity importance for finely divided numerical features, which it flatters.
- Reporting only the forest's winning label and throwing away the vote share that says how much the trees disagreed.

Practical Activities

ACTIVITY (PAPER) — Draw Bootstrap Samples with Eight Paper Slips

1. Write D1 to D8 on eight identical slips of paper and put them all in a cup.
2. Draw one slip, write its name in a row, and *put it back*. Repeat until you have eight names: that is one bootstrap sample. Underneath it, list the deliveries that never appeared and count them.
3. Repeat step 2 four more times, then average your five “left out” counts. The predicted average is $8 \times 0.3436 = 2.75$. Five rounds is a small experiment, so compare with a neighbor and average all ten rounds together.
4. From your first sample, copy each drawn delivery’s row out of Table 14.1. Toss a coin twice to choose two features (HH = Distance and Parcels, HT = Distance and Weather, TH = Parcels and Experience, TT = Weather and Experience). Using only those two, find the split with the lowest weighted Gini impurity, the way Chapter 13 did.
5. Write your tree’s one-line rule and use it to vote on Delivery Q (25 km, 11 parcels, clear weather, 6 years). Count the votes of everyone in the class: that is a random forest, grown by hand.

You need: Paper, pencil, a calculator, eight paper slips, a cup and a coin

ACTIVITY (PYTHON) — Grow the Same Forest with scikit-learn

Part 1 repeats Worked Examples 14.4, 14.5 and 14.6 on the eight deliveries of Table 14.1.

Python · NumPy · scikit-learn

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
# The eight deliveries of Table 14.1 (weather: 1 = Rain, late: 1 = Yes)
X = np.array([[5, 6, 0, 8], [42, 22, 1, 2], [8, 17, 0, 5], [36, 18, 0, 6],
              [11, 7, 0, 9], [30, 20, 1, 4], [7, 12, 1, 3], [44, 15, 1, 7]])
y = np.array([0, 1, 0, 1, 0, 1, 0, 1])
print("chance a delivery is left out:", round((1 - 1 / 8) ** 8, 4))
Q, votes = np.array([[25, 11, 0, 6]]), []
for rows, f in zip([[3,1,5,8,3,6,1,4], [2,2,2,4,5,6,7,8], [1,2,3,4,4,5,6,7]],
                  [[0,2], [1,2], [0,3]]):
    r = np.array(rows) - 1
    stump = DecisionTreeClassifier(max_depth=1).fit(X[np.ix_(r, f)], y[r])
    votes.append(int(stump.predict(Q[:, f])[0]))
    print("tree on columns", f, "cuts at", round(stump.tree_.threshold[0], 1),
          "-> votes", "Late" if votes[-1] else "On time")
print("mini-forest:", sum(votes), "of 3 trees vote Late")
```

Output

```
chance a delivery is left out: 0.3436
tree on columns [0, 2] cuts at 20.5 -> votes Late
tree on columns [1, 2] cuts at 13.5 -> votes On time
tree on columns [0, 3] cuts at 20.5 -> votes Late
mini-forest: 2 of 3 trees vote Late
```

The thresholds read 20.5 and 13.5 rather than 20 and 13 because scikit-learn cuts halfway between the two nearest rows, exactly as Chapter 13 explained; the rules are the same, and the vote is again 2 to 1

for Late. Part 2 grows a real forest on `datasets/ch14_deliveries.csv`, which holds 600 deliveries in the columns `distance_km`, `parcels`, `rain`, `driver_experience_years`, `late` and `set`.

Python · pandas · scikit-learn

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import accuracy_score

d = pd.read_csv("datasets/ch14_deliveries.csv")
c = ["distance_km", "parcels", "rain", "driver_experience_years"]
tr, te = d[d.set == "train"], d[d.set == "test"]
rf = RandomForestClassifier(n_estimators=100, oob_score=True,
                           random_state=0).fit(tr[c], tr.late)
print("forest test %:", round(accuracy_score(te.late, rf.predict(te[c])) * 100, 1),
      " out-of-bag %:", round(rf.oob_score_ * 100, 1))
print("impurity importance  :", rf.feature_importances_.round(3))
pi = permutation_importance(rf, te[c], te.late, n_repeats=20, random_state=0)
print("permutation importance:", pi.importances_mean.round(3))
```

Output

```
forest test %: 90.0  out-of-bag %: 88.2
impurity importance  : [0.446 0.376 0.018 0.16 ]
permutation importance: [0.262 0.238 0.009 0.   ]
```

Part 2 reproduces the forest row of Table 14.5 and both columns of Table 14.3. Try `n_estimators` of 1, 10 and 200 and watch the test score climb and flatten as in Figure 14.8, then set `max_features=None` to turn the feature randomness off.

You need: Python with NumPy, pandas and scikit-learn (see Appendix B), and `datasets/ch14_deliveries.csv`

CASE STUDY — Flagging Late-Delivery Risk at Sunrise Parcels

Rawa manages dispatch for Sunrise Parcels, the company whose eight deliveries run through this chapter. Today a delivery becomes a problem only once it is already late and a customer telephones to complain. Rawa wants to know before the van leaves the depot, while there is still time to move a parcel to another route, send a second driver, or simply call the customer and give them a real arrival time.

Her analyst, Karwan, starts with a single decision tree trained on one season of past deliveries. It is easy to explain in a dispatch meeting, but its predictions shift depending on which month it trains on — the jumpiness this chapter opened with. A forest of two hundred trees steadies them at once, and because Sunrise Parcels has only a few hundred usable records, Karwan leans on the out-of-bag score while trying settings, keeping a small untouched test set for one final check. The forest scores a few points above the single tree there, and scores about the same when he repeats the exercise a month later.

The importance scores start a useful argument rather than ending one. Distance and parcel count lead comfortably, weather earns a small but real score, and driver experience scores near zero on permutation importance despite a middling impurity score. Karwan reads that as a warning about the measure, not a discovery about drivers, and the team goes further: they drop the driver identity column entirely. A model that learns which driver is on a route mostly learns which routes each driver is given, and it would quietly turn “this driver gets the hardest routes” into “this driver is a risk”.

What ships is deliberately modest: every morning the forest produces a short list of the day's riskiest deliveries, each carrying its vote share, so a dispatcher can tell "97 of 100 trees" from "54 of 100 trees" and treat the second as a question, not a verdict. Three months on, Rawa reports not a dramatic rescue but a steady reduction in surprise — the same drivers, the same vans, far fewer apologetic calls.

IMPORTANT

An ensemble is usually more accurate than a single tree, and always harder to explain, because no one tree tells the whole story any more. When a prediction affects a person's opportunity — a loan, a job, a driver's route — that lost explanation is a real cost. Choose the simplest model that meets the project's accuracy need, and if a forest is that model, publish its vote shares and its limits alongside its predictions.

CHAPTER SUMMARY

- An ensemble combines many models' predictions, by voting for a category or averaging for a number, so their different mistakes cancel instead of piling up.
- Averaging B predictions leaves $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ of the original wobble, so making the members *different* matters as much as adding more of them.
- A bootstrap sample is drawn from the training set with replacement and is the same size, so it repeats some rows and leaves others out; the chance a given row is left out is $(1 - 1/n)^n$, which settles near 0.368 for any dataset bigger than a few dozen rows.
- Bagging trains many copies of a model, each on its own bootstrap sample, then combines them; it mainly reduces variance.
- A random forest is bagging applied to decision trees, plus a second source of variety: every split may choose only from a random subset of the features. It predicts by majority vote or by averaging, and the winning vote share is worth reporting as a rough confidence level.
- Out-of-bag rows are the training rows a tree's own bootstrap sample left out; scoring each row with only the trees that left it out gives a free, honest check that needs enough trees to be reliable.
- Impurity importance is each tree's own share-of-impurity score averaged across the forest; permutation importance shuffles one column and measures the damage. The two disagree in useful ways, and neither proves cause.
- Adding trees to a forest never makes it overfit; it costs time and memory, and the benefit flattens after roughly 100 trees.
- A forest cannot be read aloud like one tree, cannot extend a trend beyond its training range, and is slower and larger; growing trees in sequence instead gives the boosting of Chapter 15, which attacks bias where bagging attacks variance.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define "ensemble" in your own words, using an everyday example not from this chapter.

Problem 2 [Basic] Explain the difference between a bootstrap sample and the training set it came from, and what "with replacement" means.

Problem 3 [Basic] A random forest has 150 trees. How does it turn 150 predictions into one, and what extra number should it report alongside?

Problem 4 [Basic] Name the two sources of randomness in a random forest and say what each is for.

Problem 5 [Standard] A forest draws 6 rows with replacement from a training set of 6 rows numbered 1 to 6. The draws are 2, 4, 4, 1, 6, 2. List the distinct rows in the sample, the rows left out, and those that repeat.

Problem 6 [Standard] Five trees predict a delivery will be 12, 20, 16, 28 and 24 minutes late; it really ran 20 minutes late. Find the average prediction, its error, and the average of the five individual errors.

Problem 7 [Standard] A training set has 20 rows. Calculate the chance one row is left out of a bootstrap sample, showing every step, and estimate how many of the 20 will be out of bag.

Problem 8 [Standard] A single tree's prediction has variance $\sigma^2 = 100$. Find the variance of an average of 25 trees (a) if they were unrelated and (b) if $\rho = 0.4$, giving both as standard deviations too.

Problem 9 [Standard] Tree 1's own importance scores are Distance 0.6, Parcels 0.4; Tree 2's are 0.2 and 0.8; Tree 3's are 1.0 and 0.0. Find the forest's importance for each feature and check they add to 1.

Problem 10 [Challenge] A forest of 8 trees reports an out-of-bag score of 88% and a test score of 74% on 300 held-out rows. (a) Give two reasons the out-of-bag number could be higher. (b) Which number would you report to a manager, and what would you change before trusting the other?

Problem 11 [Challenge] A team adds "route identity number", a different whole number for each of 500 routes. Its impurity importance is high but its permutation importance is near zero. (a) Explain both results. (b) Should it stay in the model?

Problem 12 [Challenge] Using the equation of Lesson 14.1, explain why a forest whose trees may all use every feature stops improving as thousands of trees are added, while one with `max_features` set to 2 keeps improving longer. Refer to both terms.

Problem 13 [Challenge] A hospital in Sulaymaniyah wants a forest to flag patients for a follow-up call, using age, past visits and test results. (a) Name one feature you would be cautious about, and the risk. (b) How would you use feature importance responsibly here? (c) Name one thing a forest gives up compared with the tree of Chapter 13, and why that matters more here than at a parcel depot.

Boosting and Stacking

Building a strong model from many weak ones, one correction at a time

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what a weak learner is, and why models trained one after another can beat models trained side by side.
- Compute AdaBoost's error rate, learner weight and new sample weights by hand for a small dataset.
- Combine several weighted learners into one final prediction.
- Run gradient boosting for two rounds by hand, using residuals, a stump and a learning rate.
- Choose the number of rounds by early stopping, and trade a smaller learning rate against more rounds.
- Describe in plain words what modern boosting libraries add to the same idea.
- Combine finished models by hard voting, soft voting and stacking, and choose between bagging, boosting and stacking.

Introduction

A tailor in Sulaymaniyah takes your measurements and cuts a jacket. At the first fitting it is close but not right: one sleeve is a little long, and it pulls across the shoulder. She does not throw the jacket away and start again from fresh cloth. She marks only what is still wrong, and makes exactly those corrections. At the second fitting the sleeve is right and the shoulder is better, but the collar sits high, so she marks that. Three fittings later the jacket fits. Each fitting alone was a small, almost trivial piece of work; added together, they produced something no single cut could have produced.

Chapter 14 built an ensemble the other way around. A random forest grows hundreds of trees at the same time, each on its own bootstrap sample, and lets them vote; every tree does the whole job independently. This chapter grows the models one at a time instead, in a strict order, and gives every new model a much smaller job: fix what the models before it are still getting wrong. That is boosting, and it is behind a large share of the winning models in competitions on ordinary tables of data.

You will work two boosting methods entirely by hand: AdaBoost, which makes the missed examples heavier so the next model pays attention to them, and gradient boosting, which fits each new tree directly to the errors that are left. You will then meet two ways of combining models that are not boosting at all — voting and stacking — and finish with a table that says which family to reach for first.

15.1 Weak Learners and the Boosting Idea

Sunrise Parcels, the delivery company of Chapter 14, now has a forest that flags risky deliveries. Karwan, the analyst, starts from an odd-looking question: instead of building one strong model, what happens if you build a long chain of almost useless ones?

WORDS TO KNOW

Weak learner — A model only slightly better than guessing — say 55% accurate where guessing gives 50%. Far too poor to use alone.

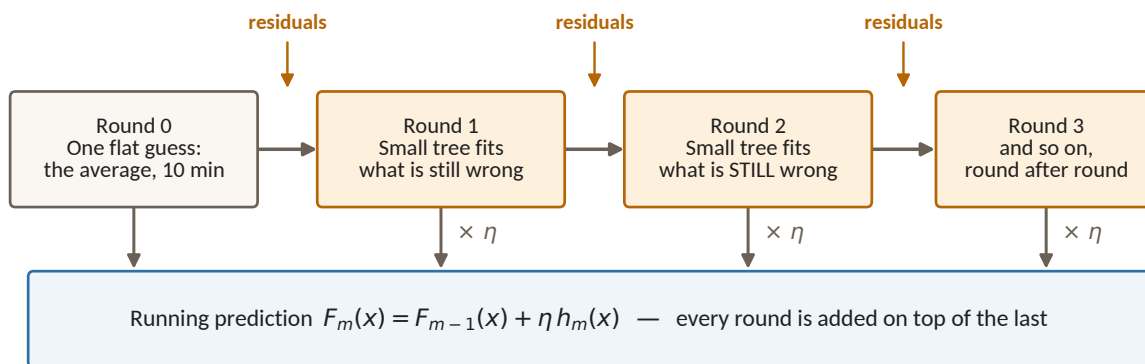
Decision stump — The smallest possible decision tree: one question, two leaves. It is the most common weak learner in this chapter, and Chapter 13 showed how to choose its single split.

In 1990 the computer scientist Robert Schapire proved something surprising about weak learners. If you can build a model that is even slightly better than guessing, again and again on cleverly re-arranged versions of the data, then those weak models can be combined into a model that is as accurate as you like: weak learnability and strong learnability are the same thing. Seven years later, in 1997, Yoav Freund and Robert Schapire turned the proof into a practical algorithm, AdaBoost, the subject of the next lesson.

WORD TO KNOW

Boosting — Building an ensemble by training models one at a time, in order, where every new model concentrates on the examples the models before it are still getting wrong. The models' outputs are then added up, usually after each one is scaled down.

The word “order” carries all the weight here. A forest’s trees can be grown in any sequence, or all at once on different computers, because no tree knows anything about the others; they can even be shuffled or thinned afterward. In boosting, model 7 cannot be built until models 1 to 6 exist, because model 7’s whole job is defined by what those six are still missing, and dropping one from the middle breaks the chain (Figure 15.1).



The finished model is the whole chain added together, never one tree on its own.

Figure 15.1 – Boosting builds a chain. Round 0 is one flat guess; every later round trains a small model on what is still wrong and adds a scaled-down version of it to the running prediction.

WHAT TO OBSERVE

Nothing in the chain is ever thrown away or retrained. Each round adds a small correction on top of everything already there, exactly like the tailor’s fittings. The multiplier η on every arrow keeps each correction small, and Lesson 15.4 is entirely about choosing it.

15.2 AdaBoost: Re-weighting the Mistakes

AdaBoost's trick is to change how much each training row matters. Every row starts equally important. After each weak learner is trained, the rows it got wrong become heavier and the rows it got right become lighter, so the next learner is pushed toward the hard cases — like a teacher who re-reads the questions a class got wrong rather than the ones everybody already answers correctly.

WORDS TO KNOW

Sample weight — A number attached to one training row saying how much that row matters while a model is being trained. All the weights add up to 1.

Weighted error rate — The share of the total weight sitting on the rows a learner got wrong. With equal weights it is just the fraction misclassified; with unequal weights, missing one heavy row costs more than missing several light ones.

Karwan takes six deliveries out of the Sunrise Parcels log, keeping two features — distance and the number of parcels on the van — and one label: did it arrive late? Table 15.1 holds those six deliveries and also the answers we are about to compute, so you can check every step against it as we go.

Table 15.1 – Six Sunrise Parcels deliveries, and how AdaBoost's weights move over three rounds

Delivery	A1	A2	A3	A4	A5	A6
Distance (km)	45	40	8	15	10	50
Parcels	9	3	11	14	5	1
Arrived	Late	Late	Late	On time	On time	On time
Weight, round 1	0.167	0.167	0.167	0.167	0.167	0.167
Stump 1 (distance)	right	right	missed	right	right	missed
Weight, round 2	0.125	0.125	0.250	0.125	0.125	0.250
Stump 2 (parcels)	right	missed	right	missed	right	right
Weight, round 3	0.083	0.250	0.167	0.250	0.083	0.167

Two deliveries break the obvious pattern: A3 traveled only 8 km but carried 11 parcels and was still late, and A6 traveled 50 km with a single parcel and arrived on time. Those two are why no stump can get all six right, and they are exactly the rows AdaBoost is built to chase. The two formulas below turn a stump's mistakes into two numbers: how bad it was, and how much its opinion should count.

EQUATION: ADABOOST: WEIGHTED ERROR RATE AND LEARNER WEIGHT

$$\varepsilon_m = \sum_{i: h_m(x_i) \neq y_i} w_i, \quad \alpha_m = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

In words: Add up the weights of every row this learner got wrong; that is its error rate. Then turn the error rate into a learner weight: divide the share it got right by the share it got wrong, take the natural logarithm, and halve the result.

When it is used: Once per round, immediately after a weak learner has been trained, to decide how loudly that learner may speak in the final vote.

Symbol	Meaning	Unit or type
ε_m	weighted error rate of learner m	between 0 and 1
α_m	weight (trust) given to learner m	a number, larger is better
w_i	current sample weight of training row i	between 0 and 1
$h_m(x_i), y_i$	learner m 's prediction for row i , and row i 's true label	+1 or -1

A learner with $\varepsilon_m = 0.5$ is pure guessing and earns $\alpha_m = \frac{1}{2} \ln 1 = 0$: no vote at all. Below 0.5 the weight is positive and grows quickly as the error falls.

The learner weight does a second job: it sets how hard the sample weights move. Rows the learner missed are multiplied by $e^{+\alpha_m}$, which is larger than 1; rows it got right are multiplied by $e^{-\alpha_m}$, which is smaller than 1. Dividing by the new total keeps the weights adding to 1.

EQUATION: ADABOOST: THE SAMPLE-WEIGHT UPDATE

$$w_i^{\text{new}} = \frac{w_i \times e^{-\alpha_m y_i h_m(x_i)}}{Z_m}, \quad Z_m = \sum_{j=1}^n w_j \times e^{-\alpha_m y_j h_m(x_j)}$$

In words: Multiply a row's weight by $e^{+\alpha_m}$ if the learner got that row wrong and by $e^{-\alpha_m}$ if it got the row right, then divide every weight by the total so that they add up to one again.

When it is used: Once per round, right after α_m has been found, to prepare the weights the next weak learner will train on.

Symbol	Meaning	Unit or type
w_i^{new}	row i 's weight going into the next round	between 0 and 1
$y_i h_m(x_i)$	+1 when the learner was right, -1 when it was wrong	+1 or -1
Z_m	the total of all the multiplied weights, used to rescale them	a number
n	number of training rows	count

After every update the rows the learner missed always carry exactly half of the total weight, no matter what ε_m was. Check this in Table 15.1: $0.250 + 0.250 = 0.5$ after round 1, and again after round 2.

WORKED EXAMPLE 15.1 — AdaBoost Round One, from Start to Finish

Every one of the six deliveries starts with the same weight, $1 \div 6 = 0.167$. The first stump asks “is the distance 30 km or more?” and answers Late if yes, On time if no. Find its error rate, its learner weight, and the six new weights. Use $\ln 2 = 0.693$, $e^{+0.347} = 1.414$ and $e^{-0.347} = 0.707$.

1. Apply the stump to all six. The long trips A1 (45 km) and A2 (40 km) are called Late and really were late; the short trips A4 (15 km) and A5 (10 km) are called On time and really were on time. But A3 (8 km) is called On time and was Late, and A6 (50 km) is called Late and was on time. Two mistakes: A3 and A6.
2. Add the weights of the wrong rows: $0.167 + 0.167 = 0.333$. So $\varepsilon_1 = 0.333$, exactly $2 \div 6$.
3. Form the ratio inside the logarithm: $(1 - 0.333) \div 0.333 = 0.667 \div 0.333 = 2$.
4. Take the logarithm and halve it: $\alpha_1 = \frac{1}{2} \times \ln 2 = \frac{1}{2} \times 0.693 = 0.347$.
5. Multiply each missed row's weight by 1.414: $0.167 \times 1.414 = 0.236$, for A3 and for A6. Multiply each correct row's weight by 0.707: $0.167 \times 0.707 = 0.118$, for each of A1, A2, A4 and A5.
6. Add all six to get Z_1 : $0.236 + 0.236 + 4 \times 0.118 = 0.472 + 0.472 = 0.943$.

7. Divide by Z_1 : each missed row becomes $0.236 \div 0.943 = 0.250$, each correct row becomes $0.118 \div 0.943 = 0.125$. Check: $2 \times 0.250 + 4 \times 0.125 = 1.000$.

Answer: $\varepsilon_1 = 0.333$ and $\alpha_1 = 0.347$: **wrong one time in three — useless alone, but better than guessing, so it earns a positive vote. A3 and A6 now weigh 0.250 each and the other four 0.125 each, so the two missed deliveries hold exactly half of the attention going into round 2.**

The next stump now trains on these new weights; Figure 15.2, at the end of this lesson, shows all three rounds side by side.

WORKED EXAMPLE 15.2 — Round Two: the Same Two Mistakes Now Cost More

With the new weights, the second stump asks “are there 7 parcels or more?” and answers Late if yes. It gets A3 and A6 right this time, but misses A2 (40 km, only 3 parcels, yet late) and A4 (15 km, 14 parcels, yet on time). Find ε_2 and α_2 , then the round-3 weights. Use $\ln 3 = 1.099$, $e^{+0.549} = 1.732$ and $e^{-0.549} = 0.577$.

1. Add the weights of the two missed rows, A2 and A4, from Table 15.1: $0.125 + 0.125 = 0.250$, so $\varepsilon_2 = 0.250$.
2. Form the ratio: $(1 - 0.25) \div 0.25 = 3$, so $\alpha_2 = \frac{1}{2} \times \ln 3 = \frac{1}{2} \times 1.099 = 0.549$.
3. Multiply the two missed rows by 1.732: $0.125 \times 1.732 = 0.217$ each. Multiply the four correct rows by 0.577: A1 and A5 give $0.125 \times 0.577 = 0.072$, and A3 and A6 give $0.250 \times 0.577 = 0.144$.
4. Add them all: $2 \times 0.217 + 2 \times 0.072 + 2 \times 0.144 = 0.433 + 0.144 + 0.289 = 0.866$.
5. Divide by 0.866: the missed rows become 0.250, A1 and A5 become $0.072 \div 0.866 = 0.083$, and A3 and A6 become $0.144 \div 0.866 = 0.167$.

Answer: $\varepsilon_2 = 0.250$ and $\alpha_2 = 0.549$, so this second stump is trusted about 1.6 times as much as the first. Notice that the same two mistakes would have scored $2 \div 6 = 0.333$ under the old equal weights; it is the weighting, not the counting, that has changed.

Once the rounds are finished, every learner votes, but the votes are not equal. Each vote is multiplied by that learner’s own α_m , and the sign of the total decides the answer. Note that α_m is a score, not a share: it has no upper limit and the α values never add up to 1.

EQUATION: ADABOOST: THE FINAL WEIGHTED VOTE

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

In words: Ask every weak learner for a +1 or −1 answer, multiply each answer by that learner’s weight, add everything up, and read off the sign: positive means the first class, negative means the second.

When it is used: Every time the finished AdaBoost model predicts for a new example.

Symbol	Meaning	Unit or type
$H(x)$	the ensemble’s final prediction for example x	+1 or −1
M	number of rounds (weak learners)	count
α_m	weight of learner m , from the first equation box	a number
$h_m(x)$	learner m ’s own prediction for x	+1 or −1

The size of the total, not only its sign, is useful: a total close to zero means the learners nearly cancelled out, which is AdaBoost’s version of “not confident”.

WORKED EXAMPLE 15.3 — Combine Three Stumps into One Weighted Vote

The run keeps three stumps. Stump 1: distance ≥ 30 km $\rightarrow$ Late, $\alpha_1 = 0.347$. Stump 2: parcels $\geq 7 \rightarrow$ Late, $\alpha_2 = 0.549$. Stump 3: parcels $\geq 2 \rightarrow$ Late, $\alpha_3 = 0.347$. Write Late as +1 and On time as -1. Predict for delivery Q (35 km, 4 parcels) and delivery R (12 km, 1 parcel).

1. Delivery Q: $35 \geq 30$, so stump 1 says Late, +1; $4 < 7$, so stump 2 says On time, -1; $4 \geq 2$, so stump 3 says Late, +1.
2. Multiply and add for Q: $0.347 - 0.549 + 0.347 = +0.145$. The total is positive, so $H(Q) = +1$: Late.
3. Delivery R: $12 < 30$, $1 < 7$ and $1 < 2$, so all three stumps say On time, -1.
4. Multiply and add for R: $-0.347 - 0.549 - 0.347 = -1.243$: On time.

Answer: Both get an answer, but with very different confidence. Q is Late by only +0.145 out of a possible 1.243 — the three stumps almost cancelled out. R is On time by the full -1.243, because every stump agreed.

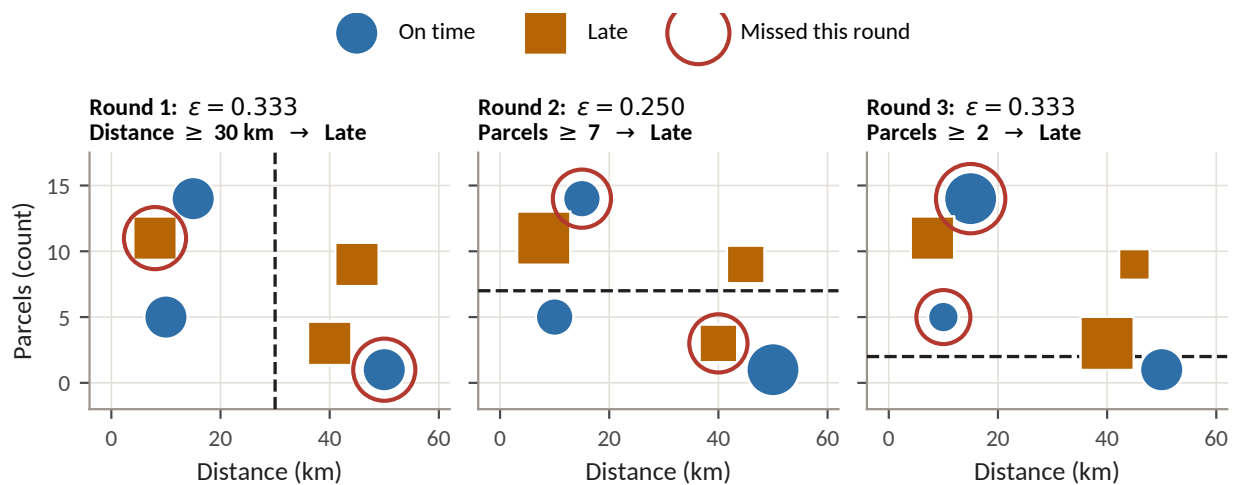


Figure 15.2 – Three rounds of AdaBoost on the six deliveries of Table 15.1. Marker area shows the current sample weight; the dashed line is the stump that round keeps; red rings mark the deliveries that stump missed.

WHAT TO OBSERVE

In round 1 every marker is the same size. By round 3 the delivery at 15 km with 14 parcels is the biggest marker on the page: missed twice, so its weight has grown twice. That is what “paying attention to the hard cases” looks like — and also the danger, because a mislabeled row grows in exactly the same way.

15.3 Gradient Boosting: Fitting the Residuals

AdaBoost changes the weights on the rows. Gradient boosting, published by the statistician Jerome Friedman in 2001, does something easier to picture: it changes the *target*. Each new tree is trained not on the original answer, but on the part of the answer that is still missing.

Dispatch now wants a number rather than a category: how many minutes late will this delivery run? Table 15.2 holds five deliveries, each with its distance and how late it actually was.

Table 15.2 – Five Sunrise Parcels deliveries, with a number to predict instead of a category

Delivery	E1	E2	E3	E4	E5
Distance (km)	8	12	16	44	50
Minutes late	2	4	6	18	20

Chapter 8 already named the gap between a true value and a predicted one: the residual, $y - \hat{y}$. Gradient boosting is built entirely out of residuals. Start with one flat guess for everybody — the average — measure how wrong it is for each row, and fit a small tree whose job is to predict those wrongnesses. Add a fraction of that tree's output to the running guess, measure the residuals again, and repeat.

WORDS TO KNOW

Gradient boosting — Boosting where each new model is fitted to the residuals left by the running prediction, and a scaled-down version of its output is added to that running prediction.

Learning rate (η) — The fraction of each new model's output that is actually added, usually between 0.01 and 0.3. It plays the same role as the learning rate of Chapter 9: small steps are slower but safer.

EQUATION: GRADIENT BOOSTING: START, RESIDUAL AND RUNNING UPDATE

$$F_0(x) = \bar{y}, \quad r_i^{(m)} = y_i - F_{m-1}(x_i), \quad F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

In words: Start by predicting the average of the training target for everybody. Before each round, find how wrong the running prediction still is for every row. Train a small tree on those numbers, and add the learning rate times that tree's output to the running prediction.

When it is used: Every round of a gradient-boosting regression model, from the first round to the last.

Symbol	Meaning	Unit or type
$F_m(x)$	the running prediction after m rounds	unit of the target
$\bar{y}$	the mean of the training target, used as round 0	unit of the target
$r_i^{(m)}$	residual of row i before round m	unit of the target
$h_m(x)$	round m 's small tree, trained on those residuals	unit of the target
η	the learning rate	a number, usually well below 1

A regression tree's leaf predicts the average of the values inside it (Chapter 13), so a stump trained on residuals simply predicts each group's average residual.

WORKED EXAMPLE 15.4 — Gradient Boosting: Round Zero and Round One

Using the five deliveries of Table 15.2, start with the average and run one round. This round's stump splits on "distance 30 km or less?" and the learning rate is $\eta = 0.5$.

1. Round 0. Add the five true values and divide by 5: $2 + 4 + 6 + 18 + 20 = 50$, and $50 \div 5 = 10$. Every delivery starts with a predicted 10 minutes late.
2. Find each residual, true minus predicted: $2 - 10 = -8$, $4 - 10 = -6$, $6 - 10 = -4$, $18 - 10 = 8$, $20 - 10 = 10$.
3. Sort by the split. Short trips (E1, E2, E3 at 8, 12 and 16 km) have residuals -8 , -6 , -4 . Long trips (E4, E5 at 44 and 50 km) have residuals 8 and 10.
4. Each leaf predicts the average residual inside it: $(-8 - 6 - 4) \div 3 = -18 \div 3 = -6$ for the short leaf, and $(8 + 10) \div 2 = 18 \div 2 = 9$ for the long leaf.
5. Update the short trips: $10 + 0.5 \times (-6) = 10 - 3 = 7$ minutes.

6. Update the long trips: $10 + 0.5 \times 9 = 10 + 4.5 = 14.5$ minutes.

Answer: After round 1 the predictions are 7 minutes for E1, E2 and E3, and 14.5 minutes for E4 and E5. The mean absolute error has fallen from $36 \div 5 = 7.2$ minutes to $18 \div 5 = 3.6$ minutes — exactly halved, because $\eta = 0.5$ removed half of every residual in a group whose residuals all had one sign.

WORKED EXAMPLE 15.5 — Gradient Boosting: Round Two

Run a second round on the same five deliveries, with the same split and the same learning rate.

1. Find the new residuals against the round-1 predictions: $2-7 = -5$, $4-7 = -3$, $6-7 = -1$, $18-14.5 = 3.5$, $20-14.5 = 5.5$.
2. The short leaf's value: $(-5-3-1) \div 3 = -9 \div 3 = -3$. The long leaf's value: $(3.5+5.5) \div 2 = 9 \div 2 = 4.5$.
3. Update the short trips: $7 + 0.5 \times (-3) = 7 - 1.5 = 5.5$ minutes.
4. Update the long trips: $14.5 + 0.5 \times 4.5 = 14.5 + 2.25 = 16.75$ minutes.
5. New residuals: -3.5 , -1.5 , $+0.5$, $+1.25$, $+3.25$, whose sizes add to 10, so the mean absolute error is $10 \div 5 = 2.0$ minutes.

Answer: After round 2 the predictions are 5.5 and 16.75 minutes, and the mean absolute error has fallen from 7.2 to 3.6 to 2.0 minutes. Two stumps, each of which alone would be a laughably crude model, have between them cut the error to under a third.

Table 15.3 collects all three rounds in one place, and Figure 15.3 draws the same numbers as bars.

Table 15.3 – Three rounds of gradient boosting on the five deliveries, with $\eta = 0.5$

Round	Prediction, E1-E3 (min)	Prediction, E4-E5 (min)	Residuals, E1-E3	Residuals, E4-E5	MAE (min)
0	10	10	-8, -6, -4	8, 10	7.2
1	7	14.5	-5, -3, -1	3.5, 5.5	3.6
2	5.5	16.75	-3.5, -1.5, 0.5	1.25, 3.25	2.0

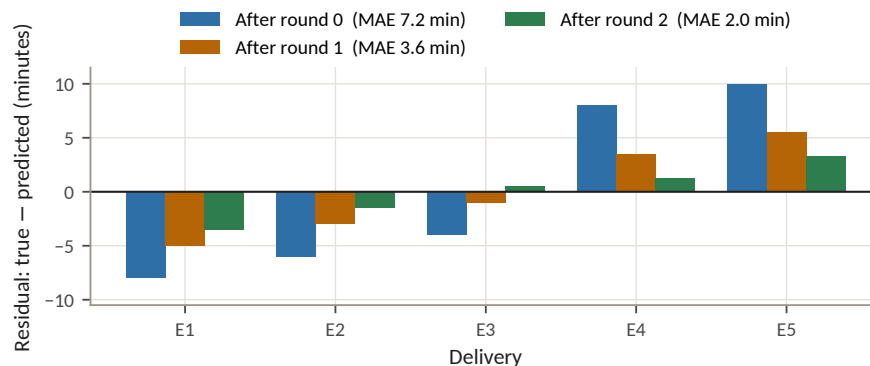


Figure 15.3 – The residuals of the five deliveries after each round. Bars above the line mean the model is predicting too low; bars below mean it is predicting too high.

WHAT TO OBSERVE

Every bar shrinks toward the zero line, but not by the same amount, and E3's bar even crosses from below the line to above it: the stump moves a whole group by one number, so an example near the middle of its group can be pushed slightly past the truth.

Five points and two stumps make a very blocky model. Figure 15.4 runs the same method on 45 deliveries with slightly deeper trees.

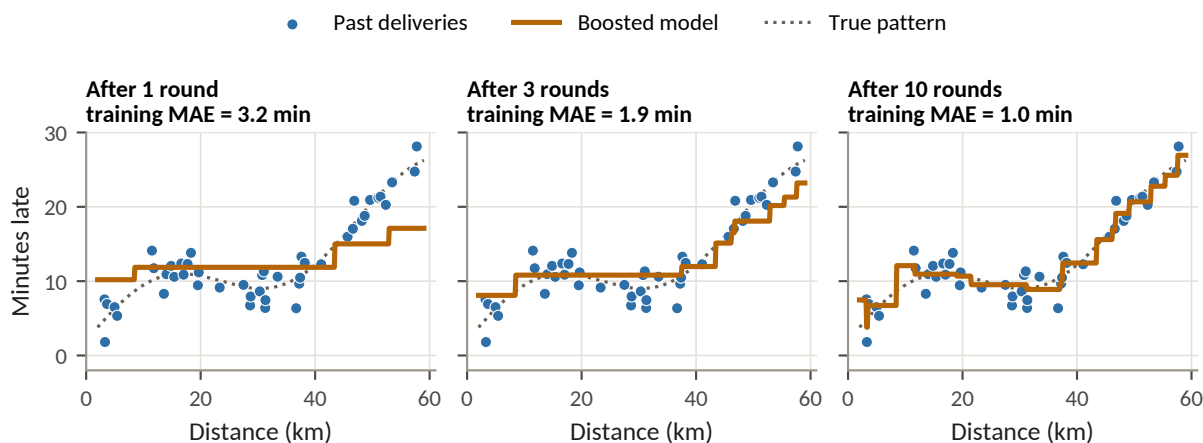


Figure 15.4 – Gradient boosting on 45 deliveries after 1, 3 and 10 rounds. Each panel shows the running prediction F_m , drawn against the true underlying pattern the simulation used.

WHAT TO OBSERVE

After one round the model is two flat shelves, barely more than the average. After three it has found the rise at long distances. After ten it traces the dip near 30 km. Each round adds one small step, and the steps accumulate into a curve no single small tree could draw.

COMMON MISTAKE

✗ Wrong: Reading a boosted model's individual trees the way Chapter 13 read a single decision tree, and quoting one of them as “the rule the model uses”.

✓ Right: After round 1, every tree predicts residuals in minutes-still-missing, not minutes late. Tree 7 on its own says nothing meaningful; only the whole chain, added together, is a prediction.

15.4 Learning Rate, Number of Rounds and Early Stopping

A random forest is famously forgiving: adding more trees almost never makes it worse. Boosting is not forgiving at all. Every extra round fits the residuals a little more closely, and once the real pattern has been used up, the only thing left in the residuals is noise. Fitting noise is overfitting (Chapter 7), so the number of rounds is a capacity dial that must be set on a validation set.

Karwan trains a boosted model on 600 past deliveries, keeping 300 more as a validation set, with $\eta = 0.1$ and depth-2 trees, and records the mean absolute error on both sets after every one of 600 rounds (Figure 15.5).

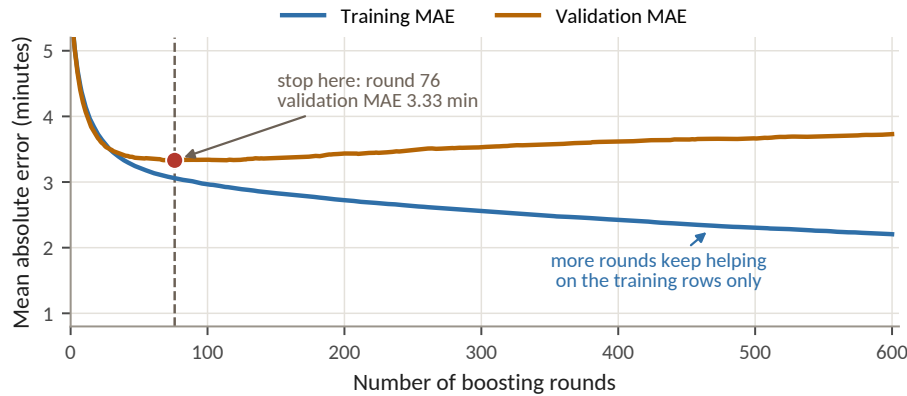


Figure 15.5 – Training and validation error of the same boosted model, round by round. The training curve never stops falling; the validation curve turns around.

WHAT TO OBSERVE

For the first thirty rounds the two curves sit almost on top of each other: everything the model learns helps on both sets. They separate near round 50, and from round 76 onward every extra tree makes the model better on deliveries it has already seen and worse on deliveries it has not — the loss-curve picture of Chapter 7, drawn by a boosted model.

WORKED EXAMPLE 15.6 — Choose the Number of Rounds by Early Stopping

Karwan's notebook records both errors at eight of the rounds. Training MAE, in minutes, after 10, 25, 50, 75, 100, 200, 400 and 600 rounds: 4.21, 3.59, 3.22, 3.06, 2.96, 2.72, 2.42, 2.20. Validation MAE at the same rounds: 4.14, 3.55, 3.36, 3.33, 3.34, 3.43, 3.62, 3.73. Choose how many rounds to keep, and find what running the full 600 would cost.

1. Ignore the training list completely; it falls at every step, so it can never choose a stopping point.
2. Read along the validation list and find the smallest value. It is 3.33 minutes, at 75 rounds.
3. Check either side: 50 rounds is worse (3.36) and 100 rounds is worse (3.34), so 75 is a real turning point and not just a wobble.
4. Find the cost of running all 600 rounds anyway: $3.73 - 3.33 = 0.40$ minutes of extra error on every prediction, with a generalization gap of $3.73 - 2.20 = 1.53$ minutes.

Answer: Stop at about 75 rounds. Left running to 600, the model looks 0.76 minutes better on its training deliveries and is 0.40 minutes worse on new ones. Checking every single round instead of these eight, the computer found the true best at round 76, with a validation MAE of 3.33 minutes.

The learning rate decides how much of each tree is added, and so how many rounds you need. Figure 15.6 runs the same data at three settings.

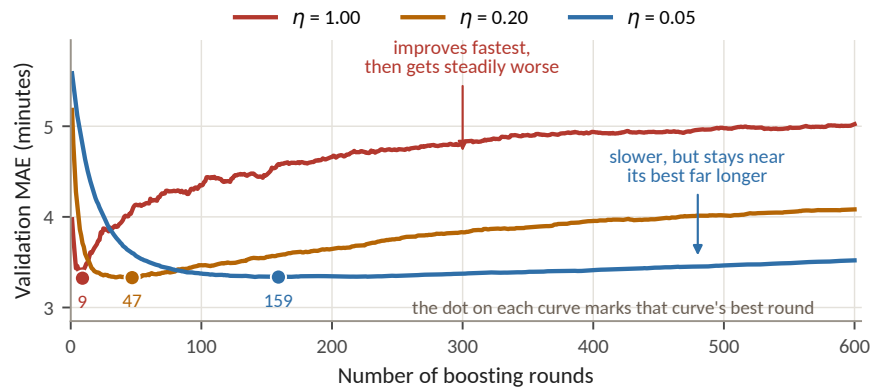


Figure 15.6 – Validation error against the number of rounds for three learning rates on the same data. Each curve’s dot marks its own best round.

WHAT TO OBSERVE

All three curves reach almost the same lowest point, near 3.33 minutes — the learning rate barely changes how good the model can get. What it changes is where that point sits and how quickly things fall apart afterward. At $\eta = 1.00$ the best arrives at round 9 and the error then climbs steeply to 5.02; at $\eta = 0.05$ the best arrives at round 159 and the curve is still nearly flat 400 rounds later. A small learning rate buys a wide, forgiving target to aim at.

Notice also that $1.00 \times 9 = 9$, $0.20 \times 47 = 9.4$ and $0.05 \times 159 = 7.95$: the learning rate multiplied by the best number of rounds stays roughly constant. That gives a useful rule of thumb whenever you change one of the two. A team whose best setting is $\eta = 0.20$ with 47 rounds, and who now wants a gentler $\eta = 0.05$, can multiply $0.20 \times 47 = 9.4$ and then divide, $9.4 \div 0.05 = 188$ rounds. The computer’s real best was 159, so the estimate is a good place to center a search rather than a final answer.

COMMON MISTAKE

X Wrong: Copying a random forest habit into boosting: “set the number of trees generously and leave it alone, since more trees never hurt.”

✓ Right: A forest’s trees are independent votes, so extra ones do no harm. A boosted model’s rounds are corrections stacked on corrections, so extra ones eventually chase noise. Always pair a number of rounds with a validation curve, or with a library’s built-in early stopping.

15.5 Modern Gradient-Boosting Libraries

Everything so far runs in scikit-learn’s plain `GradientBoostingRegressor`, or in its classifier twin, and those are the right tools for learning. On large datasets, though, most teams reach for one of a handful of specialized libraries. All of them implement the same three lines of Lesson 15.3; what they add is engineering, and one idea does most of the work.

WORD TO KNOW

Histogram binning — Sorting every numerical feature’s values into a small number of buckets, typically 255, before training starts, so that each split search compares a few hundred bucket edges instead of every distinct value in the column. It makes training dramatically faster and changes the result very little.

Table 15.4 lists the libraries you will meet most often and what each is known for.

Table 15.4 – Gradient-boosting libraries you will meet, and what each one adds

Library	What it adds to the same idea
scikit-learn GradientBoosting	The plain version used in this chapter: clear code and an exact split search, but slow past a few tens of thousands of rows
scikit-learn HistGradientBoosting	Histogram binning, built-in handling of missing values and categories, and built-in early stopping
XGBoost (Tianqi Chen and Carlos Guestrin, 2016)	Extra penalty terms on tree size and leaf values, so overfitting is fought inside each tree as well as by the learning rate
LightGBM (Guolin Ke and colleagues, 2017)	Binning plus leaf-wise growth — it grows whichever leaf promises the most improvement, instead of a whole level at a time
CatBoost (Liudmila Prokhorenkova and colleagues, 2018)	A careful way of turning category columns into numbers that avoids leaking the target into the features

Two practical points follow. First, none of these libraries changes what you learned by hand: there is still a flat starting guess, still a residual, still a learning rate, and still a number of rounds chosen on held-out data. Second, they are not magic accuracy. On a table with a few thousand rows, a carefully tuned scikit-learn model and a carefully tuned XGBoost model usually land within a hair of each other; the libraries pay off in speed and in convenience on messy columns. All of them are at their best on ordinary tables of numbers and categories — the kind of data in this book — and none is the right first tool for images, sound or free text.

15.6 Voting and Stacking Ensembles

Bagging and boosting both build many copies of *one* kind of model. The last two ensembles in this chapter combine models that are not alike at all — say a logistic regression, a k -nearest-neighbors classifier and a random forest — on the theory that very different models make very different mistakes.

WORDS TO KNOW

Voting ensemble — Several already-trained models that answer the same question, combined into one answer by counting votes or by averaging probabilities.

Hard voting — Each model announces one label, and the label named by the most models wins.

Soft voting — Each model reports a probability, the probabilities are averaged, and the average is compared with the threshold.

EQUATION: HARD AND SOFT VOTING

$$\hat{y}_{\text{hard}}(x) = \text{the label chosen by most of } \hat{y}_1(x), \dots, \hat{y}_K(x), \quad \hat{p}_{\text{soft}}(x) = \frac{1}{K} \sum_{k=1}^K p_k(x)$$

In words: Hard voting counts how many models chose each label and keeps the winner. Soft voting adds up the models' probabilities for the positive class, divides by the number of models, and compares that average with the decision threshold.

When it is used: Whenever several finished models of different kinds are available and you want one answer. Prefer soft voting when every model can report a sensible probability.

Symbol	Meaning	Unit or type
K	number of models in the ensemble	count
$\hat{y}_k(x)$	model k 's predicted label for example x	a category
$p_k(x)$	model k 's predicted probability of the positive class	between 0 and 1
$\hat{p}_{\text{soft}}(x)$	the ensemble's averaged probability	between 0 and 1

Soft voting listens to how sure each model is, so one confident model can outweigh two hesitant ones — an advantage, but only if the probabilities are well calibrated (Chapter 6).

Figure 15.7 sets three models loose on one new delivery.

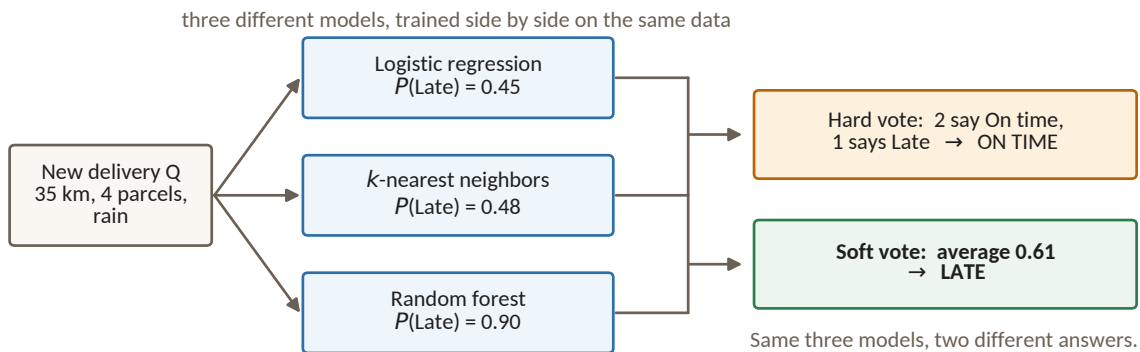


Figure 15.7 – A voting ensemble on one new delivery. The same three probabilities produce two different answers, depending on whether votes or probabilities are combined.

WHAT TO OBSERVE

Nothing in this diagram is trained on anything else's output. The three models never meet during training; they meet only at the moment of prediction. That makes voting the simplest ensemble in the book to build — and also the least clever, because it treats all three models as equally trustworthy.

WORKED EXAMPLE 15.7 — Hard Vote and Soft Vote Give Different Answers

For delivery Q, three models report the probability that it will be late: logistic regression 0.45, k -nearest neighbors 0.48, random forest 0.90. The decision threshold is 0.5. Find the hard vote and the soft vote.

1. Turn each probability into a label. $0.45 < 0.5 \rightarrow \text{On time}$. $0.48 < 0.5 \rightarrow \text{On time}$. $0.90 \geq 0.5 \rightarrow \text{Late}$.
2. Count the labels: On time gets 2 votes, Late gets 1. The hard vote is **On time**, 2 to 1.
3. Now average the probabilities: $0.45 + 0.48 + 0.90 = 1.83$, and $1.83 \div 3 = 0.61$.
4. Compare with the threshold: $0.61 \geq 0.5$, so the soft vote is **Late**.

Answer: The same three models give opposite answers: **On time** by hard voting, **Late** by soft voting. The two models saying “On time” were barely saying it at all, at 0.45 and 0.48, while the forest was very sure. Hard voting throws that information away; soft voting keeps it.

Stacking, described by David Wolpert in 1992 under the name *stacked generalization*, fixes exactly that. Instead of averaging the base models with fixed equal weights, it trains a second, small model to learn how much to trust each of them.

WORDS TO KNOW

Stacking — An ensemble in two layers: several base models are trained on the data, and their predictions become the input columns of a second model that produces the final answer.

Base model — One of the layer-1 models whose prediction becomes an input column for layer 2.

Meta-model — The small layer-2 model that combines the base models' predictions into the final answer. It is usually something simple, such as a logistic regression.

EQUATION: STACKING: THE META-MODEL

$$\hat{p}(x) = g(p_1(x), p_2(x), \dots, p_K(x)), \quad \text{with a linear meta-model} \quad \hat{p}(x) = \sum_{k=1}^K c_k p_k(x)$$

In words: The base models' predictions become the features of a new, very small dataset, and the meta-model g is trained on that dataset. When the meta-model is linear, its coefficients say how much each base model is trusted.

When it is used: When several different models each capture something the others miss, and you want the combination learned from data rather than fixed at a plain average.

Symbol	Meaning	Unit or type
$\hat{p}(x)$	the stacked ensemble's final output	between 0 and 1
g	the meta-model	any small model
$p_k(x)$	base model k 's prediction, used as an input column	between 0 and 1
c_k	the weight a linear meta-model learned for base model k	a number

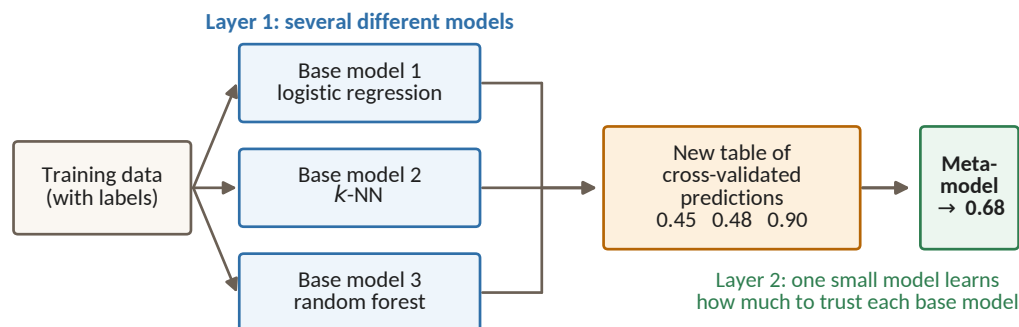
Soft voting is the special case where every c_k is fixed at $1/K$ instead of being learned.

WORKED EXAMPLE 15.8 — What the Meta-Model Adds

Trained on the deliveries, a linear meta-model learns the weights $c_1 = 0.2$ for logistic regression, $c_2 = 0.3$ for k -nearest neighbors and $c_3 = 0.5$ for the random forest. Use it on delivery Q, whose base predictions were 0.45, 0.48 and 0.90.

1. Multiply each base prediction by its own weight: $0.2 \times 0.45 = 0.090$.
2. Next: $0.3 \times 0.48 = 0.144$.
3. Next: $0.5 \times 0.90 = 0.450$.
4. Add the three products: $0.090 + 0.144 + 0.450 = 0.684$.
5. Compare with the threshold: $0.684 \geq 0.5$, so the prediction is Late.

Answer: The stacked ensemble predicts Late with probability 0.68, more confidently than soft voting's 0.61. The meta-model learned from the training data that the forest deserves half the say, so the forest's confident 0.90 pulls the answer further than a plain average would.



The layer-1 predictions must come from cross-validation, never from rows a base model was trained on.

Figure 15.8 – A stacked ensemble. Layer 1’s predictions become the columns of a small new table, and layer 2 learns how to combine them.

WHAT TO OBSERVE

The orange box is the whole idea: a brand-new dataset whose columns are other models’ opinions, so that layer 2 is ordinary supervised learning on a little table. The words “cross-validated” are not decoration — they are the difference between a stacked model that works and one that quietly cheats.

IMPORTANT

A base model always looks unrealistically good on the rows it trained on. Fed those over-optimistic predictions, the meta-model learns to trust whichever base model memorized the training set best, and the finished ensemble disappoints on new data: data leakage (Chapter 2) hiding inside an ensemble. The cure is k -fold cross-validation (Chapter 5) — predict every training row using base models fitted without it — which scikit-learn’s `StackingClassifier` does through its `cv` setting.

COMMON MISTAKE

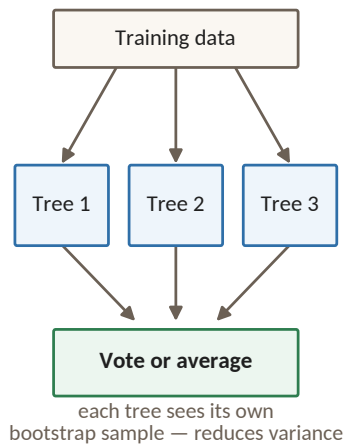
✗ Wrong: Stacking five models that are all gradient-boosted trees with slightly different settings, and expecting a large gain.

✓ Right: Stacking pays when the base models are genuinely different — a linear model, a distance-based model and a tree ensemble, for example — because their errors then fall in different places. Five near-identical models make almost the same mistakes, and averaging almost-identical mistakes changes almost nothing.

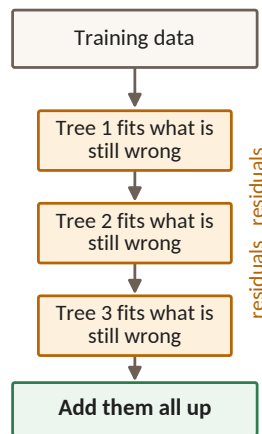
15.7 Bagging, Boosting and Stacking Side by Side

Three families, three shapes. Figure 15.9 puts them next to each other.

Bagging: side by side



Boosting: one after another



Stacking: two layers

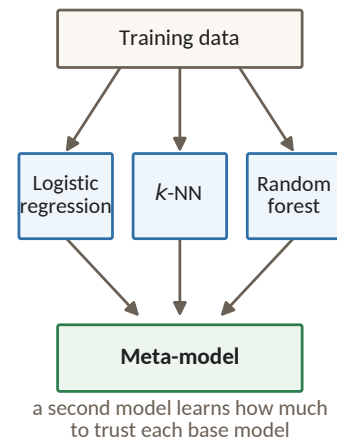


Figure 15.9 – The three ensemble families. Bagging trains its models side by side, boosting trains them one after another, and stacking trains a second layer on top of the first.

WHAT TO OBSERVE

Follow the arrows. On the left they all point down and never touch, so the models can be trained in any order or at the same time. In the middle they form one unbroken line, so nothing can be reordered. On the right they go down and then into a second model, trained last and on different inputs from everything above it.

The shapes matter because each family fixes a different problem. Chapter 7 split a model's error into bias, from being too simple to capture the pattern, and variance, from swinging with the particular rows it trained on. Bagging averages away variance; boosting chips away at bias, one correction at a time; stacking does neither directly, but covers one model's blind spots with another model's strengths. Table 15.5 sets out the differences that matter in practice.

Table 15.5 – Bagging, boosting and stacking compared

Family	How the models are built	What it mainly reduces	Main risk	Typical example
Bagging	Many copies of one model, side by side, each on its own bootstrap sample	Variance	Little; more trees rarely hurt, but a biased model stays biased	Random forest
Boosting	Many small models in a strict order, each fixing what is still wrong	Bias	Overfitting if the rounds or the learning rate are too large; sensitive to mislabeled rows	Gradient boosting, AdaBoost
Stacking	Several different models, then a small meta-model on their predictions	Whichever error the base models do not share	Leakage if layer-1 predictions are not cross-validated; harder to explain	Stacked ensemble

WORKED EXAMPLE 15.9 — Name the Ensemble

Name the family — bagging, boosting or stacking — for each description. (a) Two hundred trees are grown at the same time on bootstrap samples and vote. (b) Each new tree is trained on the errors the previous trees left behind. (c) A logistic regression is trained on the predictions of a forest, a k -NN model and a naive Bayes model. (d) A learning rate decides how much of each new model is added.

1. (a) Independent models, bootstrap samples, a vote at the end: **bagging** — specifically a random forest.
2. (b) Models trained in sequence on what is still wrong: **boosting**.
3. (c) A second model whose inputs are other models' predictions: **stacking**.
4. (d) Only a chain of corrections needs a dial to shrink each correction, so this is **boosting**. Bagging and stacking have no such dial.

Answer: (a) bagging, (b) boosting, (c) stacking, (d) boosting. The quickest test is to ask what each model was trained on: the same data resampled (bagging), the previous models' errors (boosting), or the previous models' predictions (stacking).

Which should you try first on a new table of data? A random forest, almost always: little tuning, hard to break, and an honest out-of-bag score for free. Then a boosted model, which often beats the forest by a small margin once its rounds and learning rate are tuned against a validation set. Reach for voting or stacking last, when you already have two or three good, genuinely different models and want the final percent.

RESPONSIBLE USE

Every step in this chapter makes a model harder to explain. A single tree can be read aloud and a forest can at least report a vote share, but a hundred boosted stumps predicting corrections to corrections can do neither, and a stacked ensemble adds another layer on top. When a prediction affects a person — a driver's route, a patient's appointment, a student's place — that lost explanation is a real cost. Prefer the simplest model that meets the need, keep a person in the decision, and read Chapter 24 first.

COMMON MISTAKES AND MISCONCEPTIONS

- Treating boosting as bagging with extra steps, and assuming the order of the models does not matter.
- Reading one tree out of a boosted chain as a rule, when after round 1 every tree predicts corrections, not answers.
- Expecting more boosting rounds always to help — and so choosing the number of rounds from the training curve instead of a validation curve.
- Believing a smaller learning rate makes a model more accurate; it mainly makes the stopping point easier to hit.
- Treating an AdaBoost learner weight α as an accuracy or a share, instead of an unbounded score whose size only matters relative to the others.
- Training a stacking meta-model on predictions the base models made for their own training rows, which leaks the answer into layer 2.
- Stacking several near-identical models and expecting the gain that only genuinely different models can give.

Practical Activities

ACTIVITY (PAPER) — Three Rounds of Boosting on Five Bread Deliveries

A bakery in Duhok delivers bread to five shops each morning. Yesterday's records give the distance to each shop and how many minutes late the van arrived.

Shop	S1	S2	S3	S4	S5
Distance (km)	5	9	13	38	46
Minutes late	8	12	16	28	36

1. Find round 0's prediction: the average of the five true values. Write it above every shop, then compute the five residuals and their mean absolute error.
2. Every round uses the same stump, "is the distance 25 km or less?", and a learning rate of $\eta = 0.5$. For round 1, average the residuals inside each of the two groups, multiply each average by 0.5, and add it to that group's running prediction.
3. Repeat for rounds 2 and 3, each time starting from the residuals the previous round left. Keep one row per round: prediction (short group), prediction (long group), the five residuals, and the MAE.
4. Check your work: the two predictions should be 16 and 26 after round 1, 14 and 29 after round 2, and 13 and 30.5 after round 3, and the MAE should fall 9.6, 4.8, 3.6, 3.4.
5. The gain from round 2 to round 3 is much smaller than from round 0 to round 1. Write one sentence explaining why, and one saying how you would decide when to stop on real data.

You need: Paper, pencil and a calculator

ACTIVITY (PYTHON) — Boosting Round by Round, and Where to Stop

The first half repeats Worked Examples 15.4 and 15.5 twice — once with plain NumPy and once with scikit-learn — and the second half finds the stopping point of Figure 15.5 from `datasets/ch15_deliveries.csv`, which holds 900 simulated Sunrise Parcels deliveries.

Python · NumPy · pandas · scikit-learn

```
import numpy as np, pandas as pd
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

X = np.array([[8], [12], [16], [44], [50]])      # distance (km)
y = np.array([2, 4, 6, 18, 20])                  # minutes late

F = np.full(5, y.mean())                        # round 0: one flat guess
for rnd in (1, 2):
    r = y - F                                    # what is still wrong
    short = X[:, 0] <= 30
    stump = np.where(short, r[short].mean(), r[~short].mean())
    F = F + 0.5 * stump                          # learning rate 0.5
    print("round", rnd, "residuals", r, "-> prediction", F,
          " MAE", round(float(np.abs(y - F).mean()), 2))

gb = GradientBoostingRegressor(n_estimators=2, learning_rate=0.5, max_depth=1,
                               random_state=0).fit(X, y)
for rnd, pred in enumerate(gb.staged_predict(X), start=1):
    print("scikit-learn after round", rnd, ":", pred)

d = pd.read_csv("datasets/ch15_deliveries.csv")  # 900 deliveries
cols = ["distance_km", "parcels", "rain", "driver_years"]
tr, va = d[d.set == "train"], d[d.set == "validation"]
big = GradientBoostingRegressor(n_estimators=600, learning_rate=0.1, max_depth=2,
                                random_state=0).fit(tr[cols], tr.minutes_late)
curve = [mean_absolute_error(va.minutes_late, p) for p in big.staged_predict(va[cols])]
print("best round:", int(np.argmin(curve)) + 1,
      " validation MAE:", round(min(curve), 2),
      " at 600 rounds:", round(curve[-1], 2))
```

Output

```
round 1 residuals [-8. -6. -4.  8. 10.] -> prediction [ 7.  7.  7. 14.5 14.5] MAE 3.6
round 2 residuals [-5. -3. -1.  3.5 5.5] -> prediction [ 5.5  5.5  5.5 16.75
16.75] MAE 2.0
scikit-learn after round 1 : [ 7.  7.  7. 14.5 14.5]
scikit-learn after round 2 : [ 5.5  5.5  5.5 16.75 16.75]
best round: 76 validation MAE: 3.33 at 600 rounds: 3.73
```

The NumPy loop is the equation box with nothing hidden: `r` is the residual, `stump` is each group's average residual, and `F` is the running prediction. `staged_predict` hands back that running prediction after every tree, which is how the last block scores all 600 rounds in one pass. Try `learning_rate=0.05` and watch the best round move to about 159.

You need: Python with NumPy, pandas and scikit-learn (see Chapter B), and `datasets/ch15_deliveries.csv`

ACTIVITY (PYTHON) — AdaBoost, Voting and Stacking on the Same Deliveries

This program trains three very different base models on the same 600 deliveries, combines them four ways, and scores everything on the untouched 300 validation deliveries.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.ensemble import (AdaBoostClassifier, RandomForestClassifier,
                              VotingClassifier, StackingClassifier)
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

d = pd.read_csv("datasets/ch15_deliveries.csv")
cols = ["distance_km", "parcels", "rain", "driver_years"]
tr, va = d[d.set == "train"], d[d.set == "validation"]
base = [("logreg", make_pipeline(StandardScaler(), LogisticRegression())),
        ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15))),
        ("forest", RandomForestClassifier(n_estimators=200, random_state=0))]
combos = [("AdaBoost", AdaBoostClassifier(n_estimators=200, random_state=0)),
          ("Voting (hard)", VotingClassifier(base, voting="hard")),
          ("Voting (soft)", VotingClassifier(base, voting="soft")),
          ("Stacking", StackingClassifier(base, cv=5,
                                          final_estimator=LogisticRegression()))]

for name, model in base + combos:
    model.fit(tr[cols], tr.ate)
    print("%-14s validation accuracy %.3f" % (name, model.score(va[cols], va.ate)))
```

Output

logreg	validation accuracy 0.797
knn	validation accuracy 0.820
forest	validation accuracy 0.797
AdaBoost	validation accuracy 0.820
Voting (hard)	validation accuracy 0.820
Voting (soft)	validation accuracy 0.830
Stacking	validation accuracy 0.823

The best single model reaches 0.820 and soft voting reaches 0.830, so combining three different models bought one extra correct prediction in every hundred. Hard voting bought nothing here, because it threw away the probabilities that made soft voting work. `cv=5` is what makes stacking honest: without it the meta-model would see each base model's over-confident predictions for its own training rows. Try replacing the three base models with three random forests and watch the gain disappear.

You need: Python with pandas and scikit-learn, and datasets/ch15_deliveries.csv

CASE STUDY — Sunrise Parcels Moves from a Forest to Gradient Boosting

The random forest of Chapter 14 has been producing Sunrise Parcels' morning risk list for three months, and dispatch trusts it. Karwan, the analyst, now tries a boosted model on exactly the same columns and exactly the same split of the data, to see whether the extra accuracy is real.

He is careful about one thing from the start: the forest needed almost no tuning, and the boosted model needs a lot. He fixes the learning rate at 0.1, trains to 600 rounds, and reads the validation curve rather than the training curve — which, as Figure 15.5 shows, keeps improving forever and would have told him to use every round. The validation curve turns at 76 rounds. On the untouched test set the boosted model beats the forest by a small but repeatable margin, and holds it a month later on newer deliveries. What did not change is more interesting than what did. The same two columns, distance and parcel count, still carry most of the model's weight. The driver identity column is still deliberately absent, for the same reason as before: a model that learns which driver is on a route mostly learns which routes each driver is given. The morning output is still a short ranked list a dispatcher reads, not an instruction a van obeys, and the small untouched test set is still used exactly once, at the end.

Two things did change, and both cost something. The forest could report “83 of 100 trees voted Late”, which dispatchers had learned to read as a rough confidence; the boosted model reports a single number, so Karwan must calibrate its output (Chapter 6) before anyone can read it the same way. And it needs watching: retrained on a new season with the old 600 rounds and no fresh validation curve, it would quietly slide back into the overfitting region, so Karwan writes the retraining procedure down — refit, replot the validation curve, re-choose the number of rounds. Both models are kept, with the forest running alongside as a stable second opinion, and a delivery flagged by both gets attention first.

CHAPTER SUMMARY

- A weak learner is barely better than guessing; boosting trains many of them one after another, each aimed at what the previous ones still get wrong, and adds their outputs together.
- Order is essential in boosting and irrelevant in bagging: a boosted model's trees cannot be shuffled, thinned or read one at a time.
- AdaBoost weights every training row, computes each weak learner's weighted error rate ε_m , and turns it into a learner weight $\alpha_m = \frac{1}{2} \ln((1 - \varepsilon_m)/\varepsilon_m)$.
- AdaBoost then multiplies missed rows' weights by $e^{+\alpha_m}$ and correct rows' by $e^{-\alpha_m}$ and rescales, which always leaves exactly half the total weight on the rows that were missed; it predicts with the sign of $\sum \alpha_m h_m(x)$.
- Gradient boosting starts from the mean, computes the residual $y - F(x)$ for every row, fits a small tree to those residuals, and updates with $F_m = F_{m-1} + \eta h_m$.
- The number of rounds is a capacity dial: the training error falls forever while the validation error turns around, so the rounds must be chosen by early stopping on a validation set.
- The learning rate and the number of rounds trade against each other, their product staying roughly constant; a smaller learning rate mainly makes the stopping point easier to hit.
- XGBoost, LightGBM, CatBoost and scikit-learn's HistGradientBoosting implement the same three equations, adding histogram binning, extra penalties, and better handling of categories and missing values.
- Voting combines several different finished models by counting labels (hard) or averaging probabilities (soft); stacking instead trains a small meta-model on the base models' cross-validated predictions.
- Bagging mainly reduces variance, boosting mainly reduces bias, and stacking covers one model's blind spots with another's strengths — at the price, every time, of a model that is harder to explain.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain in your own words what a weak learner is, and why boosting needs many of them rather than one good model.

Problem 2 [Basic] What is a decision stump? Say how many questions it asks and how many different answers it can give.

Problem 3 [Basic] Bagging trains its models side by side and boosting trains them one after another. Give one practical consequence of that difference — something you can do with a forest but not with a boosted model.

Problem 4 [Basic] A boosting run's running prediction for one delivery is 50 minutes. Round 1's tree predicts -12 for that delivery. With a learning rate of 0.4 , find the updated prediction, showing every step.

Problem 5 [Standard] A weak learner misclassifies 3 of 12 equally weighted training rows. (a) Find its weighted error rate ε . (b) Find its learner weight $\alpha = \frac{1}{2} \ln((1 - \varepsilon)/\varepsilon)$, using $\ln 3 = 1.099$.

Problem 6 [Standard] Continue Problem 5. Each row starts with weight $1 \div 12 = 0.0833$. Multiply the 3 missed rows by $e^{+\alpha} = 1.732$ and the 9 correct rows by $e^{-\alpha} = 0.577$, add everything to find Z , and divide. Show that the three missed rows now hold exactly half of the total weight.

Problem 7 [Standard] Three deliveries were 6, 10 and 20 minutes late. (a) Find round 0's flat prediction, the three residuals, and the mean absolute error. (b) A stump puts the two shortest deliveries in one leaf and the longest in the other. Find each leaf's value, then the three updated predictions with $\eta = 0.5$, then the new mean absolute error.

Problem 8 [Standard] Three models report the probability that a delivery will be late as 0.40 , 0.42 and 0.95 , with a threshold of 0.5 . Find the hard vote and the soft vote, and explain in one sentence why they disagree.

Problem 9 [Standard] A team's boosted model reaches its lowest validation error at 60 rounds with $\eta = 0.4$. They now want to use $\eta = 0.1$. Estimate the number of rounds they should plan for, and say why the estimate is only a starting point.

Problem 10 [Challenge] An AdaBoost run keeps three learners with weights $\alpha_1 = 0.20$, $\alpha_2 = 0.30$ and $\alpha_3 = 0.90$. For one new example, learners 1 and 2 say Late (+1) and learner 3 says On time (−1). (a) Find the weighted total and the prediction. (b) Explain why the answer disagrees with the plain majority. (c) State what must be true of ε_3 for this to happen.

Problem 11 [Challenge] A team compares two models on the same data. The random forest scores 88% on validation and 87% on the test set. The boosted model scores 99% on training and 84% on the test set. (a) Which shows the stronger sign of overfitting, and what is the evidence? (b) Name two settings you would change on the boosted model first. (c) Say what you would check before choosing between the two.

Problem 12 [Challenge] A clinic in Sulaymaniyah wants to stack a logistic regression, a k -NN model and a boosted-tree model to decide which patients are called for an earlier follow-up. One row in its training data has the wrong label. (a) Explain why the meta-model must be trained on cross-validated predictions. (b) Describe what a boosted model does with that mislabeled row over many rounds, and why a random forest is less disturbed by it. (c) Give one reason a clinic might reject the stacked ensemble even if it is the most accurate model available, and one thing the team should report alongside the accuracy.

Support Vector Machines

The widest possible street between two classes

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why many straight lines can separate the same two classes, and why some are safer than others.
- Compute the decision function $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ by hand and turn its sign into a prediction.
- Find the width of a margin from $\mathbf{w}$, the distance of any point from the boundary, and which points are the support vectors.
- Write down the hard-margin problem and check whether a candidate solution obeys its constraints.
- Measure the slack of a point that breaks the margin, and turn the dial C that decides how much slack is tolerated.
- Compute hinge loss, compare it with log loss, and predict with a support vector regression tube.
- Explain why an SVM must almost always be given standardized features.

Introduction

Imagine two football teams lining up on opposite halves of a dusty school yard, and a teacher who must draw one chalk line between them. If the groups stand well apart there are many places the line could go, and every one of them separates the teams correctly today. But the lines are not equally good. A line drawn one step from the blue team leaves no room at all if a blue player shuffles sideways. A line drawn straight down the middle of the empty ground leaves the most room on both sides.

That idea — leave as much room as possible — is the whole of this chapter. A *support vector machine* does not simply look for a line that works. It looks for the line with the widest possible empty street around it, and it measures that street in numbers instead of in feelings.

Chapter 10 gave you a straight decision boundary found by pushing probabilities toward the true labels. This chapter finds one by geometry instead. You will meet the margin, the support vectors that hold it in place, the slack that lets a few awkward points break the rules, and the dial C that decides how strict the arrangement is — all with arithmetic you can do with a pencil, a ruler and a square root.

16.1 Many Lines Separate the Data — Which One Is Best?

Nasrin runs a small workshop in Erbil that services solar water pumps. Before a repaired pump goes back to a customer she runs it on a test bench for ten minutes and writes down two readings: how much the pump body vibrates, in millimeters per second, and how much extra current it draws above its normal working value, in amperes. A quiet pump drawing normal current is healthy; a pump that shakes and pulls too much current usually has a worn bearing or a bent shaft.

Last week she tested six pumps and later opened every one, so she knows the true answer for each. Table 16.1 is her whole record. Following the usual convention for this chapter, the healthy class is written $y = -1$ and the faulty class $y = +1$; the two labels are just names.

Table 16.1 – Nasrin’s six tested pumps — two readings and the true condition

Pump	Vibration x_1 (mm/s)	Extra current x_2 (A)	Condition	Label y
1	0	1	Healthy	−1
2	1	0	Healthy	−1
3	1	1	Healthy	−1
4	2	2	Faulty	+1
5	4	2	Faulty	+1
6	3	3	Faulty	+1

Plot those six pumps and something pleasant happens: the healthy ones sit in the bottom-left corner, the faulty ones up and to the right, and no point of one class sits among the other. A single straight line can separate them with no mistakes at all — and here is the awkward part. If one line does it perfectly, so do infinitely many others, because a perfect line can be tilted a little or slid a little and still be perfect. Figure 16.1 shows three of them on the left, and the SVM’s answer on the right.

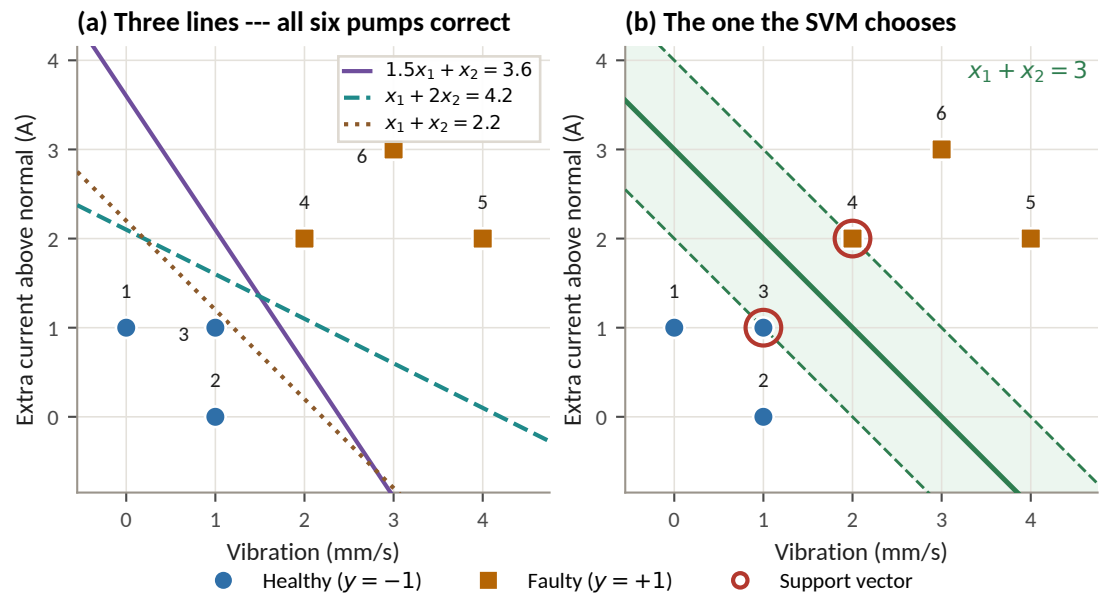


Figure 16.1 – Left: three different lines, each of which classifies all six pumps correctly. Right: the line a support vector machine chooses, shown with the empty street around it.

WHAT TO OBSERVE

All three lines on the left get every training pump right, so training accuracy cannot tell them apart. But the dotted brown line passes within a whisker of pump 3, and the dashed teal line runs close to pump 4. The line on the right keeps the largest possible empty band around itself — and the two points that touch that band are circled.

Data that can be split cleanly this way has a name.

WORD TO KNOW

Linearly separable — Two classes are linearly separable when at least one straight line (or, with more than two features, one flat surface) can be drawn so that every example of one class falls on one side and every example of the other class falls on the other side, with no mistakes at all.

Before we can measure a street, we need to write a line as arithmetic. Chapter 10 already built the machinery: give each feature a coefficient, multiply, add, and add an intercept. In this chapter the coefficients are collected into a single list called the weight vector, written $\mathbf{w}$, and the whole calculation is called the decision function.

WORDS TO KNOW

Weight vector — The list of coefficients of a linear model, written $\mathbf{w} = (w_1, w_2, \dots, w_k)$ — one number per feature. Bold letters are used for lists of numbers so they are not confused with single values.

Decision function — The raw score $f(\mathbf{x})$ a linear classifier gives an example before any decision is made. Its *sign* says which class is predicted; its *size* says how far the example sits from the boundary.

The dot in $\mathbf{w} \cdot \mathbf{x}$ is a shorthand you already know how to compute: multiply matching pairs and add the products.

EQUATION: THE DECISION FUNCTION OF A LINEAR CLASSIFIER

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = w_1x_1 + w_2x_2 + \dots + w_kx_k + b, \quad \hat{y} = \text{sign}(f(\mathbf{x}))$$

In words: Multiply every feature by its own weight, add all the products, then add the intercept. If the answer is positive predict class +1; if it is negative predict class -1.

When it is used: Every time a linear classifier — an SVM here, logistic regression in Chapter 10 — makes a prediction for one example.

Symbol	Meaning	Unit or type
$f(\mathbf{x})$	the decision function, or score, of example $\mathbf{x}$	no unit
$\mathbf{w}, w_j$	the weight vector and the weight of feature j	per unit of feature j
$\mathbf{x}, x_j$	one example and the value of its feature j	unit of that feature
$b, \hat{y}$	the intercept (or bias), and the predicted label	no unit; +1 or -1

The boundary is the set of points where $f(\mathbf{x}) = 0$, exactly the decision boundary of Chapter 10. A point with $f(\mathbf{x}) = 0$ sits on the line itself, and a program must be told which class to give it; scikit-learn answers +1.

The line the SVM chose in Figure 16.1 is $x_1 + x_2 = 3$, which means $\mathbf{w} = (1, 1)$ and $b = -3$. Lesson 16.3 shows where those numbers come from. First let us use them.

WORKED EXAMPLE 16.1 — Score Four Pumps and Classify Them

Using $\mathbf{w} = (1, 1)$ and $b = -3$, compute $f(\mathbf{x})$ for pump 1 (0, 1), pump 3 (1, 1), pump 4 (2, 2) and pump 5 (4, 2), and give each one a predicted condition.

1. Write out the function once: $f(\mathbf{x}) = 1 \times x_1 + 1 \times x_2 - 3$, which is simply $x_1 + x_2 - 3$.
2. Pump 1: $0 + 1 - 3 = -2$. The sign is negative, so the prediction is Healthy.
3. Pump 3: $1 + 1 - 3 = -1$. Negative again, so Healthy.
4. Pump 4: $2 + 2 - 3 = +1$. Positive, so Faulty.
5. Pump 5: $4 + 2 - 3 = +3$. Positive, so Faulty.
6. Compare each prediction with Table 16.1: all four match the true condition.

Answer: The scores are -2 , -1 , $+1$ and $+3$, giving Healthy, Healthy, Faulty and Faulty — all four correct. Notice that pump 5 scores $+3$ while pump 4 scores only $+1$: both are predicted Faulty, but pump 5 sits three times further into faulty territory.

COMMON MISTAKE

✗ Wrong: Reading a decision function value such as $f(\mathbf{x}) = 0.87$ as “an 87 percent chance of being faulty”.

✓ Right: An SVM score is a distance-like number, not a probability. It has no upper limit and no lower limit. Chapter 10 produces probabilities; an SVM produces a score whose sign is the answer.

So which of the many separating lines should the workshop trust? The oldest answer, from the perceptron learning rule published by Frank Rosenblatt in 1958, was simply to take the first line that makes no mistakes and stop — which works on training data and often disappoints on new data. In 1963 Vladimir Vapnik and Alexey Chervonenkis proposed a far more careful answer.

WORD TO KNOW

Support vector machine (SVM) — A classifier that chooses, out of all the boundaries that separate the classes, the one with the widest empty band around it. The idea comes from Vladimir Vapnik and Alexey Chervonenkis in 1963; the modern version that tolerates a few mistakes was published by Corinna Cortes and Vladimir Vapnik in 1995.

16.2 The Margin and the Support Vectors

Think of the boundary not as a chalk line but as the middle of a street: empty of training points, and pushed outward on both sides until it bumps into the nearest example. Those bumping points keep it from opening any wider.

WORDS TO KNOW

Margin — The width of the empty band around a classifier’s boundary — the perpendicular distance from one edge of the street to the other. A wider margin means the boundary sits further from the closest training examples of both classes.

Support vector — A training example that lies exactly on the edge of the margin and therefore holds the street in place. Moving a support vector moves the boundary; moving any other point does not.

To measure a street we need one more quantity: the length of the weight vector.

WORD TO KNOW

Norm — The length of a vector, written $\|\mathbf{w}\|$: square every entry, add the squares, take the square root — the Pythagoras calculation of Chapter 11. For $\mathbf{w} = (3, 4)$ the norm is $\sqrt{9 + 16} = 5$.

Here is the fact that makes everything else work. The score $f(\mathbf{x})$ is not itself a distance, but dividing it by $\|\mathbf{w}\|$ turns it into one. And because the two edges of the street are drawn where $f(\mathbf{x}) = +1$ and $f(\mathbf{x}) = -1$, the street is exactly $2 \div \|\mathbf{w}\|$ wide.

EQUATION: DISTANCE TO THE BOUNDARY AND THE WIDTH OF THE MARGIN

$$d(\mathbf{x}) = \frac{|f(\mathbf{x})|}{\|\mathbf{w}\|}, \quad M = \frac{2}{\|\mathbf{w}\|}, \quad \|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_k^2}$$

In words: The distance from a point to the boundary is the size of its score, ignoring the minus sign, divided by the length of the weight vector. The margin is two divided by that same length.

When it is used: To measure how far an example sits from the boundary, and to compare two candidate boundaries by the room they leave.

Symbol	Meaning	Unit or type
$d(\mathbf{x}), M$	the distance from $\mathbf{x}$ to the boundary, and the margin	feature units
$\ \mathbf{w}\ $	the length (norm) of the weight vector	no unit
$f(\mathbf{x})$	the decision function of Lesson 16.1	no unit

Making $\|\mathbf{w}\|$ smaller makes the margin wider — the two are the same statement upside down. When features are measured in different units this distance mixes them, which is exactly why Lesson 16.7 insists that features be standardized first.

WORKED EXAMPLE 16.2 — Find the Width of Nasrin's Street

The fitted model is $\mathbf{w} = (1, 1)$ and $b = -3$. Find the length of $\mathbf{w}$, the width of the margin, the distance from the boundary to each edge of the street, and the distance of pump 5, whose score is $+3$.

1. Square each weight and add: $1^2 + 1^2 = 2$; take the square root: $\|\mathbf{w}\| = \sqrt{2} = 1.41$.
2. Put that into the margin formula: $M = 2 \div 1.41 = 1.41$.
3. The boundary sits in the middle, so each edge is half the margin away: $1.41 \div 2 = 0.71$.
4. Pump 5: its distance is $|+3| \div 1.41 = 2.12$, three times as far as an edge.

Answer: The street is 1.41 units wide, each edge sits 0.71 units from the boundary, and pump 5 is 2.12 units away. Any pump landing inside the band is one the model is not confident about.

Figure 16.2 draws the street over Nasrin's six pumps, with the weight vector shown as an arrow.

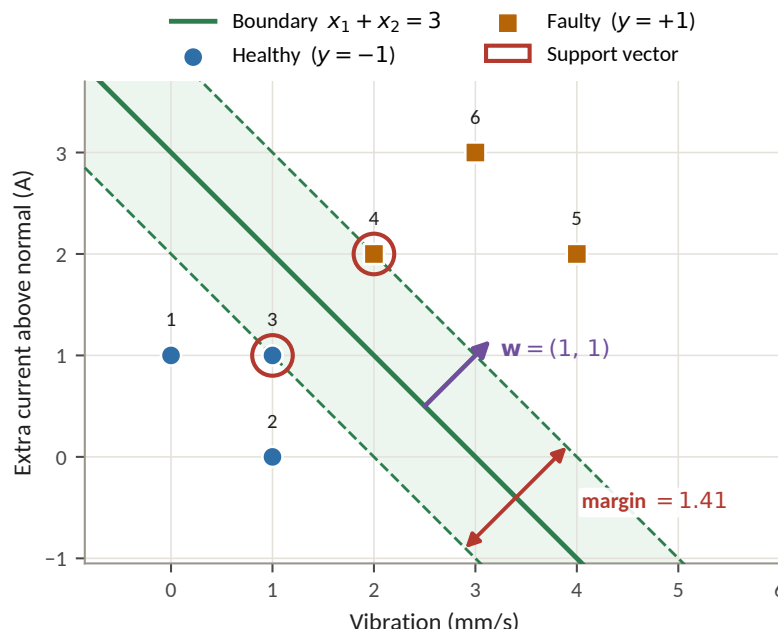


Figure 16.2 – The maximum-margin street for Nasrin's six pumps. The solid line is the boundary, the dashed lines are the two edges of the street, and the two circled pumps are the support vectors.

WHAT TO OBSERVE

The weight vector $\mathbf{w}$ points straight across the street, at a right angle to the boundary — it always does. Exactly two pumps touch the edges, pump 3 and pump 4; the other four sit outside the street and play no part in fixing where it goes.

How do we tell, without a ruler, which points are support vectors? By the score. A point on an edge of the street has $f(\mathbf{x}) = +1$ or $f(\mathbf{x}) = -1$, so multiplying its score by its own label always gives exactly 1. Table 16.2 works this out for every pump: only pumps 3 and 4 have $y f(\mathbf{x}) = 1$, so only they are support vectors. Every other pump scores more than 1 and sits outside the street. Nasrin could throw away pumps 1, 2, 5 and 6, refit the model, and get the very same line back.

Table 16.2 – Score, margin and distance for each of the six pumps, using $\mathbf{w} = (1, 1)$ and $b = -3$

Pump	x_1	x_2	Score $f(\mathbf{x})$	$y f(\mathbf{x})$	Distance $ f /\ \mathbf{w}\ $
1	0	1	-2	2	1.41
2	1	0	-2	2	1.41
3	1	1	-1	1	0.71
4	2	2	+1	1	0.71
5	4	2	+3	3	2.12
6	3	3	+3	3	2.12

The bold rows are the support vectors. A trained SVM stores only these, often a small fraction of the training set — a useful economy, and also a warning.

COMMON MISTAKE

✗ Wrong: Assuming that adding more training examples of a class always pushes the boundary toward the other class, the way adding more weight tips a balance.

✓ Right: Only points on or inside the street change an SVM's boundary. A hundred new pumps far out in faulty territory would not move the line by a hair; one new pump close to the street would move it at once.

16.3 The Hard-Margin SVM

We can now state exactly what an SVM looks for. It wants the widest street, which by the margin formula means the smallest $\|\mathbf{w}\|$, and it wants every training point on the correct side, which means every point must score at least 1 when multiplied by its own label. Together those two wishes are the hard-margin problem.

WORD TO KNOW

Hard margin — The strict version of the SVM rule: every single training example must sit outside the street, on the correct side. It can only be used when the classes are linearly separable, and it has no tolerance at all for a mislabeled or unusual example.

Two small conveniences hide in the formula: squaring the norm removes the square root, and the one-half makes a later derivation tidier. Neither changes which $\mathbf{w}$ wins, because whatever makes $\|\mathbf{w}\|$ smallest also makes $\frac{1}{2}\|\mathbf{w}\|^2$ smallest.

EQUATION: THE HARD-MARGIN SUPPORT VECTOR MACHINE

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \text{for every example } i$$

In words: Make the weight vector as short as you can — which makes the street as wide as you can — while keeping every training example at least one full unit of score on its own side of the boundary.

When it is used: As the definition of what a linear SVM computes, and to check whether a candidate $\mathbf{w}$ and b are even allowed.

Symbol	Meaning	Unit or type
$\mathbf{w}, b$	the weight vector and intercept being searched for	per feature unit; no unit
$\mathbf{x}_i, y_i$	example i and its label, +1 or −1	feature units; class name
$y_i (\mathbf{w} \cdot \mathbf{x}_i + b)$	the <i>functional margin</i> of example i	no unit

The condition after “subject to” is called a constraint: a rule the answer is not allowed to break. A solution that breaks even one constraint is rejected however wide its street.

Figure 16.3 lays out the three parallel lines that this rule creates: the boundary in the middle, where $f(\mathbf{x}) = 0$, and the two edges of the street, where $f(\mathbf{x})$ equals +1 and −1.

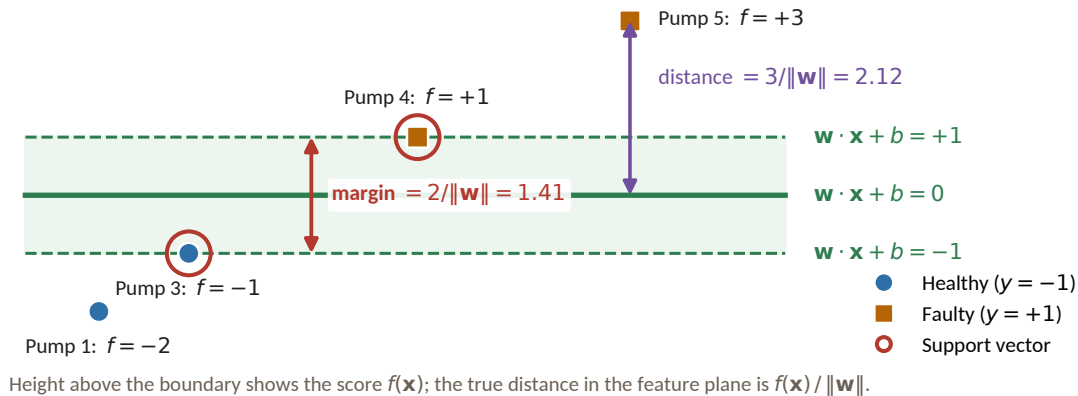


Figure 16.3 – The geometry of a hard-margin SVM, drawn with the score $f(\mathbf{x})$ as the height. The support vectors sit exactly on the lines $f = \pm 1$; every other example is further out.

WHAT TO OBSERVE

The two support vectors are pinned to the edges at $f = \pm 1$, which is what the constraint demands, while pump 5 at $f = +3$ has three units of score to spare. Squeezing $\|\mathbf{w}\|$ smaller shrinks every score toward zero and widens the street in real feature units.

Two different weight vectors can both obey every constraint. When that happens the smaller $\frac{1}{2} \|\mathbf{w}\|^2$ wins, and the next example shows what that choice buys.

WORKED EXAMPLE 16.3 — Compare Two Allowed Streets and Pick the Winner

Candidate A is $\mathbf{w} = (1, 1)$ with $b = -3$. Candidate B is $\mathbf{w} = (2, 2)$ with $b = -5$. Both keep every one of the six pumps on the correct side with a functional margin of at least 1. Which one does the hard-margin SVM choose, and how wide is each street?

1. Candidate A: $\|\mathbf{w}\|^2 = 1^2 + 1^2 = 2$, so the quantity to minimize is $\frac{1}{2} \times 2 = 1$.
2. Candidate A's margin: $\|\mathbf{w}\| = \sqrt{2} = 1.41$, so $M = 2 \div 1.41 = 1.41$.
3. Candidate B: $\|\mathbf{w}\|^2 = 2^2 + 2^2 = 8$, so the quantity to minimize is $\frac{1}{2} \times 8 = 4$.
4. Candidate B's margin: $\|\mathbf{w}\| = \sqrt{8} = 2.83$, so $M = 2 \div 2.83 = 0.71$.
5. Compare: 1 is smaller than 4, so candidate A wins; and indeed its street, 1.41 units wide, is twice candidate B's 0.71.

Answer: The SVM chooses candidate A, whose street is 1.41 units wide against candidate B's 0.71. Both obey every constraint, so accuracy on the training pumps cannot separate them — only the width of the street can.

16.4 Soft Margins: Slack and the C Dial

A week later two more pumps come back, both faulty. Pump 7 vibrates 2.5 mm/s and draws 1.0 A extra. Pump 8 is the troublemaker: it vibrates only 1.5 mm/s and draws just 0.5 A extra, yet it turned out to have a cracked impeller, and on Nasrin's plot it sits in the middle of the healthy cluster. No street can now keep all eight pumps on the correct side, so the hard-margin problem has no answer at all — not a bad answer, but none, because no line satisfies every constraint. Real data almost always overlaps somewhere. The fix, from Cortes and Vapnik's 1995 paper, is to let a point break the rule and charge it a price for doing so.

WORDS TO KNOW

Slack variable — A number ξ_i (the Greek letter xi) measuring how badly one example breaks the margin rule. It is zero for a well-behaved example, between 0 and 1 for one that strays inside the street but stays on the correct side, and greater than 1 for one that is on the wrong side altogether.

Soft margin — The practical version of the SVM rule, which allows examples to break the margin and adds up the slack as a cost instead of forbidding it.

The slack of an example is simply the amount by which its functional margin falls short of 1 — and never less than zero, because an example with room to spare is not owed anything.

EQUATION: THE SLACK OF ONE EXAMPLE

$$\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$$

In words: Work out the example's functional margin, subtract it from 1, and if the answer is negative write zero instead.

When it is used: To measure how much one training example violates the margin, once a candidate boundary is on the table.

Symbol	Meaning	Unit or type
ξ_i	the slack of example i	no unit
$y_i f(\mathbf{x}_i)$	the functional margin of example i	no unit
$\max(0, \cdot)$	keep the value if positive, otherwise use 0	no unit

$\xi_i = 0$ means outside the street on the correct side; $0 < \xi_i \leq 1$ means inside the street but still correctly classified; $\xi_i > 1$ means the example is on the wrong side of the boundary.

WORKED EXAMPLE 16.4 — Measure the Slack of Three Pumps

Keep the old street, $\mathbf{w} = (1, 1)$ and $b = -3$, and measure the slack of pump 5 (4, 2), pump 7 (2.5, 1.0) and pump 8 (1.5, 0.5). All three are faulty, so $y = +1$ for each.

1. Pump 5: $f = 4 + 2 - 3 = +3$, so $y f = 3$. Then $1 - 3 = -2$, which is negative, so $\xi_5 = 0$.
2. Pump 7: $f = 2.5 + 1.0 - 3 = +0.5$, so $y f = 0.5$. Then $1 - 0.5 = 0.5$, which is positive, so $\xi_7 = 0.5$.
3. Because ξ_7 is between 0 and 1, pump 7 is inside the street but still on the correct, faulty side.
4. Pump 8: $f = 1.5 + 0.5 - 3 = -1$, so $y f = -1$. Then $1 - (-1) = 2$, so $\xi_8 = 2$.
5. Because ξ_8 is greater than 1, pump 8 is on the wrong side of the boundary — the model calls it healthy when it is faulty.

Answer: The slacks are $\xi_5 = 0$, $\xi_7 = 0.5$ and $\xi_8 = 2$. Adding the slack of all eight pumps gives $0.5 + 2 = 2.5$, since the first six pumps still have no slack at all.

Figure 16.4 shows the two guilty pumps with an arrow marking how far each falls short.

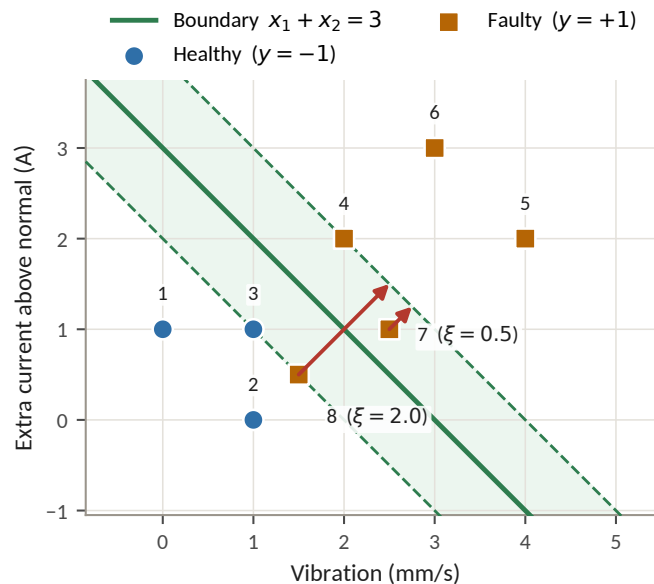


Figure 16.4 – Slack measured against the old street. Pump 7 strays inside the street; pump 8 crosses the boundary completely. Each red arrow points to the faulty edge, which is where these two pumps should have been.

WHAT TO OBSERVE

The arrows are drawn on the score scale, not the map scale: pump 8's slack of 2 means its score is two units short of the +1 it owed, and the true gap in millimeters per second and amperes is $\xi \div \|\mathbf{w}\| = 2 \div 1.41 = 1.41$. Pump 7 costs the model much less than pump 8, which is exactly the behavior we want from a fair charge.

Now the two wishes pull against each other. A wide street on this data swallows pumps 7 and 8 and collects slack; a narrow street avoids every mistake but hugs the data and may not survive the next batch. The soft-margin problem writes both wishes in one line and puts a dial between them.

EQUATION: THE SOFT-MARGIN SUPPORT VECTOR MACHINE

$$\min_{\mathbf{w}, b} \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{keep the street wide}} + C \underbrace{\sum_{i=1}^n \xi_i}_{\text{total rule-breaking}} \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

In words: Add up two costs — one for a narrow street, one for every unit of rule-breaking — and choose the boundary with the smallest total. The number C decides how expensive rule-breaking is.

When it is used: Every real SVM fit. This is what `SVC(kernel="linear")` solves, with $C = 1$ unless you say otherwise.

Symbol	Meaning	Unit or type
C	the cost charged for one unit of slack	no unit
ξ_i, n	the slack of example i , and the number of training examples	no unit; count
$\frac{1}{2} \ \mathbf{w}\ ^2$	the price of a narrow street	no unit

A large C makes mistakes expensive, so the street narrows to avoid them. A small C makes mistakes cheap, so the street widens and tolerates them. As in Chapter 10, small C therefore means strong regularization — the opposite of what the name suggests.

WORKED EXAMPLE 16.5 — Choose Between a Wide Street and a Narrow One at Two Settings of C

Street W is the old one, $\mathbf{w} = (1, 1)$, with $\frac{1}{2} \|\mathbf{w}\|^2 = 1$ and total slack 2.5 over the eight pumps. Street N is $\mathbf{w} = (4, 0)$ with $b = -5$, a narrow vertical street that gets every pump right, so its total slack is 0. Which one has the smaller total cost at $C = 0.1$, and which at $C = 10$?

1. Street N's width penalty: $\|\mathbf{w}\|^2 = 4^2 + 0^2 = 16$, so $\frac{1}{2} \times 16 = 8$. Its margin is $2 \div 4 = 0.50$.
2. At $C = 0.1$: street W costs $1 + 0.1 \times 2.5 = 1.25$, street N costs $8 + 0.1 \times 0 = 8$. Street W wins easily.
3. At $C = 10$: street W costs $1 + 10 \times 2.5 = 26$, street N still costs $8 + 10 \times 0 = 8$. Now street N wins.
4. The two are equal when $1 + 2.5C = 8$, that is when $C = 7 \div 2.5 = 2.8$.

Answer: At $C = 0.1$ the wide street W wins, mistake and all; at $C = 10$ the narrow street N wins. Nothing about the eight pumps changed — only the price of breaking a rule. The tipping point is $C = 2.8$.

Figure 16.5 shows what a real solver does with these eight pumps at two settings of C , and Table 16.3 adds two more settings.

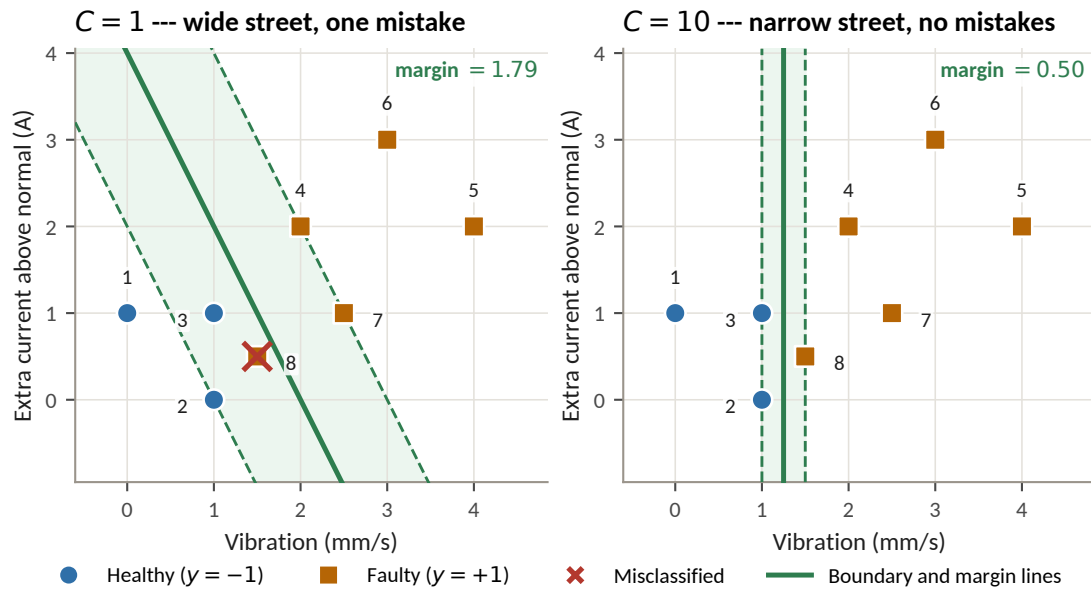


Figure 16.5 – The same eight pumps fitted at two settings of C . A small C buys a wide street and pays for it with one mistake; a large C refuses the mistake and accepts a very narrow street.

WHAT TO OBSERVE

At $C = 1$ the street is 1.79 units wide and pump 8 is left on the wrong side, marked with a cross. At $C = 10$ the boundary swings to a vertical line squeezed between vibration readings of 1.0 and 1.5: every pump is correct, but the street is only 0.50 units wide and the model rests on one odd pump.

Table 16.3 – Four settings of C on the eight pumps, fitted with `SVC(kernel="linear")`

C	Fitted w	Margin width	Mistakes (of 8)	Total slack $\sum \xi_i$
0.1	(0.40, 0.15)	4.68	1	4.17
1	(1.00, 0.50)	1.79	1	1.75
10	(4.00, 0.00)	0.50	0	0.00
100	(4.00, 0.00)	0.50	0	0.00

Read the table downward and the trade-off is plain: as C grows the street narrows and the total slack falls, and beyond $C = 10$ nothing changes because there is nothing left to fix. Which row to choose is a practical question, answered with a validation set in Lesson 16.7.

COMMON MISTAKE

✗ Wrong: Setting C to a very large number “to make the model stronger”, or assuming a large C means a large amount of regularization.

✓ Right: A large C means *less* regularization: the model is punished so hard for every mistake that it will contort itself around a single strange example. Choose C on a validation set, exactly as Chapter 23 recommends, never by feel.

16.5 The Hinge Loss View

There is a second way to read the soft-margin formula, and it connects this chapter to the loss functions of Chapter 10. Look again at the slack, $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$. Nothing in it needs a constraint — it is simply a rule for charging one example, which makes it a loss function with a name of its own.

WORD TO KNOW

Hinge loss — The loss function an SVM minimizes. It charges nothing at all once an example is correct by a full unit of margin, and then rises in a straight line as the example gets worse. Its graph looks like a door hinge: flat, then a sharp corner, then a steady slope.

Because the slack and the hinge loss are the same number, the soft-margin problem can be rewritten with no constraints at all: a penalty on large weights plus a total loss, which is exactly the shape of the regularized problems in Chapter 7 and Chapter 10.

EQUATION: HINGE LOSS AND THE SVM AS A PENALIZED LOSS

$$L_{\text{hinge}}(y, f(\mathbf{x})) = \max(0, 1 - y f(\mathbf{x})), \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n L_{\text{hinge}}(y_i, f(\mathbf{x}_i))$$

In words: Charge an example nothing when its label times its score is at least 1, and otherwise charge it the shortfall. Training minimizes the size of the weights plus C times the total charge.

When it is used: To compare an SVM with other linear models, and to see at a glance which examples a trained SVM is actually paying for.

Symbol	Meaning	Unit or type
L_{hinge}	the hinge loss of one example	no unit
$y f(\mathbf{x})$	the functional margin: label times score	no unit
C	the same dial as in Lesson 16.4	no unit

Only examples with $y f(\mathbf{x}) < 1$ contribute anything — the support vectors and the rule-breakers. Every other example contributes exactly zero, which is why an SVM can forget them.

WORKED EXAMPLE 16.6 — Compute the Hinge Loss of Three Pumps

Using the street $\mathbf{w} = (1, 1)$, $b = -3$, find the hinge loss of pump 5 (4, 2), pump 7 (2.5, 1.0) and pump 8 (1.5, 0.5), all with $y = +1$.

1. Pump 5: $f = +3$, so $y f = 3$ and $1 - 3 = -2$; the loss is $\max(0, -2) = 0$.
2. Pump 7: $f = +0.5$, so $y f = 0.5$ and $1 - 0.5 = 0.5$; the loss is $\max(0, 0.5) = 0.5$.
3. Pump 8: $f = -1$, so $y f = -1$ and $1 - (-1) = 2$; the loss is $\max(0, 2) = 2$.
4. Compare these three numbers with the slacks found in Worked Example 16.4: they are identical.

Answer: The hinge losses are 0, 0.5 and 2 — the very same numbers as the slacks. Slack and hinge loss are two names for one quantity: what one example costs the model.

Figure 16.6 draws hinge loss beside the log loss of Chapter 10 and the plain count of mistakes.

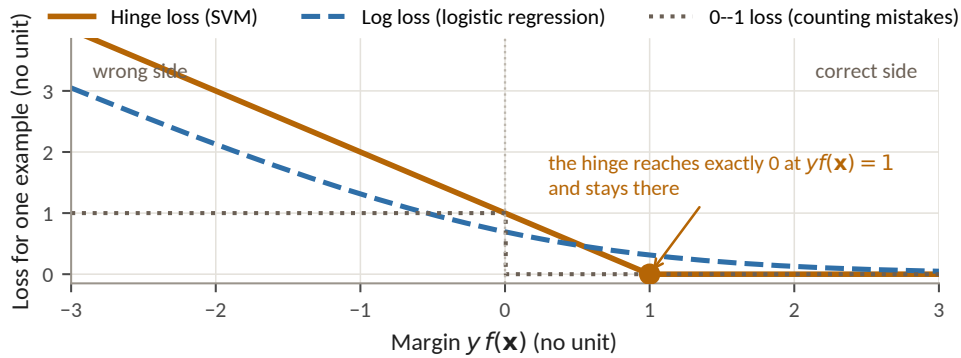


Figure 16.6 – Three ways of charging one example, drawn against its functional margin $y f(\mathbf{x})$. Positive values on the horizontal axis mean the example is on the correct side.

WHAT TO OBSERVE

The dotted 0–1 loss is what we really care about, but its flat steps give a solver no direction to move in, so it cannot be trained on; both alternatives sit above it. The key difference is on the right: hinge loss reaches exactly zero at $y f = 1$ and stays there, while log loss keeps charging a small amount forever. That is why an SVM ignores far-away points and logistic regression does not.

That single difference explains most of the practical contrast between the two methods, set out in Table 16.4. Neither is better in general; they answer slightly different questions.

Table 16.4 – Linear SVM and logistic regression, compared

Question	Linear SVM	Logistic regression (Chapter 10)
What comes out	A score $f(\mathbf{x})$; its sign is the class	A probability between 0 and 1
Loss minimized	Hinge loss	Log loss
Which points matter	Only support vectors and rule-breakers	Every training point, a little
A far-away outlier	Costs nothing and changes nothing	Always adds some cost and tugs the line
Probabilities	Need a second, calibrating fit	Built in
Feature scaling	Essential	Helpful

16.6 Support Vector Regression: a Tube Instead of a Street

The same idea works for predicting numbers. Nasrin’s electricity bill rises on busy days, and she would like to predict the workshop’s daily use from the number of pumps her team tests. Chapter 8 would fit a line by making the squared errors as small as possible, so every point, however close, pulls at the line a little. Support vector regression turns that around: draw a tube of chosen half-width around the fitted line and declare any point inside it close enough, costing nothing at all. Only points outside the tube pay, and only they shape the line.

WORDS TO KNOW

Support vector regression (SVR) — The regression form of an SVM, introduced by Harris Drucker and colleagues in 1996. It fits the flattest line whose tube covers as much of the data as possible.

Epsilon-insensitive tube — The band of half-width ε (the Greek letter epsilon) drawn above and below the fitted line. Errors smaller than ε count as no error at all.

EQUATION: THE EPSILON-INSENSITIVE LOSS

$$L_{\varepsilon}(y, f(x)) = \max(0, |y - f(x)| - \varepsilon)$$

In words: Find the size of the error, ignoring its sign, subtract the tube's half-width, and if the answer is negative write zero instead.

When it is used: To train or judge a support vector regression model, when errors below a certain size genuinely do not matter.

Symbol	Meaning	Unit or type
L_{ε}	the epsilon-insensitive loss of one example	unit of the target
$y, f(x)$	the true value and the predicted value	unit of the target
ε	half the width of the tube, chosen before training	unit of the target

This is the hinge idea again, with the corner moved: flat inside the tube, then a straight rise. Setting $\varepsilon = 0$ charges every error and turns the method back into an ordinary robust regression.

Nasrin records six days. On each day she notes how many pumps were tested and how much electricity the workshop used, in kilowatt hours. She decides that an error below 1 kWh is not worth chasing, so $\varepsilon = 1$, and her software returns the line $f(x) = 2x + 1.5$.

WORKED EXAMPLE 16.7 — Judge a Support Vector Regression Model Day by Day

The fitted model is $f(x) = 2x + 1.5$ with $\varepsilon = 1$ kWh. Compute the prediction, the residual and the loss for the day with 3 pumps (6.3 kWh used) and the day with 4 pumps (11.0 kWh used), then predict the use of a 7-pump day.

1. Three-pump day: $f(3) = 2 \times 3 + 1.5 = 7.5$ kWh.
2. Its residual is $6.3 - 7.5 = -1.2$ kWh, so the size of the error is 1.2 kWh.
3. Its loss is $\max(0, 1.2 - 1) = 0.2$ kWh, so this day is outside the tube and is a support vector.
4. Four-pump day: $f(4) = 2 \times 4 + 1.5 = 9.5$ kWh, and the residual is $11.0 - 9.5 = +1.5$ kWh.
5. Its loss is $\max(0, 1.5 - 1) = 0.5$ kWh, so this day is outside the tube as well.
6. A seven-pump day: $f(7) = 2 \times 7 + 1.5 = 15.5$ kWh.

Answer: The three-pump day costs 0.2 kWh and the four-pump day costs 0.5 kWh; adding the four remaining days, whose losses are all zero, gives a total of 0.7 kWh. The model predicts 15.5 kWh for a seven-pump day.

The remaining four days are the 1-pump day (3.5 kWh used, 3.5 predicted, residual 0.0), the 2-pump day (6.5 against 5.5, residual +1.0), the 5-pump day (10.5 against 11.5, residual -1.0) and the 6-pump day (13.4 against 13.5, residual -0.1). None of them is more than 1 kWh out, so all four cost nothing. Figure 16.7 draws the tube.

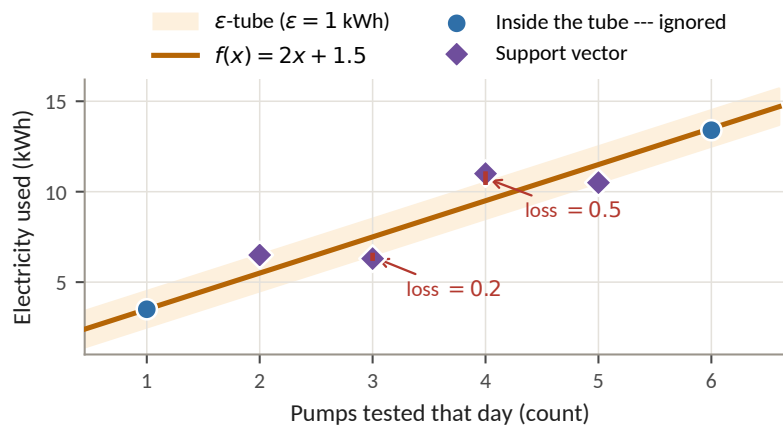


Figure 16.7 – Support vector regression on six days at the workshop. Points inside the shaded tube cost nothing; the two points outside it pay the amount marked in red.

WHAT TO OBSERVE

The 1-pump and 6-pump days sit strictly inside the tube and are ignored completely — the line would be identical if they moved anywhere else inside the band. The other four are support vectors: two exactly on an edge, two hanging outside. As with classification, only a handful of points do any work.

16.7 Scaling, Several Classes, and Practical Advice

Everything in this chapter is built on distance, and distance only means something when the numbers being compared are comparable. That makes SVMs unusually sensitive to the units a feature happens to be recorded in — far more sensitive than a decision tree (Chapter 13), which never compares two features at all. Suppose Nasrin’s meter recorded extra current in milliamperes instead of amperes. No pump would change; only the numbers on the page would.

WORKED EXAMPLE 16.8 — See a Change of Units Destroy a Distance

Pump 3 reads (1, 1) and pump 4 reads (2, 2) in millimeters per second and amperes. Find the squared distance between them, then repeat the calculation with the current recorded in milliamperes, where pump 3 becomes (1, 1000) and pump 4 becomes (2, 2000).

1. In mm/s and A: the vibration difference is $2 - 1 = 1$, and $1^2 = 1$; the current difference is $2 - 1 = 1$, and $1^2 = 1$.
2. Add: the squared distance is $1 + 1 = 2$, so the distance is $\sqrt{2} = 1.41$. Vibration supplies half of it, $1 \div 2 = 0.5$.
3. In mm/s and mA: the vibration difference is still 1, so $1^2 = 1$; the current difference is $2000 - 1000 = 1000$, and $1000^2 = 1,000,000$.
4. Add: the squared distance is 1,000,001, so the distance is 1000.0005.
5. Vibration now supplies $1 \div 1,000,001$, about one millionth, of the squared distance.

Answer: The same two pumps are 1.41 apart in one set of units and 1000.0005 apart in another. Worse, vibration went from supplying half the distance to supplying a millionth of it. The model has effectively lost a feature, and no one changed a single measurement.

The same trouble reaches the training step. The penalty $\frac{1}{2}\|\mathbf{w}\|^2$ adds up the weights as though they were comparable, but a weight on a feature measured in thousands must be about a thousand times smaller than a weight on a feature measured in single digits to have the same effect, so the balance between a wide street

and a small total slack is settled by the accident of the measuring units. On 180 records from Nasrin's files — vibration in mm/s beside a pump's running hours since its last service, a number in the thousands — a linear SVM trained at $C = 1$ on the raw numbers scored 72% on validation, leaning almost entirely on vibration and getting the running hours nearly backward. The same model trained on standardized features scored 85%, and finished training thousands of times faster — solvers struggle badly when one feature is thousands of times larger than another.

COMMON MISTAKE

X Wrong: Standardizing the whole dataset first, then splitting it into training and validation sets.

✓ Right: Fit the scaler on the training rows only and apply it to the validation and test rows, or — much safer — put the scaler and the SVM into one scikit-learn Pipeline. Chapter 5 explains why letting validation rows influence the mean and standard deviation is a form of leakage.

An SVM answers a yes-or-no question, so it needs help with three or more classes. Two standard arrangements do the job, and you have met the first already.

WORD TO KNOW

One-vs-one (OVO) — A way of handling several classes by training one SVM for every *pair* of classes and letting the pairwise winners vote: $K(K - 1)/2$ models in all — three for three classes, ten for five. Each trains on only two classes' worth of data, so each is fast. Scikit-learn's SVC uses this arrangement.

One-vs-rest, from Chapter 10, trains one model per class instead — three models for three classes — and picks whichever gives the largest score. It builds fewer models, but each sees the whole dataset; scikit-learn's LinearSVC uses this arrangement. For most problems the two give very similar accuracy.

Finally, C itself must be chosen. It is a hyperparameter, so it is chosen on a validation set and never on the test set (Chapter 5). Figure 16.8 sweeps C across six powers of ten on Nasrin's records.

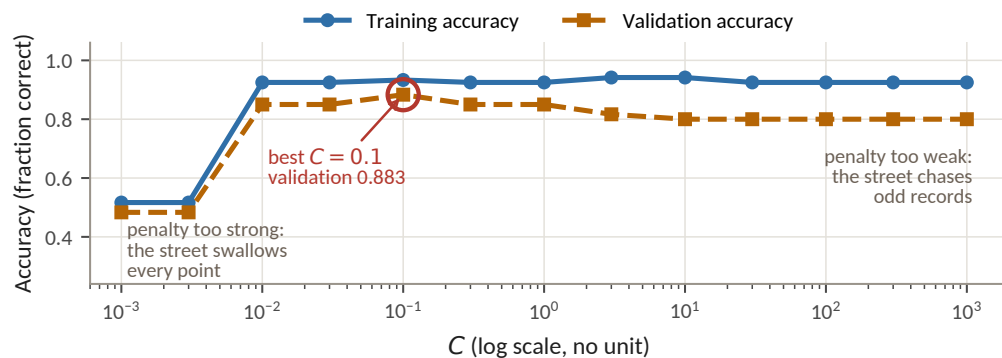


Figure 16.8 – Training and validation accuracy as C is swept from 0.001 to 1000 on 120 training and 60 validation records, with standardized features.

WHAT TO OBSERVE

The curve has the familiar hill shape of Chapter 7. Below $C = 0.01$ the penalty is so strong that the street grows wider than the data and the model guesses. The best validation accuracy, 0.883, arrives at $C = 0.1$. Above $C = 3$ the model chases odd records and slides to 0.800 while training accuracy does not improve.

Three practical notes close the lesson. Training cost grows roughly with the square of the number of training rows, so for very large datasets prefer LinearSVC or a stochastic-gradient solver; ordinary SVM training was made practical by John Platt's sequential minimal optimization, published in 1998. If one class is much rarer than the other, set `class_weight="balanced"` and judge the result with the measures of Chapter 6 rather than

accuracy. And when no straight boundary works at all, the same machinery bends into curves through kernels, the subject of Chapter 17.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing that any line separating the training data is as good as any other.
- Reading the decision function $f(\mathbf{x})$ as a probability instead of as a signed, distance-like score.
- Thinking every training example shapes an SVM's boundary; only the support vectors do.
- Getting the C dial backward: a small C is a strong penalty and a wide, forgiving street.
- Training an SVM on unscaled features, so that a change of units alone can decide where the boundary goes.
- Choosing C by trying values on the test set instead of on a validation set.

Practical Activities

ACTIVITY (PAPER) — Find the Widest Street by Hand

A second workshop in Duhok tests six pumps and records the same two readings. The healthy pumps are (1, 1), (2, 1) and (1, 2); the faulty pumps are (3, 4), (4, 3) and (4, 4), with vibration first and extra current second.

1. Plot all six points on graph paper, one large square per unit, using different symbols for the two classes.
2. Lay a ruler across the gap and slide it until you find the widest empty band separating the groups. Draw the two edges, then the boundary exactly halfway between them.
3. Mark every point that touches an edge of your band — these are the support vectors. How many did you find?
4. Read off the boundary. You should find $x_1 + x_2 = 5$, with edges at $x_1 + x_2 = 3$ and $x_1 + x_2 = 7$. Check the boundary on all six points.
5. The SVM writes those edges as $f(\mathbf{x}) = \pm 1$ with $\mathbf{w} = (0.5, 0.5)$ and $b = -2.5$. Check this by computing f for one point on each edge, then find $\|\mathbf{w}\| = \sqrt{0.5^2 + 0.5^2}$ and $M = 2 \div \|\mathbf{w}\|$. Measure your band with the ruler and compare.
6. Now erase the point (4, 4) and repeat step 2. Does the boundary move? Explain in one sentence using the words “support vector”.

You need: Graph paper, a pencil, a ruler and a calculator with a square-root key

ACTIVITY (PYTHON) — Linear SVM, the C Dial and an SVR Tube

This program fits the six pumps at a very large C , sweeps C over all eight, then fits the electricity days with SVR.

```

import numpy as np
from sklearn.svm import SVC, SVR
X = np.array([[0, 1], [1, 0], [1, 1], [2, 2], [4, 2], [3, 3]], dtype=float)
y = np.array([-1, -1, -1, 1, 1, 1])
m = SVC(kernel="linear", C=1000).fit(X, y)      # C very large = hard margin
w = m.coef_[0]
print("w =", w, " b = %.1f  margin = %.3f" % (m.intercept_[0], 2 / np.linalg.norm(w)))
print("f(x) =", m.decision_function(X).round(2), " SVs:", m.support_vectors_.tolist())
X8 = np.vstack([X, [[2.5, 1.0], [1.5, 0.5]]])  # two more faulty pumps
y8 = np.append(y, [1, 1])
for C in [0.1, 1, 10, 100]:
    s = SVC(kernel="linear", C=C).fit(X8, y8)
    slack = np.maximum(0, 1 - y8 * s.decision_function(X8)).sum()
    print("C =%7g  margin %.2f  mistakes %d  slack %.2f" % (C,
        2 / np.linalg.norm(s.coef_[0]), (s.predict(X8) != y8).sum(), slack))
days = np.arange(1, 7, dtype=float).reshape(-1, 1)  # pumps tested that day
kwh = np.array([3.5, 6.5, 6.3, 11.0, 10.5, 13.4])
r = SVR(kernel="linear", epsilon=1.0, C=10).fit(days, kwh)
resid = kwh - r.predict(days)
print("SVR: slope %.2f  intercept %.2f" % (r.coef_[0][0], r.intercept_[0]))
print("residuals:", resid.round(2) + 0.0, "losses:", np.maximum(0, abs(resid)-1).round(2))

```

Output

```

w = [1. 1.]  b = -3.0  margin = 1.414
f(x) = [-2. -2. -1.  1.  3.  3.]  SVs: [[1.0, 1.0], [2.0, 2.0]]
C =  0.1  margin 4.68  mistakes 1  slack 4.17
C =  1  margin 1.79  mistakes 1  slack 1.75
C = 10  margin 0.50  mistakes 0  slack 0.00
C = 100  margin 0.50  mistakes 0  slack 0.00
SVR: slope 2.00  intercept 1.50
residuals: [ 0.  1. -1.2  1.5 -1. -0.1] losses: [0.  0.  0.2 0.5 0.  0. ]

```

The library agrees with every hand calculation: $\mathbf{w} = (1, 1)$, $b = -3$, a margin of 1.414, scores of $-2, -2, -1, +1, +3, +3$, and exactly two support vectors. The C sweep reproduces Table 16.3, and the SVR fit returns the line $2x + 1.5$ of Worked Example 16.7 with the same residuals and losses. Then try three changes: move pump 8 to $(1.5, 2.5)$ and watch the mistakes disappear; drop pump 5 and confirm the boundary does not move at all; set $\epsilon=0.1$ and see almost every day become a support vector.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Catching Card Fraud with a Linear SVM — Erbil

A bank in Erbil wanted a fast first filter for card transactions: a model that could flag a payment for a human reviewer within a fraction of a second, before it was approved. The team chose a linear SVM because it is quick to score, gives one interpretable weight per feature, and can be retrained overnight on a laptop. The numbers below are illustrative, not published statistics.

The features were the sort a payment carries with it: the amount in Iraqi dinars, the hour of the day, the days since the card's previous payment, and the distance in kilometers between the shop and the customer's usual area. The trouble was visible from the first table. Amounts ran from a few thousand dinars to several million, distances from 0 to about 40 kilometers, and the hour from 0 to 23. On these

raw numbers the first model was almost useless: it caught only a small share of the fraud in the validation month, and training took several minutes for a dataset that should have taken seconds. As Lesson 16.7 explains, the penalty $\frac{1}{2}\|\mathbf{w}\|^2$ had been comparing weights that were not comparable, so the fit was decided by the units rather than by the evidence.

The fix was two lines of code and one careful habit: a `StandardScaler` joined to the classifier inside a single `Pipeline`, fitted only on training rows so that no validation month could leak into a mean or a standard deviation. Recall on the validation month roughly doubled and training finished in seconds. The same tidy pipeline made it easy to sweep C over powers of ten, exactly as Figure 16.8 does, and to keep the value that scored best on validation — not on the test month, which stayed sealed until the very end. Two further adjustments mattered more than the choice of algorithm. Fraud was rare, well under one payment in a hundred, so the team set `class_weight="balanced"`; without it the cheapest possible model called everything genuine and scored a magnificent accuracy while catching nothing, the trap Chapter 6 warns about. And because an SVM returns a score rather than a probability, reviewers received a ranked queue, highest score first, rather than a number pretending to be a chance of fraud; the threshold on that score was chosen by asking how many reviews the team could handle in a day. The finished filter never blocks a payment by itself — it orders a queue for people, who make every decision that touches a customer, and it is retrained monthly because fraud patterns move.

RESPONSIBLE USE

A flagged transaction is not a fraudulent transaction, and a flagged person is not a suspect. Models built from past records reproduce the patterns of those records, including patterns that reflect where people live or how much they earn rather than what they did. An SVM also gives a score, never a probability: a score of 2.4 does not mean “very likely”, and presenting one to a non-specialist as though it were a chance is misleading. Use a filter like this to order human attention, never to punish, and always keep a fast, respectful route for a customer to have a wrong decision undone. Chapter 24 returns to this in depth.

CHAPTER SUMMARY

- When two classes are linearly separable, infinitely many lines separate them; a support vector machine picks the one with the widest empty street around it.
- A linear classifier scores an example with the decision function $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$, and the sign of that score is the prediction.
- The distance from a point to the boundary is $|f(\mathbf{x})| \div \|\mathbf{w}\|$, and the margin — the full width of the street — is $2 \div \|\mathbf{w}\|$, so a shorter $\mathbf{w}$ means a wider street.
- Support vectors are the training examples with $y f(\mathbf{x}) = 1$, sitting exactly on the edges of the street; every other example could be deleted without changing the boundary.
- The hard-margin SVM minimizes $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$, and has no solution when the classes overlap.
- The slack $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$ measures how far one example falls short: zero outside the street, between 0 and 1 inside it, and above 1 on the wrong side.
- The soft-margin SVM minimizes $\frac{1}{2}\|\mathbf{w}\|^2 + C \sum \xi_i$: a small C buys a wide, forgiving street, a large C a narrow street that refuses mistakes.
- Slack and hinge loss are the same number; hinge loss charges nothing once an example is correct by a full unit, which is exactly why far-away points do not move an SVM.
- Support vector regression replaces the street with an ε -tube: errors smaller than ε cost nothing, and only points on or outside the tube shape the line.
- Features must be standardized before an SVM is trained, because a change of units alone can destroy a distance and decide the boundary; C is then chosen on a validation set.

- Several classes are handled by one-vs-one or one-vs-rest; curved boundaries need the kernels of Chapter 17.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain in your own words what the margin of a support vector machine is, and why a wider margin is usually preferred to a narrower one.

Problem 2 [Basic] Define “support vector” in one sentence. In a training set of 500 examples, is it possible for only 6 of them to be support vectors? Explain briefly.

Problem 3 [Basic] A model has $\mathbf{w} = (2, -1)$ and $b = 1$. Compute $f(\mathbf{x})$ for the point $(3, 2)$ and state which class, $+1$ or -1 , the model predicts.

Problem 4 [Basic] For $\mathbf{w} = (3, 4)$, compute $\|\mathbf{w}\|$ and then the margin $M = 2 \div \|\mathbf{w}\|$, showing every step.

Problem 5 [Basic] State whether each sentence describes a small C or a large C : (a) the street is wide and several points are allowed inside it; (b) the model bends itself to classify one unusual example correctly; (c) regularization is strong.

Problem 6 [Standard] Using $\mathbf{w} = (1, 1)$ and $b = -3$, compute $f(\mathbf{x})$ and $y f(\mathbf{x})$ for the point $(2.5, 1.5)$ with true label $y = +1$, and say whether it is a support vector, inside the street, or comfortably outside.

Problem 7 [Standard] A point has $f(\mathbf{x}) = 0.4$ and true label $y = +1$; a second point has $f(\mathbf{x}) = 0.5$ and true label $y = -1$. Compute the hinge loss of each, showing every step, and say which point costs the model more.

Problem 8 [Standard] Street P has $\frac{1}{2}\|\mathbf{w}\|^2 = 2$ and total slack 4. Street Q has $\frac{1}{2}\|\mathbf{w}\|^2 = 9$ and total slack 0.5. Compute both total costs at $C = 0.5$ and at $C = 5$, and say which street training would choose in each case.

Problem 9 [Standard] An SVR model is $f(x) = 3x + 2$ with $\varepsilon = 2$. For the three points $(1, 6)$, $(2, 11)$ and $(3, 14)$, compute each prediction, residual and epsilon-insensitive loss, and name the support vectors.

Problem 10 [Standard] A dataset has three classes. How many models does one-vs-one build, and how many does one-vs-rest build? Show the arithmetic for one-vs-one using $K(K - 1)/2$.

Problem 11 [Challenge] A hard-margin SVM on a small dataset returns $\mathbf{w} = (0.5, 0.5)$ and $b = -2.5$. (a) Write the equation of the boundary in the simplest form. (b) Find the margin. (c) A new training point $(6, 6)$ of class $+1$ is added. Will the boundary move? Explain using the value of $y f(\mathbf{x})$.

Problem 12 [Challenge] One feature is a temperature in degrees Celsius, running from 20 to 45; the other is a pressure in pascals, running from 90,000 to 110,000. (a) Compute the squared distance between the readings $(25, 100,000)$ and $(35, 100,100)$. (b) Say which feature decides that distance and by roughly what factor. (c) Describe in one sentence what standardizing would change.

Problem 13 [Challenge] Compare hinge loss and log loss for a training example that is correct and far from the boundary, with $y f(\mathbf{x}) = 5$. (a) Compute the hinge loss. (b) Is the log loss exactly zero, or only close to zero? (c) Use your answers to explain why a single far-away outlier can move a logistic regression boundary but not an SVM boundary. (d) Nasrin's validation accuracy peaks at $C = 0.1$ and falls at $C = 100$ while her training accuracy barely changes — name the problem this shows, and say why training accuracy failed to reveal it.

Kernel Methods

Bending straight lines into curves without leaving the data behind

IN THIS CHAPTER YOU WILL LEARN TO

- Recognize data that no straight line can separate, and prove it for a small example.
- Build new features by hand — squares and products — that turn a curved problem into a straight-line problem.
- Compute polynomial and RBF kernel values by hand, and check that a kernel really does equal the dot product of the mapped points.
- Explain the kernel trick in plain words, and say exactly what work it saves.
- Predict with a kernel SVM decision function from stored support vectors, their weights and a kernel.
- Choose C and γ together from a validation grid, and recognize the too-smooth and too-wiggly extremes.
- Use a kernel outside classification, and judge when a kernel method is simply too expensive.

Introduction

Think about salting a pot of soup. A little salt with a little water tastes fine. A lot of salt with a lot of water also tastes fine, because the two balance each other. But a lot of salt with only a little water is far too salty, and a lot of water with only a pinch of salt is bland. Whether the soup is good does not depend on the salt alone, and it does not depend on the water alone. It depends on the two *together*.

Every model in this book that draws a straight boundary — logistic regression in Chapter 10, the linear support vector machine in Chapter 16 — would fail on that soup. A straight boundary can say “more salt is worse”, but it cannot say “salt is bad only when water is low”. Many real problems have this shape: a reading is fine in the middle and worrying at both ends, or a machine is healthy only when two dials move together.

The fix is simple and surprisingly powerful. Instead of forcing a curved boundary into the data, we invent one new measurement — the salt multiplied by the water — and the very same straight-line model works perfectly. This chapter starts there, with features built by hand, and then shows the shortcut that made these methods famous: a small formula, called a *kernel*, that gives the answer the new features would have given without ever building them. Everything in Chapter 16 — the margin, the support vectors, the C parameter — stays as it was. Only the way two points are compared changes.

17.1 When No Straight Line Works

A farm near Zakho keeps records of four irrigation trials. Each trial recorded two settings — the water level and the fertilizer level, each written simply as low (-1) or high ($+1$) — and whether the crop came out healthy. Table 17.1 lists them (ignore the last column for now).

The two healthy fields had both settings low or both settings high. The two damaged fields each had one setting high and the other low: too much fertilizer with too little water burns the roots, and too much water with too little fertilizer washes the nutrients away. Balance is what matters, not the size of either number on its own.

Table 17.1 – Four irrigation trials near Zakho. The last column is a new feature, built in Lesson 17.2.

Field	Water level x_1	Fertilizer level x_2	Crop	New feature x_1x_2
F1	+1 (high)	+1 (high)	Healthy	+1
F2	−1 (low)	−1 (low)	Healthy	+1
F3	+1 (high)	−1 (low)	Damaged	−1
F4	−1 (low)	+1 (high)	Damaged	−1

Plot the four fields, as the left panel of Figure 17.1 does, and the problem jumps out. The healthy fields sit at opposite corners of a square, and the damaged fields sit at the other two corners. Any straight line across that square keeps one healthy field on the same side as a damaged one. In the words of Chapter 16, this data is *not linearly separable*: no single straight boundary can put one class on one side and the other class on the other.

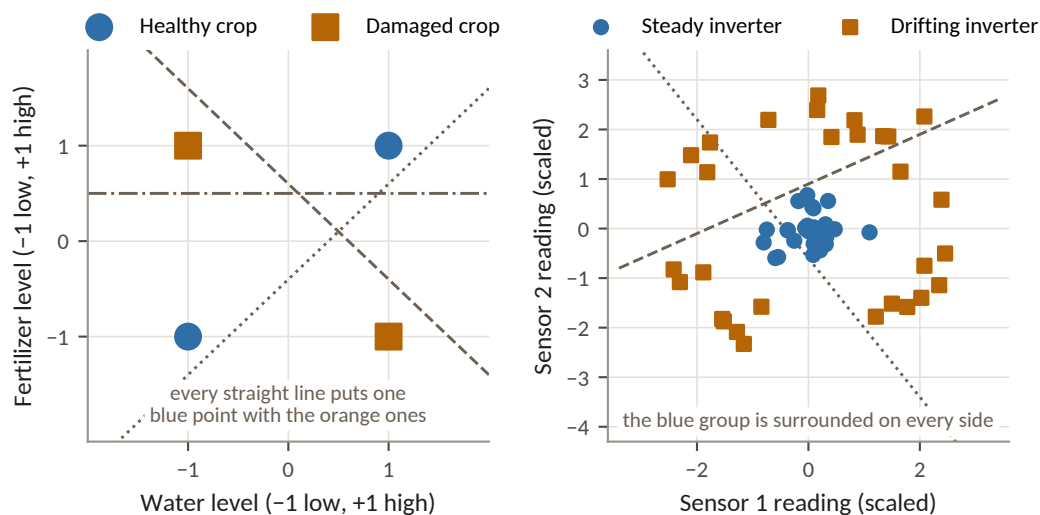


Figure 17.1 – Two datasets that no straight line can separate. Left: the four irrigation trials, where the healthy fields sit at opposite corners. Right: 60 inverters whose steady readings form a round island surrounded by drifting ones. The gray lines are attempts; every one of them fails.

WHAT TO OBSERVE

In the left panel every gray line leaves a blue point together with an orange one. In the right panel the blue group is completely surrounded, so no straight cut can fence it off. The best a straight line can do on the four fields is three correct out of four — never four.

The next worked example proves that with nothing more than addition. As in Chapter 16, a straight-line model scores a point with $f(\mathbf{x}) = w_1x_1 + w_2x_2 + b$ and predicts “healthy” when the score is positive.

WORKED EXAMPLE 17.1 — Prove That No Straight Line Separates the Four Fields

Suppose some straight-line model $f(\mathbf{x}) = w_1x_1 + w_2x_2 + b$ gets all four fields right: positive for F1 and F2, negative for F3 and F4. Show that this is impossible.

1. Write the score of each field. F1 $(+1, +1)$: $w_1 + w_2 + b$. F2 $(-1, -1)$: $-w_1 - w_2 + b$. F3 $(+1, -1)$: $w_1 - w_2 + b$. F4 $(-1, +1)$: $-w_1 + w_2 + b$.
2. The two healthy scores must both be positive, so their sum is positive: $(w_1 + w_2 + b) + (-w_1 - w_2 + b) = 2b > 0$. Therefore $b > 0$.
3. The two damaged scores must both be negative, so their sum is negative: $(w_1 - w_2 + b) + (-w_1 + w_2 + b) = 2b < 0$. Therefore $b < 0$.
4. Steps 2 and 3 demand that the same number b be both greater than zero and less than zero. That is impossible.

Answer: No straight line separates the four fields, whatever w_1 , w_2 and b we choose. The weights cancelled out completely, so this is not about choosing them better — a straight line is the wrong shape for this problem.

This pattern is famous: it is called the *XOR problem*, and Marvin Minsky and Seymour Papert used it in 1969 to show the limits of the earliest straight-line classifiers. Their point was correct, and still is — but the limit belongs to the *features*, not to the method.

COMMON MISTAKE

X Wrong: Concluding that a dataset with a curved pattern cannot be handled by a linear model, and abandoning it.

✓ Right: Keep the linear model and change the space it works in. Adding one well-chosen feature — here, the water level multiplied by the fertilizer level — often makes the very same straight-line model succeed.

17.2 Adding Features by Hand

A dairy in Sulaymaniyah stores milk in a chiller whose target temperature is fixed. Every batch is recorded by one number: how far the chiller drifted from that target, in degrees Celsius. Five batches were recorded at drifts of -3 , -1 , 0 , $+1$ and $+3$; the two extreme batches spoiled and the three middle ones were good.

With one feature the picture is a number line, and again no single cut works: the good batches sit in the middle with a spoiled batch on each side. But look what happens if we write down a second number for every batch — the drift multiplied by itself.

WORDS TO KNOW

Input space — The space of the original measured features — one axis per column of the dataset.

Feature space — A new, usually larger space whose axes are quantities computed from the original features, such as squares and products. A model trained in feature space can look curved when we draw it back in the input space.

Feature map — The rule, written φ (phi), that turns one point of the input space into its point in feature space. Every example, training or new, must go through exactly the same map.

The simplest useful map squares each feature and multiplies pairs of features together. For a single feature the map is $\varphi(x) = (x, x^2)$; for two features it is written out in the box below.

EQUATION: DEGREE-2 FEATURE MAP (SQUARES AND PRODUCTS)

$$\varphi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

In words: Replace a point's two measurements by three new ones: the first squared, the product of the two (stretched by $\sqrt{2}$), and the second squared.

When it is used: When the boundary you need is curved — a circle, an ellipse, or a rule that depends on two features acting together — but you still want to fit a straight-line model.

Symbol	Meaning	Unit or type
φ	the feature map, from input space to feature space	function
x_1, x_2	the original two features of one example	unit of each feature
x_1x_2	the interaction (product) feature	product of the two units
$\sqrt{2}$	a fixed stretch that makes the kernel of Lesson 17.3 match exactly	none

With one feature the same idea shrinks to $\varphi(x) = (x, x^2)$. The middle entry solves the irrigation problem; the squares solve the chiller problem.

WORKED EXAMPLE 17.2 — Lift the Five Chiller Batches with a Squared Feature

The five batches have drifts $x = -3, -1, 0, +1, +3$ degrees Celsius. The batches at -3 and $+3$ spoiled; the other three were good. Add the feature $u = x^2$ and find a rule that separates the two classes.

1. Compute $u = x^2$ for every batch: $(-3)^2 = 9$, $(-1)^2 = 1$, $0^2 = 0$, $1^2 = 1$, $3^2 = 9$.
2. Collect the good batches' new values: 1, 0, 1. The largest is 1.
3. Collect the spoiled batches' new values: 9 and 9. The smallest is 9.
4. Any threshold between 1 and 9 separates them. Choose the middle-ish value $u = 5$: predict spoiled when $u > 5$, good when $u < 5$.
5. Translate back to the original unit: $u = 5$ means $x^2 = 5$, so $x = \pm\sqrt{5} = \pm 2.24$ degrees Celsius.

Answer: The straight rule " $u > 5$ means spoiled" gets all five batches right. In the original feature it reads "spoiled if the drift is more than about 2.24 degrees in either direction" — a two-sided rule no single threshold on x could produce.

Figure 17.2 shows both views of those five batches side by side.

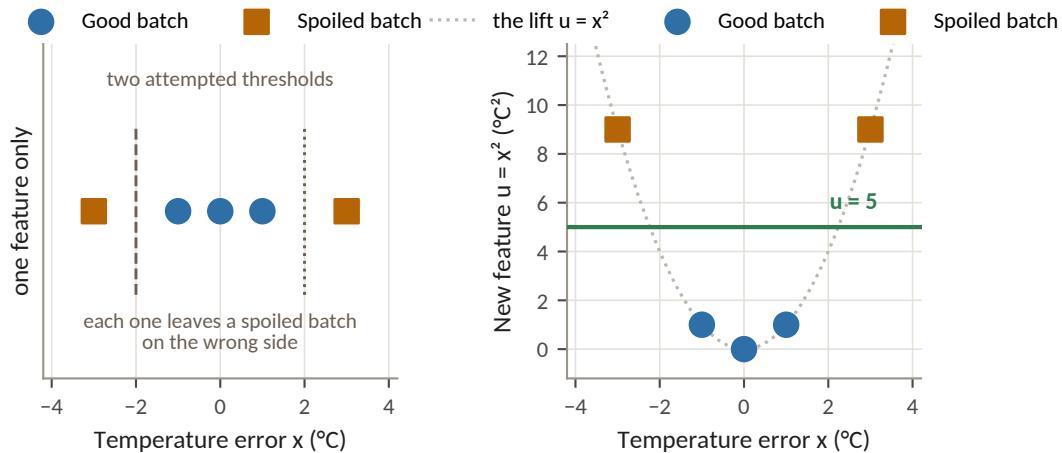


Figure 17.2 – Lifting one feature into two. Left: the five chiller batches on a number line — no single cut works. Right: the same five batches after adding $u = x^2$, where the flat green line $u = 5$ separates them cleanly.

WHAT TO OBSERVE

Nothing about the batches changed — only the picture did, as every point slid up onto the dotted parabola. The straight line in the right panel is really two cut points, at -2.24 and $+2.24$, in the left one.

The irrigation trials work the same way, using the product feature instead of the square. The last column of Table 17.1 is $x_3 = x_1x_2$: it equals $(+1)(+1) = +1$ for F1 and $(-1)(-1) = +1$ for F2, but $(+1)(-1) = -1$ for F3 and $(-1)(+1) = -1$ for F4. Both healthy fields now share one value and both damaged fields share the other, so the straight rule “healthy when $x_3 > 0$ ” — which is the score $f(\mathbf{x}) = 0x_1 + 0x_2 + 1x_3 + 0$ — is right on all four. The same trick scales up to two-dimensional data. Figure 17.3 uses 60 solar inverters whose two scaled sensor readings form a round island of steady units surrounded by drifting ones. Adding the single new feature $u = x_1^2 + x_2^2$ — how far the point sits from the center — turns the ring problem into a flat cut.

Whatever map you choose, every example must go through exactly the same one — training rows and new arrivals alike, which is what a scikit-learn Pipeline is for (Chapter 22). So far, so good — but the cost grows fast. With two features the degree-2 map produced 3 new ones. With 100 features and degree 3, the same idea produces **176,851** terms for every single example: impossible to store for a large dataset, and slow to multiply. Lesson 17.3 removes that cost completely.

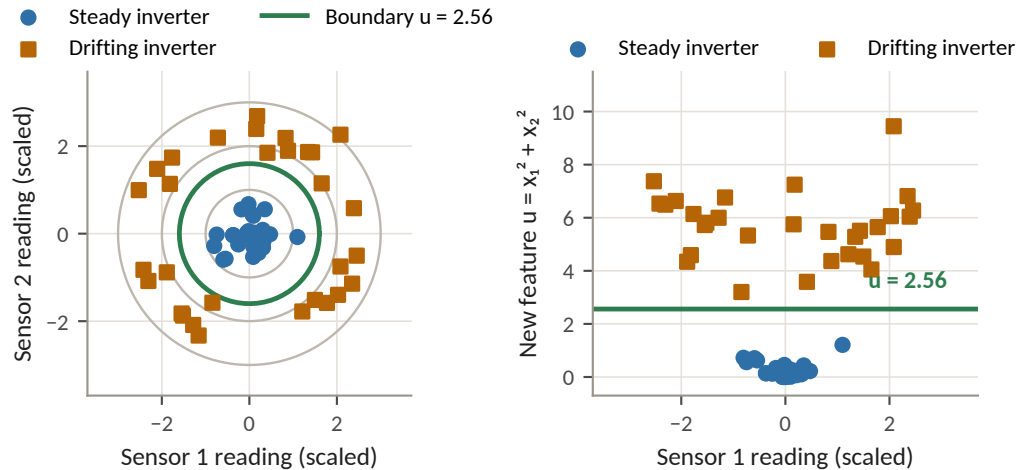


Figure 17.3 – Lifting two features. Left: the readings in the input space, with light circles showing where $x_1^2 + x_2^2$ is constant and the green circle marking the chosen boundary. Right: the same 60 points with the new feature $u = x_1^2 + x_2^2$ on the vertical axis, separated by the flat line $u = 2.56$.

WHAT TO OBSERVE

The green circle on the left and the green straight line on the right are the same boundary, drawn in two different spaces. Lifting did not bend the model; it bent the space, and left the model straight.

17.3 The Kernel Trick: Similarity Instead of Coordinates

Here is the observation that changes everything. A support vector machine never needs a point's coordinates on their own: as Chapter 16 showed, both training and prediction use the data only through *dot products* between pairs of points. So if we can get the dot product of two *mapped* points without building the maps, we get the benefit of the lift and pay none of its cost.

WORD TO KNOW

Kernel function — A function $K(\mathbf{x}, \mathbf{x}')$ of two examples that returns the dot product those two examples would have had in some feature space — a single number measuring how similar they are. Large means similar; small (or zero) means unrelated.

EQUATION: A KERNEL IS A DOT PRODUCT IN FEATURE SPACE

$$K(\mathbf{x}, \mathbf{x}') = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$$

In words: The kernel of two points equals the dot product you would get by mapping both points into the bigger feature space and multiplying them there.

When it is used: Whenever a method uses the data only through dot products — support vector machines, ridge regression, principal component analysis — so each dot product can be swapped for a kernel.

Symbol	Meaning	Unit or type
K	the kernel function; one number per pair of examples	similarity score
$\mathbf{x}, \mathbf{x}'$	two examples, each a list of features	feature units
φ	the feature map hiding behind the kernel	function
$\cdot$	the dot product: multiply matching entries, then add	number

In practice the equation is read from right to left: we choose a convenient K , and the matching φ exists in the background whether or not we ever write it down.

For the degree-2 map of the previous lesson the matching kernel is astonishingly simple: take the ordinary dot product of the two *original* points and square it.

EQUATION: POLYNOMIAL KERNEL

$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$$

In words: Take the plain dot product of the two points, add a constant, and raise the result to the power d .

When it is used: When you believe the answer depends on products of features up to a known degree — pairs of features acting together ($d = 2$), triples ($d = 3$), and so on.

Symbol	Meaning	Unit or type
d	the degree; how many features may be multiplied together	count, usually 2 or 3
c	a constant, often 0 or 1; $c > 0$ also keeps the lower-degree terms	number
$\mathbf{x} \cdot \mathbf{x}'$	dot product of the two original points	feature units squared

With $c = 0$ and $d = 2$ this kernel matches the map $\varphi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$ exactly — which is where that $\sqrt{2}$ came from. In scikit-learn the constant is `coef0`, and it defaults to 0.

WORKED EXAMPLE 17.3 — Compute a Polynomial Kernel the Long Way and the Short Way

Two examples are $\mathbf{x} = (2, 3)$ and $\mathbf{x}' = (1, 4)$. Compute $\varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$ using the degree-2 map, then compute $(\mathbf{x} \cdot \mathbf{x}')^2$, and compare.

1. Map the first point: $\varphi(2, 3) = (2^2, \sqrt{2} \cdot 2 \cdot 3, 3^2) = (4, 6\sqrt{2}, 9)$, that is about $(4, 8.4853, 9)$.
2. Map the second point: $\varphi(1, 4) = (1^2, \sqrt{2} \cdot 1 \cdot 4, 4^2) = (1, 4\sqrt{2}, 16)$, about $(1, 5.6569, 16)$.
3. Dot product in the new space, term by term: $4 \times 1 = 4$; $(6\sqrt{2})(4\sqrt{2}) = 6 \times 4 \times 2 = 48$; $9 \times 16 = 144$.
4. Add them: $4 + 48 + 144 = 196$. That is the long way, and it took nine multiplications.
5. Now the short way. Dot product of the original points: $2 \times 1 + 3 \times 4 = 2 + 12 = 14$.
6. Square it: $14^2 = 196$. Three multiplications, no map, no new features.

Answer: Both routes give exactly 196. The kernel $(\mathbf{x} \cdot \mathbf{x}')^2$ returns the dot product of the mapped points without ever building them.

Table 17.2 repeats the check for three pairs, and Figure 17.4 shows the two routes side by side.

Table 17.2 – The polynomial kernel of degree 2 ($c = 0$) checked against the long way, for three pairs of points

$\mathbf{x}$	$\mathbf{x}'$	$\mathbf{x} \cdot \mathbf{x}'$	$K = (\mathbf{x} \cdot \mathbf{x}')^2$	Long way: $\varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$
(2, 3)	(1, 4)	14	196	$4 + 48 + 144 = 196$
(2, 3)	(3, 1)	9	81	$36 + 36 + 9 = 81$
(1, 4)	(3, 1)	7	49	$9 + 24 + 16 = 49$

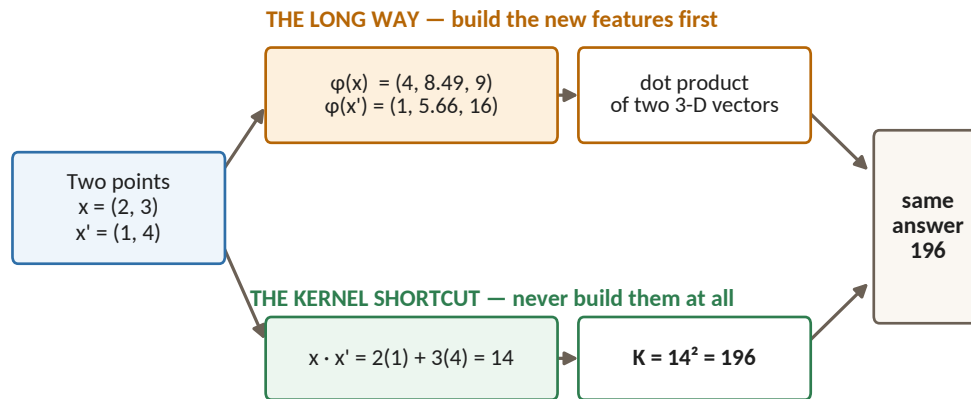


Figure 17.4 – Two routes to the same number. The long way builds three new features for each point and takes a dot product in the new space. The kernel shortcut squares one ordinary dot product.

WHAT TO OBSERVE

Both paths end at 196. The upper path stores three numbers per point — 176,851 of them with 100 features and degree 3. The lower path stores nothing extra at all, and that is the whole idea.

WORD TO KNOW

Kernel trick — Replacing every dot product inside an algorithm by a kernel function, so that the algorithm behaves as though it were working in a large feature space while all the arithmetic stays in the original, small one.

The idea of computing with such similarity functions goes back to Mark Aizerman, Emmanuil Braverman and Lev Rozonoer in 1964, and it was put together with the support vector machine by Bernhard Boser, Isabelle Guyon and Vladimir Vapnik in 1992 — the step that turned the linear SVM of Chapter 16 into one of the strongest classifiers of its time.

Not every formula of two arguments is a kernel: a valid kernel must be a genuine dot product in *some* feature space, a condition proved by the mathematician James Mercer in 1909. In practice you never check it yourself — you choose from the short list in the next lesson, all of which are known to be valid. During training the algorithm needs the kernel of every pair of training examples, and those numbers are stored in one square table.

WORD TO KNOW

Kernel matrix — The square table holding $K(\mathbf{x}_i, \mathbf{x}_j)$ for every pair of training examples. With n training examples it has $n \times n$ entries. It is also called the *Gram matrix*.

COMMON MISTAKE

X Wrong: Believing that a kernel method actually moves the data into a huge space, and worrying about how the computer stores it.

✓ Right: Nothing is ever moved. Only pairwise numbers are computed, one per pair of examples. The large space is a way of explaining why the method works, not a place where anything is stored.

17.4 Common Kernels and the Gamma Parameter

Three kernels cover almost every practical case. The first is the plain dot product itself, $K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$, called the *linear kernel*, which gives back exactly the linear SVM of Chapter 16. The second is the polynomial kernel of the previous lesson. The third, the most used of all, measures similarity by distance.

WORDS TO KNOW

RBF kernel — The radial basis function kernel, also called the Gaussian kernel: a similarity that starts at 1 for two identical points and falls smoothly toward 0 as they move apart.

Gamma (γ) — The RBF kernel's hyperparameter. It sets how quickly similarity falls off with distance: small γ means every point still notices far-away points, large γ means a point notices only its close neighbors.

EQUATION: RBF (GAUSSIAN) KERNEL

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

In words: Measure the squared distance between the two points, multiply it by γ , make it negative, and take e to that power.

When it is used: As the default first choice for a curved boundary when the number of features is small or moderate and the features have been put on comparable scales.

Symbol	Meaning	Unit or type
$\ \mathbf{x} - \mathbf{x}'\ ^2$	squared distance: add the squared differences of every feature	feature units squared
γ	how fast similarity falls with distance; $\gamma > 0$	1 / feature units squared
$\exp(z)$	the number $e \approx 2.718$ raised to the power z	number between 0 and 1
K	similarity: exactly 1 for identical points, never negative	number in $(0, 1]$

This is the same squared distance used for nearest centroids in Chapter 1 and k -nearest neighbors in Chapter 11. No square root is taken, which keeps the arithmetic easy.

WORKED EXAMPLE 17.4 — Compute RBF Kernel Values for Three Distances

A monitoring system stores three past readings: $\mathbf{a} = (2, 3)$, $\mathbf{b} = (4, 2)$ and $\mathbf{c} = (5, 6)$. A new reading arrives: $\mathbf{q} = (2, 2)$. Using $\gamma = 0.5$, compute the RBF kernel between $\mathbf{q}$ and each stored reading.

1. Squared distance to $\mathbf{a}$: $(2 - 2)^2 + (2 - 3)^2 = 0 + 1 = 1$.
2. Kernel value: $K = \exp(-0.5 \times 1) = \exp(-0.5) = 0.6065$.
3. Squared distance to $\mathbf{b}$: $(2 - 4)^2 + (2 - 2)^2 = 4 + 0 = 4$. Kernel value: $\exp(-0.5 \times 4) = \exp(-2) = 0.1353$.
4. Squared distance to $\mathbf{c}$: $(2 - 5)^2 + (2 - 6)^2 = 9 + 16 = 25$. Kernel value: $\exp(-0.5 \times 25) = \exp(-12.5) = 0.0000037$.
5. Repeat the first one with $\gamma = 2$ instead: $\exp(-2 \times 1) = 0.1353$.

Answer: The similarities are 0.6065, 0.1353 and 0.0000037, so the far reading c counts for practically nothing. Raising γ to 2 drops even the nearby reading a to 0.1353: a larger γ makes every point more of a loner.

Figure 17.5 shows the same falling-off behavior across a range of distances and three values of γ .

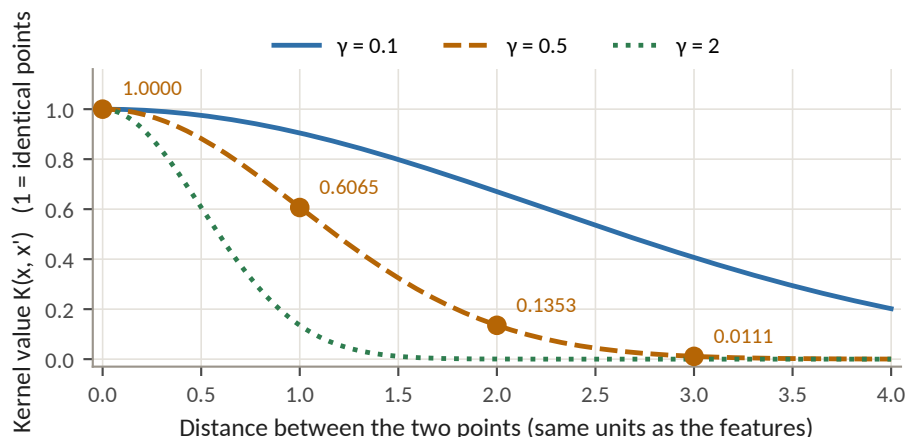


Figure 17.5 – How the RBF kernel value falls with distance, for three values of γ . The labeled dots are the values at distances of 0, 1, 2 and 3 when $\gamma = 0.5$: 1.0000, 0.6065, 0.1353 and 0.0111.

WHAT TO OBSERVE

Every curve starts at exactly 1, because a point is perfectly similar to itself. At $\gamma = 0.1$ the similarity is still 0.4 at a distance of 3, so distant points keep a vote; at $\gamma = 2$ the curve has reached zero by a distance of 1.5, so each training point speaks only about its immediate surroundings.

Table 17.3 summarizes the three kernels you will actually use.

Table 17.3 – The three kernels used in practice

Kernel	Formula	Settings to choose	Reach for it when
Linear	$\mathbf{x} \cdot \mathbf{x}'$	C only	There are many features (text, wide tables), or the dataset is very large. Try it first.
Polynomial	$(\gamma \mathbf{x} \cdot \mathbf{x}' + c)^d$	d, c, γ, C	You expect features to act in pairs or triples, and you know roughly the degree.
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	γ, C	The boundary is smooth and curved, features are few and already on comparable scales. The usual default.

Figure 17.6 trains the same support vector machine three times on the 60 inverters of Figure 17.3, changing only the kernel.

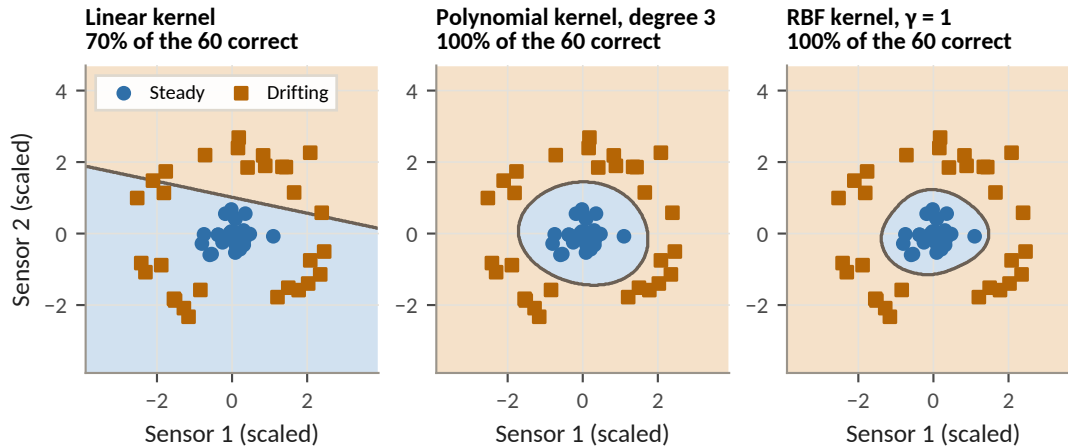


Figure 17.6 – The same SVM with three kernels on the same 60 readings, with $C = 1$. Shaded areas are the regions each model predicts; the gray curve is its boundary.

WHAT TO OBSERVE

The linear kernel gets 42 of the 60 right (70%) with a straight boundary, exactly as in Chapter 16. The polynomial and RBF kernels both reach 100%, because one round island is a shape either can draw — the polynomial as a smooth conic, the RBF following the data more closely.

COMMON MISTAKE

✗ Wrong: Feeding an RBF kernel raw features on wildly different scales — voltage in the thousands next to a ratio between 0 and 1.

✓ Right: Standardize first (Chapter 4). The RBF kernel adds up squared differences, so a large-numbered feature would swallow the distance and γ would be meaningless. This matters as much here as it did for k -nearest neighbors in Chapter 11.

17.5 Kernel SVM in Practice: Choosing C and Gamma Together

A trained kernel SVM keeps only its support vectors — the examples on or inside the margin, from Chapter 16 — each with a weight found during training, plus one intercept. To score a new point it adds up the kernel values between that point and every stored support vector.

WORD TO KNOW

Dual weight (α) — The number training attaches to each support vector, saying how much that example influences predictions. Every other example has $\alpha = 0$ and can be thrown away, and no α may exceed C .

EQUATION: KERNEL SVM DECISION FUNCTION

$$f(\mathbf{x}) = \sum_{i \in S} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

In words: For every support vector, multiply its weight, its label (+1 or -1) and its kernel similarity to the new point; add all of these up, then add the intercept.

When it is used: To predict with a trained kernel SVM: the sign of $f(\mathbf{x})$ gives the class, and its size says how far the point sits from the boundary.

Symbol	Meaning	Unit or type
S	the set of support vectors kept after training	list of examples
α_i	the dual weight of support vector i ; $0 < \alpha_i \leq C$	number
y_i	the label of support vector i , written $+1$ or -1	sign
K	the chosen kernel	similarity score
b	the intercept found during training	number

With the linear kernel this collapses back to $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ from Chapter 16. With any other kernel there is no $\mathbf{w}$ to print — the support vectors themselves are the model.

WORKED EXAMPLE 17.5 — Predict with a Kernel SVM on the Irrigation Data

An RBF kernel SVM with $\gamma = 0.5$ and $C = 1$ is trained on the four irrigation trials. All four become support vectors, each with weight $\alpha_i = 1$, and the intercept is $b = 0$. The labels are $y = +1$ for F1 (1, 1) and F2 (−1, −1), and $y = -1$ for F3 (1, −1) and F4 (−1, 1). Classify a new field $\mathbf{q} = (0.5, 0.5)$, whose water and fertilizer levels are both moderately high.

1. Squared distance from $\mathbf{q}$ to F1 (1, 1): $(0.5 - 1)^2 + (0.5 - 1)^2 = 0.25 + 0.25 = 0.5$. Kernel: $\exp(-0.5 \times 0.5) = 0.7788$.
2. To F2 (−1, −1): $(1.5)^2 + (1.5)^2 = 4.5$. Kernel: $\exp(-2.25) = 0.1054$.
3. To F3 (1, −1): $(0.5)^2 + (1.5)^2 = 2.5$. Kernel: $\exp(-1.25) = 0.2865$.
4. To F4 (−1, 1): $(1.5)^2 + (0.5)^2 = 2.5$. Kernel: $\exp(-1.25) = 0.2865$.
5. Multiply each kernel value by its $\alpha_i y_i$ (here $+1$ for F1 and F2, -1 for F3 and F4) and add: $0.7788 + 0.1054 - 0.2865 - 0.2865 = 0.3112$.
6. Add the intercept: $0.3112 + 0 = 0.3112$, which is positive.

Answer: $f(\mathbf{q}) = 0.3112 > 0$, so the new field is predicted healthy; the mirror-image field (0.5, −0.5) gives exactly -0.3112 and is predicted damaged. The four points that defeated every straight line in Lesson 17.1 are handled correctly here, with no new feature ever computed.

A kernel SVM has two hyperparameters that must be chosen together, never one at a time. C , from Chapter 16, is the price of one training mistake, so a large C pushes the model to get every training point right. γ controls how far a single training point's influence reaches. Pushed up, both produce the same disease: a boundary that wraps tightly around individual training points and fails on new data — the overfitting of Chapter 7. To see it, Figure 17.7 trains an RBF SVM on 72 of the solar inverters of this chapter's case study, holding out 48 for testing, with $C = 10$ fixed and only γ changing.

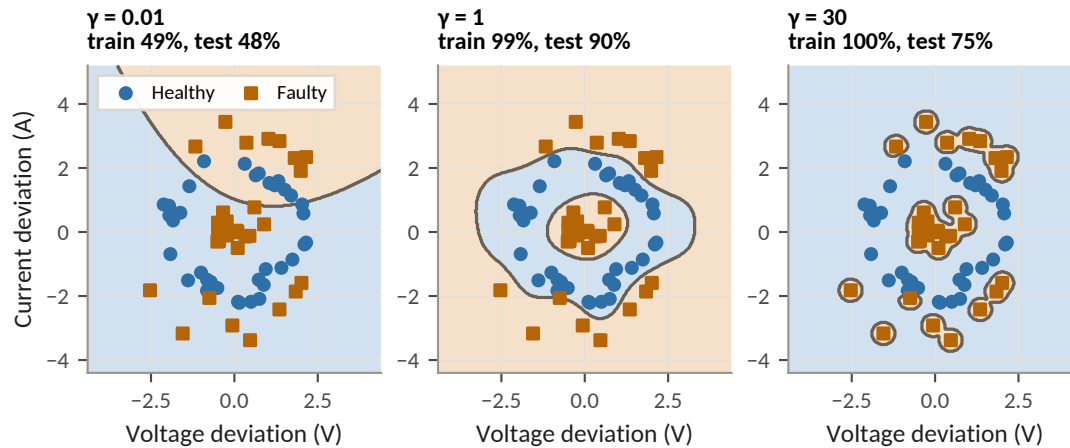


Figure 17.7 – The same data and the same $C = 10$, with three values of γ . Left: far too smooth. Middle: about right. Right: far too wiggly.

WHAT TO OBSERVE

At $\gamma = 0.01$ every point looks similar to every other, so the boundary is nearly straight and the model gets 49% of training and 48% of test points right — no better than guessing. At $\gamma = 1$ it reaches 99% and 90%, keeping only 30 of the 72 training points as support vectors. At $\gamma = 30$ it draws a tiny island around almost every faulty point: a perfect 100% on training data, 75% on test data, and now 68 of the 72 points are support vectors. A model that must remember nearly every example has memorized rather than learned.

The practical method is a grid search, which Chapter 23 treats fully: list a few values of each, spaced by multiplication (0.01, 0.1, 1, 10) rather than addition, and score every combination by cross-validation (Chapter 5) on training data only. Table 17.4 shows such a grid for the 120 inverters; Figure 17.8 draws the same numbers as a heat map.

Table 17.4 – Five-fold cross-validation accuracy for 16 combinations of C and γ , on the 120 inverters

	$\gamma = 0.01$	$\gamma = 0.1$	$\gamma = 1$	$\gamma = 10$
$C = 0.1$	0.483	0.567	0.725	0.667
$C = 1$	0.483	0.625	0.967	0.942
$C = 10$	0.483	0.833	0.975	0.950
$C = 100$	0.542	0.967	0.975	0.950

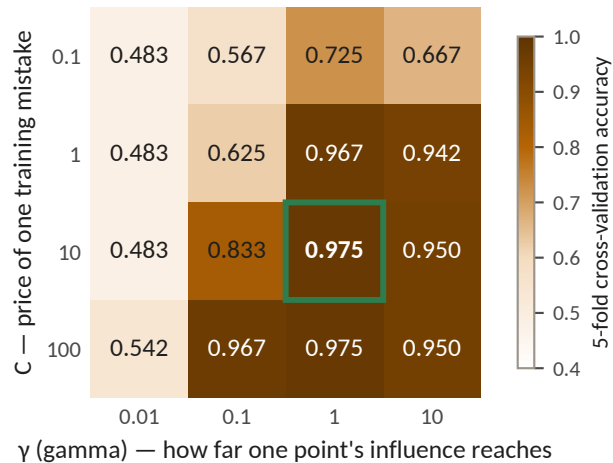


Figure 17.8 – The grid of Table 17.4 drawn as a heat map. Darker is better; the green square marks the winner.

WHAT TO OBSERVE

The whole left-hand column is stuck near 0.5 whatever C does: when γ is far too small, no C can rescue the model, and the $C = 0.1$ row says the same in the other direction. That is why the two dials must be searched together, as a grid.

WORKED EXAMPLE 17.6 — Choose C and Gamma from the Validation Grid

Using Table 17.4, choose the pair (C, γ) to train the final model with, and say how many models the search had to train.

1. Find the largest number in the table. Two cells share it: $C = 10$ with $\gamma = 1$, and $C = 100$ with $\gamma = 1$, both at 0.975.
2. Convert that score into examples: $0.975 \times 120 = 117$ of the 120 inverters classified correctly.
3. Break the tie by preferring the simpler model. A smaller C punishes training mistakes less, which leaves a wider margin, so $C = 10$ is the safer choice.
4. Count the work: the grid has $4 \times 4 = 16$ combinations, and five-fold cross-validation trains five models for each, so $16 \times 5 = 80$ models were trained.
5. Retrain one final model on all the training data with $C = 10$ and $\gamma = 1$, and only then touch the test set.

Answer: Choose $C = 10$ and $\gamma = 1$, worth about 117 correct out of 120 in cross-validation, after training 80 models. When two settings tie, prefer the one that lets the model stay simpler.

Two practical notes. Scikit-learn's default `gamma="scale"` sets γ from the number of features and the spread of the data: a sensible starting point, rarely the best value, so search around it. And never choose C or γ by looking at the test set — the grid is scored by cross-validation inside the training data, and the test set is opened once, at the end.

COMMON MISTAKE

X Wrong: Tuning γ first with C fixed, then tuning C with γ fixed, and reporting the result as the best combination.

✓ Right: Search the two together over a grid. As Table 17.4 shows, a bad γ makes every C look equally poor, so a one-at-a-time search can settle on a corner of the table that is nowhere near the best cell.

17.6 Kernels Beyond SVM: Kernel Ridge Regression

The kernel trick belongs to no single algorithm: any method that touches the data only through dot products can be kernelized. The most useful for everyday work is ridge regression, met in Chapter 7 and Chapter 8, rewritten in kernel form by Craig Saunders, Alexander Gammerman and Volodya Vovk in 1998.

WORD TO KNOW

Kernel ridge regression — Ridge regression with every dot product replaced by a kernel. It predicts a number, not a class, and its prediction is a weighted sum of kernel similarities to the training examples.

Consider a city district whose daily peak electricity demand depends on the outdoor temperature. On cold days people heat, on hot days they cool, and in between demand is low. The relationship is a valley, not a line, so a straight-line model is hopeless — but ridge regression with an RBF kernel handles it easily.

EQUATION: KERNEL RIDGE PREDICTION

$$\hat{y}(\mathbf{x}) = \sum_{i=1}^n \alpha_i K(\mathbf{x}_i, \mathbf{x}), \quad \boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$$

In words: The prediction is a weighted sum of the kernel similarities between the new point and every training point. The weights come from solving one system of equations built from the kernel matrix and the penalty λ .

When it is used: To predict a number when the pattern is clearly curved, the dataset is small or moderate (up to a few thousand rows), and you want a smooth fit without designing features by hand.

Symbol	Meaning	Unit or type
$\hat{y}$	the predicted number for a new point	unit of the target
α_i	weight of training example i ; found once, during training	number
$\mathbf{K}$	the $n \times n$ kernel matrix of the training data	similarity scores
λ	regularization strength, as in Chapter 7; larger means smoother	number ≥ 0
$\mathbf{I}$	the identity matrix, with 1 on the diagonal and 0 elsewhere	matrix

Unlike an SVM, kernel ridge regression keeps every training point, so prediction cost grows with the whole training set. Software does the second formula; the first is the one you compute by hand.

Before doing the arithmetic by hand, look at what such a model produces. Figure 17.9 fits one to 45 days of records and compares it with ordinary linear regression.

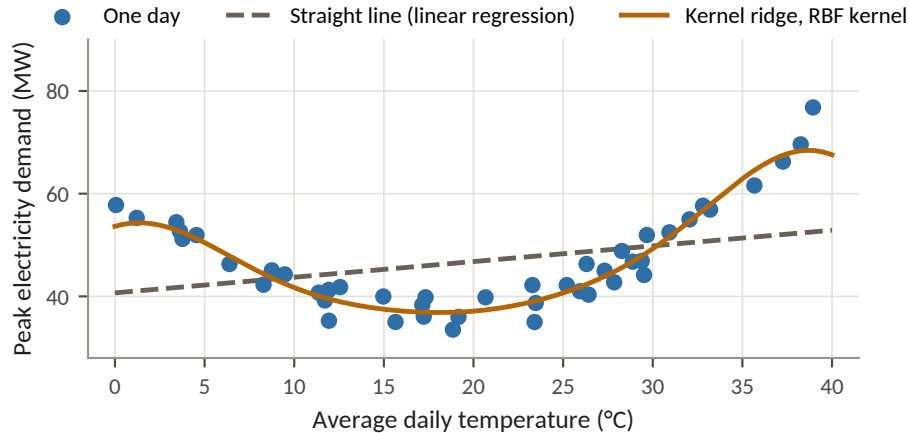


Figure 17.9 – Peak electricity demand against average daily temperature for 45 days. The straight line is ordinary linear regression; the curve is kernel ridge regression with an RBF kernel.

WHAT TO OBSERVE

The straight line rises gently and is wrong nearly everywhere, sitting far below both the cold days and the hot ones. The kernel ridge curve follows the valley, bottoming out near 17 °C. Neither model was told to expect a valley — the kernel simply let the data bend the fit.

WORKED EXAMPLE 17.7 — Predict Electricity Demand with Kernel Ridge Regression

A kernel ridge model with an RBF kernel and $\gamma = 0.01$ was trained on three representative days: 5 °C with a peak demand of 60 MW, 20 °C with 40 MW, and 35 °C with 75 MW. Training produced the weights $\alpha = 52.1, 25.1$ and 65.8 for those three days. Predict the peak demand on a day whose average temperature is 25 °C.

1. Kernel to the 5 °C day: squared distance $(25 - 5)^2 = 400$, so $K = \exp(-0.01 \times 400) = \exp(-4) = 0.0183$.
2. Kernel to the 20 °C day: $(25 - 20)^2 = 25$, so $K = \exp(-0.25) = 0.7788$.
3. Kernel to the 35 °C day: $(25 - 35)^2 = 100$, so $K = \exp(-1) = 0.3679$.
4. Multiply each weight by its kernel value: $52.1 \times 0.0183 = 0.9534$; $25.1 \times 0.7788 = 19.5479$; $65.8 \times 0.3679 = 24.2078$.
5. Add the three terms: $0.9534 + 19.5479 + 24.2078 = 44.709$.

Answer: The predicted peak demand is about 44.7 MW (scikit-learn, carrying every decimal, returns 44.67 MW). The nearby 20 °C day and the warm 35 °C day dominate; the cold 5 °C day, far away in temperature, adds less than 1 MW.

Two close relatives are worth knowing by name. *Support vector regression* with an RBF kernel, introduced by Harris Drucker and colleagues in 1996, fits the same kind of curve but keeps only the support vectors, so it predicts faster; it uses the ε -tube of Chapter 16, and in scikit-learn it is `SVR(kernel="rbf")`. *Kernel principal component analysis*, published by Bernhard Schölkopf, Alexander Smola and Klaus-Robert Müller in 1998, applies the same trick to the dimensionality reduction of Chapter 20. And a result proved by George Kimeldorf and Grace Wahba in 1971 says that for this whole family the best possible solution really can be written as a weighted sum of kernel values to the training points: the formula in the box is not an approximation but the exact shape of the answer.

17.7 Cost, Scaling and When to Prefer a Kernel Method

Kernel methods have one serious weakness, and it can be calculated in advance: training needs the kernel between every pair of training examples, so the kernel matrix has n^2 entries.

WORKED EXAMPLE 17.8 — How Much Memory Does the Kernel Matrix Need?

A team wants to train a kernel SVM on 50,000 rows. Each kernel value is stored as an 8-byte number. How much memory does the kernel matrix alone require?

1. Count the entries: $50,000 \times 50,000 = 2,500,000,000$ — two and a half billion.
2. Multiply by 8 bytes each: $2,500,000,000 \times 8 = 20,000,000,000$ bytes.
3. Convert to gigabytes by dividing by one billion: $20,000,000,000 \div 1,000,000,000 = 20$.

Answer: About 20 gigabytes for the kernel matrix alone, before anything else training needs — more than most laptops have. The 120 inverters of this chapter, by comparison, need a matrix of only $120 \times 120 = 14,400$ entries.

Table 17.5 extends that calculation: ten times the data needs a hundred times the memory.

Table 17.5 – Size of the kernel matrix as the training set grows (8 bytes per entry). At 200,000 rows it would need 320 GB.

Training rows n	Entries in K	Memory	What it means in practice
1,000	1 million	8 MB	Instant on any computer
10,000	100 million	800 MB	Still comfortable; seconds to minutes
50,000	2.5 billion	20 GB	Beyond an ordinary laptop

Prediction has its own, different cost: a kernel SVM computes one kernel value per stored support vector for every new example, so a model with 5,000 support vectors is far slower to query than a linear model, which needs one dot product. The support vector count in Figure 17.7 is therefore not only a warning about overfitting; it is the price of every future prediction. Three practical rules follow.

- **Always scale first.** Every kernel here uses dot products or distances, so a feature measured in thousands drowns one measured in tenths. Standardize inside a pipeline, using training statistics only (Chapter 4).
- **Try the linear kernel first.** With many features — text, wide tables — data is often already separable by a straight boundary, and a linear model trains in seconds on millions of rows.
- **Reach for RBF when the data is small and curved.** A few thousand rows, a handful of well-scaled features and a visibly curved boundary: here kernel methods are hard to beat.

Above roughly a hundred thousand rows, a gradient-boosted tree ensemble (Chapter 15) usually matches or beats a kernel SVM at a fraction of the cost and needs no scaling at all. Kernel methods also give no list of coefficients to read: the model is its support vectors, which makes a single decision harder to explain than with a tree (Chapter 13) or a linear model.

IMPORTANT

A kernel model cannot be read as a short list of reasons, so be careful using one where a person is owed an explanation — a loan refusal, a medical referral, a job application. Keep a simpler model for the explanation, or use the methods of Chapter 24. A model whose decisions nobody can explain should support a human decision, never replace it.

COMMON MISTAKES AND MISCONCEPTIONS

- Believing that data no straight line can separate is beyond the reach of linear models, instead of changing the space the line is drawn in.
- Thinking a kernel method physically moves the data into a huge space; it only ever computes one number per pair of examples.
- Treating a kernel value as a distance. It is a similarity: largest, exactly 1 for the RBF kernel, when two points are identical, and falling toward 0 as they separate.
- Tuning C and γ one after the other rather than together on a grid, and so missing the good corner of the table entirely.
- Running an RBF kernel on unscaled features, so that one large-numbered column decides every distance — or applying the scaler and the feature map to the training rows only.
- Reading a rising training accuracy as progress when γ grows: a large γ can reach 100% on training data while collapsing on new data, and it leaves almost every example a support vector.
- Choosing a kernel SVM for a very large dataset without checking the n^2 size of the kernel matrix first.

Practical Activities

ACTIVITY (PAPER) — The Long Way and the Short Way

Work with the two points $\mathbf{u} = (3, 1)$ and $\mathbf{v} = (2, 4)$.

1. Write down $\varphi(\mathbf{u})$ and $\varphi(\mathbf{v})$ using the degree-2 map $\varphi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$. Leave the $\sqrt{2}$ terms as they are.
2. Compute $\varphi(\mathbf{u}) \cdot \varphi(\mathbf{v})$ term by term. Count how many multiplications you performed.
3. Now compute $\mathbf{u} \cdot \mathbf{v}$, then square it. Count the multiplications again, and compare your two answers.
4. Repeat step 3 with the constant $c = 1$: compute $(\mathbf{u} \cdot \mathbf{v} + 1)^2$, then check it the long way with the six-term map $(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2)$.
5. Compute the RBF kernel of the same pair with $\gamma = 0.1$: find the squared distance, multiply by -0.1 , and take e to that power. Then explain to a classmate, in one sentence, why the short way is not an approximation.

You need: Paper, pen and a calculator with an e^x key

ACTIVITY (PYTHON) — RBF Kernel SVM and a Grid Search on the Inverter Data

This program loads the 120 solar inverters of the case study, compares kernels, searches the C - γ grid of Table 17.4, and finally checks the decision function of Worked Example 17.5 by hand against scikit-learn.

Python • pandas • scikit-learn

```
import numpy as np, pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, GridSearchCV

d = pd.read_csv("datasets/ch17_inverters.csv")
X = d[["v_dev", "i_dev"]].to_numpy()
y = d["label"].to_numpy()          # 0 = healthy, 1 = faulty

for name, m in [("linear", SVC(kernel="linear", C=1)),
                ("poly d=3", SVC(kernel="poly", degree=3, coef0=1, C=1)),
                ("rbf g=1", SVC(kernel="rbf", gamma=1, C=1))]:
    print(name, round(cross_val_score(m, X, y, cv=5).mean(), 3))

grid = {"C": [0.1, 1, 10, 100], "gamma": [0.01, 0.1, 1, 10]}
search = GridSearchCV(SVC(kernel="rbf"), grid, cv=5).fit(X, y)
print("best:", search.best_params_, round(search.best_score_, 3))
print("support vectors:", len(search.best_estimator_.support_), "of", len(X))

# Worked Example 17.5, by hand and by scikit-learn
Xf = np.array([[1, 1], [-1, -1], [1, -1], [-1, 1]]); yf = np.array([1, 1, -1, -1])
f = SVC(kernel="rbf", gamma=0.5, C=1).fit(Xf, yf)
q = np.array([0.5, 0.5])
k = np.exp(-0.5 * ((f.support_vectors_ - q) ** 2).sum(axis=1))
print("by hand:", round(float((f.dual_coef_[0] * k).sum() + f.intercept_[0]), 4),
      " sklearn:", round(float(f.decision_function([q])[0]), 4))
```

Output

```
linear      0.45
poly d=3    0.567
rbf g=1     0.967
best: {'C': 10, 'gamma': 1} 0.975
support vectors: 40 of 120
by hand: 0.3112  sklearn: 0.3112
```

The first block ranks the kernels on harder data: linear is worse than guessing (0.45), a degree-3 polynomial reaches 0.567, and RBF reaches 0.967 before any tuning. GridSearchCV then trains all 80 models of Worked Example 17.6 and finds the same winner, $C = 10$ and $\gamma = 1$, at 0.975. The last lines reproduce Worked Example 17.5 exactly: 0.3112, computed from stored support vectors and kernel values, matches decision_function to four decimals.

You need: A computer with Python, pandas and scikit-learn, and the file datasets/ch17_inverters.csv (Appendix C)

The case study below puts the whole chapter to work on the 120 solar inverters already met in Lesson 17.5; Figure 17.10 shows the boundary its finished model learned.

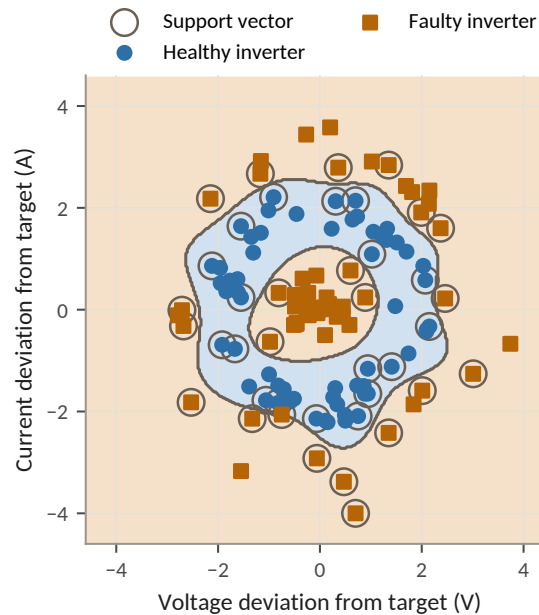


Figure 17.10 – The RBF kernel SVM learned for the 120 inverters, with $C = 10$ and $\gamma = 1$. Circled points are the 40 support vectors — the only examples the finished model keeps.

WHAT TO OBSERVE

The shaded healthy band is a closed ring with faulty regions both inside and outside it — a shape no straight line, and no single circle, could produce. Almost every support vector sits near one of the band's two edges; the comfortable points, deep inside or far outside, have weight zero and are discarded.

CASE STUDY — Finding Faulty Solar Inverters from Two Sensor Readings — Kurdistan Region

A solar installer maintains 120 rooftop systems across Erbil and Duhok. A healthy inverter constantly hunts for the panels' best operating point, stepping slightly above and below it many times a minute, so its logged readings never sit still. The maintenance team records two numbers per unit per day, on comparable scales: how far the voltage sat from its target, in volts, and how far the current sat from its target, in amps. Two very different faults appear in the field. A unit whose tracking has frozen stops hunting and its readings collapse toward zero; a unit whose electronics are failing swings much too far. So the healthy readings form a *ring*, with frozen units inside it and failing units outside it. (The dataset here is illustrative, built to match the shapes the team described.)

The team's first attempt, a linear SVM, scored 0.450 in five-fold cross-validation — worse than tossing a coin, and for a clear reason: no straight line can fence off a ring with trouble on both sides of it. Their second attempt added squares and products by hand, which is the same as a polynomial kernel, and reached only 0.567 at degree 3. The reason is worth remembering: a low-degree boundary can draw *one* circle, so it can say “inside is good, outside is bad”, but not “the middle band is good and both sides are bad”, which needs two circles at once. Degree 4 finally allowed two circles and lifted the score to 0.933. The RBF kernel needed no such reasoning: because it judges each reading by its similarity to nearby training readings, it can draw a band of any shape. A grid search (Table 17.4) settled on $C = 10$ and $\gamma = 1$ at 0.975 — about 117 of the 120 units judged correctly — keeping 40 inverters as support vectors. The team uses the model as a daily sorting tool, not a verdict. Every morning it ranks the 120 units and the three or four flagged ones join a technician's visit list. A flagged inverter is never switched off automatically, and an unflagged one is still serviced on the normal schedule, because the model has seen only two of the many things that can go wrong on a roof.

CHAPTER SUMMARY

- Data is linearly separable when one straight boundary can split the classes; the four irrigation trials prove some data is not, whatever weights are chosen.
- Adding features built from the originals — squares such as x^2 , products such as x_1x_2 — can make a curved problem straight without changing the model.
- A feature map φ carries every example, training and new, into a larger feature space, and a kernel $K(\mathbf{x}, \mathbf{x}') = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$ returns the dot product of two mapped points without building the map — the kernel trick.
- The polynomial kernel $(\mathbf{x} \cdot \mathbf{x}' + c)^d$ captures features acting in pairs or triples; the RBF kernel $\exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ measures similarity that falls smoothly with distance and is the usual default.
- γ sets how far one training point's influence reaches: too small gives a nearly straight, useless boundary; too large gives tiny islands around single points. It must be searched together with C on a grid, scored by cross-validation on training data, because a bad γ makes every C look equally poor.
- A trained kernel SVM predicts with $f(\mathbf{x}) = \sum \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$, using only its support vectors; the sign of f is the class.
- Kernel ridge regression and support vector regression apply the same trick to predicting numbers, fitting smooth curves such as the temperature–demand valley.
- The kernel matrix has n^2 entries, so kernel methods become impractical beyond a few tens of thousands of rows. Scale every feature first, and try the linear kernel before anything fancier.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain in your own words what “linearly separable” means, and give an example of data that is not.

Problem 2 [Basic] A single feature takes the values $x = -4, -2, 0, 2, 4$; the examples at -4 and 4 are “fail” and the rest “pass”. Compute x^2 for each, and give a threshold on x^2 that separates the classes.

Problem 3 [Basic] For $\mathbf{x} = (1, 5)$ and $\mathbf{x}' = (2, 2)$, compute $\mathbf{x} \cdot \mathbf{x}'$ and then the degree-2 polynomial kernel $(\mathbf{x} \cdot \mathbf{x}')^2$.

Problem 4 [Basic] Which kernel has the setting γ , and what happens to the kernel value between two fixed points when γ is made larger?

Problem 5 [Standard] For $\mathbf{u} = (2, 1)$ and $\mathbf{v} = (3, 4)$, compute $(\mathbf{u} \cdot \mathbf{v} + 1)^2$, then compute it the long way with the six-term map $(1, \sqrt{2}u_1, \sqrt{2}u_2, u_1^2, \sqrt{2}u_1u_2, u_2^2)$ and show the answers agree.

Problem 6 [Standard] Two points are 2 units apart. Compute the RBF kernel value between them for $\gamma = 0.2$, $\gamma = 1$ and $\gamma = 5$, and say in one sentence what the three numbers mean.

Problem 7 [Standard] An RBF kernel SVM with $\gamma = 0.5$ has three support vectors: $\mathbf{x}_1 = (1, 0)$, $y_1 = +1$, $\alpha_1 = 0.8$; $\mathbf{x}_2 = (0, 1)$, $y_2 = +1$, $\alpha_2 = 0.4$; $\mathbf{x}_3 = (2, 2)$, $y_3 = -1$, $\alpha_3 = 1.0$; and $b = 0.1$. Classify $\mathbf{q} = (1, 1)$, showing each kernel value.

Problem 8 [Standard] Using Table 17.4: (a) what is the best score in the $\gamma = 0.1$ column, and at which C ? (b) Why does the whole $\gamma = 0.01$ column stay near 0.5? (c) If two cells tie, which should you choose, and why?

Problem 9 [Standard] A team plans to train a kernel SVM on 30,000 rows at 8 bytes per kernel value. Compute the entries in the kernel matrix and the memory needed in gigabytes, and say whether a laptop with 16 GB can do it.

Problem 10 [Challenge] Repeat the proof of Worked Example 17.1 for healthy examples at $(2, 2)$ and $(-2, -2)$ and damaged ones at $(2, -2)$ and $(-2, 2)$. Show that the same contradiction appears, and explain why the scale of the numbers made no difference.

Problem 11 [Challenge] Show algebraically that $(\mathbf{x} \cdot \mathbf{x}')^2 = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$ for any two-feature points, using this chapter's degree-2 map. Start by expanding $(x_1x'_1 + x_2x'_2)^2$.

Problem 12 [Challenge] An RBF kernel SVM reaches 100% training accuracy but only 72% test accuracy, keeping 95% of the training examples as support vectors. (a) Name the problem. (b) Which way would you move γ , and C ? (c) What did the support vector count alone already tell you?

PART FOUR

UNSUPERVISED LEARNING: FINDING STRUCTURE IN DATA

k-Means Clustering

Finding groups in data that has no labels

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how clustering differs from classification and regression, and recognize a project that needs it.
- Run k-means by hand: assign every point to its nearest centroid, move each centroid, and repeat until nothing changes.
- Calculate the inertia and the silhouette score of a finished clustering, and use the elbow method to choose k .
- Explain why the starting centroids matter, and what repeated random starts and k-means++ do about it.
- Recognize the cluster shapes and the feature scales that make k-means fail, and name a softer alternative.
- Build a cluster profile, name the groups responsibly, and flag a point that sits far from every centroid.

Introduction

A small grocery store in Sulaymaniyah has been running a loyalty card for a year. Every time a card is scanned, the till quietly records two numbers about that customer: how many times they visited last month, and how much they spent, on average, on each visit. Nobody has ever asked a customer to describe their own shopping habits, and there is no column in the file called “customer type”.

Still, the owner is fairly sure her customers are not all the same. Some people drop in almost every day for bread and yogurt; others appear once a fortnight and fill a whole trolley. If she could see those natural groups, she could plan smarter offers: a loyalty bonus for the frequent small shoppers, a bigger one-time discount to tempt the rare large shoppers back sooner. This is the question clustering answers. Nothing in the data says who belongs where, so there is no correct answer to learn from and no accuracy to report — only a table of numbers and one honest question: which rows look enough alike to belong together? This chapter builds the most widely used answer, k-means, by hand on six customers, then checks every number with scikit-learn. Everything you need is already in Chapter 1: the centroid, and the squared distance between two points.

18.1 Grouping Without Answers

Chapter 1 named three common learning tasks. Classification predicts a category and regression predicts a number; both are *supervised* learning, because every training row already carries the right answer. Clustering is the odd one out: it is *unsupervised* learning, so the rows arrive with no answer at all and the method must find the structure from the features alone.

WORD TO KNOW

Cluster — A group of examples a clustering method has decided are similar to one another. The method invents the groups and numbers them, but never gives them a meaning; only a person, looking at a finished cluster, can decide what it represents.

To keep every number checkable with a pencil, we start with six loyalty customers and two features: visits last month, and average spend per visit in thousand IQD. Table 18.1 lists them, and Figure 18.1 plots them exactly as the computer receives them — with no colors and no labels, because none exist.

Table 18.1 – Six loyalty customers — the running example for this whole chapter

Customer	Visits last month (count)	Average spend per visit (thousand IQD)	Written as a point
A	2	18	(2, 18)
B	3	20	(3, 20)
C	4	22	(4, 22)
D	8	48	(8, 48)
E	9	50	(9, 50)
F	10	52	(10, 52)

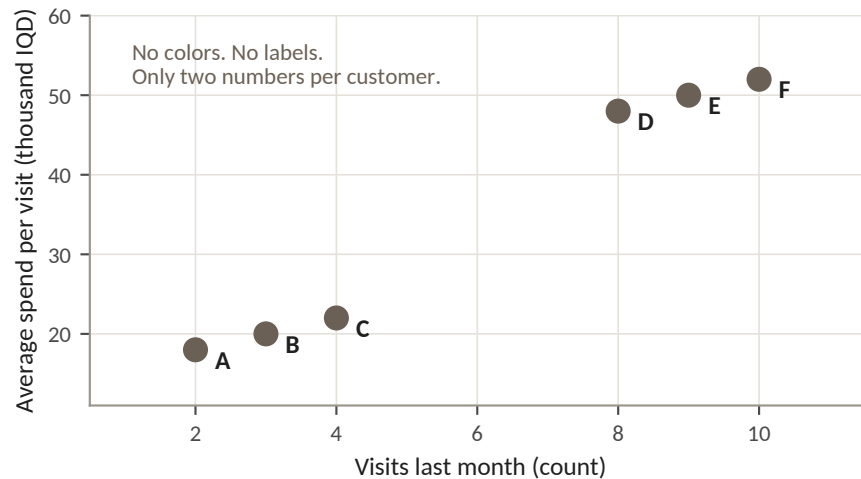


Figure 18.1 – The six loyalty customers as the computer sees them. Every point is the same color, because no group labels exist.

WHAT TO OBSERVE

Your eye splits these six points into two bunches straight away: A, B and C at the bottom left, D, E and F at the top right. A machine must reach that answer on its own, and keep reaching it with 40,000 customers and twenty features.

First, though, check that clustering is really the right family of method for the job.

WORKED EXAMPLE 18.1 — Decide Whether Four Store Projects Need Clustering

The store's data team has four projects. Project 1: group customers into similar shopping patterns, with no existing group names. Project 2: predict whether a customer will redeem a coupon, “yes” or “no”, using past redemption records. Project 3: predict how much a customer will spend next month, in thousand IQD, using past monthly totals. Project 4: group suppliers by how similar their delivery schedules are, with no existing supplier categories. Decide which projects need clustering.

1. Project 1 has no labels at all, only a request to find similar customers, so it needs **clustering**.
2. Project 2 already has a known, correct answer for every past customer, “yes” or “no”, so it is supervised learning — specifically **classification**.
3. Project 3 also has a known, correct answer for every past customer, an amount of money, so it is supervised learning — specifically **regression**.
4. Project 4 has no existing supplier categories, only a request to find similar suppliers, so it needs **clustering**, exactly like Project 1.

Answer: Projects 1 and 4 need clustering; 2 and 3 need supervised methods. What decides the family is whether a known label already exists for every row — not whether the word “group” appears in the request.

COMMON MISTAKE

X Wrong: Reaching for clustering whenever a project mentions “grouping” customers, products or records, without first checking whether a known label already exists.

✓ Right: Look for a label first. If every row carries a correct answer, a supervised method gives a sharper result that you can actually score on a test set (Chapter 6). Save clustering for data where no such answer exists.

Clustering leans on one idea Chapter 1 already built by hand: the *centroid*, the average position of a group of points. Nearest-centroid classification computed one centroid per known class. k-means turns that around: it starts from guessed centroids and uses them to discover the groups.

18.2 The k-Means Algorithm, Step by Step

WORD TO KNOW

k-means — A clustering method that sorts examples into k groups by repeating two steps: assign every example to its nearest centroid, then move each centroid to the average position of the examples now assigned to it. The number k is chosen by a person before the method runs.

The name says almost everything. The letter k is how many groups you are asking for. “Means” are the averages — the centroids — that the method keeps recomputing. The idea goes back to Hugo Steinhaus in 1956; the repeating recipe used today was written down by Stuart Lloyd at Bell Labs in 1957, and James MacQueen gave the method its name in 1967.

In plain words the whole method is four instructions. **Step 0:** choose k and place k starting centroids, often by picking k actual data points at random. **Step 1, assign:** send every example to whichever centroid it sits closest to. **Step 2, update:** move each centroid to the average position of the examples now assigned to it. **Then check:** did any example switch groups? If yes, go back to step 1. If no, stop. Only steps 1 and 2 repeat, and because a round that changes nothing would keep changing nothing forever, k-means always halts.

WORDS TO KNOW

Assignment step — The half of a k-means round that sends every example to its nearest centroid, leaving the centroids where they are.

Update step — The other half of the round, which moves each centroid to the mean of the examples just assigned to it, leaving the assignments alone.

That stopping test is why k-means repeats at all. The first centroids are usually a rough guess. Moving them to match their assigned points shifts them somewhere better, which can change which centroid is nearest for a borderline example, which shifts them again. When a full round passes with nothing changing, the method has *converged*, in the sense Chapter 9 gave that word. To decide which centroid is nearest it uses the same squared distance Chapter 1 introduced and Chapter 11 used for k-Nearest Neighbors, with the same shortcut: because only the *order* of the distances matters, the square root can be skipped.

EQUATION: SQUARED DISTANCE FROM A POINT TO A CENTROID

$$d^2(\mathbf{x}, \mathbf{c}) = \sum_{j=1}^p (x_j - c_j)^2$$

In words: Subtract the point and the centroid one feature at a time, square each difference, and add all the squares together. The smaller total marks the nearer centroid.

When it is used: In every assignment step, to decide which centroid each example belongs to.

Symbol	Meaning	Unit or type
d^2	squared distance from the point to the centroid	squared feature units
$\mathbf{x}$	one example, with features $x_1, \dots, x_p$	row of numbers
$\mathbf{c}$	one centroid, with the same p features	row of numbers
p	number of features	count

With two features this is the formula from Chapter 1: $d^2 = (x_1 - c_1)^2 + (x_2 - c_2)^2$. No square root is ever needed inside k-means.

The update step needs one more formula, and you have met it too: a centroid is the average of its points, computed one feature at a time.

EQUATION: CENTROID UPDATE (THE MEAN OF A CLUSTER)

$$\mathbf{c}_m = \frac{1}{n_m} \sum_{i \in S_m} \mathbf{x}_i$$

In words: Add up the examples now sitting in cluster m , feature by feature, then divide each total by how many examples there are.

When it is used: In every update step, once each example has been assigned to a cluster.

Symbol	Meaning	Unit or type
$\mathbf{c}_m$	new centroid of cluster m	same units as the features
S_m	the examples assigned to cluster m	list of rows
n_m	how many examples are in cluster m	count
$\mathbf{x}_i$	one example in that cluster	row of numbers

Never mix features: the centroid's visits value comes only from visits, its spending value only from spending.

Now the run itself. The owner asks for $k = 2$ and picks two of her own customers at random as the first centroids: B, at $(3, 20)$, and C, at $(4, 22)$. That is an ordinary guess — and, as you are about to see, not a good one. Watching k-means repair it is the point of the next two pages.

WORKED EXAMPLE 18.2 — k-Means Iteration 1: Assign the Six Customers, Then Move Both Centroids

The customers are $A = (2, 18)$, $B = (3, 20)$, $C = (4, 22)$, $D = (8, 48)$, $E = (9, 50)$, $F = (10, 52)$. The starting centroids are $\mathbf{c}_1 = (3, 20)$ and $\mathbf{c}_2 = (4, 22)$. Assign every customer to its nearer centroid, then recompute both centroids.

1. Squared distance to $\mathbf{c}_1 = (3, 20)$. A: $(2 - 3)^2 + (18 - 20)^2 = 1 + 4 = 5$. B: $0 + 0 = 0$. C: $1 + 4 = 5$. D: $25 + 784 = 809$. E: $36 + 900 = 936$. F: $49 + 1024 = 1073$.
2. Squared distance to $\mathbf{c}_2 = (4, 22)$. A: $4 + 16 = 20$. B: $1 + 4 = 5$. C: $0 + 0 = 0$. D: $16 + 676 = 692$. E: $25 + 784 = 809$. F: $36 + 900 = 936$.
3. Keep the smaller of each pair. A: $5 < 20$, group 1. B: $0 < 5$, group 1. C: $0 < 5$, group 2. D: $692 < 809$, group 2. E: $809 < 936$, group 2. F: $936 < 1073$, group 2.
4. New $\mathbf{c}_1$: visits $(2 + 3) \div 2 = 2.5$; spending $(18 + 20) \div 2 = 19$. So $\mathbf{c}_1 = (2.5, 19)$.
5. New $\mathbf{c}_2$: visits $(4 + 8 + 9 + 10) \div 4 = 31 \div 4 = 7.75$; spending $(22 + 48 + 50 + 52) \div 4 = 172 \div 4 = 43$. So $\mathbf{c}_2 = (7.75, 43)$.

Answer: After one round, group 1 is $\{A, B\}$ with centroid $(2.5, 19)$ and group 2 is $\{C, D, E, F\}$ with centroid $(7.75, 43)$. Customer C landed beside three much heavier shoppers only because a starting centroid sat right on top of her.

On paper, organize steps 1 to 3 as one table: a row per customer, a column for d^2 to each centroid, and a last column for the winner. Customer C has ended up beside D, E and F, who visit twice as often and spend more than twice as much — not what your eye said in Figure 18.1. This is why one round is never enough: the centroids have moved, so every customer must be checked again.

WORKED EXAMPLE 18.3 — k-Means Iterations 2 and 3: Reassign, Then Confirm

Iteration 2 starts from $\mathbf{c}_1 = (2.5, 19)$ and $\mathbf{c}_2 = (7.75, 43)$. Reassign all six customers, move both centroids, then run iteration 3 to see whether anyone still wants to switch.

1. Squared distance to $\mathbf{c}_1 = (2.5, 19)$. A: $0.25 + 1 = 1.25$. B: $0.25 + 1 = 1.25$. C: $2.25 + 9 = 11.25$. D: $30.25 + 841 = 871.25$. E: $42.25 + 961 = 1003.25$. F: $56.25 + 1089 = 1145.25$.
2. Squared distance to $\mathbf{c}_2 = (7.75, 43)$. A: $33.0625 + 625 = 658.0625$. B: $22.5625 + 529 = 551.5625$. C: $14.0625 + 441 = 455.0625$. D: $0.0625 + 25 = 25.0625$. E: $1.5625 + 49 = 50.5625$. F: $5.0625 + 81 = 86.0625$.
3. Keep the smaller of each pair. A, B and C now all belong to group 1, because 1.25, 1.25 and 11.25 are far below 658.0625, 551.5625 and 455.0625. D, E and F belong to group 2, so customer C has switched.
4. New $\mathbf{c}_1$: visits $(2 + 3 + 4) \div 3 = 3$; spending $(18 + 20 + 22) \div 3 = 20$, giving $(3, 20)$. New $\mathbf{c}_2$: visits $(8 + 9 + 10) \div 3 = 9$; spending $(48 + 50 + 52) \div 3 = 50$, giving $(9, 50)$.
5. Iteration 3, squared distance to $\mathbf{c}_1 = (3, 20)$: these are the numbers from iteration 1, because the centroid is back at $(3, 20)$. A: 5. B: 0. C: 5. D: 809. E: 936. F: 1073.
6. Iteration 3, squared distance to $\mathbf{c}_2 = (9, 50)$. A: $49 + 1024 = 1073$. B: $36 + 900 = 936$. C: $25 + 784 = 809$. D: $1 + 4 = 5$. E: 0. F: $1 + 4 = 5$.
7. Keep the smaller of each pair again: A, B and C to group 1, D, E and F to group 2 — exactly iteration 2's grouping, so nobody switched.

Answer: k-means converged after three rounds. Group 1 = {A, B, C}, the light, occasional shoppers, centroid (3, 20); group 2 = {D, E, F}, the frequent, bigger spenders, centroid (9, 50). Nobody told the method that C was misplaced; moving the centroids was enough.

Table 18.2 summarizes the whole run, and Figure 18.2 shows the three rounds side by side.

Table 18.2 – The whole run in one table. Round 3 changes nothing, which is the signal to stop

Round	c_1 at start	c_2 at start	Group 1 after	Group 2 after	Switched?
1	(3, 20)	(4, 22)	A, B	C, D, E, F	yes
2	(2.5, 19)	(7.75, 43)	A, B, C	D, E, F	yes (C)
3	(3, 20)	(9, 50)	A, B, C	D, E, F	no — stop

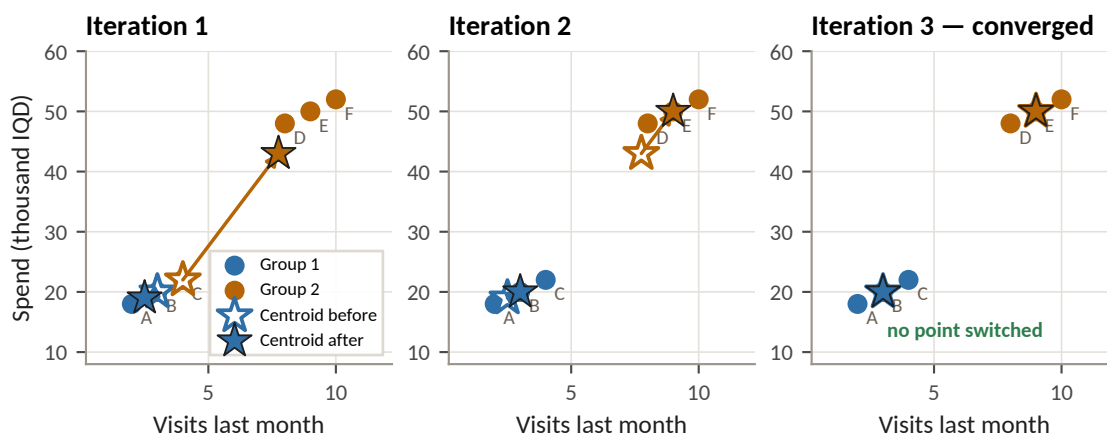


Figure 18.2 – Three rounds of k-means on the six customers. Hollow stars are where each centroid started the round, filled stars where it ended, and the arrow shows the move.

WHAT TO OBSERVE

In the left panel the two hollow stars sit almost on top of each other, on customers B and C — the poor start — and the orange arrow is long because that centroid had to travel to the middle of {C, D, E, F}. By the middle panel customer C has changed color. In the right panel the arrows are gone: nothing moved.

Never treat one assignment step plus one update step as the finished clustering: repeat until a whole round passes with no example changing group. Here, stopping after round 1 would have filed customer C with the big spenders.

18.3 Inertia and Choosing k: the Elbow Method

Every worked example so far assumed $k = 2$, but nobody told the owner that two was right. To compare that choice with three or five we need a single number scoring how tightly a finished clustering hugs its own data.

WORD TO KNOW

Inertia — The total squared distance from every example to the centroid of its own cluster, added up across the dataset. Also called the *within-cluster sum of squares*. A lower inertia means examples sit closer, on average, to their own centroid.

EQUATION: INERTIA OF A CLUSTERING

$$J = \sum_{m=1}^k \sum_{i \in S_m} d^2(\mathbf{x}_i, \mathbf{c}_m)$$

In words: For every example, find the squared distance to the centroid it was assigned to. Add all of those squared distances into one grand total.

When it is used: To compare two clusterings of the same data — one k from two different starting points, or several values of k .

Symbol	Meaning	Unit or type
J	inertia of the whole clustering	squared feature units
k	number of clusters	count
S_m	the examples assigned to cluster m	list of rows
$\mathbf{c}_m$	centroid of cluster m	row of numbers

Inertia is what each round drives downward: the assignment step and the update step can each only lower J or leave it unchanged.

WORKED EXAMPLE 18.4 — Find the Inertia for $k = 2$ and for $k = 1$

The finished $k = 2$ clustering has group 1 = {A, B, C} with centroid (3, 20) and group 2 = {D, E, F} with centroid (9, 50). Find its inertia, then find the inertia of a single cluster holding all six customers.

1. Group 1's squared distances to (3, 20) were already found in iteration 3: A = 5, B = 0, C = 5. Add them: $5 + 0 + 5 = 10$.
2. Group 2's squared distances to (9, 50), also from iteration 3: D = 5, E = 0, F = 5. Add them: $5 + 0 + 5 = 10$. The $k = 2$ inertia is $10 + 10 = 20$.
3. Now $k = 1$. The single centroid's visits value is $(2 + 3 + 4 + 8 + 9 + 10) \div 6 = 36 \div 6 = 6$, and its spending value is $(18 + 20 + 22 + 48 + 50 + 52) \div 6 = 210 \div 6 = 35$, giving (6, 35).
4. Squared distances to (6, 35). A: $16 + 289 = 305$. B: $9 + 225 = 234$. C: $4 + 169 = 173$. D: 173. E: 234. F: 305.
5. Add all six: $305 + 234 + 173 + 173 + 234 + 305 = 1424$.

Answer: The $k = 2$ inertia is 20 squared feature units; the $k = 1$ inertia is 1424, more than seventy times larger. One centroid cannot sit near both bunches, so it lands in the empty space between them.

Running k-means for $k = 3, 4$ and 5 gives 12.5, 5 and 2.5. Table 18.3 collects all five results with the drop each extra cluster buys. Inertia never rises when k grows, because more centroids always cover the same points more finely. Pushed to the extreme, $k = 6$ would give every customer a private centroid and an inertia of exactly 0: a perfect score, and a useless clustering.

Table 18.3 – Inertia of the six customers for five values of k , and what each extra cluster buys

k	Inertia	Drop from previous k	Comment
1	1424	—	one centroid, far from everybody
2	20	1404	the two real bunches are found
3	12.5	7.5	splits one bunch for very little gain
4	5	7.5	smaller and smaller pieces
5	2.5	2.5	five groups among six customers

That tension — lower is better, yet a lower score bought by chopping the data finer helps nobody — is what the elbow settles.

WORD TO KNOW

Elbow method — A way to choose k by plotting inertia against several values of k and looking for where adding another cluster stops helping much. The curve falls steeply, then bends and flattens like a bent arm; the bend is the “elbow”. Suggested by Robert Thorndike in 1953.

WORKED EXAMPLE 18.5 — Use the Elbow Method to Choose k for the Grocery Store

The inertias are: $k = 1$, 1424; $k = 2$, 20; $k = 3$, 12.5; $k = 4$, 5; $k = 5$, 2.5. Find the drop between each neighboring pair and choose a sensible k .

1. Drop from $k = 1$ to $k = 2$: $1424 - 20 = 1404$.
2. Drop from $k = 2$ to $k = 3$: $20 - 12.5 = 7.5$.
3. Drop from $k = 3$ to $k = 4$: $12.5 - 5 = 7.5$.
4. Drop from $k = 4$ to $k = 5$: $5 - 2.5 = 2.5$.
5. Line the four drops up: 1404, then 7.5, then 7.5, then 2.5. The first drop is about 187 times the second.

Answer: The elbow is at $k = 2$. Going from one group to two fixes almost the whole problem; every group added after that trims a tiny amount, so three or four promotions would cost real work for almost no extra tightness.

Figure 18.3 draws that curve beside one for a far more realistic dataset: 240 members of the same loyalty program, standardized first, who return in Lesson 18.8.

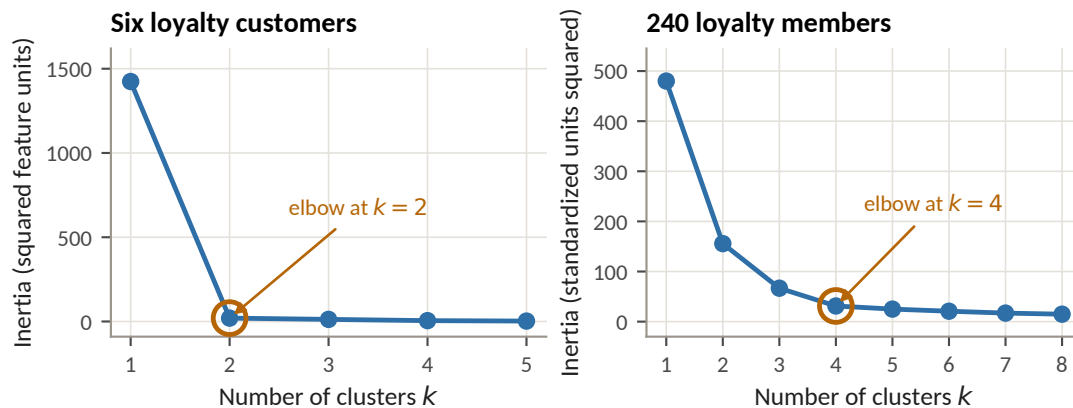


Figure 18.3 – Elbow curves. Left: the six hand-worked customers, where the bend is unmistakable. Right: 240 loyalty members, where the bend at $k = 4$ is real but gentler.

WHAT TO OBSERVE

The right-hand curve is far more typical of real data: the drops are 324.4, 88.9 and 35.5, then only 6.2 and 4.2, so after $k = 4$ extra clusters buy almost nothing. Two careful people could still argue for $k = 3$ or $k = 5$ here — the elbow is a judgment tool, not a formula.

COMMON MISTAKE

- ✗ Wrong:** Choosing whichever k gives the lowest inertia, and not noticing that inertia keeps falling all the way to 0 as k approaches the number of examples.
- ✓ Right:** Look for the bend, not the minimum. A lower inertia can simply mean the data was sliced into more, smaller, less useful pieces.

18.4 The Silhouette Score

Inertia has one blind spot: it looks only *inside* a cluster, never asking whether the clusters are far apart. A second score, invented by Peter Rousseeuw in 1987, asks both questions at once, one example at a time.

WORD TO KNOW

Silhouette score — A score from -1 to 1 for one example, comparing how close it sits to its own cluster with how close it sits to the nearest other cluster. Near 1 it is snugly inside its own group; near 0 it sits on the border; below 0 another cluster would suit it better. The mean over all examples scores the whole clustering.

For one example you need two averages. Call a the mean *true* distance to the other members of its own cluster, and b the mean true distance to every member of the nearest other cluster. Here, unlike everywhere else in k-means, the square roots really are taken, because the averages are compared as sizes, not merely ranked.

EQUATION: SILHOUETTE SCORE OF ONE EXAMPLE

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

In words: Subtract the average distance to your own group from the average distance to the nearest other group, then divide by whichever of the two averages is larger.

When it is used: To check whether clusters are genuinely separate, and to compare values of k in a way inertia cannot.

Symbol	Meaning	Unit or type
s_i	silhouette score of example i	between -1 and 1
a_i	mean distance from i to the other members of its own cluster	feature units
b_i	mean distance from i to all members of the nearest other cluster	feature units

Dividing by the larger average keeps the score between -1 and 1 . An example alone in its cluster has no a_i and scores 0 by convention.

WORKED EXAMPLE 18.6 – Find the Silhouette Score of Customer B

The finished clustering is group 1 = {A, B, C} and group 2 = {D, E, F}. Find s for customer B = $(3, 20)$. Distances here are true distances, so square roots are taken.

- Distance from B to A: $\sqrt{(3-2)^2 + (20-18)^2} = \sqrt{5} = 2.236$. Distance from B to C: $\sqrt{(3-4)^2 + (20-22)^2} = \sqrt{5} = 2.236$.
- Average them to get a : $(2.236 + 2.236) \div 2 = 2.236$.
- Distances from B to the other cluster: to D, $\sqrt{25 + 784} = \sqrt{809} = 28.443$; to E, $\sqrt{36 + 900} = \sqrt{936} = 30.594$; to F, $\sqrt{49 + 1024} = \sqrt{1073} = 32.757$.
- Average them to get b : $(28.443 + 30.594 + 32.757) \div 3 = 91.794 \div 3 = 30.598$.
- The larger of a and b is $b = 30.598$, so $s = (30.598 - 2.236) \div 30.598 = 28.362 \div 30.598 = 0.927$.

Answer: Customer B scores $s = 0.927$, near the maximum of 1. She sits about fourteen times closer to her own group than to the other one, so this is no borderline call.

Table 18.4 repeats that calculation for all six customers; their mean, 0.902, scores the whole clustering.

Table 18.4 – Silhouette score of every customer at $k = 2$. All six are close to 1

Customer	Cluster	a (own group)	b (other group)	s
A	1	3.354	32.760	0.898
B	1	2.236	30.598	0.927
C	1	3.354	28.448	0.882
D	2	3.354	28.448	0.882
E	2	2.236	30.598	0.927
F	2	3.354	32.760	0.898

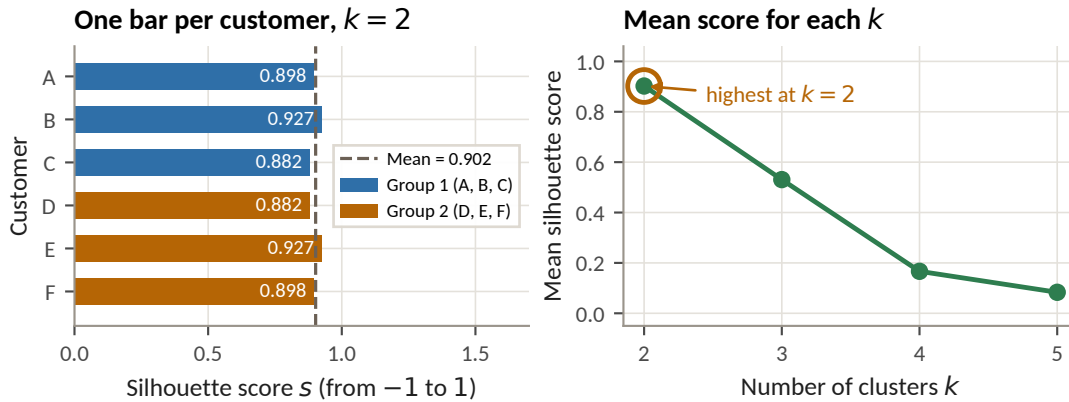


Figure 18.4 – Left: one silhouette bar per customer at $k = 2$, with the mean marked. Right: the mean silhouette for four values of k on the same six customers.

WHAT TO OBSERVE

Every bar on the left is long and positive, and the two clusters look almost identical — a clean, balanced split. The right panel is the important one: unlike inertia, which fell forever, the mean silhouette *peaks* at $k = 2$ and then collapses to 0.53, 0.17 and 0.08. Silhouette punishes the pointless extra clusters that inertia quietly rewards.

Use the two scores together: inertia says how tight the clusters are, silhouette whether they are also well separated. On the 240 loyalty members the mean silhouette is 0.569, 0.591, 0.634, 0.576 and 0.519 for $k = 2$ through 6, peaking at $k = 4$ — the same answer the elbow gave. When the two disagree, that usually means the groups overlap and no single k is clearly right. Silhouette is not accuracy, though: a mean of 0.55 does not mean “55% correct”. Read it as a shape score — above about 0.5 the clusters are reasonably separated, near 0.2 they overlap heavily, and below 0 they are wrong for the data.

18.5 Initialization Matters: Random Starts and k-means++

Iteration 1 showed that a poor starting guess costs extra rounds. On real data it can do something worse: the loop can settle into a stable grouping that is genuinely bad, then stop, because no single point wants to switch. This is the trap Chapter 9 described for gradient descent. Each round can only lower the inertia, so a run always slides downhill — but downhill from a bad start can lead into a shallow dip rather than the deepest valley. Figure 18.5 clusters the same 120 points twice with $k = 3$, changing nothing but the three starting points.

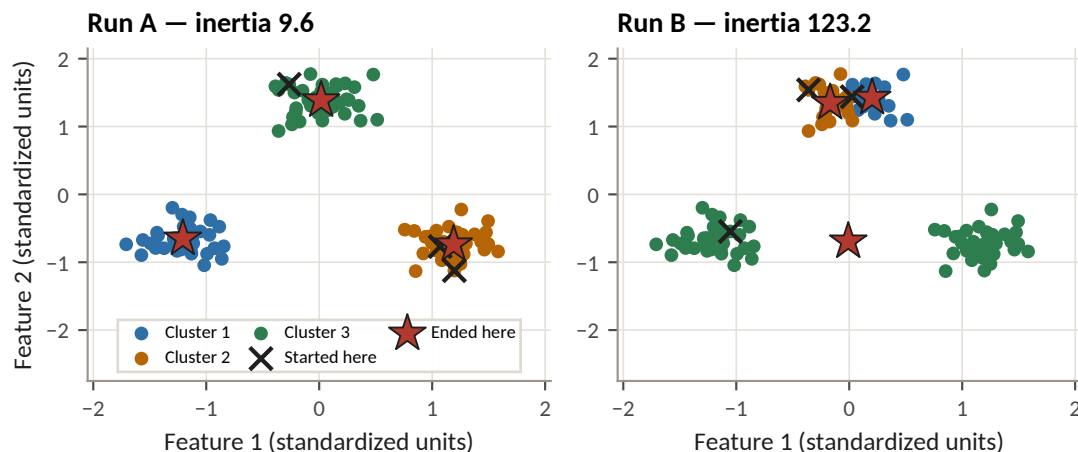


Figure 18.5 – One dataset, two runs of k -means with $k = 3$. Only the three starting points differ. Crosses show where each centroid started; red stars show where it stopped.

WHAT TO OBSERVE

Run A started with one point in each natural bunch and recovered all three, ending at inertia 9.6. Run B started with two points inside the same top bunch: it split that bunch in half, merged the two lower bunches into one wide cluster, and stranded a centroid between them — inertia 123.2, thirteen times worse. Both runs converged, and neither broke a rule.

Nothing inside one run can detect this, because it has nothing to compare itself with. The fix is to run the whole thing several times and keep the best result.

WORDS TO KNOW

Random restart — Running the same method several times from different random starting points and keeping whichever run scores best. In scikit-learn the setting `n_init` sets how many restarts k -means performs, and the run with the lowest inertia is the one you get back.

k -means++ — A rule for choosing starting centroids that spreads them out on purpose. The first is a random example; every centroid after that is also drawn at random, but with each example's chance of being picked made proportional to its squared distance from the nearest centroid already chosen, so far-away points are far more likely to be picked. It was published by David Arthur and Sergei Vassilvitskii in 2007 and is now the default almost everywhere.

Restarts are not a formality. Clustering the 240 loyalty members 50 times from 50 different random starts, with $k = 4$, eight of the fifty runs finished at an inertia near 62 instead of the best 31.21 — and no single run could tell which kind it was. Restarts help, but they are still guessing, which is what k -means++ improves on.

WORKED EXAMPLE 18.7 — k -means++ Probabilities After the First Centroid Is Chosen

Suppose k -means++ has picked customer A = (2, 18) as the first centroid and needs a second. Find each remaining customer's chance of being chosen.

1. Squared distance from each customer to A. B: $1 + 4 = 5$. C: $4 + 16 = 20$. D: $36 + 900 = 936$. E: $49 + 1024 = 1073$. F: $64 + 1156 = 1220$.
2. Add them for the total: $5 + 20 + 936 + 1073 + 1220 = 3254$.
3. Each chance is that customer's squared distance divided by the total. B: $5 \div 3254 = 0.0015$. C: $20 \div 3254 = 0.0061$. D: $936 \div 3254 = 0.288$. E: $1073 \div 3254 = 0.330$. F: $1220 \div 3254 = 0.375$.
4. Add the three far-away chances: $0.288 + 0.330 + 0.375 = 0.992$, the same as $3229 \div 3254$.

Answer: There is a 99.2% chance the second centroid comes from {D, E, F}, the group A does not belong to, and only a 0.8% chance of the near-useless start this chapter began with. That is what makes k-means++ so much more reliable than picking at random.

In practice, never run k-means once from a single guess and report the clusters as the only possible answer. Use k-means++ and several restarts — scikit-learn's defaults do both — and say in your report how many restarts you used, and whether they disagreed.

18.6 Limits: Shapes k-Means Cannot Find, and Scale

k-means is fast, simple and works well on a wide range of everyday problems, but it has three honest limits worth knowing before you point it at new data.

The first comes from its distance rule. “Nearest” is measured the same way in every direction, so the region belonging to a centroid is always a straight-edged patch around it, and k-means draws round, roughly equal-sized blobs and nothing else. Figure 18.6 shows two datasets where a person sees the groups instantly and k-means cannot.

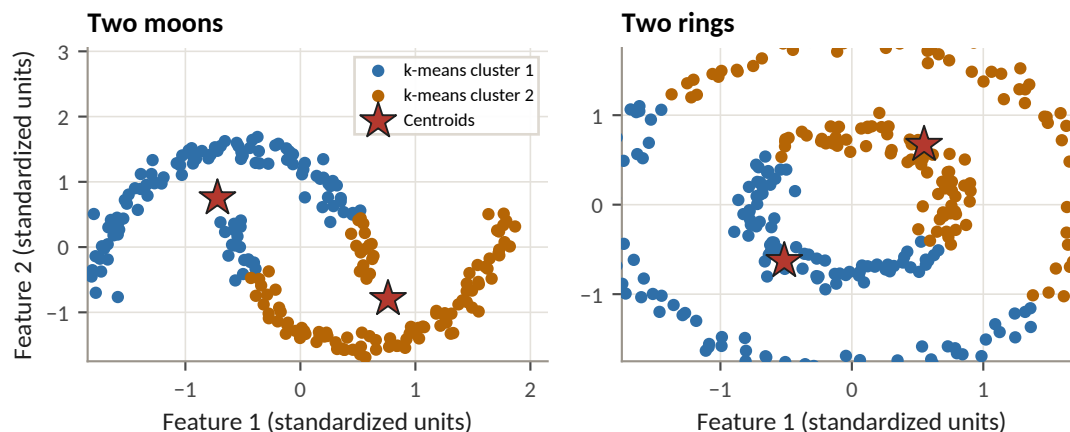


Figure 18.6 – Two shapes k-means cannot find. Colors are the clusters it produced with $k = 2$; the red stars are its centroids.

WHAT TO OBSERVE

On the left a person sees two crescents; k-means cuts straight across both, because the tip of one crescent really is closer to the other crescent's centroid than to its own. On the right the two rings share a center, so no pair of centroids can separate them, and k-means slices the picture into left and right halves. Both answers obey the method's rules perfectly, and both are useless here.

The second limit is the sensitivity to starting centroids from Lesson 18.5, which restarts and k-means++ reduce but never remove.

The third limit is scale, and it ruins more real projects than the other two together. Every k-means decision is a distance, so a feature measured in large numbers quietly takes over. Suppose the store adds lifetime spending, in IQD and in the millions, next to visits per month in single digits. A difference of two visits contributes $2^2 = 4$ to a squared distance; a difference of 50,000 IQD contributes 2,500,000,000. Visits would have no say at all, and the clusters would be lifetime-spending brackets in disguise. This is the trap Chapter 11 described for k-Nearest Neighbors, with the same cure — standardization, from Chapter 4.

COMMON MISTAKE

X Wrong: Running k-means on raw columns that sit on very different scales, such as visit counts beside lifetime spending in IQD.

✓ Right: Standardize every numerical feature first, then cluster the standardized values. Report the centroids back in the original units, because “18.2 visits per month” means something to a store owner and “1.42 standard deviations” does not.

When the groups you need really are long, curved or nested, the answer is a different method, not a better k . Chapter 19 covers two: hierarchical clustering, which builds a whole tree of groupings so you never fix k in advance, and DBSCAN, which follows dense trails of points and can call a lonely point noise instead of forcing it into a group.

18.7 Soft Clustering: Gaussian Mixture Models in Brief

Everything so far has made a hard choice for every example: group 1 or group 2, with nothing in between. For a customer deep inside a bunch that is fine. For one sitting in the gap between two bunches it throws away something true and useful — the fact that we are not sure.

WORDS TO KNOW

Hard assignment — Giving each example exactly one cluster, with no measure of how confident that choice was. Plain k-means always does this.

Soft clustering — Giving each example a share in every cluster — for instance 70% group 1 and 30% group 2 — so that uncertainty survives into the result.

Gaussian mixture model (GMM) — A soft clustering method that treats the data as a blend of k bell-shaped clouds, each with its own center, width, tilt and share of the data. It is fitted with the expectation-maximization algorithm, described by Arthur Dempster, Nan Laird and Donald Rubin in 1977, which repeats an assign-then-update loop very like k-means — except that the assignment is a set of fractions.

Responsibility — The probability that a particular example was produced by a particular cluster. Each example’s responsibilities across all k clusters always add up to 1.

Each cloud contributes a weighted score: its share of the data, multiplied by how likely it would be to produce a point right there. The responsibility is one cloud’s score as a fraction of the total. Figure 18.7 shows the difference this makes on 280 overlapping points.

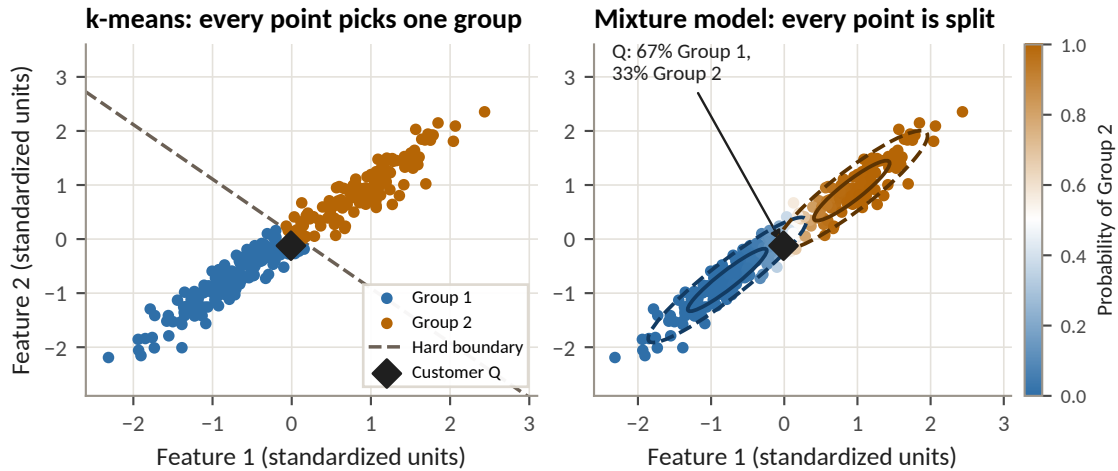


Figure 18.7 – The same 280 points, clustered twice. Left: *k*-means gives each point one group and a straight boundary. Right: a two-cluster mixture model gives each point a probability; the ellipses are the fitted clouds at one and two standard deviations.

WHAT TO OBSERVE

The *k*-means boundary on the left is a straight line, and every point falls firmly on one side of it. On the right the same points fade from blue through pale to orange in the overlap zone, and the tilted ellipses follow the stretched, diagonal shape of each cloud — a shape a round *k*-means cluster can never match.

EQUATION: RESPONSIBILITY OF ONE CLUSTER FOR ONE EXAMPLE

$$r_{im} = \frac{\pi_m \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m)}{\sum_{l=1}^k \pi_l \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}$$

In words: Work out each cloud's weighted score for this example, then divide one cloud's score by the total of all the scores.

When it is used: Whenever you want a probability of belonging rather than a yes-or-no label — for instance to leave uncertain customers out of a campaign.

Symbol	Meaning	Unit or type
r_{im}	responsibility of cluster m for example i	between 0 and 1
π_m	cluster m 's share of the whole dataset	between 0 and 1
$\mathcal{N}(\cdot)$	the bell-shaped density of a cluster at that point	density
$\boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m$	cluster m 's center, and its width-and-tilt	row of numbers; table

The bottom line is the top line added up over all k clusters, which is why the responsibilities add to exactly 1.

WORKED EXAMPLE 18.8 — Split One Borderline Customer Between Two Clusters

A mixture model with two clusters has been fitted to the standardized data in Figure 18.7. For customer Q, sitting in the gap between the clouds, scikit-learn reports weighted scores of 0.2269 for group 1 and 0.1097 for group 2. Find Q's responsibilities.

1. Add the two weighted scores to get the bottom of the fraction: $0.2269 + 0.1097 = 0.3366$.
2. Responsibility of group 1: $0.2269 \div 0.3366 = 0.674$.
3. Responsibility of group 2: $0.1097 \div 0.3366 = 0.326$.
4. Check that they add to 1: $0.674 + 0.326 = 1.000$, as they always must.

Answer: Q is 67.4% group 1 and 32.6% group 2. Plain k-means put Q in group 1 and said nothing more; the mixture model agrees on the winner but adds that a third of the evidence pointed the other way.

In scikit-learn, `GaussianMixture(n_components=4).fit(Z)` followed by `predict_proba` returns responsibilities like these. Fitted to the 240 loyalty members it reports one member — 7 visits a month, 39.4 thousand IQD a visit — as 61% in one cluster and 39% in another, and someone only 61% inside a group is a poor candidate for a campaign built around it. Soft clustering is not automatically better, though: a mixture model has far more numbers to estimate, so it needs more data and can fit noise when data is scarce (Chapter 7).

18.8 Using Clusters: Profiles, Names, and Far-Out Points

A finished clustering is not yet a result. The label column is meaningless on its own — cluster 0 is not better or bigger than cluster 1, and rerunning the method may swap the two — so the result appears only when a person describes each cluster in the language of the business.

WORD TO KNOW

Cluster profile — A short summary of one cluster giving its size and the average of each original feature inside it. Profiles are reported in the original units, never standardized ones, so that people can read them.

Take the 240 loyalty members from Figure 18.3, standardized and clustered with $k = 4$ as both scores suggested. Table 18.5 is their profile, and Figure 18.8 draws the same two averages as bars.

Table 18.5 – Cluster profile for the 240 loyalty members at $k = 4$, in original units. The names are the team's own working labels, not facts about anyone

Cluster	Members	Mean visits per month	Mean spend per visit (thousand IQD)	Working name
1	69	18.2	8.8	daily top-up shoppers
2	66	8.7	26.1	steady weekly regulars
3	58	4.1	54.2	fortnightly family shop
4	47	2.2	94.3	rare large-basket buyers

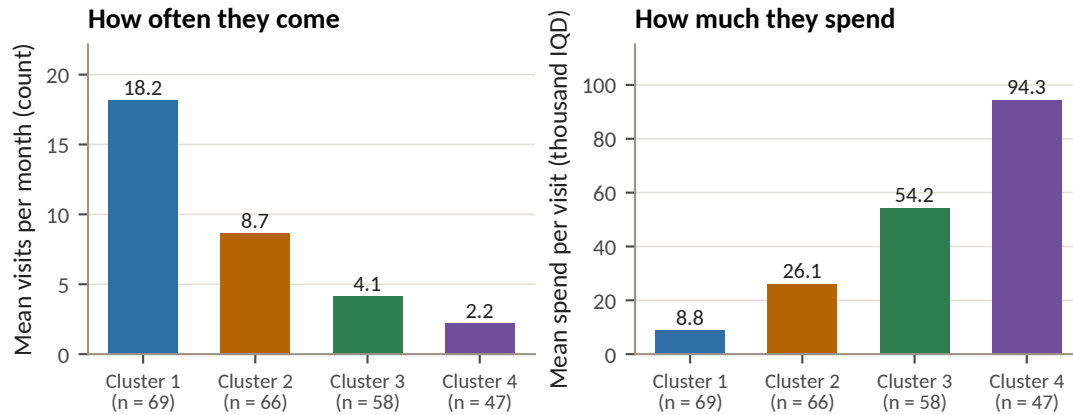


Figure 18.8 – The cluster profile as a picture: how often each group visits, and how much it spends per visit.

WHAT TO OBSERVE

The two panels run in opposite directions, and that is the whole story: the more often a group visits, the less it spends each time. Neither end is the “better customer” — clusters 1 and 4 spend roughly 160 and 207 thousand IQD a month — but they need very different offers.

RESPONSIBLE USE

A cluster is a pattern in data, not a fact about a person. Two members of one cluster can differ in ways nobody measured, and a person’s habits can drift into a different cluster next season. Use clusters to shape broad decisions, such as which kind of offer to design, and rerun the clustering regularly. Never attach a cluster name to somebody’s record as though it were permanent, and never let a cluster alone decide anything affecting a person’s price, credit or opportunity (Chapter 24).

Clusters have one more everyday use. Once the centroids exist, any new example can be measured against all of them — and an example far from every one of them is telling you something.

WORKED EXAMPLE 18.9 — Check Whether a New Customer Fits Any Existing Cluster

The finished centroids are $\mathbf{c}_1 = (3, 20)$ and $\mathbf{c}_2 = (9, 50)$. A new member, G, visited 9 times last month and spent only 16 thousand IQD per visit, so $G = (9, 16)$. Which cluster is nearest, and does G fit it?

1. Squared distance to $\mathbf{c}_1 = (3, 20)$: $(9 - 3)^2 + (16 - 20)^2 = 36 + 16 = 52$.
2. Squared distance to $\mathbf{c}_2 = (9, 50)$: $(9 - 9)^2 + (16 - 50)^2 = 0 + 1156 = 1156$.
3. The smaller is 52, so k-means assigns G to cluster 1.
4. Compare 52 with how far that cluster’s real members sit from the same centroid: A, B and C scored 5, 0 and 5, so the largest is 5.
5. $52 \div 5 = 10.4$: G is more than ten times further from the centroid than the loosest genuine member of the group.

Answer: G is assigned to cluster 1 but does not belong to it: the true distance is $\sqrt{52} = 7.2$, against 2.2 for the furthest real member. G is a new pattern — frequent visits with a tiny basket — that no cluster describes.

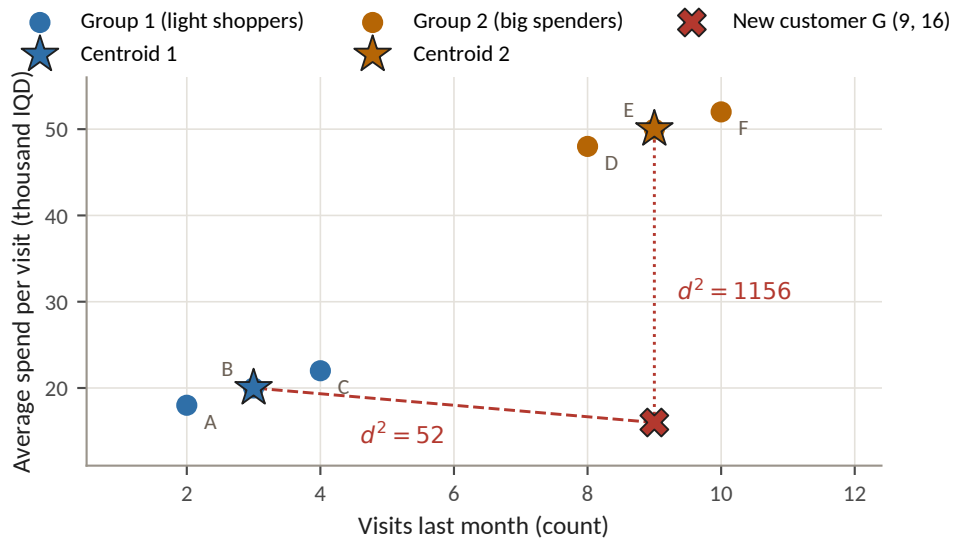


Figure 18.9 – Every point gets a cluster, even one that fits nowhere. The dashed and dotted lines carry customer G's two squared distances.

WHAT TO OBSERVE

G is nearer to centroid 1, so k-means files her there without hesitation — yet she sits below and to the right of everything, on her own. Always read the distance, not only the label: a large distance to the nearest centroid is one of the simplest anomaly detectors there is, and Chapter 21 builds several more.

COMMON MISTAKES AND MISCONCEPTIONS

- Reaching for clustering when every row already carries a known label, where a supervised method would give a sharper, checkable answer.
- Stopping after a single assign-and-update round instead of repeating until no example switches group.
- Believing k-means always finds the one best grouping, whatever starting centroids it happened to get, or choosing k by picking the lowest inertia.
- Running k-means on raw features that sit on very different scales, letting the large-number feature decide every distance by itself.
- Expecting k-means to find long, curved, nested or very unequal groups, when its rule can only draw round, straight-edged patches.
- Treating a cluster number as a permanent fact about a person, instead of a pattern that drifts and should be recomputed.

Practical Activities

ACTIVITY (PAPER) — Run Two Rounds of k-Means by Hand

Six market stalls in a Duhok bazaar are described by staff on duty and daily sales in hundred thousand IQD: $S1 = (1, 2)$, $S2 = (2, 1)$, $S3 = (3, 3)$, $S4 = (8, 7)$, $S5 = (9, 9)$, $S6 = (10, 8)$.

1. Plot all six stalls on graph paper and mark the starting centroids $c_1 = S1 = (1, 2)$ and $c_2 = S3 = (3, 3)$.

2. Round 1: build a table with one row per stall, a column for d^2 to each centroid, and a last column for the winner — every number here is a whole number. Then compute both new centroids; one lands on a decimal, so keep two decimal places.
3. Round 2: repeat the whole table with the moved centroids. Exactly one stall changes group; circle it on your plot, then compute both centroids again.
4. Would a third round change anything? Explain in one sentence how you can tell without redoing all the arithmetic. Then compute the inertia of your finished clustering using the equation from Lesson 18.3; the answer is a small whole number.

You need: Graph paper, pencil, ruler and a calculator

ACTIVITY (PYTHON) — Reproduce the Hand Clustering, Then Scale It Up

The first half repeats the hand work exactly: `init=start` forces scikit-learn to begin from customers B and C, the same poor guess, and `n_init=1` stops it restarting. The second half loads the 240 loyalty members and prints both scores for $k = 2$ to 6.

Python • pandas • scikit-learn

```
import numpy as np, pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples, silhouette_score
from sklearn.preprocessing import StandardScaler

X = np.array([[2, 18], [3, 20], [4, 22], [8, 48], [9, 50], [10, 52]])
start = np.array([[3, 20], [4, 22]]) # customers B and C
model = KMeans(n_clusters=2, init=start, n_init=1).fit(X)
print("labels      :", model.labels_, " inertia:", model.inertia_)
print("centroids  :", model.cluster_centers_.tolist())
print("silhouette:", np.round(silhouette_samples(X, model.labels_), 3),
      round(silhouette_score(X, model.labels_), 3))

df = pd.read_csv("datasets/ch18_members.csv") # the 240 loyalty members
Z = StandardScaler().fit_transform(df[["visits_per_month", "avg_spend_kiqd"]])
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=20, random_state=0).fit(Z)
    print("k =", k, " inertia =", round(km.inertia_, 1),
          " silhouette =", round(silhouette_score(Z, km.labels_), 3))
```

Output

```
labels      : [0 0 0 1 1 1] inertia: 20.0
centroids   : [[3.0, 20.0], [9.0, 50.0]]
silhouette: [0.898 0.927 0.882 0.882 0.927 0.898] 0.902
k = 2 inertia = 155.6 silhouette = 0.569
k = 3 inertia = 66.7 silhouette = 0.591
k = 4 inertia = 31.2 silhouette = 0.634
k = 5 inertia = 25.0 silhouette = 0.576
k = 6 inertia = 20.8 silhouette = 0.519
```

The first three lines match the hand work exactly: the centroids of iteration 3, the inertia 20, and the six silhouette scores of Table 18.4. The last five are Figure 18.3 as numbers, and both scores point at $k = 4$. Now change `start` to customers A and F: do the centroids change? Does the inertia?

You need: A computer with Python, pandas and scikit-learn (see Appendix B), and `datasets/ch18_members.csv`

CASE STUDY — Segmenting Shoppers for a Supermarket Chain in Pune

A supermarket chain with a dozen branches across Pune has run the same loyalty program for years, sending every member the identical weekly discount by text message. The team suspects this wastes money: an offer that excites a shopper who comes once a fortnight and fills a trolley may mean nothing to someone who drops in three times a week for milk and bread. Nobody has ever sorted members into types, so there is no answer key — only the program's own transaction records.

The team chooses a small, meaningful set of features rather than every available column: average visits per week over the last three months, average spend per basket in rupees, and the share of visits falling on a weekend. Those arrive on wildly different scales — roughly 0 to 7 visits, hundreds of rupees, and a percentage — so, following Lesson 18.6, all three are standardized before any distance is computed. They then run k-means with `k-means++` and twenty restarts for every k from 1 to 10. The inertia curve bends clearly at $k = 4$ and the mean silhouette peaks in the same place; when two scores agree, the choice is far easier to defend to a manager.

Reading the four centroids back in original units, a category manager writes a working name beside each — frequent small-basket shoppers, weekend bulk buyers, steady mid-week regulars, and a small fourth group whose basket sizes vary far more than anyone else's. Those names go on a slide, not on any customer's account. The fourth group gets extra scrutiny, for the reason Lesson 18.8 gives: its members sit unusually far from their own centroid, which often means the group is less a real type than a bin for people who fitted nowhere else, so it receives no automatic campaign, only a list for a manager to review by hand. The other three get three different weekly messages: a weekday loyalty-point bonus, a larger one-time weekend discount, and a thank-you note. The team also agrees to rerun the clustering every quarter, because a family's shopping changes when a child starts school or a shift pattern changes, and last year's segment should never quietly steer this year's messages.

CHAPTER SUMMARY

- Clustering is unsupervised learning: it groups similar examples with no labels to learn from, so there is no accuracy to report. k-means repeats two steps — assign every example to its nearest centroid, then move each centroid to the mean of its assigned examples — and stops when a full round changes nothing.
- Comparing squared distances gives the same nearest-centroid answer as true distances, so no square root is needed inside k-means.
- Inertia is the total squared distance from every example to its own centroid; it always falls as k grows, so the elbow method looks for the bend where extra clusters stop helping instead of the lowest value.
- The silhouette score compares each example's distance to its own cluster with its distance to the nearest other cluster; unlike inertia, the mean silhouette has a genuine peak.
- Different starting centroids can lead to genuinely different, worse clusterings, so use several restarts and the `k-means++` starting rule.
- k-means draws only round, roughly equal-sized groups and fails on long, curved or nested shapes; Chapter 19 gives methods that do not. Standardize every feature first, or the largest-numbered column decides every distance by itself.
- A Gaussian mixture model clusters softly, giving each example a responsibility — a probability of belonging — in every cluster.
- Clusters become useful only through a profile in original units and careful human naming; a point far from every centroid is a warning, not a member.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain the difference between clustering and classification, and give one example of each.

Problem 2 [Basic] Explain what the letter k in k-means stands for, and say who chooses its value and when.

Problem 3 [Basic] List, in order, the two steps k-means repeats every round once the starting centroids have been placed, and state the rule that tells it to stop.

Problem 4 [Basic] Define inertia in one or two sentences, and state what happens to it as k grows toward the number of examples.

Problem 5 [Basic] A hospital team wants to group patients by symptom similarity, with no existing diagnosis categories to learn from. Name the learning task and explain in one sentence why it is not classification.

Problem 6 [Standard] Two centroids are $\mathbf{c}_1 = (1, 1)$ and $\mathbf{c}_2 = (6, 6)$. Find the squared distance from point $P = (2, 3)$ to each centroid, showing every step, and state which cluster P joins.

Problem 7 [Standard] A cluster contains the three points $(2, 4)$, $(6, 8)$ and $(7, 6)$. Find its centroid, showing the average of each feature separately, then find the cluster's contribution to the inertia.

Problem 8 [Standard] A team records these inertias: $k = 1, 800$; $k = 2, 300$; $k = 3, 260$; $k = 4, 230$; $k = 5, 215$. Find every drop and identify the elbow. Explain in one sentence why $k = 5$ is not the answer even though its inertia is lowest.

Problem 9 [Standard] A point sits an average distance of 2.0 from the other members of its own cluster and 3.0 from the members of the nearest other cluster. Calculate its silhouette score. Then recalculate for a point with $a = 3.0$ and $b = 2.0$, and explain in one sentence what the sign of the second answer means.

Problem 10 [Standard] k-means++ has chosen its first centroid. Three remaining points have squared distances 4, 16 and 80 from it. Find each point's chance of being picked as the second centroid, and say in one sentence why this beats picking uniformly at random.

Problem 11 [Challenge] Four points are $P_1 = (1, 1)$, $P_2 = (2, 2)$, $P_3 = (8, 8)$ and $P_4 = (9, 9)$, with starting centroids $\mathbf{c}_1 = (1, 1)$ and $\mathbf{c}_2 = (2, 2)$. (a) Assign each point using squared distance, showing every calculation. (b) Recompute both centroids. (c) Reassign every point to the new centroids, rounding to two decimal places, and state whether the clustering has converged. (d) Run one further round and give the final centroids and the final inertia.

Problem 12 [Challenge] A mixture model with three clusters gives one customer the weighted scores 0.030, 0.012 and 0.008. (a) Find all three responsibilities. (b) State which cluster plain k-means would report. (c) Give one business decision you would make differently knowing the responsibilities rather than the hard label alone.

Problem 13 [Challenge] Suppose k-means is run twice on the same data with the same k , giving inertias of 480 and 512. (a) Which run would you keep, and why? (b) Does the better inertia prove that run found the "true" groups? Explain. (c) Name one shape of grouping k-means would miss whatever k you chose, and a method from Chapter 19 that could find it.

Hierarchical and Density-Based Clustering

Building a tree of clusters, and finding clusters of any shape

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how hierarchical clustering differs from k-means, and why it never asks you to choose the number of groups first.
- Build a distance matrix by hand and say how many distinct distances it holds.
- Run agglomerative clustering by hand, merging the two closest clusters one step at a time.
- Compare the four linkage rules — single, complete, average and Ward — and predict which one will chain.
- Read a dendrogram, cut it at a chosen height, and report the clusters the cut produces.
- Label points as core, border or noise using DBSCAN's two settings, ϵ and minPts.
- Choose between k-means, hierarchical clustering and DBSCAN for a new project.

Introduction

Ask somebody in the Kurdistan Region where they live, and the answer usually has several levels inside it. A house sits on a street; streets make a neighborhood; neighborhoods make a town; towns and villages make a district. Nobody decided how many levels there would be — they are simply there, one nested inside the next, and you pick whichever suits the conversation.

Chapter 18 built a clustering method that works very differently. Before k-means can run at all, somebody must choose k . Choose $k = 3$ and you get three groups; choose $k = 5$ and you get five, from the very same data. The elbow method and the silhouette score help, but the choice is made up front, and the result is one flat list of groups with nothing inside them.

This chapter builds two methods that answer the two most common complaints about k-means. *Hierarchical clustering* produces the whole family tree of groupings at once, from “every example alone” up to “everything together”, so you decide how many clusters you want after seeing the tree. *DBSCAN* hunts for crowded regions instead of round blobs, so it traces long curved groups k-means could never find — and may say a lonely point belongs to no group at all. Both use one ingredient you already have: the distance between two points.

19.1 Clusters Inside Clusters: the Hierarchical Idea

An agricultural office near Zakho keeps records on local farms. The director knows the farms are not all alike: some grow wheat on dry land, some grow vegetables under irrigation, a few keep animals. Those groups also have groups inside them — among the vegetable farms, some sell locally and some to a packing plant. A method that returns exactly three groups throws this nested structure away.

WORDS TO KNOW

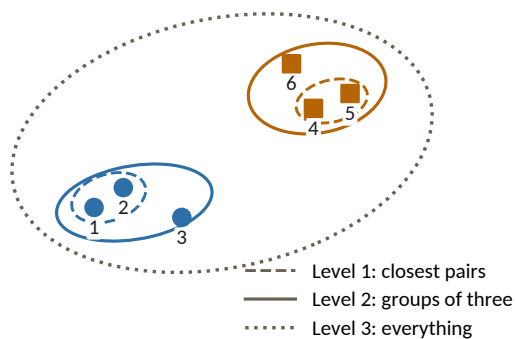
Hierarchical clustering — A family of methods that produce a whole tree of groupings at once, from every example on its own up to one group containing everything. You choose how many clusters you want after the tree is built, not before.

Agglomerative clustering — The bottom-up way to build that tree. Every example starts alone; the two closest clusters are merged; this repeats until one cluster is left.

Divisive clustering — The top-down way: everything starts in one big cluster, which is split in two, then split again. Much rarer than agglomerative clustering, because there are far more ways to split a group than to merge two of them.

This chapter uses the agglomerative version throughout, because it is the one you meet in practice and the one you can run by hand. Figure 19.1 shows the idea on six examples: closest pairs join first, small groups join into bigger ones, and finally everything joins. Nothing in the left picture says how many clusters there “really” are — four at level 1, two at level 2, one at level 3 — and the tree on the right stores all three answers at once.

Groups inside groups



The same information as a tree

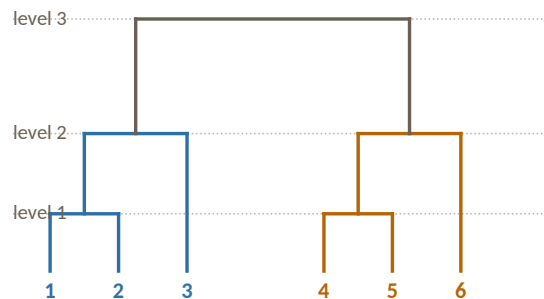


Figure 19.1 – Six points grouped at three levels (left) and the same nested grouping drawn as a tree (right). Each horizontal bar is one merge.

WORD TO KNOW

Dendrogram — The tree picture that hierarchical clustering produces. The word comes from the Greek *dendron*, meaning tree. Every merge appears as a horizontal bar, and the height of the bar shows how far apart the two merged clusters were.

Do not conclude that hierarchical clustering removes every choice. You still choose the distance measure, the linkage rule and where to cut the tree; the advantage is only that the last of those is made *after* looking at the whole tree, instead of guessed before you start.

19.2 Agglomerative Clustering Step by Step

A small non-governmental organization plans mobile health and education visits to villages in a rural district of the Kurdistan Region. For each village it has two numbers, both in kilometers: how far it is from the nearest health clinic, and how far from the nearest secondary school. Villages far from both need a very different visit from villages close to both — but nobody has sorted them into “types”, and nobody knows how many types there should be. Table 19.1 lists the six villages the team starts with.

Table 19.1 – Six villages, described by two distances (illustrative example numbers)

Village	Distance to nearest clinic (km)	Distance to nearest school (km)	Written as a point
A	1	1	(1, 1)
B	3	2	(3, 2)
C	5	4	(5, 4)
D	9	5	(9, 5)
E	10	5	(10, 5)
F	9	8	(9, 8)

Agglomerative clustering always begins the same way: work out how far every example is from every other one, and write the answers in a square table.

WORD TO KNOW

Distance matrix — A square table holding the distance between every pair of examples. Row i , column j holds the distance from example i to example j . The diagonal is all zeros, and the table is symmetric, because the distance from A to B equals the distance from B to A.

The distances are the ordinary straight-line distances from Chapter 11: square the gap in each feature, add the squared gaps, take the square root.

EQUATION: EUCLIDEAN DISTANCE AND THE DISTANCE MATRIX

$$D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^p (x_{im} - x_{jm})^2}$$

In words: To fill one cell of the table, subtract the two examples feature by feature, square each difference, add the squares, and take the square root of the total.

When it is used: As the first step of any hierarchical clustering, and whenever a method must compare how similar every pair of examples is.

Symbol	Meaning	Unit or type
D_{ij}	the cell in row i , column j of the distance matrix	same unit as the features
$\mathbf{x}_i$	example i , written as a list of p feature values	—
x_{im}	feature m of example i	unit of feature m
p	number of features	count

Here both features are in kilometers, so the distances are in kilometers too. When features use different units, standardize them first (Lesson 19.7). Unlike k-means, hierarchical clustering really does take the square root, because the heights are read off the finished tree.

WORKED EXAMPLE 19.1 — Build the Distance Matrix for the Six Villages

Using Table 19.1, find the distance between A and B, between B and C, and between D and E. Then say how many different distances the full table contains.

1. A is (1, 1) and B is (3, 2). The gaps are $3 - 1 = 2$ and $2 - 1 = 1$.
2. Square and add: $2^2 + 1^2 = 4 + 1 = 5$. Take the square root: $\sqrt{5} = 2.24$ km.

3. B is (3, 2) and C is (5, 4). The gaps are 2 and 2, so $2^2 + 2^2 = 8$ and $\sqrt{8} = 2.83$ km.
4. D is (9, 5) and E is (10, 5). The gaps are 1 and 0, so $1^2 + 0^2 = 1$ and $\sqrt{1} = 1.00$ km.
5. The full table has $6 \times 6 = 36$ cells. Six are the zero diagonal, and the other 30 come in matching pairs, so there are $30 \div 2 = 15$ different distances.

Answer: $d(A, B) = 2.24$ km, $d(B, C) = 2.83$ km and $d(D, E) = 1.00$ km. Only 15 of the 36 cells hold new information.

Repeating those steps for every pair fills the table. Figure 19.2 shows the finished matrix beside the villages themselves, and every worked example below reads its numbers from that right-hand panel.

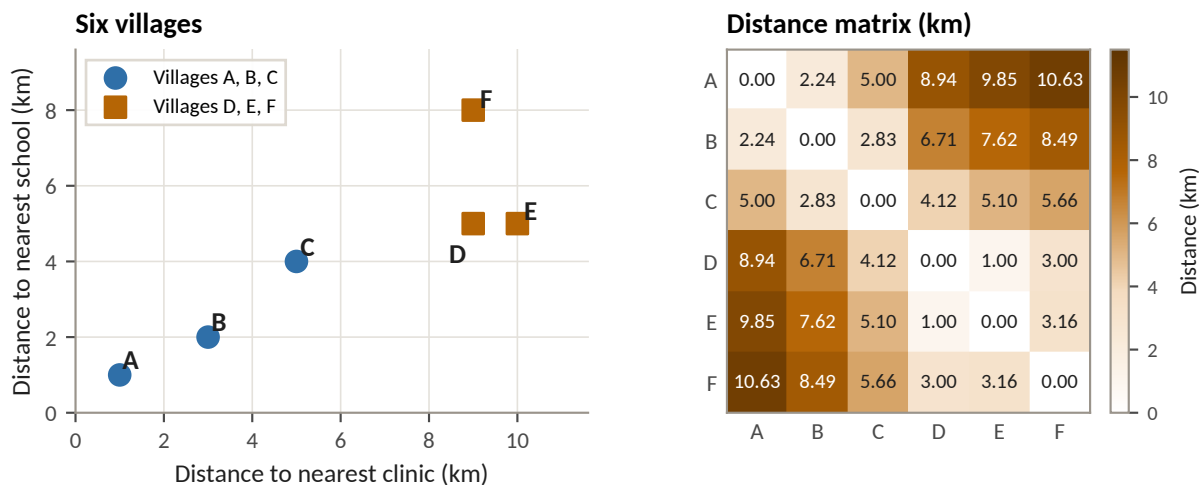


Figure 19.2 – The six villages as points (left) and their distance matrix in kilometers (right). Light cells are close pairs.

WHAT TO OBSERVE

The lightest off-diagonal cell is D–E at 1.00 km, and those two villages sit almost on top of each other on the left; the darkest is A–F at 10.63 km, at opposite corners.

With the table in hand, the algorithm is three instructions: start with every example alone; find the two closest clusters and merge them, recording how far apart they were; repeat until one cluster holds everything. Six villages therefore need exactly five merges. The only thing missing is what “closest” means once a cluster holds more than one point, and the simplest answer is to measure from the nearest member of one cluster to the nearest member of the other.

EQUATION: SINGLE LINKAGE (NEAREST-NEIGHBOR RULE)

$$d_{\text{single}}(P, Q) = \min_{i \in P, j \in Q} D_{ij}$$

In words: The distance between two clusters is the distance between their two closest members: look at every pair with one point from each cluster, and keep the smallest.

When it is used: When clusters may be long, thin or curved, and you care that a point is connected to some member of a group rather than to all of them.

Symbol	Meaning	Unit or type
P, Q	the two clusters being compared	sets of examples
D_{ij}	distance between example i and example j	same unit as the features
min	take the smallest value in the list	—

With one point in each cluster, all four linkage rules in this chapter agree: the answer is just the distance between those two points. The rules differ only once a cluster holds two or more members.

WORKED EXAMPLE 19.2 — The First Three Merges with Single Linkage

Using the distance matrix in Figure 19.2, carry out the first three merges with the single-linkage rule.

- Merge 1.** Every cluster holds one village, so the cluster distances are the 15 numbers in the matrix. The smallest is $d(D, E) = 1.00$ km. Merge D and E.
- Merge 2.** The smallest remaining distance is $d(A, B) = 2.24$ km. Merge A and B. Four clusters remain: $\{A, B\}$, $\{C\}$, $\{D, E\}$, $\{F\}$.
- Merge 3, candidate 1.** $\{A, B\}$ to C: the two distances are 5.00 and 2.83, and single linkage keeps the smaller, 2.83 km.
- Merge 3, candidate 2.** $\{D, E\}$ to F: the distances are 3.00 and 3.16, so single linkage keeps 3.00 km.
- The rest.** C to $\{D, E\}$ gives 4.12; C to F gives 5.66; $\{A, B\}$ to $\{D, E\}$ gives 6.71; $\{A, B\}$ to F gives 8.49.
- Compare the six candidates: the smallest is 2.83 km, so merge $\{A, B\}$ with C.

Answer: Single linkage merges D with E at 1.00 km, A with B at 2.24 km, then C into $\{A, B\}$ at 2.83 km. After three merges the clusters are $\{A, B, C\}$, $\{D, E\}$ and $\{F\}$.

Figure 19.3 draws these merges plus the fourth. Two more finish the job: $\{D, E\}$ takes in F at 3.00 km, and the two big clusters join at 4.12 km, the distance from C to D.

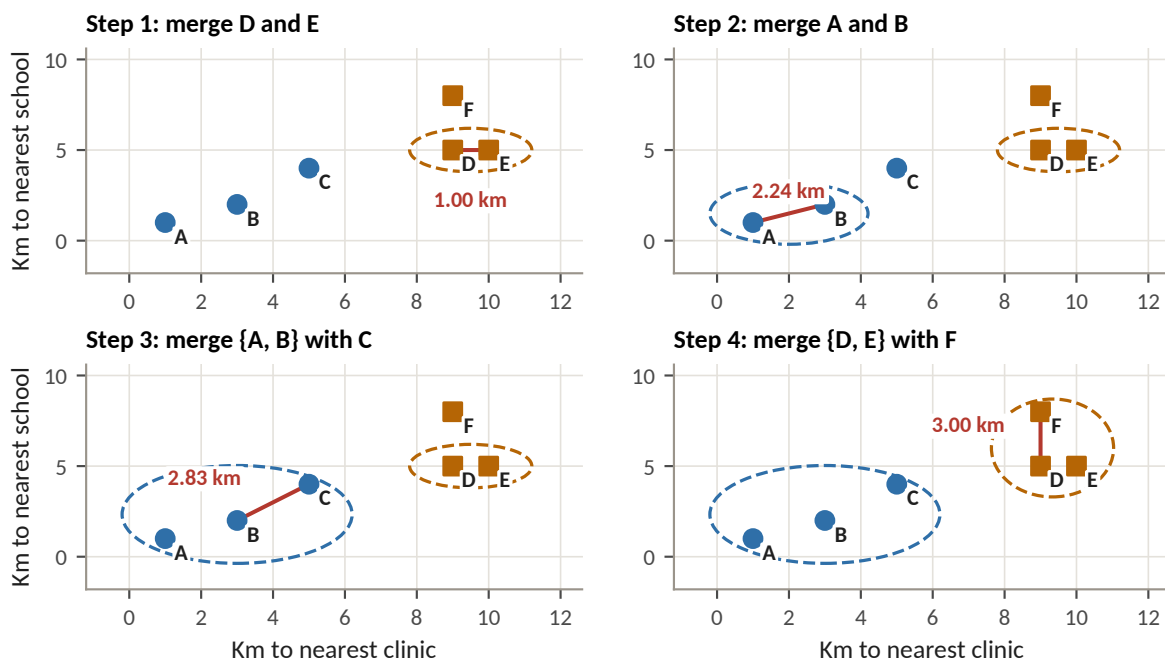


Figure 19.3 – The first four single-linkage merges. The red line marks the pair whose distance decided the merge; dashed rings show the clusters formed.

WHAT TO OBSERVE

In every panel the red line joins two *individual* villages, never two group centers — exactly what single linkage measures. After step 3 the left cluster is already stretched: A and C are 5.00 km apart, yet the rule merged them at only 2.83 km, because it looked only at the closest pair.

19.3 Linkage Rules: Single, Complete, Average and Ward

Single linkage is only one answer to “how far apart are these two groups?”. The rule that gives the answer has a name, and Figure 19.4 shows the four that are used in practice.

WORDS TO KNOW

Linkage — The rule that decides how far apart two *clusters* are, once a cluster may hold more than one example. Changing the rule can change the final clusters completely.

Single linkage — Cluster distance = the distance between the two closest members. Described for biological classification by Peter Sneath in 1957.

Complete linkage — Cluster distance = the distance between the two farthest members. Introduced by Thorvald Sørensen in 1948.

Average linkage — Cluster distance = the mean distance over every pair with one member from each cluster. Introduced by Robert Sokal and Charles Michener in 1958.

Ward's method — Merge whichever two clusters add the least extra spread, measured as the increase in total squared distance from points to their own centroid. Published by Joe H. Ward Jr. in 1963.

Chaining — The tendency of single linkage to build long, straggly clusters by following a chain of nearby points. A thin line of examples running between two real groups is enough to make single linkage merge those groups into one.

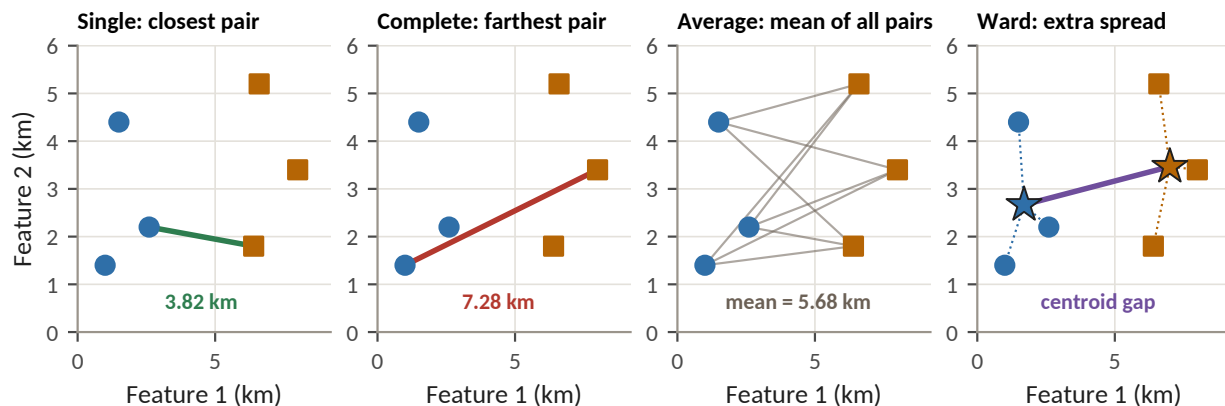


Figure 19.4 – Four ways to measure the distance between the same two clusters. Ward compares the two centroids (stars).

WHAT TO OBSERVE

The same two clusters give four different answers: 3.82 km, 7.28 km, 5.68 km, and a fourth number that is not a distance at all but an amount of extra spread. None is “correct” — they pay attention to different things, which is why the rule must be chosen deliberately.

Complete linkage is the exact opposite of single linkage. Instead of asking “are these groups touching anywhere?”, it asks “how bad is the worst pair?”. Average linkage sits between the two, using every cross-cluster pair rather than just one.

EQUATION: COMPLETE AND AVERAGE LINKAGE

$$d_{\text{complete}}(P, Q) = \max_{i \in P, j \in Q} D_{ij}, \quad d_{\text{average}}(P, Q) = \frac{1}{|P||Q|} \sum_{i \in P} \sum_{j \in Q} D_{ij}$$

In words: Complete linkage keeps the largest of all the cross-cluster distances. Average linkage adds every cross-cluster distance and divides by the number of such pairs.

When it is used: Complete linkage when you want compact clusters in which every member is close to every other. Average linkage when you do not want one very close, or one very distant, pair to decide the merge on its own.

Symbol	Meaning	Unit or type
P, Q	the two clusters being compared	sets of examples
D_{ij}	distance between example i and example j	same unit as the features
$ P Q $	the number of cross-cluster pairs	count

Complete linkage is the strictest rule: a cluster grows only if every new pair it creates is short. Average linkage is also known as UPGMA, for “unweighted pair group method with arithmetic mean”.

WORKED EXAMPLE 19.3 — The Same Three Merges with Complete Linkage

Repeat the previous worked example with complete linkage, and find the first step where the two rules disagree.

- Merges 1 and 2.** With one point per cluster both rules read the same 15 numbers, so D merges with E at 1.00 km and A merges with B at 2.24 km. No disagreement yet.
- Merge 3, candidate 1.** {A, B} to C: the distances are 5.00 and 2.83, and complete linkage keeps the larger, 5.00 km.
- Merge 3, candidate 2.** {D, E} to F: the distances are 3.00 and 3.16, so complete linkage keeps 3.16 km.
- The rest.** C to {D, E} gives 5.10; C to F gives 5.66; {A, B} to {D, E} gives 9.85; {A, B} to F gives 10.63.
- Compare: the smallest is now 3.16 km, so complete linkage merges F into {D, E} — not C into {A, B}.

Answer: Complete linkage merges D with E, then A with B, then F into {D, E} at 3.16 km. The rules first disagree at merge 3: single linkage builds {A, B, C} while complete linkage builds {D, E, F}.

WORKED EXAMPLE 19.4 — Average-Linkage Distance Between Two Clusters

Find the average-linkage distance between {A, B} and {D, E}.

- Count the pairs: $2 \times 2 = 4$, namely A–D, A–E, B–D and B–E.
- Read them from Figure 19.2: 8.94, 9.85, 6.71 and 7.62 km.
- Add: $8.94 + 9.85 + 6.71 + 7.62 = 33.12$. Divide by 4: $33.12 \div 4 = 8.28$.
- Compare with the other rules on the same two clusters: single linkage reports the smallest, 6.71 km, and complete linkage the largest, 9.85 km.

Answer: The average-linkage distance is 8.28 km, between the single-linkage answer of 6.71 km and the complete-linkage answer of 9.85 km — exactly where an average must sit.

Table 19.2 lays the two full merge sequences side by side. Both rules end with the same two big clusters here, but they reach them by different routes and at very different heights, and their three-cluster answers genuinely disagree about where village C belongs.

Table 19.2 – The five merges under single and complete linkage, with the distance at which each happened

Step	Single: merged	Height (km)	Complete: merged	Height (km)
1	D and E	1.00	D and E	1.00
2	A and B	2.24	A and B	2.24
3	{A, B} and C	2.83	{D, E} and F	3.16
4	{D, E} and F	3.00	{A, B} and C	5.00
5	the two big clusters	4.12	the two big clusters	10.63

The fourth rule, Ward's method, measures a cost rather than a distance. Chapter 18 introduced inertia: the total squared distance from every point to its own cluster's centroid. Merging two clusters always raises inertia, because the merged cluster has one centroid instead of two, and Ward's method merges whichever pair raises it least.

EQUATION: WARD'S MERGE COST (INCREASE IN INERTIA)

$$\Delta(P, Q) = \frac{|P||Q|}{|P| + |Q|} \|\mathbf{c}_P - \mathbf{c}_Q\|^2$$

In words: Find each cluster's centroid, square the distance between the two centroids, then multiply by the two cluster sizes multiplied together and divided by their total. The result is the extra inertia this merge would create.

When it is used: When you want compact, similarly sized clusters and already think in terms of inertia, as k-means does. Ward is the most common default in practice.

Symbol	Meaning	Unit or type
$\Delta(P, Q)$	extra inertia created by merging P and Q	squared feature units
$\mathbf{c}_P, \mathbf{c}_Q$	the centroid of each cluster	same unit as the features
$ P , Q $	how many examples each cluster holds	count
$\ \cdot\ ^2$	squared straight-line distance	squared feature units

With two single points this becomes $\frac{1}{2}d^2$, so Ward always agrees with the other rules about the very first merge. Software plots Ward heights on a rescaled axis, so read a Ward dendrogram for its shape, not for kilometers.

WORKED EXAMPLE 19.5 – Ward's Method Chooses the Third Merge

After D and E have merged, and A and B have merged, Ward's method must choose between adding C to {A, B} and adding F to {D, E}. Compute both costs and decide.

1. Centroid of {A, B}: $(\frac{1+3}{2}, \frac{1+2}{2}) = (2, 1.5)$; cluster {C} has centroid (5, 4).
2. Squared distance between them: $(5 - 2)^2 + (4 - 1.5)^2 = 9 + 6.25 = 15.25$.
3. The sizes are 2 and 1, so the multiplier is $\frac{2 \times 1}{2+1} = \frac{2}{3}$, and the cost is $\frac{2}{3} \times 15.25 = 10.17$.
4. Centroid of {D, E}: $(\frac{9+10}{2}, \frac{5+5}{2}) = (9.5, 5)$; cluster {F} has centroid (9, 8).
5. Squared distance between them: $(9 - 9.5)^2 + (8 - 5)^2 = 0.25 + 9 = 9.25$, so the cost is $\frac{2}{3} \times 9.25 = 6.17$, which is smaller than 10.17.

Answer: Ward's method adds F to {D, E} at a cost of 6.17, exactly as complete linkage did. Adding C to {A, B} would have cost 10.17, because C sits much farther from its neighbors than F does from D and E.

Table 19.3 sums up the four rules. The weakness in single linkage's last column has a name worth knowing.

Table 19.3 – The four linkage rules compared

Rule	What it measures	Clusters it tends to make	Main weakness
Single	Distance between the two closest members	Long, thin, curved; follows a chain of points	Chaining: one bridge of points glues two real groups together
Complete	Distance between the two farthest members	Compact, roughly equal in size	Splits a genuinely long group; sensitive to one far-out point
Average	Mean distance over all cross-cluster pairs	A middle ground between the two above	No single simple picture of what it optimizes
Ward	Extra inertia the merge would create	Compact and similarly sized, much like k-means	Only sensible for numeric features and straight-line distance

Chaining is already starting in Figure 19.3: C is 5.00 km from A, yet single linkage put them together at a height of only 2.83 km, because C happened to be near B. Figure 19.5 compares the finished trees for three rules on the same villages.

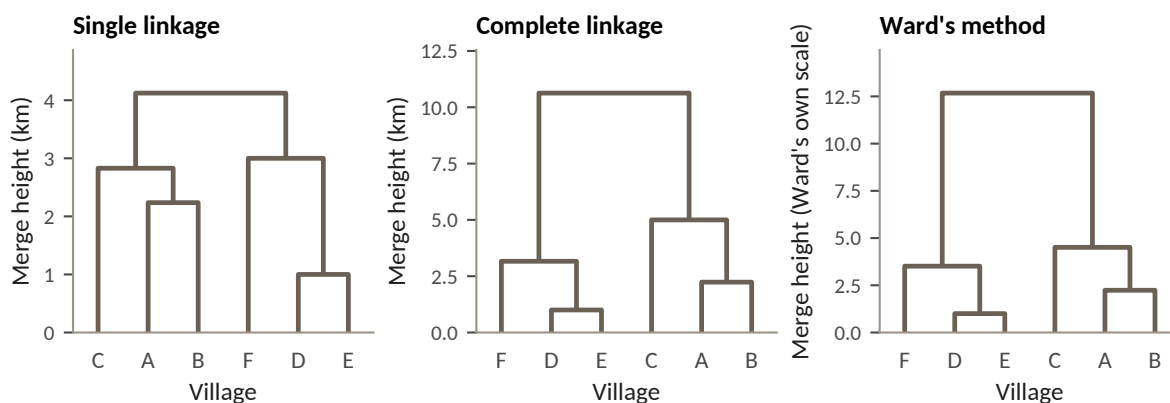


Figure 19.5 – The same six villages clustered three ways. Ward heights use a rescaled axis, so compare shapes, not numbers, across panels.

WHAT TO OBSERVE

The single-linkage tree is squashed: its last merge happens at only 4.12 km, barely above the 3.00 km merge before it, so nothing shouts “two clusters”. The complete and Ward trees have a tall empty stretch below the top — the signal that the last merge joined two groups that did not belong together.

COMMON MISTAKE

✗ Wrong: Reporting “the” hierarchical clustering of a dataset without saying which linkage rule produced it.

✓ Right: Always state the linkage rule with the result, because the four rules can give genuinely different clusters from identical data. Ward is the usual default; single linkage is the specialist choice for long, curved shapes.

19.4 Reading a Dendrogram and Cutting It

A dendrogram stores every merge in one picture, so reading one is the payoff of hierarchical clustering.

WORD TO KNOW

Merge height — The height of the horizontal bar that joins two clusters in a dendrogram. It is the linkage distance at which those clusters were merged, so a tall bar means the two clusters were far apart when they finally joined.

Three habits let you read almost any dendrogram. Read it from the bottom up, because the lowest bars are the merges the method was most confident about. Ignore the left-to-right order of the leaves, which software arranges only to keep branches from crossing. And look for tall empty stretches, because a merge that reached far up the axis joined two groups that were not alike. Figure 19.6 shows the complete-linkage tree.

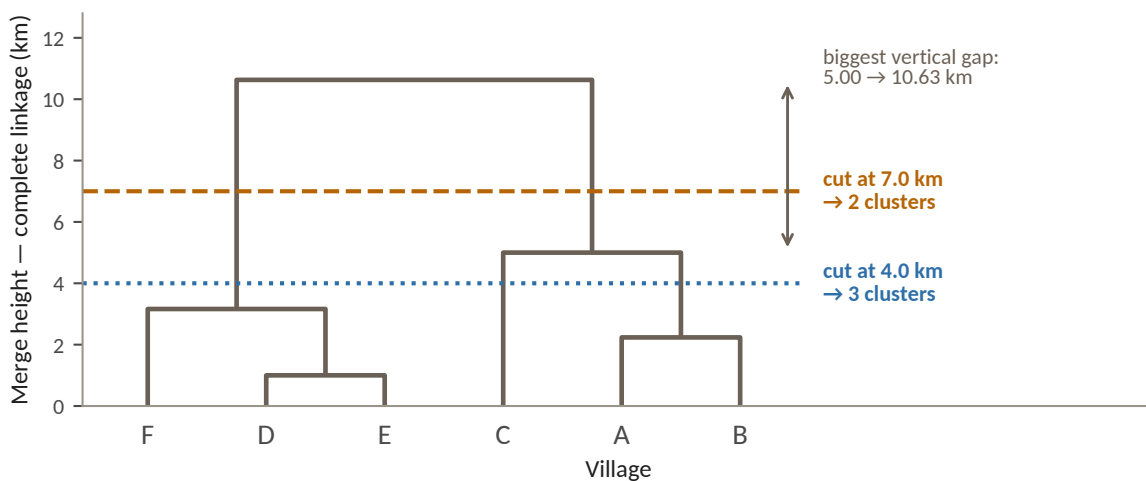


Figure 19.6 – The complete-linkage dendrogram with two candidate cuts. The double arrow marks the tallest empty stretch in the tree.

WHAT TO OBSERVE

The four lowest bars sit between 1.00 and 5.00 km, and then nothing happens until the final merge at 10.63 km. That 5.63 km of empty space is the clearest signal here: everything below joined easily, and the last merge did not.

To turn the tree into a list of clusters, draw a horizontal line across it and count the vertical lines the cut crosses. Each crossed line is one cluster, and its members are the leaves below it. Lower the line for more clusters, raise it for fewer.

WORKED EXAMPLE 19.6 — Cut the Dendrogram for Two Clusters and for Three

Using the complete-linkage merge heights 1.00, 2.24, 3.16, 5.00 and 10.63 km, report the clusters produced by a cut at 7.0 km and by a cut at 4.0 km, then say which cut the tree itself recommends.

1. A cut at height h keeps every merge below h and undoes every merge above it.
2. Cut at 7.0 km. The merges below the line are 1.00 (D with E), 2.24 (A with B), 3.16 (F joins {D, E}) and 5.00 (C joins {A, B}); the merge at 10.63 does not happen. The clusters are {A, B, C} and {D, E, F}: two clusters.

3. Cut at 4.0 km. Only 1.00, 2.24 and 3.16 are below the line, so the clusters are {A, B}, {C} and {D, E, F}: three clusters.
4. Now find the gaps between consecutive heights: $2.24 - 1.00 = 1.24$, $3.16 - 2.24 = 0.92$, $5.00 - 3.16 = 1.84$ and $10.63 - 5.00 = 5.63$.
5. The largest gap is 5.63 km, between the fourth and fifth merges, and cutting inside it gives two clusters.

Answer: A cut at 7.0 km gives {A, B, C} and {D, E, F}; a cut at 4.0 km gives {A, B}, {C} and {D, E, F}. The biggest-gap rule points to the two-cluster answer: three villages close to services, three far from them.

The biggest-gap rule is a habit, not a law — the dendrogram’s version of the elbow method from Chapter 18, to be weighed against what the clusters mean and what the organization can act on. If the health team can only run two kinds of visit, two clusters is the useful answer whatever the tree suggests; you can also compare silhouette scores across candidate cuts. And the tree proves nothing: clustering is unsupervised, so there is no label to check against, and a different rule may disagree — under single linkage, cutting for three clusters gives {A, B, C}, {D, E} and {F}, putting village C somewhere else entirely.

COMMON MISTAKE

X Wrong: Reading a dendrogram left to right and concluding that two leaves printed next to each other must be similar.

✓ Right: Judge similarity only by the height of the bar that first joins two leaves. In Figure 19.6, C is printed right next to E, but the bar joining them sits at 10.63 km — the top of the tree. They are the least similar pair of all.

19.5 Density-Based Clustering: DBSCAN

Both k-means and hierarchical clustering share two habits that can be wrong: they put every example into some cluster, however odd it is, and both prefer roughly round groups. A third family drops both habits by asking a different question — not “which center is this point near?” but “is this point in a crowd?”.

The best-known method of this kind is DBSCAN, short for “density-based spatial clustering of applications with noise”. It was published by Martin Ester, Hans-Peter Kriegel, Jörg Sander and Xiaowei Xu in 1996, has exactly two settings, and sorts every example into one of three kinds.

WORDS TO KNOW

Density-based clustering — An approach that defines a cluster as a region where points are packed closely together, separated from other such regions by emptier space. Points in no crowded region are labeled noise and left out of every cluster.

ε -neighborhood — The circle of radius ε (the Greek letter epsilon) drawn around one point. Every example inside that circle is a neighbor, so choosing ε means choosing how close counts as close.

minPts — The smallest number of points that must sit inside a point’s ε -neighborhood, counting the point itself, before the neighborhood counts as crowded.

Core point — A point with at least minPts points inside its ε -neighborhood, counting itself. Core points sit in the crowded middle of a cluster.

Border point — A point that is not core, but does fall inside the ε -neighborhood of some core point. Border points sit on the outer edge of a cluster and still belong to it.

Noise point — A point that is neither core nor within ε of any core point. It joins no cluster at all.

The core-point test is the only calculation DBSCAN really needs.

EQUATION: DBSCAN CORE-POINT CONDITION

$$|N_\varepsilon(\mathbf{x}_i)| \geq \text{minPts}, \quad \text{where } N_\varepsilon(\mathbf{x}_i) = \{j : D_{ij} \leq \varepsilon\}$$

In words: Count how many examples lie within a distance ε of this example, including the example itself. If the count reaches minPts, the example is a core point.

When it is used: Once per example, at the start of DBSCAN, to decide which examples are crowded enough to grow a cluster around.

Symbol	Meaning	Unit or type
$N_\varepsilon(\mathbf{x}_i)$	the neighbors of example i within distance ε	set of examples
$ \cdot $	how many members the set has	count
ε	the neighborhood radius chosen by you	same unit as the features
minPts	the smallest crowd size chosen by you	count

Books count the point itself differently. This book follows scikit-learn, where `min_samples` includes the point itself, so `min_samples=3` means “the point plus at least two neighbors”.

Once every point is labeled, the clusters build themselves: any two core points within ε of each other belong to the same cluster, each border point joins the cluster of a core point that reached it, and noise points are left out. Nobody chose the number of clusters — it fell out of the data.

The planning team has also mapped five small hamlets by their position, in kilometers east and north of the district center, and wants to know which are close enough to share one mobile visit. They set $\varepsilon = 1.5$ km, the distance a health worker can comfortably walk between stops, and $\text{minPts} = 3$.

WORKED EXAMPLE 19.7 — Label Five Hamlets as Core, Border or Noise

The hamlets sit at $P = (1, 1)$, $Q = (2, 1)$, $R = (2, 2)$, $S = (3, 3)$ and $T = (6, 5)$, in kilometers. Using $\varepsilon = 1.5$ km and $\text{minPts} = 3$, label every hamlet and report the clusters.

1. Compute the short distances: $d(P, Q) = \sqrt{1+0} = 1.00$; $d(P, R) = \sqrt{1+1} = 1.41$; $d(Q, R) = \sqrt{0+1} = 1.00$; $d(R, S) = \sqrt{1+1} = 1.41$; $d(Q, S) = \sqrt{1+4} = 2.24$; $d(P, S) = \sqrt{4+4} = 2.83$. T's nearest neighbor is S, at $d(S, T) = \sqrt{9+4} = 3.61$ km.
2. P's neighborhood: P itself, Q at 1.00 and R at 1.41 — 3 points, so **P is core**.
3. Q's neighborhood: Q itself, P at 1.00 and R at 1.00, since S at 2.24 is too far — 3 points, so **Q is core**.
4. R's neighborhood: R itself, P at 1.41, Q at 1.00 and S at 1.41 — 4 points, so **R is core**.
5. S's neighborhood: S itself and R at 1.41, since Q at 2.24 and P at 2.83 are outside — only 2 points, so S is not core. But S lies within 1.5 km of the core point R, so **S is a border point**.
6. T's neighborhood: T only, since its nearest neighbor is 3.61 km away, and no core point reaches it, so **T is noise**.
7. Build the clusters: P, Q and R are core points within ε of each other, so they form one cluster, and border point S joins it through R. T joins nothing.

Answer: P, Q and R are core, S is border and T is noise. DBSCAN finds one cluster, $\{P, Q, R, S\}$, and reports T as belonging to no cluster — exactly the answer the team needs: one shared visit for four hamlets, and a separate trip for T.

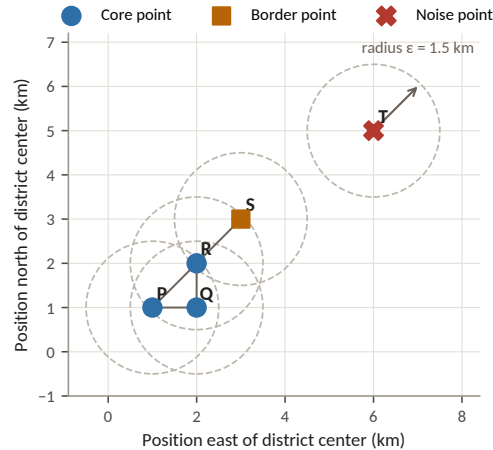


Figure 19.7 – Five hamlets, each with a circle of radius $\varepsilon = 1.5$ km. Gray lines join pairs within ε .

WHAT TO OBSERVE

Count the points inside each circle, remembering the center point itself. P, Q and R each reach three or more, so they are core. S's circle catches only R, so S cannot grow a cluster — but R's circle reaches S, which is why S still belongs. Nothing but T sits inside T's circle.

Both settings change the answer. Widen the radius to $\varepsilon = 2.5$ km, keeping $\text{minPts} = 3$, and S's neighborhood grows to three points — S itself, R at 1.41 and Q at 2.24 — so S is promoted from border point to core point. T's nearest neighbor is still 3.61 km away, so T stays noise and the cluster membership is unchanged. Shrink ε far enough and every hamlet becomes noise; widen it far enough and everything, T included, becomes one cluster.

IMPORTANT

Being labeled “noise” says nothing bad about an example. Here it means an isolated hamlet, which is very likely the one that needs the most attention, not the least. Whenever a clustering is used to plan services for people, check the noise list first, and check it by hand.

Choosing ε is the main practical difficulty, and there is a standard trick. For every point, measure the distance to its k th nearest neighbor, with k set to minPts , then sort those distances and plot them. Points inside a crowd have small k th-neighbor distances and noise points have large ones, so the curve stays low and flat and then bends sharply upward. The height of that bend is a good starting value for ε .

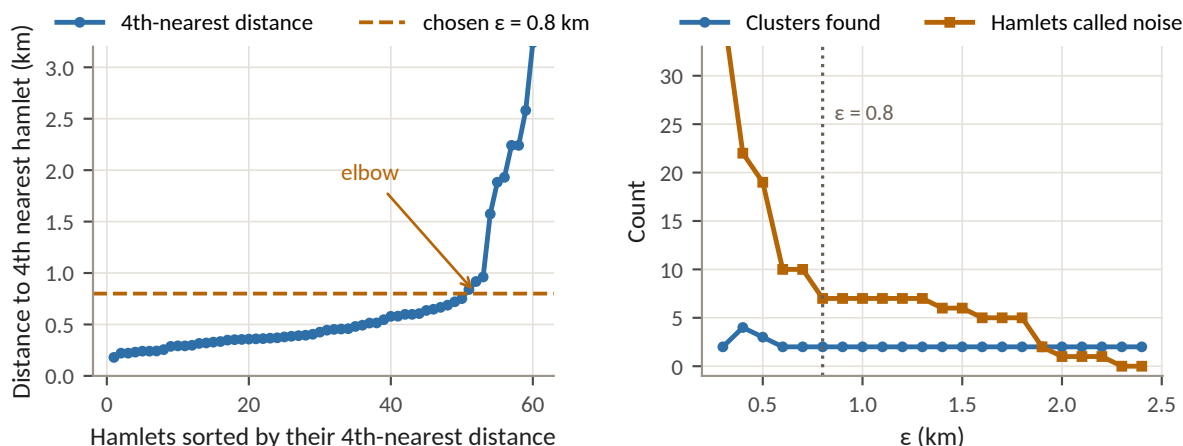


Figure 19.8 – Left: each of 60 hamlets sorted by its fourth-nearest-neighbor distance, with the elbow marked. Right: clusters and noise points as ϵ varies.

WHAT TO OBSERVE

About 50 of the 60 hamlets have a fourth-neighbor distance below 0.8 km, and the curve then shoots upward for the last ten. On the right, $\epsilon = 0.8$ km sits on a wide flat shelf where the answer is a steady two clusters. Too small an ϵ shatters the data and calls most points noise; too large an ϵ swallows everything into one cluster.

A common starting point is $\text{minPts} = 2p$, twice the number of features, with a minimum of 3 or 4; read ϵ off the k-distance plot, then adjust both while looking at the result. Too small a minPts gives thin, straggly clusters that swallow the noise; too large a minPts dismisses real but small clusters as noise. DBSCAN's one real weakness follows from having a single ϵ for the whole dataset: if one part of the data is dense and another sparse, no radius suits both. HDBSCAN, published by Ricardo Campello, Davoud Moulavi and Jörg Sander in 2013, extends DBSCAN by trying many radii and keeping the clusters that survive longest.

COMMON MISTAKE

✗ Wrong: Treating the noise points DBSCAN returns as errors, and quietly forcing each one into its nearest cluster.

✓ Right: Keep them separate and look at them. In many projects the noise list is the most valuable output of all — the isolated hamlet, the unusual machine, the suspicious transaction. Chapter 21 is entirely about points like these.

19.6 Comparing k-Means, Hierarchical Clustering and DBSCAN

You now know three clustering methods. Figure 19.9 shows the clearest difference between them, on a famous test dataset of two interlocking crescents.

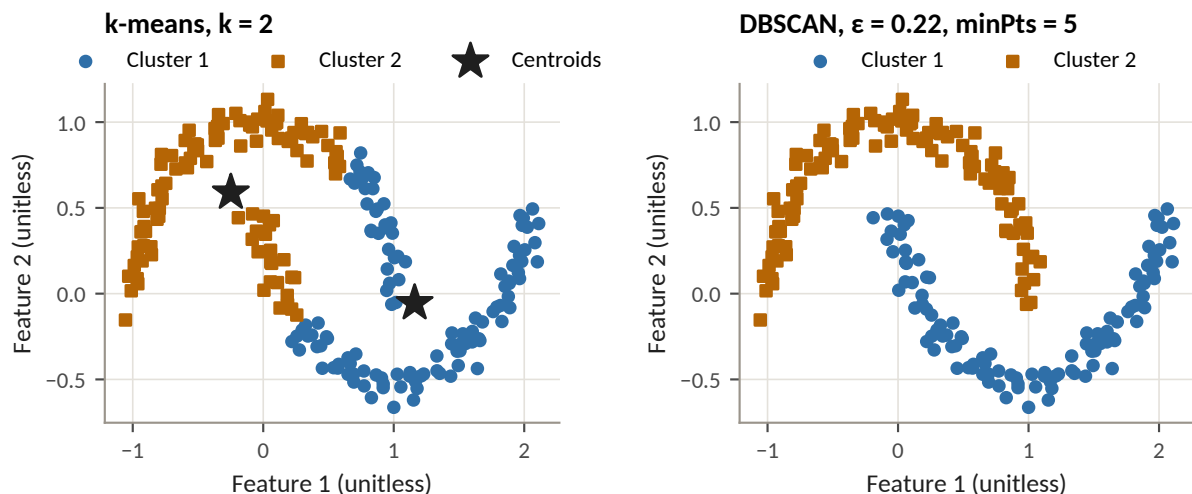


Figure 19.9 – The same 220 points clustered two ways. *k*-means splits the crescents down the middle; DBSCAN follows their shape.

WHAT TO OBSERVE

k-means fails here for a simple reason: the set of points nearest a given centroid is always a straight-sided region, and a crescent is not such a region, so no pair of centroids can describe these shapes. DBSCAN never computes a center — it follows the crowd from neighbor to neighbor — so a curved cluster costs it nothing.

Hierarchical clustering sits between the two: with Ward or complete linkage it behaves much like *k*-means, and with single linkage it can follow a crescent. Table 19.4 collects the practical differences between all three. Note especially that *k*-means can land on different answers on different runs, as Chapter 18 showed, while hierarchical clustering has no random step — but must build the whole distance matrix, the subject of the last lesson.

Table 19.4 – Choosing between the three clustering methods

Question	<i>k</i> -means (Chapter 18)	Hierarchical	DBSCAN
Do you choose the number of clusters?	Yes, before it runs	No — you cut the finished tree	No — it follows from ϵ and minPts
Cluster shapes it finds	Round, similar sizes	Depends on the linkage rule	Any shape at all
Can a point belong to no cluster?	No	No	Yes, it is called noise
Main settings to choose	k	Distance, linkage, cut height	ϵ , minPts
Repeatable, and how large?	Depends on the starting centroids; millions of rows	Always the same tree; a few thousand rows	Yes apart from border points; tens of thousands of rows
Gives a picture of structure?	Only cluster profiles	Yes, the dendrogram	A cluster map plus a noise list

Figure 19.10 puts all three methods on three different datasets at once.

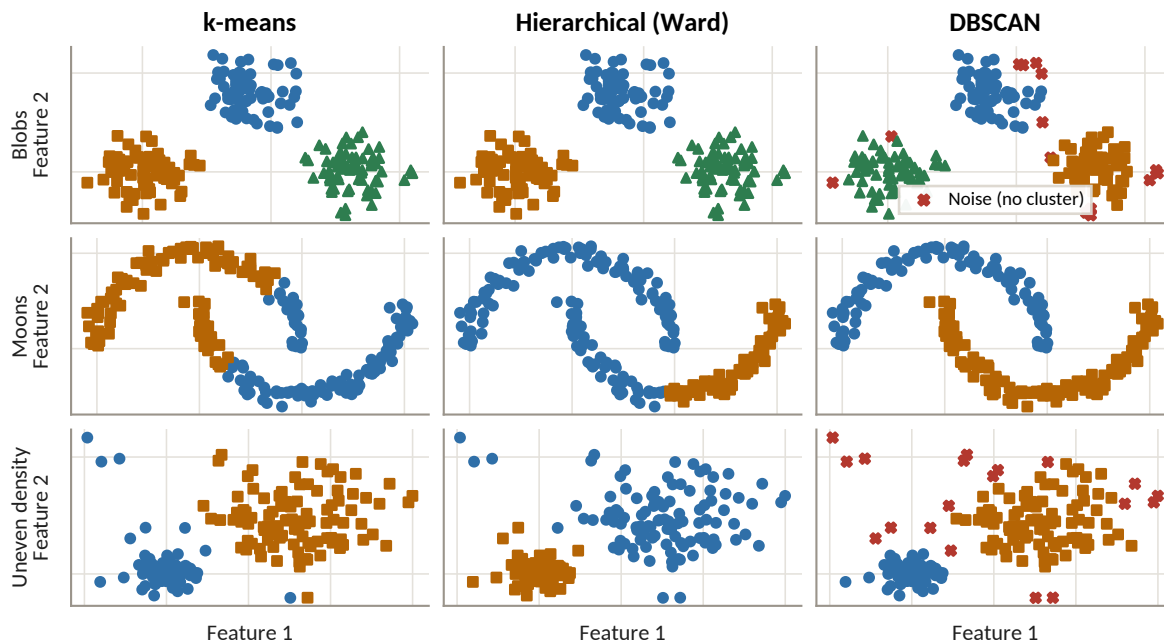


Figure 19.10 – Three methods (columns) on three datasets (rows), whose two features share one arbitrary scale. Colors mark the clusters found; red crosses are noise.

WHAT TO OBSERVE

On round blobs all three agree, apart from a few edge points DBSCAN sets aside as noise. On the crescents only DBSCAN gets the shape right. On the third dataset only DBSCAN can leave the stray points out; k-means and Ward must place every stray inside a cluster, which drags their boundaries around.

None of this makes DBSCAN simply better. On large, roughly round, well-separated groups k-means is faster, simpler to explain and easier to update with new data, while DBSCAN wins on odd shapes and on data with genuine stray points but struggles when regions of the data have very different densities. Match the method to the data, not the other way around.

19.7 Distance Matrices, Scaling and Cost

Everything in this chapter rests on the distance matrix, so it is worth knowing how big that table gets and what can spoil it.

The first issue is scale — the same trap Chapter 11 sprang for k-Nearest Neighbors and Chapter 18 for k-means. A distance adds up squared differences from every feature, so a feature measured in much larger numbers decides almost every merge on its own. Cluster shoppers on visits per month (1 to 10) and monthly spend (250 to 600 thousand IQD), and the tree will describe spending alone. The cure is the standardization of Chapter 4: subtract each feature’s mean and divide by its standard deviation, for the whole table, before computing any distance — half-scaled data is no safer than unscaled data. The only exception is a table whose features are already in the same unit, as the villages’ two distances in kilometers happen to be.

The second issue is cost. Filling in the matrix means computing every pair once, and each of the n examples pairs with the other $n - 1$, which counts every pair twice. The number of distances is therefore $n(n - 1)/2$, a count that grows much faster than n itself: doubling the number of examples multiplies the work by roughly four.

WORKED EXAMPLE 19.8 — Will the Distance Matrix Fit in Memory?

The NGO's full register lists 10,000 villages and hamlets, and each distance is stored as an 8-byte number. How many distances are there, and how much memory does the matrix need?

1. Use $n(n-1) \div 2$ with $n = 10,000$. The top is $10,000 \times 9,999 = 99,990,000$.
2. Divide by two: $99,990,000 \div 2 = 49,995,000$ pairs, nearly 50 million distances.
3. Multiply by 8 bytes each: $49,995,000 \times 8 = 399,960,000$ bytes.
4. Divide by one million to get megabytes: about 400 megabytes.
5. Compare with a 500-village pilot: $500 \times 499 \div 2 = 124,750$ pairs, about 1 megabyte.

Answer: About 50 million distances, needing roughly 400 megabytes. That fits on a modern laptop but is uncomfortable, and 100,000 villages would need about 40 gigabytes, which will not fit at all. The pilot costs four hundred times less memory for a register only twenty times smaller.

Table 19.5 – How the distance matrix grows with the number of examples

Examples n	Pairwise distances	Memory at 8 bytes each	Practical verdict
6	15	120 bytes	Do it by hand
500	124,750	about 1 MB	Instant
10,000	49,995,000	about 400 MB	Slow but possible
100,000	4,999,950,000	about 40 GB	Use k-means or DBSCAN instead

There are two ways around the limit. Cluster a random sample of a few thousand rows, read the dendrogram, decide how many clusters you want, then use another method to assign the rest. Or switch method: DBSCAN never needs the full matrix, because it only looks inside a radius of ε .

Finally, the matrix need not hold Euclidean distances. Any symmetric measure of dissimilarity that is zero between an example and itself will do — Manhattan distance from Chapter 11 is a common alternative, and text and category data have their own measures. Single, complete and average linkage work with any such matrix; Ward's method, defined through centroids and squared distances, should be used with Euclidean distance only.

COMMON MISTAKES AND MISCONCEPTIONS

- Thinking hierarchical clustering asks for no choices, when it still asks for a distance measure, a linkage rule and a cut height.
- Reporting a hierarchical clustering without naming the linkage rule, even though the four rules can give different answers on the same data.
- Reading a dendrogram left to right and assuming neighboring leaves are similar, instead of judging by the height of the bar that joins them, or expecting single linkage to give compact clusters when its whole character is to follow chains of nearby points.
- Treating DBSCAN's noise points as mistakes and forcing them into the nearest cluster, forgetting that minPts counts the point itself in scikit-learn, or choosing a single ε for data whose regions have very different densities.
- Building a distance matrix from raw features on very different scales, so the largest-numbered feature decides every merge.
- Running hierarchical clustering on a large table without checking that $n(n-1)/2$ distances will fit in memory.

Practical Activities

ACTIVITY (PAPER) — Build a Dendrogram by Hand for Five Points

Five villages are recorded as $P1 = (1, 1)$, $P2 = (2, 2)$, $P3 = (5, 4)$, $P4 = (7, 5)$ and $P5 = (3, 9)$, in kilometers.

1. Plot the five points on graph paper, one square per kilometer, and label them.
2. Compute all ten pairwise distances into a 5×5 distance matrix, rounded to two decimals. Check two of them: $d(P1, P2) = 1.41$ and $d(P3, P4) = 2.24$.
3. Run agglomerative clustering with **single linkage**. For each of the four steps, write down which two clusters merged and at what height.
4. Draw the dendrogram (five labels along the bottom, a vertical scale from 0 to 9 km, each merge a horizontal bar at its height), then draw a horizontal cut that gives two clusters and list the members of each.
5. Repeat step 3 with **complete linkage**. Which step is the first where the rules disagree, and do the two-cluster answers end up the same? In two or three sentences, say which answer you would report to the planning team, and why.

You need: Graph paper, pencil, ruler and a calculator with a square-root key

ACTIVITY (PYTHON) — Hierarchical Clustering and DBSCAN with SciPy and scikit-learn

This program reproduces the distance matrix of Figure 19.2, the merge heights of Table 19.2 for all four linkage rules, the labels of Worked Example 19.7, and the crescent comparison in Figure 19.9.

Python • SciPy • scikit-learn

```
import numpy as np
from scipy.spatial.distance import pdist, squareform
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
from sklearn.cluster import AgglomerativeClustering, DBSCAN, KMeans
from sklearn.datasets import make_moons

X = np.array([[1, 1], [3, 2], [5, 4], [9, 5], [10, 5], [9, 8]]) # villages A-F (km)
print("distance matrix:\n", np.round(squareform(pdist(X)), 2))
for rule in ["single", "complete", "average", "ward"]:
    Z = linkage(X, method=rule)
    print(rule, "heights:", np.round(Z[:, 2], 2), "2 clusters:", fcluster(Z, 2, "maxclust"))
print("sklearn Ward:", AgglomerativeClustering(n_clusters=2).fit_predict(X))
dendrogram(linkage(X, "complete"), labels=list("ABCDEF")) # draw the tree

P = np.array([[1, 1], [2, 1], [2, 2], [3, 3], [6, 5]]) # hamlets P-T (km)
db = DBSCAN(eps=1.5, min_samples=3).fit(P)
core = np.isin(np.arange(len(P)), db.core_sample_indices_)
print("DBSCAN labels:", db.labels_, " core point?", core)

Xm, _ = make_moons(n_samples=220, noise=0.06, random_state=0) # two crescents
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(Xm)
mo = DBSCAN(eps=0.22, min_samples=5).fit_predict(Xm)
print("moons sizes, k-means:", [int((km == g).sum()) for g in sorted(set(km))],
      " DBSCAN:", [int((mo == g).sum()) for g in sorted(set(mo))])
```

Output

```
distance matrix:
[[ 0.    2.24  5.    8.94  9.85 10.63]
 [ 2.24  0.    2.83  6.71  7.62  8.49]
 [ 5.    2.83  0.    4.12  5.1   5.66]
 [ 8.94  6.71  4.12  0.    1.    3.   ]
 [ 9.85  7.62  5.1   1.    0.    3.16]
 [10.63  8.49  5.66  3.    3.16  0.   ]]
single heights: [1.    2.24  2.83  3.    4.12] 2 clusters: [1 1 1 2 2 2]
complete heights: [ 1.    2.24  3.16  5.    10.63] 2 clusters: [2 2 2 1 1 1]
average heights: [1.    2.24  3.08  3.91  7.46] 2 clusters: [2 2 2 1 1 1]
ward heights: [ 1.    2.24  3.51  4.51 12.68] 2 clusters: [2 2 2 1 1 1]
sklearn Ward: [0 0 0 1 1 1]
DBSCAN labels: [ 0  0  0  0 -1] core point? [ True  True  True False False]
moons sizes, k-means: [117, 103] DBSCAN: [110, 110]
```

The merge heights match Table 19.2: single linkage tops out at 4.12 km and complete linkage at 10.63 km, from identical data. Cluster numbers are arbitrary names, so [1 1 1 2 2 2] and [2 2 2 1 1 1] describe the same split. The label –1 marks noise, so hamlet T is noise, and the core flags confirm P, Q and R as core with S only a border point. On the crescents, k-means splits 117 against 103 while DBSCAN returns two clusters of exactly 110. Change eps to 0.1 and rerun to watch the crescents shatter.

You need: A computer with Python, NumPy, SciPy, scikit-learn and matplotlib (see Appendix B)

CASE STUDY — Planning Mobile Services for Villages in the Kurdistan Region

A small non-governmental organization runs mobile health and education visits across a rural district of the Kurdistan Region. Until this year the schedule was built by hand from a paper list of 420 villages and hamlets. The office wanted a small number of village *types* instead, so that each type could get its own kind of visit, vehicle and checklist. Nobody knew how many types there should be: two seemed too few, twenty would be useless, and no field officer wanted to defend a particular number in a meeting. This is exactly the situation hierarchical clustering was built for. (All figures here are illustrative example numbers.)

The team used four features from a routine survey: distance to the nearest clinic and to the nearest secondary school, both in kilometers, number of households, and hours of mains electricity per day. Following Lesson 19.7 they standardized all four first, because household counts run into the hundreds while the distances run from 1 to 30; without that step the household count would have decided every merge. With 420 villages the distance matrix held $420 \times 419 \div 2 = 87,990$ distances, a fraction of a megabyte, so the clustering ran in under a second.

They chose Ward's method, wanting compact groups of similar size that a field officer could plan a route around. The dendrogram showed a small vertical gap below the two-cluster level and a much larger one below the four-cluster level, so they cut for four clusters and checked that the groups made sense to the people who drive the roads. Table 19.6 shows the result, with the working names the field officers gave them.

Table 19.6 – The four village types the NGO found, described by their cluster centers (illustrative)

Working name	Villages	Km to clinic	Km to school	Planned visit
Well served	168	2	2	Twice a year, health only
School far	94	3	14	Quarterly, education team
Clinic far	111	16	4	Quarterly, health team
Remote, both far	47	21	19	Monthly, full team

The team then ran DBSCAN on the same standardized table as a safety check. With ϵ read from a k-distance plot and $\text{minPts} = 8$, it marked 23 villages as noise: a few had a clinic in the village itself but no school within 30 km, and a few were tiny hamlets of three or four households. Ward's method had folded every one of them into whichever group was least wrong, so that noise list became the most useful page of the analysis. The office adopted the four types for planning, kept the noise list as a standing exception list reviewed each quarter, and agreed to rerun the analysis yearly, since a new clinic or road can move a village from one type to another overnight.

RESPONSIBLE USE

A cluster label describes a pattern in a few measured columns, not the worth or the needs of the people who live there. Two villages in the same cluster can differ in ways nobody measured, and the villages that fit no cluster are often the ones in most difficulty. Use clusters to organize work and to make plans discussable, never to rank communities or to remove anyone's case from human review. Chapter 24 returns to this in depth.

CHAPTER SUMMARY

- Hierarchical clustering builds a whole tree of groupings at once, so the number of clusters is chosen after looking at the result instead of before.
- Agglomerative clustering starts with every example alone, repeatedly merges the two closest clusters, and needs exactly $n - 1$ merges for n examples.
- The first step is always the distance matrix: a symmetric table with a zero diagonal, holding $n(n - 1)/2$ distinct distances.
- A linkage rule says how far apart two clusters are: single linkage uses the closest pair, complete linkage the farthest pair, average linkage the mean of all pairs, and Ward's method the extra inertia a merge would create.
- Different linkage rules can produce genuinely different clusters from identical data, so the rule must always be reported with the result; single linkage in particular can chain, because a thin line of points between two real groups is enough to glue them together.
- A dendrogram shows every merge as a horizontal bar whose height is the linkage distance; cutting at a chosen height gives a clustering, and a tall empty stretch hints at where to cut.
- DBSCAN defines clusters as crowded regions using two settings, the radius ϵ and the crowd size minPts , and labels every point core, border or noise; noise points join no cluster and are often the most interesting output. It finds clusters of any shape, though one single ϵ cannot suit regions of very different density.

- Every method here compares distances, so features on very different scales must be standardized first; and hierarchical clustering must store the whole distance matrix, which limits it to a few thousand examples, while k-means and DBSCAN scale much further.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain the difference between agglomerative and divisive hierarchical clustering, and state which one this chapter used.

Problem 2 [Basic] A dataset has 8 examples. (a) How many distinct pairwise distances does its distance matrix contain? (b) How many merges will agglomerative clustering perform before everything is in one cluster? (c) Name DBSCAN's two settings.

Problem 3 [Basic] Define single linkage and complete linkage in one sentence each, using the words "closest" and "farthest", then say what the height of a horizontal bar in a dendrogram means.

Problem 4 [Basic] A point's ε -neighborhood contains 5 points, counting itself, and $\text{minPts} = 4$. Is it a core point? Explain in one sentence.

Problem 5 [Standard] Cluster $P = \{(1, 1), (2, 3)\}$ and cluster $Q = \{(6, 2), (7, 5)\}$. The four cross-cluster distances are 5.10, 7.21, 4.12 and 5.39. Find the single-linkage, complete-linkage and average-linkage distances between P and Q .

Problem 6 [Standard] Using the distance matrix in Figure 19.2, find the single-linkage and the complete-linkage distance between $\{A, B, C\}$ and $\{D, E, F\}$. Which pair of villages decides each answer?

Problem 7 [Standard] A dendrogram of six examples has merge heights 0.8, 1.1, 1.5, 4.9 and 5.4. (a) List the gaps between consecutive heights. (b) Which cut does the biggest-gap rule suggest, and how many clusters does it give?

Problem 8 [Standard] Cluster P holds 3 points with centroid $(2, 2)$ and cluster Q holds 1 point at $(5, 6)$. Use Ward's formula to find the extra inertia that merging them would create, showing every step.

Problem 9 [Standard] Four points sit at $(0, 0)$, $(1, 0)$, $(2, 0)$ and $(9, 9)$. With $\varepsilon = 1.5$ and $\text{minPts} = 3$, label each as core, border or noise, showing the neighbor count you used.

Problem 10 [Standard] A team clusters 3,000 customers with Ward linkage on two features: number of orders (0 to 40) and total spend in dollars (0 to 9,000). (a) Explain what goes wrong if they skip standardization. (b) State the memory the distance matrix needs, at 8 bytes per distance.

Problem 11 [Challenge] Five weather stations sit along a straight road at $x = 0, 1, 4, 10, 12$ kilometers, all with $y = 0$. (a) Write the 5×5 distance matrix. (b) Run single linkage and list the four merges with their heights. (c) Run complete linkage and do the same. (d) Explain why the two rules give the same clusters here but different heights.

Problem 12 [Challenge] A colleague runs DBSCAN on a map of shops and gets one giant cluster containing 97% of the shops, with almost nothing labeled noise. (a) Which setting is most likely wrong, and in which direction? (b) Describe how you would use a k-distance plot to fix it. (c) Explain why simply increasing minPts might not be enough.

Problem 13 [Challenge] Six sensors sit at $(1, 1)$, $(2, 2)$, $(5, 2)$, $(6, 4)$, $(9, 3)$ and $(10, 3)$: a tight pair on the left, a pair in the middle and a tight pair on the right. (a) Predict, without computing, whether single linkage will attach the middle pair to the left pair or to the right pair, and explain your reasoning. (b) Compute the first four single-linkage merges and check your prediction. (c) Explain how this illustrates chaining.

Problem 14 [Challenge] A city has 60,000 addresses and wants to group them for waste-collection routes. (a) Explain, with a number, why plain hierarchical clustering is a poor choice. (b) Give one way to still use a dendrogram here. (c) Argue for or against DBSCAN as the main method, naming one strength and one risk.

Principal Component Analysis and Dimensionality Reduction

Fewer numbers that keep most of the information

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a table with many features becomes hard to look at, and what dimensionality reduction is for.
- Center a dataset by subtracting each feature's mean, and say what the centered numbers mean.
- Project a centered point onto a direction and compute the variance of the projections by hand.
- Find the first principal component by testing candidate directions, and build the covariance matrix that decides the winner.
- Explain why every later principal component sits at a right angle to the ones before it.
- Compute explained variance ratios, read a scree plot and a loadings table, and choose how many components to keep.
- Explain why features must be standardized before PCA, and name two projections built for other jobs: LDA and t-SNE.

Introduction

A maintenance engineer at a factory watches twelve machines. Each machine reports eight sensor readings every minute: temperature, vibration, motor current, humidity, rotation speed, feed pressure, fan speed and belt wear. That is ninety-six numbers a minute, scrolling past on one screen. Nobody can watch ninety-six numbers, so in practice nobody watches any of them, and the team learns that a machine was struggling only after it has stopped.

Now imagine a different screen. Each machine is a single dot on one chart, drawn from two numbers instead of eight. Healthy machines sit together in a familiar patch, and a machine that starts to wear out drifts away from it in plain sight, days before any reading crosses an alarm limit.

This chapter builds the method behind that second screen: *Principal Component Analysis*, or PCA. Like the clustering of Chapter 18 and Chapter 19 it is unsupervised, using only the features and never a label. PCA does not pick columns to keep and columns to drop; it builds new columns, each a blend of the originals, ranked by how much of the data's spread each one carries. You will do the whole calculation by hand on five students and two exam scores, then let Python repeat it.

20.1 Too Many Features, Too Little Insight

Every chapter so far has used a friendly number of features: a melon's weight and sound, an apartment's size and city. Real datasets are rarely so tidy — a hospital's patient file might hold forty lab measurements, a factory's sensor log hundreds of readings a second — and every extra feature makes the data harder to look at. Two features fit on an ordinary flat graph and three can still be drawn, awkwardly. Six features already give 15 different pairs you could plot against each other; eight give 28; twenty give 190. Somewhere past three or four features, “just plot the data and look at it” quietly stops being possible.

There is a quieter problem beside it. Many real features move together: a student strong in Math is usually strong in Physics, and a machine's temperature rises with its motor current, because both answer to how hard it is working. Chapter 3 gave that pattern a number, the correlation coefficient. When two features are strongly correlated, much of the second one's information already sits inside the first, so keeping both in full doubles the storing and plotting without teaching much that is new.

WORD TO KNOW

Dimensionality reduction — Cutting down the number of features used to describe each example, while keeping as much of the data's real information as possible. Each original feature is often called a *dimension*, so reducing their number is called reducing the dimensionality.

Picture it as packing for a long trip with one small suitcase: you cannot bring the whole wardrobe, so you keep a few versatile items that between them cover the widest range of situations. For the factory's eight sensors, PCA builds two new columns — blends of all eight, not a choice of two of them — that between them still carry 93% of the original spread. Instead of the 28 scatter plots of Table 20.1, the team has one chart to watch.

Table 20.1 – How many scatter plots of one pair of features a dataset offers

Number of features	2	3	4	6	8	12	20
Pairs you could plot	1	3	6	15	28	66	190

COMMON MISTAKE

✗ Wrong: Assuming the only way to shrink a dataset's features is to delete the columns that look least important and keep the rest untouched.

✓ Right: Dimensionality reduction builds brand-new features, each a blend of the original ones, so information from every column survives. Deleting columns is a different job, called feature selection, and Chapter 22 covers it.

20.2 Centering, Variance and the Direction of Greatest Spread

To see how PCA works, this lesson uses a dataset small enough to check with a pencil: two features and five students from a university department in Erbil, each sitting a Math and a Physics exam graded from 0 to 100.

WORDS TO KNOW

Principal Component Analysis (PCA) — A dimensionality-reduction method that finds new directions through a dataset, ranked by how much of the data's spread each captures, so that a few of them can stand in for many original features. It was first described by Karl Pearson in 1901 and developed independently, under the name it still carries, by Harold Hotelling in 1933.

Centering — Subtracting a feature's mean from every value of that feature, so that the column's new mean is exactly 0. After centering, each number says how far above or below average an example sits.

Table 20.2 lists the five students and Figure 20.1 draws them. Plotted with Math along the bottom and Physics up the side they do not scatter randomly: a student above average in Math is also above average in Physics, so the points stretch along a diagonal. Before PCA can search that stretch it needs to know where the middle of the data sits, and the first step is one you already know from Chapter 4.

EQUATION: CENTERING A FEATURE

$$x' = x - \mu$$

In words: Take one value, subtract the average of its own column, and keep the difference.

When it is used: As the very first step of PCA. Every feature is centered separately, using its own mean.

Symbol	Meaning	Unit or type
x	one original value of the feature	unit of the feature
μ	the mean of that feature's column	unit of the feature
x'	the centered value	unit of the feature

Centering never changes the shape of the cloud; it slides it so that its middle sits at the origin. It is the top half of the z-score of Chapter 4, without the division by σ .

WORKED EXAMPLE 20.1 — Center Five Students' Math and Physics Scores

The five students of Table 20.2 scored 73, 67, 76, 64 and 70 in Math, and 69, 61, 73, 57 and 65 in Physics. Center both features.

1. Add the Math scores: $73 + 67 + 76 + 64 + 70 = 350$. Divide by 5: the mean is $350 \div 5 = 70$.
2. Add the Physics scores: $69 + 61 + 73 + 57 + 65 = 325$. Divide by 5: the mean is $325 \div 5 = 65$.
3. Subtract 70 from each Math score: $73 - 70 = 3$, $67 - 70 = -3$, $76 - 70 = 6$, $64 - 70 = -6$, $70 - 70 = 0$.
4. Subtract 65 from each Physics score: $69 - 65 = 4$, $61 - 65 = -4$, $73 - 65 = 8$, $57 - 65 = -8$, $65 - 65 = 0$.
5. Check: each centered column must add to zero. Math gives $3 - 3 + 6 - 6 + 0 = 0$; Physics gives $4 - 4 + 8 - 8 + 0 = 0$.

Answer: The centered scores, written as (Math deviation, Physics deviation), are (3, 4), (−3, −4), (6, 8), (−6, −8) and (0, 0). Every number now says how far a student sits from the class average, and the five points sit evenly around (0, 0).

Table 20.2 – Five students' exam scores, raw and centered (illustrative example numbers)

Student	Math (0–100)	Physics (0–100)	Math deviation	Physics deviation
S1	73	69	+3	+4
S2	67	61	−3	−4
S3	76	73	+6	+8
S4	64	57	−6	−8
S5	70	65	0	0
Mean	70	65	0	0

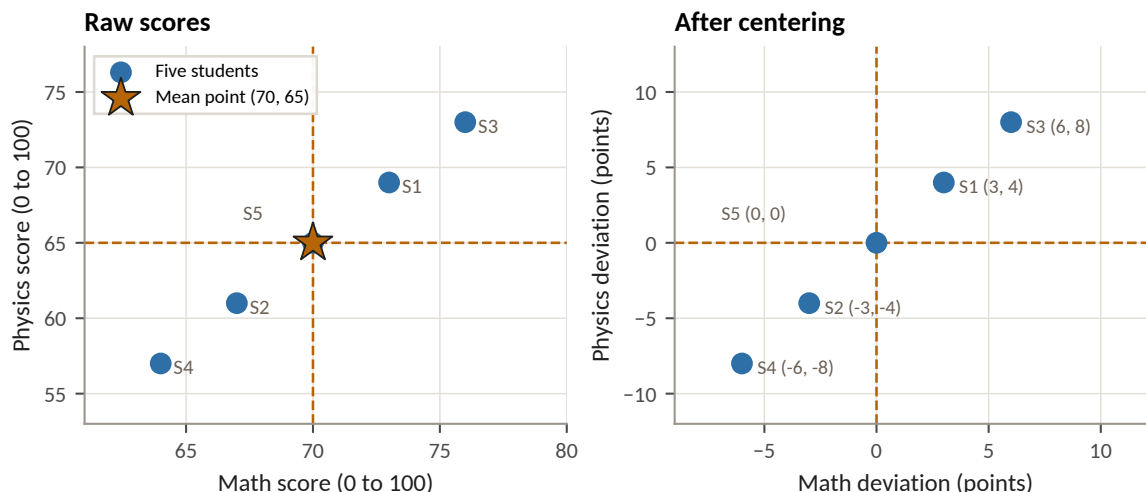


Figure 20.1 – The same five students twice. Left: raw scores, with the mean point marked by a star. Right: the centered scores, with the mean now at (0, 0).

WHAT TO OBSERVE

Centering slides the cloud but never squeezes or turns it. Student S5, exactly average in both subjects, lands on (0, 0); S3 and S4 sit at opposite ends of the diagonal PCA is about to measure.

Both subjects here use the same 0-to-100 scale, so centering alone is fair; when features arrive on very different scales, each must also be divided by its own standard deviation, which is the standardization of Chapter 4. Lesson 20.7 shows what goes wrong when that step is skipped.

With the data centered, PCA's real question can be asked precisely: among all the directions you could draw through this cloud, which one do the points spread out along the most? Answering it needs variance, from Chapter 3, and one new tool.

20.3 Projecting Points onto a Direction

Stand at the mean point and look along a chosen direction. Every student is then ahead of you or behind you along that line, and the single number saying how far is called the projection.

WORDS TO KNOW

Unit direction — A list of numbers describing a direction, chosen so that their squares add up to exactly 1. For example (0.6, 0.8) is a unit direction because $0.6^2 + 0.8^2 = 0.36 + 0.64 = 1$. Keeping every direction the same length makes comparing spreads a fair comparison.

Projection — One number saying how far along a chosen direction a centered point falls. It is found by multiplying matching numbers and adding the results.

EQUATION: PROJECTION OF A CENTERED POINT ONTO A UNIT DIRECTION

$$p = \sum_{j=1}^d u_j x'_j = u_1 x'_1 + u_2 x'_2 + \cdots + u_d x'_d$$

In words: Multiply each of the point's centered feature values by the matching number of the direction, then add all the products together.

When it is used: Whenever one number must stand in for a whole row: to measure spread along a candidate direction, and to write each example's new PCA coordinates.

Symbol	Meaning	Unit or type
p	the projection: position along the direction	unit of the features
x'_j	the point's centered value for feature j	unit of feature j
u_j	the direction's number for feature j	no unit
d	the number of features	count

With two features this is simply $p = u_1x' + u_2y'$. The direction must be a unit direction and the point must already be centered. A candidate such as $(1, 1)$ must first be rescaled to $(0.7071, 0.7071)$, because a longer direction stretches every projection and would win any comparison unfairly.

The spread along a direction is then just the variance of those projected numbers. Because the data was centered first, the projections average out to zero, so that variance is simply the average of their squares.

EQUATION: VARIANCE ALONG A DIRECTION

$$s_u^2 = \frac{1}{n} \sum_{i=1}^n p_i^2$$

In words: Square every point's projection onto the chosen direction, add the squares, then divide by the number of points.

When it is used: To score a candidate direction. PCA's whole search looks for the direction that makes this number largest.

Symbol	Meaning	Unit or type
s_u^2	the variance along direction u	unit of the features, squared
p_i	the projection of point i onto direction u	unit of the features
n	the number of points	count

We divide by n , as Chapter 4 did for standardization. Chapter 3's sample variance divides by $n - 1$; that changes every variance by the same factor and leaves the explained variance ratios of Lesson 20.5 unchanged.

Real software searches every direction at once, using matrix arithmetic beyond this book. To see the same idea on paper, the next worked example scores three sensible candidates.

WORKED EXAMPLE 20.2 — Test Three Candidate Directions for the Greatest Spread

Using the five centered scores $(3, 4)$, $(-3, -4)$, $(6, 8)$, $(-6, -8)$ and $(0, 0)$, find the variance along $(1, 0)$, the Math axis; $(0, 1)$, the Physics axis; and $(0.6, 0.8)$, a diagonal.

1. Project onto $(1, 0)$: $3(1) + 4(0) = 3$; then $-3, 6, -6, 0$. Only the Math deviation survives.
2. Square and average: $9 + 9 + 36 + 36 + 0 = 90$, and $90 \div 5 = 18$.
3. Project onto $(0, 1)$: $3(0) + 4(1) = 4$; then $-4, 8, -8, 0$. Square and average: $16 + 16 + 64 + 64 + 0 = 160$, and $160 \div 5 = 32$.
4. Check that $(0.6, 0.8)$ is a unit direction: $0.36 + 0.64 = 1$. It is.
5. Project onto it: $3(0.6) + 4(0.8) = 1.8 + 3.2 = 5$; then $-5, 6(0.6) + 8(0.8) = 3.6 + 6.4 = 10, -10, 0$.
6. Square and average: $25 + 25 + 100 + 100 + 0 = 250$, and $250 \div 5 = 50$. Compare 18, 32 and 50: the diagonal wins.

Answer: The diagonal $(0.6, 0.8)$ captures the most spread, with a variance of 50 — more than either subject captures alone. PCA keeps this winning direction as its first and most important new direction.

Figure 20.2 draws the search and Table 20.3 collects the arithmetic: each dotted line drops a student onto the candidate direction at a right angle, and the orange squares are where they land.

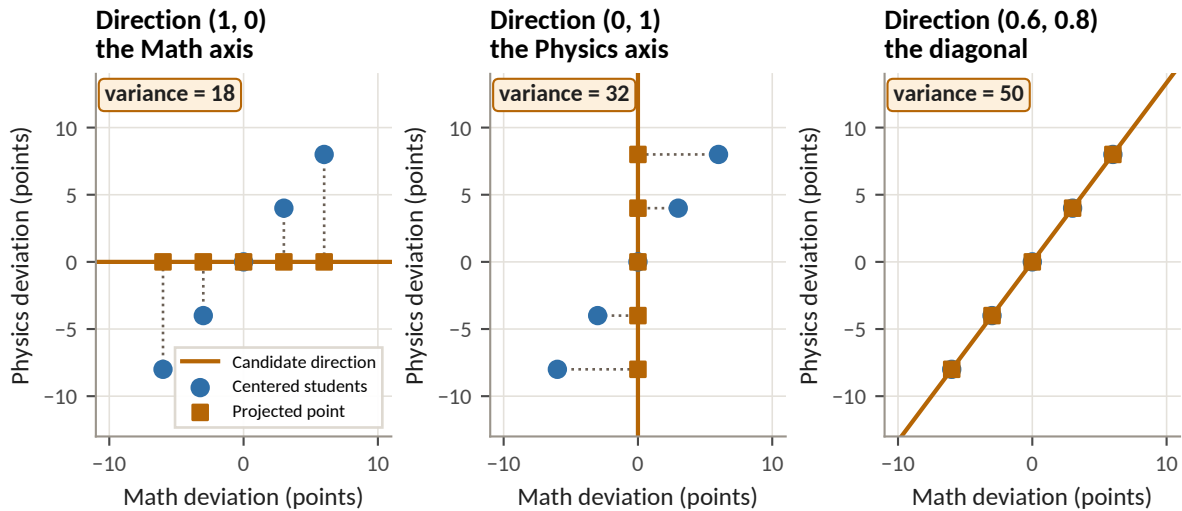


Figure 20.2 – Three candidate directions through the same five centered students. Dotted lines drop each student onto the direction; the orange squares are the projections whose spread is measured.

Table 20.3 – Projections of the five centered students onto four candidate directions

Direction	S1 (3, 4)	S2 (-3, -4)	S3 (6, 8)	S4 (-6, -8)	S5 (0, 0)	Variance
(1, 0) Math axis	3	-3	6	-6	0	18
(0, 1) Physics axis	4	-4	8	-8	0	32
(0.6, 0.8) diagonal	5	-5	10	-10	0	50
(0.8, -0.6) across	0	0	0	0	0	0

WHAT TO OBSERVE

Along the Math axis the projections are squeezed into -6 to 6 ; along the diagonal they stretch from -10 to 10 , and wider spread means larger variance. The drop lines are shortest in the third panel: the winning direction passes closest to the points.

20.4 Principal Components, One at a Time

The winning direction has a name, and so does the rule shaping every direction after it.

WORDS TO KNOW

Principal component — One of the new directions PCA finds, ranked by how much of the data's spread it captures. The first, PC1, is the single direction that captures the most spread of all; each later one captures the most spread still left over once the earlier ones are accounted for.

Orthogonal — At a right angle. Two unit directions are orthogonal when the products of their matching numbers add up to exactly 0. Principal components are always orthogonal, so each one measures something the earlier ones did not.

PC1's numbers read plainly: moving 0.6 steps along Math deviation goes with moving 0.8 steps along Physics deviation. Testing candidates by hand is honest but slow, and only ever checks the candidates you thought of. Real software instead builds one small square table from the centered data and reads the answer out of it.

WORDS TO KNOW

Covariance — A number saying how two centered features move together: multiply each example's two centered values, add the products, divide by the number of examples. Positive means they rise and fall together, negative means one rises as the other falls. It is the unstandardized cousin of the correlation coefficient of Chapter 3.

Covariance matrix — A square table holding the variance of every feature down its diagonal and the covariance of every pair off the diagonal. For d features it is $d \times d$, and it is always symmetric.

EQUATION: COVARIANCE AND THE COVARIANCE MATRIX (TWO FEATURES)

$$c_{xy} = \frac{1}{n} \sum_{i=1}^n x'_i y'_i, \quad \mathbf{C} = \begin{pmatrix} s_x^2 & c_{xy} \\ c_{xy} & s_y^2 \end{pmatrix}$$

In words: Multiply each example's two centered values together, add the products and divide by the number of examples: that is the covariance. Put the two feature variances on the diagonal and the covariance in both corners.

When it is used: To describe the shape of a centered cloud with a few numbers, and to let software find the components without testing candidates.

Symbol	Meaning	Unit or type
c_{xy}	covariance of features x and y	product of the two units
x'_i, y'_i	centered values of example i	unit of each feature
s_x^2, s_y^2	variance of each feature	unit squared
$\mathbf{C}$	the covariance matrix	table of the above

The variance along any unit direction u can be read off this table: $s_u^2 = u_1^2 s_x^2 + 2u_1 u_2 c_{xy} + u_2^2 s_y^2$. The principal components are the eigenvectors of $\mathbf{C}$ and their variances its eigenvalues; Appendix A explains those words, and no exercise here needs them.

WORKED EXAMPLE 20.3 — Build the Covariance Matrix and Read a Variance from It

Build the covariance matrix of the five centered students, then use it to find the variance along (0.6, 0.8) without projecting a single point.

1. From Worked Example 20.2, $s_x^2 = 18$ for Math and $s_y^2 = 32$ for Physics.
2. Multiply each student's two centered values: $3 \times 4 = 12$, $(-3)(-4) = 12$, $6 \times 8 = 48$, $(-6)(-8) = 48$, $0 \times 0 = 0$.
3. Add and divide by 5: $12 + 12 + 48 + 48 + 0 = 120$, and $120 \div 5 = 24$, so $c_{xy} = 24$.

4. Write the matrix: $\mathbf{C} = \begin{pmatrix} 18 & 24 \\ 24 & 32 \end{pmatrix}$, whose diagonal adds to the total variance $18 + 32 = 50$.
5. Use the note's formula with $u = (0.6, 0.8)$: $0.36 \times 18 = 6.48$; $2(0.6)(0.8) \times 24 = 0.96 \times 24 = 23.04$; $0.64 \times 32 = 20.48$.
6. Add: $6.48 + 23.04 + 20.48 = 50$.

Answer: The covariance matrix is $\begin{pmatrix} 18 & 24 \\ 24 & 32 \end{pmatrix}$, and the variance along $(0.6, 0.8)$ is 50 — the same answer as projecting all five students, from four numbers instead. The total variance, 50, is also the sum of the two feature variances, which is why PCA can never invent spread that was not already there.

Finding PC1 does not finish the job. Two original features hold two principal components in all, three hold three, and so on — but each later direction must sit at a perfect right angle to every component already found. Think of PC1 as a line through the middle of the data along its longest stretch; PC2 must cross it the way a north-south street meets an east-west one. Were PC2 allowed to point almost the same way as PC1, it would mostly measure the same spread again. In a two-feature dataset the rule leaves no freedom at all: once PC1 is fixed at $(0.6, 0.8)$, exactly one direction sits at a right angle to it, and Figure 20.3 draws the pair.

WORKED EXAMPLE 20.4 — Find the Second Principal Component

Confirm that $(0.8, -0.6)$ is a valid unit direction at a right angle to $\text{PC1} = (0.6, 0.8)$, then find its variance using the same five centered students.

1. Right-angle check: $(0.6)(0.8) + (0.8)(-0.6) = 0.48 - 0.48 = 0$ — exactly 0 means a perfect right angle. Unit-length check: $0.8^2 + (-0.6)^2 = 0.64 + 0.36 = 1$.
2. Project every student: $3(0.8) + 4(-0.6) = 2.4 - 2.4 = 0$; $-3(0.8) + (-4)(-0.6) = -2.4 + 2.4 = 0$; $6(0.8) + 8(-0.6) = 4.8 - 4.8 = 0$; likewise 0 and 0.
3. Square and average: $0 \div 5 = 0$.
4. Check the total: $50 + 0 = 50$, matching the total variance $18 + 32 = 50$ of Worked Example 20.3.

Answer: $\text{PC2} = (0.8, -0.6)$ has variance exactly 0. Every one of the five students lands precisely on the PC1 line, so once PC1 is accounted for, no spread is left in the one remaining direction.

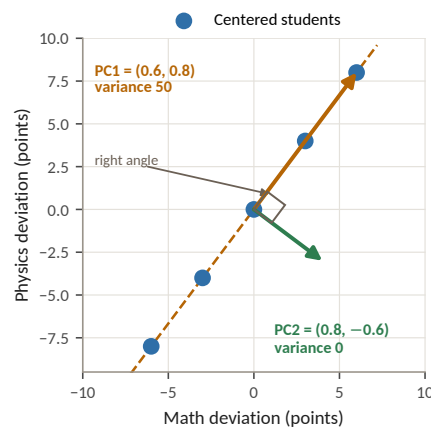


Figure 20.3 – The two principal components of the exam data, drawn on the centered points. PC1 runs along the data's longest stretch; PC2 crosses it at a right angle and finds nothing left to measure.

WHAT TO OBSERVE

All five students sit exactly on the dashed PC1 line, which is why the PC2 arrow has a variance of 0; the small square marks the right angle. In a real class the points would scatter a little and PC2 would capture a small but genuine spread.

These five students were chosen so that Math and Physics move in perfect lockstep, which keeps the arithmetic checkable by hand. The method never changes: look for the direction of greatest remaining variance, subject to a right angle with everything found so far. With three or more features, PC2 is chosen from a whole flat sheet of candidates rather than the single forced choice here.

COMMON MISTAKE

X Wrong: Reading PC1 as “really” one of the original features — saying that PC1 is the Math score because Math came first in the table.

✓ Right: A principal component is a blend of every original feature. PC1 here is 0.6 parts Math deviation and 0.8 parts Physics deviation: a new quantity that did not exist in the original table.

20.5 Explained Variance and How Many Components to Keep

Every time PCA replaces many features by a few components, some information is at risk of being left behind. The exam example lost nothing, an unusually tidy result; a real project needs a number for how much was kept, and a picture for reading it.

WORDS TO KNOW

Explained variance ratio — The share of a dataset’s total variance that one principal component accounts for, found by dividing that component’s variance by the sum of every component’s variance. A ratio close to 100% means that component carries nearly all of the spread.

Scree plot — A bar chart of the explained variance ratio of each component, in order, usually with the running total drawn on top as a line. The name is due to the psychologist Raymond Cattell in 1966: the tall early bars are the cliff, the small later ones the loose stones, the scree, at its foot.

EQUATION: EXPLAINED VARIANCE RATIO

$$r_k = \frac{s_k^2}{\sum_{m=1}^d s_m^2}$$

In words: Divide one component’s variance by the total of every component’s variance.

When it is used: Right after fitting PCA, to report each component’s worth and decide how many to keep.

Symbol	Meaning	Unit or type
r_k	explained variance ratio of component k	no unit, often shown as a percentage
s_k^2	variance along component k	unit of the features, squared
d	the number of components, equal to the number of features	count

The ratios of all d components always add to exactly 100%. Since the same divisor appears above and below the line, dividing variances by n or by $n - 1$ gives identical ratios.

WORKED EXAMPLE 20.5 — Explained Variance Ratios for the Two Exam Components

Worked Example 20.2 found PC1's variance to be 50 and Worked Example 20.4 found PC2's to be 0. Find both ratios.

1. Total variance: $50 + 0 = 50$.
2. PC1: $50 \div 50 = 1.00$, which is 100%. PC2: $0 \div 50 = 0.00$, which is 0%. The two add to 100%, as they must.

Answer: PC1 explains 100% of this class's spread and PC2 explains 0%, so each student's pair of exam scores could be replaced by a single PC1 number with nothing thrown away.

That perfect split came from choosing five students whose scores move in lockstep; realistic data spreads its variance across several components. Suppose a team at an electronics plant in Da Nang runs a pilot with two standardized sensors on one machine, and PCA returns a first component with variance 45 and a second with variance 5.

WORKED EXAMPLE 20.6 — Explained Variance for the Factory's Two Sensors

A two-sensor dataset has principal components with variances 45 and 5. Find both explained variance ratios.

1. Total variance: $45 + 5 = 50$.
2. PC1: $45 \div 50 = 0.90$, which is 90%. PC2: $5 \div 50 = 0.10$, which is 10%.

Answer: PC1 explains 90% of the sensor data's spread and PC2 the remaining 10%. Keeping only PC1 loses a small but real tenth of the machine's behavior — an everyday trade-off, quite unlike the exam example's perfect split.

With more than two features a project keeps as many leading components as it takes to reach a chosen target, usually between 90% and 95%. Table 20.4 gives an end-of-term report card: eight students, four subjects. Four follow an engineering track and four a medicine track, but PCA never sees those labels — it works from the four score columns alone.

Table 20.4 – End-of-term report card: eight students, four subjects (illustrative example numbers)

Student	Track	Math	Physics	Chemistry	Biology
Aram	Engineering	85	78	74	71
Dilan	Engineering	74	75	71	72
Hemin	Engineering	70	65	61	56
Kani	Engineering	69	68	64	67
Lana	Medicine	74	69	85	80
Media	Medicine	71	70	74	77
Nawroz	Medicine	61	54	70	67
Rezan	Medicine	64	65	69	70
Mean	—	71	68	71	70

PCA on these four columns returns components with variances 116, 58, 9 and 1. Table 20.5 turns them into ratios and Figure 20.4 draws the scree plot.

WORKED EXAMPLE 20.7 — Decide How Many Components to Keep for the Report Card

The four components have variances 116, 58, 9 and 1. Find the smallest number of components that reaches at least 90% of the total variance.

1. Total variance: $116 + 58 + 9 + 1 = 184$.
2. PC1: $116 \div 184 = 0.630$, which is 63.0%. Below the target, so one component is not enough.
3. PC2: $58 \div 184 = 0.315$, which is 31.5%.
4. Running total after two: $63.0\% + 31.5\% = 94.6\%$, or directly $174 \div 184 = 0.946$. The target is passed.
5. For completeness, PC3 gives $9 \div 184 = 4.9\%$ and PC4 gives $1 \div 184 = 0.5\%$.

Answer: Two components are enough: together they reach 94.6%. The report card shrinks from four subject scores per student to two new numbers, keeping almost 95% of the real spread.

Table 20.5 – Explained variance for the four-subject report card

Component	Variance (points ²)	Explained variance ratio	Running total
PC1	116	63.0%	63.0%
PC2	58	31.5%	94.6%
PC3	9	4.9%	99.5%
PC4	1	0.5%	100%
Total	184	100%	—

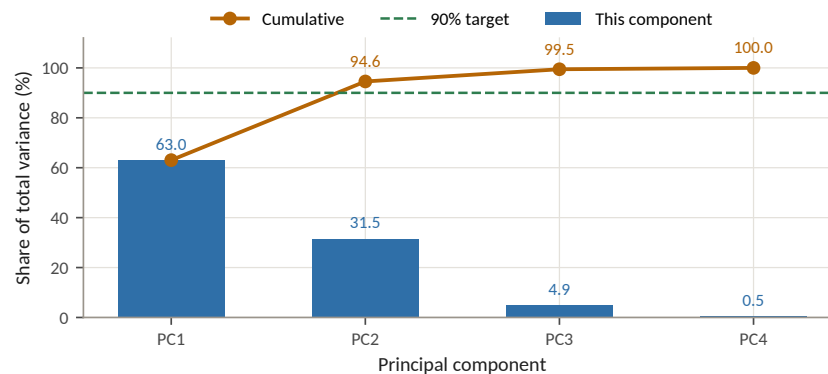


Figure 20.4 – Scree plot for the four-subject report card. Bars are each component’s own share, the line is the running total, and the dashed line is the 90% target.

WHAT TO OBSERVE

The line crosses the 90% target between PC2 and PC3, so two components are enough. The bars show a clear cliff — two tall ones, then almost nothing — the same elbow-reading habit Chapter 18 used for k .

20.6 Loadings, Visualization and Rebuilding an Example

Knowing that two components are enough is half the story. What are they made of? Table 20.6 and Figure 20.5 answer that, and the rest of the lesson prices what keeping only two costs.

WORDS TO KNOW

Loading — One of the numbers inside a principal component's direction, saying how much of one original feature is mixed into that component. A large positive loading pushes the component up, a large negative one pulls it down, and a loading near zero means the feature hardly matters there.

Reconstruction — Rebuilding an example's original feature values from the few component scores that were kept. The gap between the true row and the rebuilt row is what dimensionality reduction threw away.

Table 20.6 – Loadings of the four report-card components (each column of numbers squared adds to 1)

Subject	PC1	PC2	PC3	PC4
Math	+0.5	+0.5	+0.5	+0.5
Physics	+0.5	+0.5	−0.5	−0.5
Chemistry	+0.5	−0.5	+0.5	−0.5
Biology	+0.5	−0.5	−0.5	+0.5
Share of variance	63.0%	31.5%	4.9%	0.5%

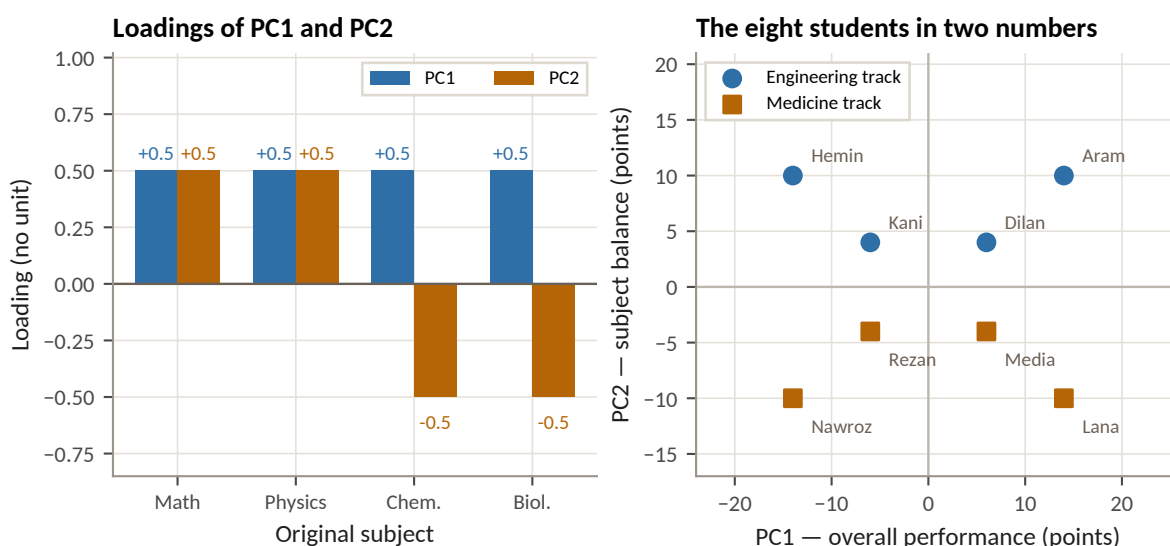


Figure 20.5 – Left: loadings of the first two report-card components. Right: the eight students drawn with PC1 across and PC2 up. PCA never saw the track labels; the colors were added afterward.

WHAT TO OBSERVE

PC1's four loadings are all +0.5, so it adds up how far above or below average a student is across every subject: an overall-performance score. PC2's are +0.5, +0.5, −0.5, −0.5, so it measures the *balance* between the two pairs of subjects — and on the right every engineering student sits above the axis and every medicine student below it, from score columns alone.

Because every loading here is ± 0.5 , a student's component scores need nothing but addition. Aram scored 85, 78, 74 and 71, and the subject means are 71, 68, 71 and 70, so his centered scores are 14, 10, 3 and 1. His PC1 score is $0.5(14 + 10 + 3 + 1) = 0.5 \times 28 = 14$, exactly twice his average deviation of 7 points; his PC2 score is $0.5(14 + 10 - 3 - 1) = 0.5 \times 20 = 10$, saying that his strength leans toward math and physics. Do the same for all eight students and the whole report card becomes the right-hand chart of Figure 20.5.

IMPORTANT

A principal component rarely has a clean, ready-made real-world meaning. “Math score” explains itself to anybody; “half the math deviation plus half the physics deviation minus half the chemistry and biology deviations” does not. Nicknames such as “overall performance” are a convenience your team invents, not a guarantee baked into the mathematics, and next term’s data can shift what a component leans on.

Keeping two components out of four is a form of compression, and the way to price it is to rebuild the original row from the components you kept, as Figure 20.6 does for one new student.

EQUATION: RECONSTRUCTION FROM THE KEPT COMPONENTS

$$\hat{x}_j = \mu_j + \sum_{k=1}^K p_k u_{kj}$$

In words: Multiply each kept component’s score by that component’s loading for the feature, add the results, then add the feature’s mean back on.

When it is used: To check what a reduced dataset has lost, to compress and later restore data, and to smooth away noise by dropping the last components.

Symbol	Meaning	Unit or type
$\hat{x}_j$	rebuilt value of feature j	unit of feature j
μ_j	mean of feature j	unit of feature j
p_k	the example’s score on component k	unit of the features
u_{kj}	loading of feature j in component k	no unit
K	number of components kept	count

Keep every component ($K = d$) and the rebuilt row equals the original exactly. Keep fewer, and the left-over gap is precisely the part carried by the components you dropped.

WORKED EXAMPLE 20.8 — Project a New Student onto PC1 and Rebuild Her Scores

Nasrin joins the class of five and scores 71 in Math and 73 in Physics. Using the class means 70 and 65 and the components $PC1 = (0.6, 0.8)$ and $PC2 = (0.8, -0.6)$, find her component scores, then rebuild her exam scores from PC1 alone.

1. Center with the *existing* class means: $71 - 70 = 1$ and $73 - 65 = 8$, so her centered point is $(1, 8)$.
2. PC1 score: $1(0.6) + 8(0.8) = 0.6 + 6.4 = 7$.
3. PC2 score: $1(0.8) + 8(-0.6) = 0.8 - 4.8 = -4$.
4. Rebuild from PC1 alone: $7 \times (0.6, 0.8) = (4.2, 5.6)$.
5. Add the means back: Math = $70 + 4.2 = 74.2$, Physics = $65 + 5.6 = 70.6$.
6. Compare with the truth: $71 - 74.2 = -3.2$ and $73 - 70.6 = +2.4$, a gap of length $\sqrt{10.24 + 5.76} = \sqrt{16} = 4$.

Answer: Nasrin’s component scores are 7 and -4 . Rebuilt from PC1 alone her scores come out as 74.2 and 70.6 instead of 71 and 73, and the size of the gap, 4, is exactly the PC2 score that was dropped. PCA kept her overall level but lost the detail that she is relatively stronger in Physics.

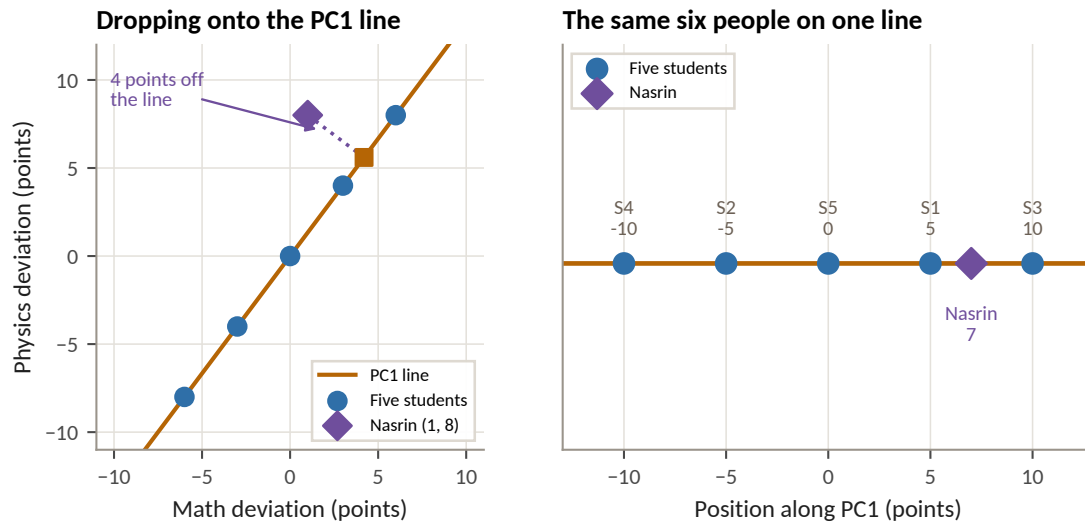


Figure 20.6 – Left: each person dropped onto the PC1 line at a right angle. Right: the same six people described by a single number, their position along PC1.

WHAT TO OBSERVE

The right-hand strip is the point of PCA: six people who needed two numbers each now need one, and still sit in a sensible order. Nasrin's dropped distance of 4 points is the only information lost.

Dropping the last few components on purpose has a second use: small, low-variance components often carry measurement noise rather than real structure, so rebuilding from the leading components alone can leave a dataset visibly cleaner — the same trade-off Chapter 7 described.

20.7 PCA Needs Scaling — and What It Cannot Do

Both exam datasets used a single 0-to-100 scale, so centering alone was fair; real features rarely agree so politely. Figure 20.7 runs PCA twice on four sensors of the Da Nang machine: temperature in degrees Celsius (spread about 5.9), vibration in millimeters per second (0.29), motor current in amperes (0.79) and rotation speed in revolutions per minute (165).

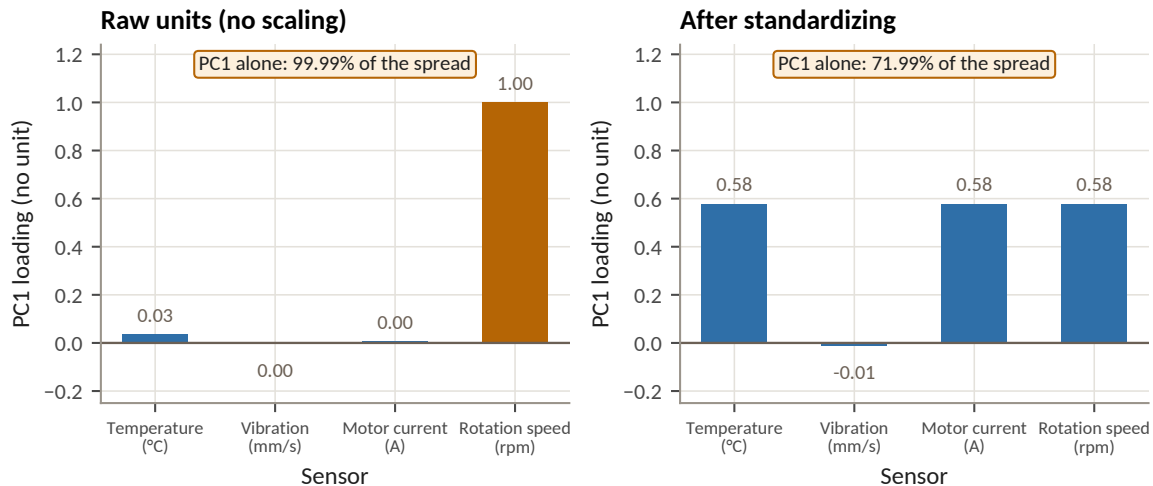


Figure 20.7 – PC1’s loadings for four sensors on the same machine, computed from raw readings (left) and after standardizing every sensor (right).

WHAT TO OBSERVE

On raw readings PC1 is almost entirely rotation speed and explains 99.99% of the variance — not because rotation speed matters most, but because rpm are the biggest numbers in the table. After standardizing it spreads its weight evenly over temperature, motor current and rotation speed and explains a believable 72%. Only the units changed.

The reason is simple: variance is measured in the square of a feature’s own unit, so a column recorded in thousands contributes thousands of times more variance than one recorded in single digits, and PCA walks straight toward it. Standardizing first — subtract the mean, divide by the standard deviation, as Chapter 4 taught and Chapter 11 required — puts every column on the same footing.

COMMON MISTAKE

X Wrong: Running PCA straight on raw columns that mix units — degrees, amperes, revolutions per minute — because “the software centers the data automatically”.

✓ Right: Fit a `StandardScaler` on the training rows first, then PCA, ideally inside one pipeline. Software does center for you; almost none of it standardizes for you.

Even with careful scaling, PCA has real limits.

- **It only sees straight-line structure.** Data lying on a curve cannot be flattened faithfully by any straight direction, just as the correlation coefficient of Chapter 3 is blind to a U-shaped relationship.
- **It ignores the label.** The direction of greatest spread is not always the one that separates two classes; Lesson 20.8 shows a case where they point almost at right angles.
- **Its components are hard to explain,** and it is **pulled around by outliers:** one wildly wrong row inflates the variance in its own direction, which is why the checks of Chapter 4 come first.
- **The mean and the components must be learned on the training rows only,** or test information leaks into the preparation — exactly the leak Chapter 5 warned about.

RESPONSIBLE USE

A PCA chart of people — patients, students, loan applicants — looks objective because it is built from arithmetic alone, but it is not a measurement of a person. Two dots that sit near each other are near *in the features that happened to be recorded*, which may leave out everything that matters most. Never let a position on such a chart decide a person's treatment, place or opportunity by itself. Chapter 24 returns to this.

20.8 Two Other Projections in Brief: LDA and t-SNE

PCA is not the only way to squeeze many features into two. Two other methods appear constantly beside it, each answering a different question.

WORDS TO KNOW

Linear discriminant analysis (LDA) — A supervised projection: it uses the labels and looks for the direction that separates the classes best, rather than the direction of greatest overall spread. Introduced by Ronald Fisher in 1936, it can produce at most $c - 1$ useful directions for c classes.

t-SNE — Short for *t*-distributed stochastic neighbor embedding, published by Laurens van der Maaten and Geoffrey Hinton in 2008. It hunts for a 2-D layout in which examples that were close together stay close together, even if larger distances get badly distorted. It is a tool for drawing pictures, not for producing features to feed to another model.

Figure 20.8 shows why the difference matters. Two sensors on the Da Nang machine were recorded during healthy running and with a worn belt; the cloud is long in one direction and narrow across it, but the two classes are separated *across* the narrow direction.

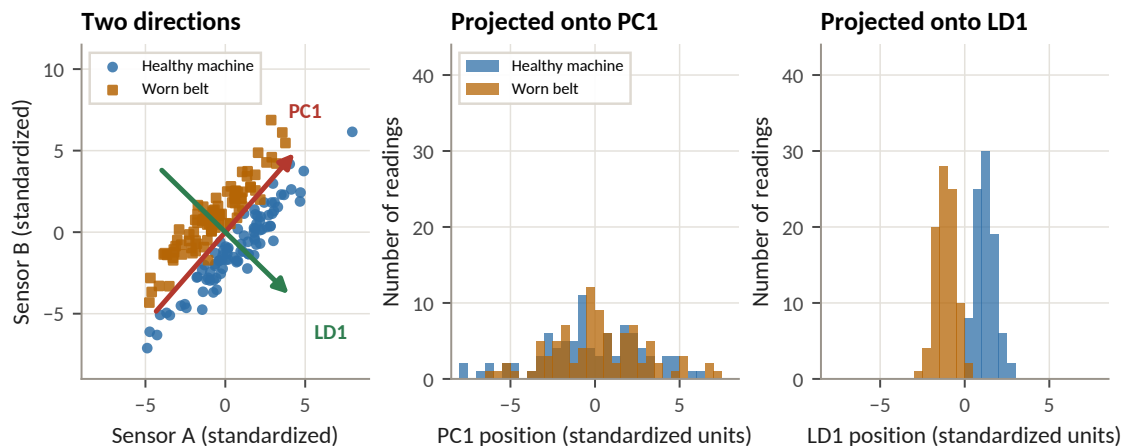


Figure 20.8 – The same 180 readings, projected onto PC1 and onto LD1.

WHAT TO OBSERVE

PC1 (red) runs along the length of the cloud, and the middle histogram shows the two classes sitting almost on top of each other: reduced to that one number, healthy and worn readings cannot be told apart. LD1 (green) points across the cloud instead, and its histograms barely touch. The widest spread and the most useful split are not the same direction.

t-SNE answers a third question: not which direction is widest, nor which separates the labels, but whether a picture can be drawn in which neighbors stay neighbors. Figure 20.9 compares it with PCA on 537 tiny 8-by-8 images of handwritten digits that ship with scikit-learn, 64 pixel features squeezed down to 2.

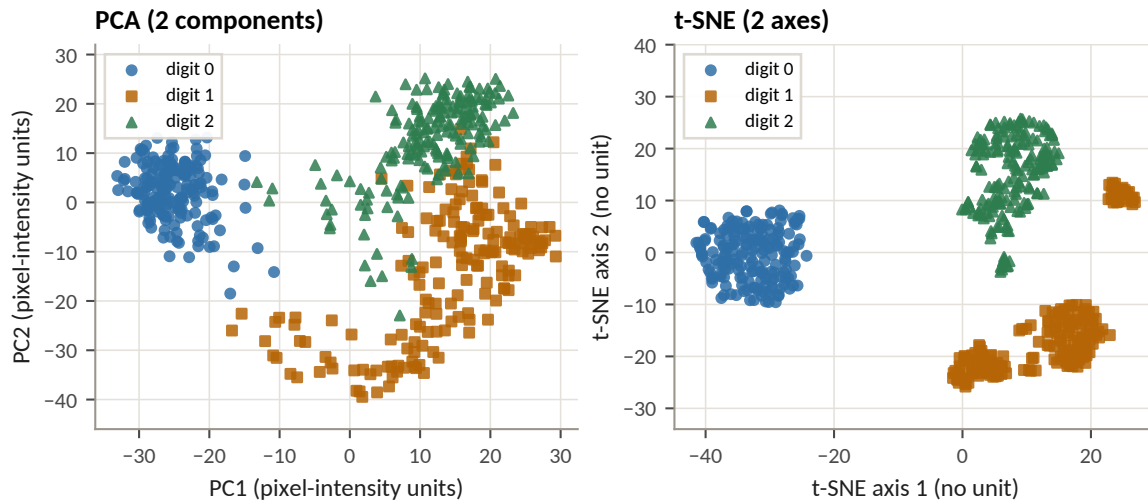


Figure 20.9 – 537 handwritten-digit images, each with 64 pixel features, drawn in two dimensions. Left: PCA. Right: t-SNE.

WHAT TO OBSERVE

PCA keeps the digits roughly apart but lets the 1s and 2s bleed into each other, because it is allowed only straight directions; t-SNE pulls each group into a tight island. Note the two separate orange islands: distances between islands carry no reliable meaning, and t-SNE axes should never be reused as features.

A cousin of t-SNE called UMAP, published by Leland McInnes, John Healy and James Melville in 2018, aims at similar pictures, runs faster on large datasets and keeps a little more of the overall shape. Choosing between the three is mostly a question of what you have and what you want: PCA when there are no labels and the new axes must be reusable, LDA when labels exist and the goal is separation, t-SNE or UMAP when the goal is a picture and nothing else.

COMMON MISTAKES AND MISCONCEPTIONS

- Deleting the features that look unimportant instead of letting PCA build blends that keep something from every column.
- Running PCA on raw columns that mix units, so that the component with the biggest numbers wins automatically.
- Comparing spreads along directions of different lengths, or believing a principal component is one original feature in disguise whose nickname is an official meaning.
- Keeping components until the explained variance reaches exactly 100%, which keeps every component and cancels the whole benefit.
- Assuming the direction of greatest spread must also separate two classes — that is LDA's job, not PCA's.
- Reading distances between clusters in a t-SNE picture as real distances, or feeding t-SNE axes into another model.
- Fitting the PCA mean and components on the training and test rows together, which leaks test information into the preparation.

Practical Activities

ACTIVITY (PAPER) — Score a Fourth Candidate Direction by Hand

The five centered students are (3, 4), (−3, −4), (6, 8), (−6, −8) and (0, 0). A classmate proposes the 45-degree direction (1, 1).

1. Show that (1, 1) is not a valid unit direction, then rescale it by dividing both numbers by $\sqrt{2} \approx 1.4142$, and check that the rescaled numbers, squared, add to 1.
2. Project all five students onto the rescaled direction. (Hint: each projection is the sum of the two centered values, times 0.7071.)
3. Square the five projections and average them to get the variance. Keep four decimals; the squares come out exactly 24.5, 24.5, 98, 98 and 0.
4. Compare your answer with the 50 that PC1 achieved, write the ratio as a percentage, then draw the five centered points, the 45-degree line and the PC1 line on graph paper and explain in two sentences why the 45-degree line does so nearly as well but still loses.

You need: Graph paper, pencil, ruler and a calculator with a square-root key

ACTIVITY (PYTHON) — PCA from Scratch, Then with scikit-learn

The first program repeats Worked Examples 20.1 to 20.5 and 20.8 exactly, first with plain NumPy arithmetic and then with scikit-learn's PCA.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.decomposition import PCA
X = np.array([[73, 69], [67, 61], [76, 73], [64, 57], [70, 65]]) # Math, Physics
Xc = X - X.mean(axis=0) # Worked Example 20.1
for u in [np.array([1, 0]), np.array([0, 1]), np.array([0.6, 0.8])]:
    print("direction", u, "projections", Xc @ u, "variance", ((Xc @ u) ** 2).sum() /
          len(X))
print("covariance matrix:", (Xc.T @ Xc / len(X)).tolist())
pca = PCA(n_components=2).fit(X)
print("components_:", pca.components_.round(4).tolist(), "ratio:",
      pca.explained_variance_ratio_.round(4),
      "explained_variance_ (n-1):", pca.explained_variance_.round(2))
z = pca.transform([[71, 73]]) # Nasrin, a new student
print("Nasrin:", z.round(2), "rebuilt from PC1:", (pca.mean_ + z[0, 0] *
      pca.components_[0]).round(2))
```

Output

```
direction [1 0] projections [ 3. -3.  6. -6.  0.] variance 18.0
direction [0 1] projections [ 4. -4.  8. -8.  0.] variance 32.0
direction [0.6 0.8] projections [ 5. -5. 10. -10.  0.] variance 50.0
covariance matrix: [[18.0, 24.0], [24.0, 32.0]]
components_: [[0.6, 0.8], [0.8, -0.6]] ratio: [1. 0.] explained_variance_ (n-1): [62.5
 0. ]
Nasrin: [[ 7. -4.]] rebuilt from PC1: [74.2 70.6]
```

Every hand result reappears: the variances 18, 32 and 50, the covariance matrix, and Nasrin's scores 7 and −4 rebuilt as 74.2 and 70.6. Note that `explained_variance_` prints 62.5 rather than 50, because scikit-learn divides by $n - 1 = 4$ while this chapter divides by $n = 5$; indeed $50 \times 5 \div 4 = 62.5$, and the

ratio is unaffected. A component and its opposite describe the same line, so another computer may print $[-0.6, -0.8]$ — every score simply changes sign with it.

The second program, run in the same session, reproduces Table 20.6, Figure 20.7 and the case-study number, then compares three projections by counting how many of each digit image's ten nearest neighbors carry the same digit.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.manifold import TSNE; from sklearn.datasets import load_digits
from sklearn.neighbors import NearestNeighbors
R = np.array([[85, 78, 74, 71], [74, 75, 71, 72], [70, 65, 61, 56], [69, 68, 64, 67],
              [74, 69, 85, 80], [71, 70, 74, 77], [61, 54, 70, 67], [64, 65, 69, 70]])
rc = PCA().fit(R)
r_ = rc.explained_variance_ratio_ * 100
print("loadings:", rc.components_.round(2).tolist())
print("ratios %:", r_.round(1), " cumulative %:", r_.cumsum().round(1))
S = pd.read_csv("datasets/ch20_sensors.csv")
f4 = ["temperature_c", "vibration_mm_s", "motor_current_a", "rotation_rpm"]
for nm, M in [("raw", S[f4].values), ("std", StandardScaler().fit_transform(S[f4]))]:
    q = PCA().fit(M)
    print(nm, "PC1", q.components_[0].round(2), round(q.explained_variance_ratio_[0], 4))
A8 = PCA().fit(StandardScaler().fit_transform(S.values))
print("8 sensors, first two: %.1f%%" % (A8.explained_variance_ratio_[:2].sum() * 100))
d = load_digits(); k = np.isin(d.target, [0, 1, 2]); Xd, yd = d.data[k], d.target[k]
def purity(Z):
    # 10 nearest neighbors of the same digit?
    ix = NearestNeighbors(n_neighbors=11).fit(Z)
    ix = ix.kneighbors(Z, return_distance=False)[: , 1:]
    return round(float((yd[ix] == yd[: , None]).mean()), 3)
print("purity  PCA:", purity(PCA(n_components=2).fit_transform(Xd)),
      "LDA:", purity(LinearDiscriminantAnalysis().fit(Xd, yd).transform(Xd)),
      "t-SNE:", purity(TSNE(n_components=2, init="pca",
                           random_state=0).fit_transform(Xd)))
```

Output

```
loadings: [[0.5, 0.5, 0.5, 0.5], [0.5, 0.5, -0.5, -0.5], [0.5, -0.5, 0.5, -0.5], [0.5,
-0.5, -0.5, 0.5]]
ratios %: [63.  31.5  4.9  0.5] cumulative %: [ 63.   94.6  99.5 100. ]
raw PC1 [ 0.03 -0.   0.   1. ] 0.9999
std PC1 [ 0.58 -0.01  0.58  0.58] 0.7199
8 sensors, first two: 93.2%
purity  PCA: 0.94 LDA: 1.0 t-SNE: 1.0
```

The loadings and ratios match Table 20.6 and Table 20.5, including the 94.6% that decided Worked Example 20.7, and the two sensor lines show the scaling trap: PC1's share falls from 0.9999 to 0.7199 once the columns are standardized. The last line shows why t-SNE is popular for pictures — in its axes every point's ten nearest neighbors carry the same digit, while PCA, restricted to straight directions, gets 94% right — but LDA reached 1.0 only because it saw the labels.

You need: A computer with Python, NumPy, pandas and scikit-learn, and the file `datasets/ch20_sensors.csv` (Appendix B and Appendix C)

CASE STUDY — Watching a Machine's Health with Two Numbers Instead of Eight — an Electronics Plant in Da Nang

An electronics assembly plant in Da Nang runs a row of automated soldering machines around the clock. Each machine reports eight sensor readings every minute: temperature, vibration, motor current, humidity inside the housing, rotation speed, feed pressure, cooling-fan speed and a wear sensor on the main belt. A maintenance engineer, Linh, is supposed to watch these across twelve machines and notice early signs of trouble, but eight numbers per machine every minute is more than anyone can follow, and most days the team learns that a machine was struggling only once it has stopped. So Linh's team tries PCA, aiming for a dashboard a busy person can glance at. The eight sensors arrive in wildly different units — degrees, millimeters per second, amperes, percent, revolutions per minute — so every column is standardized first, exactly as Lesson 20.7 insists. Running PCA on several months of normal operation, they find that the first component explains 67.8% of the standardized variance and the second a further 25.4%: 93.2% between them, comfortably past the 90% target agreed in advance. They keep two numbers per machine per minute instead of eight.

The loadings show a loose pattern. PC1 leans almost equally on temperature, motor current, rotation speed, feed pressure and fan speed (loadings near $+0.41$ each) and the other way on housing humidity (-0.38), which drops as the machine heats up; all of these move with how hard the machine is working, so the team nicknames PC1 “overall load”. PC2 is dominated by vibration and belt wear ($+0.69$ each), which move fairly independently of load, so they call it “mechanical strain”. Linh reminds the newer technicians, as the IMPORTANT box in Lesson 20.6 warns, that these nicknames are a convenience the team invented, not a meaning guaranteed by the mathematics.

The dashboard is one live scatter plot — PC1 across, PC2 up, one dot per machine, refreshed every minute — and normally all twelve dots cluster in a familiar patch. When Machine 7 begins to wear out, its dot drifts steadily away from that patch, days before any of its eight readings crosses a limit strict enough to trigger the plant's old one-sensor-at-a-time alarms; a technician notices, inspects the belt and replaces it during a routine pause. A drifting dot is never a shutdown command, only a prompt to look closer, and the team refits PCA every few months on fresh data — new means, new loadings, a new patch — so old and new charts are never compared directly.

CHAPTER SUMMARY

- Dimensionality reduction shrinks the number of features describing each example while keeping as much real information as possible; it builds blends of columns rather than deleting columns.
- PCA centers every feature first, so that each number says how far above or below average an example sits.
- Projecting a centered point onto a unit direction multiplies matching numbers and adds them; the variance along that direction is the average of the squared projections, and PC1 is the direction that makes it largest. Software finds PC1 from the covariance matrix rather than by testing candidates, and every later component sits at a right angle to all the earlier ones.
- The explained variance ratio divides one component's variance by the total; the ratios always sum to 100%, and a scree plot shows them as bars with the running total as a line.
- Keep the smallest number of components that reaches a chosen target, usually 90% to 95% — never insist on 100%.
- A loading says how much of one original feature is mixed into one component; a component is a new quantity, and any nickname for it is a convenience, not a guarantee.
- Reconstruction rebuilds an example from the components kept, and the gap between the true row and the rebuilt row is exactly what was thrown away.
- Features on different scales must be standardized before PCA, or the column with the biggest numbers wins automatically.
- PCA is unsupervised and linear. LDA uses labels to find the separating direction; t-SNE draws pictures where close neighbors stay close but distances between groups mean nothing.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] In your own words, explain what dimensionality reduction means, and say in one sentence why building a blend of columns differs from deleting columns.

Problem 2 [Basic] A dataset has 8 numerical features. (a) How many different pairs of features could you plot against each other? (b) Explain in one sentence why this makes “just look at the data” impractical.

Problem 3 [Basic] Define “principal component” in your own words, and explain in one or two sentences why the second principal component must sit at a right angle to the first.

Problem 4 [Basic] Check whether each pair is a valid unit direction, showing your arithmetic: (a) $(0.6, -0.8)$, (b) $(0.5, 0.5)$, (c) $(0, -1)$.

Problem 5 [Standard] Four students' raw Math scores are 82, 74, 90 and 78. Find the class mean and each centered score, and check that the centered scores add to zero.

Problem 6 [Standard] A centered point is $(4, -2)$. Find its projection onto $(1, 0)$, onto $(0, 1)$ and onto $(0.6, 0.8)$, showing every step.

Problem 7 [Standard] Five centered points are $(2, 1)$, $(-2, -1)$, $(0, 0)$, $(4, 2)$ and $(-4, -2)$. Find the variance along $(1, 0)$ and along $(0, 1)$, showing the squares you added, and give the total variance.

Problem 8 [Standard] A dataset's two principal components have variances 32 and 8. Find both explained variance ratios, as decimals and as percentages, and check that they add to 1.

Problem 9 [Standard] A dataset's four components have explained variance ratios 70%, 20%, 7% and 3%. (a) Write the running totals. (b) How many components reach at least 85%? (c) How many reach at least 95%?

Problem 10 [Standard] A student's four centered subject scores are $(12, 8, -4, 0)$ for Math, Physics, Chemistry and Biology. Using the loadings of Table 20.6, find her PC1 and PC2 scores, and say in one sentence what each number tells you.

Problem 11 [Challenge] Three centered points are $(3, 6)$, $(-3, -6)$ and $(0, 0)$. (a) Find the variance along $(1, 0)$, showing every step. (b) Find the variance along $(0, 1)$. (c) Explain, without computing it, why a diagonal direction must beat both. (d) Check that $(0.4472, 0.8944)$ is a unit direction, find the variance along it, and confirm it equals the total from parts (a) and (b).

Problem 12 [Challenge] The department re-reports Physics on a 0-to-20 scale by dividing every Physics score by 5, leaving Math on 0 to 100. (a) Write the new centered Physics deviations for the five students of Table 20.2. (b) Recompute the variance along $(1, 0)$ and along $(0, 1)$. (c) Explain what has happened to PC1, and name the one preparation step that would have prevented it.

Problem 13 [Challenge] A hospital records twelve lab measurements per patient and wants a single 2-D chart showing which patients resemble one another. (a) Explain why the twelve measurements must be standardized first. (b) Explain why the two axes will be hard to describe to a doctor who has not studied PCA. (c) A colleague suggests t-SNE instead because the picture looks cleaner: give one advantage and one serious risk. (d) Name one safeguard the hospital should insist on before the chart influences any decision about a patient.

Anomaly Detection

Finding the readings that do not belong

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what an anomaly is, and how it differs from the single-column outlier of Chapter 4.
- Tell point, contextual and collective anomalies apart, with an everyday example of each.
- Compute an anomaly score by hand, first as the distance from the centroid of normal data and then as a combined z-score, and set a sensible threshold.
- Compute a local density ratio by hand, and say when it beats distance from the center.
- Explain why an isolation forest needs only a few random cuts to separate an unusual example.
- Choose a threshold from an alert budget, measure precision at the top of an alert list, and act on a flagged example responsibly.

Introduction

Imagine reading your family's electricity meter on the first day of every month, for years. The number climbs by a similar amount each time — a little more in winter when the heaters run, a little less in spring — but always inside a familiar range. Then one month it barely moves at all, or jumps to four times anything you have ever seen. You do not need a statistics course to notice: you know your own normal well enough to feel it when a reading breaks the pattern.

Part Four so far has studied whole datasets at once: Chapter 18 and Chapter 19 sorted examples into groups, and Chapter 20 found the directions that carried most of the information. This chapter asks a sharper question about one example at a time: does this one belong here at all? A bank looking for a stolen card, a factory listening for a bearing about to fail and a company hunting a bypassed meter all ask exactly that.

One twist makes this different from the classification of Chapter 1. A classifier needs many labeled examples of every class, but the whole difficulty here is that nobody has enough confirmed examples of the problem — it is rare, or new, or has never been seen at all. The methods in this chapter need no labels for the unusual class. They need only a good picture of what normal looks like, and a fair way to measure how far a new example sits from it.

21.1 What Makes an Example Unusual?

Chapter 4 already met unusual values, under a different name and for a different purpose. There, an *outlier* was a single value sitting far from the rest of its own column — an apartment of 800 square meters where the others were near 80 — and the job was to check it, correct it or note it so it would not poison a model. The unusual value was dirt to be cleaned away before the real work began. This chapter turns that upside down: here the unusual example is the real work. Nobody wants to delete the strange meter reading; somebody

wants to find it, rank it, and send an inspector. The tools overlap — the z-score returns in Lesson 21.3 — but the goal has changed, and so has the vocabulary.

WORD TO KNOW

Anomaly — An example that looks noticeably different from most other examples in a dataset, once all of its features are considered together. An anomaly may point to a real problem, such as a broken sensor or a fraudulent charge, or it may simply be a rare but honest example. Finding one is only the first step; deciding *why* it is unusual needs a human look.

An outlier is about one column; an anomaly is about a whole example, and that difference matters more than it sounds. A two-year-old child is not unusual. A height of 150 centimeters is not unusual, for an adult. But a two-year-old who is 150 centimeters tall would be extraordinary, even though neither number alone raised any concern.

COMMON MISTAKE

X Wrong: Deciding that an example is normal because each of its features looks ordinary on its own, the way a height of 150 cm looks ordinary for an adult.

✓ Right: Check the combination of features together. Lesson 21.2 builds a method that does exactly this, comparing a whole example, every feature at once, against a typical example built from normal data.

Start with a single meter and a single feature. Figure 21.1 shows one household's daily usage for a month, in kilowatt-hours (kWh).

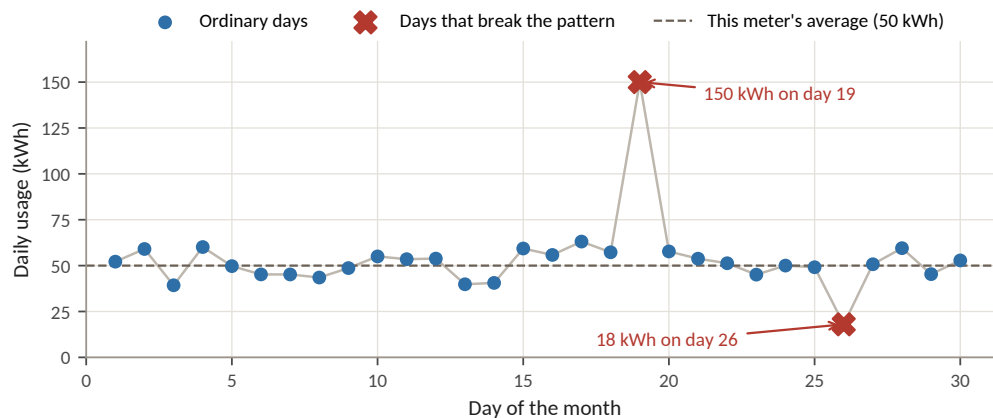


Figure 21.1 – One household's daily electricity usage over a month. Twenty-eight days sit close to the meter's own average; two do not.

WHAT TO OBSERVE

Day 19 uses about three times the household's average and day 26 about a third of it. Both matter: a system watching only for unusually *high* numbers would miss day 26, yet a low reading is exactly what a bypassed meter looks like.

With a handful of numbers, your eye is the whole method.

WORKED EXAMPLE 21.1 — Spot the Unusual Reading by Eye

A household's smart meter reports daily usage, in kWh, for five days: 42, 38, 45, 40 and 890. Which day's reading is the anomaly, and how far outside the pattern is it?

1. Compare the four smaller readings: 42, 38, 45 and 40. All sit within a few kWh of each other, and their average is $(42 + 38 + 45 + 40) \div 4 = 165 \div 4 = 41.25$ kWh.
2. Compare the fifth reading, 890, with that group. It is far larger than any of the others.
3. Find how many times larger 890 is than the next-highest reading, 45: $890 \div 45 = 19.8$ (to one decimal place).
4. Find the plain gap too: $890 - 45 = 845$ kWh, more than twenty times the whole normal daily usage.

Answer: The 890 kWh reading is the anomaly, about 19.8 times the next-highest day and 845 kWh above it. With five numbers your eye is enough. The rest of this chapter builds methods that still work when a company must check millions of readings on several features at once.

Not every anomaly announces itself so loudly. It helps to sort them into three kinds.

WORDS TO KNOW

Point anomaly — A single example that is unusual compared with the whole dataset, wherever and whenever it appeared. The 890 kWh reading is a point anomaly.

Contextual anomaly — An example whose value is perfectly ordinary in general, but wrong for its context — its time of day, its season, its location, or the customer it belongs to.

Collective anomaly — A group of examples that is unusual as a group, even though no single member of the group is unusual on its own.

Figure 21.2 shows all three side by side.

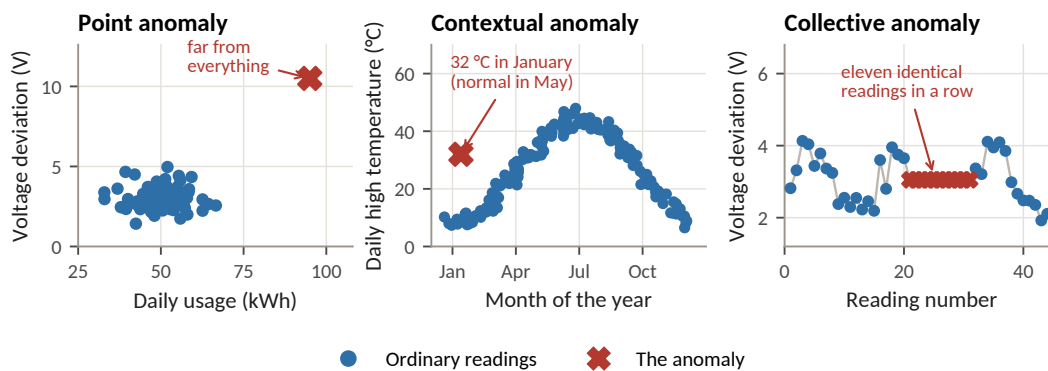


Figure 21.2 – Three kinds of anomaly. Left: a reading far from everything. Middle: a reading whose value is ordinary but whose context is wrong. Right: a run of readings that is only strange as a run.

WHAT TO OBSERVE

In the middle panel, 32 °C is a completely ordinary temperature — in May. In January it is remarkable, and only the context reveals it. In the right panel every flagged reading is about 3 V, exactly normal; what is not normal is eleven *identical* readings in a row, the signature of a frozen sensor.

Most of this chapter works on point anomalies, because the other two kinds reduce to them once the right feature is added: put “month of the year” beside the temperature, or summarize each hour into one row holding “how many different values did this sensor report?”. Chapter 22 is about such features. Two last words separate situations that look alike but are not.

WORDS TO KNOW

Novelty detection — Training on data you are confident is clean — only normal examples — and then judging new examples one at a time. Its opposite, *outlier detection*, trains on a pile of data that already contains some unknown number of anomalies and asks the method to find them inside its own training set.

Contamination — The fraction of the training data believed to be anomalous. Several scikit-learn detectors take this as a setting, and it decides how many examples they flag.

If a maintenance team can honestly say “these 5,000 hours were all fine”, your picture of normal is clean. If nobody can say that, every anomaly hiding in the training data quietly drags the idea of normal toward itself.

21.2 Distance from the Center

Real monitoring watches several features at once, so a detector needs a fair way to squeeze them into one number. Chapter 1 gave half of what is needed — the *centroid*, the average position of a group of points — and Chapter 11 gave the other half, *Euclidean distance*. Put them together and you have your first detector: collect examples you are confident are normal, find their centroid, and measure how far each new example sits from it.

WORD TO KNOW

Anomaly score — A number saying how unusual one example looks compared with normal data. A higher score means further from typical. Every method in this chapter produces an anomaly score; they differ only in how they compute it.

An electricity distribution company tracks two numbers for every smart meter: daily usage, in kWh, and how far the supply voltage strays from the standard 220 volts, called the *voltage deviation*, in volts. A large deviation can mean worn wiring, an aging transformer or a tampered connection. Table 21.1 shows six recent days from one household’s meter, all confirmed normal by an inspector.

Table 21.1 – Six confirmed-normal readings from one household’s meter

Reading	1	2	3	4	5	6
Usage (kWh)	40	50	60	45	55	50
Voltage deviation (V)	2	3	4	3	3	3

The score is the distance formula you already know, applied between a new example and the centroid of normal data.

EQUATION: ANOMALY SCORE BY DISTANCE FROM THE CENTROID

$$s(\mathbf{x}) = \sqrt{\sum_{j=1}^p (x_j - c_j)^2}$$

In words: For each feature, subtract the centroid’s value from the example’s value and square the result; add all the squares; take the square root. That distance is the anomaly score.

When it is used: As a first, simple detector whenever normal data forms one roughly round cloud and you can collect enough trustworthy normal examples to find its center.

Symbol	Meaning	Unit or type
$s(\mathbf{x})$	anomaly score of example $\mathbf{x}$	no single natural unit
$\mathbf{x}$	the example being checked, with features $x_1, \dots, x_p$	unit of each feature
$\mathbf{c}$	centroid of the normal examples, with values $c_1, \dots, c_p$	unit of each feature
p	how many features	count

This is the Euclidean distance of Chapter 11 with the centroid of Chapter 1 as the second point. How large a score must be before it counts as unusual is the job of Lesson 21.3.

WORKED EXAMPLE 21.2 — Find the Centroid, Then Score Two New Readings

Using the six readings of Table 21.1, find the centroid of normal behavior. Then find the anomaly score of Reading P (54 kWh, 6 V) and Reading Q (90 kWh, 12 V).

1. Add the six usage values and divide by 6: $(40 + 50 + 60 + 45 + 55 + 50) \div 6 = 300 \div 6 = 50$ kWh.
2. Add the six voltage deviations and divide by 6: $(2 + 3 + 4 + 3 + 3 + 3) \div 6 = 18 \div 6 = 3$ V. The centroid is (50, 3).
3. Reading P, usage: $54 - 50 = 4$, and $4^2 = 16$. Voltage deviation: $6 - 3 = 3$, and $3^2 = 9$.
4. Add and take the square root: $16 + 9 = 25$, and $\sqrt{25} = 5$. Reading P scores 5.
5. Reading Q, usage: $90 - 50 = 40$, and $40^2 = 1,600$. Voltage deviation: $12 - 3 = 9$, and $9^2 = 81$.
6. Add and take the square root: $1,600 + 81 = 1,681$, and $\sqrt{1,681} = 41$. Reading Q scores 41.

Answer: The centroid is (50 kWh, 3 V) — not any single real reading, but the “typical day” this meter’s normal behavior centers on. Reading P scores 5 and Reading Q scores 41, which is $41 \div 5 = 8.2$ times larger. Reading Q’s usage is nearly double the normal 50 kWh and its deviation is four times the normal 3 V; neither is impossible alone, but together they place Q far outside the normal cloud.

Figure 21.3 draws the same numbers: the two feature differences are the short sides of a right triangle and the score is the long side.

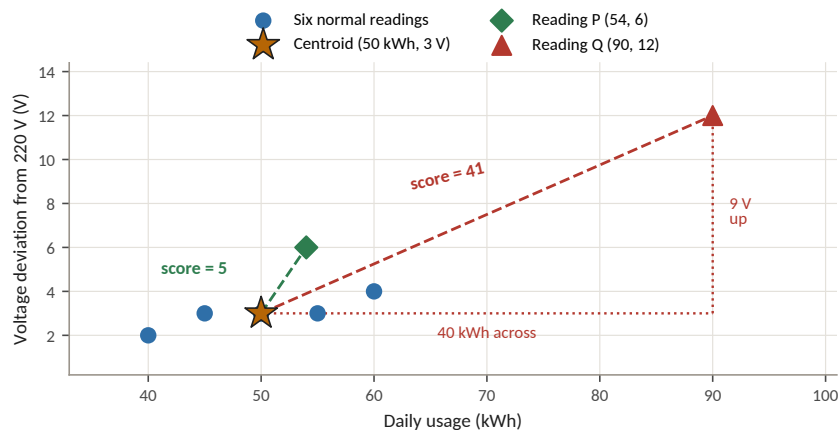


Figure 21.3 – The six normal readings, their centroid, and the two new readings. Reading Q’s score is the long side of a right triangle whose short sides are 40 kWh across and 9 V up.

WHAT TO OBSERVE

Reading P sits just outside the cloud of normal readings and scores 5; Reading Q sits far up and to the right and scores 41. Notice too that the six normal readings are not all at the centroid — normal data is spread out, and that spread is what the next lesson turns into a threshold.

This method is easy to compute and easy to explain. It has two weaknesses, the first being that it treats every feature's raw numbers as equally important.

COMMON MISTAKE

✗ Wrong: Adding up squared differences from features measured on wildly different scales — kilowatt-hours in the tens next to a bill in the tens of thousands of Iraqi dinars.

✓ Right: Put every feature on a shared scale first. If the company added “amount billed”, an ordinary difference of 4,000 IQD would add $4,000^2 = 16,000,000$ to the sum, and the usage and voltage terms — 16 and 9 — would vanish beside it. This is the trap Chapter 11 warned about, and Lesson 21.3 fixes it.

The second weakness is deeper: normal data does not always form one round cloud around one center. Lesson 21.4 shows a case where distance from the center gets the answer exactly backwards.

21.3 z-Scores and Setting a Threshold

Chapter 4 defined the z-score, $z = (x - \mu)/\sigma$: how many standard deviations a value sits from the mean of a trusted reference. There it flagged suspicious values during cleaning and, as *standardization*, rescaled whole columns before training. Here it does a third job with no new mathematics at all — a z-score is an anomaly score for one feature.

EQUATION: Z-SCORE AS A ONE-FEATURE ANOMALY SCORE

$$z = \frac{x - \mu}{\sigma}$$

In words: Subtract the mean of normal data for this feature, then divide by the standard deviation of normal data for this feature. The answer counts standard deviations, so it has no unit.

When it is used: To say how unusual one value of one feature is, on a scale that means the same for every feature — kilowatt-hours, volts, dinars or degrees.

Symbol	Meaning	Unit or type
z	the z-score	no unit (a plain number)
x	the value being checked	unit of the feature
μ	mean of this feature across normal data	unit of the feature
σ	standard deviation of this feature across normal data	unit of the feature

This is exactly the formula of Chapter 4. What changes is the reference: μ and σ must come from data you trust and, wherever there is enough history, from this meter, machine or customer.

Suppose the company keeps a running mean and standard deviation for each meter over many months. For this household, usage has a mean of 50 kWh and a standard deviation of 10 kWh; voltage deviation has a mean of 3 V and a standard deviation of 1 V.

A z-score answers “how unusual”. A working system must also answer “so what?” — flag this reading, or let it pass — and that needs a cutoff. Chapter 6 used the word *threshold* for the probability a classifier must reach before predicting the positive class; here it does the same job for an anomaly score. A common convention

flags any reading with $z \geq 3$ or $z \leq -3$. Figure 21.4 shows why that feels reasonable — and why it is only a convention.

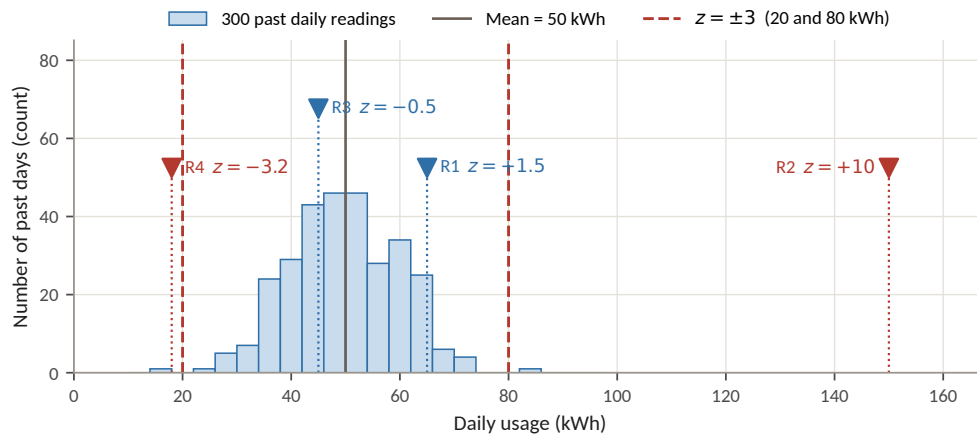


Figure 21.4 – Three hundred past daily readings from one meter, with the lines three standard deviations from the mean. Four new readings, R1 to R4, are marked with their z-scores.

WHAT TO OBSERVE

Almost every one of the 300 past readings falls between the dashed lines at 20 and 80 kWh. R1 and R3 sit comfortably inside; R4 is just outside on the low side; R2 is so far out that the axis has to be stretched to show it. The rule cuts on both sides: an unusually *small* reading is flagged too.

A threshold is a choice, not a fact of nature. A strict one catches only the most extreme cases but misses a real problem sitting just below the cutoff; a loose one catches more real problems but spends a team's limited time on false alarms. Choose it by weighing which mistake costs more — never because a number feels safe.

WORKED EXAMPLE 21.3 — Find Four z-Scores and Apply the $|z| \geq 3$ Rule

Using a mean of 50 kWh and a standard deviation of 10 kWh, find the z-score of four new readings and flag any with $z \geq 3$ or $z \leq -3$. R1: 65 kWh. R2: 150 kWh. R3: 45 kWh. R4: 18 kWh.

1. R1: subtract the mean, $65 - 50 = 15$; divide by the standard deviation, $15 \div 10 = 1.5$. Since 1.5 lies between -3 and 3 , R1 is not flagged.
2. R2: $(150 - 50) \div 10 = 100 \div 10 = 10$. Since $10 \geq 3$, R2 is flagged.
3. R3: $(45 - 50) \div 10 = -5 \div 10 = -0.5$. Since -0.5 lies between -3 and 3 , R3 is not flagged.
4. R4: $(18 - 50) \div 10 = -32 \div 10 = -3.2$. Since $-3.2 \leq -3$, R4 is flagged.

Answer: R2 and R4 are flagged; R1 and R3 are not. R2 sits ten full standard deviations above this meter's average, which is extraordinary. R4 was flagged for being unusually *low* — a bypassed meter under-reports usage, so a good rule must watch both directions.

A single feature's z-score is useful, but Lesson 21.2's strength was combining features, and the same trick works here while fixing the scale problem. After standardizing, a value equal to its own mean has a z-score of $(\mu - \mu)/\sigma = 0$, so the centroid of normal data moves to the origin and distance from the centroid becomes distance from zero.

EQUATION: COMBINED z-SCORE ANOMALY SCORE

$$s(\mathbf{x}) = \sqrt{\sum_{j=1}^p z_j^2}, \quad \text{where} \quad z_j = \frac{x_j - \mu_j}{\sigma_j}$$

In words: Turn every feature into its own z-score, square each one, add the squares, and take the square root. The result is the distance from the center of normal data measured in standard deviations rather than raw units.

When it is used: Whenever a score must combine features on different scales — almost always. It is the fair version of the distance score of Lesson 21.2.

Symbol	Meaning	Unit or type
$s(\mathbf{x})$	combined anomaly score	no unit
z_j	z-score of feature j for this example	no unit
μ_j, σ_j	mean and standard deviation of feature j across normal data	unit of feature j
p	how many features	count

With two features this is the familiar right triangle: two z-scores as the short sides, the score as the long side. The more general version, which also corrects for features that move together, is the Mahalanobis distance, introduced by Prasanta Chandra Mahalanobis in 1936.

WORKED EXAMPLE 21.4 — Combine Two Features' z-Scores into One Score

Usage has $\mu = 50$ kWh and $\sigma = 10$ kWh; voltage deviation has $\mu = 3$ V and $\sigma = 1$ V. Find the combined anomaly score for Reading R: 80 kWh with a voltage deviation of 7 V. The company flags any combined score of 4 or more.

1. Usage z-score: $(80 - 50) \div 10 = 30 \div 10 = 3$.
2. Voltage-deviation z-score: $(7 - 3) \div 1 = 4 \div 1 = 4$.
3. Square each: $3^2 = 9$ and $4^2 = 16$.
4. Add and take the square root: $9 + 16 = 25$, and $\sqrt{25} = 5$.
5. Compare with the threshold: $5 \geq 4$.

Answer: Reading R's combined score is 5, above the threshold of 4, so it is flagged. Neither raw number looked extreme by itself — 80 kWh is not shocking, and 7 V happens — but standardizing both and combining them reveals a reading that does not match this meter's normal pattern at all.

Figure 21.5 shows the same idea twice: in raw units on the left, in z units on the right.

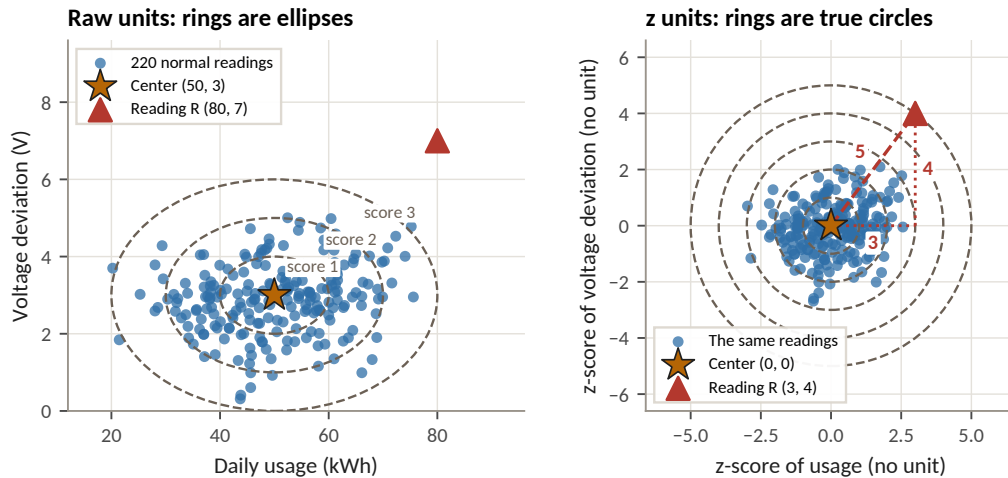


Figure 21.5 – Left: in raw units, readings that share a combined score lie on an ellipse, because one kWh and one volt mean very different amounts of “unusual”. Right: after standardizing, the rings become true circles and Reading R is the familiar 3–4–5 triangle.

WHAT TO OBSERVE

The rings on the left are stretched sideways because usage varies by about 10 kWh from day to day while voltage deviation varies by only about 1 V: 10 kWh of unusualness equals 1 V of it. Standardizing rewrites both axes in those units and the ellipses become circles. Reading R sits three circles out in usage and four out in deviation, giving the score of 5.

COMMON MISTAKE

X Wrong: Computing μ and σ from a handful of rows, or from a batch that already contains the anomaly you are hunting. Fit scikit-learn’s `StandardScaler` on the six rows of Table 21.1 and it reports standard deviations of 6.4550 kWh and 0.5774 V — not the 10 and 1 used above — and an extreme value would inflate the very spread used to judge it.

✓ Right: Estimate μ and σ from a large, trusted stretch of normal history, as the running per-meter figures above were. Chapter 4 showed the same failure for cleaning and offered the IQR fences — built from quartiles, which barely move when one value is wild — as a sturdier alternative when the reference data is small or dirty.

21.4 Local Outlier Factor: Crowded or Alone?

Distance from a center works beautifully when normal data forms one tidy cloud. Real data often has several kinds of normal, and they are not all equally tight.

Eight irrigation pumps along a canal near Zakho report how much water they deliver each day, in cubic meters (m^3). Four are small drip pumps of the same model, delivering 10, 11, 12 and 13 m^3 ; three are large flood pumps of different ages, delivering 40, 50 and 60 m^3 . One pump, E, delivers 25 m^3 — matching neither group. It is either a small pump working far too hard or a large one barely working, and an engineer should look at it. Yet the average of all eight is $221 \div 8 = 27.625 \text{ m}^3$, so pump E sits closer to that average than any other pump: distance from the center would rank it the *most* normal of the eight.

The fix is to stop asking “how far is this point from the center of everything?” and start asking “is this point in a crowd?”. That is the idea behind the best-known local method, published by Markus Breunig, Hans-Peter Kriegel, Raymond Ng and Jörg Sander in 2000. Chapter 19 used a close relative of it, when DBSCAN labeled a point in no crowded region as noise.

WORD TO KNOW

Local Outlier Factor (LOF) — A method that scores an example by comparing how spread out its own neighborhood is with how spread out its neighbors' neighborhoods are. A ratio near 1 means the example is about as crowded as the company it keeps; a ratio well above 1 means it sits in a much emptier pocket than its neighbors do, and is flagged.

The full LOF adds one smoothing step, described in the note below; its core, which you can compute by hand, is a plain ratio of average distances.

EQUATION: LOCAL DENSITY RATIO (THE CORE OF LOF)

$$\text{LDR}_k(\mathbf{x}) = \frac{\bar{d}_k(\mathbf{x})}{\frac{1}{k} \sum_{\mathbf{o} \in N_k(\mathbf{x})} \bar{d}_k(\mathbf{o})}, \quad \text{where} \quad \bar{d}_k(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{o} \in N_k(\mathbf{x})} d(\mathbf{x}, \mathbf{o})$$

In words: Measure how far this example sits from its k nearest neighbors, on average. Then measure the same thing for each of those neighbors. Divide the first by the average of the second.

When it is used: When normal data has several groups of different tightness, so that “far from the middle” and “unusual” are not the same thing.

Symbol	Meaning	Unit or type
$\text{LDR}_k(\mathbf{x})$	local density ratio of example $\mathbf{x}$	no unit
k	how many neighbors to look at, chosen by you	count
$N_k(\mathbf{x})$	the k nearest neighbors of $\mathbf{x}$	set of examples
$\bar{d}_k(\mathbf{x})$	average distance from $\mathbf{x}$ to those k neighbors	unit of the features

scikit-learn's `LocalOutlierFactor` replaces each raw distance by a reachability distance, which never lets a neighbor count as closer than that neighbor's own k th-neighbor distance. On the pumps below it returns 8.33 for pump E where the plain ratio gives 10; the verdict is the same.

Before doing any arithmetic, look at Figure 21.6, which scores all eight pumps both ways.

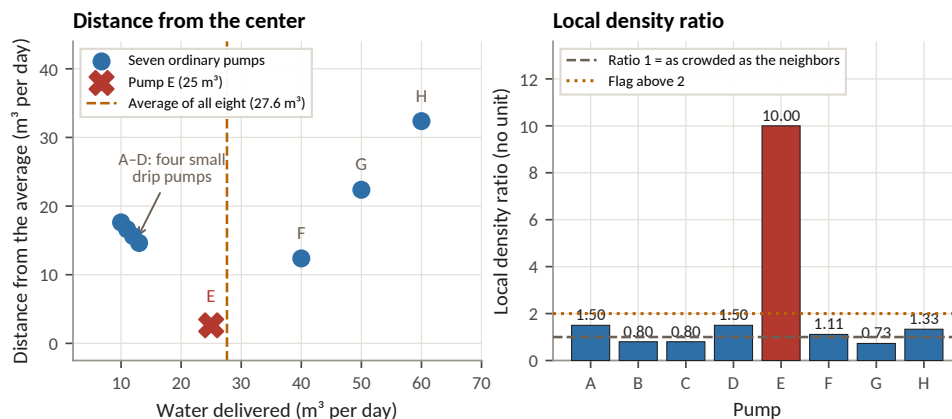


Figure 21.6 – The same eight pumps judged two ways: distance from the overall average, and the local density ratio.

WHAT TO OBSERVE

On the left, pump E's distance from the average is only 2.6 m^3 , the smallest of all eight, while the honest pump H is 32.4 m^3 away and looks worst. On the right the order reverses: tight and loose pumps alike land near a ratio of 1, and only pump E rises to 10. The ratio does not care how big the gaps are, only whether they are bigger than the neighbors'.

WORKED EXAMPLE 21.5 — Compare Local Crowding for Three Irrigation Pumps

The eight pumps deliver $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 25$, $F = 40$, $G = 50$ and $H = 60 \text{ m}^3$ per day. Using $k = 2$ neighbors, find the local density ratio of pump C (inside the tight group), pump E (alone in the gap) and pump G (inside the loose group).

1. Pump C's two nearest neighbors are B, 1 m^3 away, and D, 1 m^3 away, so $\bar{d}(C) = (1 + 1) \div 2 = 1.0$.
2. Those neighbors' own spreads: B's nearest are A (1) and C (1), so $\bar{d}(B) = 1.0$; D's nearest are C (1) and B (2), so $\bar{d}(D) = 1.5$. Their average is $(1.0 + 1.5) \div 2 = 1.25$.
3. Pump C's ratio: $1.0 \div 1.25 = 0.80$.
4. Pump E's two nearest neighbors are D, $25 - 13 = 12 \text{ m}^3$ away, and C, $25 - 12 = 13 \text{ m}^3$ away, so $\bar{d}(E) = (12 + 13) \div 2 = 12.5$.
5. Those neighbors' spreads are $\bar{d}(D) = 1.5$ and $\bar{d}(C) = 1.0$, averaging 1.25 again.
6. Pump E's ratio: $12.5 \div 1.25 = 10.0$.
7. Pump G's two nearest are F (10) and H (10), so $\bar{d}(G) = 10.0$. F's nearest are G (10) and E (15), giving 12.5; H's nearest are G (10) and F (20), giving 15.0; their average is 13.75.
8. Pump G's ratio: $10.0 \div 13.75 = 0.73$ (to two decimal places).

Answer: Pump C scores 0.80, pump G scores 0.73 and pump E scores 10.0. C and G both sit about as close to their neighbors as those neighbors sit to theirs, even though G's neighbors are ten times further away than C's. Pump E is ten times more spread out than the company it keeps, and is the one to inspect.

Three practical points follow. k matters: too small and the ratio jumps with every stray point, too large and a genuine little group of anomalies looks like its own normal neighborhood; $k = 20$ is a common start. LOF needs distances, so scale every feature first, as in Chapter 11. And do not read a ratio of 1.3 as "30% anomalous": ordinary points land a little either side of 1 — pumps A, D and H sit at 1.50, 1.50 and 1.33 simply because they are at the edges of their groups. Look for ratios far above the rest, the way pump E's 10 is.

21.5 Isolation Forest: Anomalies Are Easy to Isolate

Every method so far builds a picture of normal and measures distance from it. The isolation forest, introduced by Fei Tony Liu, Kai Ming Ting and Zhi-Hua Zhou in 2008, throws that plan away and starts from a different observation: *unusual points are easy to cut off from everything else*.

Take a dataset and cut it in two with a straight line drawn at a random place along a randomly chosen feature. Then cut the piece your point of interest fell into, again at random, and keep going until that point sits alone. A point in the middle of a crowd survives many cuts, because almost every random line has other points on both sides. A point far out on its own is separated almost immediately, because most random lines fall between it and the crowd. Figure 21.7 shows both cases on the same readings.

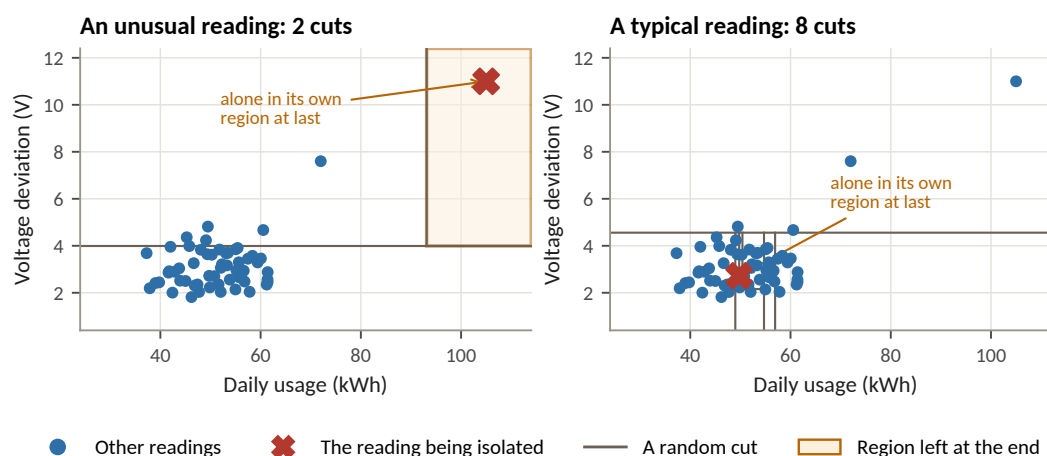


Figure 21.7 – Random cuts on the same 57 meter readings. Left: two cuts are enough to leave the unusual reading alone. Right: the same procedure needs eight cuts to isolate a reading from the middle of the crowd.

WHAT TO OBSERVE

On the left the very first cut, at 4 V, separates the top of the plot from everything below, and one more finishes the job. On the right the cuts keep landing inside the crowd, each shaving off a few readings. Nothing here counts distances or averages — only how many cuts were needed.

WORDS TO KNOW

Isolation tree — One sequence of random cuts, each on a randomly chosen feature at a randomly chosen value, continued until every example sits alone. With no labels and no purity measure, the tree of Chapter 13 becomes far simpler here: the splits are pure chance.

Isolation depth — The number of cuts needed before one example is alone in its own region. Also called the *path length*, because it counts the steps from the top of the tree down to that example's leaf.

Isolation forest — Many isolation trees built on random subsamples of the data. An example's anomaly score comes from its average isolation depth across all the trees: the shallower the average, the more unusual.

One tree is pure luck, exactly as one decision tree was in Chapter 14, and the cure is the same: build many and average. Random forests averaged *predictions*; an isolation forest averages *depths*.

EQUATION: AVERAGE ISOLATION DEPTH

$$\bar{h}(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x})$$

In words: Count how many random cuts each tree needed to leave this example alone, add those counts across all the trees, and divide by the number of trees. A small average means the example is easy to isolate, so it is unusual.

When it is used: To score examples when features are on different scales, when there are many features, or when the data is too large to compare every pair of points.

Symbol	Meaning	Unit or type
$\bar{h}(\mathbf{x})$	average isolation depth of example $\mathbf{x}$	count of cuts
$h_t(\mathbf{x})$	depth of $\mathbf{x}$ in tree t	count of cuts
T	number of trees in the forest	count

Careful: here a small number means more unusual, the opposite of every other score in this chapter. *scikit-learn* hides this by reporting $s = 2^{-\bar{h}/c(n)}$, where $c(n)$ is the depth expected by chance for n examples; that score rises toward 1 as an example gets stranger.

One meter's last eight daily usage readings are 40, 44, 46, 48, 50, 52, 56 and 120 kWh; call the 48 kWh one "Reading N" and the 120 kWh one "Reading Q". Table 21.2 lists six isolation trees as the split values each drew; a reading goes left if it is at most the split value, right if greater.

Table 21.2 – Six isolation trees on the eight readings, and the depth each one reached

Tree	Random split values, top down	Depth of N (48 kWh)	Depth of Q (120 kWh)
1	85, 47, 53, 49	4	1
2	70, 51, 47, 49	4	1
3	45, 100, 49, 47	4	2
4	95, 49, 47	3	1
5	100, 43, 45, 47, 49	5	1
6	42, 90, 47, 49	4	2

WORKED EXAMPLE 21.6 — Find the Average Isolation Depth of Two Readings

Using the six trees of Table 21.2, find the average isolation depth of Reading N (48 kWh) and Reading Q (120 kWh), and say which is more unusual.

1. Check tree 1 for Reading Q. Its first split value is 85: the seven readings from 40 to 56 go left, and 120 goes right by itself. Q is alone after 1 cut.
2. Check tree 1 for Reading N. Split at 85 keeps N with six others; split at 47 leaves N with 50, 52 and 56; split at 53 leaves N with 50 and 52; split at 49 leaves N alone. That is 4 cuts.
3. Collect N's depths from all six trees: 4, 4, 4, 3, 5, 4. Add them: $4 + 4 + 4 + 3 + 5 + 4 = 24$.
4. Divide by the number of trees: $24 \div 6 = 4.00$.
5. Collect Q's depths: 1, 1, 2, 1, 1, 2. Add them: $1 + 1 + 2 + 1 + 1 + 2 = 8$.
6. Divide by the number of trees: $8 \div 6 = 1.33$ (to two decimal places).
7. Compare: $4.00 \div 1.33 = 3$, so N sits three times deeper in the trees than Q.

Answer: Reading N needs 4.00 cuts on average and Reading Q only 1.33, so Q is the anomaly. Four times out of six a single random cut was enough to leave it alone, because almost any split value between 56 and 120 separates it from every other reading.

Three practical notes finish the method. An isolation forest never measures a distance, so features on different scales cause no trouble and standardizing is optional. Each tree uses a small random subsample — *scikit-learn*'s default is 256 examples — which makes it fast on millions of rows. And contamination moves only the cutoff, not the scores.

COMMON MISTAKE

✗ Wrong: Reading $\text{contamination}=0.05$ as a claim that 5% of the data really is anomalous, and trusting the flags that follow.

✓ Right: Treat it as an alert budget — “flag me the most unusual 5% so my team has a list to work through”. It sets how many alerts you get, not how many problems exist. Lesson 21.7 makes that idea explicit.

21.6 One-Class SVM in Brief

Chapter 16 built a classifier by finding the widest possible street between two classes. Anomaly detection has only one class, so the question changes: instead of the widest street between two groups, find the tightest boundary that still wraps almost all of the normal data. Anything outside is flagged. The method that does this, the one-class support vector machine, was introduced by Bernhard Schölkopf and colleagues in 1999.

WORD TO KNOW

One-class SVM — A support vector machine trained on normal examples only. It learns a boundary around them, using the kernel trick of Chapter 17 so the boundary can curve. A new example inside the boundary is normal; one outside is flagged, and how far outside it lies is its anomaly score.

Two settings control it. ν (nu) is a number between 0 and 1 setting an upper limit on the fraction of training examples the boundary may leave outside: with $\nu = 0.05$ and 400 training readings, at most $0.05 \times 400 = 20$ may fall outside. It is the alert budget again, chosen before training instead of after. γ (gamma) is the reach of the RBF kernel from Chapter 17: a small γ gives a smooth, loose boundary and a large one wraps tightly around every little clump. Figure 21.8 shows what changing ν alone does.

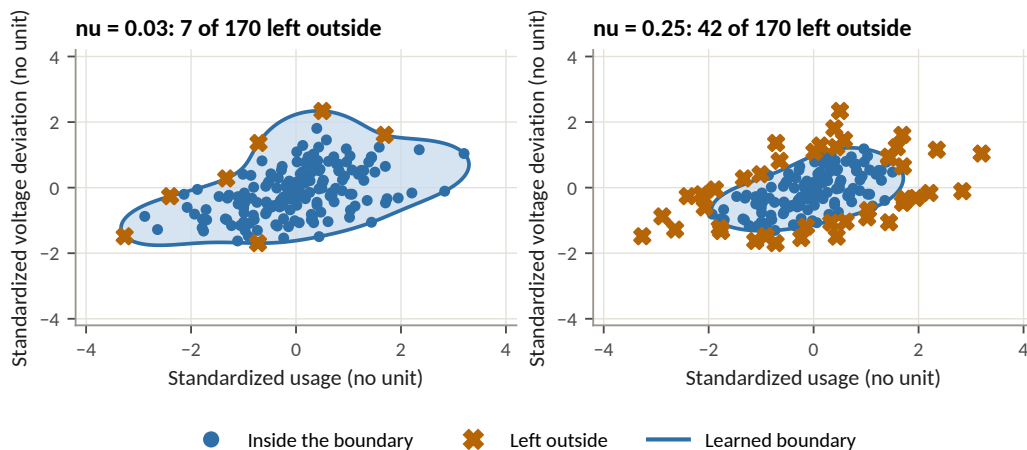


Figure 21.8 – One-class SVM boundaries on the same 170 standardized readings. Only ν changes between the panels; γ is 0.35 in both.

WHAT TO OBSERVE

With $\nu = 0.03$ the boundary stretches to include almost everything and leaves just 7 of the 170 readings outside. With $\nu = 0.25$ it pulls in tight and leaves 42 outside — about a quarter, as promised. Neither is right or wrong: ν is where you tell the method how many alerts you want.

A one-class SVM can wrap a curved region that no ellipse would fit, and it gives a definite yes-or-no rule for a new reading. The costs are real too: training time grows roughly with the square of the number of examples,

so it slows beyond a few tens of thousands of rows; every feature must be scaled; and a few anomalies mixed into training stretch the boundary out to include them, which an isolation forest tolerates far better. So standardize first (Chapter 4) and fit on data an expert has confirmed is clean. A close relative, the *support vector data description* of David Tax and Robert Duin (2004), wraps the data in the smallest enclosing sphere and behaves almost identically with an RBF kernel.

21.7 Evaluating a Detector and Acting on Alerts

Here is the awkward question that ends every anomaly detection project: if nobody labeled the anomalies, how do you know the detector works? Three honest answers get used. Check that it ranks whatever handful of past problems the team recorded near the top. Or plant them: inject a few artificial anomalies of the kind you fear and see how many come back, as the Python activity does. Or — the answer that decides whether a system survives its first year — measure what happens after the alerts go out, starting from a number that has nothing to do with statistics: how many alerts can the team handle?

WORDS TO KNOW

Alert budget — The number of examples a team can genuinely investigate in a given period — ten meter visits a day, fifty transaction reviews an hour. It is set by people and money, not by the data.

Precision at k (Precision@ k) — Of the top k examples on the alert list, the fraction that turn out to be real problems. It is the precision of Chapter 6, measured only where the team is actually looking.

EQUATION: ALERT BUDGET AND PRECISION AT THE TOP

$$m = \lceil q \times n \rceil, \quad \tau = s_{(m)}, \quad \text{Precision@}k = \frac{a_k}{k}$$

In words: To flag the most unusual q of the data, work out how many examples that is, sort every score from largest to smallest, and take the m th largest as the threshold. Afterward, count how many of the top k alerts turned out to be real problems and divide by k .

When it is used: The first pair whenever a fixed number of alerts must be produced each day; the last once some alerts have been investigated and the answers are known.

Symbol	Meaning	Unit or type
q	the alert budget, as a fraction of all examples	no unit
n	how many examples were scored	count
m	how many examples get flagged, rounded up	count
$s_{(m)}, \tau$	the m th largest score, used as the threshold: flag every example with $s \geq \tau$	same as the score
a_k	confirmed real problems among the top k alerts	count

Setting the threshold from a budget instead of from the score itself keeps the workload steady even when scores drift upward together, for instance during a heat wave when every meter's usage rises.

WORKED EXAMPLE 21.7 — Set a Threshold from an Alert Budget

A district has 200 meters, all scored this morning. The twelve highest scores are 12.4, 9.8, 8.6, 7.1, 6.3, 5.9, 5.4, 5.0, 4.6, 4.2, 3.9 and 3.5; the other 188 are all below 3.0. Find the threshold for an alert budget of 1% of meters, and then for 5%.

1. For the 1% budget, find how many meters that is: $0.01 \times 200 = 2$ meters.
2. Sort the scores from largest to smallest and take the 2nd largest: $\tau = 9.8$.
3. Check: the scores at or above 9.8 are 12.4 and 9.8 — exactly 2 meters.

4. For the 5% budget: $0.05 \times 200 = 10$ meters.
5. Take the 10th largest score: $\tau = 4.2$.
6. Check: the ten scores from 12.4 down to 4.2 are at or above the threshold, and the eleventh, 3.9, is not.

Answer: A 1% budget gives a threshold of 9.8 and 2 alerts; a 5% budget gives 4.2 and 10 alerts. The threshold was never chosen for statistical reasons at all — it was read off the sorted list once the team said how many visits they could make.

Figure 21.9 draws the same 200 scores as a histogram, with both thresholds marked.

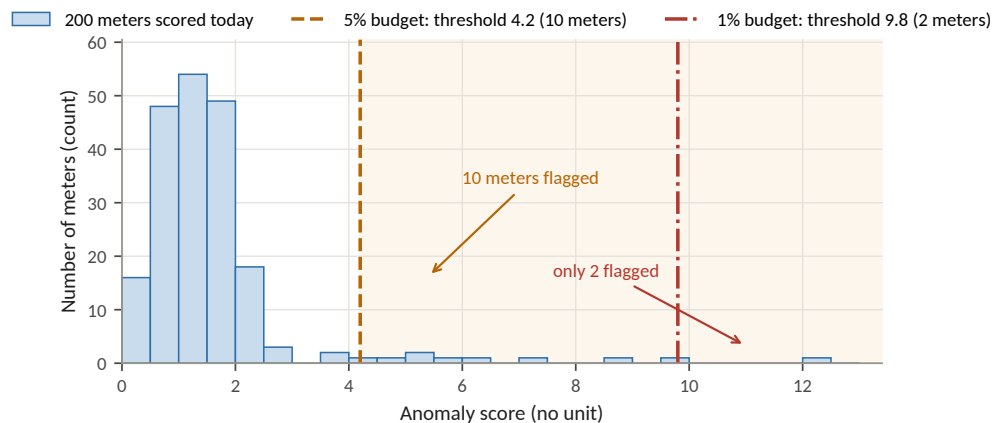


Figure 21.9 – Two hundred meters scored in one district. The two vertical lines are the thresholds produced by a 5% budget and by a 1% budget.

WHAT TO OBSERVE

Almost every meter scores below 3, and the scores thin out into a long tail. Moving the line from 9.8 to 4.2 costs eight extra visits and buys eight extra chances to find a real problem. There is no natural gap in the tail — which is why the budget, not the histogram, decides where the line goes.

Once inspectors return, the list can finally be judged.

WORKED EXAMPLE 21.8 — Measure Precision at the Top of the List

Inspectors visit the top 10 meters on today's list. Seven turn out to have a real fault or a tampered connection; the other three have honest explanations. Across the network, about 2% of meters have a real problem at any time. Find the precision at 10 and compare it with random visiting.

1. Count the confirmed problems among the top 10: $a_{10} = 7$.
2. Compute precision at 10: $7 \div 10 = 0.70$, or 70%.
3. Work out what 10 random visits would find: $0.02 \times 10 = 0.2$ problems on average.
4. Compare the two: $0.70 \div 0.02 = 35$.
5. Note the cost as well: $10 - 7 = 3$ visits found nothing wrong.

Answer: Precision at 10 is 0.70, or 70%. The detector concentrates real problems 35 times more densely than random visiting would — the number that justifies the project to a manager. The three visits that found nothing are not failures; they are the price of the seven that succeeded, and they should be counted openly.

Table 21.3 compares the four detectors of this chapter, and Figure 21.10 runs all four on one deliberately awkward dataset: a tight cluster, a loose cluster, four far-away readings, and one reading alone in the gap between the clusters.

Table 21.3 – Choosing between the four anomaly detectors

Method	What it assumes normal looks like	Main settings	Cost and cautions
Combined z-score	One roughly round cloud around one center	Threshold; a trusted μ and σ per feature	Very fast, easy to explain; needs scaling; blind to local structure
Local Outlier Factor	Any number of groups, each with its own tightness	k , the number of neighbors	Compares many pairs, so it slows on large data; needs scaling
Isolation Forest	Anything — it makes no shape assumption	Trees, subsample size, contamination	Fast on millions of rows; no scaling needed; weaker on purely local anomalies
One-class SVM	One connected region, possibly curved	ν and γ	Slow beyond tens of thousands of rows; needs scaling and clean training data

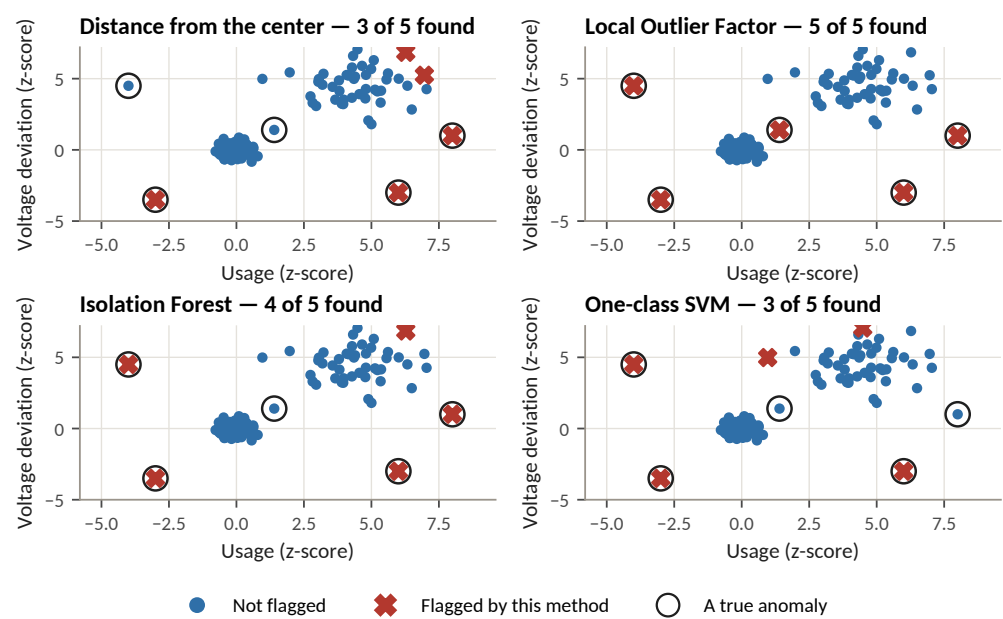


Figure 21.10 – The same 140 readings, five of them genuinely anomalous (black rings), scored by four methods. Each was allowed about five flags.

WHAT TO OBSERVE

Distance from the center finds 3 of the 5. It misses the gap reading completely — the average of all 140 readings sits at about (1.5, 1.5), right on top of it — and wastes two flags on honest members of the loose cluster, whose only crime is being far from the middle. The Local Outlier Factor finds all 5 with no

false alarms, judging each reading against its own neighborhood. The isolation forest finds 4, missing the same gap reading; the one-class SVM finds 3.

There is no permanent winner. LOF wins here because the difficulty was designed to be local; on a million rows with forty features it would be far too slow and the isolation forest would win easily. So score with two or three methods and treat a reading all of them flag as the strongest candidate. Two habits then keep a live system healthy: *re-fit* it regularly, because normal drifts with the seasons and with every new factory, and *explain* every alert — “usage 3 standard deviations high, deviation 4 high” tells an inspector what to look at, where a bare score tells them nothing. Chapter 24 returns to both.

RESPONSIBLE USE

A high anomaly score is a reason to look closer, never proof that anything is wrong. A flagged meter, a flagged transaction and a flagged medical test all deserve a quick, respectful human review before any serious action such as cutting off a supply, blocking a card or shutting down a machine. Anomaly detection exists to point limited human attention in a useful direction. It is not a judge, and being unusual is not an offense.

COMMON MISTAKES AND MISCONCEPTIONS

- Assuming every anomaly is a mistake or a fraud, instead of a candidate deserving a closer look, or deciding an example is normal because each feature looks ordinary alone, without checking the whole combination.
- Adding squared differences from features on very different scales, letting the feature with the biggest raw numbers dominate the score.
- Computing the reference μ and σ from a handful of rows, or from a batch that already contains the anomaly, so the anomaly inflates the very spread used to judge it — or comparing every example with one network-wide average instead of a baseline built for each meter, machine or customer.
- Treating a z-score threshold of 3, or any other borrowed number, as a law rather than a starting convention weighed against the cost of each mistake.
- Believing a point far from the middle must be the anomaly — Lesson 21.4 shows the only real anomaly sitting almost exactly at the average — or forgetting that a small average isolation depth means *more* unusual, the opposite of every other score here.
- Leaving a detector fitted once and never re-fitted, so ordinary seasonal drift turns it into a false-alarm machine.

Practical Activities

ACTIVITY (PAPER) — z-Scores and Thresholds for a Bakery's Meter

A bakery in Sulaymaniyah has a smart meter whose daily usage has averaged 120 kWh over two years, with a standard deviation of 15 kWh. Eight new daily readings arrive, in kWh: 117, 132, 90, 126, 171, 114, 108 and 63.

1. Make a table with the eight readings and three empty columns: z-score, flagged at $|z| \geq 3$, flagged at $|z| \geq 2$.
2. Compute each z-score using $z = (x - 120) \div 15$. Every answer comes out to one decimal place; check that 117 gives -0.2 .
3. Fill in the two flag columns. How many readings does each rule flag? The owner can investigate one day per week — which threshold suits that budget, and what does the other one cost or risk?

4. The reading of 63 kWh fell on a public holiday when the bakery was closed. Write one sentence explaining why this is a *contextual* anomaly rather than a fault, and name one extra feature that would have let a detector work this out by itself.

You need: Paper, pencil and a calculator

ACTIVITY (PYTHON) — Four Detectors, Three Planted Anomalies

This program builds 97 normal readings in two groups — 70 tightly clustered apartments and 27 loosely spread workshops — then plants three anomalies and asks four detectors to rank them. A is far from everything; B has an apartment's usage with a workshop-sized voltage deviation; C sits alone in the gap between the groups.

Python · NumPy · scikit-learn

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(21)
tight = rng.normal([50, 3], [3, 0.4], (70, 2)) # 70 apartments (kWh, V)
loose = rng.normal([90, 6], [9, 1.2], (27, 2)) # 27 workshops, far wider spread
# A far from everything; B apartment usage, huge deviation; C alone in the gap
X = np.vstack([tight, loose, [[150.0, 12.0], [50.0, 8.5], [70.0, 4.5]]])
Xs = (X - X[:97].mean(axis=0)) / X[:97].std(axis=0) # scale on the normal rows

lof = LocalOutlierFactor(n_neighbors=20).fit(X)
scores = {"z-score": np.sqrt((Xs ** 2).sum(axis=1)),
          "IsolForest": -IsolationForest(random_state=0).fit(X).score_samples(X),
          "LOF": -lof.negative_outlier_factor_,
          "OneClassSVM": -OneClassSVM(nu=0.1, gamma=0.5).fit(Xs).decision_function(Xs)}
for name, s in scores.items():
    rank = {int(j): r for r, j in enumerate(np.argsort(s)[::-1], start=1)}
    hits = sum(rank[i] <= 3 for i in (97, 98, 99))
    print("%-12s A:%3d B:%3d C:%3d -> %d of 3 in the top 3"
          % (name, rank[97], rank[98], rank[99], hits))
```

Output

```
z-score      A:  1 B:  2 C: 82 -> 2 of 3 in the top 3
IsolForest   A:  1 B:  2 C:  6 -> 2 of 3 in the top 3
LOF          A:  1 B:  3 C:  2 -> 3 of 3 in the top 3
OneClassSVM  A:  1 B:  2 C:  6 -> 2 of 3 in the top 3
```

Every score is arranged so larger means stranger, which is why three are negated: `score_samples`, `negative_outlier_factor_` and `decision_function` all return *lower* values for stranger examples. All four put A first and B in the top three. Only the Local Outlier Factor finds C: the combined z-score ranks it 82nd out of 100, because C really does sit close to the middle of everything. Try three changes and watch the ranks move: set `n_neighbors=3`; move C to `[70.0, 8.0]`; add ten more anomalies around `[70.0, 4.5]` so C is no longer alone.

You need: A computer with Python, NumPy and scikit-learn installed (see Appendix B)

CASE STUDY — Flagging Unusual Smart-Meter Readings — Kirkuk

Zainab works on the analytics team of a regional electricity distribution company serving Kirkuk and the towns around it. Its smart meters report daily usage and voltage deviation for hundreds of thousands of households and small businesses. A small fraction are genuinely faulty; a smaller fraction have been deliberately bypassed so that usage goes unbilled. Visiting every meter is impossible, and waiting for complaints finds only the faults that inconvenience somebody. Zainab's job is to decide which meters deserve a visit first.

She follows this chapter's method with one refinement. Instead of comparing every meter with one company-wide average, she builds a baseline for each meter from its own history over months the maintenance team confirmed were problem-free: a one-room apartment and a busy workshop have very different honest normals, and one shared average would bury the workshop's real problems while flagging ordinary apartments week after week. Each day she turns that meter's two features into z-scores against its own μ and σ and combines them, exactly as Worked Example 21.4 did, and also scores every meter with an isolation forest.

Choosing the threshold turns out to be the hardest part, and it is not a mathematical decision at all. A strict cutoff misses several meters the crew already suspects; a loose one catches those and also flags hundreds of households whose only crime was a wedding party or a new air conditioner. Zainab's team stops arguing about the number and asks the crew a different question: how many visits can you make in a week? The answer is about sixty. Sixty becomes the alert budget, the threshold becomes whatever score sits sixtieth on the sorted list, and the workload stays steady even in August when every score in the network rises together.

The finished system accuses nobody. It produces a ranked list, most unusual first, with a note on each meter saying which feature drove the score, and a senior analyst reviews it before any inspector is dispatched. After three months a little over half of the meters visited had a genuine fault or a tampered connection, far above the roughly two in a hundred that random visiting achieved. Two categories of false alarm were removed by adding features rather than by moving the threshold: a "day of the week" feature stopped Friday closures being flagged, and a neighborhood temperature feature stopped heat waves lighting up whole districts. Inspectors at a flagged home most often find an ordinary explanation and close the case with a friendly conversation; where it is not innocent, the company now finds the problem in weeks rather than months.

CHAPTER SUMMARY

- An anomaly is an example that looks wrong once all its features are considered together; Chapter 4 treated such values as dirt to be cleaned, while this chapter hunts them.
- Anomalies come in three kinds: point (unusual against everything), contextual (unusual for its time, place or owner) and collective (unusual only as a group). Every method produces a score that rises the less an example belongs, and a threshold turns it into a flag.
- The simplest score is the Euclidean distance to the centroid of normal data, $s(\mathbf{x}) = \sqrt{\sum_j (x_j - c_j)^2}$.
- Raw distances are unfair when features use different scales, so the combined z-score $s(\mathbf{x}) = \sqrt{\sum_j z_j^2}$ measures distance in standard deviations instead; the reference μ and σ must come from a large, trusted stretch of normal data. The Local Outlier Factor instead compares an example's own crowding with its neighbors', so it can flag a point in a sparse pocket even when that point sits close to the overall average.
- An isolation forest scores an example by its average isolation depth across many trees of random cuts; a small average depth means easy to isolate, and therefore unusual. A one-class SVM instead wraps normal data in a boundary whose tightness is set by ν and whose shape by γ ; it is powerful but slow, and needs scaled and clean training data.

- A threshold is usually best set from an alert budget — flag the top $m = \lceil q \times n \rceil$ scores — rather than from the shape of the score distribution, and precision at k is the honest way to judge the result once inspectors have reported back.
- An anomaly score is a reason to look closer, never proof of a problem; every flagged example deserves a fast, respectful human review.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define *anomaly* in your own words and explain how it differs from the *outlier* of Chapter 4, with an example where every feature is ordinary but the combination is not.

Problem 2 [Basic] Five delivery times, in minutes, are 22, 25, 20, 24 and 95. Which is the anomaly, and roughly how many times larger is it than the next-highest time?

Problem 3 [Basic] Say whether each is a point, contextual or collective anomaly: (a) a bank card used in two countries within an hour; (b) 200 kWh used by an office at 3 a.m.; (c) a sensor reporting exactly 21.0 °C forty times in a row; (d) a water reading of 9,000 m³ where the usual is 30 m³.

Problem 4 [Basic] Explain in one or two sentences what a threshold does once an anomaly score has already been computed, and give one reason a team might deliberately choose a loose one.

Problem 5 [Standard] The centroid of a set of normal machine readings is (temperature 70 °C, vibration 2 mm/s). (a) Explain in words what this centroid represents and where it came from. (b) A new reading is (74, 5): find its anomaly score using the distance-from-centroid formula, showing every step.

Problem 6 [Standard] A feature has a training mean of 100 and a training standard deviation of 20. Find the z-score of a new value of 150 and of a new value of 34, and say which of the two the rule $|z| \geq 3$ flags.

Problem 7 [Standard] A transaction has a z-score of +3 for its amount and +4 for its distance from the cardholder's home city. Find its combined anomaly score, showing every step, and say whether a threshold of 4 would flag it.

Problem 8 [Standard] Four points sit on a line at 5, 6, 7 and 20. Using $k = 2$ neighbors, find $\bar{d}_2$ for each point, then the local density ratio of the point at 20.

Problem 9 [Standard] An isolation forest of five trees isolates reading X at depths 2, 1, 3, 1 and 3, and reading Y at depths 7, 9, 8, 6 and 10. Find each average isolation depth, say which is more unusual, and explain why the smaller number means stranger.

Problem 10 [Standard] A team scores 4,000 sensors and can investigate 2% of them. (a) How many alerts is that? (b) The 79th, 80th and 81st largest scores are 6.6, 6.5 and 6.5. What threshold should the team use, and what awkward situation does the repeated 6.5 create? (c) A second team inspects 15 pumps a day out of 1,200: what alert budget, as a percentage, is that?

Problem 11 [Challenge] Eight machines report a vibration reading, in mm/s: 2.0, 2.1, 2.2, 2.3, 5.0, 9.0, 10.0 and 11.0. (a) Find the mean of all eight and show that the machine reading 5.0 is closer to that mean than any other. (b) Using $k = 2$, compute the local density ratio of the machine reading 5.0. (c) Explain in two sentences why the two methods disagree, and which answer you would give the maintenance team.

Problem 12 [Challenge] A hospital wants to flag unusual overnight vital-sign readings for a quick nurse check. (a) Name two features and why. (b) Explain why the baseline should probably be built per patient rather than once for the whole ward. (c) Give one risk of a threshold that is too strict and one of a threshold that is too loose, and say which you would accept.

Problem 13 [Challenge] A factory's detector was fitted in winter and never changed. Through the summer the daily alert count climbs from 12 to 140, although no machine has broken. (a) Explain what happened, using the word *drift*. (b) Why would setting the threshold from an alert budget have hidden the symptom without fixing the cause? (c) Suggest one feature that would let the detector handle summer correctly.

Problem 14 [Challenge] You must choose a detector for 3 million smart meters with 25 features each, refreshed daily, where the training history is known to contain some undetected tampering. Using Table 21.3, name the method you would try first and the one you would rule out, justifying both.

PART FIVE

BUILDING RELIABLE AND RESPONSIBLE MODELS

Feature Engineering and Feature Selection

A good feature is worth more than a fancy model

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a well-chosen feature often helps more than a more complicated model.
- Build ratio, difference, date and interaction features by hand from raw columns.
- Straighten a skewed column with a log transform, cut a numeric column into bins, and replace a crowded category column with a frequency encoding.
- Choose a smaller feature set using a correlation filter, forward selection and feature importance.
- Read a feature importance chart carefully, especially when two features carry the same information.
- Check every engineered feature for leakage before it reaches a model.

Introduction

Two teams are given the same spreadsheet of wheat records from farms near Zakho and the same task: predict how much grain each field will produce. The first team spends three weeks trying model after model. The second spends one afternoon with a farmer, learns that what matters is how much water each kilogram of fertilizer gets, adds one column that divides rainfall by fertilizer, and trains the simplest model in the book. The second team wins.

This is not a trick. A model can only find patterns inside the exact numbers you hand it, so when the useful pattern lives in the *relationship between* two columns rather than in either column alone, handing the model that relationship already worked out beats hoping a fancier model will find it unaided.

This chapter covers the two halves of that work. Feature engineering *creates* new columns from the ones you have: ratios, gaps between dates, products, bins, logs and encodings. Feature selection *removes* the columns that did not earn their place. Both use arithmetic you can do with a pencil, and both are among the highest-value hours you will spend on a project.

22.1 A Good Feature Is Worth More Than a Fancy Model

The Zakho Farmers' Cooperative keeps a record book for every wheat field its members farm: the village, the total rainfall in millimeters, the fertilizer in kilograms, the area in dunams (one dunam is 2,500 square meters), the planting and harvest dates and, at the end of the season, the yield in tons per hectare. Table 22.1 shows six of those fields, and every worked example in the first half of this chapter uses them.

Table 22.1 – Six wheat fields from the cooperative's record book, exactly as they are written down

Field	Village	Rain (mm)	Fert. (kg)	Area (dunam)	Planted (day)	Harvested (day)	Yield (t/ha)
A	Zakho Center	300	30	40	70	215	2.0
B	Batifa	300	60	12	95	218	1.2
C	Zakho Center	600	60	90	68	231	2.0
D	Faysh Khabur	600	30	25	110	245	2.8
E	Zakho Center	450	45	60	74	227	2.4
F	Levo	450	45	8	91	208	1.9

Diyar, the cooperative's new analyst, trains a model directly on these raw columns and is disappointed: the predictions are barely better than guessing each village's average yield from past years. The columns are not wrong. They are just not yet asking the model the kind of question it is good at answering.

WORDS TO KNOW

Feature engineering — Creating new features from a dataset's existing columns, using arithmetic and knowledge of the subject, so that a model can find a useful pattern more easily. Good feature engineering can let a simple model match or beat a complicated one.

Domain knowledge — What the people who actually do the work already understand — a farmer, a nurse, a mechanic, a shopkeeper. It tells you which combinations of columns are worth building, long before any model is trained.

The raw material for this work is rarely more mathematics; it is usually a conversation. When Diyar asks an experienced farmer why two fields with the same rainfall gave such different harvests, the answer arrives in one sentence: "It is not the rain, it is how much rain each bag of fertilizer gets." That sentence is a recipe for a new column. The two simplest recipes divide one column by another, or subtract one column from another.

EQUATION: RATIO AND DIFFERENCE FEATURES

$$r = \frac{a}{b} \quad d = a - b$$

In words: A ratio feature divides one column by another, giving "how much of a per unit of b ". A difference feature subtracts one column from another, giving the gap between them.

When it is used: Whenever two raw columns only make sense together: rain per kilogram of fertilizer, price per square meter, days between two dates.

Symbol	Meaning	Unit or type
a	the first raw column	its own unit
b	the second raw column	its own unit
r	the new ratio feature	unit of a per unit of b
d	the new difference feature	the shared unit of a and b

A ratio needs b to be non-zero, and dividing must make sense: subtracting two dates is safe, subtracting a price from a weight is not.

WORKED EXAMPLE 22.1 — See How a Ratio Reveals a Pattern the Raw Numbers Hide

Fields A, B, C and D of Table 22.1 report rainfall, fertilizer and yield. Field A: 300 mm and 30 kg, yield 2.0 t/ha. Field B: 300 mm and 60 kg, yield 1.2 t/ha. Field C: 600 mm and 60 kg, yield 2.0 t/ha. Field D: 600 mm and 30 kg, yield 2.8 t/ha. Find the rainfall-to-fertilizer ratio of each field and compare the four ratios with the four yields.

1. Field A's ratio: $300 \div 30 = 10$ mm per kg.
2. Field B's ratio: $300 \div 60 = 5$ mm per kg.
3. Field C's ratio: $600 \div 60 = 10$ mm per kg.
4. Field D's ratio: $600 \div 30 = 20$ mm per kg.
5. Line the ratios up against the yields: $5 \rightarrow 1.2$, $10 \rightarrow 2.0$, $10 \rightarrow 2.0$, $20 \rightarrow 2.8$. The smallest ratio goes with the smallest yield, the largest with the largest, and fields A and C — whose raw numbers share nothing — have the same ratio 10 and the same yield 2.0.

Answer: The rainfall-to-fertilizer ratio lines up with yield far more cleanly than either raw column does alone. A model given only the two raw columns would have to discover the division for itself; a model given the ratio can use it immediately.

Figure 22.1 draws the same four fields twice: once against raw rainfall, and once against the engineered ratio.

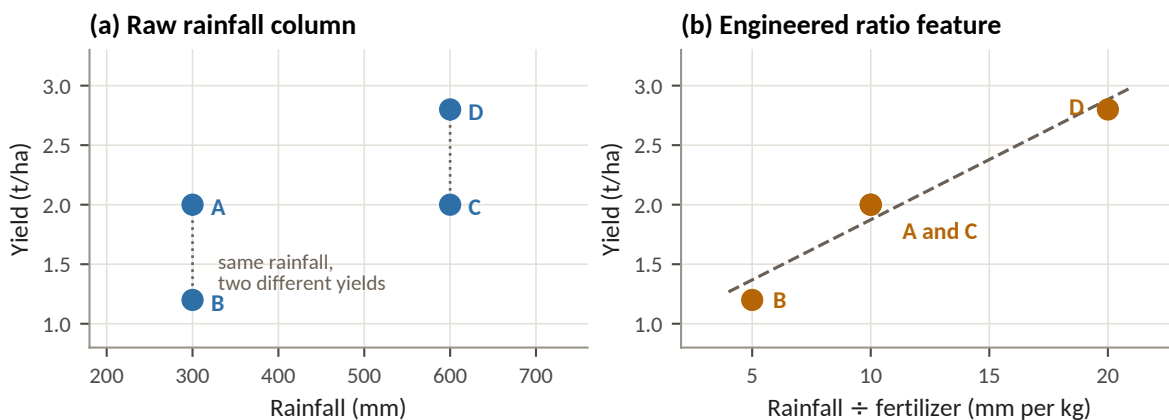


Figure 22.1 – The same four fields, plotted against the raw rainfall column (left) and against the engineered rainfall-to-fertilizer ratio (right).

WHAT TO OBSERVE

On the left, two fields sit at 300 mm with yields of 1.2 and 2.0 and two more at 600 mm with yields of 2.0 and 2.8: rainfall alone cannot tell those pairs apart. On the right the four points fall almost on a line, and A and C land on the same point because they share a ratio. The pattern was always there; the ratio made it visible.

COMMON MISTAKE

X Wrong: Reaching straight for a more complicated model when accuracy disappoints, without asking whether the features carry enough information.

✓ Right: Ask a person who knows the subject what really drives the outcome, and build that quantity as a column. A simple model with good features usually beats a complicated model fed raw ones, and is far easier to explain.

22.2 Combining Columns: Dates, Differences and Interactions

A raw date is one of the least useful things a model can receive. The text 2024-03-15 is not naturally larger or smaller than 2024-08-15 in any way a model can use, even though the gap between them — the number of days — is very often exactly what matters. The difference feature above fixes this in one subtraction, and in the record book both dates are already day numbers within the year.

WORKED EXAMPLE 22.2 — Engineer a Growing-Season-Length Feature from Two Dates

Field E was planted on day 74 of the year (15 March) and harvested on day 227 (15 August). Field F was planted on day 91 (1 April) and harvested on day 208 (27 July). Build a new feature, growing-season length in days, for each field.

1. Use the difference formula $d = a - b$ with a = harvest day and b = planting day.
2. Field E: $227 - 74 = 153$ days.
3. Field F: $208 - 91 = 117$ days.
4. Compare the two: $153 - 117 = 36$ days.

Answer: Field E grew for 153 days and field F for only 117, a gap of 36 days. Neither raw date showed this by itself; one subtraction makes it obvious to a model and to a person reading the table.

Figure 22.2 puts both fields on the same calendar.

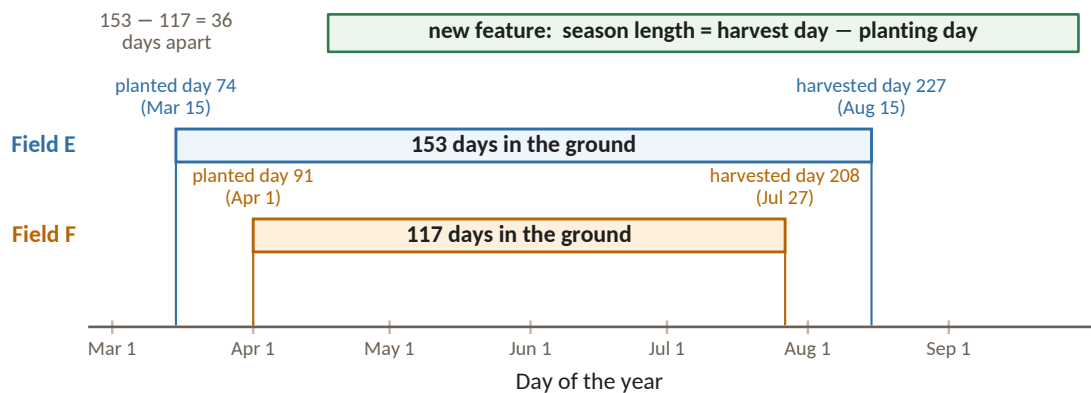


Figure 22.2 – Two raw dates per field become one useful feature. The length of each bar is the new growing-season-length column.

WHAT TO OBSERVE

The two raw date columns give four numbers a model cannot compare meaningfully. The lengths of the bars — 153 days and 117 days — give one number per field that lines up with how much time the crop had. Field F was also planted two weeks later in the calendar, which is a second, separate piece of information.

A date can also be broken into its calendar parts whenever the position in the year, and not just the gap, is likely to matter. Planting month is one example: a field planted in early spring behaves differently from one planted a month later. For a shop, day of the week and a public-holiday flag are often the two most valuable columns in the whole dataset.

Ratios and gaps each combine two columns with one operation. A third kind of combination captures something neither can: the way two features work *together*.

WORD TO KNOW

Interaction feature — A new feature built by multiplying two existing features, so that a model can see their combined effect directly instead of only their two separate effects. An interaction is large only when both original features are large at the same time.

EQUATION: INTERACTION FEATURE (TWO-WAY PRODUCT)

$$x_{\text{int}} = x_1 \times x_2$$

In words: Multiply two features together to make one new number, which is only large when both of the original features are large at once.

When it is used: When the effect of one feature depends on the value of another — fertilizer that only helps if there is enough water, a discount that only works during a holiday, a drug dose that depends on body weight.

Symbol	Meaning	Unit or type
x_{int}	the new interaction feature	product of the two units
x_1	the first feature	its own unit
x_2	the second feature	its own unit

Two-way products are by far the most common. Tree models can approximate an interaction by splitting twice; linear models cannot, so they benefit most.

WORKED EXAMPLE 22.3 — Detect an Interaction Between Rainfall and Fertilizer

Four more fields from a neighboring village report rainfall, fertilizer and yield. Field G: 300 mm and 20 kg, yield 1.5 t/ha. Field H: 300 mm and 60 kg, yield 1.6 t/ha. Field I: 600 mm and 20 kg, yield 2.0 t/ha. Field J: 600 mm and 60 kg, yield 3.5 t/ha. Find how much the extra 40 kg of fertilizer changed the yield at each rainfall level, then build the interaction feature.

1. At low rainfall, 300 mm, the yield change from 20 kg to 60 kg of fertilizer is $1.6 - 1.5 = 0.1$ t/ha.
2. At high rainfall, 600 mm, the same change is $3.5 - 2.0 = 1.5$ t/ha.
3. Compare: $1.5 \div 0.1 = 15$, so the very same extra 40 kg of fertilizer bought fifteen times more grain when the water was there to use it.
4. Build the interaction feature, rainfall $\times$ fertilizer, for each field. Field G: $300 \times 20 = 6,000$. Field H: $300 \times 60 = 18,000$. Field I: $600 \times 20 = 12,000$. Field J: $600 \times 60 = 36,000$.
5. Field J is the only field with both values high at once, and it has by far the largest interaction, 36,000, matching its standout yield of 3.5 t/ha.

Answer: Fertilizer's benefit depends on rainfall: 0.1 t/ha at low rainfall against 1.5 t/ha at high rainfall. Multiplying the two columns hands a model that combined effect directly, instead of hoping it discovers the connection from two separate columns.

The four fields make a clearer picture as two lines, one per fertilizer level, in Figure 22.3.

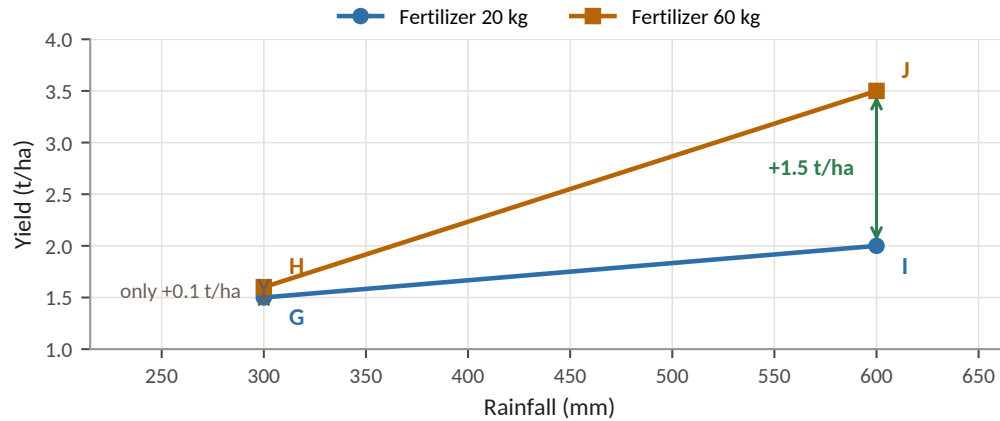


Figure 22.3 – Yield against rainfall at two fertilizer levels. Two lines that are not parallel are the signature of an interaction.

WHAT TO OBSERVE

If fertilizer helped by the same amount at every rainfall level the two lines would be parallel. They are not: the orange line climbs steeply while the blue one is almost flat. That widening gap — 0.1 t/ha at 300 mm, 1.5 t/ha at 600 mm — is what an interaction feature captures in one column.

Table 22.2 collects everything built so far for the six fields of Table 22.1. The last new column, village frequency, is built in Lesson 22.4.

Table 22.2 – The same six fields after feature engineering. Every new column comes from arithmetic on Table 22.1.

Field	Rain ÷ fert. (mm/kg)	Season length (days)	Planting month	Rain × fert.	Village frequency	Yield (t/ha)
A	10	145	March	9,000	0.40	2.0
B	5	123	April	18,000	0.20	1.2
C	10	163	March	36,000	0.40	2.0
D	20	135	April	18,000	0.25	2.8
E	10	153	March	20,250	0.40	2.4
F	10	117	April	20,250	0.01	1.9

Fields E and F are the interesting pair. Their rainfall and fertilizer are identical, so their ratio and interaction are identical too, yet E yielded 2.4 t/ha and F only 1.9. The engineered columns explain it: E had 36 more growing days and sits in a large village, while F was planted in April in a tiny hill village. Without those columns the two rows would look like one field with two different answers.

COMMON MISTAKE

✗ Wrong: Engineering a feature from information that would not actually exist yet at the moment the prediction has to be made — for example a field's total season rainfall, when the forecast is needed in May.

✓ Right: Write down the exact moment the prediction must be made, then build every feature only from data available by that moment. This is the same data leakage trap Chapter 2 warned about, wearing a new disguise.

22.3 Reshaping One Column: Log Transforms and Bins

Sometimes a single column is fine in principle but badly shaped, so a handful of extreme values crowd everything else into a corner. Field area is the clearest example: across the cooperative's 180 recorded fields the smallest is 3 dunams, the middle one 32, and the largest 360.

WORDS TO KNOW

Right-skewed column — A column in which most values are small and a few are very large, so the histogram has a long tail to the right. Areas, incomes, populations, order sizes and waiting times are almost always right-skewed.

Log transform — Replacing a feature's values by their logarithms, so that equal *multiplications* become equal *steps*. It pulls the long tail in and spreads the crowded small values out.

Binning — Replacing a numeric column by a small number of ordered groups, such as Low, Medium and High. Binning hides small differences on purpose, so a model sees only the broad level.

EQUATION: LOG TRANSFORM OF A SKEWED FEATURE

$$x' = \log_{10}(x)$$

In words: Replace each value by the power of ten it represents, so that 10 becomes 1, 100 becomes 2 and 1000 becomes 3.

When it is used: When one feature spans several orders of magnitude and a few huge values would otherwise dominate a linear model, a distance calculation or a plot.

Symbol	Meaning	Unit or type
x	the original value, which must be greater than zero	its own unit
x'	the transformed value	no unit (a power of ten)

When the column can contain zeros, use $x' = \log_{10}(x + 1)$ so that 0 maps to 0. Software often uses the natural logarithm (`np.log1p`); the shape is identical, only the axis numbers change.

WORKED EXAMPLE 22.4 — Log-Transform Four Field Areas

Four fields have areas of 1, 10, 100 and 1000 dunams. Apply the log transform and compare the gaps before and after.

1. $\log_{10}(1) = 0$, because $10^0 = 1$.
2. $\log_{10}(10) = 1$, because $10^1 = 10$.
3. $\log_{10}(100) = 2$, because $10^2 = 100$.
4. $\log_{10}(1000) = 3$, because $10^3 = 1000$.
5. Gaps before the transform: $10 - 1 = 9$, then $100 - 10 = 90$, then $1000 - 100 = 900$.
6. Gaps after the transform: $1 - 0 = 1$, then $2 - 1 = 1$, then $3 - 2 = 1$.

Answer: The four areas become 0, 1, 2 and 3. Before the transform the last gap was a hundred times the first, so the biggest field alone would have driven any distance or any straight-line fit. Afterward the four fields are evenly spaced, and “ten times larger” means the same one-step move everywhere on the scale.

Figure 22.4 applies the same transform to all 180 field areas at once.

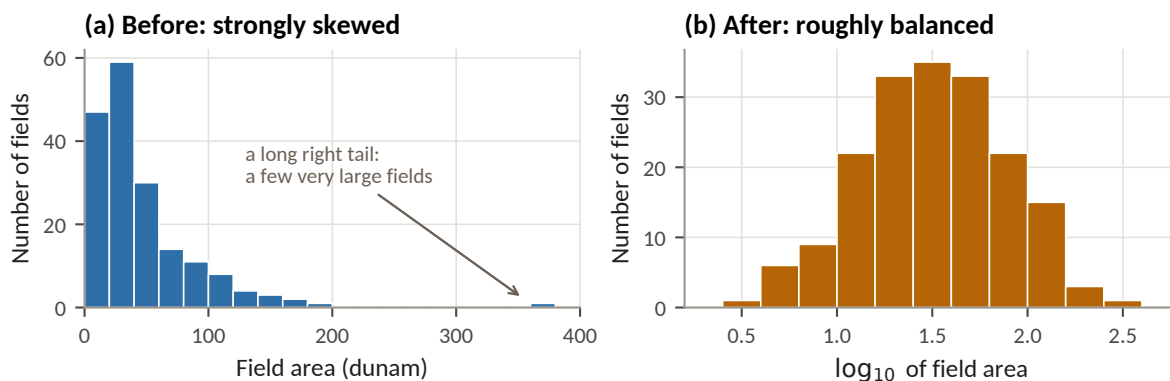


Figure 22.4 – The cooperative’s 180 field areas before (left) and after (right) the log transform.

WHAT TO OBSERVE

On the left, more than half the fields squeeze into the first two bars while one field sits alone at 360 dunams. On the right the same 180 fields spread into a rounded, balanced shape with no lonely extreme. Nothing was deleted and no field was called an outlier — only the scale changed.

The log transform belongs to a family of shape-fixing transforms studied by George Box and David Cox in 1964; the square root is a gentler member. Which to use is decided the way every choice in this book is: try it, and compare validation scores as in Chapter 5. Binning, the third term in the box above, goes the opposite way: instead of stretching the scale it throws part of it away on purpose, as Figure 22.5 does to the rainfall column.

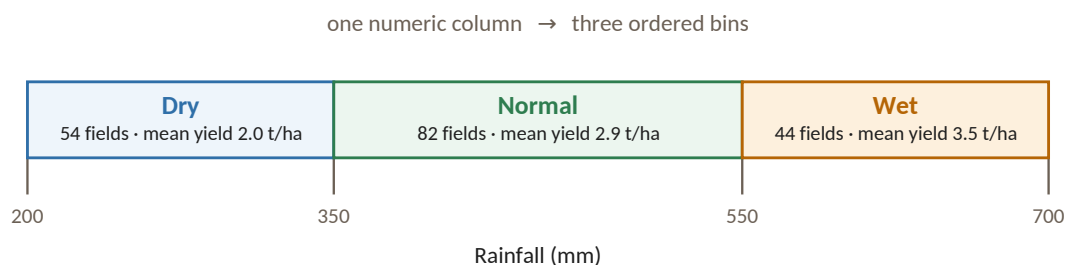


Figure 22.5 – Binning the rainfall column of the cooperative’s 180 fields into three ordered groups.

WHAT TO OBSERVE

The three bins hold 54, 82 and 44 fields, and their average yields climb steadily from 2.0 to 2.9 to 3.5 t/ha. The bins keep that trend while throwing away the difference between 351 mm and 352 mm, which no farmer would call meaningful.

Bins can be cut at equal widths, as here, or at equal counts. Either way, choose the edges from the training rows or from domain knowledge — never by sliding a cut point until the groups differ as much as possible in the label, which is leakage exactly like fitting a scaler on the test set (Chapter 4). Binning helps most when a relationship is bumpy, when a column is measured roughly, or when the model must be explained to people who think in categories such as “a dry year”. Tree models already cut numeric columns into ranges themselves (Chapter 13) and gain least; linear and distance-based models (Chapter 11) gain most from a log transform, because a single enormous value distorts both.

22.4 Encoding Text and Crowded Category Columns

Chapter 4 introduced one-hot encoding for a categorical column with no natural order: one new 0/1 column per category. That works beautifully for four or five values, and badly for the cooperative’s Village column.

WORDS TO KNOW

High-cardinality category — A categorical column with a large number of different values — village names, postal codes, product codes, customer identifiers. One-hot encoding it produces a huge number of columns that are almost always zero.

Target encoding — Replacing each category by the average value of the label for that category in the training data, such as each village’s average yield. Described by Daniele Micci-Barreca in 2001, it is powerful but must be built with great care.

Across the cooperative’s whole record book the Village column names 18 different places, so one-hot encoding turns one column into 18, and in a typical row 17 of them are 0. A rare village gets a column that is 1 in only one or two rows, from which no model can learn anything reliable. A better idea replaces each category by how often it appears.

EQUATION: FREQUENCY ENCODING OF A CATEGORY

$$\text{freq}(c) = \frac{n_c}{n}$$

In words: Replace each category name by the share of the training rows that carry that name.

When it is used: For a category column with many values, when one-hot encoding would create dozens or hundreds of nearly empty columns.

Symbol	Meaning	Unit or type
$\text{freq}(c)$	the encoded value for category c	share, between 0 and 1
n_c	number of training rows in category c	count
n	total number of training rows	count

Count on the training rows only, then apply the same lookup table to every other row. A category never seen in training gets a frequency of 0.

WORKED EXAMPLE 22.5 — Frequency-Encode a Crowded Village Column

Take a round 100-row extract of the training records, chosen so that the arithmetic stays easy. “Zakho Center” appears 40 times, “Faysh Khabur” 25 times, “Batifa” 20 times, and 15 other villages appear once each. Find the frequency-encoded value for a Zakho Center row, a Faysh Khabur row, a Batifa row and a rare-village row.

1. Check the total: $40 + 25 + 20 + 15 \times 1 = 100$ rows, so $n = 100$.
2. Zakho Center: $40 \div 100 = 0.40$.
3. Faysh Khabur: $25 \div 100 = 0.25$.
4. Batifa: $20 \div 100 = 0.20$.
5. Each rare village: $1 \div 100 = 0.01$. One-hot encoding would have made 18 columns; this makes 1.

Answer: The village names become 0.40, 0.25, 0.20 and 0.01 in a single numeric column instead of 18 mostly empty ones. All 15 rare villages land on the same small number, which is an honest summary of the fact that the model has barely seen them.

Two other repairs are worth knowing. The simplest keeps the common categories by name and lumps the rare ones into a single value called “Other”; the other is target encoding, defined in the box above.

COMMON MISTAKE

X Wrong: Building a target encoding from the very same rows the model will then train on, so that each row's own label helps decide its own encoded value.

✓ Right: Compute the encoding on separate folds, as in cross-validation (Chapter 5), so that no row ever sees its own label smuggled back through the encoded column. Treat target encoding as a leakage risk first and a convenient shortcut second.

IMPORTANT

A high-cardinality column such as a village name, a postal code or a customer identifier can quietly stand in for something far more sensitive — ethnicity, religion, income or neighborhood — even when nobody intended it. A feature that acts as a stand-in for a protected attribute is called a *proxy variable*, and Chapter 24 studies the problem in depth. Before adding such a feature to any model that affects people, pause and ask what it might really be measuring underneath.

The record book also holds a free-text column: a short note the agronomist writes after each visit. A model cannot read a sentence, but two simple features capture part of its value with no advanced text processing: the number of words, and a 0-or-1 flag for each keyword worth watching, such as “drought”, “pest” or “flood”.

WORKED EXAMPLE 22.6 — Build Two Simple Features from an Agronomist's Notes

Note 1 reads “Healthy stand, slight drought stress in the eastern corner.” Note 2 reads “Strong growth across the whole field, no concerns this visit.” For each note find the word count and a drought flag, written as 1 for yes and 0 for no.

1. Count note 1 word by word: Healthy, stand, slight, drought, stress, in, the, eastern, corner — 9 words.
2. Count note 2 word by word: Strong, growth, across, the, whole, field, no, concerns, this, visit — 10 words.
3. Search note 1 for “drought”: it appears once, so the flag is 1.
4. Search note 2 for “drought”: it does not appear, so the flag is 0.

Answer: Note 1 becomes (9 words, drought flag 1) and note 2 becomes (10 words, drought flag 0). The word counts are almost identical, so that feature carries little here — but the drought flag hands the model a genuine early warning from a column it previously could not read at all.

22.5 Selecting Fewer, Better Features

The cooperative's dataset has grown, not shrunk: alongside the raw columns it now holds a ratio, a season length, an interaction, a planting month, a frequency-encoded village and two note-based features. It is time to trim.

Three arguments push in the same direction. Every extra feature gives a flexible model one more way to fit accidents in the training data, which is the capacity problem of Chapter 7. Distance-based methods such as k-nearest neighbors (Chapter 11) weaken as columns pile up, because with enough useless columns every example ends up about equally far from every other. And a model a cooperative board can read in one page is worth more than a slightly better one nobody trusts.

WORDS TO KNOW

Feature selection — Choosing a smaller, more useful subset of the available features to give a model, instead of feeding it every column that happens to exist. Fewer features usually mean less overfitting, faster training and a model far easier to explain.

Filter method — Scores each feature on its own, before any model is trained — for example by its correlation with the target — and keeps the best. Fast, but blind to how features work together.

Wrapper method — Trains a model again and again on different subsets of features and keeps the subset that validates best. Slow, but it judges features the way they will actually be used.

Embedded method — Lets the model itself do the selecting while it trains, as lasso does by driving weak coefficients to exactly zero (Chapter 8) and as a forest does through its importance scores (Chapter 14).

The simplest filter measures how strongly each feature moves together with the target, using the correlation coefficient introduced by Karl Pearson in 1895.

EQUATION: CORRELATION BETWEEN A FEATURE AND THE TARGET

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

In words: Multiply each feature deviation by the matching target deviation and add them up; then divide by the square root of the two sums of squared deviations. The result runs from -1 to $+1$.

When it is used: As a first, cheap filter on a wide dataset: keep the features whose correlation with the target is far enough from zero, and set the rest aside for a closer look.

Symbol	Meaning	Unit or type
r	correlation between the feature and the target	between -1 and $+1$
$x_i, \bar{x}$	feature value of row i , and the feature's mean	unit of the feature
$y_i, \bar{y}$	target value of row i , and the target's mean	unit of the target
n	number of training rows	count

A value near ± 1 means a strong straight-line relationship; a value near 0 means no straight-line relationship, which is not the same as no relationship. Compute r on the training rows only.

WORKED EXAMPLE 22.7 — Filter Two Candidate Features by Correlation

Four fields report rainfall of 200, 400, 600 and 800 mm, yields of 1.0, 3.0, 2.0 and 4.0 t/ha, and the farmer's shoe sizes of 42, 44, 38 and 40. The team keeps any feature with $|r| \geq 0.3$. Decide which of the two features survives.

- Means: rainfall $\bar{x} = 500$ mm, yield $\bar{y} = 2.5$ t/ha, shoe size $\bar{x} = 41$.
- Rainfall deviations: $-300, -100, +100, +300$. Yield deviations: $-1.5, +0.5, -0.5, +1.5$. Shoe-size deviations: $+1, +3, -3, -1$.
- Top of the fraction for rainfall: $(-300)(-1.5) + (-100)(0.5) + (100)(-0.5) + (300)(1.5) = 450 - 50 - 50 + 450 = 800$.
- Sums of squares: $90,000 + 10,000 + 10,000 + 90,000 = 200,000$ for rainfall, and $2.25 + 0.25 + 0.25 + 2.25 = 5$ for yield.
- Bottom of the fraction: $\sqrt{200,000 \times 5} = \sqrt{1,000,000} = 1000$.
- Rainfall's correlation: $r = 800 \div 1000 = 0.80$.
- Top of the fraction for shoe size: $(1)(-1.5) + (3)(0.5) + (-3)(-0.5) + (-1)(1.5) = 0$, so its correlation is $r = 0 \div \sqrt{20 \times 5} = 0$.

Answer: Rainfall scores $r = 0.80$ and passes the filter; shoe size scores $r = 0.00$ and is dropped. A column recorded by accident during an old paper survey has just been caught by four multiplications and a square root.

Running the same calculation on all ten candidates of the cooperative's 90 training fields gives Table 22.3, where the keep rule is again $|r| \geq 0.3$: three features pass and seven do not.

Table 22.3 – Correlation of each candidate feature with yield, computed on the 90 training fields

Feature	r	Keep?	Feature	r	Keep?
Rainfall $\times$ fertilizer	+0.89	yes	Rainfall $\div$ fertilizer	+0.09	no
Rainfall	+0.78	yes	Field area	−0.07	no
Fertilizer	+0.52	yes	Village frequency	−0.06	no
Season length	+0.15	no	Farmer's shoe size	+0.06	no
Tractors owned	−0.15	no	Planting day	+0.01	no

The engineered interaction tops the whole table, a satisfying reward for one multiplication. But a filter judges every feature in isolation, and that is its weakness: a feature that says little alone can still be valuable once other features are in the model. Keep Table 22.3 in mind — season length and village frequency both fail it — and watch what a wrapper does with the same ten columns.

WORD TO KNOW

Forward selection — A wrapper method that starts with no features at all and adds, one at a time, whichever remaining feature improves the validation score most, stopping when no remaining feature improves it enough to be worth keeping. It is the forward half of stepwise regression, described by M. A. Efroymson in 1960.

EQUATION: FORWARD SELECTION: ONE ROUND

$$j^* = \arg \min_{j \notin S} \text{Err}_{\text{val}}(S \cup \{j\})$$

In words: Take the features already chosen, try adding each remaining feature in turn, train and score a model each time, and keep whichever addition gives the smallest validation error.

When it is used: On a dataset with a modest number of candidate features, when you can afford to train a model once for every candidate in every round.

Symbol	Meaning	Unit or type
S	the set of features already chosen	set of columns
j	one candidate feature not yet chosen	a column
j^*	the feature this round adds	a column
Err_{val}	validation error of a model trained on a feature set	unit of the target

Repeat until the best improvement falls below a threshold fixed in advance. With k candidates the search trains about $k^2/2$ models, far cheaper than trying all 2^k subsets.

WORKED EXAMPLE 22.8 — Run the First Two Rounds of Forward Selection

The cooperative splits its 180 fields into 90 training and 90 validation fields and measures each candidate set by the validation RMSE of a linear model, in tons per hectare. Predicting the training mean scores 0.824. Round 1 gives rainfall $\times$ fertilizer 0.382, rainfall 0.540, fertilizer 0.741, and the other seven candidates between 0.816 and 0.834. Round 2, with rainfall $\times$ fertilizer already chosen, gives rainfall 0.312, rainfall $\div$ fertilizer 0.323, fertilizer 0.336, and every other candidate 0.361 or worse. Carry out both rounds.

1. Round 1: scan the ten scores for the smallest. It is 0.382, for rainfall $\times$ fertilizer.
2. Check that this is really an improvement on predicting the mean: $0.824 - 0.382 = 0.442$ t/ha better. Add the feature, so $S = \{\text{rainfall} \times \text{fertilizer}\}$.
3. Round 2: scan the nine remaining scores. The smallest is 0.312, for rainfall.
4. Measure the improvement: $0.382 - 0.312 = 0.070$ t/ha. Add rainfall, so S now holds two features.
5. Note that the seven losing candidates all scored 0.816 to 0.834 in round 1 — no better than the 0.824 of predicting the mean. They carry nothing about yield.

Answer: After two rounds the chosen set is {rainfall $\times$ fertilizer, rainfall}, with a validation RMSE of 0.312 t/ha, down from 0.824. The single engineered interaction did more work in round 1 than any raw column in the record book.

Continuing to the end produces Table 22.4. The cooperative decided in advance to stop as soon as a round improves the validation RMSE by less than 0.01 t/ha, smaller than the accuracy of their own weighbridge.

Table 22.4 – Every round of forward selection on the cooperative's ten candidate features

Round	Feature added	Validation RMSE (t/ha)	Improvement	Decision
0	(predict the training mean)	0.824	—	starting point
1	Rainfall $\times$ fertilizer	0.382	0.442	keep
2	Rainfall	0.312	0.070	keep
3	Village frequency	0.289	0.023	keep
4	Season length	0.248	0.041	keep
5	Fertilizer	0.244	0.004	stop here

Figure 22.6 draws the same five rounds, and the five that follow them, as a curve.

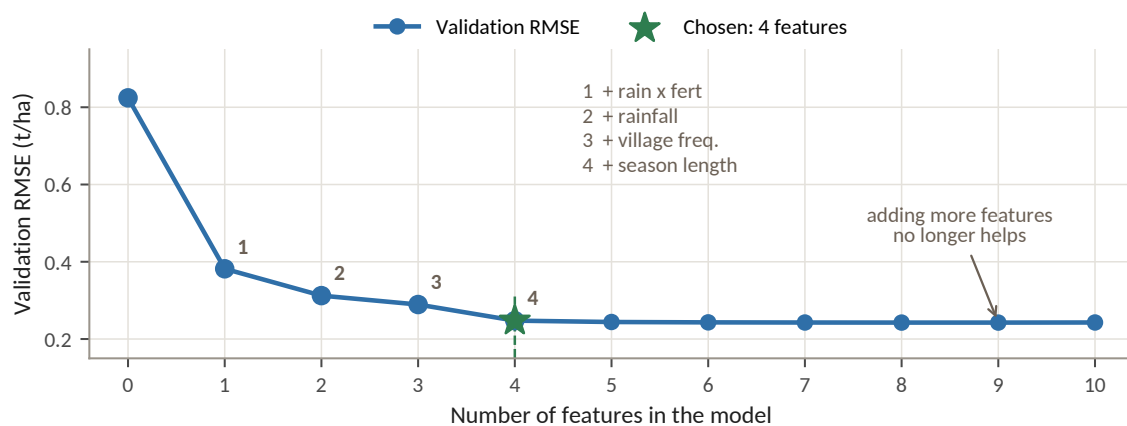


Figure 22.6 – Validation error against the number of features chosen. The curve falls steeply, then flattens.

WHAT TO OBSERVE

The first four features cut the validation RMSE from 0.824 to 0.248 t/ha; the remaining six together change it by less than 0.01, and the last two make it slightly worse. Once the curve stops falling, extra columns buy nothing and cost simplicity.

Compare the winners with Table 22.3. Village frequency scored $r = -0.06$ and season length $r = +0.15$; the filter would have thrown both away, yet forward selection kept them, because they add information the other chosen features do not carry.

Two embedded methods you already know select while the model trains. Lasso regression (Chapter 8), proposed by Robert Tibshirani in 1996, drives weak coefficients to exactly zero, so raising the penalty ranks features by the order in which they drop out; a random forest (Chapter 14), introduced by Leo Breiman in 2001, reports an importance score for every column. Recursive feature elimination, published by Isabelle Guyon and colleagues in 2002, is the backward relative of forward selection. Whichever you use, select with training and validation data only: Chapter 5 showed that a set you choose from stops being an honest measure.

22.6 Feature Importance Charts and Their Traps

Chapter 14 showed how to read importance scores out of a trained forest: impurity importance, each feature's share of the impurity the trees removed, and permutation importance, the damage done by shuffling one column. Both give a ranked list, and the everyday use of that list is a simple cut-off rule. Trimming is not only about tidiness: Chapter 7 showed that unearned capacity widens the gap between training error and validation error, so the same worked example checks both at once.

WORKED EXAMPLE 22.9 — Select Features by Importance, Then Check the Gap

A forest trained on 12 columns reports these example importance scores for its top five: rainfall 45%, fertilizer 30%, growing-season length 20%, farmer's shoe size 3%, tractors owned 2%. The team keeps every feature at or above 5%. The wide model had a training error of 0.05 t/ha and a validation error of 0.35 t/ha; retrained on the kept features it reports 0.12 and 0.18. Decide which features stay and which model to trust.

1. Check the scores add up: $45 + 30 + 20 + 3 + 2 = 100\%$, as impurity importances always do.
2. Rainfall 45%, fertilizer 30% and growing-season length 20% are all above 5%: keep all three.
3. Shoe size 3% and tractors 2% are below 5%: drop both. That keeps $45 + 30 + 20 = 95\%$ of the measured usefulness and loses only 5%.
4. Now the wide model's gap: $0.35 - 0.05 = 0.30$ t/ha.
5. The trimmed model's gap: $0.18 - 0.12 = 0.06$ t/ha.
6. Compare: $0.30 \div 0.06 = 5$, so the wide model's gap is five times larger — it was fitting noise the small model was not.

Answer: Keep rainfall, fertilizer and growing-season length. The trimmed model looks worse on its training data, 0.12 against 0.05, and is much better on data it has never seen, 0.18 against 0.35. A rising training error together with a falling validation error is the signature of a healthy trim.

Figure 22.7 runs the same idea on the cooperative's ten real candidates.

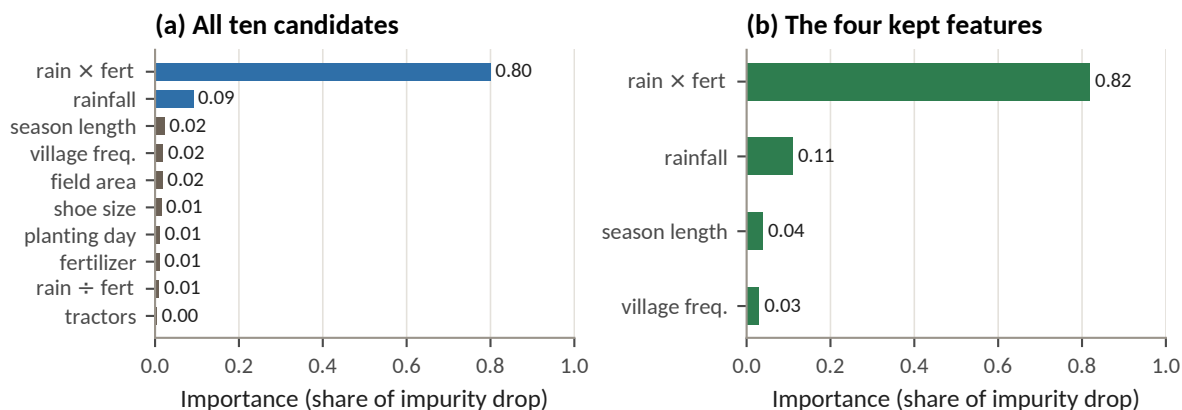


Figure 22.7 – Impurity importance from a 200-tree forest, before selection (left) and after the four chosen features are refitted alone (right).

WHAT TO OBSERVE

On the left, one engineered feature takes 0.80 of the total importance and the six weakest columns together take less than 0.07. On the right the four kept features share the whole scale and the ranking barely changes: dropping six near-useless columns did not disturb what the model was really using.

Importance charts are also the most misread chart in machine learning, and the largest trap appears whenever two features carry the same information. The cooperative records rainfall twice — once from the village weather station and once from a gauge at the field's own edge — and the two columns agree almost perfectly, with a correlation of 0.996. Figure 22.8 shows what that does to their importance scores.

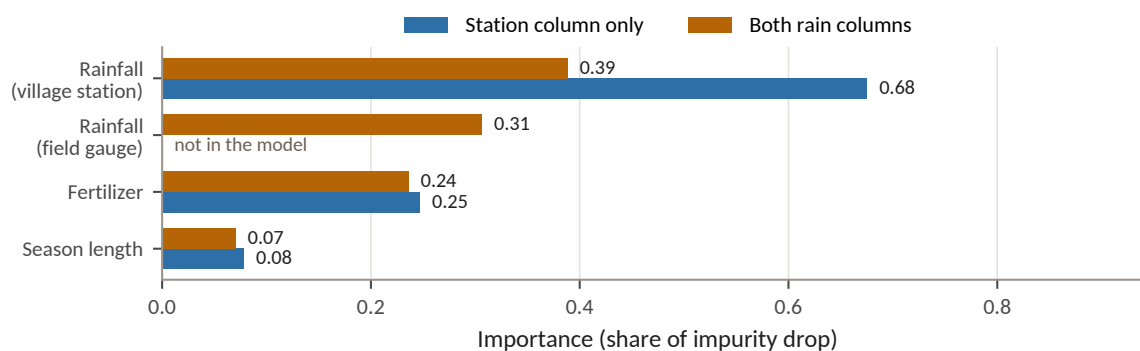


Figure 22.8 – What happens to importance scores when a second, nearly identical rainfall column joins the model.

WHAT TO OBSERVE

With only the station column, rainfall scores 0.68. Add the field gauge, which measures the same rain, and the score splits into 0.39 and 0.31 — adding to about the same 0.69. Rainfall did not become less important: its credit was divided between two columns saying the same thing, and either alone now looks weaker than fertilizer.

COMMON MISTAKE

X Wrong: Deleting a feature because its importance score is low, without checking whether a near-twin beside it is taking half the credit.

✓ Right: Look for pairs of features that measure the same thing, using a correlation between features rather than with the target. Keep one of each pair, then read the importance chart again.

IMPORTANT

An importance score describes one model's own behavior, never the real world, and never proves that a feature causes an outcome. Be especially careful when a column describes a person: if "farmer's age" scored highly here, the honest reading is that older members farm different land, not that any farmer is at fault. Use importance to start a human review, never to justify a penalty. Chapter 24 returns to this.

22.7 A Feature-Engineering Checklist

The whole chapter fits into one loop, drawn in Figure 22.9: understand the timing, build features, encode categories, select, check for leakage, write the list down — then repeat next season.

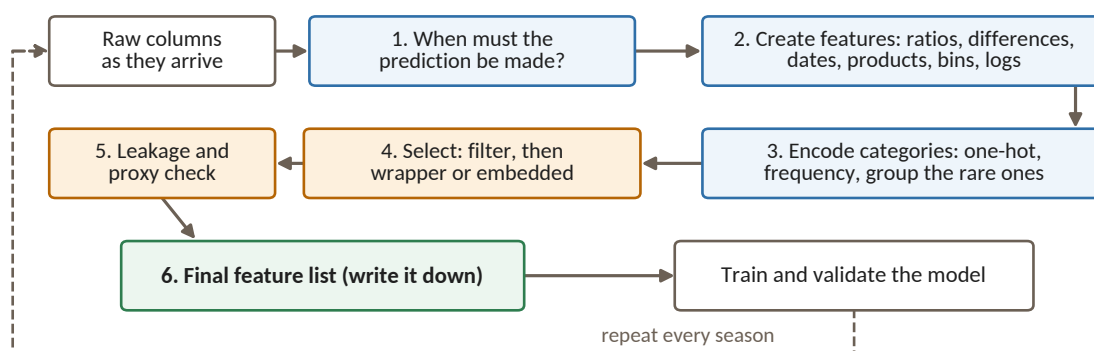


Figure 22.9 – The feature-engineering loop. Steps 1 to 3 create columns, steps 4 to 6 remove and record them.

WHAT TO OBSERVE

The very first step contains no arithmetic at all: it asks when the prediction must be made, and that answer decides which columns are even allowed to exist. Notice too that the loop closes — a feature list is a living document, not a decision made once.

Table 22.5 turns the same loop into questions you can work through with a pencil for any dataset in any field. One last habit: write the final feature list down, one line per feature, saying where it came from and why it is allowed. That page lets a colleague reproduce the work next season, and it is the first thing anyone asks for when a prediction turns out to be wrong.

Table 22.5 – A feature-engineering checklist to run on any new dataset

#	Step	Question to ask	Warning sign
1	Timing	At what exact moment must the prediction be ready?	A column that is only filled in after the outcome is known
2	Combine	Do two columns only make sense together?	A model asked to discover a ratio or a product on its own
3	Shape	Is a date used as a plain number, or a column spread over several orders of magnitude?	No gap or month extracted; a long right tail
4	Categories	How many values does each category column hold?	Dozens of one-hot columns that are almost always zero
5	Trim	Does every feature earn its place on validation data?	A wide feature set with a large training-validation gap
6	Check	Could any feature stand in for something sensitive?	An identifier or location column inside a model about people

COMMON MISTAKES AND MISCONCEPTIONS

- Reaching for a more complicated model before asking whether better features could show the pattern more directly.
- Building a feature from information that would not exist yet at the moment the prediction has to be made.
- Handing a model a raw date, instead of the gap between two dates or the month extracted from one.
- One-hot encoding a column with hundreds of categories, or building a target encoding, or choosing bin edges, from the same rows the model then trains on.
- Assuming an extra feature can never hurt, or trusting a filter's verdict on a single feature that might shine only in company with others.
- Deleting a feature with a low importance score without checking whether a near-identical column beside it is taking half the credit, or selecting features with test-set scores and then reporting those same scores as honest performance.

Practical Activities

ACTIVITY (PAPER) — Engineer Three Features for a Bakery in Erbil

A bakery in Erbil wants to predict whether a bread order will arrive late. Six past orders are recorded below.

Order	Placed (day)	Delivered (day)	Loaves	Staff on duty	Distance (km)	Late?
1	12	13	200	3	4	No
2	15	18	900	3	12	Yes
3	19	20	150	4	6	No
4	23	27	1200	2	15	Yes
5	26	27	300	4	3	No
6	30	34	800	2	11	Yes

1. One column must never be used as a feature here, because its value is only known after the outcome. Name it and explain why in one sentence.
2. Engineer three new features from the remaining columns: one ratio, one product and one of your own choosing. Compute all three for all six orders, showing your arithmetic.
3. For each new feature write one sentence saying why a baker would expect it to matter, then rank the three by how well they separate the Late orders from the On-time ones, judging by eye.
4. Swap sheets with a classmate and try to break each other's features: for each one, describe an order for which the feature would give a misleading answer.

You need: Paper, pen and a calculator

ACTIVITY (PYTHON) — Engineer and Select Features with pandas and scikit-learn

This program loads the cooperative's 180 fields, builds three engineered features with pandas, then compares a filter (SelectKBest), a wrapper (SequentialFeatureSelector) and permutation importance on the very same eight candidate columns.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, f_regression, SequentialFeatureSelector
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance

d = pd.read_csv("datasets/ch22_fields.csv")
d["season_days"] = d.harvest_day - d.plant_day      # a difference feature
d["rain_x_fert"] = d.rainfall_mm * d.fertilizer_kg  # an interaction feature
tr = d[d.set == "train"]
d["village_freq"] = d.village.map(tr.village.value_counts(normalize=True)).fillna(0)
tr, va = d[d.set == "train"], d[d.set == "valid"]
c = ["rain_x_fert", "rainfall_mm", "fertilizer_kg", "season_days",
     "village_freq", "area_dunam", "shoe_size", "tractors"]

k = SelectKBest(f_regression, k=3).fit(tr[c], tr.yield_t_ha)
print("filter keeps :", [n for n, ok in zip(c, k.get_support()) if ok])
s = SequentialFeatureSelector(LinearRegression(), n_features_to_select=4)
s.fit(tr[c], tr.yield_t_ha)
print("forward keeps:", [n for n, ok in zip(c, s.get_support()) if ok])

rf = RandomForestRegressor(n_estimators=200, random_state=0).fit(tr[c], tr.yield_t_ha)
pi = permutation_importance(rf, va[c], va.yield_t_ha, n_repeats=10, random_state=0)
for name, score in sorted(zip(c, pi.importances_mean), key=lambda t: -t[1]):
    print("%-13s %6.3f" % (name, score))
```

Output

```
filter keeps : ['rain_x_fert', 'rainfall_mm', 'fertilizer_kg']
forward keeps: ['rain_x_fert', 'rainfall_mm', 'season_days', 'village_freq']
rain_x_fert    1.041
rainfall_mm    0.127
village_freq   0.017
season_days    0.017
fertilizer_kg  0.012
shoe_size      0.004
area_dunam     0.001
tractors       -0.000
```

The two selectors disagree exactly as Table 22.3 and Table 22.4 predicted: the filter keeps the three strongest correlations, while forward selection swaps fertilizer for season length and village frequency. SequentialFeatureSelector scores subsets by cross-validation on the training rows rather than on a separate validation set, and still reaches the same four columns found by hand. Try `n_features_to_select=6`.

You need: Python with pandas and scikit-learn (see Appendix B), and `datasets/ch22_fields.csv`

CASE STUDY — Better Features for Wheat Yield at a Farm Cooperative Near Zakho

Diyar joined the Zakho Farmers' Cooperative as its first data analyst. The record book stretches back a decade: planting and harvest dates, rainfall, fertilizer, field area, village, the agronomist's visit notes and each field's final yield. The board wants a forecast good enough to plan next season's fertilizer purchase and to decide which villages most need irrigation, but Diyar's first model, trained straight on the raw columns, is barely better than predicting each village's past average.

Diyar suspects the columns rather than the model, and starts with the timing question. The forecast has to be ready in May, while the crop is still in the ground, so the harvest date is not yet known — but the planting date is. Season length is therefore rebuilt from the planting date and the region's usual harvest window, and total season rainfall is dropped entirely because in May it does not exist. Then comes the building: a rainfall-to-fertilizer ratio, a growing-season length, a rainfall $\times$ fertilizer interaction and a planting month all take one afternoon. The Village column, with its long tail of tiny hill villages, becomes a single frequency-encoded number instead of a separate, mostly empty one-hot column per village. The agronomist's notes give two more features: the note's word count and a "mentions drought" flag. On the cooperative's own validation fields, a linear model that scored 0.33 t/ha of RMSE on the seven raw columns scores 0.24 t/ha once the engineered columns join it — roughly a quarter better, with no change to the model at all.

Feature selection then goes the other way. Forward selection, stopping when a round gains less than 0.01 t/ha, keeps four columns out of ten: rainfall $\times$ fertilizer, rainfall, village frequency and season length. Validation RMSE holds at 0.25 t/ha with six fewer columns — the same accuracy in a model the board can read in one page. Among the columns that go is a long-forgotten "farmer's shoe size", sitting in the spreadsheet since the cooperative's first paper survey and correlated with yield at exactly +0.06. Before presenting the result Diyar makes two final checks: is every remaining feature knowable in May, and could village frequency stand in for something it was never meant to measure, such as which communities are wealthier? Because the forecast will steer irrigation money, Diyar writes that risk into the report and asks the board to review village-level decisions by hand rather than by score. One habit then joins the yearly calendar: revisit the whole feature list every season, because a village can grow, a new fertilizer can arrive, and a feature that mattered this year can quietly stop mattering the next.

CHAPTER SUMMARY

- Feature engineering builds new columns from existing ones so a model can see a pattern directly instead of having to discover it.
- A ratio feature, $a \div b$, can reveal a relationship neither raw column shows alone.
- A difference feature, $a - b$, turns two raw dates into one number such as a growing-season length; a date can also be split into month, day of week or holiday flag.
- An interaction feature, $x_1 \times x_2$, hands a model the combined effect of two features; non-parallel lines on a plot are the sign that one exists.
- A log transform straightens a strongly right-skewed column, and binning replaces a numeric column by a few ordered groups when only the broad level matters.
- A category column with many values is better handled by frequency encoding, or by grouping rare values into “Other”, than by one-hot encoding; target encoding is powerful but leaks the label unless computed on separate folds.
- Simple text features such as a word count or a keyword flag can extract real value from a column a model cannot read.
- Feature selection comes in three families: filters score each feature alone, wrappers such as forward selection judge whole subsets by validation score, and embedded methods such as lasso and forest importance select while training.
- A filter can miss a feature that only becomes useful in company with others, and two features carrying the same information split their importance score, so a low score is not proof that a feature is useless.
- Removing unhelpful features usually raises training error slightly and lowers validation error, and every engineered feature must still be checked for leakage and for standing in as a proxy for something sensitive.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher’s Guide.

Problem 1 [Basic] Explain, in your own words, what feature engineering means, and why it can matter more than choosing a fancier model.

Problem 2 [Basic] A dataset has separate columns for a shipment’s weight in kilograms and its volume in cubic meters. Suggest one ratio feature built from these two columns, and say in one sentence what it would measure.

Problem 3 [Basic] Explain in one or two sentences why one-hot encoding a column with 300 different city names could cause problems, and name one better option.

Problem 4 [Basic] Four fields have areas of 10, 100, 1000 and 10,000 dunams. Write the base-10 log transform of each, and state the gap between neighboring values before and after the transform.

Problem 5 [Standard] Four fields report rainfall and fertilizer: K (200 mm, 10 kg), L (400 mm, 20 kg), M (400 mm, 10 kg), N (800 mm, 20 kg). Find the rainfall-to-fertilizer ratio of each, showing every step, and name any fields that share a ratio.

Problem 6 [Standard] A payment-method column has these training counts in 1000 rows: Cash 500, Card 450, and 50 other rare types appearing once each. Find the frequency-encoded value for a Cash row and for one rare row, showing every step, and say how many one-hot columns this replaces.

Problem 7 [Standard] A model reports these importance scores: location 40%, price 35%, store size 20%, day of week 5%. Using a rule of keeping every feature at or above 10%, decide which features stay, and state what share of the total importance is dropped.

Problem 8 [Standard] Four crops report temperature (in °C) of 10, 20, 30, 40 and growth (in cm) of 2, 6, 4, 8. Compute the correlation r between temperature and growth using the equation in this chapter, showing every step, and decide whether it passes a filter of $|r| \geq 0.3$.

Problem 9 [Standard] In one round of forward selection, the current set scores a validation RMSE of 0.50. Adding feature P scores 0.47, feature Q scores 0.44 and feature R scores 0.51. Which feature is added, what is the improvement, and what would happen under a stopping rule of 0.05?

Problem 10 [Challenge] A team predicting customer churn engineers a feature, “total complaints this customer has ever filed”, computed from the customer’s whole history including complaints filed after the churn decision. (a) Explain exactly what is wrong. (b) Write a corrected version of the same feature that would be safe to use.

Problem 11 [Challenge] A forest is trained twice on the same data. In the first run, “rainfall” scores an importance of 0.60. In the second run a new column, “rainfall measured by a second gauge”, is added, and rainfall now scores 0.31 while the new column scores 0.29. (a) Explain what happened. (b) Explain why deleting both columns because “neither reaches 0.35” would be a serious mistake. (c) Describe a check that would have caught the problem.

Problem 12 [Challenge] A hiring dataset includes an engineered feature, “distance from the office to the candidate’s home”. (a) Give one legitimate reason it might help predict on-time attendance. (b) Explain how the same feature could quietly act as a proxy for economic background or ethnicity. (c) State what you would do before allowing it into the model.

Hyperparameter Tuning, Model Comparison, and Model Selection

Turning the dials fairly, and choosing the model you can defend

IN THIS CHAPTER YOU WILL LEARN TO

- Tell a model parameter from a hyperparameter, and name the main dials of the models taught in Chapters 8 to 21.
- Count the combinations in a grid search, and the model fits that searching them costs.
- Explain how random search fixes the cost in advance, and estimate its chance of finding a good combination.
- Set up a nested cross-validation, count its fits, and say what its score actually measures.
- Explain why a test set may be opened only once, and what a peeked score really reports.
- Compare candidate models on the same folds using mean $\pm$ standard deviation and paired fold differences.
- Choose a final model from score, cost, speed and interpretability together, and write an honest model report.

Introduction

Bahar bakes bread rolls in Erbil and delivers them every morning to twelve small shops. Her oven has two dials, temperature and time, and they are not independent: a hotter oven needs a shorter bake, a cooler oven a longer one. If she turns only the temperature dial, leaving the time wherever it happens to sit, she will never find her best bread.

Every model in this book has dials of exactly this kind. The number of neighbors k , a tree's maximum depth, a boosting model's learning rate, an SVM's C and γ — all set by a person before training starts, and all interacting. This chapter shows how to turn them systematically instead of by guesswork, how much computer time that costs, and how to avoid the mistake that quietly ruins the whole exercise: letting the test set help you choose.

The second half is about a bigger decision. Once every candidate has been tuned, which one do you actually ship? You will learn to compare models on the same folds, to report a score with its spread, to decide whether one model is genuinely ahead or merely luckier, and to weigh the score against cost, speed and whether a person can explain the prediction. Bahar's forecast runs through the whole chapter, and every number in it comes from a real calculation you can repeat.

23.1 Every Model Has Dials to Turn

Chapter 7 introduced the *hyperparameter*: a setting chosen before training begins, rather than a value the learning algorithm discovers on its own. It is worth naming the other half of that pair, because beginners mix the two up constantly.

WORDS TO KNOW

Model parameter — A number the learning algorithm finds by itself during training: a slope, an intercept, a split threshold, a leaf value. You never set these by hand.

Search space — The complete list of hyperparameter values you are willing to consider, one list per dial. A search can only ever return something from inside this space.

Information flows one way: hyperparameters go in before training, parameters come out after it. No single training run can discover a good hyperparameter, because the hyperparameter decides how that run behaves in the first place. The only way to choose one is to train several models and compare them.

Table 23.1 lists the dials of the models you have already met. Copy two habits from it. Ranges are spaced by multiplication, not addition — 0.01, 0.1, 1, 10, 100 rather than 1, 2, 3, 4, 5 — because these settings act on very different scales. And most models have only two or three dials that really matter; leave the rest at their defaults on a first pass.

Table 23.1 – The main dials of the models taught in Chapters 8 to 21

Model (chapter)	Main dials	Typical values	What they control
Ridge, lasso, polynomial regression (8)	λ ; degree	0.01–100; 1, 2, 3	Size of coefficients; bend of the curve
Logistic regression (10)	C	0.01–100	How closely the boundary follows points
k -nearest neighbors (11)	k ; weighting	1, 3, 5, 10, 20; uniform or distance	How far a prediction looks
Naïve Bayes (12)	Smoothing α	0.1, 0.5, 1	Weight given to unseen combinations
Decision tree (13)	Max depth; min leaf size	2–20; 1, 5, 20	How finely the tree may cut
Random forest (14)	Trees; max depth; features per split	100–500; 5, 10, none; square root	Number of voters and their depth
Gradient boosting (15)	Learning rate; rounds; max depth	0.01–0.3; 100–1000; 1, 2, 3	Size and number of corrections
SVM, kernel SVM (16, 17)	C ; kernel; γ	0.1–100; RBF; 0.01–10	Price of a mistake; reach of one point

Bahar’s problem is a regression task: predict how many bread rolls the twelve shops will want tomorrow, from the day of the week, the forecast temperature, whether it is a public holiday, and whether a promotion is running. She has 364 days of records. Following Chapter 5, she keeps the last 60 days as a test set and does everything else on the first 304 days. Her metric is mean absolute error (MAE) in rolls per day, from Chapter 6.

She starts with a boosted-tree model (Chapter 15) and two dials: the learning rate and the maximum depth of each tree. It is tempting to tune them one at a time: fix one dial at a convenient value, find the best value of the other, then go back. Four real settings, each scored by five-fold cross-validation, show what that shortcut costs.

WORKED EXAMPLE 23.1 — Tuning One Dial at a Time Misses the Best Pair

Bahar's four validation scores are: rate 0.05 with depth 1, MAE 37.4; rate 0.05 with depth 3, MAE 26.1; rate 0.3 with depth 1, MAE 24.0; rate 0.3 with depth 3, MAE 28.6. She tunes one dial at a time: first hold the rate at 0.05 and pick the better depth, then hold that depth and pick the better rate. Where does she land, and what does it cost her?

1. Hold the rate at 0.05 and compare depths: 37.4 against 26.1. The lower MAE wins, so choose depth 3.
2. Hold depth at 3 and compare rates: 26.1 against 28.6. Again the lower wins, so keep rate 0.05.
3. One dial at a time therefore stops at rate 0.05, depth 3, with MAE 26.1.
4. Compare all four cells directly: 37.4, 26.1, 24.0, 28.6. The lowest is 24.0, at rate 0.3 with depth 1 — a pair the one-at-a-time path never tried.
5. Find the loss: $26.1 - 24.0 = 2.1$ rolls per day.
6. As a percentage of the score she settled for: $2.1 \div 26.1 = 0.080$, or 8.0%.

Answer: One dial at a time lands on 26.1 rolls per day, while the best of the four is 24.0. The shortcut gives away 2.1 rolls per day, about 8.0% of the error, because the two dials interact.

COMMON MISTAKE

X Wrong: Assuming the best value of one hyperparameter, found while every other dial was held fixed, stays best when the other dials change.

✓ Right: Search the dials together. Depth 3 beat depth 1 at a learning rate of 0.05, and lost to it at a rate of 0.3: the best value of one dial depends on where the other one sits.

23.2 Grid Search: Trying Every Combination

Checking four cells by hand was easy. Real projects need a name for the method and a way to count its cost.

WORD TO KNOW

Grid search — A tuning method that lists a few values for each hyperparameter and tries every possible combination of them, training and scoring one model for each combination.

The name comes from the picture. Values of one dial form the rows, values of another form the columns, and every cell is one combination the search must try. Each cell must then be scored, and a fair score means cross-validation (Chapter 5), which trains one model per fold. So the cost is two multiplications, never an addition — and that single fact explains why grid searches get expensive so fast.

EQUATION: COMBINATIONS IN A GRID, AND THE MODEL FITS THEY COST

$$N = v_1 \times v_2 \times \cdots \times v_p = \prod_{j=1}^p v_j, \quad F = N \times k$$

In words: Count how many values you listed for each dial and multiply those counts together; then multiply by the number of cross-validation folds to get the number of models the computer must train.

When it is used: Before starting a search, to turn a grid into a time estimate by multiplying F by the time one fit takes.

Symbol	Meaning	Unit or type
N	number of combinations in the grid	count
v_j	how many values are listed for dial j	count
p	how many hyperparameters are being searched	count
F	total number of model fits	count
k	number of cross-validation folds	count

Adding one value to one list multiplies N , it does not add one to it: $4 \times 3 \times 4 = 48$ becomes $4 \times 4 \times 4 = 64$. Scikit-learn adds one more fit at the end to refit the winner on all the training data, so the true total is $F + 1$; with a single validation set instead of cross-validation, $k = 1$.

WORKED EXAMPLE 23.2 — Count the Combinations and the Fits for Two Grids

(a) Bahar first tunes a k -nearest-neighbor model (Chapter 11) over $k = 3, 5, 10, 20$ and two weighting schemes, uniform and distance, scored with five-fold cross-validation. (b) She then plans a boosted-tree grid with 4 tree counts, 3 depths and 4 learning rates, also five-fold, on a much larger regional dataset where one fit takes about 2 minutes. Count the combinations and the fits for each, and the hours for (b).

1. (a) The lists hold 4 values of k and 2 weighting schemes. Multiply: $4 \times 2 = 8$ combinations.
2. Multiply by the folds: $8 \times 5 = 40$ model fits, or 41 including the final refit.
3. (b) The lists hold 4, 3 and 4 values. Multiply all three: $4 \times 3 \times 4 = 48$ combinations.
4. Multiply by the folds: $48 \times 5 = 240$ model fits.
5. Multiply by 2 minutes per fit: $240 \times 2 = 480$ minutes, and $480 \div 60 = 8$ hours.

Answer: The small grid needs 8 combinations and 40 fits — a few seconds of work. The larger grid needs 48 combinations and 240 fits, which at 2 minutes per fit is 8 hours of computer time. Nothing changed except two extra lists.

Table 23.2 continues the pattern. Every extra value on any list multiplies the whole total, so a grid that felt reasonable on Monday can be impossible by Friday: adding a fourth depth to that $4 \times 3 \times 4$ grid means 80 more complete training runs, from one extra number on one list.

Table 23.2 – How fast a grid grows (five-fold cross-validation, 2 minutes per fit)

Values per dial	Dials	Combinations	Model fits	Hours
4×3	2	12	60	2
$4 \times 3 \times 4$	3	48	240	8
$4 \times 4 \times 4$	3	64	320	10.7
$4 \times 4 \times 4 \times 4$	4	256	1,280	42.7

Bahar's own grid is small enough to look at in full. She lists four learning rates and four maximum depths, giving $4 \times 4 = 16$ combinations and $16 \times 5 = 80$ fits, and draws every resulting score as the heat map in Figure 23.1. Reading a whole grid, rather than only its winner, tells you far more than a single best score ever could.

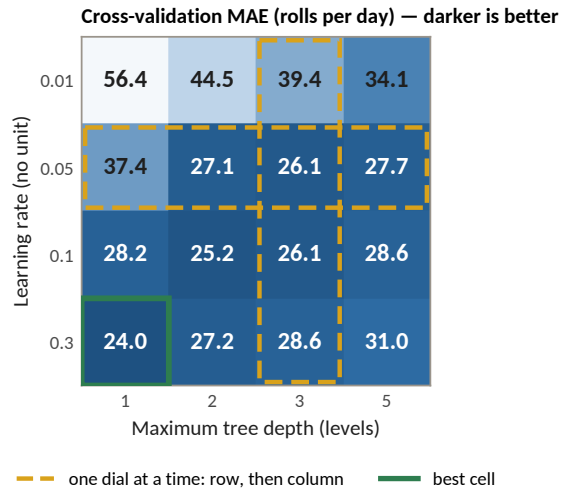


Figure 23.1 – All 16 combinations of learning rate and maximum depth, on Bahar's 304 training days. Gold marks the cells a one-dial-at-a-time search would visit; green marks the best cell.

WHAT TO OBSERVE

The top row is poor whatever the depth: a learning rate of 0.01 is too small for 100 rounds. The good cells form a diagonal band — a fast rate wants shallow trees, a slow rate wants deeper ones. The gold row and column cross at 26.1, exactly where Worked Example 23.1 stopped, two cells away from the green winner. And that winner, 24.0 at rate 0.3 and depth 1, sits on the edge of the grid, which is a warning sign.

Scikit-learn runs the whole search in a few lines, from the same dictionary of dials and values.

Python • scikit-learn

```
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV, KFold

days = pd.read_csv("datasets/ch23_bakery.csv").iloc[:304] # training days only
X = days[["day", "temp_c", "holiday", "promo"]]
y = days["rolls"]
grid = {"learning_rate": [0.01, 0.05, 0.1, 0.3], "max_depth": [1, 2, 3, 5]}
folds = KFold(n_splits=5, shuffle=True, random_state=0)
search = GridSearchCV(GradientBoostingRegressor(random_state=0), grid, cv=folds,
                      scoring="neg_mean_absolute_error").fit(X, y)
print(search.best_params_, round(-search.best_score_, 1))
```

Output

```
{'learning_rate': 0.3, 'max_depth': 1} 24.0
```

The default settings of this model are a learning rate of 0.1 with depth 3, which scores 26.1 on the same folds. Tuning therefore bought Bahar $26.1 - 24.0 = 2.1$ rolls per day, an 8.0% cut in error, for eighty short training runs. That is a typical result: tuning is usually worth a modest, honest improvement, not a miracle.

COMMON MISTAKE

✗ Wrong: Accepting a winning cell that sits in a corner or along an edge of the grid, without asking what lies just beyond it.

✓ Right: When the best cell is on the edge, the true best value may be outside your search space. Bahar extended the grid to a learning rate of 0.5 and scored 24.1 — no better — so 0.3 really was the top. Widening the grid once is cheap; assuming is not.

23.3 Random Search: Trying Many, Not All

Grid search fixes its cost the moment the lists are written: every combination, no exceptions. A different method lets you fix the cost first and let the search fit inside it.

WORDS TO KNOW

Random search — A tuning method that draws a fixed number of hyperparameter combinations at random from the search space, scores each one, and keeps the best it found.

Search budget — The number of combinations you decide in advance to try. It is chosen from the computer time available, not from the size of the grid.

Random search was studied carefully by James Bergstra and Yoshua Bengio in 2012, and their key picture is Figure 23.2. Both panels spend exactly 16 fits, but the grid spends them on only 4 distinct learning rates while the random search spends them on 16. If the learning rate matters far more than the other dial — which is common — random search has looked at the important dial four times more finely for the same money.



Figure 23.2 – Sixteen fits spent two ways. The tick marks along the bottom show how many different learning rates each method actually tested.

WHAT TO OBSERVE

Panel (a) has 16 dots but only 4 tick marks: the grid repeats the same four rates at four row heights. Panel (b) has 16 dots and 16 tick marks. When only one dial really matters, the random search learns much more about it.

The cost rule is the same equation as before, with the budget B in place of the grid size N : total fits = $B \times k$.

WORKED EXAMPLE 23.3 — Compare a Random Budget of 20 With the Full Grid

Instead of the 48-combination grid of Worked Example 23.2, part (b), Bahar’s regional team runs a random search with a budget of 20 combinations drawn from the same value lists, still scored with five-fold cross-validation, still 2 minutes per fit. Find (a) the total fits, (b) the total time, (c) how many fewer fits than the full grid, as a count and as a percentage.

1. Multiply the budget by the folds: $20 \times 5 = 100$ model fits.
2. Multiply by 2 minutes per fit: $100 \times 2 = 200$ minutes.
3. Convert: $200 \div 60 = 3$ with 20 left over, so 3 hours and 20 minutes.
4. Subtract from the grid’s fits: $240 - 100 = 140$ fewer fits.
5. As a percentage of the grid’s cost: $140 \div 240 = 0.583$, or 58.3%.

Answer: The random search needs 100 fits and about 3 hours 20 minutes, against the grid’s 240 fits and 8 hours. It saves 140 fits, 58.3% of the computer time, while still trying 20 genuinely different combinations.

With only 20 draws, what is the chance of missing everything good? It can be calculated, and it is smaller than most people expect.

EQUATION: CHANCE THAT A RANDOM SEARCH REACHES THE TOP

$$P = 1 - (1 - q)^B$$

In words: One draw misses the good region with probability $1 - q$. All B draws miss it with that probability multiplied by itself B times. Subtract from 1 to get the chance of at least one hit.

When it is used: To choose a search budget before running anything, or to defend a budget afterward.

Symbol	Meaning	Unit or type
P	chance of drawing at least one combination from the best fraction	probability from 0 to 1
q	the fraction of combinations counted as “good”	fraction
B	search budget: how many combinations are drawn	count

$q = 0.05$ means the best 5 settings in every 100. The rule assumes every draw is made independently from the same space, which is exactly how random search works.

WORKED EXAMPLE 23.4 — Chance That 20 Random Draws Land in the Best 5 Percent

Bahar’s team wants at least one setting from the best 5% of the search space, so $q = 0.05$. Find the chance of getting one with a budget of 20 draws.

1. Find the chance that one draw misses: $1 - 0.05 = 0.95$.
2. Raise it to the power of the budget, in steps: $0.95^{10} = 0.5987$, so $0.95^{20} = 0.5987 \times 0.5987 = 0.3585$.
3. That 0.3585 is the chance that every one of the 20 draws misses.
4. Subtract from 1: $1 - 0.3585 = 0.6415$, or 64.2%.

Answer: A budget of only 20 draws has about a 64% chance of finding at least one setting from the best 5% of the whole space — without ever knowing how large that space is.

Table 23.3 repeats the calculation for other budgets. Notice the shape: the first draws buy a lot of certainty, later ones buy less and less. Going from 10 draws to 20 adds 24 percentage points; going from 40 to 60 adds only 8.

Table 23.3 – What a random-search budget buys (five-fold cross-validation, 2 minutes per fit)

Budget B	Model fits	Hours	Chance of a top-5% setting
10	50	1.7	40%
20	100	3.3	64%
40	200	6.7	87%
60	300	10.0	95%

The change in code is one word plus a budget: swap `RandomizedSearchCV` for `GridSearchCV` in the listing above and add `n_iter=6` to set B . On Bahar's small grid, those six random draws — about a third of the work — found a learning rate of 0.1 with depth 2, scoring 25.2 rolls against the full grid's 24.0. That is the honest trade: most of the benefit for a fraction of the cost, but no guarantee. Grid search remains the better choice when there are only one or two dials with a handful of sensible values each, since it is cheap and certain. Random search wins once there are three or more dials, or once any one dial has many plausible values.

COMMON MISTAKE

X Wrong: Assuming random search must be worse than grid search because it does not try everything.

✓ Right: With a sensible budget it usually lands very close to the grid's best, in a fraction of the time — and it looks at each individual dial far more finely, which matters when only one or two dials really drive the score.

One warning applies to both methods. A very large search checks many settings against the same folds, so with enough attempts one setting will score well by luck alone — and that lucky one is the one you keep. Keep the search space reasonable, and treat a surprisingly perfect validation score with suspicion rather than celebration.

23.4 Nested Cross-Validation and the Test-Set Trap

Every safeguard so far depends on one rule: the test set stays closed until all tuning is finished. Breaking it is easy, tempting, and almost invisible.

WORD TO KNOW

Test-set peeking — Looking at the test score while still deciding what to try next. Each look turns the test set a little more into a second validation set, and the final reported number stops being an honest estimate.

Suppose Bahar tries five settings and checks each one on her 60 test days, then reports the best of the five. Figure 23.3 shows what happens.

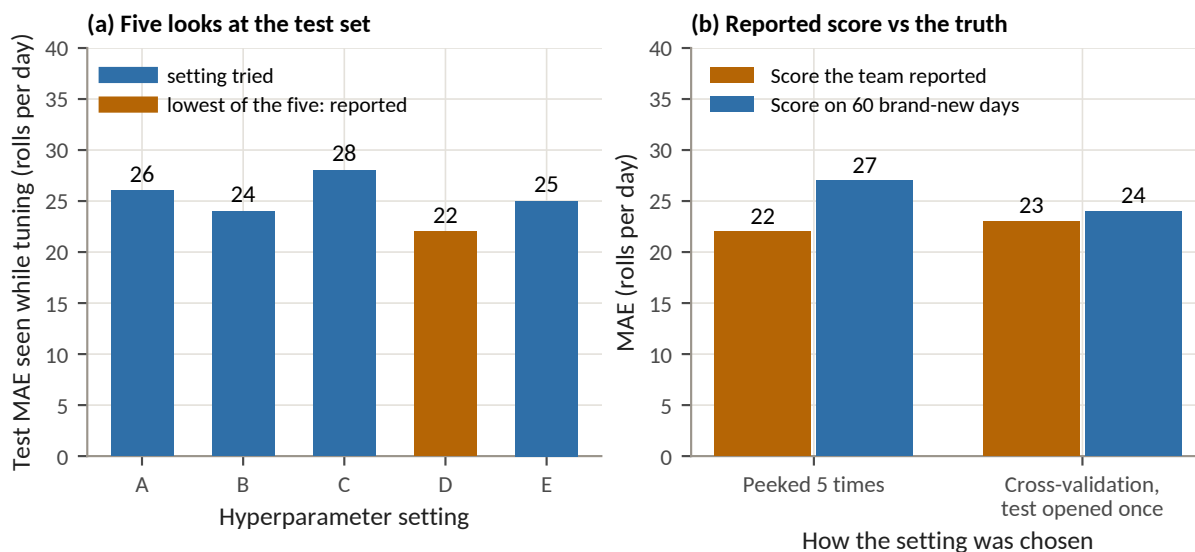


Figure 23.3 – Five looks at the test set (a) and what the reported score is really worth (b). Illustrative numbers.

WHAT TO OBSERVE

In panel (a) the five settings differ by only a few rolls, and setting D happens to be lowest — partly because it suits those particular 60 days. In panel (b), the 5-roll gap on the left is the price of peeking; the 1-roll gap on the right is what honest practice looks like.

WORKED EXAMPLE 23.5 — Peeking, and the Same Choice Made Safely

(a) Bahar tries five settings and checks each on the same 60 test days: A scores MAE 26, B 24, C 28, D 22, E 25 rolls per day. She reports setting D at 22. Sixty brand-new days arrive, and setting D scores 27 on them. Find the size of the illusion. (b) She repeats the comparison honestly, scoring the same five settings by five-fold cross-validation on the 304 training days only: A averages 25, B 24, C 27, D 24, E 23. She picks the lowest and opens the test set once, scoring 24. Compare the two gaps.

1. (a) The five peeked scores are 26, 24, 28, 22, 25; the lowest, and the one reported, is 22 from setting D.
2. Compare it with setting D on 60 completely fresh days: 27.
3. Find the gap: $27 - 22 = 5$ rolls per day, which is $5 \div 22 = 0.23$, or about 23% of the reported number.
4. (b) The lowest cross-validation average is 23, from setting E — not D, whose earlier win was partly luck on those particular test days.
5. Compare E's estimate, 23, with its single test score, 24: the gap is $24 - 23 = 1$ roll per day.

Answer: Peeking made the reported score too good by 5 rolls per day, about 23%. Choosing by cross-validation and opening the test set once left a gap of only 1 roll. The test set was used for measuring, never for choosing.

There is a subtler version of the same problem. Even a cross-validated grid search reports the *best of many* scores, so the winning cell's own score is a little lucky. If you also want an honest estimate of how well the whole tuning procedure performs, you need one more layer.

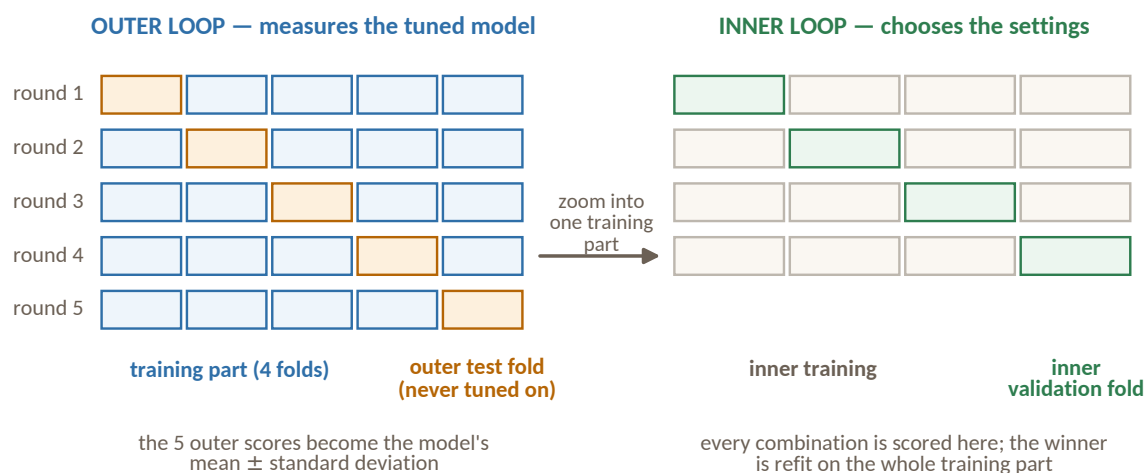
WORDS TO KNOW

Nested cross-validation — Cross-validation with a second cross-validation inside it. The inner one chooses the hyperparameters; the outer one measures the tuned model on data the inner search never saw.

Inner loop — The search that picks the settings, run separately inside each outer training part.

Outer loop — The folds that score the tuned model. Their scores are never used to choose anything.

Figure 23.4 draws both loops. The important discipline is that the whole search is repeated from scratch inside every outer round, so no outer test fold ever influences the settings that are later scored on it. Sudhir Varma and Richard Simon showed in 2006 that skipping this layer can make a tuned model's reported score noticeably too optimistic.



The outer test folds are used only to score. They never help choose a setting.

Figure 23.4 – Nested cross-validation. The inner loop chooses the settings; the outer loop measures the result.

WHAT TO OBSERVE

Count the orange blocks: five outer test folds, one per round, each used once and only for scoring. Inside a round, the green inner validation folds are carved out of the blue training part alone. The two never mix.

EQUATION: TOTAL MODEL FITS IN A NESTED CROSS-VALIDATION

$$F = K_{\text{out}} \times (N \times K_{\text{in}} + 1)$$

In words: Inside one outer round, the search fits one model per combination per inner fold, plus one refit of the winner. Multiply all of that by the number of outer rounds.

When it is used: To budget an honest comparison of tuned models, before starting it.

Symbol	Meaning	Unit or type
F	total number of model fits	count
K_{out}	number of outer folds	count
N	number of combinations in the search	count
K_{in}	number of inner folds	count

The $+1$ is the refit of the winning combination on the whole outer training part, which is the model the outer test fold then scores.

WORKED EXAMPLE 23.6 — Count the Fits in a Nested Cross-Validation

Bahar compares her boosted-tree model honestly. Its grid has 3 learning rates and 3 depths, the inner loop uses 4 folds, and the outer loop uses 5. How many models does the computer train in total?

1. Count the combinations: $3 \times 3 = 9$.
2. Fits in one inner search: $9 \times 4 = 36$.
3. Add the refit of the winner on the outer training part: $36 + 1 = 37$.
4. Multiply by the 5 outer rounds: $5 \times 37 = 185$.

Answer: 185 model fits. Nested cross-validation costs roughly K_{out} times a plain search — five times here — which is why it is used for the final comparison of a few candidates rather than for everyday tinkering.

COMMON MISTAKE

X Wrong: Believing that because a grid search used cross-validation, the test set no longer matters.

✓ Right: Cross-validation protects the *tuning*. The test set still has its own separate job: one honest, final check, opened after every choice has already been locked in.

23.5 Comparing Models Fairly: Same Splits, Same Metric, With Spread

Bahar now has five tuned candidates plus a baseline. Choosing between them is a race, and a race is only meaningful if everyone runs the same track. Three rules make it fair. **Same splits:** every model is scored on exactly the same folds, from the same random seed, because comparing scores from different splits measures the splits, not the models. **Same metric:** one number, chosen before the race (Chapter 6) — here MAE in rolls per day for every candidate. **Same effort:** every candidate gets its own small grid search inside the same nested cross-validation, because comparing a carefully tuned model with a rival left at its defaults compares attention, not models.

IMPORTANT

Any preprocessing that learns from data — scaling, imputing, encoding — must live inside the pipeline (Chapter 4) so that it is refitted within every fold. Fitting a scaler on all the data before splitting leaks information from the validation folds into training, and every score in the comparison becomes too good.

Table 23.4 shows the result: five outer folds for six candidates, each MAE rounded to whole rolls so the arithmetic stays easy to check by hand.

Table 23.4 – MAE (rolls per day) of six candidates on the same five outer folds, each tuned by an inner grid search

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std. dev.
Baseline (mean)	63	63	63	62	76	65.4	5.94
Ridge regression	49	51	51	48	56	51.0	3.08
k -nearest neighbors	37	35	32	30	35	33.8	2.77
Decision tree	27	30	27	31	33	29.6	2.61
Random forest	25	27	25	28	29	26.8	1.79
Gradient boosting	21	25	21	27	25	23.8	2.68

A single mean is never enough. Chapter 5 already showed why, and gave both formulas: the mean of the k fold scores, $\bar{s} = \frac{1}{k} \sum_{j=1}^k s_j$, and their standard deviation s_{sd} , which divides by $k - 1$ (`std(ddof=1)` in NumPy). Every cross-validation result in this book is written as $\bar{s} \pm s_{sd}$, in a table, a chart and the final report alike.

WORKED EXAMPLE 23.7 — Mean and Spread of Five Fold Scores

From Table 23.4, the random forest scored 25, 27, 25, 28, 29 and gradient boosting scored 21, 25, 21, 27, 25. Find each model's mean and standard deviation.

1. Forest mean: $(25 + 27 + 25 + 28 + 29) \div 5 = 134 \div 5 = 26.8$.
2. Forest deviations $-1.8, 0.2, -1.8, 1.2, 2.2$ squared and added: $3.24 + 0.04 + 3.24 + 1.44 + 4.84 = 12.8$. Divide by $k - 1 = 4$: 3.2 ; square root: $\sqrt{3.2} = 1.79$.
3. Boosting mean: $(21 + 25 + 21 + 27 + 25) \div 5 = 119 \div 5 = 23.8$.
4. Boosting deviations $-2.8, 1.2, -2.8, 3.2, 1.2$ squared and added: $7.84 + 1.44 + 7.84 + 10.24 + 1.44 = 28.8$. Divide by 4: 7.2 ; square root: $\sqrt{7.2} = 2.68$.

Answer: The random forest scores 26.8 ± 1.79 and gradient boosting scores 23.8 ± 2.68 rolls per day. Boosting has the lower mean but the wider spread, and the two ranges overlap heavily — so the means alone cannot settle the question.

Here is where the “same splits” rule pays for itself. A fold that was hard for one model was hard for the other too, so subtracting within each fold cancels that shared difficulty out.

WORD TO KNOW

Paired comparison — Comparing two models by subtracting their scores fold by fold, then studying the five differences, instead of comparing two separate averages. It only works when both models were scored on exactly the same folds.

EQUATION: PAIRED DIFFERENCE BETWEEN TWO MODELS

$$d_j = a_j - b_j, \quad \bar{d} = \frac{1}{k} \sum_{j=1}^k d_j, \quad SE_d = \frac{s_d}{\sqrt{k}}$$

In words: On each fold, subtract model B's score from model A's. Average those differences, and divide their standard deviation by the square root of the number of folds.

When it is used: To decide whether one model is genuinely ahead of another. Call the difference clear only when $|\bar{d}| > 2 SE_d$.

Symbol	Meaning	Unit or type
d_j	difference on fold j	same unit as the score
$\bar{d}$	mean of the differences	same unit as the score
s_d	standard deviation of the differences	same unit as the score
SE_d	standard error of the mean difference	same unit as the score
k	number of folds	count

The two-standard-error rule is a rough guide, in the spirit of the fold-by-fold comparison described by Thomas Dietterich in 1998. Folds share training rows, so treat it as a warning light rather than a proof.

WORKED EXAMPLE 23.8 — Is Gradient Boosting Really Better Than the Forest?

Using the same five folds, subtract gradient boosting's MAE from the random forest's on each fold, and decide whether the difference is real.

1. Fold by fold: $25 - 21 = 4$, $27 - 25 = 2$, $25 - 21 = 4$, $28 - 27 = 1$, $29 - 25 = 4$.
2. Every difference is positive: the forest was worse on all five folds.
3. Mean difference: $(4 + 2 + 4 + 1 + 4) \div 5 = 15 \div 5 = 3.0$ rolls per day.
4. Deviations: 1, -1, 1, -2, 1. Squares: 1, 1, 1, 4, 1; sum 8; divided by 4 gives 2; $s_d = \sqrt{2} = 1.41$.
5. Standard error: $1.41 \div \sqrt{5} = 1.41 \div 2.236 = 0.632$.
6. Two standard errors: $2 \times 0.632 = 1.26$. Compare with the mean difference: $3.0 > 1.26$.

Answer: Gradient boosting is genuinely better, by about 3.0 rolls per day. The $\pm$ bars of Worked Example 23.7 overlapped, but the paired differences do not: because the same folds were used, the shared fold-to-fold difficulty cancels and a real 3-roll gap becomes visible.

COMMON MISTAKE

✗ Wrong: Deciding that two models are tied because their $\bar{s} \pm s_{sd}$ ranges overlap on a chart.

✓ Right: Overlapping bars are not the test. When both models were scored on the same folds, compare the *paired differences*. Here the bars overlap and yet the winner is clear on every single fold.

23.6 Model-Comparison Tables and Charts

The comparison now has to be communicated to people who will not read the code. Two pictures do the work, and Figure 23.5 draws both: a ranked bar chart whose whiskers carry the spread, and a box plot of the individual folds underneath it, because a bar chart alone hides where the surprises live.

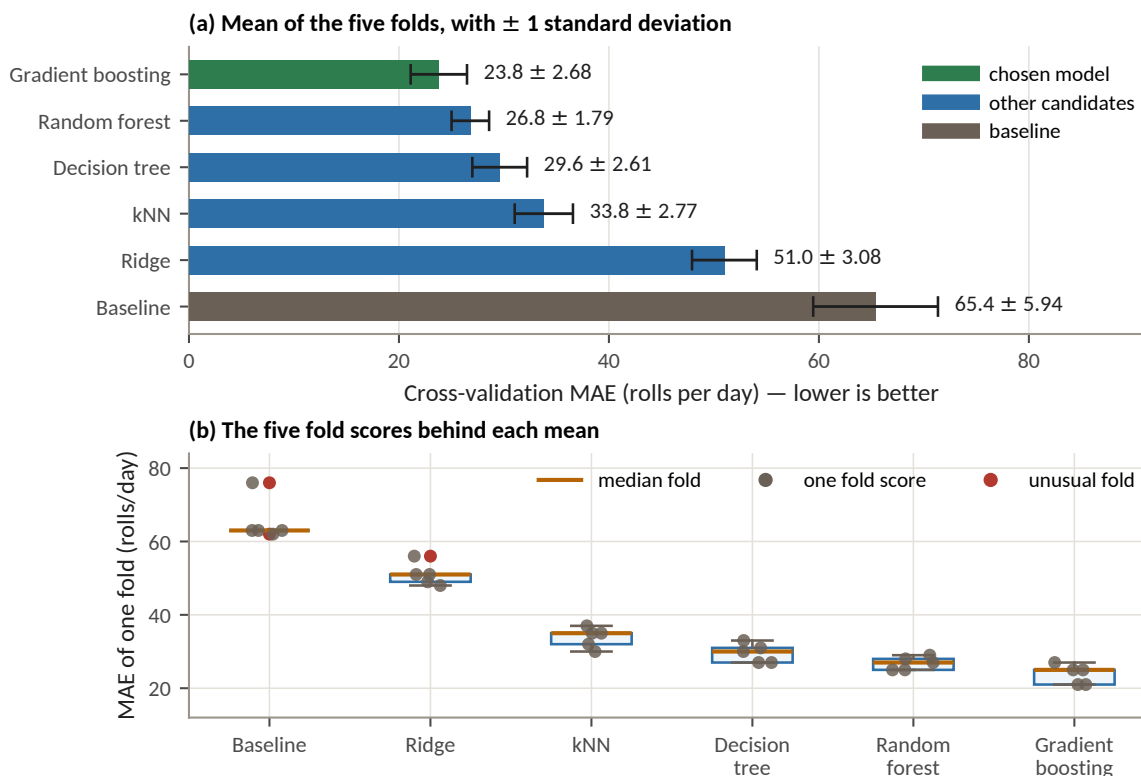


Figure 23.5 – The six candidates ranked by mean cross-validation MAE (a), and the five fold scores behind each mean (b).

WHAT TO OBSERVE

In panel (a) the biggest single step is from the baseline to ridge regression, and the next biggest from ridge to k -nearest neighbors; after that each model buys less than the one before, and the last two whiskers overlap — exactly what the paired comparison was invented for. In panel (b), fold 5 was the hardest fold for every candidate, and for the baseline it lands as an unusual point at 76 rolls. A bad fold that hurts everyone is a property of that slice of days, perhaps a heat wave or a holiday week, not a fault of any model; because all six met the same fold 5, the ranking is unaffected.

Two percentages belong in every report of this kind. Gradient boosting removes $65.4 - 23.8 = 41.6$ rolls of the baseline’s error, which is $41.6 \div 65.4 = 63.6\%$ of it. But ridge regression — the simplest real model, fitted in two milliseconds — already removes $65.4 - 51.0 = 14.4$ rolls, or 22.0%, on its own. Report the total gain and the share the cheapest model got for free, so nobody mistakes a complicated model for the whole reason a forecast works.

Table 23.5 is the table Bahar would actually put in a report: the score with its spread, the two costs, and one plain-language note per model.

Table 23.5 – Model-comparison table for the bread-roll forecast. Times measured once on one ordinary laptop and rounded; they vary between machines.

Model	CV MAE (rolls/day)	Fit time (ms)	Predict 1000 rows (ms)	Can a person follow it?
Baseline (mean)	65.4 ± 5.94	< 1	< 0.01	Yes, trivially
Ridge regression	51.0 ± 3.08	2	0.07	Yes: one coefficient per feature
k -nearest neighbors	33.8 ± 2.77	2	2.31	Partly: shows similar past days
Decision tree	29.6 ± 2.61	1	0.03	Yes: read the branch
Random forest	26.8 ± 1.79	140	5.36	No: 100 trees vote
Gradient boosting	23.8 ± 2.68	55	0.45	No, but importances help

Sometimes two candidates really are tied. A useful habit then comes from the authors of the decision-tree book *Classification and Regression Trees* — Leo Breiman, Jerome Friedman, Richard Olshen and Charles Stone, in 1984.

WORDS TO KNOW

One-standard-error rule — Among all models whose score is within one standard error of the best score, choose the simplest one, rather than the one with the best number.

Model selection — The whole decision — which algorithm, with which settings — made using validation evidence plus cost, speed and interpretability, and never using the test set.

Bahar's best two are not tied, so she keeps gradient boosting. But if the forest had scored 24.0 ± 1.8 against boosting's 23.8, the standard error of the difference would have swallowed the gap, and the rule would have told her to keep whichever model is simpler to run and explain — a decision she can defend to anyone.

23.7 Beyond the Score: Cost, Speed, Interpretability, Data Needs

A model that wins on MAE can still be the wrong choice. Four other questions decide almost as often as the score does.

WORDS TO KNOW

Inference time — How long a trained model takes to produce predictions, usually quoted per thousand rows. It matters when predictions must appear while somebody waits.

Interpretability — How easily a person can follow why a model produced a particular prediction, in words another person would accept.

Training cost is what you pay each time the model is refreshed; **inference time** is what you pay on every prediction, forever; **interpretability** decides whether a shopkeeper, a doctor or a judge can be told why; and **data needs** decide whether the model can work at all, since a boosted ensemble needs far more rows than a ridge regression before it stops memorizing. Figure 23.6 plots two of them together for Bahar's candidates.

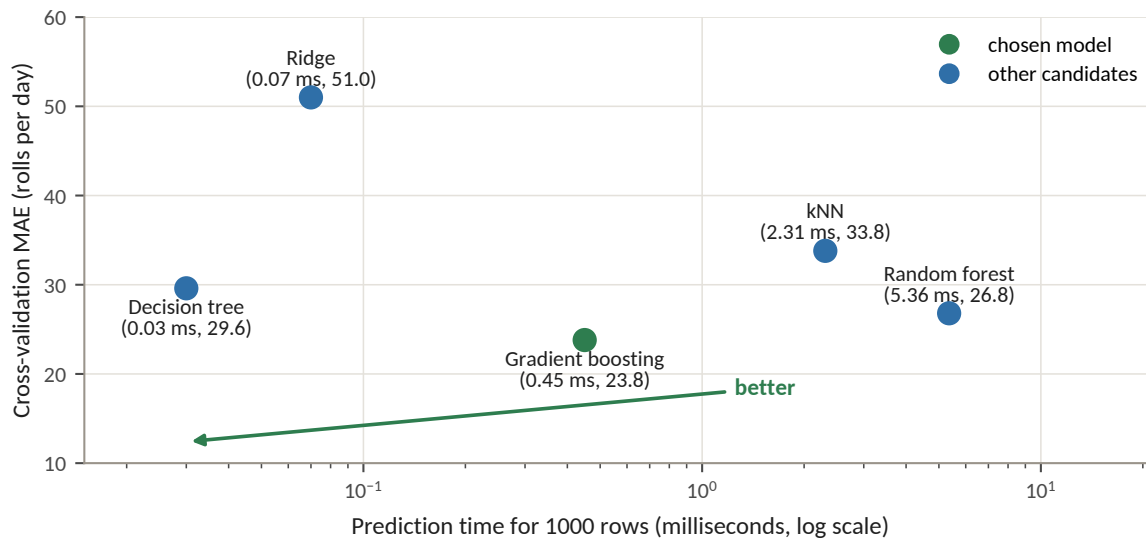


Figure 23.6 – Error against prediction speed for five tuned candidates. Down and to the left is better.

WHAT TO OBSERVE

The random forest is both worse and about twelve times slower to predict than gradient boosting, so nothing recommends it here. The decision tree is the fastest of all and only 5.8 rolls behind the winner — the trade Bahar would make if she needed a model she could print on one page.

The same picture holds across the whole toolbox of Chapters 8 to 21. Ridge, lasso and logistic regression are cheap to train, almost free to predict and easy to explain, but they need scaled features (Chapter 4). A single decision tree is nearly as cheap and the easiest of all to read aloud. Naive Bayes is cheap and needs no scaling. k -nearest neighbors costs nothing to train but the most of any model to predict, because it searches the stored data every time. Tree ensembles — random forests and gradient boosting — cost the most to train and cannot be read directly, though feature importances help. Kernel SVMs are expensive on both counts and need scaling as well.

RESPONSIBLE USE

A model chosen for the best average score can still be the worst choice for a small group inside the data. Before selecting a model that affects people — patients, borrowers, students, job applicants — compare the candidates' errors *inside meaningful subgroups* as well as overall, and count a model that fails one group badly as a failed candidate, whatever its headline number. Chapter 24 returns to this in full.

The safest habit is to write the project's hard limits down *before* the comparison, and to treat any candidate that breaks them as disqualified, however good its score looks afterward.

23.8 A Checklist, and Reporting the Final Result

Figure 23.7 collects the chapter into one route, from “we have data” to “we have a model we can defend”.

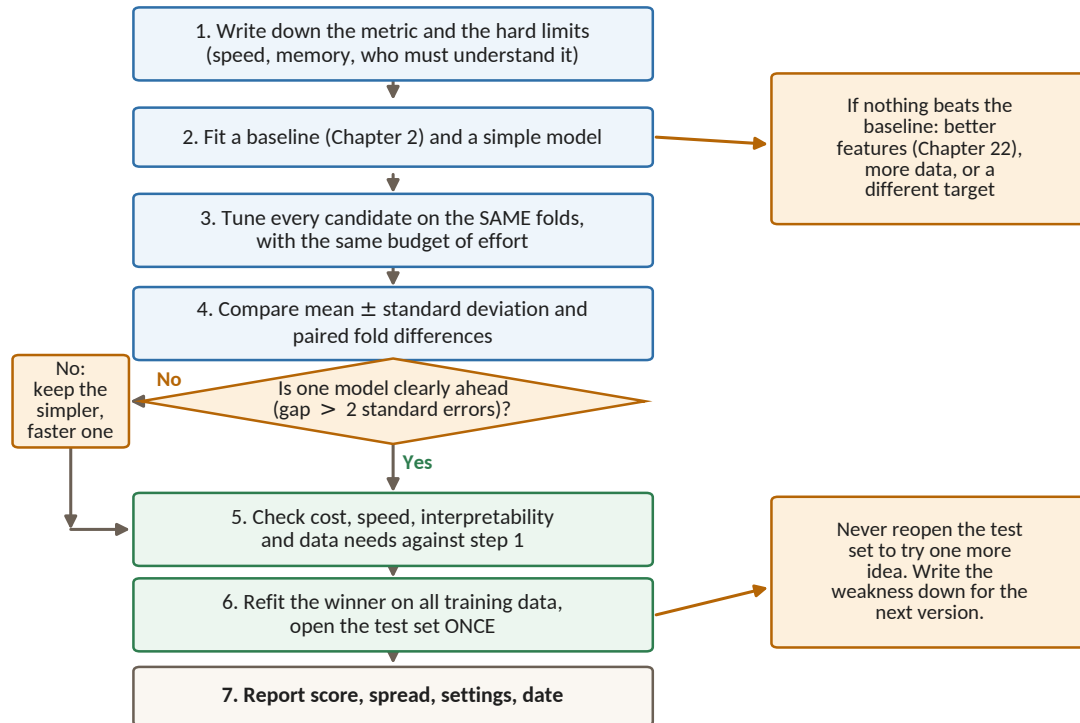


Figure 23.7 – The model-selection route. Steps 1 to 4 use only training data; step 6 opens the test set once.

WHAT TO OBSERVE

Only one box touches the test set, and it comes after every decision is already made; everything above it happens inside the training data. The two orange notes mark where teams most often go wrong: giving up too early, and reopening the test set too late.

Bahar has followed steps 1 to 5. Her winner is gradient boosting with a learning rate of 0.3 and a maximum depth of 1, tuned entirely within the 304 training days. She refits it on all of them and opens her 60 test days once.

WORKED EXAMPLE 23.9 – Read the Final Test Score Honestly

Bahar’s nested cross-validation estimated her tuned boosted-tree model at 23.8 rolls per day. On the 60 test days, opened for the first and only time, it scores 23.5. Should this worry her?

1. Write down the two numbers: the estimate, 23.8, and the one-time test score, 23.5.
2. Find the difference: $23.8 - 23.5 = 0.3$ rolls per day, a little over 1% of an error of about 24 rolls.
3. Compare with the baseline on the same test days, so the number means something: it is far behind, at about 68.

Answer: No. The test score, 23.5 rolls per day, sits 0.3 rolls from the estimate — a small difference in either direction is normal, and a rise is in fact slightly more common than a fall. Because the test days were never used to choose anything, 23.5 is the number Bahar reports.

The last step is writing it down. A model that nobody can reproduce is not finished, however good its score.

WORD TO KNOW

Model report — A short, fixed record of the chosen model: what it predicts, on what data, with which settings, scored how, by whom and when. It lets a future colleague reproduce the result instead of starting over.

Table 23.6 – The model report template, filled in for Bahar’s bread-roll forecast

What to record	Bahar’s forecast
Task and metric	Predict tomorrow’s total bread rolls for twelve shops; MAE in rolls per day
Data and split	364 days; first 304 for training and tuning, last 60 held out as a time-based test set
Candidates compared	Baseline, ridge, k -NN, decision tree, random forest, gradient boosting
How tuned	Grid search inside a 5-outer, 4-inner nested cross-validation; 185 fits for the winner
Chosen model and settings	Gradient boosting, learning rate 0.3, maximum depth 1, 100 rounds, seed 0
Validation and test results	MAE 23.8 ± 2.68 over 5 outer folds; 23.5 on the 60 held-out days, opened once
Known weaknesses	Only one year of data; no Ramadan or Newroz week in the test period
Author, date and review	Name, the date the test set was opened, and a review three months after launch

COMMON MISTAKE

✗ Wrong: Reporting only the winning score: “our model gets 23.5”.

✓ Right: Report the score, its spread, the metric, the split, the exact settings and the date. A number without those five things cannot be checked, reproduced or trusted.

COMMON MISTAKES AND MISCONCEPTIONS

- Tuning one dial at a time and assuming the winner still wins once another dial moves.
- Forgetting to multiply by the number of folds when estimating a search, or thinking an extra value on a list adds to the cost instead of multiplying it.
- Accepting a winning cell that sits on the edge of the grid without checking what lies just outside it.
- Checking the test set after each new setting, then reporting the best of those looks as if it were one fair score, or believing that cross-validation during tuning makes the test set unnecessary.
- Comparing models scored on different splits, with different metrics, or with very different amounts of tuning effort.
- Declaring two models tied because their $\pm$ bars overlap, when the paired fold differences settle the question clearly.
- Choosing the top of the leaderboard without checking speed, memory, explanation and subgroup limits, or reporting a score with no spread, no settings and no date.

Practical Activities

ACTIVITY (PAPER) — Fill In a 3 by 3 Grid and Choose

A clinic in Sulaymaniyah tunes a decision tree that predicts a patient's waiting time in minutes, over maximum depth (2, 4 or 6) and minimum leaf size (1, 5 or 20). Five-fold cross-validation gives these MAE values, in minutes — depth 2: 14.8, 14.6, 15.1; depth 4: 13.4, 12.5, 13.1; depth 6: 12.2, 12.9, 13.6 (leaf sizes 1, 5, 20 in that order).

1. Draw a 3 by 3 table, depth down the side and minimum leaf size across the top, and write each value in its cell.
2. Use the equations of Lesson 23.2 to state how many combinations the grid holds and how many models were trained.
3. Circle the best cell. Is it on an edge or a corner? Write one sentence saying what you would try next because of that.
4. Now tune one dial at a time, starting at depth 2 and leaf size 1. Which cell do you end on, and how many minutes worse is it than the best cell?
5. The best cell's five fold scores have a standard deviation of 0.9 minutes. Find its standard error ($0.9 \div \sqrt{5}$), add it to the best score to get a threshold, and list every cell at or below it. Then apply the one-standard-error rule: of those cells, choose the one that gives the simplest tree, and explain the choice to the clinic manager in one sentence.

You need: Paper, pen and a calculator

ACTIVITY (PYTHON) — Nested Cross-Validation and a Paired Comparison

Python • scikit-learn

```
import numpy as np, pandas as pd
from sklearn.model_selection import KFold, GridSearchCV, cross_val_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
days = pd.read_csv("datasets/ch23_bakery.csv").iloc[:304] # training days only
X = days[["day", "temp_c", "holiday", "promo"]].to_numpy(float)
y = days["rolls"].to_numpy(float)
outer = KFold(n_splits=5, shuffle=True, random_state=0) # same folds for everyone
inner = KFold(n_splits=4, shuffle=True, random_state=1)
MAE = "neg_mean_absolute_error"

def nested(name, model, grid):
    # tune inside, score outside
    search = GridSearchCV(model, grid, cv=inner, scoring=MAE)
    folds = np.round(-cross_val_score(search, X, y, cv=outer, scoring=MAE))
    print(name, folds, "mean %.1f sd %.2f" % (folds.mean(), folds.std(ddof=1)))
    return folds

forest = nested("Forest ", RandomForestRegressor(n_estimators=100, random_state=0),
                {"max_depth": [5, 10, None]})
boost = nested("Boosting", GradientBoostingRegressor(random_state=0),
               {"learning_rate": [0.05, 0.1, 0.3], "max_depth": [1, 2, 3]})
d = forest - boost # paired differences, fold by fold
se = d.std(ddof=1) / np.sqrt(len(d))
print("differences", d, "mean %.1f" % d.mean(), "2 x SE %.2f" % (2 * se),
      "clearly better?", abs(d.mean()) > 2 * se)
```

Output

```
Forest    [25. 27. 25. 28. 29.] mean 26.8 sd 1.79
Boosting  [21. 25. 21. 27. 25.] mean 23.8 sd 2.68
differences [4. 2. 4. 1. 4.] mean 3.0 2 x SE 1.26 clearly better? True
```

This program repeats Table 23.4 for the two closest candidates and then runs the paired test of Worked Example 23.8; it takes about half a minute. GridSearchCV sits *inside* `cross_val_score`: the search is repeated from scratch on each outer training part, so no outer fold helps choose its own settings. The printed folds, means and standard deviations are exactly the last two rows of Table 23.4, and the last four lines reproduce Worked Example 23.8. Change `random_state=0` in `outer` to 1 and rerun: the numbers move, but boosting still wins.

You need: A computer with Python, NumPy, pandas and scikit-learn, and the file `datasets/ch23_bakery.csv` (see Appendix C)

CASE STUDY — From Tuning in Mexico to Choosing in Erbil

A supermarket chain with more than eighty stores across several Mexican cities forecasts how much of each product to stock each week. Order too little and shelves are empty during the weekend rush; order too much and fresh produce spoils. Their boosted-tree model, left at its default settings, reaches a mean absolute error of about 18 units per product per week — barely better than the old rule of thumb, and worst around holidays, exactly when it matters most.

The team writes down a full grid first: four dials — tree count, depth, learning rate and minimum leaf size — with four values each. That is $4 \times 4 \times 4 \times 4 = 256$ combinations, and with five-fold cross-validation $256 \times 5 = 1,280$ model fits, far more than the shared server can deliver before the next planning cycle. So they fix the cost instead: a random search with a budget of 60 combinations, $60 \times 5 = 300$ fits, sized to finish in one overnight run. By the equation of Lesson 23.3, a budget of 60 has about a 95% chance of reaching the best 5% of the space — a number they can show when someone asks why they did not try everything. Halfway through the run an analyst asks whether they could peek at the test set, just to see how the best setting is doing. The team says no. When the search finishes, the winning combination is checked against the held-out weeks exactly once, and the error falls to about 11 units per product per week — a number they can defend, because the test weeks never influenced a single choice.

Two thousand kilometers away, an estate agency in Erbil, Hawler Homes, faces the second half of the same problem. They are not tuning one model; they are choosing between four. Following Lesson 23.5, all four are scored on identical five-fold splits with the same metric, MAE in million IQD, each inside the same pipeline so that scaling and encoding are refitted within every fold. The baseline scores 28.4, ridge regression 16.2, the random forest 12.8 and gradient boosting 11.9 million IQD.

Boosting has the lowest mean, but the team runs the paired test before celebrating. Fold by fold, the forest minus boosting differences are 1.8, -0.6 , 2.2, 0.1 and 1.0, with mean 0.9 and standard deviation 1.16, so the standard error is $1.16 \div \sqrt{5} = 0.52$ and two standard errors is 1.04. The mean gap, 0.9, is smaller than that, and one fold even favored the forest — so on this evidence the two are tied. The choice is therefore made on the other columns: the boosted model predicts about twelve times faster and its saved file is a fraction of the size, so it goes into production. Opened once, the test set gives 12.4 million IQD against the validation estimate of 11.9, a rise of 0.5, the small expected step from an estimate to an untouched measurement.

The team writes the model report of Table 23.6, including one weakness they will not hide: homes above 150 million IQD are rare in the training data and are predicted noticeably worse. They schedule a review in three months, and they resist the strongest temptation of the whole project — retuning the model and checking those same test homes again to see whether the new settings look better. That weakness becomes the first goal of the next version instead.

CHAPTER SUMMARY

- Parameters are learned during training; hyperparameters are set before it, and no single fit can discover a good one.
- Hyperparameters interact, so they must be searched together — the best depth at one learning rate can be the worst depth at another.
- Grid search tries every combination, $N = v_1 \times v_2 \times \cdots \times v_p$, at a cost of $F = N \times k$ model fits plus one refit. Each extra value multiplies the total, it does not add to it.
- Random search fixes the budget in advance. With budget B , the chance of reaching the best fraction q is $P = 1 - (1 - q)^B$: 20 draws give about a 64% chance of reaching the best 5%.
- The test set must never influence a choice. Peeking after each setting turns the reported score into the best of several looks.
- Nested cross-validation puts the search in an inner loop and the scoring in an outer loop, costs $K_{\text{out}}(N \times K_{\text{in}} + 1)$ fits, and measures the whole tuning procedure honestly.
- Fair comparison means the same folds, the same metric and the same tuning effort for every candidate, with all preprocessing inside the pipeline.
- Report every cross-validation score as mean $\pm$ standard deviation, and settle close races with paired fold differences: the gap is clear only when $|\bar{d}| > 2 \text{SE}_d$.
- The score is one column among several: training cost, prediction time, interpretability, data needs and subgroup fairness can all overrule it.
- Finish with a model report: task, metric, split, candidates, settings, validation result with spread, one-time test result, weaknesses, author and date.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain the difference between a model parameter and a hyperparameter in your own words, and give one example of each from any model in Chapters 8 to 21.

Problem 2 [Basic] A grid lists 3 values for one dial and 6 values for another. How many combinations does the grid hold?

Problem 3 [Basic] Define grid search and random search in one sentence each, and state the one thing a team must decide before starting a random search.

Problem 4 [Basic] Explain, in two or three sentences, why the test set must not be used while hyperparameters are being chosen.

Problem 5 [Basic] A random search has a budget of 15 combinations, scored with 4-fold cross-validation. Find the total number of model fits.

Problem 6 [Standard] A grid lists 5 values for dial A, 2 for dial B and 4 for dial C. (a) Find the number of combinations, showing each multiplication. (b) Find the total fits with 10-fold cross-validation. (c) One fit takes 90 seconds; find the total time in hours.

Problem 7 [Standard] Model P scores 0.82, 0.86, 0.80, 0.88 and 0.84 on five folds. Find its mean and standard deviation, showing every step, and write the result as mean $\pm$ standard deviation.

Problem 8 [Standard] Model Q scores 0.85, 0.87, 0.83, 0.90 and 0.85 on the *same* five folds as Model P in Problem 7. Find the five paired differences, their mean, their standard deviation and their standard error, and decide whether Q is clearly better than P.

Problem 9 [Standard] A nested cross-validation uses 5 outer folds, 3 inner folds and a search with 12 combinations. Find the total number of model fits, showing every step.

Problem 10 [Standard] Using Table 23.5: (a) which model would you choose if predictions had to be produced for one million rows every minute? (b) Which would you choose if a shopkeeper had to be told, in one sentence, why today's number is what it is? Justify each answer with a column from the table.

Problem 11 [Challenge] A grid search covers 4 dials with 4 values each, scored with 5-fold cross-validation. (a) Find the combinations and the total fits. (b) At 45 seconds per fit, find the total time in hours. (c) Suggest two practical ways to cut this cost that do not amount to guessing at good values.

Problem 12 [Challenge] The best cell of a 4×4 grid turns out to be in a corner, at the largest learning rate and the smallest depth. (a) What does that position suggest about the search space? (b) Describe exactly what you would run next, and what result would convince you that the corner really is the best. (c) A neighboring cell scores within one standard error of the corner and uses a simpler model; state which you would ship and why.

Problem 13 [Challenge] Two teams report on the same dataset. Team 1 says "accuracy 0.91". Team 2 says "accuracy 0.89 ± 0.02 over 5 folds with seed 0, chosen from 12 settings by nested cross-validation, test set opened once at 0.88 on 4 March". (a) Which report would you trust, and why? (b) Name three specific things Team 1 must add before their number can be checked. (c) Explain how Team 1's higher number could be true and still be worse than Team 2's.

Interpretability, Fairness, Privacy, and Responsible Machine Learning

Why “it works” is not enough

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a strong test score alone does not prove that a model is safe, fair or trustworthy, and read a model's coefficients and importance scores for what each can and cannot tell you.
- Compute permutation importance and a partial-dependence value by hand.
- Compare group-level outcomes — selection rate and false-negative rate — and calculate the gaps between two groups.
- Explain why two reasonable fairness definitions can conflict mathematically, so that a person must choose between them.
- Identify a protected attribute and the proxy variables that can quietly rebuild it, and apply the k -anonymity check to a small table.
- Detect data drift with a percent-change calculation, and decide when an alert crosses a pre-agreed rollback line.

Introduction

Imagine a pharmacist in Sulaymaniyah who hands you a medicine every morning but refuses to say what it treats. The medicine might even work, most days, for most people. Would you keep taking it forever without ever asking why? Almost nobody would. We want to understand things that affect our health, our money and our future — not only to see that they usually work.

A model that makes decisions about people deserves the same question. The first twenty-three chapters taught you to build a model and measure it honestly, but a strong score answers only one question: does this model perform well on data like the data it was tested on? It says nothing about three others. Can a person explain why the model decided what it decided? Does it treat every group of people equally well? And was the personal data behind it collected in a way the people in it would accept? This chapter is about those three questions, and about a fourth that only appears after launch: is the model still right today?

You will read coefficients and importance scores, break features on purpose, count errors group by group, check a small table for k -anonymity, and measure drift with an average and a percent change — every calculation small enough for one sheet of paper.

24.1 Why “It Works” Is Not Enough

A team in Erbil trains a model that decides which loan applications an officer reads first. It reaches 91% test accuracy, measured honestly the way Chapter 6 teaches. But 91% is a summary of the past: it does not say which features the model leaned on, whether it works as well for a farmer near Zakho as for a shop owner downtown, or what happens next year when rates change.

WORD TO KNOW

Interpretability — How well a person can understand why a model made a particular prediction, and what patterns it relies on in general. An interpretable model is easy to check, question and explain to somebody else.

Models differ enormously here, as Figure 24.1 shows. A linear or logistic regression (Chapter 8, Chapter 10) hands you one coefficient per feature, and a shallow decision tree (Chapter 13) is a short list of yes/no questions. A random forest (Chapter 14) or a boosted ensemble (Chapter 15) may add hundreds of trees together, and a kernel SVM (Chapter 17) draws its boundary in a space nobody can picture. Such models are called “black boxes” — not because they hide anything, but because their calculation is too large to walk through step by step.

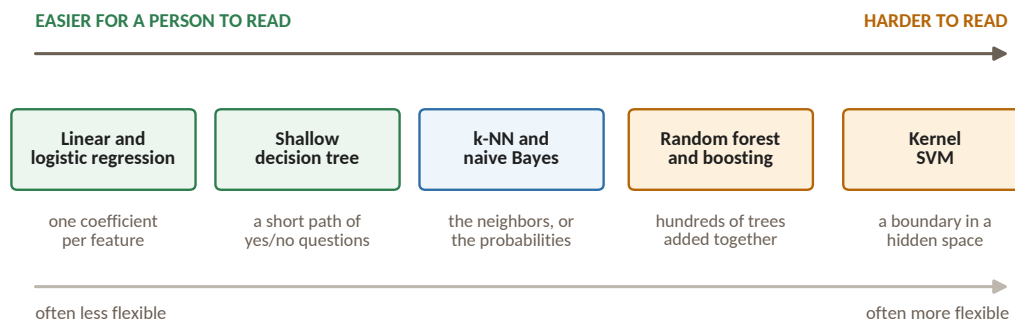


Figure 24.1 – The interpretability spectrum. Models on the left hand you numbers you can read directly; models on the right need extra tools before a person can see inside them.

WHAT TO OBSERVE

The two ends of the arrow pull against each other: the models easiest to read are often the least flexible. This chapter does not ask you to give up accuracy — it opens the box far enough to check a flexible model while you keep using it.

Two kinds of explanation answer two different questions.

WORDS TO KNOW

Global explanation — What a model does *in general*, across all examples — for instance, which features it relies on most.

Local explanation — Why the model gave *one particular example* the answer it did — the explanation a rejected applicant actually wants.

This chapter builds global explanations, the ones you can compute by hand. Local explanations have their own tools — LIME, published by Marco Ribeiro, Sameer Singh and Carlos Guestrin in 2016, and SHAP values, by Scott Lundberg and Su-In Lee in 2017 — which fit a small, readable model around one example.

24.2 Reading Coefficients and Feature Importance

The most direct way to understand a model is to read the numbers it already carries. Sunrise Parcels, the delivery company of Chapter 14, fits a linear regression predicting a delivery's delay in minutes from four features:

$$\widehat{\text{delay}} = 4 + 1.5 x_{\text{distance}} + 2.0 x_{\text{stops}} - 0.8 x_{\text{experience}} + 6.0 x_{\text{rain}}$$

Distance is in kilometers, stops counts the deliveries made before this one, experience is the driver's years on the job, and rain is 1 on a heavy-rain day. A coefficient (Chapter 8) is one of the most readable numbers in machine learning: its sign gives the direction of the effect and its size gives how far the prediction moves for one extra unit, with every other feature held still.

WORKED EXAMPLE 24.1 — Read What a Coefficient Predicts, and What Changing It Would Do

Find the predicted delay for a delivery 12 km away with 3 stops before it, a driver with 0 years of experience and no rain. Then find the new prediction with 4 stops instead, everything else unchanged.

1. Multiply the distance coefficient by the distance: $1.5 \times 12 = 18$.
2. Multiply the stops coefficient by the number of stops: $2.0 \times 3 = 6$.
3. The experience and rain terms are zero here, so add the intercept and the two terms: $4 + 18 + 6 = 28$.
4. Change only the stops, from 3 to 4: $2.0 \times 4 = 8$.
5. Add again: $4 + 18 + 8 = 30$.

Answer: 28 minutes, rising to 30 with one extra stop — exactly the 2-minute stops coefficient. A coefficient gives the direction and the size without retraining anything.

There is a trap in that list. Rain has the biggest coefficient, 6.0, so it is tempting to call rain the most important feature. But a coefficient's size depends on the *scale* of its feature: rain only moves from 0 to 1, while distance moves by tens of kilometers. Figure 24.2 compares the raw coefficients with the same coefficients multiplied by a typical change — one standard deviation, here 6 km for distance, 1.5 for stops, 3 years for experience and 0.4 for rain.

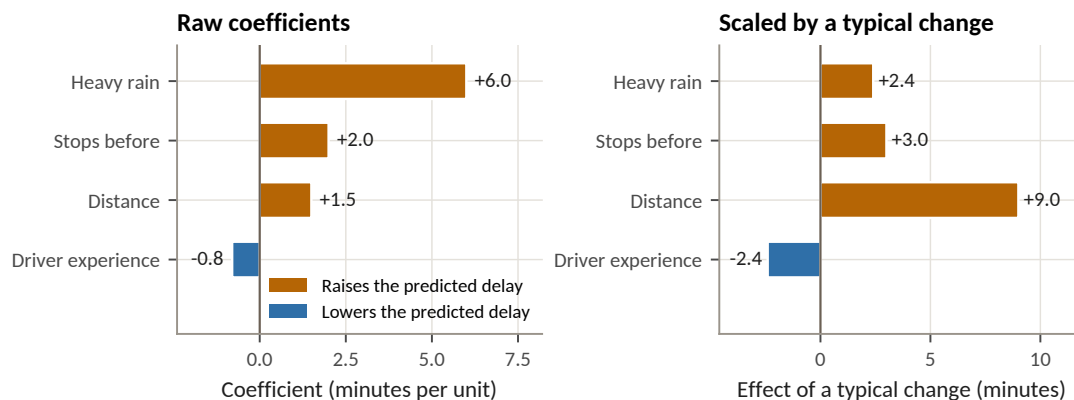


Figure 24.2 – The delay model's coefficients. Left: the raw numbers. Right: each coefficient multiplied by one standard deviation of its own feature, which is how far that feature typically moves.

WHAT TO OBSERVE

The ranking flips. Raw, rain looks four times as important as distance (6.0 against 1.5); scaled by a typical change, distance moves the prediction by 9 minutes and rain by only 2.4. The signs never change.

COMMON MISTAKE

X Wrong: Comparing two raw coefficients, such as 6.0 for a 0/1 rain flag and 1.5 for a distance in kilometers, and concluding that rain “matters more” because 6.0 is the bigger number.

✓ Right: Compare coefficients only after standardizing the features (Chapter 4), or multiply each coefficient by the range that feature really moves across, as Figure 24.2 does.

Tree ensembles hand you a different number. Chapter 14 built impurity-based *feature importance*: how much each feature’s splits reduced impurity, added up across the trees and turned into a share — in that chapter’s mini-forest, distance scored 0.667, parcels 0.333 and weather 0.000. It answers a narrower question than a coefficient does.

WORKED EXAMPLE 24.2 — Notice What a Feature Importance Score Cannot Tell You

Karwan wants to know one more thing: does a *longer* distance make a late delivery more likely, or less likely? Can distance’s importance score of 0.667 answer this?

1. Recall what 0.667 measures: the share of all impurity removed that distance’s splits were responsible for. Check that the shares are a complete share-out: $0.667 + 0.333 + 0.000 = 1.000$.
2. Impurity removed is never negative, so a share can never be negative either. The smallest possible score is 0.
3. Because the number can never be negative, it cannot say whether long distances push predictions toward “late” or toward “on time”. It only ranks how much each feature helped the trees split.
4. To get the direction, Karwan needs a tool that asks the finished model what it predicts as distance changes. Lesson 24.3 supplies it.

Answer: An importance score gives size without direction. A coefficient’s sign carries the direction inside the very same number; an importance score does not.

24.3 Permutation Importance and Partial Dependence

Chapter 14 introduced a second way to score a feature, proposed by Leo Breiman alongside random forests in 2001: break it on purpose and watch what happens. Shuffle one column so its values no longer match their own rows, leave everything else untouched, and re-score the model.

EQUATION: PERMUTATION IMPORTANCE OF ONE FEATURE

$$PI(f) = s - \frac{1}{R} \sum_{r=1}^R s_{f,r}$$

In words: Take the model’s ordinary score and subtract the average score it gets after that one feature’s column is shuffled, repeating the shuffle several times so one unlucky shuffle cannot decide the answer.

When it is used: After training, on held-out data, to see how much a finished model of *any* kind depends on each feature.

Symbol	Meaning	Unit or type
$PI(f)$	permutation importance of feature f	same unit as the score
$s, s_{f,r}$	the score before shuffling, and on shuffle r of feature f	F1, accuracy, or any metric
R	number of shuffles averaged	count

For an error measure where lower is better, reverse the subtraction. Unlike impurity importance, these scores do not add up to 1.

Because it needs only a shuffled column and a re-scored model, permutation importance works the same way on a linear regression, a k -nearest-neighbors model (Chapter 11), a naive Bayes classifier (Chapter 12), a forest or an SVM, while impurity importance only makes sense for trees.

WORKED EXAMPLE 24.3 — Find the Permutation Importance of Three Delivery Features

Karwan checks Sunrise Parcels' forest on a held-out test set of 315 deliveries. Its baseline F1 is 0.87. Shuffling distance alone drops F1 to 0.62, stops alone to 0.78 and heavy rain alone to 0.85. Find each importance and rank the three.

1. Distance: $0.87 - 0.62 = 0.25$.
2. Stops: $0.87 - 0.78 = 0.09$.
3. Heavy rain: $0.87 - 0.85 = 0.02$.
4. Rank the drops from largest to smallest: 0.25, then 0.09, then 0.02.

Answer: Distance leads with 0.25: breaking it costs the model a quarter of its F1 score, while breaking rain costs almost nothing.

Permutation importance has a blind spot of its own. Karwan adds a fourth feature, estimated travel time, which mostly restates distance in a different unit — nothing about deliveries has changed, only the spreadsheet.

WORKED EXAMPLE 24.4 — See How a Near-Copy Feature Hides Distance's Importance

With estimated travel time added, the baseline F1 is 0.88. Shuffling distance alone drops it to 0.78, estimated travel time alone also to 0.78, and both together to 0.65. Find all three importances and compare with distance's earlier score of 0.25.

1. Distance, shuffled alone: $0.88 - 0.78 = 0.10$.
2. Estimated travel time, shuffled alone: $0.88 - 0.78 = 0.10$.
3. Add the two separate importances: $0.10 + 0.10 = 0.20$.
4. Both shuffled together: $0.88 - 0.65 = 0.23$, larger than either alone and larger than their sum.
5. Compare distance's new score, 0.10, with the 0.25 it earned before the near-copy was added.

Answer: Adding a near-copy cut distance's importance from 0.25 to 0.10 although deliveries did not change. The information is shared between two columns, so shuffling either alone leaves the model its twin; only breaking both reveals the true dependence of 0.23.

Figure 24.3 shows both experiments run for real on 900 simulated deliveries, with error bars from repeating each shuffle fifteen times.

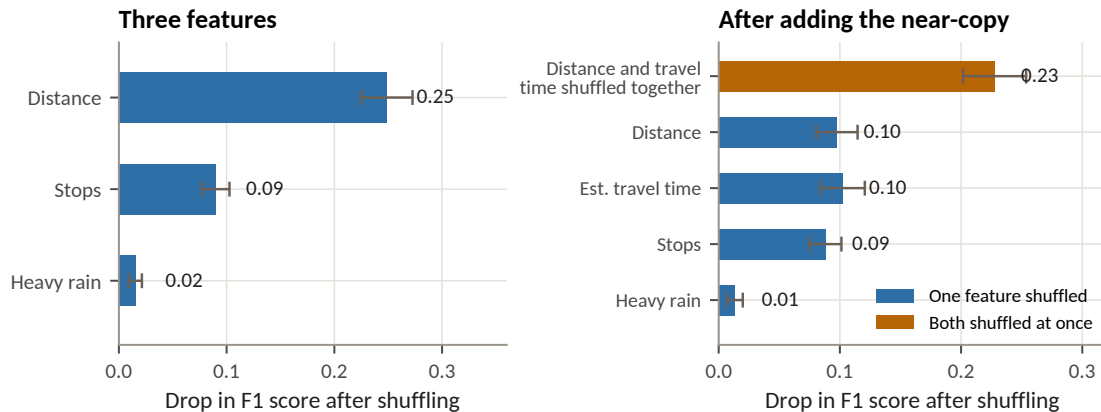


Figure 24.3 – Permutation importance, measured as the drop in F1 score after shuffling. Left: the three-feature model. Right: the same model after estimated travel time — a near-copy of distance — is added.

WHAT TO OBSERVE

The bars reproduce both worked examples, and the error bars are small enough that the ranking is not the luck of one shuffle. A low bar never proves a feature is useless in the world — only that *this* model can manage without it.

COMMON MISTAKE

X Wrong: Treating a large permutation importance score as proof that changing that feature in the real world would change the outcome.

✓ Right: Permutation importance measures how much a fitted model depends on a feature, not what causes what. Shortening routes might not reduce late deliveries; only a real trial on real routes can show that.

Neither importance measure answers Worked Example 24.2’s question about direction. For that we ask the model what it predicts as one feature moves while the others keep their real values — a partial-dependence plot, introduced by Jerome Friedman in 2001.

WORD TO KNOW

Partial dependence plot — A curve showing the model’s *average* prediction as one feature is set to each value in turn, with every other feature left as it is in the data. It shows the direction and shape of a feature’s effect, not only its size.

The recipe is short: pick a value, force it into every row while leaving each row’s other features alone, ask the model to predict, and average. Try it on four test deliveries by hand. Set all four to 30 km and the model answers 0.90, 0.80, 0.95 and 0.75, which add to 3.40 and average $3.40 \div 4 = 0.85$; set the same four to 10 km and it answers 0.10, 0.20, 0.05 and 0.25, adding to 0.60 for an average of $0.60 \div 4 = 0.15$. The rise, $0.85 - 0.15 = 0.70$, is the direction no importance score could give. One warning: the curve describes the model rather than the world, so where two features are strongly related, forcing one to an unusual value creates rows that could never happen. Figure 24.4 runs the same calculation on the real test set at twenty distances instead of two.

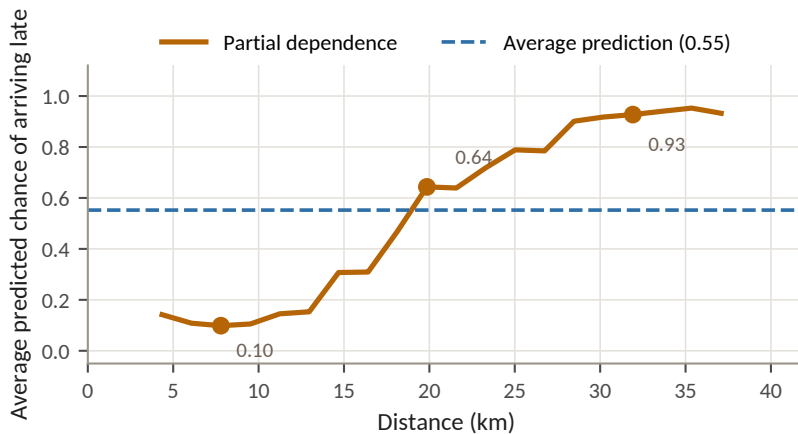


Figure 24.4 – Partial dependence of the forest’s predicted chance of being late on distance, averaged over all 315 test deliveries.

WHAT TO OBSERVE

The curve climbs from about 0.10 at 8 km to 0.64 at 20 km and 0.93 at 32 km, and it is not a straight line: nearly flat below 12 km, steep between 13 and 20 km, then flat again.

Both tools are one call each in scikit-learn. The program below reproduces Worked Example 24.3’s three importances and the three marked points of Figure 24.4, using the 900 deliveries in `datasets/ch24_deliveries.csv`.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance, partial_dependence
d = pd.read_csv("datasets/ch24_deliveries.csv") # 900 deliveries
cols = ["distance_km", "stops", "heavy_rain"]
tr, te = d[d.set == "train"], d[d.set == "test"]
rf = RandomForestClassifier(400, min_samples_leaf=5,
                           random_state=0).fit(tr[cols], tr.late)
pi = permutation_importance(rf, te[cols], te.late, scoring="f1",
                           n_repeats=15, random_state=0)
print("importance", pi.importances_mean.round(2), "+/-",
      pi.importances_std.round(2))
pdp = partial_dependence(rf, te[cols], features=[0], grid_resolution=20)
print("distance", pdp["grid_values"][0][[2, 9, 16]].round(1),
      "km -> late", pdp["average"][0][[2, 9, 16]].round(2))
```

Output

```
importance [0.25 0.09 0.02] +/- [0.02 0.01 0.01]
distance [ 7.8 19.9 31.9] km -> late [0.1 0.64 0.93]
```

Add `"eta_min"` to `cols` and run it again: distance falls to about 0.10 while estimated travel time picks up about 0.10, exactly as Worked Example 24.4 predicted. The class `PartialDependenceDisplay` draws the curve instead of printing it.

24.4 Fairness: The Same Word, Different Meanings

The last two lessons asked what a model depends on. This one asks a larger question: even once you understand a model fully, could it still treat one group of people worse than another?

Historical data is not a neutral photograph of the world. It records past decisions, made by people, inside systems that were not always equally fair, and a model trained faithfully on that record learns its patterns just as faithfully — including the unfair ones, without anybody intending harm. The word “bias” now returns with a second meaning, and the two must not be mixed up.

WORDS TO KNOW

Algorithmic bias — A pattern in a model’s predictions or errors that systematically disadvantages one group of people more than another — a mismatch between how a model treats one group and another, not, as in Chapter 7, between predictions and true values.

Fairness — The requirement that a model’s benefits, errors and decisions do not systematically disadvantage one group more than another in ways that matter here. What counts as fair depends on context and needs more than one check.

Checking fairness starts with a habit from Chapter 6: instead of trusting one combined score, split it by group. Peshraw University ranks scholarship applicants with a model whose shortlist a committee reviews. Six hundred people apply — 200 from rural secondary schools, 400 from urban ones — and the committee judged 40 rural and 160 urban applicants genuinely qualified. Figure 24.5 splits the confusion matrix by group.

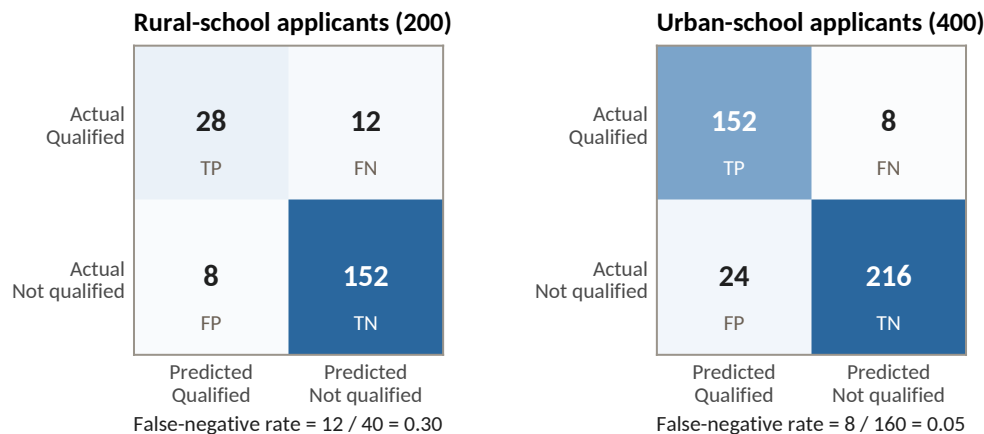


Figure 24.5 – The scholarship model’s confusion matrix, computed separately for each group. A “positive” prediction means a place on the shortlist.

WHAT TO OBSERVE

Both matrices look healthy at a glance. But read the top-right cell as a share of its own row: 12 of 40 qualified rural applicants missed, against 8 of 160 urban — six times the rate.

Two group measures do most of the work, and both are built from numbers Chapter 6 already defined.

EQUATION: GROUP FAIRNESS MEASURES

$$SR(g) = \frac{TP_g + FP_g}{n_g}, \quad FNR(g) = \frac{FN_g}{TP_g + FN_g} = 1 - \text{Recall}(g)$$

$$\Delta_{DP} = |SR(A) - SR(B)|, \quad \Delta_{EO} = |FNR(A) - FNR(B)|$$

In words: A group's selection rate is the share of everyone in it who received the positive decision; its false-negative rate is the share of the truly positive people it missed. The gaps are the differences between groups, ignoring the sign.

When it is used: Whenever a model's output changes what happens to a person. Compute both for every group you can identify.

Symbol	Meaning	Unit or type
$SR(g), n_g$	selection rate of group g , and its size	0 to 1; count
$FNR(g)$	false-negative rate, or miss rate, of group g	0 to 1
Δ_{DP}, Δ_{EO}	demographic parity difference; equal-opportunity gap	0 to 1

$\Delta_{DP} = 0$ is demographic parity; $\Delta_{EO} = 0$ is equal opportunity. The four-fifths rule of the United States Uniform Guidelines of 1978 treats a selection-rate ratio below 0.8 as worth investigating.

WORKED EXAMPLE 24.5 — Compare the False-Negative Rate Across Two Groups

Of the 40 qualified rural applicants the model missed 12; of the 160 qualified urban applicants it missed 8. Find each group's false-negative rate, the combined rate for all 200 qualified applicants, and the equal-opportunity gap.

1. Rural false-negative rate: $12 \div 40 = 0.30$, or 30%.
2. Urban false-negative rate: $8 \div 160 = 0.05$, or 5%.
3. Combined: $(12 + 8) \div (40 + 160) = 20 \div 200 = 0.10$, or 10%.
4. Equal-opportunity gap: $|0.30 - 0.05| = 0.25$; as a ratio, $0.30 \div 0.05 = 6$.

Answer: The combined 10% looks acceptable, but it hides a gap of 0.25: qualified rural applicants are missed six times as often as qualified urban ones.

Alarmed, the committee retunes the model so both groups reach a false-negative rate of 10%, satisfying equal opportunity exactly. Does that settle fairness?

WORKED EXAMPLE 24.6 — Check a Second Fairness Measure After Fixing the First

After the fix, 56 of the 200 rural applicants and 160 of the 400 urban applicants receive an offer. Find each selection rate and the demographic parity difference. Then find the rates a *perfect* model would produce, knowing 40 rural and 160 urban applicants are genuinely qualified.

1. Rural selection rate: $56 \div 200 = 0.28$.
2. Urban selection rate: $160 \div 400 = 0.40$.
3. Demographic parity difference: $|0.40 - 0.28| = 0.12$.
4. A perfect model shortlists exactly the qualified: $40 \div 200 = 0.20$ rural and $160 \div 400 = 0.40$ urban.
5. Its demographic parity difference would still be $0.40 - 0.20 = 0.20$.

Answer: Equalizing the false-negative rate left a selection-rate gap of 0.12, and even a flawless model would leave 0.20. Closing that gap completely means shortlisting rural applicants the committee itself judged unqualified.

That is not a trick. It is a small version of a mathematical result, proved independently by Jon Kleinberg, Sendhil Mullainathan and Manish Raghavan in 2016 and by Alexandra Chouldechova in 2017: when two groups genuinely differ in how often the outcome occurs, demographic parity, equal error rates and equal precision cannot all hold at once except in trivial cases. Table 24.1 lists the definitions that most often compete; Table 24.2 collects both versions of the scholarship model.

Table 24.1 – Four common group-fairness definitions and what each one makes equal

Definition	What it makes equal	The question it answers, and its cost
Demographic parity	Selection rate	Does each group get the same share of positive decisions? Ignores real differences in the outcome.
Equal opportunity	False-negative rate (recall)	Is the model equally good at finding those who really qualify? Silent about false alarms. Named by Moritz Hardt, Eric Price and Nathan Srebro in 2016.
Equalized odds	False-negative <i>and</i> false-positive rate	Are both mistakes equally common in each group? The strictest rule; usually costs accuracy.
Predictive parity	Precision	When the model says yes, is it right equally often for each group? Conflicts with equalized odds when base rates differ.

Table 24.2 – The scholarship model before and after the false-negative rates were equalized

Group	Applicants	Truly qualified	Offers, v1	FNR, v1	Sel. rate, v1	Offers, v2	FNR, v2
Rural	200	40	36	0.30	0.18	56	0.10
Urban	400	160	176	0.05	0.44	160	0.10

IMPORTANT

No formula settles whether a model is fair. Fair-sounding rules genuinely conflict when groups differ, so a person must choose which measure matters most for this decision, in this place, under this law — and write that choice down where others can question it. What this chapter insists on is only that you measure the gaps and publish them.

24.5 Protected Attributes and Proxy Variables

Comparing outcomes by group only works if you know which group each person is in — harder than it looks.

WORDS TO KNOW

Protected attribute — A personal characteristic — gender, age, ethnicity, religion, disability, region of origin — that law or policy says must not be the reason somebody receives a worse outcome. Which attributes are protected varies by country.

Proxy variable — A feature that is not itself a protected attribute but carries almost the same information, so a model can rebuild the protected pattern even after that column is deleted.

A common instinct is to keep a protected attribute out of the features altogether: a model cannot discriminate by region if it never sees a region column. That is not wrong, but it is not enough.

Peshaw University’s team deletes the rural/urban flag, satisfied that this closes the gap. But in a sample of 200 applicants, school district matches rural or urban status exactly for all but 10, who attend an urban boarding school while living in a rural area. Count the ones district gets right, $200 - 10 = 190$, and divide by the total: $190 \div 200 = 0.95$, or 95%. Deleting the flag did not remove that information — it was still sitting inside the district column, which the model remained free to use, as Figure 24.6 shows.

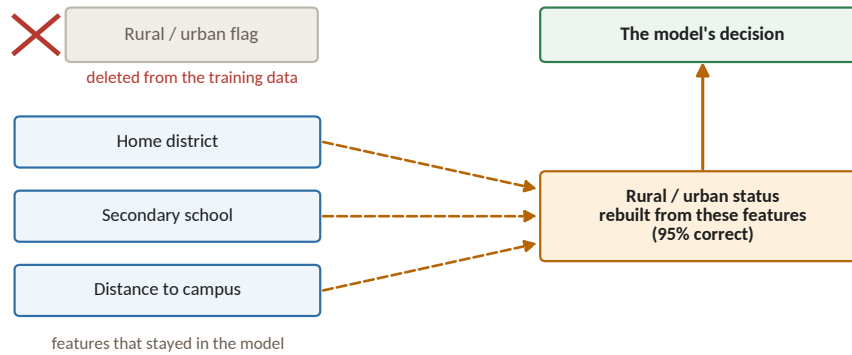


Figure 24.6 – Deleting a protected column does not delete its information. Three ordinary features rebuild the removed attribute, and the model uses the rebuilt version without anyone asking it to.

WHAT TO OBSERVE

Nobody added the district column in order to rebuild the flag; it was there to route paperwork to the right office. A proxy almost never announces itself, which is why it needs a deliberate check — see Table 24.3.

Table 24.3 – Everyday features that often act as proxies for a protected attribute

Feature	Can stand in for	Why
Home postcode or district	Ethnicity, income, region of origin	Neighborhoods are rarely mixed evenly, so location carries community information.
Secondary school name	Income, region, sometimes religion	Schools serve particular areas and fee levels.
First name	Gender, ethnicity, religion	Naming conventions differ strongly between communities.
Hours available to work	Care responsibilities, disability	Availability often reflects circumstances outside a person’s control.

COMMON MISTAKE

X Wrong: Believing that deleting a protected attribute’s column makes a model fair, and skipping the group-level checks because “the model cannot see it”.

✓ Right: Check the remaining features for proxies, and measure the group-level outcomes of Lesson 24.4 directly. Only the outcomes can tell you whether a gap exists.

RESPONSIBLE USE

Sometimes the safest choice is the opposite of deleting a protected attribute. Keeping it in a separate, access-controlled file — used only to measure group outcomes, never as a feature the model sees — lets a team check for unfairness directly instead of guessing through proxies.

24.6 Privacy: Personal Data, Anonymization and Consent

Every dataset in this chapter describes people, which changes what a team may do with it — and what a careful team *chooses* to do, well before any law requires it.

WORDS TO KNOW

Personal data — Any information relating to an identifiable person — a name, an ID number, an address, an exam record, a medical result, or a combination of ordinary fields that together point to one individual.

Consent — A person's freely given, informed and specific agreement that their data may be used for a stated purpose. It is not valid if the purpose was hidden, vague or changed afterward.

Data minimization — Collecting and keeping only the fields needed for that purpose, and deleting them once it has been served.

Minimization is the cheapest protection there is: data you never collected cannot leak, be misused, or become a proxy. The European Union's General Data Protection Regulation, in force since 2018, made minimization and purpose limitation legal requirements across much of the world's economy, and many other countries have since passed similar rules. Figure 24.7 shows the life of a personal dataset.

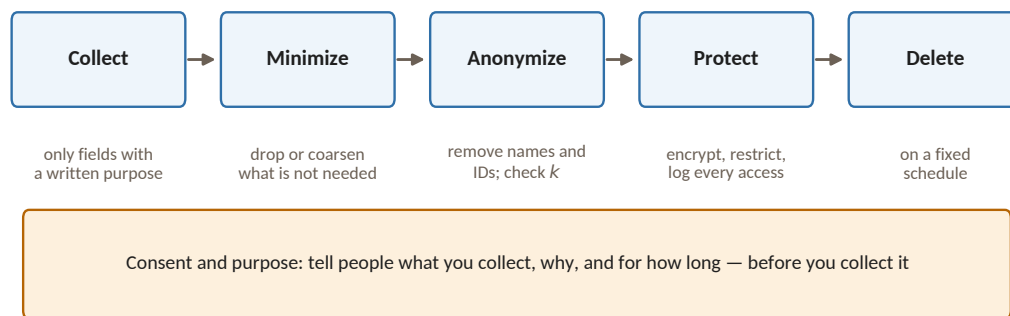


Figure 24.7 – The life of a personal dataset. Every step is a decision somebody has to make on purpose; none of them happens by default.

WHAT TO OBSERVE

The pipeline begins and ends with an action, not with a model: “Collect” has a gate in front of it — is this field needed at all? — and “Delete” has a date attached.

WORDS TO KNOW

Anonymization — Changing a dataset so no row can be traced back to a person, even by somebody who combines it with other information. Removing names is *not* anonymization.

Pseudonymization — Replacing an identifier with a code and keeping the key elsewhere. It reduces risk but is reversible, so the data stays personal data.

Quasi-identifier — A field that is not an identifier alone but can single out a person combined with others — age band, district, sex, job title, date of a visit.

Removing names is not enough because a released table can be re-identified by joining its quasi-identifiers to another list — a voter roll, a staff directory, a social-media profile. The standard defense, published by Latanya Sweeney in 2002 after joint work with Pierangela Samarati in 1998, requires every row to hide in a crowd.

EQUATION: THE k -ANONYMITY CONDITION

$$k = \min_{q \in Q} n_q \geq k_{\min}$$

In words: Group the rows by their combination of quasi-identifiers and count each group. The table's k is the size of the smallest group, and the table is k -anonymous only if that group is at least as big as the level you require.

When it is used: Before releasing or sharing any table about people, even inside your own organization.

Symbol	Meaning	Unit or type
Q, n_q	the distinct quasi-identifier combinations, and how many rows share each	set; count
k	the smallest group size in the table	count
$k_{\min}$	the level required, often 5 or 10 in practice	count

Raising k means widening bands or dropping a column: safer data and coarser data. There is no setting that gives both.

WORKED EXAMPLE 24.7 — Check Whether a Small Table Is 2-Anonymous

A clinic in Duhok wants to share Table 24.4 with a research team. Names are removed. Age band, district and sex are quasi-identifiers; diagnosis is the sensitive value. Decide whether the table is 2-anonymous, and fix it if it is not.

1. Group the six rows by (age band, district, sex) and count: (31–40, Erbil, F) has 2 rows; (41–50, Duhok, F) has 2; (41–50, Duhok, M) has 1; (51–60, Duhok, M) has 1.
2. Take the smallest group size: $k = 1$.
3. Compare with the required level: 1 is less than 2, so the table is **not** 2-anonymous. Rows 5 and 6 stand alone, so anyone who knows a 55-year-old man from Duhok visited the clinic learns his diagnosis.
4. Widen the bands 41–50 and 51–60 into one band 41–60 and count again: 2, 2 and 2.
5. The smallest group now holds 2 rows, so $k = 2$.

Answer: As published the table is only 1-anonymous and two people are exposed. After widening one age band it is 2-anonymous — nothing deleted, one column simply less precise.

Table 24.4 – Six clinic records. The last column shows the widened age band that makes the table 2-anonymous

Row	Age band	District	Sex	Diagnosis	Widened age band
1	31–40	Erbil	F	Diabetes	31–40
2	31–40	Erbil	F	Asthma	31–40
3	41–50	Duhok	F	Asthma	41–60
4	41–50	Duhok	F	Diabetes	41–60
5	41–50	Duhok	M	Anemia	41–60
6	51–60	Duhok	M	Diabetes	41–60

A table can pass the k -anonymity check and still leak: if everyone in one group shares the same diagnosis, an attacker learns it without knowing which row is which. The strongest widely used guarantee instead adds calibrated random noise to every published answer.

WORD TO KNOW

Differential privacy — A mathematical guarantee that the answers released from a dataset are almost the same whether or not any one person's record is included, achieved by adding a measured amount of random noise. It was defined by Cynthia Dwork, Frank McSherry, Kobbi Nissim and Adam Smith in 2006.

Differential privacy is why national statistics offices can publish counts and averages without exposing individuals. The price is accuracy: more privacy means more noise, and a small dataset cannot afford much of either. Table 24.5 compares the techniques.

Table 24.5 – Privacy techniques, what each one protects against, and what it costs

Technique	What it protects against	What it costs
Data minimization	Every risk at once: leaks, misuse, proxies	Some questions cannot be asked later
Generalization to reach k -anonymity	Re-identification by joining quasi-identifiers to another list	Coarser bands, weaker models
Differential privacy	Learning anything about one person from a released answer	Noise in every answer; needs a large dataset
Access control, logging, scheduled deletion	Misuse by insiders, and old data leaking years later	Administration; no long-term trends

COMMON MISTAKE

X Wrong: Calling a dataset “anonymous” because the name and ID columns were deleted.

✓ Right: Ask what a determined outsider could join the remaining columns to. Age band, district and one date are often enough to single somebody out. Compute k before you share.

24.7 Drift, Monitoring and Safe Rollback After Launch

Every check so far happens before a model launches. The world does not stop once it does.

WORDS TO KNOW

Data drift — A change over time in the data a live model sees, compared with the data it trained on. It can appear in the features themselves or in how they relate to the true answer.

Monitoring — Regularly checking a live model’s incoming data, its predictions and its real outcomes, so problems are caught quickly instead of found by accident much later.

Training-serving skew — A mismatch between how a feature is computed for training and for live predictions, so the model receives different information than it learned from although nothing in the world changed.

Rollback — Switching a live system back to its last known-good version, quickly, when monitoring shows the current one is behaving badly.

Roj Solar Supply, a solar-equipment retailer with stores in Erbil, Duhok and Sulaymaniyah, forecasts next week’s battery sales every Sunday night. One feature counts the inquiries its website receives each week; for months that number stays in a narrow band, and then the government announces a subsidy for home solar batteries. The team treats any feature whose average moves by more than 20% as worth a closer look.

EQUATION: TWO SIMPLE MONITORING CHECKS

$$\text{shift} = \frac{\bar{x}_{\text{recent}} - \bar{x}_{\text{reference}}}{\bar{x}_{\text{reference}}} \times 100\%, \quad \rho = \frac{\text{error}_{\text{live}}}{\text{error}_{\text{validation}}}$$

In words: The population shift is how far a feature’s recent average has moved from its average when the model was built, as a percentage of the old average. The error ratio is the recent error divided by the validation error.

When it is used: The shift on every important feature, on a fixed schedule, as an early warning; the error ratio as soon as true outcomes are known.

Symbol	Meaning	Unit or type
$\bar{x}_{\text{reference}}, \bar{x}_{\text{recent}}$	the feature's average when the model was built, and now	unit of the feature
ρ	error ratio, live against validation	a plain number

$\rho = 1$ means the model performs exactly as well as in validation; $\rho = 2$ means twice the error. Agree on the alert level for both before launch, not during an incident.

WORKED EXAMPLE 24.8 — Detect Drift in Weekly Website Inquiries

In the 8 weeks before the announcement, weekly inquiries were 36, 40, 42, 38, 41, 39, 43 and 41; in the 8 weeks after, 66, 70, 72, 68, 71, 69, 73 and 71. Find both averages and the population shift, and compare with the 20% rule.

1. Add the “before” counts: $36 + 40 + 42 + 38 + 41 + 39 + 43 + 41 = 320$, and $320 \div 8 = 40$.
2. Add the “after” counts: $66 + 70 + 72 + 68 + 71 + 69 + 73 + 71 = 560$, and $560 \div 8 = 70$.
3. Find the difference: $70 - 40 = 30$.
4. Divide by the reference average and convert: $30 \div 40 = 0.75$, and $0.75 \times 100 = 75\%$.

Answer: Inquiries rose by 75%, far past the 20% threshold. This is real drift, not week-to-week noise, and the next few forecasts deserve a closer look.

Drift rarely announces itself with one dramatic failure. It shows up as a slow slide — a few more stockouts, a few more unsold pallets, each small enough to blame on a bad week. Figure 24.8 shows the feature moving first and the error following.

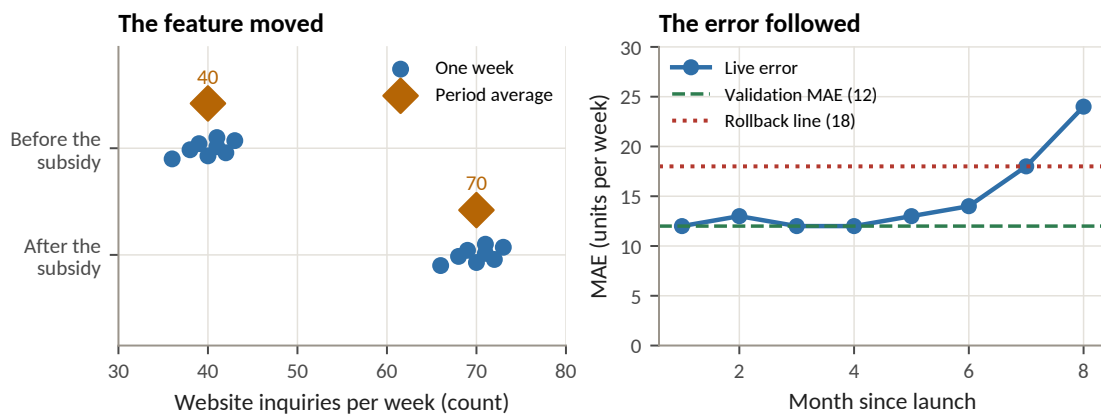


Figure 24.8 – Left: weekly website inquiries before and after the subsidy announcement. Right: the forecasting model’s error over the same period, against its validation error and the team’s rollback line.

WHAT TO OBSERVE

The feature’s average jumps from 40 to 70 at once, while the error stays near its validation value of 12 for five more months before climbing to 18 and then 24. Watching only the error would have cost months of quietly worsening forecasts.

A monitoring plan therefore watches three layers (Table 24.6, Figure 24.9): incoming data, which warns within minutes; the model’s own output; and real outcomes, the slowest layer but the only one that checks the thing that matters.

Table 24.6 – Three monitoring layers, and what each one can catch

Layer	What it watches, and an example alert	How fast it warns
1. Incoming data	Ranges, formats, missing values, volume, feature averages against reference: “inquiries up 75%”	Minutes
2. Predictions	The pattern of the model's own output, before any true answer is known: “high demand predicted for 90% of products”	Hours
3. Real outcomes	Live error against validation error, calibration, group gaps: “error ratio 1.5 two months running”	Days to weeks

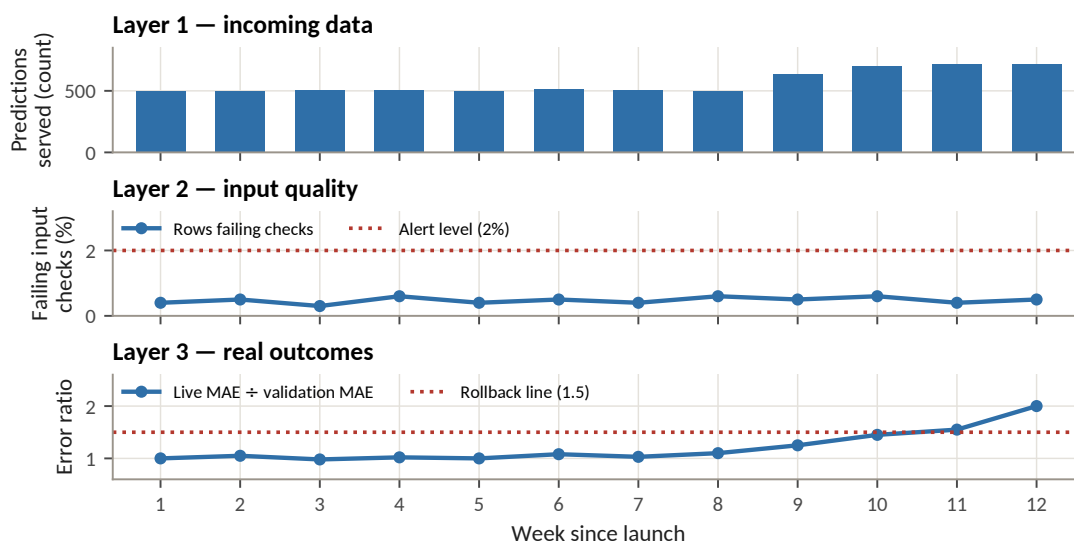


Figure 24.9 – A monitoring dashboard for the demand-forecasting model, one row per layer, over twelve weeks.

WHAT TO OBSERVE

The three rows together diagnose the incident. Volume jumps in week 9 but the input checks stay flat, so the extra rows are real and not a broken feed; then the error ratio crosses 1.5. A skew bug would instead have spiked the middle row.

Skew is the mirror image of Chapter 3’s data leakage: leakage sneaks information into training that would not be available at prediction time, while skew feeds the live model poorer information than it learned from. If Roj Solar’s “average daily inquiries” feature was computed over 7 full days during training but reports only the current, incomplete day when live, the model receives about 15 where it expects about 41 — with no change in customer behavior at all. Reusing one saved pipeline (Chapter 4) for training and serving is the strongest defense, because only one copy of the logic can go wrong. Fairness drifts too, so the group checks of Lesson 24.4 belong on the monitoring schedule beside accuracy.

WORKED EXAMPLE 24.9 — Decide Whether an Incident Crosses the Rollback Line

Roj Solar’s team agreed in advance that if the error ratio reaches 1.5 for two consecutive months they roll back rather than keep investigating live. Validation MAE is 12 units per week; in month 7 the live MAE is 18 units and in month 8 it is 24. Decide whether the incident crosses the line, and when.

1. Find the threshold in the model’s own units: $1.5 \times 12 = 18$ units per week.

2. Month 7: $\rho = 18 \div 12 = 1.5$, which meets the threshold exactly and so counts as crossing it.
3. Month 8: $\rho = 24 \div 12 = 2.0$, well above 1.5.
4. Both consecutive months meet or exceed the threshold, so the two-month rule is satisfied.

Answer: The line is crossed as early as month 7, and month 8 confirms it was not a fluke. The pre-agreed rule calls for an immediate rollback, not more investigation while the purchasing team acts on bad forecasts.

COMMON MISTAKE

✗ Wrong: Retraining on the newest data the moment a monitoring alert fires, as an automatic, reflexive fix.

✓ Right: Diagnose first. Retraining on top of an undiagnosed skew bug bakes the bug in deeper. Confirm whether the cause is real drift, a skew bug or a short-lived blip — then decide.

24.8 A Responsible-ML Checklist and the Human in the Loop

Everything in this chapter fits into one loop that never really ends (Figure 24.10).

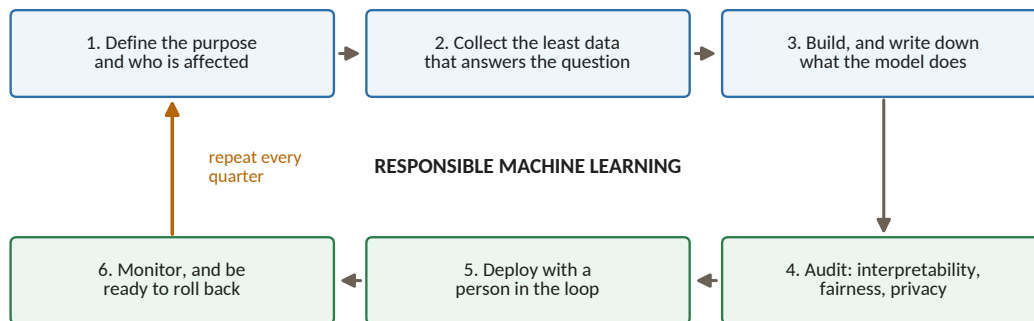


Figure 24.10 – The responsible machine learning lifecycle. The arrow on the left is the part teams forget: the loop closes, and the audit runs again.

WHAT TO OBSERVE

Stages 1 and 2 happen before any modeling and stages 5 and 6 continue for as long as the model is used, so the modeling itself is one box out of six — roughly the right proportion for a real project.

WORDS TO KNOW

Human in the loop — A design in which a model's output is a recommendation for a trained person to review, not an automatic decision. The person can overrule the model, and those overrules are recorded and studied.

Model card — A short, plain-language document published with a model, stating what it is for, what data it learned from, how well it performs overall and by group, and what it must not be used for. Proposed by Margaret Mitchell and colleagues in 2019.

A human in the loop is not a rubber stamp. It works only if the reviewer has time, the authority to disagree, and enough context to disagree well — usually the local explanation, not only the score — and if the person

affected can ask for a review and get an honest answer about what the model did. Table 24.7 is the short version of this chapter.

Table 24.7 – A responsible-ML checklist: the question to answer before each stage is finished

Stage	The question that must have a written answer
Purpose	Who is affected, and what is the worst a wrong prediction could do to them?
Data	Which fields do we need, who consented, when do we delete, and what is the table's k : before we share it?
Features	Which remaining columns could act as a proxy for a protected attribute?
Interpretability	Can we state which features the model leans on, and in which direction?
Fairness	What are the selection-rate and false-negative-rate gaps, and which definition did we choose?
Deployment and monitoring	Who reviews the output and can overrule it, how does an affected person ask for a review, what are the drift and error thresholds, and when does the audit repeat?

RESPONSIBLE USE

A prediction is not a fact about a person and not proof of what will happen. Use a sensitive prediction to offer help and to trigger a caring human review — never to remove somebody's opportunity automatically. A model that cannot be explained, audited and monitored should not be deployed for decisions about people, however good its test score.

COMMON MISTAKES AND MISCONCEPTIONS

- Trusting a high test score without asking which features the model relies on, or whether it treats every group equally well.
- Comparing raw coefficients across features on very different scales, and calling the bigger number the more important feature.
- Reading an importance score as if it revealed the *direction* of a feature's effect, or proved that the feature causes the outcome.
- Reporting one combined fairness number instead of group-by-group outcomes, expecting one formula to settle what counts as fair, or believing that deleting a protected column makes a model fair.
- Calling a table “anonymous” because the names were removed, without computing k ; or assuming a model that launched well stays well, then retraining reflexively when an alert fires, before diagnosing drift, skew or noise.

Practical Activities

ACTIVITY (PAPER) — Audit Twenty Decisions by Group

1. Draw a table with 20 rows and four columns: *Person*, *Group* (A or B), *Truly qualified?* and *Model said yes?*. Fill it in as you like, but give Group A 8 people and Group B 12.
2. For each group count the four confusion-matrix cells of Chapter 6 — TP, FN, FP, TN — and check that they add up to the group's size.
3. Compute each group's selection rate, $(TP + FP) \div n_g$, and false-negative rate, $FN \div (TP + FN)$, then the demographic parity difference and the equal-opportunity gap.

4. Change two decisions — only two — to make the equal-opportunity gap smaller, recompute both gaps, and write one sentence saying whether the other gap improved or got worse.

You need: Paper, pen and a calculator

ACTIVITY (PYTHON) — Audit the Scholarship Model for Group Differences

This repeats Worked Examples 24.5 and 24.6 on `datasets/ch24_scholarship.csv`: 600 applicants with the columns `school_type`, `exam_score`, `essay_score`, `qualified`, `shortlisted` (version 1) and `shortlisted_v2`.

Python · pandas

```
import pandas as pd
d = pd.read_csv("datasets/ch24_scholarship.csv")      # 600 applicants, 2 groups

for decision in ["shortlisted", "shortlisted_v2"]:
    sr, fnr = [], []
    for group, g in d.groupby("school_type"):
        recall = g[g.qualified == 1][decision].mean()    # of the truly qualified
        sr.append(g[decision].mean())                    # of everyone in the group
        fnr.append(1 - recall)
    print(f"{decision:15s} {group:6s} n={len(g):3d} selected {sr[-1]:.2f}"
          f" recall {recall:.2f} FNR {fnr[-1]:.2f}")
    print(f" parity difference {max(sr) - min(sr):.2f}"
          f" equal-opportunity gap {max(fnr) - min(fnr):.2f}")
```

Output

```
shortlisted    rural    n=200 selected 0.18 recall 0.70 FNR 0.30
shortlisted    urban    n=400 selected 0.44 recall 0.95 FNR 0.05
parity difference 0.26 equal-opportunity gap 0.25
shortlisted_v2 rural    n=200 selected 0.28 recall 0.90 FNR 0.10
shortlisted_v2 urban    n=400 selected 0.40 recall 0.90 FNR 0.10
parity difference 0.12 equal-opportunity gap 0.00
```

Version 2 closes the equal-opportunity gap completely and cuts the demographic parity difference from 0.26 to 0.12 — but not to zero, and Worked Example 24.6 showed why.

You need: Python with pandas (see Appendix B), and `datasets/ch24_scholarship.csv`

CASE STUDY — Auditing a Hiring-Screening Model — Chennai, India

Chalo Rides, a ride-hailing company with a large customer-support center in Chennai, receives thousands of applications a month for a small number of openings. It builds a screening model, trained on past applications together with each hired employee's first-year performance rating, to produce a shortlist for a recruiter. Before the model goes near a real applicant, the team runs this chapter's checklist. Importance scores show it leans most on typing-test score and months of prior customer-service experience, plus a smaller, curious weight on distance from the office. Comparing group-level outcomes among past applicants who had scored equally well on the company's own structured interview, the team finds the model shortlists men at a noticeably higher rate than women. Distance is acting as a proxy, exactly like Lesson 24.5's school district: past women applicants more often lived farther away, in neighborhoods the company had not served well with evening transport, and the old labels reflected that barrier rather than any difference in job performance.

Chalo Rides neither throws the model away nor ignores the finding. It removes distance from the office, retrains, and confirms that the shortlist gap closes. It keeps gender in a separate, access-controlled file used only for the quarterly audit, never as a feature, and publishes a one-page model card. The model still only produces a shortlist: a recruiter reviews every shortlisted application personally, and anyone not shortlisted may ask a person to review their case by hand.

CASE STUDY — Monitoring a Demand Forecast After Launch — Roj Solar Supply, Kurdistan Region

Roj Solar Supply deployed its weekly demand forecast as a batch job that runs every Sunday night, following the workflow of Chapter 2: a clear baseline, a validated pipeline, and an honest test score before anyone trusted it. For four months it performed close to its validation MAE of 12 units per week. Then the government announced the battery subsidy, and within two weeks the dashboard of Figure 24.9 showed all three layers reacting: prediction volume jumped, the input checks stayed clean, and over the following months the error ratio climbed past 1.5. A flat input-check layer ruled out a broken feed early. Following Lesson 24.7's order of operations, the team ruled out training-serving skew next, confirming that the live system still called the same saved pipeline used in training, then compared feature distributions directly and confirmed the shift was real.

With the cause identified as drift rather than a bug, they did not retrain in a panic. They rolled back to a simpler baseline forecast for two months, leaning more on a regional manager's judgment, while they collected enough post-subsidy data to retrain properly — then repeated the full workflow and redeployed gradually. Looking back, the team treats the incident as proof the system worked: the dashboards caught a real change within two weeks, and the pre-agreed rollback line meant nobody argued about thresholds during it. Their manager tells new staff that the model is not a finished product on a shelf; it is more like a solar panel, producing value only if somebody checks it, cleans it and adjusts it as the seasons change.

CHAPTER SUMMARY

- A strong test score is necessary but not sufficient: interpretability, fairness, privacy and monitoring are separate checks, and models range from easy to read (linear models, shallow trees) to hard (forests, boosting, kernels).
- A coefficient gives the direction and size of a feature's effect, but only once the features are on comparable scales.
- An importance score gives size without direction, because it can never be negative; a partial-dependence plot supplies the direction and the shape.
- Permutation importance is the drop in score after one column is shuffled: it works for any model, measures dependence and never cause, and splits the credit between two features that carry the same information.
- Fairness is checked by splitting a metric by group: selection-rate gaps measure demographic parity, false-negative-rate gaps equal opportunity. When base rates differ both cannot be zero, so a person must choose and say which they chose.
- Proxies such as district, school or first name rebuild a deleted protected attribute, and removing names is not anonymization: privacy means collecting less, generalizing until every row hides in a group of at least k , controlling access and deleting on schedule.
- After launch, watch the incoming data, the predictions and the real outcomes; a large population shift or an error ratio past a pre-agreed line triggers a diagnosis and, if needed, a rollback.
- Every decision affecting a person needs a human in the loop with the time, context and authority to disagree with the model.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Define interpretability in your own words, and name one model from this book that is easy to interpret and one that is not.

Problem 2 [Basic] Explain in one or two sentences the difference between Chapter 7's statistical meaning of "bias" and this chapter's algorithmic bias.

Problem 3 [Basic] Define protected attribute, give two examples of your own that this chapter did not use, and explain in one sentence why deleting such a column does not guarantee a fair model.

Problem 4 [Basic] A dataset has had every name and ID number removed. Explain why it may still not be anonymous, using the term *quasi-identifier*.

Problem 5 [Standard] A linear model predicts a delay in minutes as $6 + 2x_{\text{distance}} + 3x_{\text{stops}}$. (a) Find the predicted delay 10 km away with 2 stops. (b) Find it again with 5 stops, and check that the rise matches the stops coefficient.

Problem 6 [Standard] A model's baseline validation accuracy is 0.88. Shuffling feature X drops it to 0.81 and feature Y to 0.86. Find each permutation importance, state which feature the model depends on more, and explain why adding a near-copy of X would lower X's score.

Problem 7 [Standard] Group A has 50 truly qualified applicants and the model misses 5. Group B has 30 truly qualified applicants and the model misses 9. Find each group's false-negative rate, the combined rate, and the equal-opportunity gap.

Problem 8 [Standard] Forced to a price of 200 IQD a model predicts 0.40, 0.50, 0.30, 0.60 and 0.20 for five rows; forced to 400 IQD it predicts 0.10, 0.20, 0.05, 0.15 and 0.00. Find the partial dependence at each price and say which way price pushes the prediction.

Problem 9 [Standard] A feature reads 480, 510, 495, 505, 515 and 495 over six weeks before a campaign, then 640, 660, 645, 655, 660 and 640 during it. Find both averages and the population shift. If a validation MAE of 8 becomes a live MAE of 14, find the error ratio too.

Problem 10 [Standard] A table's quasi-identifier groups have sizes 6, 4, 3, 1 and 7. (a) What is the table's k ? (b) Is it 3-anonymous? (c) Name one change that would raise k to 3, and one cost of making it.

Problem 11 [Challenge] A hospital's readmission-risk model has an overall false-negative rate of 12%. (a) Give one reason this single number could hide a serious problem. (b) Describe two group-level checks you would run before trusting it. (c) Name one non-model factor that should also be part of the decision to use the tool.

Problem 12 [Challenge] Two groups apply for a training program: 300 people from Group A, 60 of them qualified, and 200 from Group B, 80 qualified. (a) Find each base rate. (b) Find the selection rates a perfect model would produce and the demographic parity difference it would leave. (c) Explain what a team would have to do to reach demographic parity, and what that would cost the people involved.

Problem 13 [Challenge] A ride-hailing app's arrival-time model shows rising error every day for a week after a city map update. (a) List three checks you would run, in order, before deciding whether to roll back. (b) State what result from each check would point to drift and what would point to training-serving skew. (c) State one condition under which you would roll back immediately.

Problem 14 [Challenge] Choose a model that could be used in your own school, workplace or town, and write a model card answering the six questions of Table 24.7, one line each, in language a person affected by the model could read.

Capstone Projects

Two complete projects from the decision to the final test — and ideas for your own

IN THIS CHAPTER YOU WILL LEARN TO

- Carry one project all the way from a plain decision sentence to a monitoring plan, using every stage of the workflow of Chapter 2.
- Audit a raw dataset for duplicates, gaps, impossible values, data leakage and circular features before any model is trained.
- Split data by time and by customer, and compare a baseline against three real models on the same validation data.
- Read a final test score honestly, then break it into slices to find the group the model serves worst.
- Turn a probability into a decision by choosing a threshold from a team's real capacity and a campaign's real net value.
- Plan and present your own project on one page, including what the model must never be used for.

Introduction

Twenty-four chapters ago this book began with a market inspector guessing which watermelon would be sweet. Since then you have cleaned data, split it honestly, chosen a performance measure, fitted lines, grown trees, boosted them, found clusters, tuned dials and asked hard questions about fairness — one idea at a time, so that each idea was clear.

Real projects do not arrive one idea at a time. A manager asks a question on a Sunday morning, the data is messier than anyone promised, the obvious feature leaks the answer, the fancy model barely beats the simple one, and the number everybody wants — one honest score — can be produced only once, at the very end.

Project 1 is a regression project: a small real-estate service in Erbil wants a starting price for a home a seller is about to list. Project 2 is a classification project with a sharper edge: a phone company wants to know which customers are about to leave, so a small team can call the ones most worth calling. Both run from the first sentence to the monitoring plan. Then come six project ideas you can carry out yourself, and a one-page template for writing up whatever you build. Nothing in this chapter is new. That is the point.

25.1 Project 1: Framing, Auditing and Splitting

Meet Lana, a data analyst at Hawler Homes, a small real-estate service in Erbil. Today a seller's starting price comes from one agent's memory and a glance at a few nearby listings. An experienced agent in a district she knows does this skillfully; a new agent, or one working where towers go up every year, does not. The director asks whether years of the company's own sales records could give every agent a steadier starting point.

WORD TO KNOW

Capstone project — A project you carry through every stage of the workflow yourself, from the first plain sentence about the decision, through cleaning, splitting, baselines and model comparison, to one honest test score, an explanation, and a plan for watching the model after launch.

Figure 25.1 is the map of this lesson and the next: nine stages, each one a chapter of this book, each carrying the number Lana actually got.

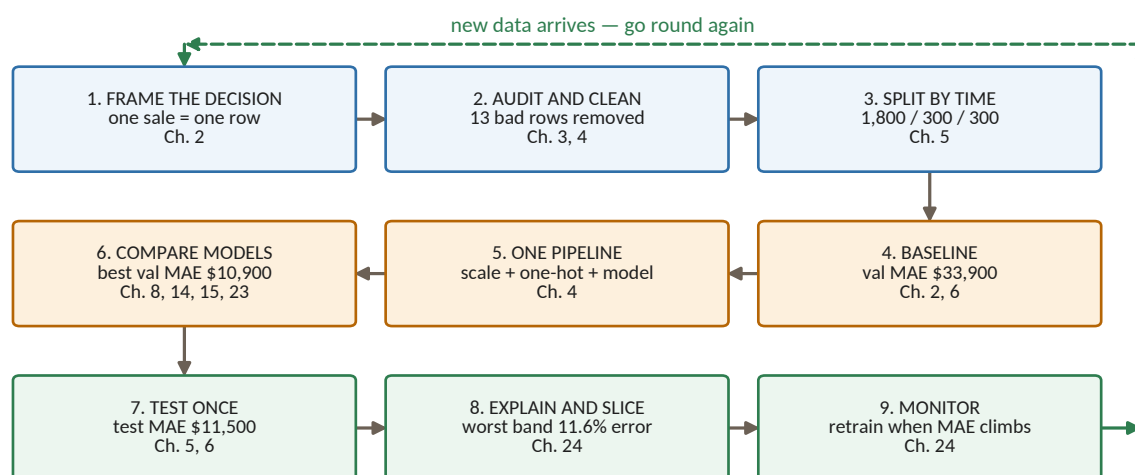


Figure 25.1 – The nine stages of Project 1, with the chapter that teaches each stage and the number Lana obtained. Read left to right along the top row, right to left along the middle row, then left to right again.

WHAT TO OBSERVE

Most of the diagram happens *before* a model is trained: framing, auditing and splitting fill the whole top row. And the last box loops back to the first — a launched model is not a finished project.

Chapter 2 gave the safe way to begin: one plain sentence of the form “At prediction time, the model will use ___ to predict ___, so that ___.”

WORKED EXAMPLE 25.1 — Write the Decision Sentence for Hawler Homes

Prediction happens the moment a seller first sits down with an agent, before the home is listed anywhere. At that moment the agent knows the home’s size, bedrooms, bathrooms, age, neighborhood and distance from the city center. Nobody yet knows what the home will sell for. Fill in the three blanks.

1. Decide what is genuinely known at that moment: size, bedrooms, bathrooms, age, neighborhood and distance from the center, because an agent can note all of these during one visit.
2. Decide the target: the price this home is likely to sell for.
3. Decide the purpose: so the agent can offer the seller an honest starting range for discussion, instead of a guess from memory.
4. Join the three parts into one sentence.

Answer: “At prediction time, the model will use the home’s size, bedrooms, bathrooms, age, neighborhood and distance from the city center to predict the price it is likely to sell for, so that the agent can offer the seller an honest starting range for discussion.” Written before any data file is opened, that sentence already fixes the unit of observation, the target and the prediction time.

Table 25.1 answers the same four questions for both projects. Project 2 arrives properly in Lesson 25.3; the table is here so you can see from the start that two very different projects begin in exactly the same way.

Table 25.1 – The same four questions, answered for both capstone projects

Part	Project 1 — Hawler Homes	Project 2 — Zagros Mobile
Unit of observation	One completed home sale	One customer, at one monthly snapshot
Target	The price the home sells for, in US dollars	Whether that customer churns within the next 30 days
Prediction time	The first meeting with the seller, before any listing	The first day of the month, using data through the end of last month
Action it supports	The agent offers an honest starting range for discussion	The retention team calls the customers most at risk

IMPORTANT

Hawler Homes’ tool produces a starting range for a conversation. It is not a legal appraisal, not a bank valuation and not a promise of what any home will sell for. A bank, a court or a tax office may require a formal appraisal following rules this project never claims to follow. Every range the model produces should say so, right next to the number.

COMMON MISTAKE

X Wrong: A teammate suggests “let us use gradient boosting, it is usually the strongest model”, before anybody has written down which decision the prediction supports.

✓ Right: Write the plain decision sentence first, as Chapter 2 recommends. Choose a model only once the target, the unit of observation and the prediction time are clear.

Lana pulls 2,413 past sales covering the last 24 months, one row per completed sale. Before fitting anything she does what Chapter 3 and Chapter 4 demand: look at the data honestly, with no model running. Twelve rows are exact duplicates, most likely the same sale typed in twice; one lists a size of 0 m² beside a price of \$180,000; forty have no bathroom count at all. She drops the twelve duplicates and the impossible row, leaving 2,400 clean sales, exactly 100 per month, and fills each missing bathroom count with the typical count for homes of a similar size, computed *only* from training rows.

Then she checks every candidate column against the prediction time — the leakage check of Chapter 2 and Chapter 5, and the step that most often saves a project from failing after launch. Table 25.2 is the result. The last row is the interesting one: the agent’s own asking price really is known in time, so it is not leakage in the strict sense — but Hawler Homes built this project to help set that very number.

WORD TO KNOW

Circularity — Using as a feature the very quantity the model is meant to produce or replace. A circular feature is not always leakage — it may be known in time — but it hides the model’s real skill, because the model can lean on the human judgment it was supposed to improve.

Table 25.2 – The leakage audit for Project 1. Every candidate column faces one question: would this really be known at the first meeting with the seller?

Candidate column	Verdict	Reason
Size in m ²	Safe	Measured during the first visit
Neighborhood average of earlier sales	Safe	Built only from sales that already closed
Days on the market	Leaks	Only exists after the home has sold
Agent's asking price	Circular	Known in time, but it is the number the tool is meant to help set

A column that only exists after the sale closes can never help predict a price before it, whatever it does to the training score. Nine safe features survive: size in m², bedrooms, bathrooms, age in years, neighborhood, distance to the center in km, garden, covered parking, and house or apartment. Age is not stored directly; Lana builds it by subtracting each home's construction year from its sale year — a small piece of the feature engineering of Chapter 22.

Prices in a growing city do not sit still. Over Lana's 24 months, construction costs rose and a new road opened up land that used to be far away. A random split would drop a few of next month's sales into training by chance, letting the model learn a trend before that trend was knowable. Chapter 5 gave the fix: a time-based split.

WORKED EXAMPLE 25.2 — Split 2,400 Home Sales by Time

The 2,400 clean sales are spread evenly across 24 months, 100 per month. Lana holds out the most recent 3 months as a test set, the 3 months before that as a validation set, and trains on everything older. How many sales fall into each group?

1. Count the training months: $24 - 3 - 3 = 18$.
2. Training sales: $18 \times 100 = 1,800$.
3. Validation sales: $3 \times 100 = 300$; test sales: $3 \times 100 = 300$.
4. Check the total: $1,800 + 300 + 300 = 2,400$, matching the cleaned dataset.

Answer: 1,800 sales from months 1–18 train the model, 300 from months 19–21 validate it, and 300 from months 22–24 test it. Every training sale happened before every validation sale, and every validation sale before every test sale — exactly the protection a time-based split exists to give.

Figure 25.2 shows both things Lana now knows about her data: how prices are spread, and how the 24 months are cut.

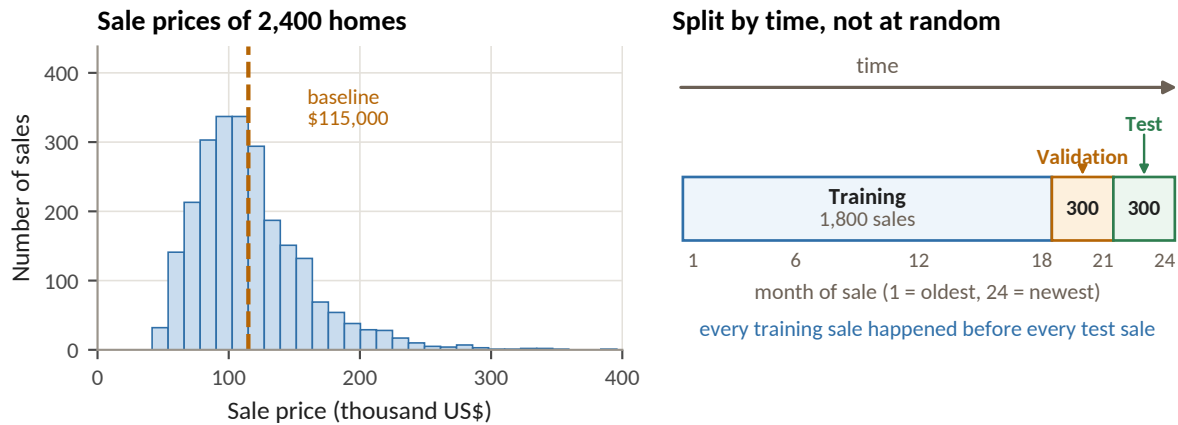


Figure 25.2 – Left: the 2,400 cleaned sale prices, with the training-set average of \$115,000 marked. Right: the time-based split, with the number of sales in each group.

WHAT TO OBSERVE

The price histogram has a long right tail: most homes sell for \$70,000 to \$160,000, a few for over \$300,000. Expensive homes are rare, so the model gets little practice on them — a weakness that returns, with a number attached, in Lesson 25.2.

COMMON MISTAKE

✗ Wrong: Using a plain random split on the 2,400 sales, because it is the fastest and most familiar option.
✓ Right: Sort the sales by date, then cut the three groups along that timeline. A random split lets next month's prices slip into training, making validation scores look better than the live tool will ever perform.

25.2 Project 1: Baseline, Models, Test and Explanation

Before anything clever, Lana builds the simplest possible predictor: always guess the average training price. Chapter 2 insists on this step because it is so cheap that skipping it has no excuse.

WORKED EXAMPLE 25.3 — Build and Test a Baseline Price Predictor

Five representative training sales closed at \$80,000, \$95,000, \$110,000, \$130,000 and \$160,000. Find the baseline's prediction, then its absolute error on a validation home that sold for \$124,000.

1. Add the five prices: $80,000 + 95,000 + 110,000 + 130,000 + 160,000 = 575,000$.
2. Divide by $n = 5$: $575,000 \div 5 = 115,000$.
3. The baseline predicts \$115,000 for every home, whatever its size, neighborhood or age.
4. Absolute error on that home: $|124,000 - 115,000| = 9,000$.

Answer: The baseline predicts \$115,000 and misses this home by \$9,000. Across all 1,800 training sales the average is the same \$115,000, so that is what the baseline predicts for every validation and test home too.

Run against all 300 validation homes, the flat \$115,000 guess reaches a mean absolute error of \$33,900 and a root mean squared error of \$45,600. Neither number means much alone, as Chapter 6 warned. What they give Lana is a floor: any real model must clear \$33,900 by a wide margin or it is not worth its complexity.

Chapter 4 warned about a trap waiting here: any scaling or encoding rule must be learned from training rows only, then applied unchanged. The safest guarantee is to chain every step into one pipeline, so they can never

run out of order. Lana's has two stages: a `ColumnTransformer` that standardizes the seven numeric columns and one-hot encodes the two category columns, and the model itself. The PYTHON activity at the end of this chapter builds exactly this pipeline, line by line.

COMMON MISTAKE

X Wrong: Computing each numeric feature's average and spread across all 2,400 homes — training, validation and test together — before standardizing.

✓ Right: Fit the scaler and encoder on the 1,800 training homes only, then apply them unchanged. A pipeline makes this the default instead of something you must remember.

Lana keeps the preprocessing identical and swaps only the last step, so every candidate is judged on identical data. First comes ridge regression (Chapter 8), the L2-penalized linear model of Arthur Hoerl and Robert Kennard, published in 1970; she tries $\lambda = 1, 5$ and 20 and keeps $\lambda = 20$, though the three barely differ. Second is a random forest (Chapter 14), introduced by Leo Breiman in 2001: 300 trees on bootstrap samples, voting together. Third is gradient boosting (Chapter 15), published by Jerome Friedman in 2001; here she runs the small grid search of Chapter 23 over two learning rates and three depths and keeps the winner, 400 trees at learning rate 0.10 and depth 2. To say how much of the baseline's error each model removed, she reports the improvement $I = (\text{MAE}_{\text{base}} - \text{MAE}_{\text{model}}) / \text{MAE}_{\text{base}}$: the error that disappeared, divided by the error there was to begin with. Both numbers must come from the same measure and the same rows, or the comparison means nothing.

WORKED EXAMPLE 25.4 — Compare Four Models Using Validation MAE

On the same 300 validation homes, the baseline scores a MAE of \$33,900, ridge regression \$12,300, the random forest \$11,400 and gradient boosting \$10,900. Rank them, then find each real model's improvement over the baseline.

1. Rank from worst to best: baseline \$33,900, ridge \$12,300, random forest \$11,400, gradient boosting \$10,900.
2. Ridge: $33,900 - 12,300 = 21,600$, and $21,600 \div 33,900 = 0.637$, that is 63.7%.
3. Random forest: $33,900 - 11,400 = 22,500$, and $22,500 \div 33,900 = 0.664$, that is 66.4%.
4. Gradient boosting: $33,900 - 10,900 = 23,000$, and $23,000 \div 33,900 = 0.678$, that is 67.8%.
5. Compare the best ensemble with ridge: $(12,300 - 10,900) \div 12,300 = 0.114$, about 11.4% better.

Answer: Gradient boosting wins at \$10,900, which is 67.8% below the baseline. But look where the gain came from: plain ridge regression already removed 63.7 of those 67.8 percentage points. All the trees, boosting rounds and tuning together bought the last 4 points.

Table 25.3 lists all four scores side by side.

Table 25.3 – Project 1 model comparison on the 300 validation homes. The spread column is the standard deviation of MAE across five rolling time-based folds.

Model	Validation MAE (\$)	Validation RMSE (\$)	Spread over 5 folds (\$)	Below baseline
Baseline (average price)	33,900	45,600	$\pm 2,400$	—
Ridge regression ($\lambda = 20$)	12,300	17,600	± 600	63.7%
Random forest (300 trees)	11,400	16,200	± 400	66.4%
Gradient boosting (400 trees)	10,900	16,900	± 400	67.8%

Figure 25.3 draws the same four scores.

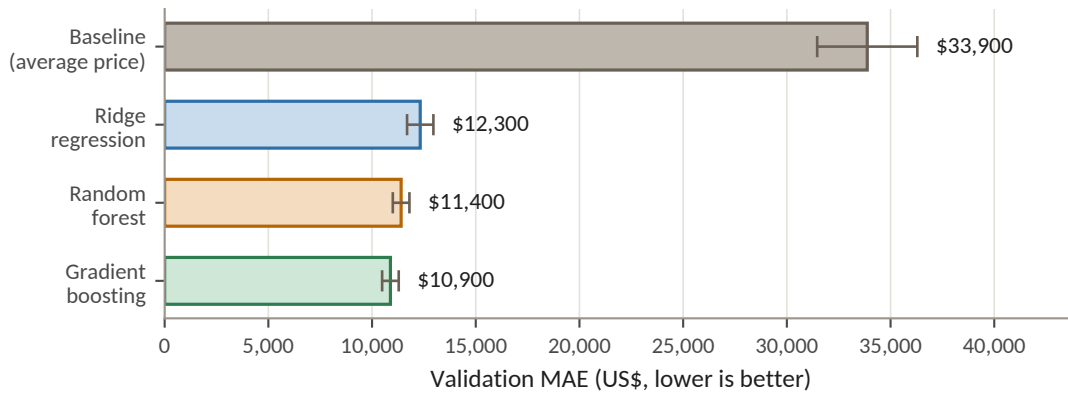


Figure 25.3 – Validation MAE for the baseline and the three real models. The whiskers show the spread across five rolling time-based folds.

Two details deserve a careful reader. The gap between the random forest and gradient boosting, \$500, is close to the spread of either across folds, \$400, and Chapter 23 warned that a difference no larger than the wobble between folds is not one to defend loudly. And gradient boosting wins on MAE while the random forest wins on RMSE, because RMSE punishes large errors more heavily (Chapter 6). Lana chose MAE *before* fitting anything, so she keeps gradient boosting — and writes the disagreement into her report rather than hiding it.

WHAT TO OBSERVE

The baseline's bar dwarfs the other three and its whisker is by far the longest: a flat guess is not only worse, it is less stable from period to period. The three real models sit so close that their whiskers almost touch.

Gradient boosting won, so it is the one model Lana runs on the 300 test homes — the most recent three months, opened for the first and only time, with every setting already locked. It scores a MAE of \$11,500 and an RMSE of \$16,400. Against the validation MAE of \$10,900 that is a rise of \$600, or $600 \div 10,900 = 5.5\%$ — exactly the small rise Chapter 5 predicts, because the validation set helped choose the model and its settings and therefore flatters them slightly. The \$11,500 is the honest number Lana reports, and Figure 25.4 shows what it looks like home by home.

One overall number hides an uneven pattern, so Lana asks the question Chapter 6 always pushes her toward: honest *for whom*?



Figure 25.4 – Actual against predicted price for the 300 test homes. Points on the dashed line are perfect predictions; the dotted vertical lines mark the two band boundaries used below.

WHAT TO OBSERVE

The cloud hugs the dashed line on the left and fans out on the right. Above \$200,000 the points sit mostly *below* it: the model under-predicts the most expensive homes, because it has seen few of them and because what makes one expensive — a rare view, a custom finish — is in no column.

WORD TO KNOW

Error slice — The error of a model computed on one meaningful subgroup of the test set — one price band, one region, one age group — instead of on the whole set at once. Slices reveal who the model serves badly.

EQUATION: MEAN ABSOLUTE ERROR INSIDE ONE SLICE

$$\text{MAE}_g = \frac{1}{n_g} \sum_{i \in g} |y_i - \hat{y}_i|$$

In words: Keep only the test examples belonging to group g , add up their absolute errors, and divide by how many examples that group has.

When it is used: After the overall test score, to check whether one group of examples — or one group of people — is served much worse than the rest.

Symbol	Meaning	Unit or type
MAE_g	mean absolute error inside group g	unit of the target
n_g	number of test examples in group g	count
$i \in g$	each test example belonging to group g	index
$y_i, \hat{y}_i$	true value and predicted value of example i	unit of the target

The overall MAE is the weighted average of the slices: $\text{MAE} = \sum_g n_g \text{MAE}_g / n$. Dividing MAE_g by the group's typical value turns it into a relative error, which is what makes bands at different price levels comparable.

WORKED EXAMPLE 25.5 — Break the Test Error Down by Price Band

Of the 300 test homes, 51 sold for under \$80,000 with a band MAE of \$6,500 and a typical price of \$67,000; 185 sold for \$80,000 to \$150,000 with a band MAE of \$9,500 and a typical price of \$113,000; 64 sold for over \$150,000 with a band MAE of \$21,400 and a typical price of \$184,000. Find each band's relative error, then check that the bands rebuild the overall test MAE.

1. Lowest band: $6,500 \div 67,000 = 0.097$, about 9.7%.
2. Middle band: $9,500 \div 113,000 = 0.084$, about 8.4%.
3. Highest band: $21,400 \div 184,000 = 0.116$, about 11.6%.
4. Weighted average: $(51 \times 6,500 + 185 \times 9,500 + 64 \times 21,400) \div 300 = 3,458,600 \div 300 \approx 11,529$.

Answer: The middle band is the model's best ground at 8.4%, because that is where most of Hawler Homes' past sales sit. Homes over \$150,000 are its weakest at 11.6%, and their absolute error, \$21,400, is more than three times the lowest band's. The weighted average, about \$11,500, matches the overall test MAE, confirming that the three slices account for the whole score.

Table 25.4 collects the three slices.

Table 25.4 – Test error by price band. One overall score of \$11,500 hides errors that differ by a factor of three.

Price band	Homes	Band MAE (\$)	Typical price (\$)	Error as % of price
Under \$80,000	51	6,500	67,000	9.7%
\$80,000 to \$150,000	185	9,500	113,000	8.4%
Over \$150,000	64	21,400	184,000	11.6%

IMPORTANT

An overall test score can hide a model that serves one group far worse than another. When the groups are price bands, the cost is an awkward conversation. When the groups are people — patients, applicants, students — the cost is unfairness. Always compute error slices, not just the headline number (Chapter 24).

COMMON MISTAKE

✗ Wrong: Noticing the weak result on expensive homes, retuning the model, and checking the same 300 test homes again to see whether the new settings look better.

✓ Right: Write the weakness down and make it a goal for the next version. Once a test set has been opened and reported, using it to guide more changes quietly turns it into a second validation set — the trap of Chapter 5.

A price range with no explanation is hard for an agent to defend to a doubtful seller. Chapter 24 gives two tools. Impurity-based importance reads the finished trees: size 71%, neighborhood 10%, distance 6%, everything else below 5%. Permutation importance, proposed by Leo Breiman alongside random forests in 2001, tests the finished model instead: shuffle one feature's values across the validation set and see how much worse the score gets. Shuffling size alone pushes validation MAE from \$10,900 to \$30,500 — a rise of \$19,600, or 179.8% of the original error — while shuffling neighborhood pushes it to \$16,900, a rise of \$6,000, or 55.0%. Figure 25.5 ranks all nine features.

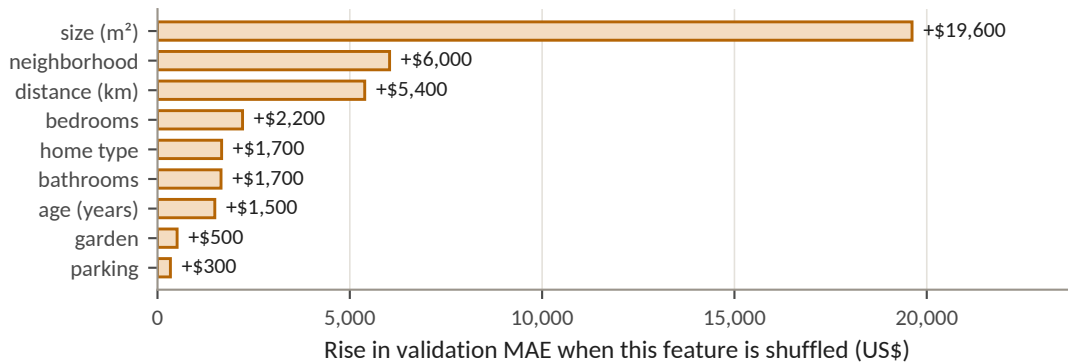


Figure 25.5 – Permutation importance for the price model: how much the validation MAE rises when each feature’s values are shuffled.

WHAT TO OBSERVE

Both measures agree on the backbone — size, then neighborhood, then distance — and disagree about the small features, where home type and bathrooms are identical to the nearest hundred dollars. Prefer permutation importance where they disagree, and where bars are this close do not rank them at all. Neither proves that a feature *causes* a higher price (Chapter 24).

For one seller across the table the agent needs something smaller than a global ranking, so the tool shows the five most similar past sales beside every prediction. For a 165 m² house in Ankawa, four years old, 3.5 km from the center, it predicts about \$221,000 — well above the \$115,000 average — and those five similar sales make it obvious why.

A finished model is still not a finished project. Following Chapter 24, Lana and the director agree on a monitoring plan *before* launch: track live MAE every month against the \$11,500 test score; confirm that the exact fitted pipeline used for testing is the one running in the agents’ app, not a hand-rebuilt copy; and keep the previous pipeline ready for a rollback. Figure 25.6 is what that plan produced.

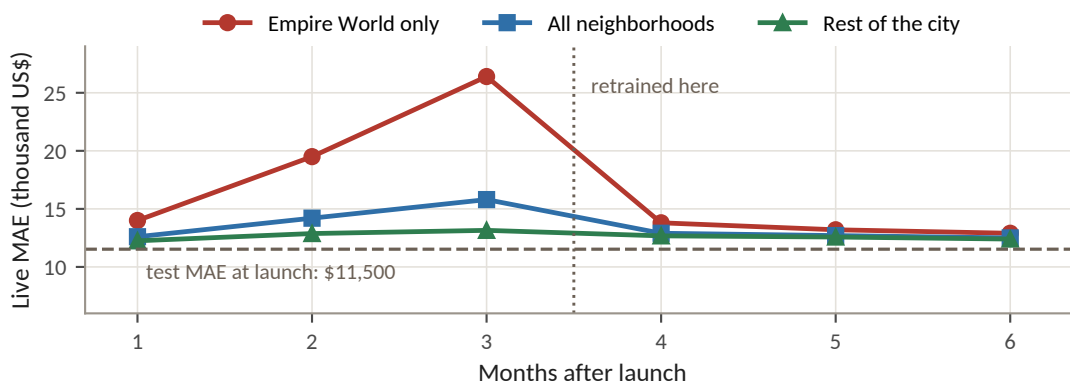


Figure 25.6 – Live error in the six months after launch, for the whole city and for two groups of neighborhoods separately.

WHAT TO OBSERVE

The blue line, the only number most teams watch, creeps from \$12,600 to \$15,800 and gives no hint where the problem is. The slices give it away instantly: the rest of the city is stable while Empire World doubles its error. Slices turn “something is wrong” into “here is what is wrong”.

25.3 Project 2: Framing a Churn Decision and Splitting It Safely

Meet Darya, a data analyst at Zagros Mobile, a phone company serving about 20,000 postpaid customers from Duhok in the north to Basra in the south. Like every subscription business it loses some customers every month. Her manager Ronahi leads the small retention team, and asks a simple question: could a model tell us, early enough to matter, which customers are about to leave?

WORDS TO KNOW

Churn — A customer ending their relationship with a business: canceling a subscription, closing an account, or simply not renewing. In a churn project the label usually marks whether a customer leaves within a fixed future window, such as the next 30 days.

Silent churn — A customer who stops using a service but never cancels, so no churn label is ever recorded even though the business is, in practice, already lost.

A churn prediction is only worth making if it leads to a helpful action, so Darya writes: “At prediction time, on the first day of the month, the model will use a customer’s tenure, usage, billing and support history through the end of last month to predict whether that customer will churn in the next 30 days, so that the retention team can call the customers most at risk while there is still time to help.” The right-hand column of Table 25.1 records its four parts. Notice what the sentence rules out: a score with no owner and no next step, and any column not sitting in the company’s systems on the first of the month. It also settles in advance what happens once the model flags somebody — a phone call offering real help, never a penalty, never a locked account, never a note the customer can see.

RESPONSIBLE USE

A churn prediction is not a fact about a customer’s loyalty and not proof that anyone has decided to leave. Use a high-risk prediction to offer a genuinely helpful reason to stay — fixing a real problem, or a fair offer — never to pressure, guilt or trap a customer into staying.

Darya’s table holds one row per customer per month, so the same customer appears many times. That shape creates two leakage risks. The first is the familiar time leakage of Chapter 5: a row from next month must never train a model that is then tested on a row from last month, and the fix is the time-based split Lana used. The second comes from having many rows per person.

WORD TO KNOW

Customer leakage — Information about one particular person or object crossing from the training set into the test set, because several rows describe the same person or object. A flexible model can partly recognize that individual instead of learning the general pattern, and the test score is then flattered.

If one customer’s January row lands in training and the same customer’s March row lands in test, the two rows are not independent: same city, same plan, nearly identical tenure and usage, because they describe one person two months apart. The fix is to split whole customers rather than rows.

WORKED EXAMPLE 25.6 — Split Zagros Mobile’s Customers, Not Their Rows

Six months of snapshots give 42,000 rows from 7,000 unique customers, up to 6 rows each. The team splits the 7,000 *customers* into 70% training, 15% validation and 15% test, moving every row belonging to one customer into that customer’s single group. How many customers land in each group, and roughly how many rows travel with them?

1. Training customers: $0.70 \times 7,000 = 4,900$.
2. Validation customers: $0.15 \times 7,000 = 1,050$; test customers: 1,050 as well.
3. Check: $4,900 + 1,050 + 1,050 = 7,000$, matching the original count.

4. Rows travel with their customer, about 6 each: $4,900 \times 6 = 29,400$ training rows, and $1,050 \times 6 = 6,300$ rows for each of validation and test.
5. Check: $29,400 + 6,300 + 6,300 = 42,000$, matching the original rows.

Answer: 4,900 customers and about 29,400 rows go to training; 1,050 customers and about 6,300 rows go to each of validation and test. No customer's rows are ever split across two groups, so the model is always tested on customers it has genuinely never seen.

COMMON MISTAKE

X Wrong: Splitting a table of monthly customer snapshots randomly, row by row, so the same customer's snapshots land in both training and test.

✓ Right: Split by customer, not by row, and keep each customer's test snapshots later in time than their training snapshots. Both leaks need fixing, and both fixes are cheap.

A churn label sounds obvious until you write its exact rule. Does a customer count as churned when they stop using the phone, or only when they formally cancel? What about a card payment that fails and is never fixed? Zagros Mobile agrees on one written definition first: a customer counts as churned if their status becomes "canceled", or becomes "suspended for nonpayment" and stays suspended for more than 30 days, checked within the 30 days after prediction time. Applied to the window 1–31 March, that rule says: a customer who canceled on 12 March is churn; one whose payment failed on 5 March and was restored on 9 March is not, because four days is far short of 30; one suspended on 20 March and still suspended on 31 March is not either, because only 11 days have passed when the label must be recorded; and one who stopped using the service on 2 March but kept paying automatically is not, even though that customer is, in every practical sense, already lost.

That last case is a real blind spot. No cleverer formula fixes it; the team can only be honest about it and consider tracking usage decline as a second project. A label definition is a choice, not a fact of nature, and every choice leaves something out — so write yours down, share it with the people who will act on it, and say plainly what it does not catch.

The feature list gets the same scrutiny. Two candidates are clearly safe: trailing three-month data use, and late payments in the last 90 days. Two are clearly unusable: "days since the account was marked pending cancellation" and "final closing balance" exist only once a customer has already started leaving. One is genuinely hard: the support system logs a "retention save attempt" tag whenever an agent notices a customer sounding unhappy. It passes the timing check — but an agent applies it only after noticing the very warning signs the model is meant to learn, so the model would partly copy a human hunch. Darya leaves it out. "We are not sure, so we leave it out for now" is a legitimate result of an audit.

Churn is the classic case where one accuracy number hides a useless model, because most customers do not churn in any given month. In the validation set, 630 of the 6,300 customer-months end in churn and 5,670 do not.

WORKED EXAMPLE 25.7 — Evaluate Two Opposite Baselines

Using those 6,300 validation customer-months, find the accuracy and recall of a baseline that always predicts "Stay", then the recall and precision of the opposite baseline that always predicts "Churn".

1. "Always Stay" is right on all 5,670 stayers and wrong on all 630 churners, so accuracy = $5,670 \div 6,300 = 0.90$, or 90%.
2. It never predicts "Churn", so TP = 0 and FN = 630, giving recall = $0 \div 630 = 0$.
3. "Always Churn" catches every churner: TP = 630, FN = 0, so recall = $630 \div 630 = 1.0$, or 100%.
4. It also flags all 5,670 stayers, so precision = $630 \div 6,300 = 0.10$, or 10%.

Answer: "Always Stay" reaches 90% accuracy with exactly 0% recall: impressive on a slide, useless to the retention team. "Always Churn" reaches 100% recall with 10% precision: nine wasted calls in every

ten. Every useful model lives between these two useless extremes — which is why a churn model's accuracy must never be reported without precision and recall beside it (Chapter 6).

25.4 Project 2: Calibrate, Choose a Threshold, Test Once, Watch

Darya trains three candidates on the training customers: a logistic regression (Chapter 10), a random forest (Chapter 14) and a gradient boosting model (Chapter 15). Table 25.5 compares them on validation at the default threshold of 0.5, reporting precision, recall and F1 together as Chapter 6 requires. Gradient boosting wins, which echoes what Chapter 15 said honestly about ordinary tabular business data. Darya keeps the other two saved, in case a later fairness check favors a simpler model.

Table 25.5 – Project 2 model comparison on the validation set, at the default threshold of 0.5, with both baselines included so every score has something to beat

Method	Precision	Recall	F1
Baseline: always predict “Stay”	—	0%	0%
Baseline: always predict “Churn”	10%	100%	18.2%
Logistic regression	45%	28%	34.5%
Random forest	60%	35%	44.2%
Gradient boosting	65%	39%	48.8%

A strong F1 is not the last question. A model can rank customers perfectly while its actual numbers run too high or too low, and calibration (Chapter 6) compares predicted probabilities with how often those customers really churned. Darya gathers the 50 validation customers whose predicted probability fell between 0.70 and 0.80: if the probabilities were trustworthy about 75% should churn, but $44 \div 50 = 0.88$, or 88%, did. In this range the model is *underconfident*. So she adds a calibration step after training, of the kind John Platt proposed in 1999, fitted on validation data only (Figure 25.7).

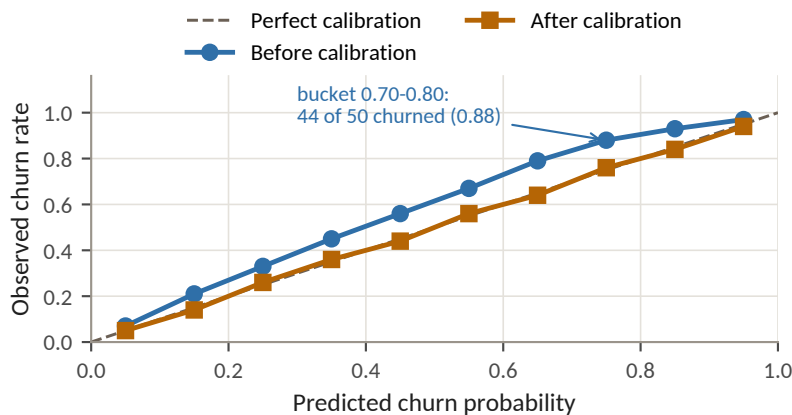


Figure 25.7 – Calibration of the churn model before and after the calibration step. Points above the dashed line mean the model is underconfident.

WHAT TO OBSERVE

Before calibration every point sits above the dashed line. After it the points lie close, so a predicted 0.60 really does mean about 60 customers in 100. Every money figure from here on rests on that repair.

A trained model gives a probability, not yet a decision. Chapter 6 showed that the 0.5 cutoff can be moved when one kind of mistake costs more than another. Zagros Mobile's version of that choice has two parts.

WORDS TO KNOW

Capacity — How many flagged cases the people who act on a model can actually handle in one cycle. Capacity turns a ranking into a shortlist of a fixed length.

Save rate — The share of genuinely at-risk customers who stay *because* they were contacted. It is a business estimate, not something the model predicts.

Net value — What a campaign is worth after its costs: the value of the customers it keeps, minus the cost of contacting everybody who was flagged.

Ronahi's team has 4 staff, each able to make about 200 thoughtful, unrushed calls a month. That is a hard capacity of 800 calls, against 20,000 customers scored on the first of the month. Whatever threshold Darya chooses must flag close to 800 people: many more would overwhelm the team, many fewer would waste capacity Ronahi already pays for.

EQUATION: EXPECTED CUSTOMERS SAVED AT A THRESHOLD

$$S = F \times p \times r$$

In words: Multiply the number of customers flagged at this threshold by the share of those flags that are genuine (the precision), then multiply again by the share of genuine at-risk customers a call actually keeps.

When it is used: Once a threshold is proposed, to turn a precision into a count of real people the campaign expects to keep.

Symbol	Meaning	Unit or type
S	expected customers kept who would otherwise have left	count
F	customers flagged at this threshold	count
p	precision at this threshold, from the validation set	value from 0 to 1
r	save rate: share of contacted at-risk customers who stay	value from 0 to 1

Only flagged customers who were genuinely about to leave can be “saved”, which is why precision appears. The save rate r is an assumption until an experiment measures it, as the case study at the end of this chapter finally does.

EQUATION: NET VALUE OF A CAMPAIGN

$$V = S \times v - F \times c$$

In words: Take the money gained from the customers you keep, then subtract the money spent contacting everybody you flagged — including the flagged customers who were never going to leave.

When it is used: To decide whether a threshold is worth acting on at all, and to compare two thresholds fairly.

Symbol	Meaning	Unit or type
V	net value of the campaign for one cycle	currency
S	expected customers saved, from the previous equation	count
v	value of keeping one customer	currency
F	customers contacted at this threshold	count
c	cost of one contact, including staff time and any offer	currency

A threshold that flags more people is not automatically better: more flags mean more chances to save somebody, but also more money spent on people who were never at risk. Dividing V by F gives the net value per call.

Every currency figure below is an illustrative planning number, not a verified statistic. Zagros Mobile estimates that keeping one customer is worth \$90 in future revenue, that a retention call costs about \$5 in staff time and offers, and — from past campaigns — that a call keeps about 30% of the truly at-risk customers it reaches.

WORKED EXAMPLE 25.8 — Expected Saves and Net Value at the Chosen Threshold

The chosen threshold, 0.62, flags 800 of this month's 20,000 customers, with a validation precision of 0.75. Using a save rate of 0.30, a value of \$90 per save and a cost of \$5 per contact, find the expected saves and the net value.

1. Expected true churners among the flagged: $F \times p = 0.75 \times 800 = 600$.
2. The other $800 - 600 = 200$ flagged customers were never at risk; a call cannot save somebody who was not leaving.
3. Apply the save rate to the 600 only: $S = 0.30 \times 600 = 180$.
4. Money gained: $S \times v = 180 \times 90 = 16,200$ dollars.
5. Money spent: $F \times c = 800 \times 5 = 4,000$ dollars.
6. Net value: $V = 16,200 - 4,000 = 12,200$ dollars.

Answer: The month's campaign expects to keep about 180 customers and to be worth about \$12,200, which is $12,200 \div 800 = \$15.25$ of net value per call. The other 620 calls either reach somebody who was never leaving (200 of them) or reach a true churner who leaves anyway (420).

Figure 25.8 shows why 0.62 and not some other number, and Table 25.6 works the same arithmetic out at six thresholds.

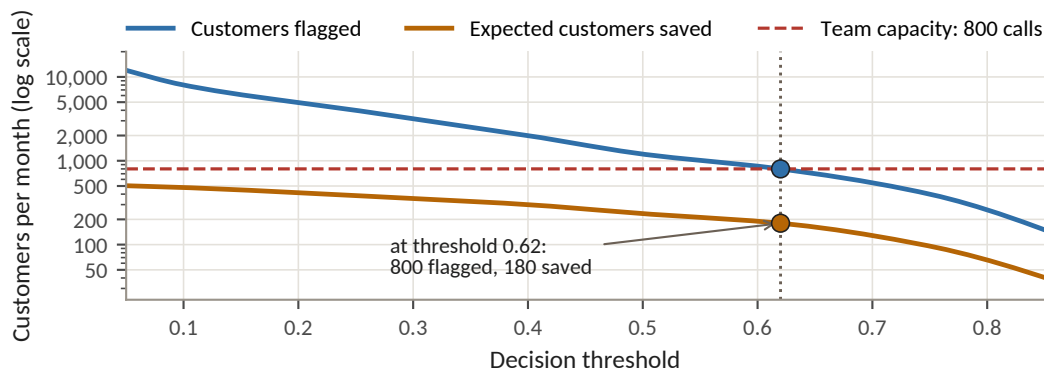


Figure 25.8 – How many customers a threshold flags, and how many the campaign expects to keep. Both curves are counts, so they share one log scale.

WHAT TO OBSERVE

Lowering the threshold flags many more people but keeps far fewer extra customers: from 0.62 down to 0.10 the flagged count grows tenfold while expected saves do not even triple. That widening gap is the cost of a looser threshold, drawn as a picture.

Table 25.6 – Six thresholds for the churn model, scored against 20,000 customers of whom 2,000 truly churn. Save rate 0.30, value \$90 per save, cost \$5 per contact.

Threshold	Flagged	Precision	Expected saved	Net value (\$)	Value per call (\$)
0.75	400	0.80	96	6,640	16.60
0.62	800	0.75	180	12,200	15.25
0.50	1,200	0.65	234	15,060	12.55
0.40	2,000	0.50	300	17,000	8.50
0.25	4,000	0.32	384	14,560	3.64
0.10	8,000	0.20	480	3,200	0.40

Figure 25.9 puts the same six thresholds on two curves: what the model is doing, and what the campaign is worth.

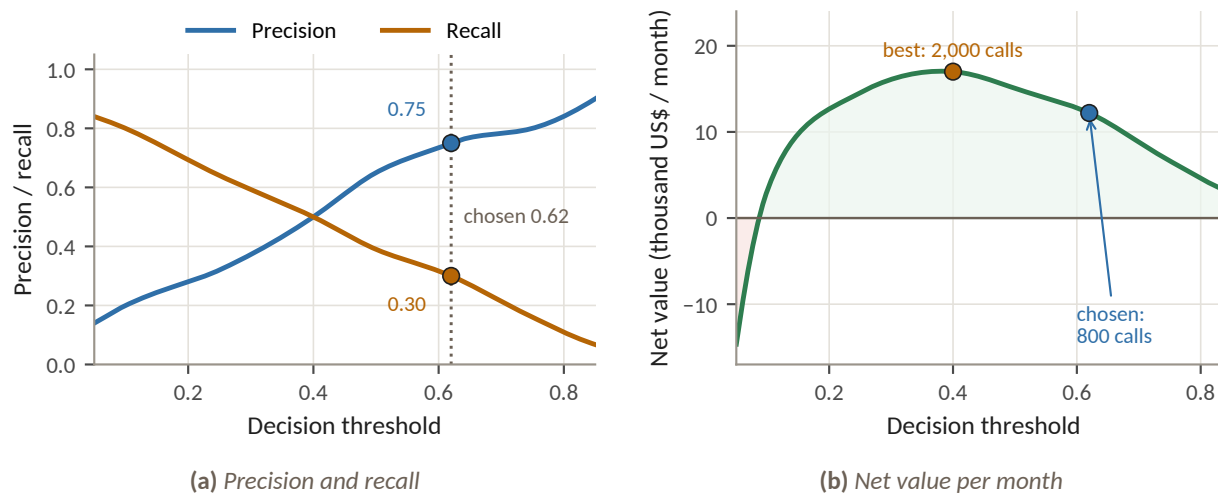


Figure 25.9 – Two views of the same threshold. Left: precision and recall, with the chosen operating point marked. Right: net value, which rises, peaks and then falls below zero, because every extra call costs money whether or not it reaches somebody at risk.

WHAT TO OBSERVE

In the left panel the two curves move in opposite directions, as Chapter 6 showed: at 0.62 the model is right about three flags in four but reaches only 30% of the customers who will leave. In the right panel the value-maximizing threshold is not the chosen one — flagging 2,000 customers would be worth \$17,000 a month against \$12,200 — but it needs 2,000 calls, and Ronahi has 800. Below a threshold of about 0.09 the campaign turns into a loss.

WORKED EXAMPLE 25.9 — Check Whether a Looser Threshold Would Create More Net Value

Ronahi asks whether hiring enough help for 1,200 calls a month would pay. At the looser threshold of 0.50 the model flags 1,200 customers with a precision of 0.65. Using the same save rate, value and cost, find the net value and compare it with the 800-call plan.

1. True churners among the flagged: $0.65 \times 1,200 = 780$.
2. Expected saves: $S = 0.30 \times 780 = 234$.
3. Money gained: $234 \times 90 = 21,060$ dollars; money spent: $1,200 \times 5 = 6,000$ dollars.
4. Net value: $V = 21,060 - 6,000 = 15,060$ dollars.
5. Compare the extra: $15,060 - 12,200 = 2,860$ dollars from $1,200 - 800 = 400$ extra calls, that is $2,860 \div 400 = \$7.15$ per extra call.

Answer: The looser plan is worth more in total, \$15,060 against \$12,200. But the 400 extra calls earn only \$7.15 each, less than half the \$15.25 the first 800 earn. Ronahi should expand only if extra call capacity costs less than about \$7 per call — and that is a staffing question, not a modeling one.

COMMON MISTAKE

X Wrong: Leaving the threshold at the default 0.5, or choosing whichever threshold gives the highest recall, without asking how many calls the team can make or what a call is worth.

✓ Right: Choose the threshold from capacity and value together. The best threshold is almost never the one with the highest recall, and rarely the one with the highest precision.

Every piece is ready: audited features, a safe split, a calibrated model and a threshold chosen from real capacity and real value. Darya opens the test set for the first and only time.

WORKED EXAMPLE 25.10 — Report the Final Test Result, One Time Only

On the untouched test set of 6,300 customer-months, 630 of which end in churn, the finished pipeline at threshold 0.62 produces $TP = 175$, $FP = 75$, $FN = 455$ and $TN = 5,595$. Find precision, recall and F1, then compare with the validation figures at the same threshold: precision 0.75, recall 0.30.

1. Check the total: $175 + 75 + 455 + 5,595 = 6,300$, matching the test set.
2. Precision = $TP \div (TP + FP) = 175 \div 250 = 0.70$, or 70%.
3. Recall = $TP \div (TP + FN) = 175 \div 630 = 0.278$, or 27.8%.
4. $F1 = 2 \times 0.70 \times 0.278 \div (0.70 + 0.278) = 0.389 \div 0.978 = 0.398$, about 39.8%.

Answer: The honest final score is 70% precision, 27.8% recall and an F1 near 39.8%, against 75% and 30% on validation, where F1 was 42.9%. Both slipped a little, which is normal and healthy: the validation set helped choose the threshold, so the test set is the truer measure.

Darya then does what Chapter 24 demands before a single customer is called: she recomputes recall for the five largest service regions. Four sit between 28% and 30%, close to the overall 27.8%; Basra sits at 20%. The cause is not mysterious — postpaid plans launched there only eight months ago, so the training data holds far fewer complete histories. She also re-examines the features, because a column can carry unfairness without naming a group. Handset price tier is the risk: a phone model tracks income. Darya keeps it, because it carries real information about usage, but the team commits to something stronger: whatever a flagged customer is offered depends only on their own risk and their own account history, never on their region, their phone or any group they belong to.

Fairness is also a property of what happens after the model speaks. Every flagged customer is reviewed by an agent before any offer is made, never contacted automatically. An agent can see that a flagged customer simply had one hard month — a late payment after a hospital visit — and choose a warmer conversation than a bare risk number would suggest. The model decides who gets a closer look; a person still decides what to say.

Chapter 24 taught that a live model needs three kinds of watching. The first is data drift: the team tracks each feature's weekly average against the training average, because when a new unlimited-data plan launches, usage jumps overnight. The second is the model's own honesty: as each 30-day window closes the true outcome becomes known, so recall, precision and calibration can be recomputed on live customers, and a steady drop warns that a rollback may be needed.

The third is the most important and the easiest to get wrong. Suppose called customers churn less than customers who were never flagged. That is not evidence the calls work: the called customers were, by design, the riskiest people in the base, so the two groups differed before any phone rang.

WORD TO KNOW

Holdback group — A randomly chosen slice of the flagged cases that is deliberately *not* acted on, kept only so the team has a fair comparison group of equally at-risk cases. Without it, nobody can tell whether the action changed anything.

IMPORTANT

A prediction is never proof that acting on it changes anything. Only a controlled comparison — equally at-risk customers contacted, against equally at-risk customers not contacted — can show whether a retention call changes what a customer does next.

So each month one flagged customer in ten is randomly held back. Because both groups were chosen the same way, any difference between them can honestly be credited to the call. What that experiment found closes Zagros Mobile's story below.

25.5 Six Project Ideas You Can Do

Table 25.7 sketches six projects a student, a small team or a small organization could realistically carry out, each framed as Chapter 1 and Chapter 2 taught: a unit of observation, a target, the data you would need, and the number you would report.

Start simple. Linear or ridge regression (Chapter 8) is the right first model for solar output, shop demand and accident counts; logistic regression (Chapter 10) for no-shows, support flags and crop disease. Then try a random forest (Chapter 14) and a boosted model (Chapter 15), tuned as Chapter 23 describes, and keep whichever wins on validation by more than the spread across folds. Each project needs a baseline too — the clinic's no-show rate, yesterday's output, last week's sales — and each has one trap that will spoil it if you do not plan for it:

- **Clinic no-shows:** the reminder call itself changes the outcome, so a model trained after reminders start no longer predicts the same thing — and a no-show prediction must never be used to refuse or deprioritize care.
- **Solar output and shop demand:** split by time, never at random. The measured irradiance for the hour you are predicting is leakage, and a stock-out looks exactly like low demand in till records.
- **Exam-support flags and crop disease:** split by student and by field, not by term or by leaf; several leaves from one field, like several terms of one student, are not independent examples. Flags go to a caring adult, never onto a record.
- **Road-accident risk:** a predicted risk is not a cause, and a segment list must never become a way to blame the people who live near it.

Table 25.7 – Six projects you could carry out, each stated as task, experience and performance

Project	One row, and what to predict (T)	Data you would need (E)	What to report (P)
Clinic no-shows, Sulaymaniyah	One appointment; will the patient miss it?	Two years of bookings: lead time, day, hour, age band, attendance	Recall and precision at the clinic's reminder-call capacity
Solar output, Duhok	One hour of one rooftop system; how many kWh?	A year of inverter logs plus hourly weather data	MAE in kWh against "same hour yesterday"
Exam-support flags, Erbil	One student in one term; will they need support?	Attendance, homework counts, early quiz marks	Recall inside every group, and flags a counselor can act on
Crop disease, near Zakho	One field visit; is the crop diseased?	Leaf measurements, soil moisture, humidity, a verdict	Recall for the diseased class, with the cost of a miss
Shop demand, Duhok	One product on one day; how many units sold?	Two years of till records, holidays, weather	MAE in units against "same weekday last week"
Road-accident risk	One road segment in one month; how many accidents?	Length, curves, lighting, traffic, past accidents	MAE, plus recall of the worst segments

RESPONSIBLE USE

Two of these ideas make predictions about named people — patients and students. For those, write down before you start: who sees the output, what help it triggers, who may overrule it, and what it must never be used for. If you cannot answer all four, build one of the other four projects instead.

25.6 How to Present a Machine-Learning Project

A project nobody can read is a project nobody can trust. The last skill this book teaches is the smallest: fitting an honest account of your work onto one page.

WORD TO KNOW

Model card — A short, plain-language document that travels with a trained model, describing what it predicts, what data it learned from, how well it scored, where it is weakest, and what it must not be used for. The format was proposed by Margaret Mitchell and colleagues in 2019.

Table 25.8 is a template you can copy for any project here or of your own. Fill it in as you go: most rows can be written before a single model is trained. Rows 8, 9 and 10 are the ones beginners leave out and experienced reviewers read first. A report that gives one headline score and stops is not a report; it is an advertisement.

Table 25.8 – A one-page report template for a machine-learning project

Section	What goes in it
1. Decision sentence	“At prediction time, the model will use ___ to predict ___, so that ___”, plus the unit of observation and the exact moment of prediction
2. Data	Source, rows, period covered, and what cleaning was done to how many rows
3. Excluded columns	Every column dropped for leakage or circularity, with the reason
4. Split	How the data was split, why that way, and the size of each group
5. Baseline	What the baseline predicts and the score it reached
6. Models compared	Each model, its settings, and its validation score with a spread
7. Chosen model	Which one and why, including cost, speed and interpretability, not only the score
8. Final test result	One number, reported once, beside the baseline it must be judged against
9. Error slices	The score inside meaningful subgroups, and the group served worst
10. Limits and monitoring	What the model must never be used for; what is tracked, how often, and the line that triggers a rollback

COMMON MISTAKES AND MISCONCEPTIONS

- Choosing an algorithm before writing the plain decision sentence that names the unit of observation, the target and the prediction time.
- Keeping a feature that would not truly be known at prediction time, such as days on the market, or a circular one, such as the asking price the tool is meant to help set.
- Splitting rows at random when the data has an order in time, or when several rows describe the same person.
- Skipping the baseline, or fitting a scaler on all the data before splitting instead of inside a pipeline on training rows only.
- Reporting one overall score without slices, so a group the model serves badly stays invisible; or reopening the test set to check whether a change helped.
- Trusting a model’s probabilities as real chances without checking calibration, then multiplying them by money; or leaving the threshold at 0.5 without asking how many cases the team can handle.
- Reading “the people we contacted did better” as proof that contacting them helped, instead of running a holdback comparison.

Practical Activities

ACTIVITY (PYTHON) — Rebuild Project 1 End to End

This program repeats the whole of Project 1 on the 2,400 simulated Hawler Homes sales in `datasets/ch25_homes.csv`: a time split, one pipeline, the flat baseline and the tuned boosting model, scored on validation and then once on the test months.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

homes = pd.read_csv("datasets/ch25_homes.csv")
train = homes[homes.sale_month <= 18] # oldest 18 months
val = homes[homes.sale_month.between(19, 21)]
test = homes[homes.sale_month >= 22] # newest 3 months
num = ["size_m2", "bedrooms", "bathrooms", "age_years",
       "distance_km", "garden", "parking"]
cat = ["neighborhood", "home_type"]
pipe = Pipeline([("prep", ColumnTransformer([
    ("scale", StandardScaler(), num),
    ("encode", OneHotEncoder(handle_unknown="ignore"), cat)])),
    ("model", GradientBoostingRegressor(n_estimators=400, learning_rate=0.1,
                                       max_depth=2, random_state=0))])

pipe.fit(train[num + cat], train.price_usd)
flat = train.price_usd.mean()
print("baseline prediction: $%.0f" % flat)
for name, part in [("validation", val), ("test", test)]:
    base = mean_absolute_error(part.price_usd, [flat] * len(part))
    mae = mean_absolute_error(part.price_usd, pipe.predict(part[num + cat]))
    print("%-10s baseline MAE $%.0f    model MAE $%.0f" % (name, base, mae))
```

Output

```
baseline prediction: $114974
validation baseline MAE $ 33877    model MAE $ 10885
test          baseline MAE $ 33002    model MAE $ 11528
```

The three `homes[...]` lines cut the data by month, so no training sale ever happens after a test sale, and one `Pipeline` learns the scaler, the encoder and the 400 trees together from training rows only. The printed numbers are the ones this chapter quotes. Now change one thing at a time — drop `max_depth=2`, or split at random with `sample(frac=0.75)` instead of by month — and watch which numbers move.

You need: A computer with Python, pandas and scikit-learn (see Appendix B), and `datasets/ch25_homes.csv`

ACTIVITY (PAPER) — Write a One-Page Project Plan

1. Choose one of the six ideas in Table 25.7, or invent one for your own school, workplace or town.
2. Write the decision sentence with all three blanks filled in, then name the unit of observation, the target and the prediction time.
3. List eight candidate columns. Mark each one *safe*, *leaks* or *circular*, with one short reason. You should end up with at least one of each mark.
4. Choose the split — random, stratified, by time or by person — and write one sentence saying why that choice and not another.

5. Write the baseline you would build, the performance measure you would report, and the score the baseline would have to beat before you would call a model useful.
6. Name two error slices you would compute, then finish with two sentences: what the model must never be used for, and who may overrule it.

You need: Paper and pen; the template of Table 25.8

CASE STUDY — Two Launches — Hawler Homes in Erbil and Zagros Mobile Across Iraq

Hawler Homes launches the pricing tool inside the app its agents already use. Every range carries one sentence, exactly as the decision sentence promised: a starting point for a discussion, not an appraisal. The rollout is deliberately small — only agents who volunteer. Three months in, Lana checks the live numbers against her test-set report (Figure 25.6). Overall MAE has crept from \$12,600 to \$15,800, well above the \$11,500 reported at launch. Rather than retraining blindly, she slices: the rest of the city holds near \$13,000, while Empire World alone has doubled to \$26,400. The cause is outside the data entirely — a new construction phase opened there halfway through the quarter, and homes in it sell for far more than anything in the training months.

Lana adds the newest sales, refits the pipeline and — this is the part that matters — confirms the refreshed model scores better on a fresh, more recent test period *before* replacing anything. The old pipeline stays ready for a rollback, and from month four the blue line falls back to \$12,900, then \$12,500. The agent who was most skeptical at the start tells her the ranges now land close to his own estimate more often than not. He still overrides it sometimes, and Lana is glad: an experienced agent catching a model's occasional bad guess is exactly the human check this project was built to keep.

At Zagros Mobile the churn model goes live six months after Darya wrote her decision sentence. On the first of every month it scores all 20,000 active customers, flags about 800, and sends the list to Ronahi's team, highest risk first, with each customer's account history attached. Nothing about a flag ever appears on a bill or an account status; it creates only a private internal task. The team also runs the holdback comparison, leaving one flagged customer in ten uncalled. After three months the results arrive: among flagged customers who were called, 68% churned in the following month; among the held-back group — equally risky by the model's own scoring, but never contacted — 75% churned. The gap is 7 percentage points, and it rests on an experiment instead of a hope.

It is also far smaller than the 30% save rate assumed in planning: 7 points out of the 75% who would have left means the calls keep about 9% of true churners. Darya reworks the net value with the measured figure: $800 \times 0.07 = 56$ customers kept, worth $56 \times \$90 = \$5,040$ against \$4,000 of calls, for about \$1,040 a month instead of \$12,200. The campaign still pays for itself, but only just, and Ronahi now argues with a real number instead of an assumption. The Basra gap gets a real fix too: with six more months of histories Darya retrains and recall there rises from 20% to 26% — closer to the rest of the network, still not equal, and recorded as an open item rather than a solved problem.

RESPONSIBLE USE

Both models here suit one modest job each: giving an agent a starting range to open a conversation, and ranking customers so a limited team can spend its time where it likely matters most. Neither can decide anyone's fate automatically, explain with certainty why one home or person is what it is, or prove that an action changed an outcome. A model earns continued use only by staying honest about where it works, and by being watched, tested and retrained — never launched once and forgotten.

CHAPTER SUMMARY

- A real project starts with one plain decision sentence naming the unit of observation, the target and the prediction time, and it names the action the prediction supports before naming any algorithm.

- Auditing raw data catches duplicates, gaps and impossible values; auditing candidate columns against the prediction time catches data leakage and, sometimes, circularity.
- Split by time when the target drifts, and split by person when several rows describe the same person; both ask whether the test set is genuinely new information.
- A baseline costs almost nothing and sets the floor every real model must clear: \$33,900 for Hawler Homes, 90% accuracy with 0% recall for Zagros Mobile.
- Improvement over the baseline, $(MAE_{base} - MAE_{model})/MAE_{base}$, is what makes a score mean something; here ridge alone captured 63.7 of gradient boosting's 67.8 percentage points, and a gap smaller than the wobble across folds is not a result at all.
- The test set is opened once, after every setting is locked, and a small rise from validation to test is a sign of an honest process, not a failure.
- Slice the test score: the price model's error ran from 8.4% of price in its best band to 11.6% in its worst, and the churn model's recall fell from about 29% to 20% in one region. Permutation importance, which tests the finished model on data it never trained on, says which features carry that score — though it never proves a cause.
- A probability is not yet a decision: calibrate it, then choose a threshold from the team's real capacity and the campaign's real net value, $V = Sv - Fc$.
- A finished model is not a finished project: watch for drift, recompute the live score as outcomes arrive, keep a tested pipeline ready for rollback, and hold back a random slice of flagged cases so you can tell whether acting changed anything.
- Present the work on one page: decision sentence, data, split, baseline, comparison, one test number, slices, limits and a monitoring plan.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Explain in your own words why Hawler Homes' tool must never be described as a "legal appraisal".

Problem 2 [Basic] Name the unit of observation, the target and the prediction time for the Hawler Homes project, and then for the Zagros Mobile project.

Problem 3 [Basic] A dataset holds 3,600 home sales spread evenly over 36 months. A team holds out the most recent 4 months for testing and the 4 months before that for validation. How many months, and how many sales, are left for training?

Problem 4 [Basic] Define churn in your own words, and give one example of a churn event that is not a formal cancellation.

Problem 5 [Standard] A model's validation MAE is \$14,500 and the baseline's, on the same homes, is \$29,000. Find the model's improvement over the baseline as a percentage, showing every step.

Problem 6 [Standard] A test set of 200 homes splits into two bands. Band A has 140 homes with a MAE of \$8,000; band B has 60 homes with a MAE of \$22,000. Find the overall MAE across all 200 homes, showing every step.

Problem 7 [Standard] A model's validation MAE is \$10,000. After shuffling one feature's values it rises to \$15,500. Find that feature's permutation importance in dollars and as a percentage of the original MAE.

Problem 8 [Standard] A validation set holds 2,000 customer-months, of which 200 end in churn. Find the accuracy and the recall of an “always predict Stay” baseline, showing every step, and say in one sentence why the two numbers disagree so violently.

Problem 9 [Standard] A churn model flags 500 of the 10,000 customers scored this month, with a precision of 0.60 at that threshold, and past campaigns show a save rate of 25%. Find the expected number of customers saved. Then, with a value of \$80 per save and a cost of \$6 per contact, find the campaign’s net value and its net value per call.

Problem 10 [Standard] Explain, in your own words, why comparing contacted customers with customers who were never flagged cannot prove that a retention call changed anyone’s behavior, and describe what comparison would.

Problem 11 [Challenge] A colleague proposes using the agent’s original asking price as a feature, since it is always known before the home sells. (a) Explain why this would not count as data leakage under the definition in Chapter 2. (b) Explain the separate reason it should still be left out. (c) Describe one check that would show how much the model is really worth without it.

Problem 12 [Challenge] A team scores 15,000 customers and must choose between two thresholds. Threshold A flags 600 customers at a precision of 0.70; threshold B flags 1,000 at a precision of 0.50. Using a save rate of 30%, a value of \$100 per save and a cost of \$8 per contact, find the net value of each, showing every step. State which you would choose and what you would need to know about the team before being sure.

Problem 13 [Challenge] Choose one of the six projects in Table 25.7, or one of your own, and write its one-page report using the ten rows of Table 25.8. Rows 1 to 5 can be written now, before any data exists; write those in full, and for rows 6 to 10 write what you would do and what could go wrong.

PART SIX

REFERENCE

APPENDIX A

Mathematics When You Need It

The small amount of mathematics this book leans on, in one place

Introduction

This appendix is not a chapter. It is a shelf. Nothing here has to be read before Chapter 1, and most readers will never read it straight through. Instead, when a chapter uses a symbol you would like explained more slowly — a $\sum$ sign, a dot between two bold letters, a $\ln$, a ∂ — turn to the matching section, read two pages, work the example with a pencil, and go back to your chapter.

Every idea collected here has already done real work somewhere in the book, so nothing in it is new. Nine short sections cover everything the twenty-five chapters lean on, and every worked example uses numbers small enough to check on paper. Table A.1 shows which section belongs to which chapter.

Table A.1 – Which section of this appendix each chapter leans on

Section	What it explains	Chapters
A.1 Numbers, rounding and percentages	Significant figures, rounding, percent against percentage point	3, 4, 6, 7
A.2 Scalars, vectors and matrices	Notation, shape, transpose, matrix–vector product	3, 8, 16, 17, 20
A.3 Sums and the sigma notation	Reading a $\sum$, sums of products, index conventions	3, 8, 9, 20
A.4 Mean, variance, standard deviation	Averages and spread; divisor n against $n - 1$	3, 4, 20
A.5 The dot product	Multiply matching entries and add; alignment	8, 10, 16, 17, 20
A.6 Distance and norm	Euclidean and Manhattan distance, L1 and L2 norms	7, 8, 11, 16
A.7 Probability basics	Joint and conditional probability, Bayes, expected value	6, 10, 12
A.8 Slopes, derivatives and gradients	Slope, derivative, chain rule, partial derivative, gradient	9, 10, 16
A.9 Exponentials and logarithms	The number e , natural log, log rules, log scales	7, 10, 12

A.1 Numbers, Rounding and Percentages

A pharmacist in Sulaymaniyah checks a delivery. The label says 200 milliliters; her cylinder says 199.6. Is the delivery wrong? It depends on how many digits of that reading are trustworthy — a question about significant figures, the idea that stops every number in this book from looking more precise than it is.

WORDS TO KNOW

Significant figures — The digits of a number that actually carry information. Non-zero digits always count; zeros between non-zero digits count; zeros written only to place the decimal point, as in 0.0042, never count.

Decimal places — How many digits are written after the decimal point — not the same thing: 0.0042 has four decimal places but two significant figures.

So 0.85 has two significant figures, 105 has three (the zero sits between two non-zero digits), and 1.60 has three, because a final zero after the point was measured or nobody would have written it.

Rounding throws digits away on purpose. Look at the first digit you are about to drop: if it is 4 or less, leave the digit before it alone; if it is 5 or more, add one to it; then delete everything from there rightward. One habit matters more than the rule itself: *round once, at the very end*. Rounding at every step lets small errors pile up.

WORKED EXAMPLE A.1 — Report One Accuracy Three Different Ways

A classifier gets 1,753 of 2,100 test predictions right, an accuracy of 0.834762 in full. Report it to three decimal places, to two decimal places, and as a percentage to one decimal place. Then show what goes wrong if the rounding is done in stages.

1. Three decimal places: the first dropped digit is 7, so the 4 rounds up, giving 0.835.
2. Two decimal places, from the original number: the first dropped digit is 4, so nothing changes, giving 0.83.
3. As a percentage: $0.834762 \times 100 = 83.4762$, which rounds to 83.5%.
4. Rounding in stages instead: $0.834762 \rightarrow 0.8348 \rightarrow 0.835 \rightarrow 0.84$.
5. Compare the two two-decimal answers: rounding once gives 0.83, rounding in stages gives 0.84.

Answer: The accuracy is 0.835, 0.83 and 83.5%. Chained rounding gives 0.84, wrong by a whole hundredth — always round once, from the original number.

COMMON MISTAKE

X Wrong: Copying every digit Python prints and reporting an accuracy as “0.8347619048”.

✓ Right: Report only the digits the data can support. With about 2,000 test rows, two or three significant figures is honest; the rest is noise dressed as precision.

A percentage is a fraction written out of 100. Next comes the distinction that causes more confusion in machine learning reports than any other: one model scores 70% and another 75%, and that gap has two correct descriptions that are not equal.

WORDS TO KNOW

Percentage point — The unit for the plain difference between two percentages. From 70% to 75% is a rise of 5 percentage points.

Relative change — The difference measured against the starting value. From 70% to 75% is a relative rise of about 7.1%, because 5 is about 7.1% of 70.

EQUATION: PERCENTAGES, POINTS AND RELATIVE CHANGE

$$\text{percent} = \frac{\text{part}}{\text{whole}} \times 100, \quad \text{point change} = \text{new} - \text{old}, \quad \text{relative change} = \frac{\text{new} - \text{old}}{\text{old}} \times 100$$

In words: A percentage divides the part by the whole and multiplies by a hundred. A point change simply subtracts two percentages. A relative change divides that difference by the old value first, so it says how big the improvement was next to where you started.

When it is used: Whenever a fraction is reported, and whenever two model scores are compared (Chapter 6, Chapter 7). Say which of the two changes you mean, every time.

Symbol	Meaning	Unit or type
part, whole	the count of interest, and the total it came from	count
point change	the plain difference between two percentages	percentage points
relative change	the difference as a share of the old value	percent

The two changes can look wildly different. A rise from 1% to 2% is 1 percentage point but a relative change of 100%. Neither is wrong; only an unlabeled one is.

A ratio is a third way of comparing counts: write them side by side separated by colons and divide out any common factor, so 150, 30 and 20 become 15 : 3 : 2. Ratios keep whole numbers where percentages would need decimals, which is why Chapter 10 writes odds as a ratio of two probabilities and Chapter 22 builds ratio features.

WORKED EXAMPLE A.2 — Compare Two Models, and Read a Sample's City Mix

(a) Model A scores 70% accuracy and Model B scores 75%. Describe the improvement both honest ways. (b) The rental website of Chapter 3 exports 200 listings: 150 from Erbil, 30 from Sulaymaniyah and 20 from Duhok, while the real regional market is about 50% Erbil. Find each city's share, the size of the Erbil bias, and the ratio of the counts.

1. (a) Point change: $75 - 70 = 5$, so Model B is 5 percentage points better.
2. Relative change: $5 \div 70 = 0.0714$, and $\times 100$ gives about 7.1% better than Model A.
3. (b) Erbil: $150 \div 200 = 0.75 = 75\%$. Sulaymaniyah: $30 \div 200 = 15\%$. Duhok: $20 \div 200 = 10\%$, and the three shares add to 100.
4. Erbil bias: the sample has 75% where the market has 50%, so $75 - 50 = 25$ percentage points of over-representation.
5. Ratio: divide 150, 30 and 20 by 10 to get 15 : 3 : 2.

Answer: (a) Model B is 5 percentage points better, or about 7.1% better in relative terms — two correct descriptions of one gap. (b) The sample is 75%, 15% and 10%, over-representing Erbil by 25 percentage points, in the ratio 15 : 3 : 2.

A.2 Scalars, Vectors and Matrices

Numbers in this book come arranged in three shapes: one number alone, a short ordered list describing one example, or a whole rectangular table of examples. Each has a name, and every chapter has been using all three since Chapter 1.

WORDS TO KNOW

Scalar — A single number on its own. A melon's weight, 4.2 kilograms, is a scalar; so is a model's accuracy, 0.85.

Vector — An ordered list of numbers, usually one example with one number per feature in a fixed agreed order. The melon (4, 8) is a vector: weight 4 kilograms, then sound score 8.

Matrix — A rectangular table of numbers in rows and columns, usually a whole dataset: one row per example, one column per feature.

A vector with two entries can also be drawn as an arrow from (0, 0) to the point its numbers name, as Figure A.1 does; both pictures are used later, in Sections A.5 and A.6.

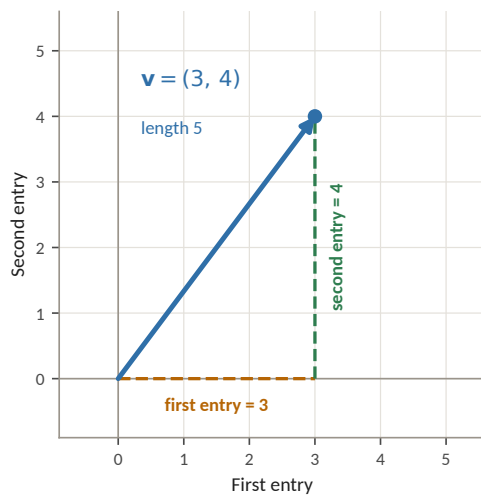


Figure A.1 – The vector (3, 4) drawn as an arrow from the origin: its first entry is a step across, its second a step up, and its length is 5.

WHAT TO OBSERVE

The two dashed lines are the vector's entries, 3 across and 4 up. The arrow's length, 5, is a third number computed from the first two — the norm of Section A.6.

This book follows one fixed set of symbols, and formulas become far easier to read once each symbol's shape is known at a glance. A plain italic letter is one number: n a count, b an intercept, y_i one example's target. A bold lowercase letter is a vector: $\mathbf{x}$ one example's features, $\mathbf{w}$ the model's coefficients, $\mathbf{y}$ the whole column of targets. A bold capital is a table: $\mathbf{X}$, the feature matrix. And two index letters run through everything — i counts examples from 1 to n , j counts features from 1 to k .

The size of a matrix is its *shape*, written rows first: 4 examples with 2 features make a 4×2 matrix. A wrong shape is the most common bug in beginner code, which is why Chapter 3 checks the shapes of $\mathbf{X}$ and $\mathbf{y}$ before anything else. Turning a matrix on its side is its *transpose*, $\mathbf{X}^T$, and Chapter 20 uses exactly that move to build a covariance matrix.

Take Chapter 1's four melons, (4, 8), (6, 10), (2, 2) and (2, 4), stacked one per row. Four melons and two features make $\mathbf{X}$ a 4×2 matrix; counting down, row 3 is (2, 2), so the entry in row 3 column 2 is 2, the first unripe melon's sound score. Turning the columns into rows gives $\mathbf{X}^T = \begin{pmatrix} 4 & 6 & 2 & 2 \\ 8 & 10 & 2 & 4 \end{pmatrix}$, of shape 2×4 . Note that this appendix counts from 1, as mathematics does, while Python counts from 0, so that same entry is written `X[2, 1]` in code.

One operation on a matrix is genuinely needed in this book: multiplying a matrix by a vector. It sounds forbidding and is not — it applies the same short recipe to every row at once.

EQUATION: THE MATRIX-VECTOR PRODUCT

$$(\mathbf{X}\mathbf{w})_i = \sum_{j=1}^k X_{ij} w_j = X_{i1}w_1 + X_{i2}w_2 + \cdots + X_{ik}w_k$$

In words: To get entry i of the answer, walk along row i of the matrix and along the vector together, multiplying each matching pair and adding the products. Repeat for every row.

When it is used: To score a whole dataset in one line: every prediction of Chapter 8, every score of Chapter 10 and every projection of Chapter 20 is a matrix-vector product.

Symbol	Meaning	Unit or type
$\mathbf{X}, \mathbf{w}$	feature matrix of shape $n \times k$; coefficient vector of k entries	table; list
$\mathbf{X}\mathbf{w}$	the answer, one entry per example	vector of n entries

The shapes must agree: the matrix's column count must equal the vector's entry count. Each entry of the answer is a dot product (Section A.5) of one row with $\mathbf{w}$.

WORKED EXAMPLE A.3 — Score Three Homes at Once with a Matrix-Vector Product

Chapter 8 fitted $\hat{y} = 5 + 0.4 \times \text{size} + 9 \times \text{appliances}$. Three homes measure (120, 4), (120, 5) and (80, 2). Stack them into $\mathbf{X}$, compute $\mathbf{X}\mathbf{w}$ with $\mathbf{w} = (0.4, 9)$, and add the intercept.

1. Check the shapes: $\mathbf{X}$ is 3×2 and $\mathbf{w}$ has 2 entries, so the answer will have 3 entries.
2. Row 1: $0.4 \times 120 = 48$ and $9 \times 4 = 36$; add, $48 + 36 = 84$.
3. Row 2: 48 and $9 \times 5 = 45$; add, $48 + 45 = 93$. Row 3: $0.4 \times 80 = 32$ and $9 \times 2 = 18$; add, $32 + 18 = 50$.
4. Collect: $\mathbf{X}\mathbf{w} = (84, 93, 50)$. Add the intercept 5 to each: 89, 98 and 55.

Answer: The three predicted bills are 89, 98 and 55 dollars; the first two match Worked Example 8.7 exactly. One matrix-vector product replaced three separate hand calculations, which is why software writes models this way.

Two more words appear once each in this book, in Chapter 20, and both describe a matrix's private directions. Most directions get turned when a matrix acts on them, the way a mirror turns most arrows; a few do not turn at all.

WORDS TO KNOW

Eigenvector — A direction that a particular matrix does not turn. Multiplying the matrix by that vector gives back the same direction, only stretched or shrunk.

Eigenvalue — The stretch factor belonging to one eigenvector. A large eigenvalue marks an important direction.

That is the whole idea behind Chapter 20: the principal components of a dataset are the eigenvectors of its covariance matrix, and the variance along each one is that eigenvector's eigenvalue. Software finds them with a routine no exercise here needs; the words are simply worth recognizing.

A.3 Sums and the Sigma Notation

The capital Greek letter sigma, $\sum$, is shorthand for “add all of these up”. It looks intimidating and carries almost no information: it compresses a long chain of plus signs, and it always has exactly three parts to read.

WORD TO KNOW

Sigma notation — A short way of writing a long addition. Below the $\sum$ sits the index and its starting value; above it sits the stopping value; to its right sits the thing to be added, once for each value of the index.

EQUATION: THE SUMMATION SIGN

$$\sum_{i=1}^n x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

In words: Start the counter i at 1 and write down x_i . Add one to i and write down the next one. Keep going until i reaches n , then add everything you wrote.

When it is used: Everywhere a list is totaled: the mean and variance of Section A.4, the losses of Chapter 8 and Chapter 9, the projections of Chapter 20.

Symbol	Meaning	Unit or type
i	the index, or counter, starting at the value written below the sign	count
n	where the counter stops	count

The letter is arbitrary — $\sum_i x_i$ and $\sum_t x_t$ mean the same thing — but this book always uses i for examples and j for features.

Three small rules make most formulas easier to read: adding one constant n times gives $\sum_{i=1}^n c = nc$; a constant multiplier pulls outside, $\sum c x_i = c \sum x_i$, which is why the $\frac{1}{n}$ of a mean may sit in front of its sum; and two sums add term by term, $\sum (x_i + y_i) = \sum x_i + \sum y_i$.

WORKED EXAMPLE A.4 — Read Two Sums Written Over the Same Five Numbers

The five apartment sizes of Chapter 3 are 50, 70, 110, 120 and 150 square meters. Evaluate $\sum_{i=1}^5 x_i$, then $\frac{1}{5} \sum_{i=1}^5 x_i$, then $\sum_{i=1}^5 x_i^2$, and compare the last with $(\sum_{i=1}^5 x_i)^2$.

1. Read the first sum: start at $i = 1$, stop at $i = 5$: $50 + 70 + 110 + 120 + 150 = 500$.
2. The second is the same sum with $\frac{1}{5}$ outside: $500 \div 5 = 100$ square meters.
3. For the third, square each size first, then add: $2,500 + 4,900 + 12,100 + 14,400 + 22,500 = 56,400$.
4. For the comparison, add first and square afterwards: $500^2 = 250,000$.

Answer: The sum is 500, the mean 100 m², the sum of squares 56,400 and the square of the sum 250,000. Do exactly what is written to the right of the $\sum$, once per example, and only then add: $\sum x_i^2$ squares first, $(\sum x_i)^2$ adds first, and the two are nothing like each other.

A sum whose terms are products of two matching lists appears again and again: in Chapter 3's correlation coefficient, Chapter 9's gradient and the dot product of Section A.5. It is read the same way — one term per index value, then add.

WORKED EXAMPLE A.5 — Evaluate a Sum of Products

Chapter 9 trains a model on four homes with 1, 2, 3 and 4 air conditioners whose daily costs are \$3, \$5, \$7 and \$9. At the starting guess $w = 0$, $b = 0$ every prediction is 0, so each residual r_i equals the true cost. Evaluate $\sum_{i=1}^4 r_i$ and $\sum_{i=1}^4 x_i r_i$.

1. Write the residuals: $r_1 = 3$, $r_2 = 5$, $r_3 = 7$, $r_4 = 9$.
2. First sum, plain residuals: $3 + 5 + 7 + 9 = 24$.

3. Second sum, one product per example: $1 \times 3 = 3$, $2 \times 5 = 10$, $3 \times 7 = 21$, $4 \times 9 = 36$.
4. Add the four products: $3 + 10 + 21 + 36 = 70$.

Answer: $\sum r_i = 24$ and $\sum x_i r_i = 70$. These are the two numbers Chapter 9 feeds into its gradient, and Section A.8 shows where the surrounding formula comes from.

A.4 Mean, Variance and Standard Deviation

Three numbers summarize a column: where its middle sits, and how widely its values scatter around that middle. Chapter 3 computes all three for apartment sizes, Chapter 4 uses them to standardize features, and Chapter 20 uses variance as the very thing it maximizes.

The mean adds every value and divides by how many there are. Variance averages each value's squared distance from the mean, and the standard deviation takes the square root to return to the data's own unit. Squaring stops positive and negative distances from cancelling, and punishes a far-away value more heavily than a near one — exactly as least squares does in Chapter 8. But the book uses two slightly different divisors here, and that difference confuses many readers, so it is worth stating plainly: dividing by n describes the numbers you actually have, while dividing by $n - 1$ estimates the spread of the larger group those numbers came from.

WORDS TO KNOW

Population variance — The average squared distance from the mean, dividing by n . Use it when the numbers in front of you are the whole group you care about.

Sample variance — The same calculation dividing by $n - 1$. Use it when the numbers are a sample and you want to estimate the spread of the wider population behind them.

EQUATION: MEAN, VARIANCE AND STANDARD DEVIATION

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

In words: The mean adds every value and divides by the count. For either variance, subtract the mean from every value, square each difference and add the squares; then divide by n for the population form or $n - 1$ for the sample form. The square root of a variance is its standard deviation.

When it is used: The sample form s^2 describes a column in Chapter 3. The population form σ^2 is used inside standardization in Chapter 4 and inside Chapter 20, matching what scikit-learn does there.

Symbol	Meaning	Unit or type
$\bar{x}$	the mean, said “x bar”	same unit as the data
σ^2, σ	population variance and standard deviation	data unit squared; data unit
s^2, s	sample variance and standard deviation	data unit squared; data unit
n	the number of values	count

The mean is also the rule behind Chapter 1's centroid: average each feature column separately, never mixing two columns together.

Why does the sample form divide by the smaller number? Because the mean was computed from the very same values, so each value sits a little closer to its own list's mean than it would to the true mean of the wider

population, and the squared distances come out slightly too small. Dividing by $n - 1$ corrects that bias. When n is 1,000 the two barely differ; when n is 5 they differ noticeably.

WORKED EXAMPLE A.6 — Find the Mean and Both Variances of Five Temperatures

A weather station in Sulaymaniyah records five daily high temperatures in degrees Celsius: 30, 32, 28, 35 and 30. Find the mean, then the variance and standard deviation with each divisor.

1. Add the five values: $30 + 32 + 28 + 35 + 30 = 155$, and divide by 5: the mean is 31°C .
2. Subtract the mean from each value: $-1, 1, -3, 4, -1$.
3. Square each difference: 1, 1, 9, 16, 1. Add them: $1 + 1 + 9 + 16 + 1 = 28$.
4. Population form, divide by $n = 5$: $\sigma^2 = 5.6$, and $\sigma = \sqrt{5.6} \approx 2.366$.
5. Sample form, divide by $n - 1 = 4$: $s^2 = 7$, and $s = \sqrt{7} \approx 2.646$.

Answer: The mean is 31°C ; the population standard deviation is about 2.366°C and the sample standard deviation about 2.646°C . Both describe the same five days but answer different questions. NumPy's `np.std` gives the first by default; pandas' `.std()` gives the second.

The two forms always differ by the same fixed factor, $\sqrt{n/(n-1)}$, and nothing else. Chapter 3's five apartment sizes — 50, 70, 110, 120 and 150 m^2 , mean 100 — have squared differences adding to 6,400, so $\sigma = \sqrt{6400 \div 5} \approx 35.78\text{ m}^2$ while $s = \sqrt{6400 \div 4} = 40\text{ m}^2$, and $40 \div 35.78 \approx 1.118 = \sqrt{1.25}$. Because that factor is identical for every column, ratios of variances are unaffected — which is why Chapter 20 can divide by n and print 50 where scikit-learn divides by $n - 1$ and prints 62.5, the explained-variance ratios coming out the same either way.

A.5 The Dot Product

Chapter 8 built a prediction by multiplying each feature by its own coefficient and adding the products. That “multiply matching entries, then add” pattern turns up in so many corners of machine learning that it has its own name and its own symbol: a single raised dot.

WORD TO KNOW

Dot product — A way of combining two vectors of the same length into one single number: multiply each pair of matching entries, then add all the products. Written $\mathbf{w} \cdot \mathbf{x}$ and said “w dot x”.

It has a second, geometric meaning too. Draw both vectors as arrows from the origin: the dot product measures how well they line up, through their lengths and the angle between them. Only one fact about cos is needed — close to +1 for a small angle, exactly 0 at a right angle, negative for a wide one, reaching -1 when the arrows point in opposite directions.

EQUATION: THE DOT PRODUCT AND WHAT IT MEASURES

$$\mathbf{w} \cdot \mathbf{x} = \sum_{j=1}^k w_j x_j = w_1 x_1 + \cdots + w_k x_k, \quad \mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

In words: Pair the first entry with the first, the second with the second, multiply each pair and add every product into one number. That same number equals the two arrows' lengths multiplied together, times an alignment factor between -1 and $+1$.

When it is used: Every linear score in the book: $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ in Chapter 8, $z = \mathbf{w} \cdot \mathbf{x} + b$ in Chapter 10, $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ in Chapter 16, the projections of Chapter 20 and the kernels of Chapter 17.

Symbol	Meaning	Unit or type
$\mathbf{w} \cdot \mathbf{x}$	the dot product: one single number, never a vector	product of the two units
$\ \mathbf{a}\ , \theta$	a vector's length (Section A.6); the angle between two	unit of the vector; degrees

Both vectors must have the same number of entries, since each entry needs a partner. Rearranged, $\cos \theta = (\mathbf{a} \cdot \mathbf{b}) \div (\|\mathbf{a}\| \|\mathbf{b}\|)$ strips out both lengths and leaves pure direction; under the name cosine similarity that is a standard way of comparing two documents.

Row 1 of Worked Example A.3 was already a dot product without being called one: with $\mathbf{w} = (0.4, 9)$ and $\mathbf{x} = (120, 4)$, multiplying the matching pairs gives $0.4 \times 120 = 48$ and $9 \times 4 = 36$, and adding them gives $\mathbf{w} \cdot \mathbf{x} = 84$, one intercept short of Chapter 8's \$89. Every linear prediction in this book is one dot product plus one intercept, however many features there are.

Figure A.2 pairs one vector with three partners and shows what the sign of the answer records.

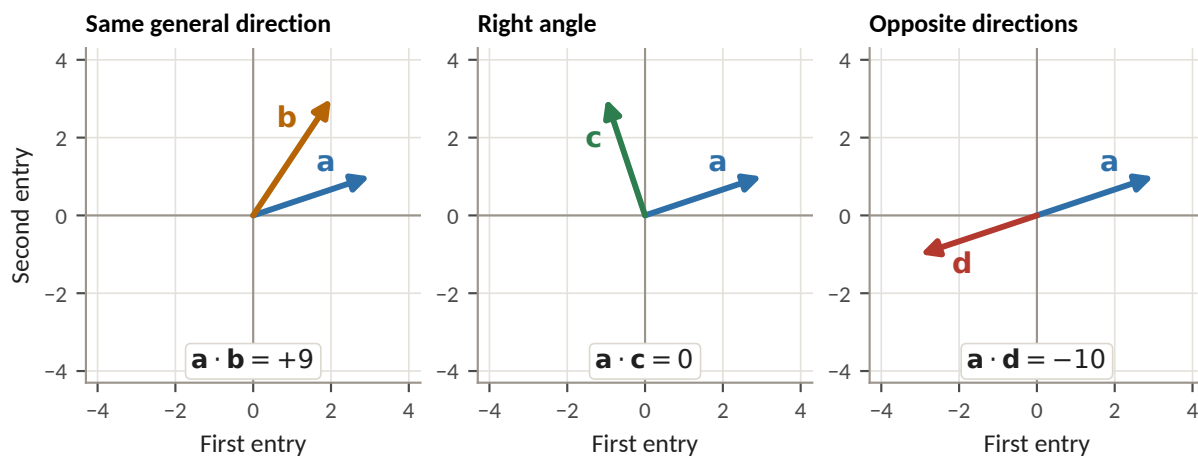


Figure A.2 – The same vector $\mathbf{a} = (3, 1)$ paired with three partners. The sign of the dot product records whether the arrows point the same way, at a right angle, or against each other.

WHAT TO OBSERVE

Identical arithmetic gives three different signs. Positive means the arrows share a general direction; exactly zero means a right angle, the situation Chapter 20 calls *orthogonal*; negative means they pull against each other. That sign is what a classifier reads when it decides which side of a boundary a point falls on.

WORKED EXAMPLE A.7 – Read the Sign of Three Dot Products, and One Angle

Let $\mathbf{a} = (3, 1)$. Compute $\mathbf{a} \cdot \mathbf{b}$ for $\mathbf{b} = (2, 3)$, $\mathbf{a} \cdot \mathbf{c}$ for $\mathbf{c} = (-1, 3)$ and $\mathbf{a} \cdot \mathbf{d}$ for $\mathbf{d} = (-3, -1)$. Then find $\cos \theta$ between $\mathbf{a}$ and $\mathbf{b}$.

1. $\mathbf{a} \cdot \mathbf{b} = 3 \times 2 + 1 \times 3 = 6 + 3 = 9$, positive.
2. $\mathbf{a} \cdot \mathbf{c} = 3 \times (-1) + 1 \times 3 = -3 + 3 = 0$, exactly zero: a right angle.
3. $\mathbf{a} \cdot \mathbf{d} = 3 \times (-3) + 1 \times (-1) = -9 - 1 = -10$, negative: opposite directions.
4. Lengths: $\|\mathbf{a}\| = \sqrt{9 + 1} = \sqrt{10} \approx 3.162$ and $\|\mathbf{b}\| = \sqrt{4 + 9} = \sqrt{13} \approx 3.606$.
5. Multiply the lengths, $3.162 \times 3.606 \approx 11.402$, then divide: $\cos \theta = 9 \div 11.402 \approx 0.789$.

Answer: The three dot products are +9, 0 and −10, matching the three panels of Figure A.2. The alignment factor between **a** and **b** is about 0.789 — comfortably positive but short of 1, so the arrows point broadly the same way without being parallel.

A.6 Distance and Norm

Chapter 1’s nearest-centroid rule and Chapter 11’s k-Nearest Neighbors both had to answer one question: how far apart are two examples? Both compared squared distances without ever taking a square root, since only the order mattered. This section takes that last step.

WORDS TO KNOW

L2 norm — The ordinary straight-line length of a vector: square every entry, add the squares, take the square root. Written $\|\mathbf{v}\|$ or $\|\mathbf{v}\|_2$.

L1 norm — The length you get by dropping every minus sign and adding, with no squaring and no square root. Written $\|\mathbf{v}\|_1$.

The distance between two points is the norm of their difference: subtract entry by entry, then measure the length of what is left. Subtracting in the other order changes nothing, since squaring and dropping signs both erase direction.

EQUATION: THE L2 AND L1 NORMS, AND THE MATCHING DISTANCES

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{j=1}^k v_j^2}, \quad \|\mathbf{v}\|_1 = \sum_{j=1}^k |v_j|, \quad d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|$$

In words: For the L2 norm, square every entry, add the squares and take the square root. For the L1 norm, drop every minus sign and add. For a distance, subtract the two points first and take the norm of what is left.

When it is used: L2 gives Euclidean distance (Chapter 11), the margin of Chapter 16 and the ridge penalty of Chapter 7. L1 gives Manhattan distance (Chapter 11) and the lasso penalty of Chapter 8.

Symbol	Meaning	Unit or type
$\ \mathbf{v}\ _2$	the L2, or Euclidean, norm	unit of the entries
$\ \mathbf{v}\ _1$	the L1, or Manhattan, norm	unit of the entries

Both belong to one family, the Minkowski distances; scikit-learn selects between them with $p=1$ and $p=2$. When features use different units, standardize first (Chapter 4), or the largest-numbered feature decides every distance by itself.

The two norms measure the same journey differently, as Figure A.3 shows: L2 flies straight, L1 walks the streets of a city grid — which is where its nickname comes from.

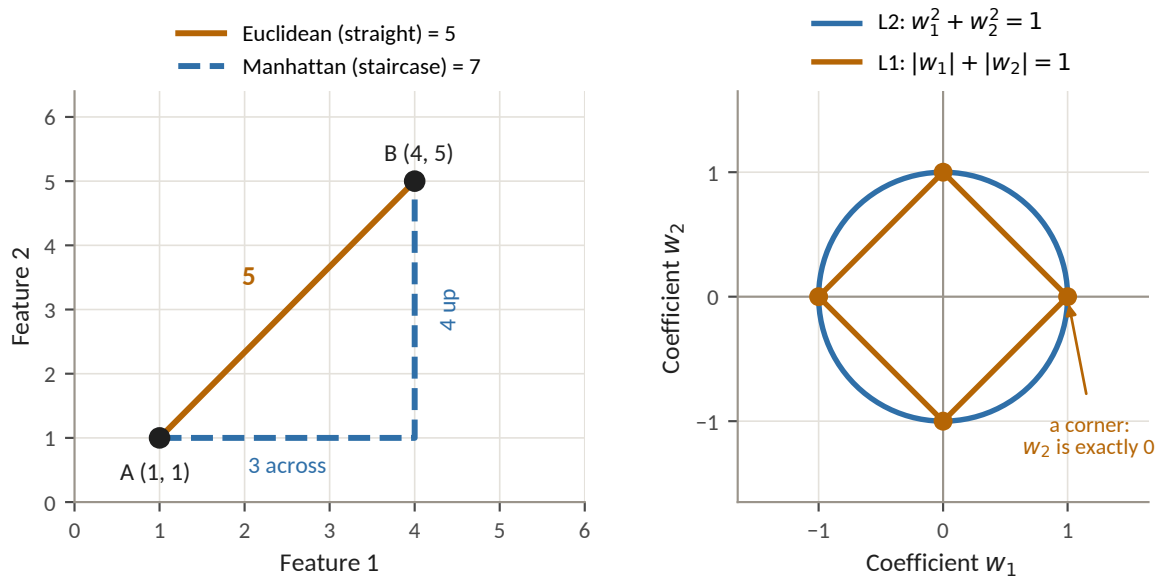


Figure A.3 – Left: the straight Euclidean route and the Manhattan staircase between the same two points. Right: every point one L2 unit from the origin forms a circle, while every point one L1 unit away forms a diamond whose corners sit on the axes.

WHAT TO OBSERVE

On the left, two routes join the same two points and give different answers, 5 and 7; neither is wrong, they measure different journeys. On the right, the L1 diamond has sharp corners exactly where one coefficient is zero, while the L2 circle is smooth everywhere. That difference in shape is why the lasso pushes coefficients to exactly zero and ridge only shrinks them toward it.

WORKED EXAMPLE A.8 — Find a Norm and Two Distances

(a) Find the L2 norm of the vector (3, 4) drawn in Figure A.1. (b) Two examples sit at $A = (1, 1)$ and $B = (4, 5)$. Find the Euclidean and the Manhattan distance between them.

1. (a) Square each entry, $3^2 = 9$ and $4^2 = 16$; add, $9 + 16 = 25$; take the root, $\sqrt{25} = 5$.
2. (b) Subtract entry by entry: $4 - 1 = 3$ and $5 - 1 = 4$.
3. Euclidean: square and add, $9 + 16 = 25$, then $\sqrt{25} = 5$.
4. Manhattan: drop the signs and add, $3 + 4 = 7$.

Answer: $\|(3, 4)\|_2 = 5$, the Euclidean distance from A to B is 5 and the Manhattan distance is 7. The straight route is always the shorter of the two, and they agree only when the points differ in a single feature.

The choice between the norms really bites in regularization. Chapter 7 adds a penalty to the training loss to stop coefficients growing wildly: ridge penalizes the squared L2 norm, $\lambda\|\mathbf{w}\|_2^2$, and the lasso the L1 norm, $\lambda\|\mathbf{w}\|_1$. Squaring makes ridge's penalty smooth, so it shrinks every coefficient a little; the L1 penalty has corners, and a corner is exactly a place where one coefficient is zero, so the lasso will switch a feature off completely.

WORKED EXAMPLE A.9 — Compute a Ridge Penalty and a Lasso Penalty

A two-feature model has coefficients $\mathbf{w} = (0.5, 6)$, the pair used in Chapter 8. With $\lambda = 0.1$, find the ridge penalty $\lambda \|\mathbf{w}\|_2^2$ and the lasso penalty $\lambda \|\mathbf{w}\|_1$.

1. Squared L2 norm: $0.5^2 + 6^2 = 0.25 + 36 = 36.25$.
2. Ridge penalty: $0.1 \times 36.25 = 3.625$.
3. L1 norm: $|0.5| + |6| = 6.5$, so the lasso penalty is $0.1 \times 6.5 = 0.65$.

Answer: The ridge penalty is 3.625 and the lasso penalty 0.65. Squaring makes ridge react far more strongly to the one large coefficient, 6, which is why ridge suits problems where many features matter a little and lasso suits problems where only a few matter at all.

A.7 Probability Basics

Many chapters have already produced a number between 0 and 1 and called it a probability: Chapter 10's sigmoid output, Chapter 12's posterior, every rate from Chapter 6's confusion matrix. This section defines that number and states the rules the book needs.

WORD TO KNOW

Probability — A number between 0 and 1 measuring how likely an event is. Zero means the event never happens, 1 means it always happens, and 0.5 means about half the time. Multiplying by 100 turns it into a percentage.

The most honest way to estimate one from data is to count how often the event actually happened. Two events can then be looked at together in two ways: the *joint* probability asks how often both happen out of everybody, while the *conditional* probability narrows the crowd first — given that one already happened, how often does the other?

EQUATION: PROBABILITY FROM COUNTS, JOINT AND CONDITIONAL

$$P(A) = \frac{\text{count of } A}{\text{total count}}, \quad P(\text{not } A) = 1 - P(A), \quad P(A | B) = \frac{P(A \text{ and } B)}{P(B)}$$

A and B are independent when $P(A \text{ and } B) = P(A) P(B)$

In words: Divide the count of the event by the number of chances it had; the probability it did not happen is one minus that. A conditional probability divides the joint probability by the probability of the condition, changing the question from “out of everybody” to “out of this group”. Two events are independent when knowing one tells you nothing about the other.

When it is used: To set the prior and likelihoods of Chapter 12, whose “naive” assumption is exactly the independence rule, and to read Chapter 6's precision, a conditional probability wearing a different hat.

Symbol	Meaning	Unit or type
$P(A)$	the estimated probability of event A	between 0 and 1
$P(A \text{ and } B)$	probability that both happen	between 0 and 1
$P(A B)$	probability of A given that B happened	between 0 and 1

$P(A | B)$ and $P(B | A)$ are different numbers built from the same cases, because they divide by different groups. Confusing them is the most expensive mistake in this section.

WORKED EXAMPLE A.10 — Read Probabilities from One Clinic's Records

A clinic records 200 visits. 60 patients had flu, 80 had a cough and 45 had both. Find $P(\text{flu})$, $P(\text{cough})$, the joint probability, both conditional probabilities, and check whether flu and cough are independent.

1. Counts to probabilities: $P(\text{flu}) = 60 \div 200 = 0.30$ and $P(\text{cough}) = 80 \div 200 = 0.40$.
2. Joint: $P(\text{flu and cough}) = 45 \div 200 = 0.225$.
3. For $P(\text{flu} \mid \text{cough})$, keep only the 80 coughing patients; 45 had flu, so $45 \div 80 = 0.5625$.
4. For $P(\text{cough} \mid \text{flu})$, keep only the 60 flu patients; 45 coughed, so $45 \div 60 = 0.75$.
5. Independence check: if they were independent the joint would be $0.30 \times 0.40 = 0.12$, far from the counted 0.225.

Answer: $P(\text{flu} \mid \text{cough}) = 0.5625$ and $P(\text{cough} \mid \text{flu}) = 0.75$ — the same 45 patients, two answers, because the questions divide by different groups. And $0.225 \neq 0.12$, so flu and cough are not independent, which is exactly why a cough is useful evidence for flu.

Bayes' theorem turns a conditional probability around: it takes what you believed before the evidence arrived, adjusts it by how strongly the evidence points one way, and returns an updated belief. Chapter 12 builds a whole classifier from it.

EQUATION: BAYES' THEOREM

$$P(C \mid x) = \frac{P(x \mid C) P(C)}{P(x)}, \quad P(x) = P(x \mid C)P(C) + P(x \mid \text{not } C)P(\text{not } C)$$

In words: The updated belief equals the likelihood of the evidence inside the class, times the prior belief in that class, divided by how likely the evidence was overall.

When it is used: Whenever evidence must update a belief: the naive Bayes classifier of Chapter 12, and any time a test result has to be read against a rare condition.

Symbol	Meaning	Unit or type
$P(C \mid x)$	posterior: the class after seeing the evidence	between 0 and 1
$P(x \mid C), P(C)$	likelihood of the evidence inside the class; prior belief before it	between 0 and 1
$P(x)$	evidence: how likely the observation was overall	between 0 and 1

The bottom line is the same for every class, so when you only need to know which class wins you may compare the top halves alone and skip it.

WORKED EXAMPLE A.11 — Update a Belief After a Positive Test

A condition affects 1 person in 100. A test catches 95% of those who have it and wrongly flags 5% of those who do not. A customer tests positive. Find the probability that the customer really has the condition.

1. Write the prior: $P(\text{condition}) = 0.01$, so $P(\text{no condition}) = 0.99$.
2. Top half for "has it": likelihood times prior, $0.95 \times 0.01 = 0.0095$.
3. The same product for "does not have it": $0.05 \times 0.99 = 0.0495$.
4. Add the two for the evidence: $0.0095 + 0.0495 = 0.059$.
5. Divide: $0.0095 \div 0.059 \approx 0.161$.

Answer: About 0.161, or roughly 16%. A positive result raises the belief sixteen-fold, from 1% to about 16%, and still leaves it far below certainty — because the 5% of false alarms is drawn from a very much larger group of healthy people.

RESPONSIBLE USE

A probability describes a group, never one individual. Saying that 12% of similar applicants do not repay is not the same as saying a particular person will not repay. When a probability is attached to a person — a patient, a borrower, a student — it should start a careful human conversation, not end one. Chapter 24 returns to this at length.

The last idea here connects a probability to a decision. If each outcome carries a cost or a value, the *expected value* $E = \sum_m P(\text{outcome } m) \times \text{value of outcome } m$ weighs each outcome by how likely it is and adds the products up. It answers the question a manager actually asks — on average, what will this cost? — and it is how Chapter 6 and Chapter 21 choose a threshold when the two kinds of mistake cost different amounts. An expected value need not be a value any single case can take: an expected loss of \$6 can come from a 2% chance of losing \$300 and a 98% chance of losing nothing.

WORKED EXAMPLE A.12 — Decide Whether a Manual Check Is Worth Its Cost

An online store estimates that 2% of the orders its model flags are fraudulent. Shipping a fraudulent order costs \$300; a staff member can check a flagged order for \$5. Should flagged orders be checked?

1. Outcomes of shipping unchecked: fraud with probability 0.02 costing \$300, or no fraud with probability 0.98 costing \$0.
2. Expected cost of shipping: $0.02 \times 300 + 0.98 \times 0 = 6$ dollars.
3. Cost of checking: \$5 with certainty, so an expected cost of exactly 5 dollars.
4. Compare: $6 > 5$, so checking is cheaper by \$1 per flagged order; over 500 flagged orders that is $1 \times 500 = 500$ dollars.

Answer: Check them: \$6 expected cost against \$5, saving about \$1 per order and \$500 over 500 orders. Notice how the decision would flip if a check cost \$8, or if the fraud rate fell to 1% — expected value is what makes that trade-off visible.

A.8 Functions, Slopes, Derivatives and Gradients

Chapter 9 described gradient descent as walking downhill in the dark, feeling the ground's slope at each step. This section names that slope and shows exactly where Chapter 9's two formulas come from.

WORDS TO KNOW

Function — A rule that turns each input into exactly one output, written $y = f(x)$. A trained model is a function: features go in, a prediction comes out.

Derivative — The slope of a function at one exact point: how much the output changes for a tiny change in the input, measured right there. Written $f'(x)$.

A straight line has one slope that serves everywhere: pick any two points, divide the rise by the run. A curve does not. Chapter 9's loss curves like a valley — steep on the walls, nearly flat at the bottom — so we must speak of its slope *at one place*. Imagine laying a ruler so it just touches the curve at your point without cutting through; the slope of that ruler is the derivative there, and Figure A.4 puts both pictures side by side.

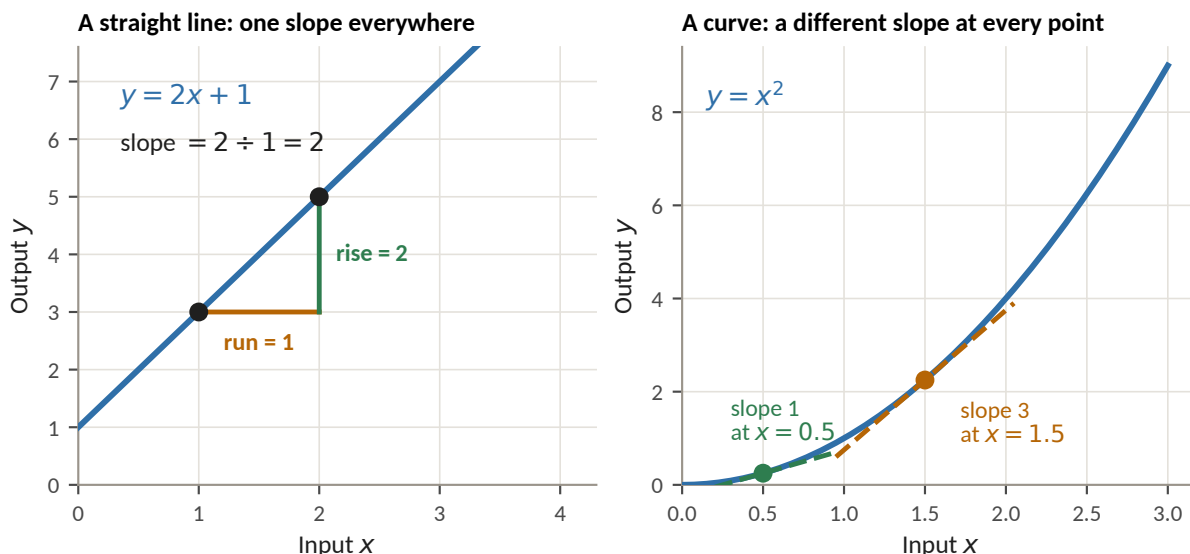


Figure A.4 – Left: a straight line, where rise over run gives the same slope at every point. Right: the curve $y = x^2$, whose touching line is gentle at $x = 0.5$ and three times steeper at $x = 1.5$.

WHAT TO OBSERVE

The left panel needs one number for the whole line, and for a fitted model that number is the coefficient w of Chapter 8. The right panel needs a new number at every point — slope 1 at $x = 0.5$, slope 3 at $x = 1.5$ — which is why gradient descent must recompute its gradient at every step.

EQUATION: SLOPE OF A LINE, AND THE DERIVATIVE AT A POINT

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{y_2 - y_1}{x_2 - x_1}, \quad f'(x) \approx \frac{f(x + h) - f(x)}{h} \text{ for a very small } h$$

In words: For a line, pick two points and divide the change in output by the change in input. For a curve, take a tiny step h to the right, see how much the output moved, and divide; as the step shrinks, that ratio settles on the exact slope at the point.

When it is used: The line form reads any linear model's coefficient; the derivative form measures how steeply a loss is falling at the model's current settings — the one measurement Chapter 9 needs at every iteration.

Symbol	Meaning	Unit or type
slope, $f'(x)$	rise per unit of run; the slope at the point x	output unit per input unit
h	a small step in the input	unit of the input

Three rules cover everything this book needs: the derivative of a constant is 0, of cx is c , and of x^2 is $2x$. Squared losses are built from exactly these.

WORKED EXAMPLE A.13 — Find the Slope of a Line and of a Curve

(a) Chapter 9 fits $\hat{y} = 2x + 1$ to four homes. Find its slope from two points. (b) For $f(x) = x^2$, estimate the derivative at $x = 1.5$ with a step of $h = 0.1$, then $h = 0.01$, and compare with the rule $f'(x) = 2x$.

- (a) At $x = 1$ the line gives $2 \times 1 + 1 = 3$; at $x = 2$ it gives $2 \times 2 + 1 = 5$.

2. Rise $5 - 3 = 2$ dollars, run $2 - 1 = 1$ air conditioner, so the slope is $2 \div 1 = 2$ — exactly the coefficient $w = 2$ Chapter 9 searches for.
3. (b) With $h = 0.1$: $f(1.6) = 2.56$ and $f(1.5) = 2.25$, so $(2.56 - 2.25) \div 0.1 = 3.1$.
4. With $h = 0.01$: $f(1.51) = 2.2801$, so $(2.2801 - 2.25) \div 0.01 = 3.01$.
5. The exact rule gives $f'(1.5) = 2 \times 1.5 = 3$.

Answer: The line's slope is 2 everywhere. The curve's estimates, 3.1 and 3.01, close in on the exact answer 3 as the step shrinks — which is what “the slope at a point” means.

Real losses are functions built out of other functions: the prediction depends on w , the residual depends on the prediction, and the loss depends on the residual. One rule handles the whole chain.

The rule is the *chain rule*, and it fits on one line: if $y = f(g(x))$ then $dy/dx = f'(g(x)) \times g'(x)$ — outside slope times inside slope. For a squared loss the outside is r^2 , whose slope is $2r$, and the inside is r , whose slope with respect to w is $-x$. That single line is where the gradients of Chapter 9 and Chapter 10 come from.

WORD TO KNOW

Partial derivative — The slope of a function when one input is changed and every other input is held perfectly still. Written $\partial L / \partial w$ and read “the partial derivative of L with respect to w ”.

WORKED EXAMPLE A.14 — Derive the Mean Squared Error Gradient with the Chain Rule

The mean squared error of a one-feature line is $L = \frac{1}{n} \sum_{i=1}^n r_i^2$, where $r_i = y_i - (wx_i + b)$. Find $\partial L / \partial w$ and $\partial L / \partial b$, then evaluate both for Chapter 9's four homes at $w = 0$, $b = 0$.

1. Outside function: each term is r_i^2 , whose derivative with respect to r_i is $2r_i$.
2. Inside function for w : since $r_i = y_i - wx_i - b$, raising w by one unit lowers r_i by x_i , so $\partial r_i / \partial w = -x_i$.
3. Chain rule: multiply them, $2r_i \times (-x_i) = -2x_i r_i$; averaging over n examples gives $\partial L / \partial w = -\frac{2}{n} \sum x_i r_i$.
4. Inside function for b : raising b by one unit lowers r_i by 1, so $2r_i \times (-1) = -2r_i$ and $\partial L / \partial b = -\frac{2}{n} \sum r_i$.
5. Put in the numbers from Worked Example A.5: $\sum x_i r_i = 70$, $\sum r_i = 24$ and $n = 4$.
6. Evaluate: $\partial L / \partial w = -\frac{2}{4} \times 70 = -35$ and $\partial L / \partial b = -\frac{2}{4} \times 24 = -12$.

Answer: The two partial derivatives are -35 and -12 , exactly the gradient Chapter 9 computes by recipe in its Worked Example 9.2. The chain rule is where that recipe — and both of its minus signs — comes from.

Collect one partial derivative for every adjustable number in a model and you have the gradient.

EQUATION: THE GRADIENT

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_k}, \frac{\partial L}{\partial b} \right), \quad \text{new } w = w - \alpha \frac{\partial L}{\partial w}$$

In words: The gradient is the list of all the partial derivatives, one per adjustable number. Gradient descent subtracts a small multiple of each one, moving every parameter in the direction that lowers the loss.

When it is used: Once per iteration of training in Chapter 9, and inside every model scikit-learn fits by gradient descent, including the logistic regression of Chapter 10 and the linear support vector machines of Chapter 16.

Symbol	Meaning	Unit or type
∇L	the gradient of the loss, said “grad L”	one slope per parameter
$\partial L / \partial w$	the slope in the w direction alone	loss unit per unit of w
α	the learning rate: how big a step to take	no unit

The minus sign in the update is the whole trick. The gradient points uphill, so subtracting it walks downhill — which is where a smaller loss lives.

Why does the gradient point uphill? Each entry answers one question about one parameter: if this number grew a little and nothing else moved, would the loss rise, and how fast? Moving along the gradient raises every entry’s contribution at once, so among all directions you could step in, that combination raises the loss fastest. Reverse the sign and you get the direction of fastest decrease, as Figure A.5 shows on Chapter 9’s own loss curve.

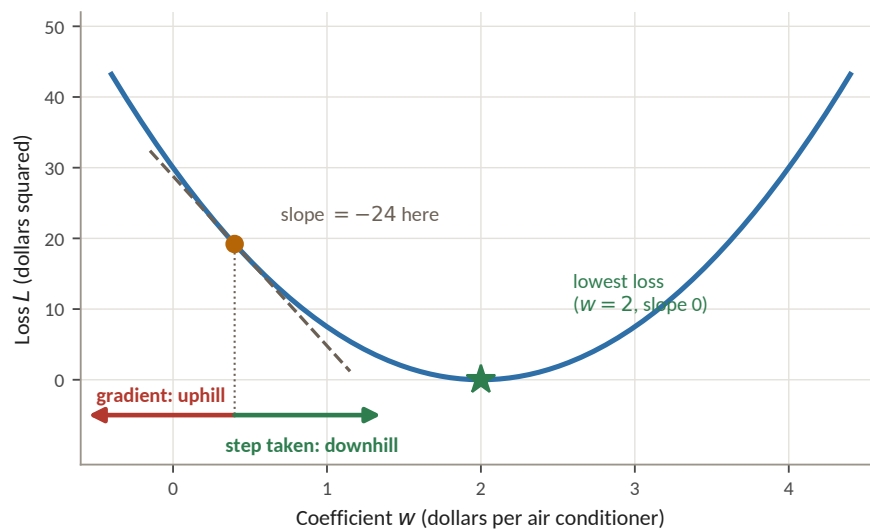


Figure A.5 – The loss of Chapter 9’s four homes as the coefficient w varies. At $w = 0.4$ the slope is -24 , so uphill lies to the left and gradient descent steps to the right.

WHAT TO OBSERVE

The slope at $w = 0.4$ is negative, saying the loss falls as w grows, so the uphill direction points left and the step actually taken points right. At the bottom of the valley the slope is 0, the gradient vanishes, and the update leaves w where it is — which is how gradient descent knows it has arrived.

WORKED EXAMPLE A.15 — Take One Step of Gradient Descent by Hand

On the loss curve of Figure A.5 the current coefficient is $w = 0.4$, where the loss is 19.2 and the slope is -24 . With learning rate $\alpha = 0.02$, find the new w and the new loss.

1. Write the update rule: new $w = w - \alpha \times \text{slope}$.
2. Multiply first: $0.02 \times (-24) = -0.48$.
3. Subtracting a negative adds it: $0.4 + 0.48 = 0.88$.
4. Read the new loss from the curve $L = 7.5(2 - w)^2$: $7.5 \times (2 - 0.88)^2 = 7.5 \times 1.2544 \approx 9.41$.

Answer: One step moves w from 0.4 to 0.88 and cuts the loss from 19.2 to about 9.41. The step went right, opposite the uphill gradient, and its size came from the slope and the learning rate together.

COMMON MISTAKE

✗ Wrong: Adding the gradient to the parameters, reasoning that a bigger number must be better.
✓ Right: Subtract. The gradient points toward higher loss; training wants lower loss. If the loss climbs after every iteration, a lost minus sign is the first thing to check and an oversized learning rate the second (Chapter 9).

A.9 Exponentials and Logarithms

Chapter 10's sigmoid used a number called e and its log loss used the natural logarithm, neither of which that chapter stopped to explain. This closing section fills both gaps with whole numbers you can check on paper.

WORDS TO KNOW

The number e — A fixed number, about 2.71828, that appears in every formula describing smooth growth. Use it exactly as you use π for the area of a circle, without deriving it.

Logarithm — The power a fixed base must be raised to in order to reach a given number — so a logarithm undoes an exponent, the way subtraction undoes addition. Base 10 is written log; base e is the natural logarithm, written ln.

EQUATION: THE EXPONENTIAL AND THE NATURAL LOGARITHM UNDO EACH OTHER

$$\ln(e^x) = x, \quad e^{\ln(x)} = x \text{ (for } x > 0), \quad \ln(1) = 0, \quad \ln(e) = 1$$

In words: Raising e to a power and taking a natural logarithm are opposite moves: doing both, in either order, returns the number you started with.

When it is used: To read the sigmoid of Chapter 10, which raises e to a negative score, and its log loss, which takes the natural logarithm of a probability. Chapter 12 uses the same pair to add log-probabilities instead of multiplying probabilities.

Symbol	Meaning	Unit or type
e^x	e raised to the power x	always positive
$\ln(x)$	the natural logarithm of x	any number; defined only for $x > 0$

Two facts do most of the work here: $e^0 = 1$, which is why the sigmoid of a score of 0 is exactly 0.5, and $\ln(x)$ plunges without limit as x nears 0, which is why a confident wrong prediction is punished so heavily.

Figure A.6 draws both curves; they are mirror images, the exponential climbing slowly then exploding, the logarithm rising quickly then flattening almost to a standstill.

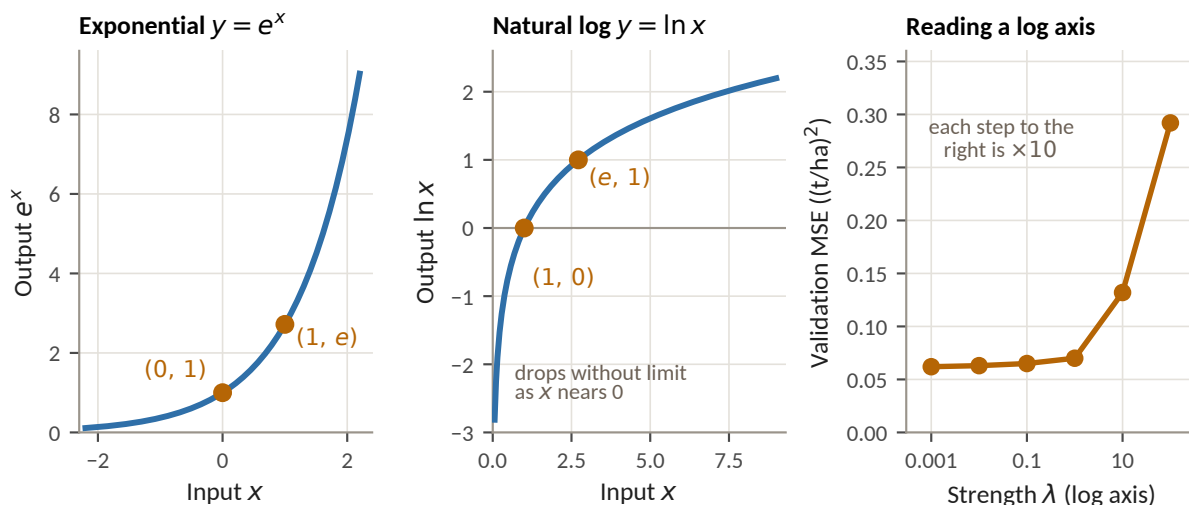


Figure A.6 – Left: $y = e^x$, always positive and growing faster and faster. Middle: $y = \ln x$, defined only for positive x and crossing zero at $x = 1$. Right: Chapter 7’s regularization dial on a logarithmic axis, where every equal step multiplies λ by ten.

WHAT TO OBSERVE

The exponential never touches zero however far left you go, which is why a sigmoid probability is never exactly 0 or exactly 1. The logarithm mirrors this, dropping without limit as x nears 0. On the right, six values spanning a factor of one hundred thousand fit comfortably on one axis — the entire reason log axes exist.

EQUATION: THE THREE LOGARITHM RULES

$$\ln(ab) = \ln a + \ln b, \quad \ln\left(\frac{a}{b}\right) = \ln a - \ln b, \quad \ln(a^p) = p \ln a$$

In words: A logarithm turns multiplication into addition, division into subtraction, and a power into a plain multiplication. The same three rules hold for base-10 logarithms.

When it is used: To turn a long product of probabilities into a sum of log-probabilities — exactly what Chapter 12 does to classify a message, and what lets Chapter 10’s log loss add over examples instead of multiplying.

Symbol	Meaning	Unit or type
a, b	any two positive numbers	no unit
p	any power	no unit

The first rule is why logarithms were invented in the seventeenth century, and it is still why machine learning code uses them: adding is safe where multiplying overflows or vanishes.

WORKED EXAMPLE A.16 — Turn a Product into a Sum with Logarithms

(a) Find $\log(1000)$ by reversing a power of ten, and confirm that $\log(100 \times 10) = \log(100) + \log(10)$. (b) A naive Bayes classifier multiplies twenty probabilities, each 0.1. Compare multiplying them with adding their natural logarithms.

1. (a) Ask the reversing question: 10 raised to what power gives 1000? Since $10^3 = 1000$, $\log(1000) = 3$.

2. Multiply first: $100 \times 10 = 1000$, whose logarithm is 3. Add the logs instead: $2 + 1 = 3$. The routes agree.
3. (b) Multiplying: 0.1 to the power 20 is 10^{-20} , so small that many programs round it to plain 0.
4. Taking logs instead: $\ln(0.1) \approx -2.3026$, and twenty of them add to $20 \times (-2.3026) = -46.05$.
5. Compare: -46.05 is an ordinary number a computer stores exactly, while 10^{-20} sits at the edge of what it can hold.

Answer: $\log(1000) = 3$ either way, and twenty probabilities of 0.1 become the harmless sum -46.05 instead of the fragile product 10^{-20} . Comparing sums of logs ranks the classes in exactly the same order as comparing products, because the logarithm never reorders positive numbers.

Chapter 10's log loss uses the same function to punish confident mistakes: the loss for one example is $-\ln(p)$, where p is the probability the model gave the answer that turned out to be true. Because $\ln$ dives toward minus infinity as p nears 0, the minus sign turns that dive into a very large penalty.

WORKED EXAMPLE A.17 — Compute Three Log Losses

Using Chapter 10's numbers, find the log loss when the model gave the true answer a probability of 0.881, then 0.269, then 0.01.

1. Confident and right: $-\ln(0.881) = -(-0.127) = 0.127$.
2. Unsure and leaning the wrong way: $-\ln(0.269) = -(-1.313) = 1.313$.
3. Confident and badly wrong: $-\ln(0.01) = -(-4.605) = 4.605$.
4. Compare: from 0.127 to 1.313 to 4.605, the penalty grows far faster than the probability shrinks.

Answer: The three losses are 0.127, 1.313 and 4.605 — a confidently wrong model is punished about 36 times more heavily than a confidently right one. That is exactly the behavior Chapter 10 wants: honest uncertainty costs little, false confidence costs a great deal.

A logarithmic axis, like the right panel of Figure A.6, spaces numbers by their logarithms rather than their size, so equal steps along it mean equal *multiplications*. That is how Chapter 7 fits a regularization strength running from 0.001 to 100 onto one readable line, and how Chapter 23 lays out a search over learning rates.

One consequence catches everybody out. Halfway along the axis between the ticks 0.01 and 0.1 is not their ordinary average, 0.055. Take logarithms — $\log(0.01) = -2$ and $\log(0.1) = -1$ — and halfway between those is -1.5 , so the middle value is $10^{-1.5} \approx 0.0316$. The middle of a log axis is the *geometric* middle, $\sqrt{0.01 \times 0.1}$, which is exactly why searching a learning rate or a regularization strength means trying 0.001, 0.01, 0.1, 1 rather than 0.2, 0.4, 0.6, 0.8.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Round 0.4567 to two decimal places, and state how many significant figures 0.00305 has.

Problem 2 [Basic] A model gets 36 of 48 test predictions right. Write its accuracy as a fraction, as a decimal and as a percentage.

Problem 3 [Basic] A matrix has 6 rows and 3 columns. (a) State its shape. (b) State the shape of its transpose. (c) Could it be multiplied by a vector with 3 entries?

Problem 4 [Basic] Evaluate $\sum_{i=1}^4 x_i$ and then the mean, for the values $x = 4, 7, 9, 12$.

Problem 5 [Standard] Find the dot product of $(2, 5)$ and $(4, 1)$, and the L2 norm of $(6, 8)$, showing every step.

Problem 6 [Standard] Five service times in minutes are 3, 5, 8, 8 and 11. Find the mean, the sum of squared deviations, both variances (divisor n and divisor $n - 1$) and both standard deviations. State which one pandas would print.

Problem 7 [Standard] Points $P = (1, 2)$ and $Q = (6, 14)$. Find the Euclidean and the Manhattan distance between them, showing every step.

Problem 8 [Standard] Of 120 patients, 45 had a fever, and 18 of those with a fever also had the illness. Find $P(\text{illness} \mid \text{fever})$, and explain in one sentence why this is not the same as $P(\text{fever} \mid \text{illness})$.

Problem 9 [Challenge] A model's accuracy improves from 62% to 68%. (a) State the change in percentage points and the relative change, each to one decimal place. (b) A manager reads the relative figure as if it were the point figure. Explain in two sentences what they would wrongly conclude.

Problem 10 [Challenge] Using $L = \frac{1}{n} \sum r_i^2$ with $r_i = y_i - (wx_i + b)$ and Chapter 9's four homes ($x = 1, 2, 3, 4$ and $y = 3, 5, 7, 9$), compute both partial derivatives at $w = 1, b = 0$. (a) Show the residuals, the two sums and both derivatives. (b) Say which way each parameter should move, and why both numbers are smaller than at $w = 0, b = 0$.

APPENDIX B

Python, NumPy, pandas and scikit-learn

Everything you need to run the code in this book, and nothing else

Every chapter of this book ends with a PYTHON activity, and every one of those activities uses the same small set of tools. This appendix is the field guide for those tools. It is deliberately short. It does not teach you all of Python; it teaches the part of Python that appears in this book, and stops there.

You do not need to read this appendix from beginning to end. Install the software once using Section B.1, read Section B.2 and Section B.3 before Chapter 1, and then return to one short section whenever a chapter's code moves faster than you would like. Every code block below is complete and runnable exactly as printed, and every output box shows what really appeared on the screen when the code was run.

Two conventions run through the whole book, and it helps to name them once. A capital X always holds the feature table — one row per example, one column per feature. A lowercase y always holds the target: one value per row, and nothing else.

B.1 Installing Python and the Libraries

Python by itself can add and subtract, but it cannot fit a model. The machine learning work is done by five extra pieces of software that you install once and then import whenever you need them.

WORD TO KNOW

Library — A collection of ready-made code that someone else has written, tested and shared, which your own program can load and use. NumPy, pandas, matplotlib, scikit-learn and SciPy are libraries. A library is also called a *package*.

Table B.1 lists everything this book needs. Nothing else is required: there is no extra plotting tool, no database, no deep learning framework.

Table B.1 – The five libraries used in this book, and the versions the outputs were produced with

Package	What it does in this book	Version used
python	The language itself; anything 3.10 or newer works	3.11
numpy	Arrays of numbers, and fast arithmetic on all of them at once	2.4
pandas	Tables with column names, read from and written to CSV files	3.0
matplotlib	All plots: scatter plots, histograms, bar charts, lines	3.10
scikit-learn	Every model, split, metric, scaler and pipeline	1.8
scipy	A few statistics helpers that scikit-learn also depends on	1.17

There are two normal ways to install these. Choose one; do not mix them.

Route 1: Anaconda or Miniconda

Anaconda is a single download that brings Python and most of these libraries with it. Miniconda is the same tool with a much smaller download, and you add the libraries yourself. Either is a good choice for a classroom, because one installer sets up everything for every student in the same way.

After installing, open the Anaconda Prompt (Windows) or a terminal (macOS, Linux) and type these three lines, one at a time. They create a separate workspace called `mlbook`, switch into it, and install the five libraries plus Jupyter.

Terminal • conda

```
conda create -n mlbook python=3.11
conda activate mlbook
conda install numpy pandas matplotlib scikit-learn scipy jupyter
```

The word `mlbook` is just a name you chose; anything works. The first line builds the workspace, the second switches into it, and the third fills it. You type the first and third lines only once. You type `conda activate mlbook` every time you open a new terminal to work on this book.

Route 2: pip and a virtual environment

If Python is already installed on the computer, the built-in installer `pip` is enough. Check first that Python is there, then build a workspace of your own and install into it.

Terminal • pip

```
python3 --version
python3 -m venv mlbook
source mlbook/bin/activate      # Windows: mlbook\Scripts\activate
python3 -m pip install numpy pandas matplotlib scikit-learn scipy notebook
```

A *virtual environment* is a private folder holding one Python and its libraries. It keeps this book's software separate from anything else on the computer, so that upgrading a library for one project can never break another. As with conda, you create it once and activate it each time you start work.

COMMON MISTAKE

X Wrong: Typing `pip install scikit-learn`, seeing “Successfully installed”, and then getting `ModuleNotFoundError` in Python. This happens when the computer has more than one Python and `pip` belongs to a different one than the Python you are running.

✓ Right: Always install with `python3 -m pip install ...`, using the same `python3` you use to run your code. That form asks that exact Python to install into itself, so the library always lands where it will be found.

Checking that everything worked

Whichever route you took, run this program once. It prints the version of every piece of software, and it is also the first thing to run whenever something stops working.

Python • all libraries	
<pre>import sys import numpy, pandas, sklearn, matplotlib, scipy print("Python ", sys.version.split()[0]) print("numpy ", numpy.__version__) print("pandas ", pandas.__version__) print("scikit-learn", sklearn.__version__) print("matplotlib ", matplotlib.__version__) print("scipy ", scipy.__version__)</pre>	
Output	
<pre>Python 3.11.15 numpy 2.4.4 pandas 3.0.2 scikit-learn 1.8.0 matplotlib 3.10.9 scipy 1.17.1</pre>	

Note that the library is called `scikit-learn` when you install it but `sklearn` when you import it. That single difference confuses almost everyone once.

Your numbers will not match these exactly, and they do not need to. Anything from NumPy 1.24, pandas 2.0 and scikit-learn 1.3 onward will run every example in this book. Where a newer version prints something in a slightly different style, the text says so.

A notebook or a plain file?

There are two ways to run the code, and both are fine.

- A **plain Python file** is a text file whose name ends in `.py`. You write the whole program, save it, and run it from the terminal with `python3 my_script.py`. It runs from top to bottom and prints its output. This is the better choice for a program you want to keep, share or hand in.
- A **Jupyter notebook** is a page in a web browser, divided into cells. You type a few lines into one cell, press Shift+Enter, and see the result underneath immediately, with plots appearing on the page. Start one with `jupyter notebook` in the terminal. This is the better choice while you are exploring data and trying things out.

WORD TO KNOW

Jupyter notebook — A document that mixes code cells, their output and written notes on one page in a web browser. Cells can be run one at a time and in any order — which is convenient, but also means a notebook can show results that the code, run from top to bottom, would no longer produce.

One last, genuinely useful fact: after the installation finishes, none of this needs the internet. Every dataset in this book is a small CSV file (Appendix C), every library runs on your own computer, and no example sends data anywhere. A classroom with one afternoon of internet access can install everything and then work offline for the rest of the year.

B.2 The Small Amount of Python This Book Uses

This book uses seven pieces of Python: variables, numbers and strings, lists, indexing, for loops, if, and functions — plus `print` to see what happened. That is all. If you already know these, skip to Section B.3.

A *variable* is a name for a value, created with a single equals sign: `rooms = 3` stores 3 under the name `rooms`. A value can be a whole number (an *int*), a number with a decimal point (a *float*), or a piece of text in quotation marks (a *string*). A *list* holds several values in order inside square brackets: `sizes = [85, 60, 105, 75]`.

Positions in a list are counted from 0, not from 1. So `sizes[0]` is the first item, `sizes[2]` is the third, and `sizes[-1]` is the last one. This is the single most common source of confusion for beginners, and it is worth saying out loud every time: **Python counts from zero.**

A `for` loop repeats the same indented block once for each item in a list. An `if` statement runs its indented block only when a condition is true. A *function*, written with `def`, gives a name to a few lines so they can be used again with different inputs. In every case the indentation — four spaces — is what tells Python which lines belong inside the block. Python is strict about this, and Section B.10 returns to the error you get when the indentation is wrong.

Finally, an *f-string* is a string with the letter `f` in front of it, in which anything inside curly braces is replaced by its value: `f"{s} m2"` prints the value stored in `s`, then the letters `m2`. Adding `:.2f` inside the braces rounds to two decimal places. The program below uses all seven ideas at once.

Python • no libraries

```
city = "Duhok"           # a string: text inside quotation marks
rooms = 3                 # an int: a whole number
rent = 550.0              # a float: a number with a decimal point

sizes = [85, 60, 105, 75] # a list of four numbers
print(sizes[0], sizes[2], sizes[-1], len(sizes))

total = 0
for s in sizes:           # repeat once for each item
    total = total + s
print("total size:", total, "m2")

def price_per_m2(rent, size): # a function with two inputs
    return rent / size

for s in sizes:
    if s >= 80:
        p = price_per_m2(rent, s)
        print(f"{s} m2 in {city}: large, {p:.2f} thousand IQD per m2")
    else:
        print(f"{s} m2 in {city}: small")
```

Output

```
85 105 75 4
total size: 325 m2
85 m2 in Duhok: large, 6.47 thousand IQD per m2
60 m2 in Duhok: small
105 m2 in Duhok: large, 5.24 thousand IQD per m2
75 m2 in Duhok: small
```

Read the first output line against the code: `sizes[0]` gave 85 (the first size), `sizes[2]` gave 105 (the third), `sizes[-1]` gave 75 (the last), and `len(sizes)` gave 4 (how many there are). Everything after the `#` symbol on a line is a *comment*: Python ignores it completely, and it is there only for the human reader.

B.3 NumPy Arrays

A list is fine for four numbers. A dataset with four hundred numbers, on which you want to compute a mean, subtract that mean and square the result, needs something better.

WORD TO KNOW

Array — A grid of numbers, all of the same type, built by the NumPy library. An array has a *shape*: the number of rows and columns it holds. Arithmetic on an array is applied to every entry at once. Every feature matrix in this book is an array (or a table built on top of one).

Nearly every program in this book begins with the line `import numpy as np`, which loads NumPy and gives it the short nickname `np`. The difference between a list and an array shows up immediately in something as simple as multiplying by 2.

Python • NumPy

```
import numpy as np

plain = [2, 4, 6]
print(plain * 2)
print(np.array(plain) * 2)
```

Output

```
[2, 4, 6, 2, 4, 6]
[ 4  8 12]
```

Multiplying a list repeats it; multiplying an array multiplies every entry. That one difference is the whole reason NumPy exists, and it is why every list of numbers in this book becomes an array before any real arithmetic happens.

The next program is the one to keep. It builds a small feature table — four apartments, with size in square meters and number of rooms — and then does everything this book ever asks of an array.

Python • NumPy

```
import numpy as np

X = np.array([[85, 2], [60, 1], [105, 3], [75, 2]]) # size (m2), rooms
print("shape:", X.shape, " dtype:", X.dtype)
print("row 0:", X[0], " entry [2, 1]:", X[2, 1])
print("size column:", X[:, 0])
print("rows 1 and 2:")
print(X[1:3])
print("column means (axis=0):", X.mean(axis=0))
print("row sums (axis=1):", X.sum(axis=1))
print("size std:", X[:, 0].std().round(2))
print("sizes + 10:", X[:, 0] + 10)
print("centered:")
print(X - X.mean(axis=0))
```

Output

```
shape: (4, 2)    dtype: int64
row 0: [85  2]    entry [2, 1]: 3
size column: [ 85  60 105  75]
rows 1 and 2:
[[ 60   1]
 [105   3]]
column means (axis=0): [81.25  2. ]
row sums      (axis=1): [ 87  61 108  77]
size std: 16.35
sizes + 10: [ 95  70 115  85]
centered:
[[  3.75  0. ]
 [-21.25 -1. ]
 [ 23.75  1. ]
 [-6.25  0. ]]
```

Work through the output one line at a time. The shape (4, 2) means four rows and two columns, which is exactly “four apartments, two features”. Indexing uses [row, column] and counts from zero, so X[2, 1] is the third row, second column: 3 rooms. A colon means “all of them”, so X[:, 0] is the whole first column. A slice 1:3 means “from 1 up to but not including 3”, which is why it returns two rows, not three.

The two axis lines deserve special attention, because axis is the single most confusing word in NumPy. The rule is short: **axis=0 runs down the rows and gives you one answer per column; axis=1 runs across the columns and gives you one answer per row.** Column means [81.25, 2.] are the average size and the average number of rooms — one number per feature, which is what a scaler needs. Row sums [87, 61, 108, 77] add size to rooms within each apartment, which is meaningless here, and that is a useful reminder that the computer will happily compute a number you did not want.

COMMON MISTAKE

X Wrong: Writing X.mean() with no axis when you wanted the average of each feature. With no axis, NumPy averages every entry in the whole table together — here it would mix square meters and room counts into a single meaningless number, 41.625.

✓ Right: Say which direction you mean: X.mean(axis=0) for one value per column (per feature), X.mean(axis=1) for one value per row (per example). Check the length of the answer: with 2 features, axis=0 must give 2 numbers.

The last block, X - X.mean(axis=0), does something worth naming.

WORD TO KNOW

Broadcasting — NumPy’s rule for combining arrays of different shapes: the smaller one is repeated automatically to match the larger one, without you writing a loop. Subtracting a row of 2 column means from a table of shape (4, 2) subtracts them from every one of the four rows.

Broadcasting is why standardization is one line rather than a loop, and why X[:, 0] + 10 adds 10 to all four sizes at once. It works whenever the shapes match from the right-hand side, or one of them is 1. Table B.2 collects every NumPy operation this book uses.

Table B.2 – The NumPy operations used in this book, with results for the array *x* above

Code	What it does	Result
<code>np.array([[85, 2], ...])</code>	Build an array from lists	shape (4, 2)
<code>np.arange(1, 7)</code>	Whole numbers 1 up to (not including) 7	[1 2 3 4 5 6]
<code>X.shape</code>	(rows, columns)	(4, 2)
<code>X.reshape(2, 4)</code>	Rearrange the same values	needs rows × cols to match
<code>X[2, 1]</code>	One entry: row 2, column 1	3
<code>X[:, 0]</code>	A whole column	[85 60 105 75]
<code>X[1:3]</code>	Rows 1 and 2	shape (2, 2)
<code>X[X[:, 0] > 80]</code>	Keep rows meeting a condition	shape (2, 2)
<code>X + 10, X * 2, X.round(2)</code>	Arithmetic and rounding, on every entry	shape unchanged
<code>X.mean(axis=0)</code>	One mean per column	[81.25 2.]
<code>X.sum(axis=1)</code>	One sum per row	[87 61 108 77]
<code>X.std(), X.min(), X.max()</code>	Spread and extremes	one number each

B.4 pandas DataFrames

An array is a grid of numbers with no column names and one single type. Real data arrives as a CSV file with a header row, mixed text and numbers, and sometimes gaps. That is what pandas is for.

WORD TO KNOW

DataFrame — A table with named columns, built by the pandas library. Each column may hold a different type — numbers in one, text in another — and each has a name you can use instead of a position number. `pd.read_csv` turns a CSV file into a DataFrame. A single column on its own is called a *Series*.

The examples below use `datasets/ch03_apartments.csv`, the 40 rental listings from Chapter 3. The first thing to do with any new file, always, is look at it.

Python • pandas

```
import pandas as pd

d = pd.read_csv("datasets/ch03_apartments.csv")
print(d.shape)
print(d.head(3))
print(d.dtypes)
```

Output

```
(40, 7)
   id  size_m2  rooms  age_years  distance_km  city  rent
0   1      85      2         5           2.0  Erbil   550
1   2      60      1        12           4.5  Duhok   300
2   3     105      3         3           1.5  Erbil   700
id          int64
size_m2      int64
```

```
rooms          int64
age_years      int64
distance_km    float64
city           str
rent           int64
dtype: object
```

`d.shape` says 40 rows and 7 columns. `d.head(3)` prints the first three rows, and the plain numbers 0, 1, 2 down the left are the *index*, pandas' own row labels. `d.dtypes` reports what kind of value each column holds: `int64` for whole numbers, `float64` for decimals, and `str` for text. (pandas 2 and earlier print `object` instead of `str` for a text column; it means the same thing.)

The second program summarizes, selects, filters and groups.

Python • pandas

```
import pandas as pd

d = pd.read_csv("datasets/ch03_apartments.csv")
print(d[["size_m2", "rent"]].describe().round(1))
print(d["city"].value_counts())
print(d.isna().sum().sum(), "missing values in the whole table")

big = d[d["size_m2"] > 100]           # filtering: keep some rows
print(len(big), "apartments larger than 100 m2")
print(d.loc[0, "city"], d.loc[0, "rent"]) # one cell by label

print(d.groupby("city")["rent"].mean().round(0))
big.to_csv("large_apartments.csv", index=False)
```

Output

```
count    size_m2    rent
mean      98.8    485.0
std       39.2    199.9
min       50.0    220.0
25%       70.0    347.5
50%       90.0    420.0
75%      112.5    582.5
max      200.0   1150.0
city
Erbil          18
Sulaymaniyah   12
Duhok          10
Name: count, dtype: int64
0 missing values in the whole table
14 apartments larger than 100 m2
Erbil 550
city
Duhok          386.0
Erbil          583.0
Sulaymaniyah   420.0
Name: rent, dtype: float64
```

`describe()` is the fastest honest look at a numeric column: count, mean, standard deviation, smallest, largest, and the three quartiles of Chapter 3. Here it already tells a story — the largest apartment, 200 m², is more than twice the median of 90 m², and the largest rent is more than five times the smallest.

`value_counts()` does the same job for a text column, counting how many rows carry each category: 18 Erbil, 12 Sulaymaniyah, 10 Duhok. `isna().sum()` counts missing values in each column, and adding `.sum()` a second time totals them; the answer 0 means this file has no gaps. Run that line on every new file before anything else — Chapter 4 explains what to do when the answer is not 0.

Selecting is worth one careful sentence. `d["rent"]` with single brackets returns one column as a Series; `d[["size_m2", "rent"]]` with double brackets returns a smaller DataFrame with two columns. Filtering uses a condition inside square brackets: `d[d["size_m2"] > 100]` keeps only the rows where the condition is true. And `d.loc[0, "city"]` reads one cell by its row label and column name.

Finally, `groupby("city")["rent"].mean()` splits the 40 rows into three groups by city, takes the mean rent inside each, and returns one number per group — the single most useful line in pandas. `to_csv(..., index=False)` writes a table back out; `index=False` stops pandas from adding its own row numbers as an extra first column. Table B.3 collects the whole set.

Table B.3 – The pandas operations used in this book

Code	What it does
<code>pd.read_csv("file.csv")</code>	Read a CSV file into a DataFrame
<code>d.head(3), d.tail(3)</code>	Show the first or last few rows
<code>d.shape, d.columns</code>	(rows, columns), and the list of column names
<code>d.dtypes</code>	The type stored in each column
<code>d.describe()</code>	count, mean, std, min, quartiles, max for numeric columns
<code>d["rent"], d[["rooms", "rent"]]</code>	One column as a Series; double brackets give a DataFrame of several columns
<code>d.loc[0, "city"]</code>	One cell, by row label and column name
<code>d[d["rooms"] == 3]</code>	Keep only the rows where the condition is true
<code>d["city"].value_counts()</code>	How many rows carry each category
<code>d.isna().sum()</code>	How many values are missing in each column
<code>d.groupby("city")["rent"].mean()</code>	One summary number per group
<code>d.to_numpy()</code>	Convert to a plain NumPy array
<code>d.to_csv("out.csv", index=False)</code>	Write the table back to a CSV file

COMMON MISTAKE

X Wrong: Changing a filtered table and expecting the original to change: `d[d["size_m2"] > 100]["rent"] = 999`. Depending on the pandas version this prints a `SettingWithCopyWarning` or raises a `ChainedAssignmentError`, and in every version the original table `d` is left completely unchanged.

✓ Right: Do it in one step with `.loc`: `d.loc[d["size_m2"] > 100, "rent"] = 999`. If you only want the filtered rows as their own table, keep them under a new name and add `.copy()`: `big = d[d["size_m2"] > 100].copy()`.

B.5 The scikit-learn Pattern

Here is the most useful single fact in this appendix. Every model in this book — linear regression, logistic regression, k-nearest neighbors, naive Bayes, decision trees, random forests, gradient boosting, support vector

machines, k-means, PCA — is used through the same four lines of code: import it, fit it, use it, score it. Learn them once and every new chapter is only a new name on line one. (The unsupervised tools of Chapter 18 to Chapter 21 have no *y*, so their *fit* takes only *X* — but the shape of the code is unchanged.)

WORD TO KNOW

Estimator — Any scikit-learn object that learns something from data. Every estimator has a *fit* method. Models add *predict*; preprocessing steps such as a scaler add *transform*. Because they all share this interface, one can be swapped for another without changing the rest of the program.

Table B.4 is the whole interface. There is nothing else to learn.

Table B.4 – The estimator interface: every scikit-learn tool in this book uses these methods

Method	What it does	What comes back
<i>fit</i> (<i>X</i> , <i>y</i>)	Learn from the training data. A model needs <i>x</i> and <i>y</i> ; a scaler needs only <i>x</i>	The fitted estimator itself
<i>predict</i> (<i>X</i>)	Give one answer for each row of <i>X</i>	An array of length <i>n</i>
<i>predict_proba</i> (<i>X</i>)	Give the probability of each class, for classifiers that support it	An array $n \times (\text{classes})$
<i>transform</i> (<i>X</i>)	Apply an already-learned preprocessing step to new rows	A changed version of <i>x</i>
<i>fit_transform</i> (<i>X</i>)	<i>fit</i> then <i>transform</i> in one call — training data only	A changed version of <i>x</i>
<i>score</i> (<i>X</i> , <i>y</i>)	One default quality number (R^2 for regressors, accuracy for classifiers)	A single number

Before any of that, the data must be split, because a score measured on rows the model has already seen is not a score at all (Chapter 5). *train_test_split* does it in one line, and *random_state* makes the split repeatable.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

d = pd.read_csv("datasets/ch03_apartments.csv")
X = d[["size_m2", "rooms", "age_years", "distance_km"]]
y = d["rent"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)
print("train:", X_train.shape, " test:", X_test.shape)

model = LinearRegression()           # 1. choose the estimator
model.fit(X_train, y_train)          # 2. fit on the training set only
pred = model.predict(X_test)         # 3. predict for new rows
print("R2 on the test set:", round(model.score(X_test, y_test), 3))
print("first three predictions:", pred[:3].round(0))
```


Output

```
train: (30, 4)   test: (10, 4)
R2 on the test set: 0.81
first three predictions: [440. 821. 575.]
```

`train_test_split` returns four things in a fixed order — training features, test features, training target, test target — so the order of the four names on the left matters. With `test_size=0.25` and 40 rows, 30 rows go to training and 10 to testing, exactly as the printed shapes confirm. For a classification problem, add `stratify=y` so both parts keep the same mix of classes.

Now the promised claim. Add these lines to the end of the program above and swap only the name on line one; everything else is unchanged.

Python • scikit-learn

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

for model in [LinearRegression(),
               DecisionTreeRegressor(random_state=0),
               RandomForestRegressor(n_estimators=100, random_state=0)]:
    model.fit(X_train, y_train)                    # same line
    score = model.score(X_test, y_test)            # same line
    print(type(model).__name__, "R2:", round(score, 3))
```

Output

```
LinearRegression R2: 0.81
DecisionTreeRegressor R2: 0.899
RandomForestRegressor R2: 0.709
```

Three completely different algorithms, one `fit` line and one `score` line. (Do not read too much into which one wins on ten test apartments; Chapter 23 explains why a single small test set cannot settle that question.)

Classifiers add one method. `predict` returns the chosen class, while `predict_proba` returns how sure the model is — which is what Chapter 6 needs in order to move a threshold.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

v = pd.read_csv("datasets/ch05_clinic_visits.csv")
X = v[["hour", "clinicians"]]
y = v["long_wait"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

clf = LogisticRegression().fit(X_train, y_train)
print("classes:", clf.classes_)
print("predicted labels:", clf.predict(X_test)[:5])
print("predicted probabilities:")
print(clf.predict_proba(X_test)[:5].round(2))
```

```
print("accuracy:", round(clf.score(X_test, y_test), 3))
```

Output

```
classes: [0 1]
predicted labels: [1 1 0 0 0]
predicted probabilities:
[[0.29 0.71]
 [0.47 0.53]
 [0.53 0.47]
 [0.51 0.49]
 [0.55 0.45]]
accuracy: 0.733
```

Each row of `predict_proba` holds one probability per class, in the order given by `classes_`, and the two add to 1. The first test visit gets 0.71 for class 1, so `predict` chose 1. Notice also `LogisticRegression().fit(X_train, y_train)` on one line: because `fit` returns the fitted estimator itself, creating and fitting can be chained.

B.6 Pipelines and ColumnTransformer

Almost every model needs its features prepared first: numbers scaled, gaps filled, cities turned into columns of 0s and 1s. Doing that preparation in the wrong order is the most expensive mistake in this whole book, and it is completely invisible — the code runs, no warning appears, and the reported score is simply too good.

The problem is that a scaler *learns* something. `StandardScaler` learns a mean and a standard deviation. If it learns them from all 40 apartments, then the training rows have been adjusted using a mean that was partly computed from the test rows — and information has crossed a line it must never cross. Chapter 5 calls this **data leakage**. The two-line experiment below shows the leak concretely.

Python • scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

d = pd.read_csv("datasets/ch03_apartments.csv")
X = d[["size_m2"]]
X_train, X_test = train_test_split(X, test_size=0.25, random_state=0)

leaky = StandardScaler().fit(X)           # WRONG: sees all 40 apartments
safe = StandardScaler().fit(X_train)      # RIGHT: sees only the 30 training ones
print("mean used by the leaky scaler:", leaky.mean_.round(2))
print("mean used by the safe scaler: ", safe.mean_.round(2))
```

Output

```
mean used by the leaky scaler: [98.75]
mean used by the safe scaler:  [90.17]
```

The two means differ by more than 8 m², and the difference comes entirely from the ten test apartments. A model trained on the leaky version has been quietly told something about the rows it is about to be graded on. The cure is not to be careful; the cure is to make the safe order automatic.

WORDS TO KNOW

Pipeline — A chain of preprocessing steps ending in a model, treated as one estimator. Calling `fit` on the pipeline fits each step in order, on the training rows only; calling `predict` applies the learned steps to new rows and then predicts.

ColumnTransformer — A tool that sends different columns down different preprocessing paths — numeric columns to a scaler, text columns to a one-hot encoder — and joins the results back into one table.

Here is a complete, safe program for mixed numeric and categorical data. It is the template for every serious example in the book.

Python • pandas • scikit-learn

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_absolute_error

d = pd.read_csv("datasets/ch03_apartments.csv")
num = ["size_m2", "rooms", "age_years", "distance_km"]
cat = ["city"]
X_train, X_test, y_train, y_test = train_test_split(
    d[num + cat], d["rent"], test_size=0.25, random_state=0)

prep = ColumnTransformer([
    ("num", Pipeline([("fill", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
])
pipe = Pipeline([("prep", prep), ("model", Ridge(alpha=1.0))])
pipe.fit(X_train, y_train)           # fits every step on train only
pred = pipe.predict(X_test)         # applies every step to test

print("columns after preprocessing:",
      pipe.named_steps["prep"].get_feature_names_out().size)
print("test MAE:", round(mean_absolute_error(y_test, pred), 1),
      "thousand IQD")
cv = cross_val_score(pipe, d[num + cat], d["rent"], cv=5,
                     scoring="neg_mean_absolute_error")
print("5-fold MAE:", (-cv).round(1))
```

Output

```
columns after preprocessing: 7
test MAE: 69.5 thousand IQD
5-fold MAE: [66.5 48.6 53.2 51.4 30.7]
```

Read the structure rather than the details. Each step is a (name, tool) pair; inside a `ColumnTransformer` it is a (name, tool, columns) triple. The numeric columns go through two steps of their own — fill the gaps with the median, then standardize — and the one text column goes to `OneHotEncoder`, which turns

city into three 0/1 columns. Four numeric columns plus three city columns is the 7 the program prints. `handle_unknown="ignore"` tells the encoder to output all zeros rather than crash if a city it never saw during training appears later.

The payoff is the last three lines. `cross_val_score` refits the entire pipeline from scratch inside each of the five folds, so the scaler's mean is recomputed on each fold's own training rows. That is only safe because the scaler lives inside the pipeline. Hand `cross_val_score` a pre-scaled `X` instead, and every fold leaks.

COMMON MISTAKE

✗ Wrong: Calling `fit_transform` on the test set: `X_test_s = scaler.fit_transform(X_test)`. This throws away the mean learned from training and re-learns a new one from the test rows, so training and test data end up on two different scales.

✓ Right: `fit_transform` on training data, plain `transform` on everything else — or, better, put the scaler in a Pipeline and never call either one by hand.

B.7 Plotting with matplotlib

Every plot in this book is built from the same three pieces: a *figure* (the whole picture), one or more *axes* (a single panel with its own x and y), and marks drawn onto an axes. The command `fig, ax = plt.subplots()` creates both and hands them to you.

Four kinds of mark cover everything this book needs: `plot` for a line, `scatter` for points, `hist` for the shape of one column, and `bar` for a count per category. The program below draws all four from the apartments file, and Figure B.1 is exactly what it produces.

Python • pandas • matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt

d = pd.read_csv("datasets/ch03_apartments.csv")
fig, ax = plt.subplots(2, 2, figsize=(6.5, 4.6))    # 2 rows, 2 columns

by_rooms = d.groupby("rooms")["rent"].mean()
ax[0, 0].plot(by_rooms.index, by_rooms.values, marker="o")
ax[0, 0].set_xlabel("Rooms")
ax[0, 0].set_ylabel("Mean rent (thousand IQD)")

for city in ["Erbil", "Duhok", "Sulaymaniyah"]:
    part = d[d["city"] == city]
    ax[0, 1].scatter(part["size_m2"], part["rent"], label=city, s=18)
ax[0, 1].set_xlabel("Size (m2)")
ax[0, 1].set_ylabel("Rent (thousand IQD)")
ax[0, 1].legend(fontsize=7)

ax[1, 0].hist(d["rent"], bins=8, edgecolor="white")
ax[1, 0].set_xlabel("Rent (thousand IQD)")
ax[1, 0].set_ylabel("Number of apartments")

counts = d["city"].value_counts()
ax[1, 1].bar(counts.index, counts.values)
ax[1, 1].set_xlabel("City")
ax[1, 1].set_ylabel("Number of apartments")
```

```
fig.tight_layout()
fig.savefig("figures/appB_plots.pdf")
plt.show()
```

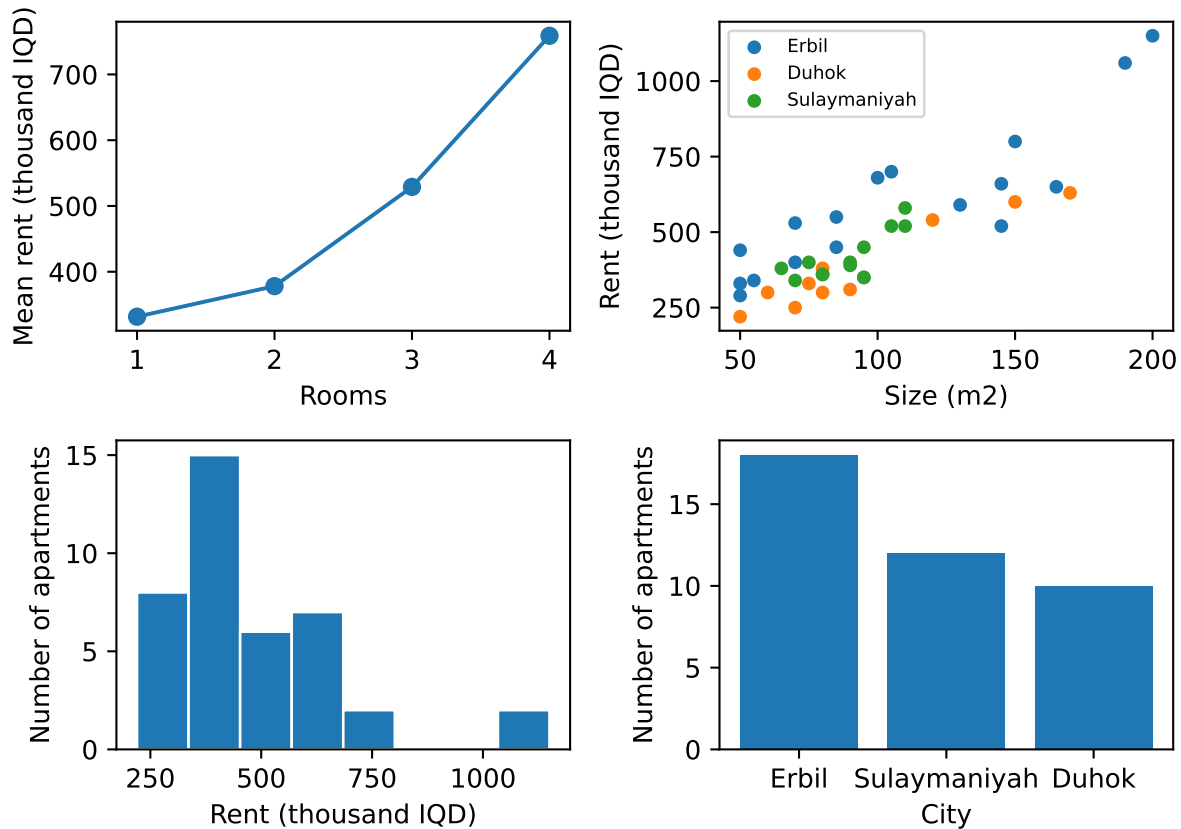


Figure B.1 – The four plot types used in this book, all drawn from `ch03_apartments.csv`: a line of mean rent against rooms, a scatter of rent against size, a histogram of rents, and a bar chart counting apartments per city.

WHAT TO OBSERVE

Every panel has both axis labels, with units, and the only panel with more than one series has a legend. Those two habits are the whole difference between a plot a reader can use and a plot they have to guess at. Notice too how much the four views differ: the scatter shows that rent rises with size, while the histogram shows that most rents sit between 300 and 700 thousand IQD with two much higher ones out on the right.

With `plt.subplots(2, 2)`, `ax` is a grid, so each panel is addressed as `ax[row, column]`. With a plain `plt.subplots()` there is one panel and you write `ax.scatter(...)` directly. `figsize` is given in inches. `fig.tight_layout()` spaces the panels so labels do not collide — run it after everything is drawn, and if a label still gets cut off, make the figure slightly larger. `fig.savefig("name.pdf")` writes the picture to a file (.png works too), and `plt.show()` opens it in a window; inside a Jupyter notebook the plot appears under the cell without `plt.show()` at all.

B.8 Reproducibility

Several methods in this book use randomness on purpose: `train_test_split` shuffles before dividing, random forests draw random samples and random features, k-means starts from random centers. Randomness is what makes those methods work. It also means that running the same program twice can give two different answers, which makes debugging and honest comparison much harder.

WORD TO KNOW

Random seed — A fixed starting number that makes a program's random choices come out the same way every time it runs. In scikit-learn it is passed as `random_state=0`; in NumPy it is given to `np.random.default_rng(0)`. The same seed always gives the same result; a different seed gives a different but equally valid one.

Python • NumPy • scikit-learn

```
import numpy as np
from sklearn.model_selection import train_test_split

rows = np.arange(10)
print("seed 42, run 1:", train_test_split(rows, test_size=0.3,
                                          random_state=42)[1])
print("seed 42, run 2:", train_test_split(rows, test_size=0.3,
                                          random_state=42)[1])
print("seed 7, run 1:", train_test_split(rows, test_size=0.3,
                                         random_state=7)[1])
print("no seed, run 1:", train_test_split(rows, test_size=0.3)[1])
print("no seed, run 2:", train_test_split(rows, test_size=0.3)[1])

rng = np.random.default_rng(0)          # NumPy's own seeded generator
print("rng draw 1:", rng.integers(0, 100, 4))
rng = np.random.default_rng(0)          # same seed, same numbers again
print("rng draw 2:", rng.integers(0, 100, 4))
```

Output

```
seed 42, run 1: [8 1 5]
seed 42, run 2: [8 1 5]
seed 7, run 1: [8 5 0]
no seed, run 1: [7 6 9]
no seed, run 2: [4 5 6]
rng draw 1: [85 63 51 26]
rng draw 2: [85 63 51 26]
```

The two seeded lines agree, and they agree again tomorrow and on any other computer. Seed 7 gives a different, equally legitimate split. The two unseeded lines disagree with each other in this run, and they will print yet other numbers the next time the program runs — try it. Imagine chasing a two-point drop in accuracy that was caused only by this.

Set the seed everywhere it is offered: in `train_test_split`, in every model that accepts `random_state`, in `KFold(shuffle=True, random_state=...)`, and in `np.random.default_rng(...)` whenever you generate data yourself. Use the same number throughout one project so that any difference you see really came from a change you made.

Two warnings. First, a fixed seed makes a result *repeatable*, not *reliable*. One seeded split can still be a lucky split; only cross-validation across several splits (Chapter 5) shows whether a score is stable. Second, reproduc-

ing a result a year later needs more than a seed: write down the library versions from Section B.1, the exact dataset file, the split rule and the list of features. Saving that version output next to your results costs one line and settles arguments that would otherwise take an afternoon.

B.9 A Safe Checklist for Every Code Example

Every code example in this book obeys the twelve rules in Table B.5. They are worth more than any single model. A careful beginner who follows all twelve will beat a careless expert who skips number 3, because rule 3 is the difference between a score that means something and a score that does not.

Table B.5 – The twelve checks behind every code example in this book

#	Check	Why
1	Look at the data first	head, shape, describe, value_counts take ten seconds and catch wrong units, stray zeros and impossible values
2	Say what one row is, and what the target is	If you cannot name them in one sentence, the model has no defined job
3	Split before anything else	Every gap filled, mean computed or column encoded before the split leaks test information into training
4	Fit only on the training set	fit_transform on train, transform on test; never the other way round
5	Put preprocessing in a Pipeline	It makes the safe order automatic instead of something you must remember
6	Check the shapes	print(X.shape, y.shape): the row counts must match, and the column count must be the number of features
7	Set a seed	random_state on the split and on every model, so a rerun is comparable
8	Compare against a baseline	Predicting the mean, or the most common class, is the score a real model has to beat
9	Choose the metric deliberately	Accuracy hides a rare class; report the metric that matches the real cost
10	Never tune on the test set	Use validation data or cross-validation for every choice; open the test set once, at the very end
11	Report in real units	“69.5 thousand IQD off on average” means something; “MAE 69.5” means less
12	Save the whole pipeline and its limits	The fitted preprocessing is part of the model; write down what it was trained on and where it should not be used

Notice what the list does not contain: any mention of which model to use. That is deliberate. The safe order protects a project whichever model ends up inside it, and a project that follows these twelve rules with a plain linear regression is in far better shape than one that ignores them with the most powerful method in the book.

B.10 Common Error Messages and What They Mean

An error message is not a punishment. It is the computer telling you, in a fixed format, exactly which line it could not run and why. Almost every error you meet while working through this book is one of the twelve in Table B.6; the first column quotes the part of the message that identifies it.

Table B.6 – Common error messages, their usual cause and the fix

Message	Usual cause	Fix
<code>ModuleNotFoundError: No module named 'sklearn'</code>	The library is not installed, or belongs to a different Python	<code>python3 -m pip install scikit-learn</code> , then rerun the version program
<code>FileNotFoundError: No such file or directory</code>	The path is wrong, or you are running from another folder	Check the spelling and run from the folder that holds datasets
<code>KeyError: 'Rent'</code>	A column name is misspelled or capitalized differently	<code>print(d.columns)</code> and copy it exactly: it is <code>rent</code> , not <code>Rent</code>
<code>ValueError: could not convert string to float</code>	A text column was handed to a model that takes only numbers	Encode it first with <code>OneHotEncoder</code> in a <code>ColumnTransformer</code>
<code>ValueError: inconsistent numbers of samples</code>	<code>X</code> and <code>y</code> have different numbers of rows	<code>print(X.shape, y.shape)</code> and find where rows were dropped
<code>ValueError: shapes (3,2) and (3,2) not aligned</code>	Two arrays were multiplied whose inner sizes do not match	The columns of the first must equal the rows of the second; transpose with <code>.T</code>
<code>ValueError: Expected 2D array, got 1D array</code>	<code>X</code> was passed as a flat list of numbers	Use double brackets, <code>d[["size_m2"]]</code> , or <code>X.reshape(-1, 1)</code>
<code>NotFittedError: ... is not fitted yet</code>	<code>predict</code> was called before <code>fit</code>	Call <code>fit</code> first; in a notebook, check that the cell really ran
<code>ConvergenceWarning: lbfgs failed to converge</code>	The solver ran out of iterations, usually because the features have very different scales	Scale in a pipeline, or raise <code>max_iter</code> . A warning, not an error: results exist but may be poor
<code>SettingWithCopyWarning (ChainedAssignmentError in pandas 3)</code>	A value was set on a filtered copy of a <code>DataFrame</code>	One step instead: <code>d.loc[mask, "rent"] = 999</code> , or an explicit <code>.copy()</code>
<code>IndentationError: expected an indented block</code>	The line after <code>for</code> , <code>if</code> or <code>def</code> is not indented	Indent by four spaces; never mix tabs and spaces in one file
<code>... does not accept missing values encoded as NaN</code>	The table still has gaps	Count them with <code>isna().sum()</code> and fill them with <code>SimpleImputer</code> in the pipeline

How to read a traceback

When a program stops, Python prints more than the message itself. It prints several lines listing where the problem was found, ending with the actual complaint.

WORD TO KNOW

Traceback — The block of text Python prints when a program stops with an error. Read it *from the bottom up*: the last line names the error and explains it, and just above it is the line of your own file that caused it.

Output

```
Traceback (most recent call last):
  File "/home/mlbook/rent_report.py", line 4, in <module>
    print(size.mean())
    ^^^^^
NameError: name 'size' is not defined. Did you mean: 'sizes'?
```

Four lines, and every one is useful. The last line names the error (`NameError`) and explains it in plain English, even suggesting the fix. Above it is the offending code, `print(size.mean())`, with arrows under the exact word. Above that is the file and the line number: line 4 of `rent_report.py`. When a traceback is long, most of it is inside the libraries; scroll to the last line first, then find the topmost line that names *your* file.

Two habits make all of these easier. First, when a message is unfamiliar, paste the last line of the traceback into a search engine: these are among the most-asked questions on the internet. Second, when a program is long and something looks wrong, add `print(X.shape, y.shape)` right after the split. Most confusing bugs are shape bugs, and that one line finds them.

CHAPTER SUMMARY

- This book needs Python 3.10 or newer plus five libraries: `numpy`, `pandas`, `matplotlib`, `scikit-learn` and `scipy`. Install them once with `conda` or with `python3 -m pip`; after that, nothing needs the internet.
- The Python used here is small: variables, numbers, strings, lists, indexing from zero, `for`, `if`, functions, `print` and f-strings.
- A NumPy array is a grid of numbers with a shape; arithmetic applies to every entry at once. `axis=0` gives one answer per column, `axis=1` one answer per row.
- A pandas DataFrame is a table with named columns. Start every new file with `shape`, `head`, `dtypes`, `describe`, `value_counts` and `isna().sum()`.
- Every scikit-learn estimator shares one interface: `fit` to learn, then `predict`, `predict_proba`, `transform` or `score` to use what was learned.
- Every model in this book is used through the same four lines; only the name on line one changes.
- Fitting a scaler on all the data before splitting leaks test information into training, and the score that results is too good. A Pipeline plus a ColumnTransformer makes the safe order automatic.
- Every matplotlib plot is a figure, one or more axes, and marks: `plot`, `scatter`, `hist`, `bar` — always with axis labels, units and a legend when there are two or more series.
- `random_state` and `np.random.default_rng` make random results repeatable; repeatable is not the same as reliable, and the library versions belong in your notes.
- Read a traceback from the bottom up: the last line says what went wrong, the line above says where.

Practice Problems

Try each problem on paper first, then check your arithmetic with Python where you can. Full step-by-step solutions are in the companion Teacher's Guide.

Problem 1 [Basic] Write three lines of code that turn the list `[12, 15, 30, 45]` into a NumPy array, print its shape, and print its mean. State what the shape will be before you run it.

Problem 2 [Basic] This code fails with `IndentationError: expected an indented block after 'for' statement`. Write the corrected version.

Python • no libraries

```
sizes = [85, 60, 105]
for s in sizes:
    print(s)
```

Problem 3 [Basic] Write the four pandas lines that read `datasets/ch03_apartments.csv`, print how many rows and columns it has, print how many apartments are in each city, and print how many values are missing in each column.

Problem 4 [Standard] An array `x` has shape `(40, 4)`. Write the line that gives the mean of each feature and the line that gives the mean of each row, and state the shape of each answer. Which of the two would a `StandardScaler` need?

Problem 5 [Standard] Fix the leak. Rewrite these lines so that the scaler never sees the test rows, using a `Pipeline`.

Python • scikit-learn

```
X_scaled = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=0)
model = Ridge().fit(X_train, y_train)
```

Problem 6 [Standard] Each of these three lines raises an error. Name the error and write the corrected line.
(a) `model.predict([85, 2])` for a model trained on two features. (b) `d["Rent"].mean()` on the `apartments` file.
(c) `LinearRegression().predict(X_test)` written before any fit.

Problem 7 [Challenge] A file has numeric columns `age`, `income` and `visits`, one text column `region`, and a target `y`. Some incomes are missing. Write a complete `ColumnTransformer` plus `Pipeline` that fills the gaps with the median, standardizes the numbers, one-hot encodes `region` and ends in a `Ridge` model — then the two lines that fit it and score it honestly.

Problem 8 [Challenge] A classmate reports 96% accuracy but gets 71% when a teacher reruns the same notebook. The program has no `random_state` anywhere, calls `fit_transform` on both `X_train` and `X_test`, and was run cell by cell out of order. Name the three separate problems, say which one explains the 96%, and list the changes that would make the two runs agree.

Datasets Used in This Book

Every dataset used in this book, and where to find it

Every dataset in this book is small enough to understand and, in the lessons, small enough to work by hand. The tiny datasets used inside worked examples — four melons, six customers, eight deliveries — are printed in full in the chapter itself, so you can copy them from the page. The larger files listed here are the ones the PYTHON activities read; they are supplied as plain CSV files in the datasets folder that accompanies this book.

All of these files are *simulated*: they were generated with a fixed random seed so that everyone who runs the code gets exactly the numbers printed in the book. They are built to behave like real measurements of the situation they describe, but they are not records of any real person, household, patient, farm or company.

Reading a dataset file

Every file has one header row naming the columns, then one row per example. Load one with pandas:

Python • pandas

```
import pandas as pd
df = pd.read_csv("datasets/ch03_apartments.csv")
print(df.shape)      # (rows, columns)
print(df.head())      # the first five rows
print(df.dtypes)      # the type of each column
```

If Python cannot find the file, give it the full path, or start Python from the folder that contains datasets. Appendix B, Section B.4 covers pandas in more detail.

The files

File	Rows	Cols	Chapter	What it holds
ch03_apartments.csv	40	7	3	Rental listings used for histograms, box plots, scatter plots and the correlation heat map.
ch04_listings.csv	8	6	4	The same listings with missing values, duplicates and an entry error, for cleaning practice.
ch05_clinic_visits.csv	100	4	5	Clinic appointments used for the cross-validation folds and the group-leakage example.
ch07_wheat_fields.csv	30	4	7	Wheat fields used for the underfitting/overfitting curves and learning curves.
ch11_apples.csv	120	3	11	Graded apples used for k-nearest-neighbor classification and the accuracy-versus-k curve.
ch12_clinic_days.csv	14	5	12	Clinic days with weather and day type, used for categorical naive Bayes.
ch13_machines.csv	280	5	13	Factory machines with vibration, temperature and age, used to grow and prune a decision tree.
ch14_deliveries.csv	600	7	14	Parcel deliveries used for bagging, random forests and out-of-bag scoring.
ch15_deliveries.csv	900	8	15	A larger delivery set used for boosting rounds, early stopping and stacking.
ch17_inverters.csv	120	4	17	Solar-inverter sensor readings whose healthy region is a ring, used for the RBF kernel.
ch18_members.csv	240	3	18	Supermarket loyalty members used for k-means, the elbow curve and silhouette scores.
ch20_sensors.csv	300	8	20	Eight machine sensors used for principal component analysis and the scree plot.
ch22_fields.csv	180	12	22	Farm-cooperative fields used for feature engineering and forward selection.
ch23_bakery.csv	364	5	23	Daily bakery demand used for grid search, random search and model comparison.
ch24_deliveries.csv	900	6	24	Deliveries used for permutation importance, partial dependence and drift monitoring.
ch24_scholarship.csv	600	7	24	Scholarship applications used for the group-fairness audit.
ch25_homes.csv	2400	12	25	Home sales used for the Hawler Homes capstone project.

Columns, file by file

The column names below are exactly the ones in the file's header row.

File	Columns
ch03_apartments.csv	id, size_m2, rooms, age_years, distance_km, city, rent
ch04_listings.csv	id, size, age, city, condition, rent
ch05_clinic_visits.csv	hour, waiting, clinicians, long_wait
ch07_wheat_fields.csv	field, rainfall_mm, yield_t_ha, set
ch11_apples.csv	weight_g, firmness, grade
ch12_clinic_days.csv	day, weather, day_type, holiday, crowded
ch13_machines.csv	machine_id, temperature_c, vibration_mm_s, needs_inspection, set
ch14_deliveries.csv	delivery_id, distance_km, parcels, rain, driver_experience_years, late, set
ch15_deliveries.csv	delivery_id, distance_km, parcels, rain, driver_years, minutes_late, set, late
ch17_inverters.csv	v_dev, i_dev, status, label
ch18_members.csv	member_id, visits_per_month, avg_spend_kiqd
ch20_sensors.csv	temperature_c, vibration_mm_s, motor_current_a, humidity_pct, rotation_rpm, feed_pressure_bar, fan_speed_rpm, belt_wear_index
ch22_fields.csv	field, village, rainfall_mm, gauge_mm, fertilizer_kg, area_dunam, plant_day, harvest_day, shoe_size, tractors, yield_t_ha, set
ch23_bakery.csv	day, temp_c, holiday, promo, rolls
ch24_deliveries.csv	distance_km, stops, heavy_rain, eta_min, late, set
ch24_scholarship.csv	applicant_id, school_type, exam_score, essay_score, qualified, shortlisted, shortlisted_v2
ch25_homes.csv	sale_month, size_m2, bedrooms, bathrooms, age_years, distance_km, neighborhood, home_type, garden, parking, days_on_market, price_usd

Datasets printed inside the chapters

These are the small tables used for the hand calculations. They are not supplied as files, because typing eight rows is part of the exercise.

Chapter	Dataset	Size
1	Melons: weight, sound score, sweet	4 rows
1	Ripe and unripe melons for the nearest-centroid example	4 rows
2	Clinic visits for the baseline example	9 rows
3	Apartment listings with size, rooms, age, city and rent	8 rows
4	Listings with missing values, a duplicate and an entry error	8 rows
6	Pump inspections for the confusion matrix	50 predictions
6	Ten patients with predicted probabilities, for threshold moving	10 rows
7	Wheat fields at three model capacities	3 models
8	Rania's homes: size, appliances, monthly bill	5 rows
9	Homes with air conditioners, for gradient descent by hand	4 rows
10	Students: hours studied, attendance, pass or fail	8 rows
11	Apples: weight, color score, grade	8 rows
12	Clinic days: weather, day type, holiday, crowded	14 rows
12	Ten short messages for the spam classifier	10 messages
13	Machines: vibration, temperature, age, needs inspection	10 rows
14	Deliveries: distance, parcels, weather, experience, late	8 rows
15	Deliveries with minutes late, for boosting rounds	5–6 rows
16	Pumps in two classes, for the margin example	6 rows
17	Irrigation trials for the XOR and kernel examples	4–5 rows
18	Customers: visits per month, average spend	6 rows
19	Villages with two coordinates, for linkage and DBSCAN	6 rows
20	Students' mathematics and physics scores	5 rows
20	Four-subject report cards	8 rows
21	Confirmed-normal meter readings	6 rows
22	Farm fields: rainfall, fertilizer, area, yield	6 rows
23	Fold-by-fold errors of six candidate models	6 models
24	Clinic records for the k -anonymity check	6 rows
25	Threshold table for the churn campaign	8 thresholds

Making your own version of any dataset

Every figure in this book is produced by a Python script, and those scripts also write the CSV files listed above. If you want a bigger version of a dataset, or one with different numbers, open the script for that chapter, change the number of rows or the random seed, and run it again. Changing the seed changes the data, so your answers will no longer match the book's — which is exactly the point of the reproducibility discussion in Appendix B, Section B.8.

IMPORTANT

None of these files contains real personal data. If you replace one with a real dataset of your own — patient records, student records, customer records — then privacy law and your institution's rules apply to it. Chapter 24 explains what that means in practice: collect only what you need, remove direct

identifiers, check that rare combinations cannot re-identify anyone, and never publish a file you have not checked.

Glossary

Every dataset used in this book, and where to find it

Every important term used in this book, in one place. The chapter that introduces a term explains it more fully; this page is for when you meet the word again later and want a one-line reminder.

A

Accuracy — The fraction of predictions a model gets exactly right, usually written as a percentage.

AdaBoost — A boosting method that gives every training row a weight, raises the weights of the rows each weak learner gets wrong, and lets every learner vote with a strength set by how few mistakes it made.

Agglomerative clustering — The bottom-up way of building a cluster tree: every example starts alone in its own cluster, the two closest clusters are merged, and this repeats until a single cluster holds everything. It needs exactly one merge fewer than the number of examples.

Alert budget — The number of flagged examples a team can genuinely investigate in a given period, such as ten meter visits a day. It is decided by people and money, not by the data, and it is usually what sets the threshold.

Algorithm — A clear, exact set of steps that solves a problem. A learning algorithm is the set of steps a computer follows to find a pattern in data.

Algorithmic bias — A pattern in a model's predictions or errors that systematically disadvantages one group of people more than another. It is a mismatch between how the model treats one group and another, not the statistical bias of Chapter 7.

Anomaly — An example that looks noticeably different from most other examples once all of its features are considered together. It may be a real problem or simply a rare but honest example, so it is always a candidate for a human check, never a verdict.

Anomaly score — A number saying how unusual one example looks compared with normal data. A higher score means further from typical; every detector produces one, and they differ only in how it is computed.

Anonymization — Changing a dataset so that no row can be traced back to a particular person, even by somebody who combines it with other information. Deleting names and ID numbers on its own is not anonymization.

Array — A grid of numbers of one type, built by the NumPy library. An array has a shape — the number of rows and columns it holds — and arithmetic written on it is applied to every entry at once.

Assignment step — The half of a k-means round that sends every example to its nearest centroid, leaving the centroids where they are.

AUC (area under the ROC curve) — One number, from 0.5 (random guessing) to 1.0 (perfect), for how well a model ranks positives above negatives. It equals the probability that a random positive example scores higher than a random negative one (Hanley and McNeil, 1982).

Average linkage — A linkage rule that measures the distance between two clusters as the mean of every pair of distances with one point in each cluster. It sits between single and complete linkage, and is also known as UPGMA.

Axis-aligned boundary — A decision boundary built only from straight pieces at right angles to the feature axes. Decision trees can draw only this boxy kind of boundary, because every question compares one feature with one threshold.

B

Bag of words — A way of turning a document into numbers: choose a fixed list of words, called the vocabulary, and count how many times each of them appears in the document. The order of the words is thrown away, so two messages with the same words in a different order give exactly the same vector.

Bagging — Short for bootstrap aggregating: training many copies of the same kind of model, each on its own bootstrap sample, and then combining every copy's prediction by voting or averaging. Bagging mainly reduces variance.

Base model — One of the first-layer models in a stacked ensemble, whose prediction becomes an input column for the second layer.

Base rate — How common a class is in the population before any evidence is examined — the prior expressed as a rate. Judging a positive test result by the test's accuracy alone, while ignoring a low base rate, is the base-rate fallacy.

Baseline — A deliberately simple predictor used as the minimum standard of comparison: the training mean for regression, the most common class for classification. A useful model must clearly beat it.

Batch gradient descent — Gradient descent in which every step is computed from all of the training examples at once. Each step is accurate but needs a full pass through the data.

Bayes' theorem — A rule for updating a belief after new evidence arrives: multiply what you believed beforehand by how well the evidence fits that belief, then divide by how likely the evidence was overall.

Bias — A model's tendency to make the same kind of mistake, in the same direction, again and again, because its assumptions are too simple for the real pattern. High bias shows up as underfitting.

Bias-variance trade-off — The pattern where lowering a model's bias, usually by giving it more capacity, tends to raise its variance, and lowering its variance tends to raise its bias. The best model is usually not the simplest one and not the most flexible one, but somewhere in between.

Binning — Replacing a numeric column by a small number of ordered groups, such as Low, Medium and High, so a model sees only the broad level.

Boosting — Building an ensemble by training models one at a time, in order, where every new model concentrates on the examples the models before it are still getting wrong. Their outputs are then added together.

Bootstrap sample — A new dataset built by drawing rows at random from an original dataset, with replacement, until it has as many rows as the original. It usually repeats some rows and leaves others out completely.

Border point — In DBSCAN, a point that is not itself crowded enough to be a core point, but that lies inside the neighborhood of some core point. It sits on the outer edge of a cluster and still belongs to it.

Box plot — A picture of a column: a box from Q_1 to Q_3 with a line at the median, whiskers to the most extreme values that are not outliers, and a dot for each outlier. It was introduced by the statistician John Tukey in 1977.

Branch — One of the two paths leaving a node of a decision tree, taken according to the answer to that node's question.

Broadcasting — NumPy's rule for combining arrays of different shapes: the smaller one is repeated automatically to match the larger one, so a row of column means can be subtracted from every row of a table without writing a loop.

C

Calibration — How well a model's predicted probabilities match real-world frequencies. A model is well calibrated if, among all the cases where it predicts a 70% chance, the event really happens about 70% of the time. A reliability diagram shows this check for several probability buckets at once.

Capacity — How many flagged cases the people who act on a model can actually handle in one cycle — for example the number of phone calls a small team can make in a month. Capacity turns a ranked list into a shortlist of a fixed length.

Capstone project — A project carried through every stage of the workflow by one person or team: the decision sentence, cleaning, splitting, a baseline, a model comparison, one honest test score, an explanation, and a plan for watching the model after launch.

Categorical feature — A value chosen from a fixed set of names or codes. A nominal feature has no natural order (city, blood type). An ordinal feature has an order but no fixed distance between steps (condition: needs repair, fair, good, excellent — nobody can say that the step from fair to good equals the step from good to excellent). A binary feature has exactly two values (furnished: yes or no).

Centering — Subtracting a feature's mean from every value of that feature, so the column's new mean is exactly zero and each number says how far above or below average an example sits.

Centroid — The average position of a group of points, found by averaging each feature separately.

Chain rule — A rule for finding the slope of a function built out of another function: multiply the slope of the outside step by the slope of the inside step.

Chaining — The tendency of single linkage to build long, straggly clusters by following a line of nearby points. A thin bridge of examples running between two genuinely separate groups is enough to glue them into one cluster.

Churn — A customer ending their relationship with a business, such as canceling a subscription, closing an account or not renewing. In a churn project the label usually records whether a customer leaves within a fixed future window, such as the next 30 days.

Circularity — Using as a feature the very quantity a model is meant to produce or replace. A circular feature may be known in time, so it is not always leakage, but it hides the model's real skill because the model can lean on the human judgment it was supposed to improve.

Class imbalance — A situation where one class has far more examples than another. The common class is the majority class, the rare class the minority class.

Class weight — A number attached to a class before training that multiplies how much a mistake on that class costs inside the learning algorithm's loss; a high weight on the minority class pushes the algorithm to avoid missing it.

Classification — A learning task where the model predicts a category from a short, fixed list of possible answers.

Closed-form solution — An answer given by a formula that can be worked out once, directly, instead of being searched for step by step. Linear regression with one feature has a closed-form solution for its slope and intercept.

Cluster — A group of examples that a clustering method has decided are similar to one another. The method invents and numbers the groups; only a person can decide afterward what each group seems to represent.

Cluster profile — A short summary of one cluster giving its size and the average of each original feature inside it, always reported in the original units so that people can read it.

Clustering — A learning task where the model groups similar examples together without being told the groups in advance.

Coefficient — The number a feature is multiplied by inside a regression equation, also called a weight, or the slope when there is only one feature. A coefficient of 0.7 means the prediction rises by 0.7 for every extra unit of that feature.

R^2 (**coefficient of determination**) — A score comparing a model's squared errors with those of always predicting the mean: 1 is perfect, 0 is no better than the mean, and a negative value is worse than the mean. The name was introduced by the geneticist Sewall Wright in 1921.

Collective anomaly — A group of examples that is unusual as a group, even though no single member of the group is unusual on its own — for example eleven identical sensor readings in a row.

ColumnTransformer — A scikit-learn tool that sends different columns down different preprocessing paths — numeric columns to a scaler, text columns to an encoder — and joins the results back into one table.

Complete linkage — A linkage rule that measures the distance between two clusters as the distance between their two farthest members. It is the strictest rule, and it tends to produce compact clusters of similar size.

Conditional independence (the naive assumption) — The assumption that, within a single class, the features carry no information about each other. It is what lets naive Bayes replace the probability of a whole feature combination with the separate feature probabilities multiplied together.

Conditional probability — The probability that one event happens once another is already known to have happened, written $P(A \mid B)$ and read “the probability of A given B”. It divides by the smaller group of cases where B happened, not by everybody.

Confusion matrix — A table that sorts every prediction of a classifier into one of four groups by comparing the prediction with the truth. Rows are the actual classes, columns the predicted classes.

Consent — A person’s freely given, informed and specific agreement that their data may be used for a stated purpose. It is not valid if the purpose was hidden, vague, or changed afterward.

Contamination — The fraction of the training data believed to be anomalous. Several scikit-learn detectors take it as a setting; it decides how many examples get flagged, not how many problems really exist.

Contextual anomaly — An example whose value is perfectly ordinary in general but wrong for its context — its time of day, its season, its place, or the customer it belongs to. A temperature of 32 degrees Celsius is ordinary in May and remarkable in January.

Convergence — The point at which further training rounds no longer change a model’s parameters or its loss in any useful way. A run that keeps getting worse instead is said to diverge.

Convex function — A function shaped like a bowl, with one lowest region and no misleading dips along the way. A convex loss can always be minimized by walking downhill.

Core point — In DBSCAN, a point that has at least `minPts` points inside its neighborhood, counting itself. Core points sit in the crowded middle of a cluster and are the only points that can grow one.

Correlation coefficient r — A number between -1 and $+1$ that measures how closely two numerical columns follow a straight line together. Values near $+1$ mean they rise together, values near -1 mean one falls as the other rises, and values near 0 mean no straight-line relationship.

Correlation matrix — A square table with one row and one column per numerical feature, holding the correlation of every pair; the diagonal is always 1 and the table is symmetric.

Covariance — A number saying how two centered features move together: positive when they rise and fall together, negative when one rises as the other falls, zero when there is no straight-line relationship.

Covariance matrix — A square table holding each feature’s variance down its diagonal and each pair’s covariance off the diagonal. Software reads the principal components out of this table.

Curse of dimensionality — The way distances stop being meaningful as features are added, because every example ends up almost equally far from every other example.

Customer leakage — Information about one particular person or object crossing from the training set into the test set because several rows describe that same person or object. The model can then partly recognize the individual instead of learning the general pattern, and the test score looks better than it should.

D

Data drift — A change over time in the data a live model sees, compared with the data it was trained on. It can appear in the features themselves or in how those features relate to the true answer.

Data leakage — Information entering training that would not actually be available at the real moment of prediction. It makes test scores look better than the model will perform in real use.

Data minimization — Collecting and keeping only the fields actually needed for a stated purpose, and deleting them once that purpose has been served. Data you never collected cannot leak or be misused.

DataFrame — A table with named columns, built by the pandas library. Different columns may hold different types, and each column is used by name rather than by position. A single column on its own is a Series.

Dataset — An organized collection of examples used to train or test a model; usually a table with one row per example.

Date/time feature — A date or a moment, such as the day a listing was posted; usually turned into numerical features (month, day of the week, days since an event) before training.

Decimal places — How many digits are written after the decimal point. Not the same as significant figures: 0.0042 has four decimal places but only two significant figures.

Decision boundary — The line, or surface, that separates the region where a classifier predicts one category from the region where it predicts the other. For logistic regression with the usual one-half cutoff it lies exactly where the linear score is zero: a single point when there is one feature, a straight line with two, a flat surface with more.

Decision function — The raw score a linear classifier works out for one example before any decision is made. Its sign says which class is predicted, and its size says how far the example sits from the boundary. It is a score, never a probability.

Decision sentence — One plain sentence written before any data work: “At prediction time, the model will use ___ to predict ___, so that ___.”

Decision stump — The smallest possible decision tree: one question and two leaves. It is the usual weak learner inside a boosted model.

Decision tree — A model that predicts by asking a sequence of simple yes-or-no questions about an example’s features, one question at a time, until it reaches a final answer.

Demographic parity — A fairness rule that asks every group to receive the positive decision at the same rate. The gap between two groups’ selection rates is the demographic parity difference.

Dendrogram — The tree picture produced by hierarchical clustering. Each merge appears as a horizontal bar, and the height of the bar shows how far apart the two merged clusters were. Cutting the tree with a horizontal line turns it into a clustering.

Density-based clustering — An approach that treats a cluster as a region where points are packed closely together, separated from other such regions by emptier space. Points that lie in no crowded region are labeled noise instead of being forced into a cluster.

Deployment — Putting a trained model to work on real, new examples, for instance connecting it to a clinic’s check-in screen.

Depth — How many questions an example has answered by the time it reaches a node of a

tree. The root sits at depth 0, and the deepest leaf gives the depth of the whole tree.

Derivative — The slope of a function at one exact point: how much the output changes for a tiny change in the input, measured right there.

Difference feature — A new feature made by subtracting one column from another, such as the number of days between a planting date and a harvest date.

Differential privacy — A mathematical guarantee that the answers released from a dataset are almost the same whether or not any one person’s record is included, achieved by adding a carefully measured amount of random noise.

Dimensionality reduction — Cutting down the number of features used to describe each example while keeping as much of the data’s real information as possible. It usually builds a few new, blended columns rather than deleting original ones.

Distance matrix — A square table holding the distance between every pair of examples. Its diagonal is all zeros and it is symmetric, so a table of n examples holds only $n(n - 1)/2$ different distances.

Distance-weighted voting — A version of the kNN vote in which each neighbor’s vote is multiplied by one divided by its distance, so closer neighbors count more than distant ones.

Divisive clustering — The top-down way of building a cluster tree: everything starts in one big cluster, which is split in two, then split again. It is much rarer than agglomerative clustering, because there are far more ways to split a group than to merge two of them.

Domain knowledge — What the people who actually do the work already understand about how things behave. It tells you which new columns are worth building, before any model is trained.

Dot product — A way of combining two lists of numbers of the same length into one single number: multiply each pair of matching entries, then add all the products.

Dual weight — The number training gives to each support vector, saying how much that example influences a kernel model’s predictions. Every example that is not a support vector has a dual weight of zero and can be thrown away.

Dummy-variable trap — Redundancy caused by keeping every one-hot column of a category together with a linear model's intercept term. Because the three city columns always add up to 1, any one of them can be computed from the other two ($\text{City_Erbil} = 1 - \text{City_Duhok} - \text{City_Sulaymaniyah}$), so a linear model (a later chapter) has two ways to express the same thing and its weights become ambiguous. The fix is to drop one column (scikit-learn: `drop="first"`); tree-based models are not affected. A related point: the category list is learned from the training rows, so a test listing from an unseen city should be encoded as all zeros (`handle_unknown="ignore"`) rather than crash the program.

Duplicate record — A row that describes the same example as another row. An exact duplicate matches in every column; a near duplicate matches in every column except one that is generated automatically, such as an ID or a time stamp, or differs only by spelling or spacing.

E

Early stopping — Stopping training at the round where the validation loss is lowest, instead of continuing until the training loss is lowest.

Eigenvalue — The stretch factor belonging to an eigenvector: how much longer or shorter that direction becomes. A large eigenvalue marks an important direction.

Eigenvector — A direction that a particular matrix does not turn. Multiplying the matrix by it gives back the same direction, only stretched or shrunk.

Elbow method — A way of choosing the number of clusters by plotting inertia against several values of k and picking the bend, where adding another cluster stops helping much.

Embedded method — A feature-selection method in which the model itself does the selecting while it trains, as lasso does by driving weak coefficients to zero.

Encoding — Turning a categorical column into numbers a model can use.

Ensemble — A model built by combining the predictions of many smaller models instead of relying on one. An ensemble usually votes for a category or averages for a number.

Entropy — An impurity measure borrowed from information theory. It is zero for a node whose examples all share one label and, for two classes, reaches its largest value of one bit when the node is split evenly.

Epoch — One complete pass through every training example. How many parameter updates happen in an epoch depends on how many examples are used for each step.

Epsilon neighborhood — The circle of a chosen radius, written with the Greek letter epsilon, drawn around one point. Everything inside that circle counts as a neighbor, so choosing the radius means choosing how close counts as close.

Epsilon-insensitive tube — The band drawn above and below the line fitted by support vector regression. Any error smaller than the tube's half-width counts as no error at all, so points inside the tube cost the model nothing and do not shape the line.

Equal opportunity — A fairness rule that asks every group to have the same false-negative rate, so the model is equally good at finding the people who really qualify. The gap between two groups' false-negative rates measures it.

Error slice — The error of a model computed on one meaningful subgroup of the test set — one price band, one region, one age group — instead of on the whole test set at once. Slices show which group a model serves worst.

Estimator — Any scikit-learn object that learns from data. Every estimator has a `fit` method; models add `predict`, and preprocessing steps add `transform`. Because they share this interface, one can be swapped for another without changing the rest of a program.

Euclidean distance — The straight-line distance between two points, found by squaring the difference in each feature, adding the squares, and taking the square root of the total.

Expected value — An average over many cases: every possible outcome's value multiplied by how likely that outcome is, with all the products added together.

Experience (E) — The exact data a model learns from: how many examples and where they came from.

Explained variance ratio — The share of a dataset's total variance that one principal component

accounts for. The ratios of all the components always add up to 100 percent.

Exploratory data analysis (EDA) — The habit of describing and plotting a dataset — one column at a time, then pairs, then everything — before building any model, in order to understand what the data holds and to catch problems early.

Extrapolation — Using a model to predict for feature values outside the range covered by its training data. The model has never seen such values, so the prediction rests on nothing the data can support.

F

F1 score — A single score combining precision and recall with the harmonic mean, an average that stays high only when both are high. It is the best-known member of the F-measure family proposed by C. J. van Rijsbergen in 1979.

Fairness — The requirement that a model's benefits, errors and decisions do not systematically disadvantage one group of people more than another in ways that matter for the situation. What counts as fair depends on context and needs more than one check.

False negative (FN) — Actually positive, predicted negative — a miss.

False positive (FP) — Actually negative, predicted positive — a false alarm.

Feature — One measured property of an example, used as an input to a model.

Feature engineering — Creating new, more useful columns from the columns a dataset already has, using arithmetic and knowledge of the subject, so that a model can find a pattern more easily.

Feature importance — A score for each feature saying how much of a tree's total impurity drop that feature was responsible for. The scores are shares of one whole and add up to 1.

Feature map — The rule, usually written with the Greek letter phi, that turns one point of the original data into its point in a larger space of new features such as squares and products. Every example, training or new, must go through the same map.

Feature matrix X — All the feature values arranged as a grid with n rows (one per sample) and p

columns (one per feature); row i holds the feature values of sample i .

Feature scaling — Rescaling a numerical column so that different columns have comparable ranges. The two common methods are standardization and min-max normalization.

Feature selection — Choosing a smaller, more useful subset of the available features to give a model, instead of using every column that happens to exist.

Feature space — The larger space whose axes are quantities computed from the original features. A boundary that is straight in feature space can look curved when it is drawn back in the original features.

Feature subsampling — Choosing a node's split from only a small, randomly chosen subset of the features, drawn afresh at that one node. The setting that controls the subset size is usually called max features.

Filter method — A feature-selection method that scores each feature on its own, before any model is trained, and keeps the best-scoring ones.

Fit and transform — The two halves of every preprocessing step: fit learns the rule from the training rows (the median, the category list, the mean and standard deviation); transform applies an already-learned rule to any rows — training, test or brand-new.

Fold — One of the k equal-sized pieces into which the rows are cut for cross-validation.

Forward selection — A wrapper method that starts with no features and repeatedly adds whichever remaining feature improves the validation score most, stopping when no addition helps enough.

Frequency encoding — Replacing each category name by the share of the training rows that carry it, turning a crowded category column into a single number.

Function — A rule that turns each input into exactly one output. A trained model is a function: features go in, a prediction comes out.

G

Gamma — The setting of the RBF kernel that decides how quickly similarity falls off with distance. A small gamma lets far-away examples still count; a large gamma makes each example notice only

its close neighbors, and can lead to memorizing the training data.

Gaussian mixture model (GMM) — A soft clustering method that treats the data as a blend of bell-shaped clouds, each with its own center, width, tilt and share of the data.

Gaussian naive Bayes — The version of naive Bayes used for numerical features. For each feature and each class it stores only a mean and a standard deviation, and it uses the normal (bell-shaped) curve as the likelihood.

Generalization — A model's ability to perform well on new examples it has never seen, not only on the examples it trained on. Generalization, not training performance, is the true goal of every machine learning project.

Generalization gap — The validation error minus the training error. A large gap means the model does much better on the examples it memorized than on new ones: the signature of overfitting.

Gini impurity — The most common way to measure how mixed a node's labels are. Square each label's share of the node, add the squares together, and subtract the total from one.

Global explanation — A description of what a model does in general, across all examples — for instance, which features it relies on most and in which direction.

Global minimum — The lowest point of the whole loss surface: the best parameter values that exist for a given model and dataset.

Gradient — The list of the slopes of the loss, one for each parameter. It points in the direction in which the loss rises fastest, so its opposite points straight downhill.

Gradient boosting — Boosting in which each new model is fitted to the residuals left by the running prediction, and a scaled-down version of its output is added to that running prediction.

Gradient descent — A training method that repeatedly measures which direction lowers the loss and takes one small step that way, until the steps stop making a difference.

Greedy algorithm — An algorithm that takes the best-looking step available right now, without checking whether a slightly worse step now would allow much better steps later. Tree growing is greedy.

Grid search — A tuning method that lists a few values for each hyperparameter and tries every possible combination of them, training and scoring one model for each combination.

Group leakage — Leakage that happens when rows belonging to the same person, machine or place land on both sides of a split, so the model can recognize the group instead of learning a general rule.

H

Hard assignment — Giving each example exactly one cluster, with no measure of how confident that choice was. Plain k-means always does this.

Hard margin — The strict form of the support vector machine rule, which demands that every single training example sit outside the margin on its own correct side. It can only be used when the two classes can be separated perfectly by a straight boundary.

Hard voting — Combining several finished models by letting each announce one label and keeping the label named by the most models.

Heat map — A grid whose cells are colored by their value, so that patterns can be seen without reading every number. A correlation heat map colors the correlation matrix.

Hierarchical clustering — A family of clustering methods that build a whole tree of groupings at once, from every example on its own up to a single group containing everything. The number of clusters is chosen after the tree is built, not before.

High-cardinality category — A category column with a large number of different values, such as village names or product codes, which one-hot encoding would turn into a huge number of almost always empty columns.

Hinge loss — The loss function a support vector machine makes as small as possible. It charges nothing once an example is correct by a full unit of margin, and then rises in a straight line as the example gets worse. The graph looks like a door hinge.

Histogram — A plot that cuts the range of a numerical column into equal intervals called bins and draws one bar per bin, with height equal to the number of values in it. Too few bins hide the shape and too many make it ragged; 6 to 15 bins

is a sensible start for 30 to 300 values. The name was introduced by the statistician Karl Pearson in 1895.

Histogram binning — Sorting a numerical feature's values into a small number of buckets before training, so that each split search compares a few hundred bucket edges instead of every distinct value. It makes boosting much faster and changes the result very little.

Holdback group — A randomly chosen part of the flagged cases that is deliberately not acted on, kept only so that the team has a fair comparison group of equally at-risk cases. Without one, nobody can tell whether acting on the prediction changed anything.

Holdout set — Any group of rows kept out of training so that the model can be scored on data it did not learn from; the validation set and the test set are both holdout sets.

Human in the loop — A design in which a model's output is a recommendation for a trained person to review and act on, rather than an automatic decision. The person can overrule the model, and those overrules are recorded and studied.

Hyperparameter — A setting chosen before training begins, rather than a value the learning algorithm discovers from the data. Changing a hyperparameter changes how a model learns, including how much freedom it has to bend.

I

Identifier — A column, such as an ID number, that names a row but carries no information about it; kept for bookkeeping, dropped before training.

Impurity — A number measuring how mixed the labels are inside one node. A node holding only one label has zero impurity; a node with an even mix has the highest impurity.

Imputation — Filling a missing value with a substitute learned from the training data, such as the column's median, mean or most frequent value.

Independent events — Two events are independent when knowing that one happened does not change the probability of the other, so the probability that both happen is simply the two probabilities multiplied together.

Indicator column — An extra 0/1 column that records whether the original value was missing

(1) or present (0). It lets a model notice that "unknown" may itself carry information.

Inertia — The total squared distance from every example to the centroid of its own cluster, added up across the whole dataset. Also called the within-cluster sum of squares. A lower inertia means tighter clusters.

Inference time — How long a trained model takes to produce predictions, usually quoted per thousand rows. It matters most when a prediction has to appear while somebody is waiting for it.

Information gain — How much impurity a split removes: the parent node's impurity minus the weighted impurity of the two children it would create. Larger is better.

Inner loop — In a nested cross-validation, the search that picks the hyperparameters. It is run separately inside each outer training part and never sees the outer test fold.

Input space — The space of the original measured features, with one axis for each column of the dataset.

Instance-based learning — A family of methods, also called lazy learning, that keep the training examples themselves instead of summarizing them into an equation, so all the work happens when a prediction is asked for.

Interaction feature — A new feature made by multiplying two features together, so a model can see their combined effect directly. It is large only when both original features are large at the same time.

Intercept — The fixed number added at the end of a regression equation, after every coefficient has been multiplied by its feature. It is the model's prediction when every feature equals zero.

Internal node — A node below the root of a tree that still asks a question and still splits its group of examples in two.

Interpretability — How well a person can understand why a model made a particular prediction, and what patterns it relies on in general. An interpretable model is easy to check, question and explain to somebody else.

Interquartile range (IQR) — The distance between the first and third quartiles, $Q_3 - Q_1$: the width of the middle half of the sorted values. The two cut-off lines $Q_1 - 1.5 \text{ IQR}$ and $Q_3 + 1.5 \text{ IQR}$ are called

the fences; any value outside them is flagged as a possible outlier.

C (inverse regularization strength) — The dial in scikit-learn that decides how strongly a logistic regression model is punished for large coefficients. It runs backward from the usual penalty strength: a small C means a strong penalty and small, cautious coefficients, while a large C means almost no penalty at all. It is a hyperparameter, so it is chosen on a validation set.

Irreducible noise — Randomness that no model can predict away: two fields with identical rainfall still give slightly different yields. It sets a floor below which no error can fall.

Isolation depth — The number of random cuts needed before one example sits alone in its own region of an isolation tree. Also called the path length. A small depth means the example was easy to separate, and therefore unusual.

Isolation forest — Many isolation trees built on random subsamples of the data. An example's anomaly score comes from its average isolation depth across all the trees: the shallower the average, the more unusual the example.

Isolation tree — One sequence of random cuts, each made on a randomly chosen feature at a randomly chosen value, continued until every example sits alone. Unlike a decision tree, it uses no labels and no measure of purity.

Iteration — One complete round of gradient descent: predict every example, find the residuals, compute the gradient, and update every parameter once.

J

Joint probability — The probability that two events both happen, written $P(A \text{ and } B)$. Estimate it by dividing the number of cases where both happened by the total number of cases.

Jupyter notebook — A document that mixes code cells, their output and written notes on one page in a web browser. Cells can be run one at a time and in any order, which is convenient but can also show results the code would no longer produce if run from top to bottom.

K

k-anonymity — A safety check for a table about people: group the rows by their quasi-identifiers,

count each group, and take the smallest count. A table is k-anonymous when every row shares its quasi-identifiers with at least k minus 1 others.

Kernel function — A short formula that takes two examples and returns one number saying how similar they are. That number equals the dot product the two examples would have had in a much larger space of built features, without anyone ever building them.

Kernel matrix — The square table holding the kernel value of every pair of training examples. With a thousand training rows it has a million entries, which is why kernel methods become expensive on large datasets. It is also called the Gram matrix.

Kernel ridge regression — Ridge regression with every dot product replaced by a kernel, so that it predicts a number with a smooth curve instead of a straight line. Its prediction is a weighted sum of the kernel similarities between the new point and the training points.

Kernel trick — Replacing every dot product inside an algorithm with a kernel function, so the algorithm behaves as though it worked in a huge space of new features while all of its arithmetic stays in the original, small one.

k-fold cross-validation — A procedure that trains and scores a model k times, each time holding out a different fold for validation and training on the other folds, then averages the k scores.

k-means — A clustering method that sorts examples into k groups by repeating two steps: send every example to its nearest centroid, then move each centroid to the average position of the examples now assigned to it.

k-Nearest Neighbors (kNN) — A method that predicts the answer for a new example by finding the k training examples most similar to it and letting them vote on a class, or averaging their values for a number.

kNN regression — Predicting a number for a new example by averaging the target values of its k nearest training examples.

L

L1 norm — The length of a list of numbers found by dropping every minus sign and adding, with no squaring and no square root. The lasso penalty uses it.

L2 norm — The ordinary straight-line length of a list of numbers: square every entry, add the squares, take the square root. The ridge penalty uses it.

Label — The known, correct answer attached to a training example. When it is a number it is often called the target.

Laplace smoothing (add-one smoothing) — Adding a small constant, usually 1, to every count before dividing, and adding the matching amount to the denominator so the probabilities still add to 1. It removes zero counts, which would otherwise wipe out a whole class score, at the cost of pulling every estimate slightly toward “all values equally likely”. Pierre-Simon Laplace introduced the idea in 1814.

Lasso regression — Linear regression with a penalty equal to the regularization strength times the sum of the absolute values of the coefficients. Lasso can shrink a weak coefficient all the way to exactly zero, which drops that feature from the model.

Leaf node — An ending point of a tree that asks no further question and simply gives a prediction.

Learner weight — In AdaBoost, the number that says how loudly one weak learner may speak in the final vote. It grows as that learner’s weighted error rate falls, and is zero for a learner that is no better than guessing.

Learning curve — A plot of training error and validation error against the size of the training set. The shape of the two lines together shows whether a model is underfitting, overfitting or generalizing well, and whether more data would help.

Learning rate — A small number chosen before training that decides how big each gradient descent step is. Too small wastes time; too large makes the loss grow instead of shrink.

Learning rate (boosting) — The fraction of each new model’s output that is actually added to the running prediction, usually well below 1. A smaller learning rate needs more rounds but makes the best stopping point far easier to hit.

Least squares — The rule that the best line is the one making the sum of the squared residuals as small as possible. It is the standard way of fitting a linear regression model.

Leave-one-out cross-validation — Cross-validation in which every single row is its own fold, so one

model is trained per row, each on all the other rows.

Library — A collection of ready-made code that someone else has written, tested and shared, which your own program can load and use. NumPy, pandas, matplotlib, scikit-learn and SciPy are libraries. Also called a package.

Likelihood — How likely the evidence you actually saw would be if a class were true, written $P(\text{evidence} \mid \text{class})$. It is a conditional probability read backwards, from class to evidence.

Linear discriminant analysis (LDA) — A probabilistic classifier that models each class as a bell-shaped cloud of points and forces every class to share one common shape and spread. Its decision boundary is always a straight line, or a flat plane in more dimensions. Ronald A. Fisher introduced it in 1936.

Linear regression — A method that predicts a number by multiplying each feature by its own coefficient, adding the results together, and adding one fixed number at the end. With a single feature it is a straight line drawn through the data.

Linearly separable — Describes two classes that at least one straight line, or one flat surface in more than two dimensions, can separate with no mistakes at all: every example of one class on one side, every example of the other class on the other.

Linkage — The rule that decides how far apart two clusters are once a cluster may hold more than one example. Single, complete, average and Ward are the four rules in common use, and they can give genuinely different clusters from the same data.

Loading — One of the numbers inside a principal component’s direction, saying how much of one original feature is mixed into that component. A loading near zero means the feature hardly matters there.

Local explanation — A description of why a model gave one particular example the answer it did — the kind of explanation a rejected applicant actually wants.

Local minimum — A dip in the loss surface that is lower than everything immediately around it but still higher than the global minimum. A downhill search can get stuck in one.

Local Outlier Factor (LOF) — A method that scores an example by comparing how spread out its own neighborhood is with how spread out its neighbors' neighborhoods are. A ratio near 1 is ordinary; a ratio well above 1 means the example sits in a much emptier pocket than the company it keeps.

Log loss — A loss function for probabilities, also called cross-entropy loss. It compares the true yes/no answer with the predicted probability and charges a very heavy penalty for a confident prediction that turns out wrong. A perfect, fully confident, correct prediction costs nothing at all, and a prediction of one-half always costs about 0.693.

Log transform — Replacing a feature's values by their logarithms, so that equal multiplications become equal steps. It pulls in the long tail of a strongly skewed column.

Logarithm — The power a fixed base must be raised to in order to reach a given number. A logarithm undoes an exponent, the way subtraction undoes addition.

Logarithmic axis — A chart axis on which equal steps mean equal multiplications rather than equal additions, so a very wide range of values fits on one readable line.

Logistic regression — A method that predicts the probability of a yes-or-no answer. It builds the same kind of score as linear regression, a coefficient for each feature plus an intercept, then squeezes that score into the range from zero to one so it can honestly be called a probability. Despite its name it is used for classification.

Log-odds (logit) — The natural logarithm of the odds. Log-odds run from very negative to very positive, and a log-odds of zero means a probability of exactly one-half. A logistic regression model's score is exactly the log-odds, which is why the model behaves like a straight line on this scale even though its probability curve bends.

Log-probability — The natural logarithm of a probability. Every log-probability is zero or negative, and a bigger (less negative) value means a more likely outcome. Adding log-probabilities is the same as multiplying the probabilities, which stops long products from shrinking to zero inside a computer.

Loss — The number a learning algorithm tries to make as small as possible during training; it measures how badly the predictions miss the known answers. In this chapter the loss is the mean squared error (MSE).

Loss curve — A plot of the loss against the number of training rounds (iterations or epochs). Plotting the training loss and the validation loss on the same axes shows when the model stops learning the pattern and starts learning the noise.

Loss surface — A picture of a model's loss for every possible choice of its parameters, imagined as a landscape in which low ground means a good model and high ground a bad one.

M

Machine learning — A way to build computer programs that find useful rules from data instead of a person writing every rule by hand.

Macro average — The plain average of a metric across the classes, so that every class counts equally, however rare.

Majority class — The category that appears most often in the training data. The majority-class baseline predicts it for every example.

Majority vote — The way a classification ensemble combines its members: count how many members chose each category, and answer with the category chosen most often.

Manhattan distance — The distance found by adding the sizes of the feature differences, with no squaring and no square root, like walking around city blocks instead of cutting across.

Margin — The width of the empty band around a classifier's boundary, measured straight across from one edge to the other. A support vector machine chooses, among all the boundaries that separate the classes, the one whose margin is widest.

Matrix — A rectangular table of numbers arranged in rows and columns, usually holding a whole dataset: one row per example, one column per feature.

Maximum likelihood — The principle of choosing the coefficients that make the outcomes actually seen in the training data as probable as possible. For logistic regression it is the same thing as making the average log loss as small as possible.

Mean — The everyday average: the sum of the values divided by how many there are. Every value pulls the mean toward itself, so one very large value can move it a lot.

Mean absolute error (MAE) — The average of the absolute errors over a group of predictions; how far off a regression model typically is, in the unit of the target.

Mean squared error (MSE) — The average of the squared errors; large errors count for much more than small ones, and the unit is the target's unit squared.

Median — The middle value after sorting. Half of the values are below it and half above it, so a single huge value hardly moves it.

Merge height — The height of a horizontal bar in a dendrogram, which is the linkage distance at which the two clusters below it were merged. A tall bar means the two clusters were far apart when they finally joined.

Meta-model — The small second-layer model in a stacked ensemble that combines the base models' predictions into the final answer. It is usually something simple, such as a logistic regression.

Metric — A number used to measure how well a model performs. Different metrics pay attention to different kinds of mistakes, so choosing the right metric matters as much as choosing the right model.

Micro average — The metric computed from the pooled counts of all classes. For a single-label problem, micro-averaged precision, recall and F1 all equal the accuracy, so it is dominated by the common classes.

Mini-batch gradient descent — Gradient descent in which every step is computed from a small group of training examples, usually between 16 and 256. It is the compromise most software uses.

Minimum leaf size — The smallest number of training examples a leaf is allowed to contain. Any split that would create a smaller leaf is forbidden, so a tree cannot build a rule around one or two lucky examples.

Min-max normalization — Subtracting the training minimum and dividing by the training range, so that the training values run from exactly 0 to exactly 1.

minPts — One of DBSCAN's two settings: the smallest number of points that must sit inside a point's neighborhood, counting the point itself, before that neighborhood is treated as crowded.

Missing value — A feature value that was not recorded, and is simply blank in the table (software often shows it as NaN, "not a number"). It is not automatically zero and not automatically an error; it means the true value is unknown.

Mode — The value that appears most often — the only "typical value" that also makes sense for a categorical column (the most common city).

Model — The rule a learning algorithm produces after studying data; it turns a new input into a prediction.

Model capacity — How wide a range of patterns a model is able to represent. A low-capacity model can only represent simple patterns, such as a straight line; a high-capacity model can represent complicated, wiggly ones.

Model card — A short, plain-language document that travels with a trained model, saying what it predicts, what data it learned from, how well it scored, where it is weakest, and what it must not be used for.

Model parameter — A number the learning algorithm finds by itself during training, such as a slope, an intercept, a split threshold or a leaf value. A person never sets these by hand.

Model report — A short, fixed record of a chosen model: what it predicts, on what data, with which settings, scored how, by whom and when. It lets a future colleague reproduce the result instead of starting over.

Model selection — The whole decision of which algorithm to use and with which settings, made from validation evidence together with cost, speed and interpretability, and never from the test set.

Monitoring — Regularly checking a live model's incoming data, its predictions and its real outcomes, so problems are caught quickly instead of found by accident much later.

Multinomial logistic regression — A single model that works out one score for every class and converts them all at once with softmax, instead of training a separate yes/no model for each class.

Multiple linear regression — Linear regression with more than one feature. Each feature gets its own coefficient, all the products are added together, and a single intercept is added at the end.

N

Naive Bayes classifier — A classifier that counts how common each class is and how common each feature value is inside each class, multiplies those numbers together under the naive assumption, and predicts whichever class gives the largest product.

Natural logarithm — The logarithm that uses the number e as its base, written $\ln$. It is the one used by log loss and by log-probabilities.

Nested cross-validation — Cross-validation with a second cross-validation inside it. The inner one chooses the hyperparameters; the outer one measures the tuned model on data the inner search never saw.

Net value — What a campaign or program is worth after its costs: the value of the outcomes it saves or gains, minus the cost of acting on every case that was flagged, including the cases that were never at risk.

Noise point — In DBSCAN, a point that is neither a core point nor within the chosen radius of any core point, so it joins no cluster at all. Being called noise says nothing bad about an example — noise points are often the most interesting ones.

Norm — The length of a vector. Square every entry, add the squares together and take the square root, exactly as when finding a distance. The margin of a support vector machine is two divided by the norm of its weight vector.

Novelty detection — Training a detector on data you are confident is clean, containing only normal examples, and then judging new examples one at a time. Its opposite, outlier detection, trains on data that already contains an unknown number of anomalies.

Numerical feature — A number whose size means something: 100 m² really is twice 50 m². A continuous numerical feature can take any value on a scale (size, rent, temperature); a discrete one counts whole things (rooms, children, visits).

O

Odds — The probability that something happens divided by the probability that it does not. A probability of 0.8 gives odds of 4, usually read as “four to one on”; odds start at zero and have no upper limit. Adding one unit to a feature multiplies the odds by a fixed number, called the odds multiplier or odds ratio, found by raising e to the power of that feature’s coefficient.

One-class SVM — A support vector machine trained on normal examples only. It learns a boundary that wraps almost all of them; anything outside is flagged. The setting ν fixes how many training examples may be left outside, and γ fixes how tightly the boundary can curve.

One-hot encoding — Replacing an unordered (nominal) categorical column — city, blood type, day type — by one new 0/1 column per category. In each row exactly one of the new columns is 1 (“hot”) and the rest are 0.

One-standard-error rule — Among all models whose score is within one standard error of the best score, choose the simplest one rather than the one with the best number.

One-vs-one (OvO) — A way of handling three or more classes by training one classifier for every pair of classes and letting the pairwise winners vote. With five classes that means ten small models, each trained on only two classes’ worth of data.

One-vs-rest (OvR) — A way of handling several classes by training one ordinary yes/no model for each class, “this class versus everything else”, and then predicting whichever class its own model is most confident about. Also called one-vs-all.

Ordinal encoding — Replacing ordered categories by ordered whole numbers, such as Needs Repair = 0, Fair = 1, Good = 2, Excellent = 3. Correct only when the order is real (condition, pain level, shirt size).

Ordinary least squares (OLS) — Fitting a line by choosing the coefficient and the intercept that make the sum of squared errors as small as it can possibly be.

Orthogonal — At a right angle. Two unit directions are orthogonal when the products of their matching numbers add up to exactly zero. Principal components are always orthogonal to one another.

Outer loop — In a nested cross-validation, the folds that score the tuned model. Their scores are used only for measuring, never for choosing anything.

Outlier — A value that sits far from most of the other values in its column. It may be a genuine but unusual example, such as a real luxury apartment, or a mistake, such as an extra digit typed by accident; only checking can tell the two apart.

Out-of-bag example — For one tree in a forest, a training row that the tree's own bootstrap sample happened to leave out. The tree never learned from that row, so its prediction for it is honest.

Out-of-bag score — An accuracy estimate for a whole forest, made by predicting each training row with only the trees that left it out, then comparing those predictions with the true labels. It costs nothing extra and needs enough trees to be reliable.

Overfitting — Learning the noise and small accidents of the training data so well that the model performs worse on new data. An overfit model has memorized its training examples instead of learning a general rule.

Oversampling — Adding more copies, or new similar examples, of the minority class to the training set.

P

Pair plot — A grid of scatter plots showing every pair of a few chosen features, with the histogram of each feature on the diagonal.

Paired comparison — Comparing two models by subtracting their scores fold by fold and studying the differences, instead of comparing two separate averages. It only works when both models were scored on exactly the same folds.

Partial dependence plot — A curve showing a model's average prediction as one feature is set to each value in turn, with every other feature left as it is in the data. It shows the direction and shape of that feature's effect, not only its size.

Partial derivative — The slope of a function when one variable is changed and everything else is held still. It says whether increasing that variable would raise or lower the result, and how quickly.

Percentage point — The unit for the plain difference between two percentages. A rise from 70% to 75% is 5 percentage points, but a relative rise of about 7.1%.

Percentile — The value below which a given percentage of the column falls; the 25th percentile is the value that a quarter of the values are below.

Performance (P) — The measure used to judge whether a trained model is good enough.

Permutation importance — A feature's importance measured by damage: score a trained model, shuffle that one feature's column so its values no longer match their rows, score again, and take the drop. Best computed on held-out data.

Personal data — Any information relating to an identifiable person — a name, an ID number, an address, an exam record, a medical result, or a combination of ordinary fields that together point to one individual.

Pipeline — An ordered chain of preprocessing steps and a model, bundled into one object. Fitting it fits every step in order on the training rows only; using it afterward transforms new rows with the learned rules and then predicts.

Point anomaly — A single example that is unusual compared with the whole dataset, wherever and whenever it appeared — a household using 890 kilowatt-hours in a day when it normally uses 40.

Polynomial regression — Linear regression fitted to powers of a feature — the feature, its square, its cube and so on — as if each power were a separate feature. The result is a curve, but the model is still linear in its coefficients.

Population — The complete group a model is meant to work for — every apartment for rent in the three cities, including the ones nobody has listed yet.

Population variance — The average squared distance from the mean, dividing by the number of values. Use it when the numbers in hand are the whole group you care about.

Posterior probability — The updated belief about a class after the evidence has been taken into account, written $P(\text{class} \mid \text{evidence})$. It is the number a probabilistic classifier reports for each class.

Precision — Of all the examples the model predicted as positive, the fraction that really were positive: "when this model raises an alarm, how often is it right?"

Precision at k — Of the top k examples on an alert list, the fraction that turn out to be real problems

after they have been investigated. It is ordinary precision measured only at the top of the list, where a team is actually looking.

Precision-recall curve — A plot of precision against recall at every threshold. It looks only at the positive class, which makes it more informative than the ROC curve when positives are rare; the area under it (average precision) summarizes it.

Prediction — The output a model gives for one new example — a category or a number.

Prediction time — The exact moment in the real world at which a model must produce its answer. Only information that already exists at that moment may be used as a feature.

Preprocessing — All the steps that prepare raw data for a model before training begins. Data cleaning — finding and fixing missing values, errors, outliers, inconsistent spellings and duplicate rows — comes first; turning categories into numbers and putting numbers on a shared scale come next.

Principal component — One of the new directions PCA finds. The first captures the most spread of all; each later one captures the most spread still left over once the earlier ones are accounted for.

Principal Component Analysis (PCA) — A dimensionality-reduction method that finds new directions through a dataset, ranked by how much of the data's spread each one captures, so that a few of them can stand in for many original features.

Prior probability — What you believe about a class before looking at the new example. In a trained classifier it is usually just how common that class was in the training data.

Probability — A number between 0 and 1 measuring how likely an event is. Zero means it never happens, 1 means it always happens, 0.5 means about half the time.

Projection — One number saying how far along a chosen direction a centered point falls. It is found by multiplying matching numbers and adding the results.

Protected attribute — A personal characteristic, such as gender, age, ethnicity, religion, disability or region of origin, that law or an organization's own policy says must not be the reason somebody receives a worse outcome.

Proxy variable — A feature that is not itself a protected attribute but carries almost the same information, so a model can rebuild the protected pattern even after that column is deleted. Home district and school name are common examples.

Pruning — Cutting branches off an already-grown tree, replacing a branch with a single leaf whenever that branch does not really help on held-out data.

Pseudonymization — Replacing a direct identifier with a code and keeping the key somewhere else. It lowers the risk but is reversible, so the data still counts as personal data.

Q

Quadratic discriminant analysis (QDA) — The same idea as LDA, except that each class is allowed its own shape and spread instead of sharing one. Its decision boundary is curved, and it needs more training data because it has more numbers to estimate.

Quartiles — The three values that cut the sorted column into four equal parts: the first quartile Q_1 (25th percentile), the median (50th) and the third quartile Q_3 (75th).

Quasi-identifier — A field that is not an identifier on its own but can single out a person when combined with others — age band, district, sex, job title, or the date of a visit.

Query time — The work a model must do to answer one prediction request. For kNN it grows with both the number of stored examples and the number of features.

R

Random forest — An ensemble of decision trees in which every tree is grown on its own bootstrap sample and every split is chosen from a random subset of the features. It predicts by majority vote for a category, or by averaging for a number.

Random restart — Running the same method several times from different random starting points and keeping whichever run scores best. In scikit-learn this is the `n_init` setting.

Random search — A tuning method that draws a fixed number of hyperparameter combinations at random from the search space, scores each one, and keeps the best it found.

Random seed — A fixed starting number that makes a program's random choices come out the same way every time it runs. Written as `random_state` in scikit-learn and passed to `np.random.default_rng` in NumPy. The same seed always gives the same result.

Random split — A split made by shuffling all the rows and cutting the shuffled list at the chosen counts, so every row has the same chance of landing in each set.

Range — The largest value minus the smallest: a quick but fragile measure of spread, because it depends on only two values.

Ratio feature — A new feature made by dividing one column by another, such as rainfall per kilogram of fertilizer or price per square meter.

RBF kernel — The radial basis function kernel, also called the Gaussian kernel. It gives a similarity of exactly 1 for two identical examples and a value that falls smoothly toward 0 as they move apart. It is the usual first choice for a curved boundary.

Recall (sensitivity, true positive rate) — Of all the examples that really were positive, the fraction the model found: “of all the true cases, how many did the model catch?”

Reconstruction — Rebuilding an example's original feature values from the few component scores that were kept. The gap between the true row and the rebuilt row is what was thrown away.

Regression — A learning task where the model predicts a number that can take almost any value on a scale.

Regression tree — A decision tree whose leaves predict a number instead of a category. Each leaf predicts the plain average of the target values of the training examples that landed in it.

Regularization — Adding a penalty, inside training itself, that discourages a model from becoming more complex than the data justifies. The size of the penalty is set by the regularization strength λ (lambda), a hyperparameter: $\lambda = 0$ means no penalty, and a large λ forces the model to stay simple.

Reinforcement learning — A kind of learning in which an agent learns from rewards and penalties rather than from labeled examples.

Relative change — The difference between two values measured against the starting value and

written as a percent, so it says how big a change was next to where it started.

Repeated cross-validation — Running k-fold cross-validation several times with a fresh shuffle each time and averaging all the fold scores, to smooth out the luck of any one set of folds.

Representative sample — A sample whose mix of examples looks like the population's mix, so that what is true of the sample is roughly true of the population.

Residual — The true value minus the predicted value for one example. A positive residual means the model predicted too low, a negative one that it predicted too high, and zero that the prediction was exactly right.

Residual plot — A plot of the residuals against the predicted values, with a horizontal line at zero. A good fit shows a shapeless band around zero; any visible shape means the model is missing a pattern.

Responsibility — The probability that a particular example was produced by a particular cluster. An example's responsibilities across all the clusters always add up to one.

Ridge regression — Linear regression with a penalty equal to the regularization strength times the sum of the squared coefficients. Ridge shrinks every coefficient toward zero, the largest ones most, but almost never all the way to zero.

Right-skewed column — A column in which most values are small and a few are very large, so its histogram has a long tail stretching to the right.

ROC curve — A plot of the true positive rate (up the side) against the false positive rate (along the bottom) for every threshold from 1 down to 0.

Rollback — Switching a live system back to its last known-good version, quickly, when monitoring shows that the current version is behaving badly. It is usually safer than debugging in front of real users.

Root mean squared error (RMSE) — The square root of MSE, which brings the number back into the target's real unit.

Root node — The starting point of a decision tree, where the first question is asked, using every training example.

S

Sample — The rows actually collected, always only part of the population. (Machine learning also uses sample for one single row; this book says “a sample of 40 listings” for the group and “one sample” or “one example” for a row.)

Sample variance — The average squared distance from the mean, dividing by one less than the number of values. Use it to estimate the spread of the wider population a sample came from.

Sample weight — A number attached to one training row saying how much that row matters while a model is being trained. All the weights of a training set add up to 1.

Sampling bias — A systematic difference between sample and population caused by the way the data was collected, such as collecting listings from only one city.

Save rate — The share of genuinely at-risk customers who stay because they were contacted. It is a business estimate, not something a model predicts, and only a controlled comparison can measure it.

Scalar — A single number on its own, with no other numbers attached to it.

Scatter plot — A plot with one feature on the horizontal axis and another on the vertical axis, and one point per sample. It shows whether two columns move together, how tightly, and whether the relationship is a straight line or a curve.

Scree plot — A bar chart of each component’s explained variance ratio, in order, usually with the running total drawn on top as a line. It shows at a glance how many components are worth keeping.

Search budget — The number of hyperparameter combinations a team decides in advance to try. It is chosen from the computer time available, not from the size of the grid.

Search space — The complete list of hyperparameter values a team is willing to consider, one list per dial. A search can only ever return something from inside this space.

Selection rate — The share of everybody in a group who received the positive decision, whether or not they truly qualified. Comparing selection rates across groups tests demographic parity.

Shape — The size of a matrix, written as rows first and columns second. A table of 4 examples with 2 features has shape 4 by 2.

Sigma notation — A short way of writing a long addition, using the sign $\sum$: below it sits the counter and its starting value, above it the stopping value, and to its right the thing to be added.

Sigmoid function — A function that squeezes any number, however large or however negative, into a value strictly between zero and one, tracing a smooth S-shaped curve. Logistic regression uses it to turn a plain score into a probability.

Significant figures — The digits of a number that actually carry information. Non-zero digits always count; zeros written only to place the decimal point never do.

Silent churn — A customer who stops using a service but never cancels, so no churn label is ever recorded even though the business is, in practice, already lost.

Silhouette score — A score from minus one to one for a single example, comparing its average distance to its own cluster with its average distance to the nearest other cluster. Near one it sits snugly inside its group; below zero another cluster would suit it better.

Single linkage — A linkage rule that measures the distance between two clusters as the distance between their two closest members. It can follow long, thin or curved shapes, but it is the rule most prone to chaining.

Slack variable — A number measuring how badly one training example breaks the margin rule. It is zero for an example safely outside the margin, between zero and one for an example inside the margin but still correct, and above one for an example on the wrong side.

Soft clustering — Giving each example a share in every cluster, such as seventy percent in one group and thirty percent in another, so that uncertainty survives into the result.

Soft margin — The practical form of the support vector machine rule, which lets a few examples break the margin and charges for the total amount of rule-breaking instead of forbidding it. Nearly all real data needs it.

Soft voting — Combining several finished models by averaging the probabilities they report and comparing that average with the decision threshold.

Softmax function — A function that turns several raw class scores into several probabilities that are all positive and add up to exactly one. It is the version of the sigmoid used when there are three or more classes.

Specificity (true negative rate) — Of all the examples that really were negative, the fraction the model correctly left alone. Its complement, $1 - \text{specificity}$, is the false positive rate.

Split — One question at a node, made of a feature and a threshold, that sends every example in the node down one of two branches.

Stacking — An ensemble in two layers: several different base models are trained on the data, and their cross-validated predictions become the input columns of a second model that produces the final answer.

Standard deviation — The square root of the variance: a typical distance of a value from the mean, in the column's own unit. Small means the values sit close to the mean; large means they are spread out.

Standard error — The standard deviation of the fold scores divided by the square root of the number of folds; a rough measure of how uncertain the cross-validation mean is.

Standardization — Subtracting the training mean and dividing by the training standard deviation, so that the column is centered on 0 and most values fall between about -3 and 3 .

Stochastic gradient descent (SGD) — Gradient descent in which every step is computed from a single randomly chosen training example. Steps are cheap and fast but noisy.

Stratified k-fold — Cross-validation in which each fold is built separately inside every class, so all folds carry the same class shares as the whole dataset.

Stratified split — A random split made separately inside each class, so that every set contains the same share of each class as the whole dataset.

Sum of squared errors (SSE) — The total obtained by squaring every residual and adding all the squares together. Squaring removes the signs, so

errors above and below the line can never cancel out.

Supervised learning — Learning from examples that already carry the correct label.

Support vector — A training example lying exactly on the edge of the margin, holding the boundary in place. Moving a support vector moves the boundary; moving any other example does not move it at all.

Support vector machine (SVM) — A classifier that chooses, out of all the boundaries separating two classes, the one with the widest empty band around it. It stores only its support vectors, and it needs features on comparable scales.

Support vector regression (SVR) — The version of a support vector machine that predicts a number instead of a category. It fits the flattest line whose tube covers as much of the data as possible, ignoring every point already inside the tube.

T

Target encoding — Replacing each category by the average label value seen for that category in the training data. It is powerful but leaks the label unless it is computed on separate folds.

Target vector $\mathbf{y}$ — The n target values, one per sample, in the same row order as $\mathbf{X}$. Together, $\mathbf{X}$ and $\mathbf{y}$ are what a supervised learning algorithm receives.

Task (T) — A plain sentence describing what a model must predict, and for what kind of example.

Test set — Examples kept completely separate from training and used once, at the end, to check the finished model.

Test-set peeking — Looking at the test score while still deciding what to try next. Each look turns the test set a little more into a second validation set, and the final reported number stops being an honest estimate.

Text feature — Free sentences typed by a person, such as a listing's description. Text must be turned into numbers before a model can use it.

The number e — A fixed number, about 2.71828, that appears in every formula describing smooth growth. It is used the way π is used for circles.

Threshold — The probability a model's output must reach before the prediction counts as positive. A threshold of 0.5 is the common default; lowering it makes the model flag more examples as positive, raising it makes the model flag fewer.

Tie — A vote in which two or more classes collect the same total, so the majority rule has no winner. A tie-breaking rule should be chosen before the model is used.

Time-based split — A split made by date instead of by chance: the oldest rows train the model and the newest rows validate and test it, so the model is scored on data from after the period it learned from.

Traceback — The block of text Python prints when a program stops with an error. It is read from the bottom up: the last line names the error and explains it, and the line above it points at the code that caused it.

Training — The process of using examples to fit, or adjust, a model.

Training set — The examples a model learns from.

Training-serving skew — A mismatch between how a feature is computed for training and how it is computed for live predictions, so the model receives different information than it learned from although nothing in the world changed.

Transpose — A matrix turned on its side, so that its rows become columns and its columns become rows. A 4 by 2 matrix transposes to a 2 by 4 one.

True negative (TN) — Actually negative, predicted negative — correctly left alone.

True positive (TP) — Actually positive, predicted positive — a correct alarm.

t-SNE — A method that arranges examples in two dimensions so that close neighbors stay close. It is meant for drawing pictures only: its axes have no unit and distances between groups mean nothing.

U

Underfitting — Failing to learn enough of the real pattern, so that the model performs poorly even on the training data. An underfit model is usually too simple for the problem.

Undersampling — Removing examples of the majority class from the training set.

Unit direction — A list of numbers describing a direction, chosen so that their squares add up to exactly one. Using the same length for every direction makes comparisons of spread fair.

Unit of observation — The real-world thing that one row of a dataset represents, such as one patient visit, one parcel delivery or one field in one season.

Unsupervised learning — Learning from examples that have no labels; the model sees only the features.

Update step — The half of a k-means round that moves each centroid to the mean of the examples just assigned to it, leaving the assignments alone.

V

Validation set — A group of examples, separate from both the training and test sets, used to compare candidate models and choose settings before the test set is opened.

Variance — The average squared distance of the values from their mean (dividing by $n - 1$ for a sample). Its unit is the column's unit squared, which is why it is rarely reported on its own.

Vector — An ordered list of numbers, usually describing one example with one number per feature, always in the same agreed order.

Virtual environment — A private folder holding one copy of Python and its libraries, kept separate from the rest of the computer, so that changing a library for one project cannot break another.

Vote share — The fraction of an ensemble's members that voted for the winning category. It works as a rough confidence level for the prediction.

Voting ensemble — Several already-trained models that answer the same question, combined into one answer by counting their votes or by averaging their probabilities.

W

Ward's method — A linkage rule that merges whichever two clusters add the least extra spread, measured as the increase in the total squared distance from points to their own centroid. It favors compact, similarly sized clusters, much like k-means.

Weak learner — A model that is only slightly better than guessing — far too poor to use alone, but useful as one link in a long boosted chain.

Weight vector — The list of coefficients of a linear model, one number for each feature, written as a bold letter. In a support vector machine it points straight across the margin, at a right angle to the boundary.

Weighted error rate — The share of the total sample weight sitting on the rows a learner got wrong. With unequal weights, missing one heavy row costs more than missing several light ones.

Weighted impurity — The impurity of a split's two children combined into one number, with each child counting in proportion to how many examples landed in it.

Workflow — An ordered set of steps used to complete a project in a safe, repeatable way.

Wrapper method — A feature-selection method that trains a model on different subsets of features and keeps the subset that validates best.

Z

z-score — How many standard deviations a value sits above or below the mean of its reference data. A z-score of 0 means exactly average; +2 means two standard deviations above average; -2 means two below.

References

Every dataset used in this book, and where to find it

This book names a person and a year whenever it introduces a method, so that you can find the original work if you want to. Those sources are listed here, grouped by subject, together with the textbooks and software the book relies on. Within each group the entries are ordered by year, so the list also reads as a short history of the subject.

Textbooks and general references

- [1] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill. — The source of the Task, Experience and Performance definition used in Chapter 1.
- [2] Duda, R. O., Hart, P. E., and Stork, D. G. (2001). *Pattern Classification* (2nd ed.). Wiley.
- [3] Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- [4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [5] Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- [6] James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.
- [7] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly.
- [8] Shalev-Shwartz, S., and Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.
- [9] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley. — The box plot of Chapter 3.
- [10] Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23.

Project workflow, data and evaluation

- [11] Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product*. Van Nostrand. — Control charts, the ancestor of the monitoring plots in Chapter 24.
- [12] Lachenbruch, P. A., and Mickey, M. R. (1968). Estimation of error rates in discriminant analysis. *Technometrics*, 10(1), 1–11. — Leave-one-out cross-validation.
- [13] Stone, M. (1974). Cross-validated choice and assessment of statistical predictions. *Journal of the Royal Statistical Society, Series B*, 36(2), 111–147.
- [14] Geisser, S. (1975). The predictive sample reuse method with applications. *Journal of the American Statistical Association*, 70(350), 320–328.
- [15] Hanley, J. A., and McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36.
- [16] Swets, J. A. (1988). Measuring the accuracy of diagnostic systems. *Science*, 240(4857), 1285–1293.
- [17] Murphy, A. H., and Winkler, R. L. (1977). Reliability of subjective probability forecasts of precipitation and temperature. *Applied Statistics*, 26(1), 41–47. — Reliability (calibration) diagrams.

- [18] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of IJCAI 1995*, 1137–1143.
- [19] Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- [20] Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., and Wirth, R. (2000). *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- [21] Varma, S., and Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. — Nested cross-validation.
- [22] Kaufman, S., Rosset, S., and Perlich, C. (2011). Leakage in data mining: formulation, detection, and avoidance. *Proceedings of KDD 2011*, 556–563.
- [23] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [24] Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 61–74. MIT Press. — Probability calibration.
- [25] Bergstra, J., and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.

Statistics and the mathematics behind the methods

- [26] Bayes, T. (1763). An essay towards solving a problem in the doctrine of chances. *Philosophical Transactions of the Royal Society of London*, 53, 370–418. Communicated after Bayes’s death by Richard Price.
- [27] Laplace, P.-S. (1774). Mémoire sur la probabilité des causes par les événements. *Mémoires de l’Académie Royale des Sciences*, 6, 621–656.
- [28] Laplace, P.-S. (1814). *Essai philosophique sur les probabilités*. Courcier. — The rule of succession, the origin of add-one smoothing.
- [29] Legendre, A.-M. (1805). *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot. — First published account of least squares.
- [30] Gauss, C. F. (1809). *Theoria Motus Corporum Coelestium*. Perthes and Besser.
- [31] Cauchy, A.-L. (1847). Méthode générale pour la résolution des systèmes d’équations simultanées. *Comptes Rendus de l’Académie des Sciences*, 25, 536–538. — Gradient descent.
- [32] Galton, F. (1886). Regression towards mediocrity in hereditary stature. *Journal of the Anthropological Institute*, 15, 246–263. — The origin of the word “regression”.
- [33] Pearson, K. (1895). Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242. — The correlation coefficient.
- [34] Mercer, J. (1909). Functions of positive and negative type and their connection with the theory of integral equations. *Philosophical Transactions of the Royal Society A*, 209, 415–446.
- [35] Wright, S. (1921). Correlation and causation. *Journal of Agricultural Research*, 20, 557–585. — The coefficient of determination R^2 .
- [36] Fisher, R. A. (1922). On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society A*, 222, 309–368. — Maximum likelihood.
- [37] Mahalanobis, P. C. (1936). On the generalised distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1), 49–55.
- [38] Robbins, H., and Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407. — Stochastic gradient descent.
- [39] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27, 379–423 and 623–656. — Entropy.
- [40] Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1–17. — Momentum.

- [41] Box, G. E. P., and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society, Series B*, 26(2), 211–252.
- [42] Geman, S., Bienenstock, E., and Doursat, R. (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1), 1–58.
- [43] Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12, 2121–2159. — AdaGrad.

Regression and regularization

- [44] Hoerl, A. E., and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- [45] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.
- [46] Zou, H., and Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society, Series B*, 67(2), 301–320.

Classification

- [47] Verhulst, P.-F. (1838). Notice sur la loi que la population suit dans son accroissement. *Correspondance Mathématique et Physique*, 10, 113–121. — The logistic curve.
- [48] Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188. — Linear discriminant analysis.
- [49] Berkson, J. (1944). Application of the logistic function to bio-assay. *Journal of the American Statistical Association*, 39(227), 357–365. — The name “logit”.
- [50] Fix, E., and Hodges, J. L. (1951). *Discriminatory Analysis: Nonparametric Discrimination, Consistency Properties*. USAF School of Aviation Medicine, Report 4. — The nearest-neighbor rule.
- [51] Kent, A., Berry, M. M., Luehrs, F. U., and Perry, J. W. (1955). Machine literature searching VIII: operational criteria for designing information retrieval systems. *American Documentation*, 6(2), 93–101. — Precision and recall.
- [52] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
- [53] Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society, Series B*, 20(2), 215–242. — Logistic regression.
- [54] Vapnik, V. N., and Chervonenkis, A. Ya. (1963). On a class of algorithms of learning recognition. *Automation and Remote Control*, 25(6). — The maximal-margin idea.
- [55] Cover, T. M., and Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- [56] Minsky, M., and Papert, S. (1969). *Perceptrons*. MIT Press. — The XOR problem of Chapter 17.
- [57] McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior. In *Frontiers in Econometrics*, 105–142. Academic Press. — Multinomial logistic regression.
- [58] Dudani, S. A. (1976). The distance-weighted k -nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, 6(4), 325–327.
- [59] Domingos, P., and Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29, 103–130.
- [60] Hand, D. J., and Yu, K. (2001). Idiot’s Bayes — not so stupid after all? *International Statistical Review*, 69(3), 385–398.
- [61] Ng, A. Y., and Jordan, M. I. (2001). On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes. *Advances in Neural Information Processing Systems*, 14.

Trees, forests and boosting

- [62] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth. — CART, Gini impurity, cost-complexity pruning, the one-standard-error rule.
- [63] Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1, 81–106. — ID3.
- [64] Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- [65] Efron, B. (1979). Bootstrap methods: another look at the jackknife. *Annals of Statistics*, 7(1), 1–26.
- [66] Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning*, 5, 197–227.
- [67] Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259.
- [68] Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24, 123–140.
- [69] Freund, Y., and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139. — AdaBoost.
- [70] Ho, T. K. (1998). The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8), 832–844.
- [71] Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32. — Also the source of permutation importance.
- [72] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. — Also partial-dependence plots.
- [73] Geurts, P., Ernst, D., and Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63, 3–42.
- [74] Chen, T., and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *Proceedings of KDD 2016*, 785–794.
- [75] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- [76] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

Support vector machines and kernel methods

- [77] Aizerman, M. A., Braverman, E. M., and Rozonoer, L. I. (1964). Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25, 821–837. — The kernel idea.
- [78] Kimeldorf, G., and Wahba, G. (1971). Some results on Tchebycheffian spline functions. *Journal of Mathematical Analysis and Applications*, 33(1), 82–95. — The representer theorem.
- [79] Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of COLT 1992*, 144–152. — The kernel trick applied to SVMs.
- [80] Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297. — The soft margin.
- [81] Drucker, H., Burges, C. J. C., Kaufman, L., Smola, A., and Vapnik, V. (1996). Support vector regression machines. *Advances in Neural Information Processing Systems*, 9, 155–161.
- [82] Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5), 1299–1319. — Kernel PCA.
- [83] Saunders, C., Gammerman, A., and Vovk, V. (1998). Ridge regression learning algorithm in dual variables. *Proceedings of ICML 1998*, 515–521. — Kernel ridge regression.
- [84] Platt, J. C. (1998). *Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines*. Microsoft Research Technical Report MSR-TR-98-14.
- [85] Schölkopf, B., Williamson, R., Smola, A., Shawe-Taylor, J., and Platt, J. (1999). Support vector method for novelty detection. *Advances in Neural Information Processing Systems*, 12, 582–588. — One-class SVM.
- [86] Tax, D. M. J., and Duin, R. P. W. (2004). Support vector data description. *Machine Learning*, 54, 45–66.

Clustering, dimensionality reduction and anomaly detection

- [87] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572. — Principal component analysis.
- [88] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
- [89] Sørensen, T. (1948). A method of establishing groups of equal amplitude in plant sociology. *Biologiske Skrifter*, 5, 1–34. — Complete linkage.
- [90] Thorndike, R. L. (1953). Who belongs in the family? *Psychometrika*, 18(4), 267–276. — The elbow method.
- [91] Steinhaus, H. (1956). Sur la division des corps matériels en parties. *Bulletin de l'Académie Polonaise des Sciences*, 4(12), 801–804.
- [92] Sneath, P. H. A. (1957). The application of computers to taxonomy. *Journal of General Microbiology*, 17(1), 201–226. — Single linkage.
- [93] Lloyd, S. P. (1957/1982). Least squares quantization in PCM. Bell Laboratories technical note, published in *IEEE Transactions on Information Theory*, 28(2), 129–137. — The k -means algorithm.
- [94] Sokal, R. R., and Michener, C. D. (1958). A statistical method for evaluating systematic relationships. *University of Kansas Science Bulletin*, 38, 1409–1438. — Average linkage (UPGMA).
- [95] Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236–244.
- [96] Bellman, R. (1961). *Adaptive Control Processes: A Guided Tour*. Princeton University Press. — The curse of dimensionality.
- [97] Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276.
- [98] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium*, 1, 281–297. — The name “ k -means”.
- [99] Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9), 509–517. — The k -d tree.
- [100] Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1), 1–38.
- [101] Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- [102] Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD 1996*, 226–231. — DBSCAN.
- [103] Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). LOF: identifying density-based local outliers. *Proceedings of SIGMOD 2000*, 93–104.
- [104] Arthur, D., and Vassilvitskii, S. (2007). k -means++: the advantages of careful seeding. *Proceedings of SODA 2007*, 1027–1035.
- [105] Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. *Proceedings of ICDM 2008*, 413–422.
- [106] van der Maaten, L., and Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.
- [107] Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Proceedings of PAKDD 2013*, 160–172. — HDBSCAN.
- [108] McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for dimension reduction. *arXiv:1802.03426*.

Feature engineering and feature selection

- [109] Efron, M. A. (1960). Multiple regression analysis. In *Mathematical Methods for Digital Computers*, 191–203. Wiley. — Stepwise and forward selection.

- [110] Guyon, I., Weston, J., Barnhill, S., and Vapnik, V. (2002). Gene selection for cancer classification using support vector machines. *Machine Learning*, 46, 389–422. — Recursive feature elimination.
- [111] Guyon, I., and Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- [112] Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations*, 3(1), 27–32. — Target encoding.

Interpretability, fairness and privacy

- [113] Equal Employment Opportunity Commission and others (1978). *Uniform Guidelines on Employee Selection Procedures*. 29 CFR Part 1607. — The four-fifths rule.
- [114] Samarati, P., and Sweeney, L. (1998). *Protecting Privacy When Disclosing Information: k -Anonymity and Its Enforcement through Generalization and Suppression*. SRI International Technical Report.
- [115] Sweeney, L. (2002). k -anonymity: a model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5), 557–570.
- [116] Dwork, C., McSherry, F., Nissim, K., and Smith, A. (2006). Calibrating noise to sensitivity in private data analysis. *Proceedings of TCC 2006*, 265–284. — Differential privacy.
- [117] Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). “Why should I trust you?” Explaining the predictions of any classifier. *Proceedings of KDD 2016*, 1135–1144. — LIME.
- [118] Hardt, M., Price, E., and Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems*, 29, 3315–3323.
- [119] Kleinberg, J., Mullainathan, S., and Raghavan, M. (2017). Inherent trade-offs in the fair determination of risk scores. *Proceedings of ITCS 2017*, 43:1–43:23.
- [120] Chouldechova, A. (2017). Fair prediction with disparate impact: a study of bias in recidivism prediction instruments. *Big Data*, 5(2), 153–163.
- [121] Lundberg, S. M., and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774. — SHAP.
- [122] Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model cards for model reporting. *Proceedings of FAT* 2019*, 220–229.
- [123] Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., and Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- [124] European Parliament and Council of the European Union (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. In force since 25 May 2018.
- [125] Barocas, S., Hardt, M., and Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.

Software used to write this book

- [126] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [127] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [128] Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- [129] Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

- [130] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. — pandas.
- [131] Buitinck, L., Louppe, G., Blondel, M., et al. (2013). API design for machine learning software: experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

A note on finding these papers. Most of the older articles are freely readable through university libraries or through the authors' own pages; many of the newer ones are on arxiv.org or in open conference proceedings. Where a work exists in several versions — a conference paper later extended into a journal article — the version named here is the one the book's statement is drawn from.